

DATA STRUCTURES · ALGORITHMS · SYSTEM DESIGN

क्रम

KRAMA

The Ordered Path

180 Days from First Principles to the Product-Company Interview

NISHANT GUPTA - GRANTH SKILLS

VOLUME IV OF EIGHT

Links and Recursion

DAYS 078 – 097

DAY 001

THE ORDERED PATH

DAY 180



TWO TRACKS

20 OF 180 DAYS IN THIS VOLUME

180 STEPS

K R A M A

THE ORDERED PATH

क्रम

K R A M A

The Ordered Path

180 Days from First Principles to the Product-Company Interview

Nishant Gupta - Granth Skills

VOLUME IV · LINKS AND RECURSION

DAYS 78 TO 97

KRAMA — THE ORDERED PATH

180 Days from First Principles to the Product-Company Interview

Volume IV of eight: **Links and Recursion**. Days 78 to 97.

Copyright © 2026 Nishant Gupta - Granth Skills. All rights reserved.

ABOUT THE TEXT

This edition is generated from the Krama course sources. The lessons, stories, code, diagrams and interview material are original work written for this course.

Practice problems are named, with their source and one line on what they test. No problem statement is reproduced. Trade marks and problem titles belong to their respective owners and are used here only to identify where a problem can be found.

ABOUT THE MAKING OF IT

Typeset from Markdown by rendering to a browser engine and printing to PDF. Every tool in that chain is free software and every typeface is published under the SIL Open Font Licence. Appendix F lists all of them with their licences.

Text face: Source Serif 4 — SIL Open Font Licence 1.1

Display face: Fraunces — SIL Open Font Licence 1.1

Sans face: Inter — SIL Open Font Licence 1.1

Monospace face: JetBrains Mono — SIL Open Font Licence 1.1

Devanagari: Noto Serif Devanagari — SIL Open Font Licence 1.1

Diagrams: Mermaid — MIT Licence

Code colouring: Pygments — BSD 2-Clause

EDITION

First edition, 2026. Set in Source Serif 4 on a 6.75 by 9.5 inch page.

क्रम

krama, Sanskrit: order; sequence; the step that follows the step before it.

YOU CANNOT SKIP A STEP. THAT IS THE WHOLE METHOD.

Why this book teaches two things at once

This book teaches two subjects at the same time, on purpose.

Most preparation separates them. You spend three months on data structures and algorithms, then start again from nothing on system design, and by the time the second subject makes sense the first has gone quiet. Interviews do not work that way. A single loop at a product company will ask you to reverse a linked list in one room and design a rate limiter in the next, on the same day, from the same head.

So every chapter here teaches one topic from each track, side by side. Day 42 gives you binary search and the object-oriented design of a parking lot. Day 130 gives you graph colouring and message queues. You will not finish this book with one strong subject and one weak one.

Who this is written for

Someone who has never studied this before. Not a computer science graduate. Not someone brushing up after a few years in the job.

If you need to be told what a server is, you are the reader. The first chapter explains what happens when a program runs. The fourteenth explains what happens when you type an address into a browser. Nothing before that is assumed.

The other half of that promise matters just as much: you are someone who will sit in a real interview. So the writing never stops at understanding. Every chapter ends by putting the idea in an interviewer's mouth, giving you the first ninety seconds of an answer, and writing out a full model response you can compare yours against.

The one question every chapter answers

Every lesson in this book was written to satisfy a single test:

Does this leave the reader able to answer one real interview question out loud, from memory, to a stranger?

That test decided what went in and, more usefully, what stayed out. There are no formal proofs here. No potential functions, no decision-tree lower bounds, no interpreter internals for their own sake. If a thing will not be asked, and does not make an asked thing clearer, it is not in this book.

What is different about how it is written

Every lesson opens with a story about a person. Not an analogy dressed up as a story: an actual scene, with a name in it, told in ordinary words with no technical vocabulary at all. Someone sorting socks. Someone in a canteen queue. Someone laying tables for a wedding. The story comes before the definition because a beginner needs somewhere to put the idea before they have the word for it.

Every solution is shown in full. This book does not hide answers. Code is built up in fragments of a few lines, each one explained, and then given again complete and ready to run. You are never told to work it out yourself as a test of character.

The arithmetic is always done out loud. Not “this is slow” but “this line runs about twenty-five million times”. Not “that is a lot of data” but `50M x 20 x 2KB = 2TB`. Numbers are worked through where you can see them.

Errors are printed exactly as they appear. When something breaks, the real message is on the page - `IndexError: list index out of range` - not a description of it. You will recognise it when your own screen shows it.

How this edition was made

The text is generated from the course repository, so the book and the working material cannot drift apart. The typesetting was done by rendering the pages in a browser engine and printing them to PDF.

Every part of the toolchain is free software, and every typeface is published under the SIL Open Font Licence. Appendix F lists all of it, with licences. No proprietary tool, font, or copyrighted third-party text was used to produce this book.

That extends to the practice problems. Problems are named, with their source and one line on what they are really testing. Their statements are never reproduced. You are meant to read them where they live, and solve them there.

How to use this book

The book is one hundred and eighty chapters long. One chapter is one day's work. That is the only pacing instruction in it, and it is deliberate: a day here is a unit of subject, not a unit of hours. Some days will take you longer than others. That is information about the topic, not about you.

The shape of every chapter

Each chapter holds three pieces, in this order.

1. **The DSA lesson.** Nine sections, always the same nine, always in the same order. Appendix A sets out what each one is for.
2. **The system design lesson.** The same nine sections, on a different subject.
3. **The practice sheet.** Named problems to solve, and questions to answer out loud.

The repetition is the point. After a week you stop navigating and start reading. You know that section 5 has the complete code, that section 7 has the mistakes you are about to make, and that section 8 is the interview itself.

How to read a lesson

Read the story first, and do not skip it. It is section 2, it has no technical words in it, and it is the part that makes the idea stick. People who skip it can define the term and cannot use it.

Type the code. Do not read it. Section 5 builds the solution in fragments and then gives it complete. Type the fragments as they come. Run the finished program. Change a number and run it again.

Say section 9 out loud. Every lesson ends with a recall card - the two or three sentences that are the whole lesson. Say them from memory, aloud, in full sentences. Not in your head. Aloud. An answer you have only ever thought is an answer you have never tested.

Treat section 8 as a rehearsal, not a reading. It gives you real interviewer phrasings, a script for the opening ninety seconds, the three follow-ups that usually come next, and a written-out model answer. Deliver yours before you read theirs.

How to use the practice sheet

Solve the problems where they live. The sheet names each one, tells you where to find it, and says in one line what it is really testing - which is usually not what it appears to be about.

Solve it, then say what you did out loud, as if someone had asked. If you cannot narrate your own solution, you have not finished the problem.

When you get stuck

Go back one chapter, not forward. This course is built so that each day rests on the one before it. A chapter that will not open is almost always a chapter whose foundation is thin, and the foundation is behind you.

If you want the vocabulary, Appendix D is a glossary of every term the book defines, with the day it first appears. If you want to see the whole route, Appendix C prints all one hundred and eighty days on two facing pages.

Revision

Certain chapters are revision days, and they are marked as such in the contents. Do not read past them. They exist because recall fades on a schedule, and the schedule is not negotiable.

Appendix B collects the recall cards for every part in this volume. Cover the answer, say it, then check. That is the whole drill.

Where this volume sits

This volume is marked in colour below.

- **Volume I — Foundations**
Days 1 to 26. How Code Costs, and How the Internet Works
- **Volume II — Scanning and Searching**
Days 27 to 50. Pointers, Windows and Binary Search · Databases from Zero
- **Volume III — Order and Access**
Days 51 to 77. Sorting, Hashing, Stacks and Queues · Objects, SOLID and Patterns
- **Volume IV — Links and Recursion**
Days 78 to 97. Linked Lists, Recursion and Backtracking · Low-Level Design
- **Volume V — Trees and Priorities**
Days 98 to 124. Trees, Heaps and Tries · Scaling Fundamentals
- **Volume VI — Graphs**
Days 125 to 142. Graph Algorithms · Distributed Systems Core
- **Volume VII — Optimal Choices**
Days 143 to 170. Dynamic Programming and Greedy · High-Level Design Case Studies
- **Volume VIII — The Last Mile**
Days 171 to 180. Bits, Maths and Mock Interviews · Reliability and Security

The nineteen parts

PART I	Foundations: how code costs	Days 1–8
PART II	Arrays	Days 9–18
PART III	Strings	Days 19–26
PART IV	Two pointers and sliding window	Days 27–36
PART V	Prefix sums	Days 37–41
PART VI	Binary search	Days 42–50
PART VII	Sorting	Days 51–59
PART VIII	Hashing: maps and sets	Days 60–67
PART IX	Stacks and queues	Days 68–77

Part X	Linked lists	Days 78–86
Part XI	Recursion and backtracking	Days 87–97
PART XII	Trees and binary search trees	Days 98–112
PART XIII	Heaps and priority queues	Days 113–119
PART XIV	Tries	Days 120–124
PART XV	Graphs	Days 125–142
PART XVI	Dynamic programming	Days 143–163
PART XVII	Greedy and intervals	Days 164–170
PART XVIII	Bits and maths	Days 171–176
PART XIX	Final mocks and revision	Days 177–180

What is in this volume

PART X · LINKED LISTS	1
078 Nodes and links: pointers without pointers	2
DSA Nodes and links: pointers without pointers	3
DESIGN Design a parking lot	21
PRACTICPractice — Nodes and links: pointers without pointers	40
079 Traversal, insertion, and deletion	46
DSA Traversal, insertion, and deletion	47
DESIGN Design an elevator system	65
PRACTICPractice — Traversal, insertion, and deletion	83
080 The dummy head trick	89
DSA The dummy head trick	90
DESIGN Design an ATM	107
PRACTICPractice — The dummy head trick	127
081 Reversing a linked list	133
DSA Reversing a linked list	134
DESIGN Design a vending machine	151
PRACTICPractice — Reversing a linked list	169
082 Finding the middle, and the runner technique	175
DSA Finding the middle, and the runner technique	176
DESIGN Design a library management system	193
PRACTICPractice — Finding the middle, and the runner technique	212
083 Cycle detection, and why Floyd's algorithm works	218
DSA Cycle detection, and why Floyd's algorithm works	219
DESIGN Design tic-tac-toe, and then chess	235
PRACTICPractice — Cycle detection, and why Floyd's algorithm works	255
084 Merging and sorting linked lists	261
DSA Merging and sorting linked lists	262
DESIGN Design a deck of cards and a card game	281
PRACTICPractice — Merging and sorting linked lists	299
085 Doubly and circular linked lists	305
DSA Doubly and circular linked lists	306
DESIGN Design Splitwise	325
PRACTICPractice — Doubly and circular linked lists	343
086 Linked lists revision and mock round	349
DSA Linked lists revision and mock round	350
DESIGN Design BookMyShow	368

PRACTIC	Practice — Linked lists revision and mock round	385
PART XI · RECURSION AND BACKTRACKING		391
087	Recursion: the leap of faith	392
DSA	Recursion: the leap of faith	393
DESIGN	Design a food delivery order flow	411
PRACTIC	Practice — Recursion: the leap of faith	429
088	The call stack, drawn	435
DSA	The call stack, drawn	436
DESIGN	Design a ride-hailing booking flow	454
PRACTIC	Practice — The call stack, drawn	474
089	Writing a recursive function that terminates	480
DSA	Writing a recursive function that terminates	481
DESIGN	Design a rate limiter, at the object level	499
PRACTIC	Practice — Writing a recursive function that terminates	518
090	Recursion on arrays and strings	524
DSA	Recursion on arrays and strings	525
DESIGN	Design an in-memory cache with eviction	542
PRACTIC	Practice — Recursion on arrays and strings	562
091	Subsets: the include-or-exclude tree	568
DSA	Subsets: the include-or-exclude tree	569
DESIGN	Design a logging framework	586
PRACTIC	Practice — Subsets: the include-or-exclude tree	604
092	Permutations	610
DSA	Permutations	611
DESIGN	Design a notification service	630
PRACTIC	Practice — Permutations	649
093	Combinations and combination sum	655
DSA	Combinations and combination sum	656
DESIGN	Design a file system	674
PRACTIC	Practice — Combinations and combination sum	692
094	Backtracking: the undo step	697
DSA	Backtracking: the undo step	698
DESIGN	Design snake and ladder	717
PRACTIC	Practice — Backtracking: the undo step	739
095	N-Queens and constraint grids	745
DSA	N-Queens and constraint grids	746
DESIGN	Design an online auction	764
PRACTIC	Practice — N-Queens and constraint grids	786
096	Sudoku, word search, and grid backtracking	791
DSA	Sudoku, word search, and grid backtracking	792

DESIGN	Low-level design revision and full mock	811
PRACTICE	Practice — Sudoku, word search, and grid backtracking	833
097	Recursion and backtracking revision and mock round	839
DSA	Recursion and backtracking revision and mock round	840
DESIGN	What scale actually means, in numbers	859
PRACTICE	Practice — Recursion and backtracking revision and mock round	874

BACK MATTER

A	Appendix A - The nine sections, and what each is for	881
B	Appendix B - Recall cards	883
C	Appendix C - The complete 180-day map	910
D	Appendix D - Glossary	911
E	Appendix E - References and further reading, by section	930
F	Appendix F - Colophon: how this book was made	932
G	Appendix G - Problem index	933
H	Index	939

List of figures

78.1	System design - Design a parking lot	2
78.2	System design - Design a parking lot	2
79.1	System design - Design an elevator system	46
79.2	System design - Design an elevator system	46
80.1	System design - Design an ATM	89
80.2	System design - Design an ATM	89
81.1	System design - Design a vending machine	133
81.2	System design - Design a vending machine	133
82.1	System design - Design a library management system	175
82.2	System design - Design a library management system	175
83.1	System design - Design tic-tac-toe, and then chess	218
83.2	System design - Design tic-tac-toe, and then chess	218
84.1	System design - Design a deck of cards and a card game	261
85.1	System design - Design Splitwise	305
86.1	System design - Design BookMyShow	349
86.2	System design - Design BookMyShow	349
87.1	System design - Design a food delivery order flow	392
87.2	System design - Design a food delivery order flow	392
88.1	System design - Design a ride-hailing booking flow	435
88.2	System design - Design a ride-hailing booking flow	435
89.1	System design - Design a rate limiter, at the object level	480
90.1	System design - Design an in-memory cache with eviction	524
92.1	System design - Design a notification service	610
92.2	System design - Design a notification service	610
93.1	System design - Design a file system	655
94.1	System design - Design snake and ladder	697
94.2	System design - Design snake and ladder	697
95.1	System design - Design an online auction	745
95.2	System design - Design an online auction	745
96.1	System design - Low-level design revision and full mock	791
96.2	System design - Low-level design revision and full mock	791
97.1	Data structures and algorithms - Recursion and backtracking revision and mock round	839
97.2	System design - What scale actually means, in numbers	839



PART X

Linked lists

Days 78 to 86 · 9 chapters

Alongside, on the system design track:

Low-level design case studies

078 Nodes and links: pointers without pointers

079 Traversal, insertion, and deletion

080 The dummy head trick

081 Reversing a linked list

082 Finding the middle, and the runner technique

083 Cycle detection, and why Floyd's algorithm works

084 Merging and sorting linked lists

085 Doubly and circular linked lists

086 Linked lists revision and mock round

Nodes and links: pointers without pointers

DATA STRUCTURES AND ALGORITHMS

Nodes and links: pointers without pointers

You can draw a linked list in memory and say why it is not an array.

SYSTEM DESIGN

Design a parking lot

You can produce classes, an interface for pricing, and a spot-allocation strategy.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *What is a linked list, and when is it better than an array?*
- *Design a parking lot. Multiple floors, multiple vehicle types.*

Nodes and links: pointers without pointers

1 What this is, and why they ask it

A **linked list** is a chain of small objects. Each object — a **node** — holds one value and a reference to the next node. You keep hold of the first node, called the **head**, and everything else is reached by following the chain.

That is the entire structure, and everything about it follows from one difference with an array: an array's elements sit **next to each other in memory** and are found by arithmetic, while a linked list's nodes sit **wherever they happen to be** and are found by following references. So an array can jump straight to element 500 and a linked list cannot. And a linked list can insert in the middle without moving anything, while an array must shift everything after the gap.

They ask it because “when would you use a linked list instead of an array?” is a question with a genuinely honest answer that most candidates get wrong in both directions. Some say “linked lists are faster for insertion” without the qualifier that makes it true. Some say “never use them” without noticing that the LRU cache they just built on [day 076](#) needs one. It is also the foundation for the next eight days, and for trees and graphs after that — a tree is a node with two links instead of one, and nothing else changes.

2 The story

There are forty flats in Prakash's building and the association has a way of passing on news that was set up long before he moved in.

It used to be that the secretary called everybody. Forty calls, on a Sunday, about the water tanker being late or the lift being serviced on Tuesday. He was seventy-one and it took him most of a morning, and by the end of it he had usually forgotten two flats and rung one twice.

So they changed it. Now every flat has exactly one number saved for this — the next flat along. 101 has 102's number. 102 has 103's. 103 has 201's, because that is how the building runs. And the last flat, 404, has nobody, which is how you know the message has finished going round.

The secretary makes one call. Then he goes back to his tea.

Prakash likes it and he has also noticed the two things about it that everyone in the building has noticed.

The first is that you cannot skip. When the plumber told him he would be at flat 302 in ten minutes and Prakash wanted to warn them, he could not — he does not have 302's number. Nobody in the building has anybody's number except the one they were given. To get a message to 302 you start at 101 and you go through everybody in between, or you walk up.

The second is what happened when the Menons moved into 204, which had been empty for two years. It took about a minute. 203 saved the Menons' number instead of 205's, and the Menons saved 205's, and that was it. Nobody else in the building changed anything or even knew. When old Mr Iyer in 302 passed away and the flat was locked up, it was the same in reverse: 301 saved 303's number and the chain closed over the gap, and again nobody else was affected.

The one thing that goes wrong, and it goes wrong about twice a year, is when somebody does not answer. Then the message stops there. Everybody after them hears nothing, and they do not know they have heard nothing, and the first anyone finds out is when half the building turns up on the wrong day.

3 The idea in plain English

Each flat is a **node**. The number saved in each flat's phone is the **link** to the next node. The secretary's own number for 101 is the **head**. And 404 having nobody is how the chain ends.

A node, in code

```
class Node:
    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value          # the flat's news
        self.next = next           # the one number they have saved
```

Two fields. That is the whole data structure. A list of three values is three of these objects, each holding the next:

```
third = Node(3)
second = Node(2, third)
first = Node(1, second)
head = first
```

`head` is not a special object. It is just a name for the first node, and losing that name loses the whole list, because nothing else points at it.

“Pointers without pointers”

In C you would write `struct Node *next` and think about memory addresses. Python has no pointers and you never need to think about addresses — but the mechanism is the same one, and it is worth being precise about it because interviewers probe here.

In Python, a variable does not hold an object. It holds a reference to one. When you write `second = Node(2, third)`, the name `second` refers to a node object living somewhere on the heap, and `first.next` refers to *the same object*. Two names, one object. That is why:

```
first.next.value = 99
print(second.value)           # 99 – same object, seen through a different name
```

So `node.next` is exactly a pointer: something that lets you get to another object without containing it. What Python removes is address arithmetic and manual freeing, not the idea.

The difference that decides everything

An array — a Python `list` — is a **contiguous block of slots**. From [day 009](#): the list knows the address of slot 0, and element `i` is at `start + i × slot_size`. One multiplication and one addition, whatever `i` is. That is why `numbers[500_000]` is as fast as `numbers[0]`.

A linked list has no such arithmetic available, because the nodes are **not next to each other**. Node 5 could be a megabyte away from node 4. The only way to reach the 500,000th node is to start at the head and follow `next` 500,000 times.

array:	one multiply and one add	→ O(1) access
linked list:	follow k references	→ O(k) access

Everything else in this phase is a consequence of that one sentence.

What you get in exchange

Insertion and deletion in the middle are O(1) — if you already have the node. That qualifier is the entire honest answer and most people leave it out.

To delete a node from an array you must shift every element after it left by one, which is O(n). To delete a node from a linked list you set one reference:

```
previous.next = node_to_remove.next           # the chain closes over the gap
```

One assignment. 203 saving 205's number. Nothing else in the building changes, and it costs the same whether the building has forty flats or forty thousand.

But if all you have is a *position* — “delete the 500th element” — you must walk to it first, and that walk is $O(n)$. So the honest complexity is:

```
delete, given the node:      0(1)
delete, given the position:  0(n) to find + 0(1) to unlink = 0(n)
delete from an array, given the position: 0(1) to find + 0(n) to shift = 0(n)
```

Both are $O(n)$ when you are given a position. The linked list wins only when the node is already in your hand — which is exactly the LRU cache, where a hash map hands you the node.

The other two differences, which are real

No reallocation. A Python list that grows past its capacity allocates a bigger block and copies everything across. A linked list never copies anything; it just makes one more small object. That matters when elements are huge or when a copy pause is unacceptable.

Worse memory and worse speed per element. Measured, at a million elements:

```
python list:  8 bytes per element (a pointer in the block)
Node object:  48 bytes with __slots__, 336 bytes without
```

Six times the memory with `__slots__`, forty times without. And walking the chain was $2.0\times$ **slower** than walking the list, because each step is an attribute lookup on an object that may be anywhere in memory, rather than the next slot in a block the processor has already fetched.

That second effect is **cache locality**, and it is the reason experienced engineers reach for arrays by default. When the processor fetches memory it fetches a whole 64-byte line, so reading an array brings the next several elements along for free. Reading a linked list brings a node and whatever happens to be beside it, which is usually nothing useful.

So when is a linked list actually right?

Four honest cases, and you should be able to give them without hedging:

- 1. You already hold the node and must splice it out in $O(1)$.** The LRU cache. This is the big one.
- 2. You are always inserting or removing at the front.** Measured: 100,000 front insertions took **1.443 s** with `list.insert(0, x)` and **0.0246 s** building a chain — about **59 $\times$** . Though in Python you would reach for a `deque`, which is a linked list of blocks and gets both.
- 3. Copying is unacceptable.** Real-time systems where a reallocation pause matters, or elements too large to copy.

4. **The structure is the point.** A tree is a node with two links. A graph adjacency list is a list of chains. Everything after this phase is built out of nodes.

And the honest fifth case: **because the interviewer said so.** Half the problems in the next eight days exist to test pointer manipulation, not to be good engineering.

The failure mode

Mr Iyer not answering the phone is a **broken link**. In Python, a `next` that is `None` when you expected a node gives you:

```
AttributeError: 'NoneType' object has no attribute 'next'
```

That single error will account for most of your debugging for the next eight days, and the two defences are: check `while node is not None`, and use a **dummy head** so the first node is never a special case. That is [day 080](#), and it is the most useful trick in the phase.

4 The picture

An array and a linked list holding the same four values.

ARRAY – one block, elements adjacent, found by arithmetic

address	1000	1008	1016	1024
	+-----+-----+-----+-----+			
value	7	3	9	4
	+-----+-----+-----+-----+			
index	0	1	2	3

`numbers[2] → 1000 + 2*8 = 1016 → one multiply, one add, 0(1)`

LINKED LIST – separate objects, scattered, found by following

head			
v			
+-----+-----+-----+-----+			
7	----->	3	----->
		9	----->
		4	None
+-----+-----+-----+-----+			
@ 4200		@ 9016	@ 4408
			@ 7712

the third node → `head.next.next` → follow 2 links, 0(k)
the addresses are in no order at all, and that is the whole point

What to notice: the array's addresses go up by 8 every time and the linked list's addresses are random. That randomness is what buys the cheap insertion and what costs the fast access.

Now the insertion, which is the operation the structure exists for:

```
insert 5 between 3 and 9
```

```
before:    ... [3| ]---->[9| ]---->[4|None]
```

```
step 1: new node points at 9      [5| ]---->[9| ]
```

```
step 2: 3 points at the new node [3| ]---->[5| ]
```

```
after:     ... [3| ]---->[5| ]---->[9| ]---->[4|None]
```

Two assignments. Node 9 and node 4 were not touched, moved, or copied. ORDER MATTERS: do step 1 first. If you set 3.next = 5 first, you have lost your only reference to 9 and the rest of the list is gone.

And the deletion:

```
delete 9, given the node BEFORE it
```

```
before:    [3| ]---->[9| ]---->[4|None]
           previous node
```

```
previous.next = node.next
```

```
after:     [3| ]----->[4|None]
```

One assignment. Node 9 still points at 4, but nothing points at 9, so Python collects it. 203 saving 205's number.

5 The code, built step by step

Step 1 — the node

```
class Node:
    """One link in the chain. A value and a reference to the next node."""

    __slots__ = ("value", "next")          # 48 bytes instead of 336

    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value
        self.next = next
```

`__slots__` stops each node from carrying its own attribute dictionary. Mention it once and move on; in an interview the plain version is fine.

Step 2 — building a list, and the loop that appears in every problem

```
def from_values(values: list[int]) → Node | None:
    head: Node | None = None
    for value in reversed(values):          # build backwards
        head = Node(value, head)          # each new node points at the old head
    return head
```

Building **backwards** is the trick, and it is worth understanding rather than memorising: pushing at the front is the only $O(1)$ insertion when you hold nothing but the head. Reverse the input, push each value at the front, and the list comes out in the original order.

Step 3 — the traversal loop

```
node = head
while node is not None:
    ...                                # do something with node.value
    node = node.next                  # step forward
```

This loop is the phase. It will appear in every problem for the next eight days. Two things about it: the condition is `is not None`, not `while node`, because a node holding the value `0` is perfectly real and `while node` happens to work only because `Node` has no `__bool__` — do not rely on that. And the last line must be inside the loop, or it never ends.

Step 4 — length and access by position

```
def length(head: Node | None) → int:
    count = 0
    node = head
    while node is not None:
        count += 1
        node = node.next
    return count
```

$O(n)$, and there is no way to make it $O(1)$ without storing a count separately. An array knows its length; a chain does not. That is worth saying out loud when the interviewer asks for the `k`th element.

```
def value_at(head: Node | None, index: int) → int:
    node = head
    for _ in range(index):
        # walk `index` links
        if node is None:
            raise IndexError("index out of range")
        node = node.next
    if node is None:
        raise IndexError("index out of range")
    return node.value
```

$O(\text{index})$. Compare with `numbers[index]` on a list, which is one multiply and one add. This function is the reason you do not use a linked list when you need random access.

Step 5 — insertion after a node you hold

```
def insert_after(node: Node, value: int) → Node:
    node.next = Node(value, node.next)    # build first, then relink
    return node.next
```

One line, and it is correct because `Node(value, node.next)` is fully constructed *before* `node.next` is reassigned. Written as two statements in the wrong order, it loses the tail.

Step 6 — deletion, and the two ways it is asked

```
def delete_after(node: Node) → None:
    if node.next is not None:
        node.next = node.next.next    # close the gap
```

Given the node *before* the target: one assignment, $O(1)$.

Given only the target and no head — a genuine interview question, LeetCode 237 — you cannot reach the node before it. The trick is to **become the next node**:

```
def delete_this_node(node: Node) → None:
    """Delete a node when you are given only that node (never the tail)."""
    node.value = node.next.value    # copy the successor's value into me
    node.next = node.next.next    # then unlink the successor
```

You have not deleted the node you were given; you have deleted the one after it, having first stolen its value. The effect is identical from the outside. It fails on the last node, and saying so unprompted is the point of the question.

The complete solution

```
class Node:
    """One link in a singly linked list: a value and a reference to the next."""

    __slots__ = ("value", "next")

    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value
        self.next = next

    def __repr__(self) → str:
        return f"Node({self.value})"

def from_values(values: list[int]) → Node | None:
    """Build a list from a Python list. Build BACKWARDS: pushing at the front
    is the only O(1) insertion available when you hold only the head."""
    head: Node | None = None
    for value in reversed(values):
        head = Node(value, head)
    return head

def to_values(head: Node | None) → list[int]:
    """The traversal loop, which is every problem in this phase."""
    values: list[int] = []
    node = head
    while node is not None:
        values.append(node.value)
        node = node.next
    return values

def length(head: Node | None) → int:
    """O(n). A chain does not know its own length; an array does."""
    count = 0
    node = head
    while node is not None:
        count += 1
        node = node.next
    return count

def value_at(head: Node | None, index: int) → int:
    """O(index), against O(1) for an array. This is the cost of the structure."""
    if index < 0:
        raise IndexError("index must be non-negative")
    node = head
    for _ in range(index):
        if node is None:
            raise IndexError("index out of range")
        node = node.next
    if node is None:
        raise IndexError("index out of range")
    return node.value

def find(head: Node | None, value: int) → Node | None:
    """Return the first node with this value, or None. O(n) – no shortcut."""
    node = head
    while node is not None:
        if node.value == value:
```

```

        return node
        node = node.next
    return None

def push_front(head: Node | None, value: int) → Node:
    """O(1), and the reason linked lists are good at stacks.
    Returns the NEW head – the caller must reassign."""
    return Node(value, head)

def insert_after(node: Node, value: int) → Node:
    """O(1) given the node. Construct first, THEN relink, or the tail is lost."""
    node.next = Node(value, node.next)
    return node.next

def delete_after(node: Node) → int | None:
    """Remove the node following `node`. O(1). Returns the removed value."""
    target = node.next
    if target is None:
        return None
    node.next = target.next          # close the gap over the target
    target.next = None              # not required, but keeps stale links out
    return target.value

def delete_value(head: Node | None, value: int) → Node | None:
    """Remove the FIRST node with this value. Returns the (possibly new) head.

    Note the special case for the head – this is exactly the ugliness that the
    dummy-head trick on day 080 removes.
    """
    if head is None:
        return None
    if head.value == value:          # the special case
        return head.next

    node = head
    while node.next is not None:
        if node.next.value == value:
            node.next = node.next.next
            return head
        node = node.next
    return head

def delete_this_node(node: Node) → None:
    """LeetCode 237: delete a node given ONLY that node, never the tail.

    You cannot reach the node before it, so instead you steal the successor's
    value and unlink the successor. Indistinguishable from outside, and
    impossible for the last node – say that before being asked.
    """
    if node.next is None:
        raise ValueError("cannot delete the last node without its predecessor")
    node.value = node.next.value
    node.next = node.next.next

if __name__ == "__main__":
    head = from_values([7, 3, 9, 4])
    print(to_values(head))          # [7, 3, 9, 4]

```

```

print(length(head), value_at(head, 2))      # 4 9

node = find(head, 3)
insert_after(node, 5)
print(to_values(head))                     # [7, 3, 5, 9, 4]

delete_after(node)
print(to_values(head))                     # [7, 3, 9, 4]

head = push_front(head, 1)
print(to_values(head))                     # [1, 7, 3, 9, 4]

head = delete_value(head, 1)                # the head case
print(to_values(head))                     # [7, 3, 9, 4]
head = delete_value(head, 9)                # the middle case
print(to_values(head))                     # [7, 3, 4]
head = delete_value(head, 99)               # not present
print(to_values(head))                     # [7, 3, 4]

print(to_values(from_values([])))           # []
print(length(None))                        # 0

victim = find(head, 3)
delete_this_node(victim)                   # steals 4's value
print(to_values(head))                     # [7, 4]

try:
    value_at(head, 9)
except IndexError as error:
    print(f"IndexError: {error}")           # IndexError: index out of range

```

6 What it costs

The operations, side by side with an array

	linked list	Python list (array)
access by index	$O(n)$ walk	$O(1)$ one multiply, one add
search for a value	$O(n)$	$O(n)$
insert at the front	$O(1)$	$O(n)$ shift everything
insert at the back	$O(n)$ without a tail reference;	
	$O(1)$ with one	$O(1)$ amortised
insert given the node	$O(1)$	n/a
insert at a known position	$O(n)$ walk + $O(1)$	$O(1)$ find + $O(n)$ shift = both $O(n)$
delete given the node	$O(1)$ one write	n/a
delete at a known position	$O(n)$	$O(n)$
length	$O(n)$ unless stored	$O(1)$
memory per element	48 B (<code>__slots__</code>)	8 B

The two rows that decide real arguments are `insert at the front` and `insert given the node`. Everything else is a draw or a loss.

The measurements

All on a million elements unless stated, on this machine.


```
walking the whole structure and summing:
python list      0.0621 s
linked list      0.1232 s      2.0x slower
```

Two times, not a hundred, and it is worth being accurate about that. The slowdown is one attribute lookup per step plus poor cache behaviour, not an asymptotic difference — both are $O(n)$.

```
100,000 insertions at the FRONT:
list.insert(0, x)  1.443 s
push_front on nodes 0.0246 s      about 59x faster
```

That is the linked list's real win, and it is the same measurement as `pop(0)` from [day 073](#) seen from the other end. In Python you would use a `deque`, which is a linked list of blocks and gets the $O(1)$ front for free.

```
memory per element:
python list slot      8 B (a pointer; the int objects are shared or separate)
Node with __slots__  48 B
Node without          336 B
```

Six times with `__slots__`, forty times without. At ten million elements that is 80 MB against 480 MB — and against 3.4 GB if you forget `__slots__`, which is the difference between fitting in memory and not.

Space

```
the list itself:  O(n) nodes
traversal:        O(1) — one variable, and this matters
```

Every algorithm in this phase should use $O(1)$ extra space. If your solution converts the list to a Python list, works on that, and rebuilds — which is legal and sometimes the right answer — say that you are trading $O(n)$ space for simplicity, because the interviewer is watching for it.

7 The traps

Trap 1 — losing the head

```
while head is not None:
    print(head.value)
    head = head.next      # the parameter now points at None
```

The loop works, prints correctly, and the caller's list is unaffected because Python passes the reference by value — but *inside* your function you no longer have the head, so anything after the loop that needs it is broken. **Always walk with a separate variable:** `node = head`, then move `node`.

Trap 2 — relinking in the wrong order

```
node.next = new_node          # WRONG ORDER
new_node.next = node.next     # now points at ITSELF
```

`node.next` has already been changed, so `new_node.next` is set to `new_node`. You have built a cycle of one node, and the next traversal hangs for ever with no error at all. **Point the new node at the rest of the list first, then attach it.**

Trap 3 — `AttributeError` on `None`

```
while node.next.value != target:    # two dots, two chances to be None
    node = node.next
```

```
AttributeError: 'NoneType' object has no attribute 'value'
```

The single most common error in this phase. Every `.next.next` is a promise that two more nodes exist. Guard it: `while node.next is not None and node.next.value != target`.

Trap 4 — the head as a special case

```
def delete_value(head, value):
    node = head
    while node.next is not None:
        if node.next.value == value:
            node.next = node.next.next
            return head
        node = node.next
    return head                    # never checks the head itself
```

Deleting the first element silently does nothing. Every problem in this phase has this bug available, which is exactly why [day 080](#) exists — a dummy node before the head makes the first element ordinary.

Trap 5 — `while node:` instead of `while node is not None:`

These happen to be equivalent for a plain `Node` class, and stop being equivalent the moment somebody adds `__len__` or `__bool__` to it — at which point a node holding an empty value is treated as the end of the list. Write `is not None`. It also reads as what

you mean.

Trap 6 — forgetting that the caller's head does not update

```
push_front(head, 5)           # return value discarded
print(to_values(head))        # the 5 is not there
```

Python passes a reference *by value*, so rebinding `head` inside a function does not change the caller's variable. Any operation that can change which node is first must **return the new head**, and the caller must write `head = push_front(head, 5)`. This is the second-most-common bug in the phase and it produces no error at all.

Trap 7 — building a cycle by accident

```
last.next = head              # now it never ends
```

```
KeyboardInterrupt
```

is what you get, eventually, from `to_values` on a cyclic list — or a `MemoryError` when the output list eats your RAM. There is no exception for “this list has a cycle”; detecting it is [day 083](#).

Trap 8 — assuming a linked list is faster because it is “ $O(1)$ insertion”

The claim is true only when you already hold the node. If you have a position, you must walk to it, and that walk is $O(n)$ — the same as the array's shift, but with worse constants and worse cache behaviour. Say the qualifier every time: **$O(1)$ insertion *given the node***.

8 In the interview

How it gets asked

- The direct version, usually as a warm-up before a harder problem: “What is a linked list, and when would you use one instead of an array?”
- The implementation version: “Implement a singly linked list with insert, delete and search.”
- The trick version: “Delete a node from a linked list when you are given only that node.” LeetCode 237. It is a test of whether you will say “that is impossible for the tail”.
- The memory version: “Why is iterating a linked list slower than iterating an array, if both are $O(n)$?” — they want cache locality.

What to say out loud, in the first ninety seconds

1. **Define it by the one difference.** “An array’s elements are contiguous in memory, so element `i` is found by arithmetic. A linked list’s nodes are scattered and each holds a reference to the next, so the only way to reach the `k` th is to follow `k` links.”
2. **Give both consequences immediately.** “So access by index is $O(n)$ instead of $O(1)$, and in exchange, inserting or deleting does not move anything.”
3. **Say the qualifier that makes the trade honest.** “Insertion is $O(1)$ *given the node*. If I am given a position, I have to walk there first, so both structures are $O(n)$ — the array pays for the shift and the list pays for the walk.”
4. **Name the real use.** “Where it genuinely wins is when something else already hands me the node — the LRU cache, where a hash map maps a key straight to its node, so unlinking is two assignments.”
5. **Concede the costs.** “And the costs are real: about six times the memory per element in Python with `__slots__`, and about twice the traversal time, because each step is a reference to somewhere else in memory rather than the next slot in a block the processor already has.”

The follow-ups

“**If both traversals are $O(n)$, why is the linked list slower?**” “Cache locality. When the processor reads memory it fetches a whole cache line, typically 64 bytes, so reading an array element brings the next several along for free — the loop mostly runs out of cache. A linked list’s nodes are wherever the allocator put them, so each step is likely to be a fresh fetch, and the prefetcher cannot predict where you are going next because the address is inside the data. I measured about twice the time on a million elements. Same complexity, different constant, and the constant is what you feel.”

“**When would you actually use one?**” “Four cases. When something else hands me the node and I must splice it out in $O(1)$ — that is the LRU cache and it is the main one. When I only ever add and remove at the front, where 100,000 front insertions were 1.4 seconds with `list.insert(0, x)` and 0.02 seconds with nodes. When a reallocation copy is unacceptable, in real-time code or with very large elements. And when the structure itself is the point — a tree is a node with two links, so everything after this phase is built out of these. In day-to-day Python I would use a `list`, or a `deque` if I need both ends, because a deque is a linked list of blocks and gets both properties.”

“**Delete a node given only that node.**” “I cannot reach the node before it, so I cannot unlink it directly. Instead I copy the next node’s value into this node and unlink the next node — from outside, the effect is identical. It fails on the tail, because there is no suc-

cessor to steal from, and I would state that as a precondition rather than let you find it. It also breaks any reference someone else is holding to the successor node, which is worth mentioning if the nodes are shared.”

“How would you make length $O(1)$?” “Store it. Wrap the head in a small list class that keeps a `size` field and updates it on every insertion and deletion. That is what real implementations do. The cost is that every mutation now has to go through the class — a bare node graph cannot maintain it — and that is exactly the trade a `LinkedList` class exists to make.”

“Singly or doubly linked?” “Doubly if I need to remove a node in $O(1)$ when I only have that node, because unlinking needs both neighbours. That is why the LRU cache is doubly linked. The cost is one extra reference per node — eight more bytes — and twice as many pointer updates to get wrong, which is a real source of bugs. Singly linked when I only ever walk forwards.”

A model answer

Asked: *what is a linked list, and when is it better than an array?*

“A linked list is a chain of small objects. Each node holds a value and a reference to the next node, and you keep the first one, the head. Following `next` repeatedly is the only way to get anywhere.

The whole structure comes out of one difference from an array. An array is a contiguous block, so the runtime knows where slot zero is and finds element `i` with one multiplication and one addition — constant time, whatever `i` is. A linked list’s nodes are wherever the allocator happened to put them; node five might be a megabyte from node four. There is no arithmetic that gets you to the five-hundredth node, so you follow five hundred references.

So access is $O(n)$ instead of $O(1)$. What I get in exchange is that inserting and deleting move nothing. Deleting from the middle of an array shifts every following element left; deleting from a linked list is one assignment — the previous node points at the next one, and the chain closes over the gap. That is the same whether the list has forty elements or forty thousand.

But I want to state the qualifier, because it is where people overclaim. That insertion is $O(1)$ *given the node*. If I am given a position — ‘delete the five-hundredth element’ — I have to walk there first, which is $O(n)$, and then both structures are $O(n)$: the array pays for the shift and the list pays for the walk. So ‘linked lists are faster for insertion’ is only true when something else already put the node in my hand.

Which is exactly the LRU cache. A hash map maps the key straight to the node, so unlinking it is two assignments and never a search. That is the case where I would genuinely reach for one. The others are: when I only ever work at the front — I measured 100,000 front insertions at 1.4 seconds with `list.insert(0, x)` and 0.02 seconds building a chain, about sixty times — when a reallocation copy is unacceptable, and when the structure is the point, because a tree is just a node with two links.

The costs are real and I would say them without being asked. Memory: a Python list slot is eight bytes; a node with `__slots__` is forty-eight, and without `__slots__` it is over three hundred. So six times the memory at best. Speed: I measured walking a million nodes at about twice the time of walking a million list elements. Same complexity, worse constant, and the reason is cache locality — the processor fetches a whole cache line at a time, so an array brings the next few elements along for free, while each node is a fresh trip to memory that the prefetcher cannot anticipate.

So in ordinary Python I use a `list`, or a `deque` when I need both ends cheaply — a deque is internally a linked list of blocks, which is how it gets $O(1)$ at both ends without the per-element overhead. I write an explicit linked list when I need $O(1)$ splicing of a node I already hold, or when the interviewer has asked me to.”

9 Recall card

- A node is a value plus a reference to the next node; the head is just a name for the first one, and losing it loses the list. In Python a variable holds a **reference**, not an object — so `node.next` is exactly a pointer, minus the address arithmetic.
- One difference explains everything: an array is contiguous and finds element `i` by arithmetic; a linked list is scattered and must follow `k` links. So access is $O(n)$ vs $O(1)$ — and in exchange, insertion and deletion **move nothing**.
- Say the qualifier or the claim is wrong: insertion and deletion are $O(1)$ *given the node*. Given only a **position**, both structures are $O(n)$ — the array pays for the shift, the list pays for the walk. The genuine win is when a **hash map hands you the node** (the LRU cache).
- The measurements, at 1,000,000 elements: traversal 0.062 s list vs 0.123 s linked — $2.0 \times$ (cache locality, not complexity) · 100,000 front insertions: 1.443 s vs 0.0246 s — $\sim 59 \times$ · memory 8 B per list slot vs 48 B per node with `__slots__` (336 B without) — $6 \times$ at best.
- The three bugs that will cost you the next eight days. Relink in the **wrong order** and a node points at itself — build the new node first, then attach. `AttributeError: 'NoneType' object has no attribute 'next'` — every `.next.next` promises two nodes exist. And **rebinding head** inside a function does not change the caller's **variable** — return the new head, always. The head as a special case is what **day 080** deletes.

Design a parking lot

1 What this is, and why they ask it

“Design a parking lot” is the most-asked low-level design question there is. You get forty minutes, a one-line prompt, and no further detail. The output is classes: what objects exist, what each is responsible for, how they relate, and real code for the two or three that carry the design.

Yesterday you learned the [six-move script](#). Today you run it end to end on the canonical prompt, and the thing to take away is not the diagram. It is the habit of finding the **one place where the design is actually decided** — here, which spot a vehicle gets — putting an interface there, and showing two implementations.

They ask this one because everybody has been in a car park, so no domain knowledge separates candidates, and because it has a perfect curve ball available at minute thirty-five: “*now add electric-vehicle charging bays*”. A design where the allocation rule is buried inside the main method needs surgery for that. A design with the rule isolated needs one class. The interviewer can see the difference in ninety seconds.

2 The story

The wedding hall on the main road has a ground beside it that holds about ninety cars, and on a Saturday in December it is full by seven.

Munna has run that ground for nine years. He stands at the gate in a reflective jacket that is too big for him, and he decides where every car goes.

There is nothing marked on the ground. No lines, no numbers. He knows it the way you know your own house in the dark. He knows the corner by the generator where a big car cannot turn, and the strip along the back wall where the water collects after rain, and the six places near the gate that he keeps empty until half past seven, because that is when the older relatives arrive and they should not have to walk.

When a car comes in he looks at it for about a second — the size of it, and whether the driver is going to be any good — and then he points. Left, left, straight, stop. He hands over a small numbered token and drops the matching one into his shirt pocket in the order they arrived, and that pocket is how he finds any car in ninety seconds when somebody wants to leave at eleven.

The two things that go wrong, go wrong the same way every time.

The first is when the ground is nearly full and a very large car arrives. There is space — three or four gaps — but not one gap that will take it. He has to turn the car away, and the driver never believes him, because from the gate it looks like there is plenty of room.

The second is the other gate. On big functions they open the side gate and put Munna's nephew on it, and twice now the two of them have sent cars into the same gap at the same moment, because neither could see what the other had just done. The first time everybody laughed. The second time a man reversed into a scooter.

3 The idea in plain English

Munna's job is the design. He does four things, and the design has four parts.

He decides where each car goes. That is **spot allocation**, and it is the interesting part — because his rule is not “the first free space”. It is “the first free space that fits, except the six near the gate before half past seven”. Rules like that arrive constantly, and they must land in one place.

He looks at the car and judges its size. That is the **vehicle type**, and the rule that a large gap can take a small car but not the reverse. Munna's first failure — the ground looks empty and the bus cannot fit — is a *fallback* rule, and it belongs with allocation, not with the ground.

He hands out a token and keeps the matching one. That is the **ticket**: a record linking a vehicle to a spot and a time, which is what makes exit and payment possible at all.

Two gates cannot see each other. That is **concurrency**, and it is the part that separates candidates. Finding a free spot and claiming it are two steps, and something can happen in between.

The classes, straight out of that

Seven, each with one line of responsibility. If you cannot write the line, the class is wrong.

- **Vehicle** — a number plate and a size. Nothing else.
- **Spot** — a location, a size, and the vehicle currently in it if any.
- **Floor** — holds spots and reports what is free. Bookkeeping only, no policy.
- **Ticket** — links a plate to a spot with an entry time. Immutable once issued.
- **ParkingLot** — the entry point: `park`, `unpark`, and the free-space display.
- **SpotAllocation** (*interface*) — decides which free spot a vehicle gets.
- **PricingPolicy** (*interface*) — turns a size and a duration into an amount.

Two interfaces, five plain classes. The gate for an interface is always the same: **can you name a second implementation somebody would actually want?** Allocation: nearest-to-entrance, and spread-across-floors for lift congestion. Pricing: flat hourly, and weekend surge. Both pass. Nothing else does, so nothing else gets one.

Version one, and what breaks it

Start by saying the simple version out loud, because presenting the finished design as if it appeared fully formed is what makes an interviewer suspect memorisation.

```
def park(self, vehicle):
    for floor in self._floors:
        for spot in floor.spots:
            if spot.is_free() and spot.size == vehicle.size:
                spot.assign(vehicle)
                return Ticket(...)
    raise ParkingFull()
```

Fifteen lines, correct, and fine for a long time. It breaks the moment anyone says any of these:

- “A car should be allowed to use a bus spot when no car spots are free.” → a fallback ordering.
- “Spread cars across floors so the lift queue is shorter.” → a completely different rule.
- “Reserve floor one for electric vehicles.” → a filter on features.
- “Monthly pass holders get the reserved bay.” → a rule about the *driver*, not the vehicle.

Every one of those adds a branch to the method that every single car goes through. Four rules later it is ninety lines and two teams edit it. So:

```
class SpotAllocation(Protocol):
    def find(self, floors: list[Floor], vehicle: Vehicle) → Spot | None: ...
```

Three lines of interface, and `ParkingLot.park` now contains **no rules at all** — only sequencing.

The size rule, and where it belongs

Sizes are ordered: motorcycle < car < bus. A vehicle fits a spot when the spot is at least as large. But you do not want a motorcycle taking a bus bay while a bus is waiting, so the rule is: **try the exact size first, then progressively larger.**

That is a *policy* — a different car park might allow no upsizing at all — so it lives in the allocation implementation, not in `Floor`. `Floor` answers “which spots of exactly this size are free?” and nothing more. Keeping bookkeeping and policy apart is the single

most useful instinct in low-level design.

The ticket, and why it is immutable

A ticket records what happened: this plate, that spot, at this time. Nothing about it should change afterwards. Make it a frozen dataclass and the whole class of bugs where the exit code adjusts the entry time to make the fee come out differently simply cannot be written.

Payment produces a **separate** record, so that an unpaid ticket and a paid one are different objects rather than the same object with a mutated flag. That also gives you the audit trail for free.

4 The picture

The ground itself, so the size and fallback rules are visible:

```
FLOOR 1                                     size legend:  M = motorcycle  C = car  B = bus
+---+---+---+---+---+---+---+---+
|M | M | C | C | C | C | B | B |           entrance
|free|used|used|free|used|free|used|free|  ← this side
+---+---+---+---+---+---+---+---+

a CAR arrives:
  exact size (C) free?  yes, spots 4 and 6  → take the one nearest the entrance
a BUS arrives:
  exact size (B) free?  yes, spot 8         → take it
another BUS arrives:
  exact size (B) free?  no
  larger than B?       nothing is larger    → REJECT, even though 3 spots are free
                                                (Munna at the gate, being disbelieved)
a MOTORCYCLE arrives:
  exact size (M) free?  yes, spot 1         → take it
  (only if no M were free would it consider a C spot)
```

What to notice: the bus is rejected while three spots stand empty. That is not a bug and it is the thing to say out loud — **“free spots” and “usable spots” are different numbers, and the display board has to be honest about which it shows.**

The class diagram:

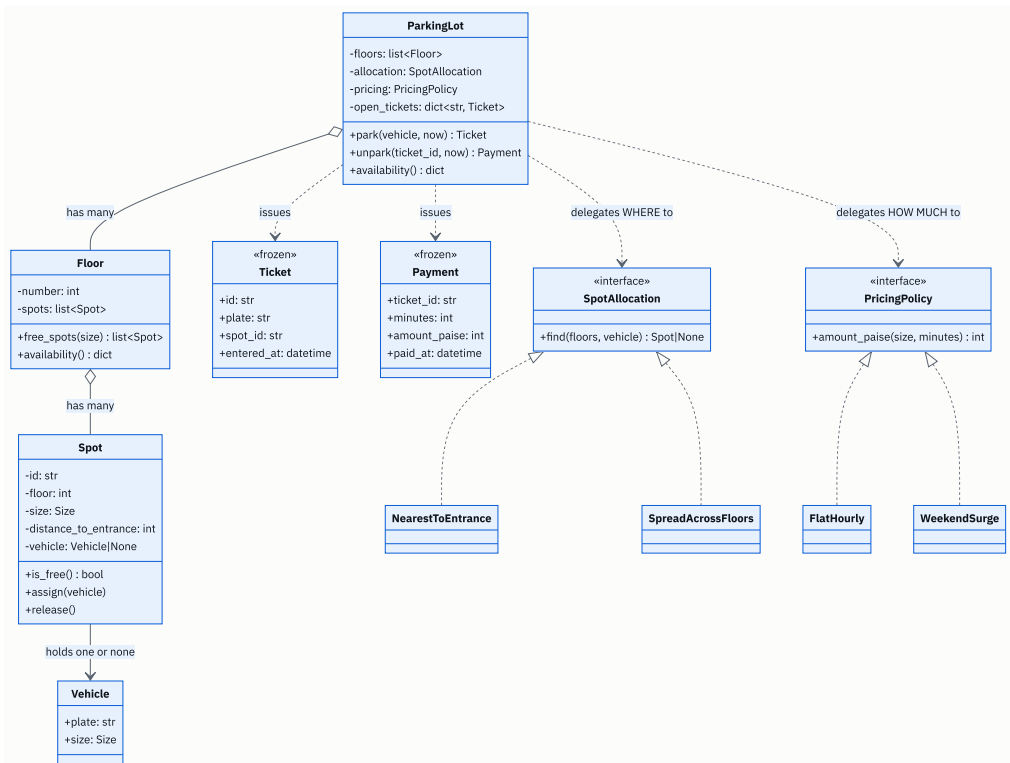


FIGURE 78.1

What to notice: the two dashed arrows out of `ParkingLot`. They are the design. Everything else is containment and record-keeping, and if you removed those two arrows you would have described a car park rather than designed one.

The entry sequence, with the race visible:

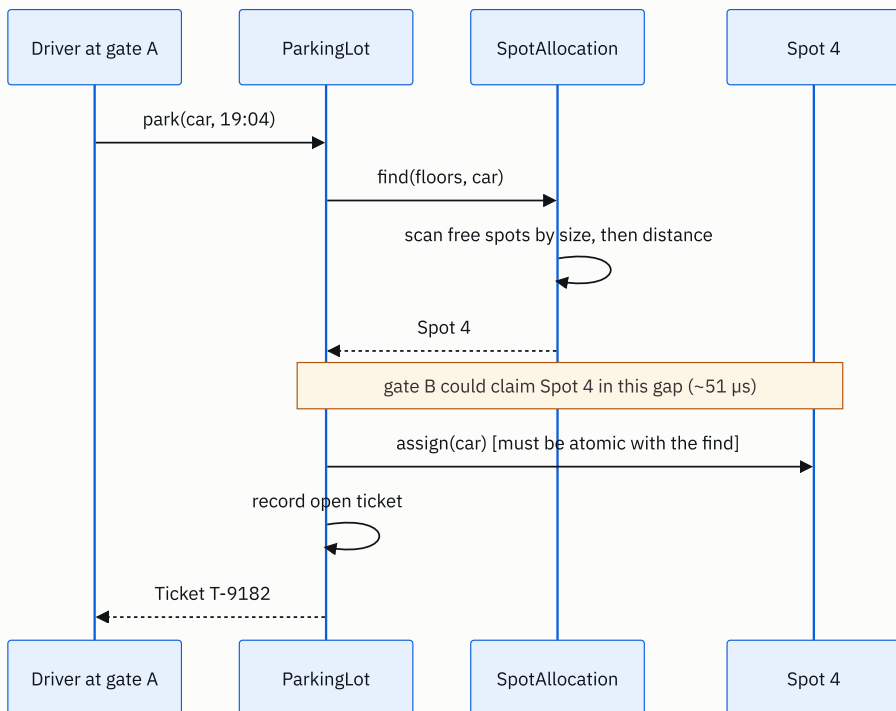


FIGURE 78.2

5 How it actually works

Move 1 · Clarify (minutes 0–5)

Four questions whose answers change the design, then the assumptions:

“Multiple floors, or one level?” — Multiple. **“How many vehicle sizes, and can a small vehicle use a larger spot?”** — Motorcycle, car, bus. Yes, but only when nothing of its own size is free. **“Is payment on exit, and is it time-based?”** — On exit, per hour, different rate per size. **“Roughly how many spots?”** — About a thousand.

“I will assume a ticket is issued at entry and settled at exit, that one vehicle occupies exactly one spot, and that we are designing the in-process object model rather than a database schema. I am not designing the payment gateway, number-plate recognition, or the mobile app.”

Move 2 · The nouns (minutes 5–13)

The seven classes from §3, each with its one-line responsibility, said aloud as you write them.

Move 3 · The value objects, written first

```
from dataclasses import dataclass
from datetime import datetime
from enum import IntEnum

class Size(IntEnum):
    MOTORCYCLE = 1
    CAR = 2
    BUS = 3
```

`IntEnum` rather than `Enum`, deliberately, so `Size.CAR ≤ Size.BUS` is true and the fallback rule is a comparison rather than a lookup table. Say that when you write it — small choices with a stated reason read very well.

```
@dataclass(frozen=True)
class Vehicle:
    plate: str
    size: Size

@dataclass(frozen=True)
class Ticket:
    id: str
    plate: str
    spot_id: str
    entered_at: datetime
```

Frozen, both of them. A ticket is a record of something that happened, and nothing downstream should be able to edit the entry time to change the fee.

Move 4 · The bookkeeping classes

```
class Spot:
    __slots__ = ("id", "floor", "size", "distance_to_entrance", "vehicle")

    def __init__(self, id: str, floor: int, size: Size, distance_to_entrance: int) → None:
        self.id = id
        self.floor = floor
        self.size = size
        self.distance_to_entrance = distance_to_entrance
        self.vehicle: Vehicle | None = None

    def is_free(self) → bool:
        return self.vehicle is None

    def assign(self, vehicle: Vehicle) → None:
        if self.vehicle is not None:
            raise SpotTaken(f"spot {self.id} already holds {self.vehicle.plate}")
        self.vehicle = vehicle
```

The check inside `assign` is not defensive noise. It is the **last line of defence against the two-gate race**, and it turns a silent double-park into an exception you can catch and retry. Point at it when you write it.

```
class Floor:
    def __init__(self, number: int, spots: list[Spot]) → None:
        self.number = number
        self.spots = spots

    def free_spots(self, size: Size) → list[Spot]:
        """Free spots of EXACTLY this size. Fallback is policy, not bookkeeping."""
        return [s for s in self.spots if s.is_free() and s.size == size]
```

One sentence of docstring doing a lot of work. `Floor` answers a factual question. It does not decide whether a motorcycle may take a car bay, because that is a rule that varies between car parks.

Move 5 · The interesting part — allocation

```
class SpotAllocation(Protocol):
    def find(self, floors: list[Floor], vehicle: Vehicle) → Spot | None: ...

class NearestToEntrance:
    """Exact size first, then progressively larger; within a size, nearest first."""

    def find(self, floors: list[Floor], vehicle: Vehicle) → Spot | None:
        for size in Size:
            # MOTORCYCLE, CAR, BUS in order
            if size < vehicle.size:
                continue
            # never smaller than the vehicle
            candidates = [s for floor in floors for s in floor.free_spots(size)]
            if candidates:
                return min(candidates, key=lambda s: (s.distance_to_entrance, s.id))
        return None
```

Read the loop out loud: *try the exact size, and only if nothing is free try the next size up*. The `if size < vehicle.size: continue` is the fallback rule, and it is three words long because `Size` is an `IntEnum`.

```
class SpreadAcrossFloors:
    """Send vehicles to the emptiest floor, to keep lift and ramp queues even."""
    def find(self, floors: list[Floor], vehicle: Vehicle) → Spot | None:
        best: Spot | None = None
        best_free = -1
        for size in Size:
            if size < vehicle.size:
                continue
            for floor in floors:
                free = floor.free_spots(size)
                if free and len(free) > best_free:
                    best_free, best = len(free), min(free, key=lambda s: s.id)
            if best is not None:
                return best
        return None
```

Two implementations, not one. This is the artefact the round is scored on: it demonstrates that the design survives a changed requirement rather than asserting it.

Move 6 • Pricing, the second interface

```
class PricingPolicy(Protocol):
    def amount_paise(self, size: Size, minutes: int) → int: ...

class FlatHourly:
    """Part hours round up. The first 15 minutes are free."""
    RATES = {Size.MOTORCYCLE: 1000, Size.CAR: 3000, Size.BUS: 8000} # paise per hour
    FREE_MINUTES = 15
    def amount_paise(self, size: Size, minutes: int) → int:
        if minutes ≤ self.FREE_MINUTES:
            return 0
        hours = -(-minutes // 60) # ceiling division
        return hours * self.RATES[size]
```

`-(-minutes // 60)` is ceiling division without importing anything, and rounding *up* is a business rule you should state: “part hours round up, which is what car parks do — I would confirm that.”

The lot itself, which holds no rules

```
class ParkingLot:
    def __init__(self, floors, allocation, pricing) → None:
        self._floors = floors
        self._allocation = allocation
        self._pricing = pricing
        self._open: dict[str, Ticket] = {}
        self._spots = {s.id: s for floor in floors for s in floor.spots}
        self._lock = threading.Lock()

    def park(self, vehicle: Vehicle, now: datetime) → Ticket:
        with self._lock:
            # find and claim, atomically
            spot = self._allocation.find(self._floors, vehicle)
            if spot is None:
                raise ParkingFull(f"no spot available for a {vehicle.size.name}")
            spot.assign(vehicle)
            ticket = Ticket(new_id(), vehicle.plate, spot.id, now)
            self._open[ticket.id] = ticket
        return ticket
```

Nine lines and **not one business rule**. It asks where, it claims, it records. Say that when you finish typing it — a class that only sequences is the goal, and it is what makes the electric-vehicle follow-up cheap.

```
def unpark(self, ticket_id: str, now: datetime) → Payment:
    ticket = self._open.pop(ticket_id, None)
    if ticket is None:
        raise UnknownTicket(ticket_id)
    # lost, or already settled
    minutes = max(0, int((now - ticket.entered_at).total_seconds() // 60))
    spot = self._spots[ticket.spot_id]
    amount = self._pricing.amount_paise(spot.size, minutes)
    spot.release()
    return Payment(ticket.id, minutes, amount, now)
```

`pop` rather than a lookup-then-delete, so the same ticket cannot be settled twice by two exit gates racing. That is a one-word decision worth narrating.

Move 7 • The display board

```
def availability(self) → dict[int, dict[str, int]]:
    return {
        floor.number: {size.name: len(floor.free_spots(size)) for size in Size}
        for floor in self._floors
    }
```

Per floor and per size, because a single total is a lie — as Munna’s rejected bus shows. If the board says “3 FREE” and a bus is turned away, the board is wrong even though the number is right.

Real systems that work this way

Every commercial car-park management system — **Amano McGann**, **Skidata**, **Flowbird** — has this shape: a spot inventory, a rate engine that is configured rather than coded, and a barrier controller. The rate engine being *configuration* is the real-world version of `PricingPolicy`: nobody ships a release to change a Sunday rate. Airport car parks add reservations, which turn the spot inventory into an availability calendar and change the concurrency question completely.

6 The numbers

The model fits in memory, and that decides the design

A thousand spots across five floors:

Spot with __slots__	$\sim 120 \text{ B} \times 1,000$	$= 120 \text{ KB}$
Floor	$\sim 200 \text{ B} \times 5$	$= 1 \text{ KB}$
Ticket (one per parked vehicle, at most)	$\sim 200 \text{ B} \times 1,000$	$= 200 \text{ KB}$
Vehicle	$\sim 120 \text{ B} \times 1,000$	$= 120 \text{ KB}$

total		$\approx 441 \text{ KB}$

Under half a megabyte. **So there is no reason to design the object model around a database.** Say the number and then say the conclusion — that is what this section is for, and it is the difference between “it will be small” and an argument.

Traffic, so you know what you are building for

1,000 spots, average stay 3 hours, open 16 hours	
turnovers per spot per day:	$16 / 3 \approx 5.3$
vehicles per day:	$1,000 \times 5.3 \approx 5,300$
events per day:	$5,300 \text{ entries} + 5,300 \text{ exits} = 10,600$
average:	$10,600 / (16 \times 3,600) \approx 0.18 \text{ events per second}$

Under one event every five seconds on average. Even at a peak twenty times the average, this is four events per second. That number matters because it tells you that a single lock around find-and-claim is *completely adequate* — you do not need lock-free anything, and proposing it would be a mistake.

The allocation scan

NearestToEntrance, worst case: scan all 1,000 spots $\times$ 3 size passes	$= 3,000 \text{ checks}$
at $\sim 50 \text{ ns}$ per check:	$150 \text{ } \mu\text{s}$

150 microseconds per arrival, at four arrivals per second, is 0.06 percent of one core. Fine. But if this were a 10,000-spot airport car park with a peak of 50 arrivals a second, the scan becomes 1.5 ms and 7.5 percent of a core — at which point you keep a per-floor, per-size list of free spots and the allocation is a `pop` from a list instead of a scan. **Say the threshold at which you would change it, not just that you might.**

The concurrency window, in real units

```
find a free spot    ~150 µs
assign it           ~1 µs
-----
unguarded window   ~151 µs

two gates, 4 arrivals/s at peak:
probability two arrivals land inside the same window ≈ 4 × 151 µs ≈ 6 × 10-4 per arrival
over 5,300 arrivals/day: about 3 collisions per day
```

Three double-allocations a day. That is Munna’s nephew and the scooter, quantified, and it is the strongest possible argument for holding the lock across find-and-claim rather than around each step.

Revenue arithmetic, if pricing comes up

```
5,300 vehicles/day × average 3 hours × ₹30/hour (car rate) = ₹4,77,000/day
a 1-minute error in the entry time, on every ticket:
5,300 × 1 min = 88 hours/day of unbilled time ≈ ₹2,650/day ≈ ₹9.7 lakh/year
```

Which is why the ticket is immutable and the clock is the server’s, not the barrier’s. A small correctness argument with a rupee figure behind it lands much harder than “we should be careful with times”.

7 The trade-offs

What this design gives up

Allocation is a scan, not an index. `free_spots` filters the whole floor every call. At a thousand spots that is 150 microseconds and irrelevant; at a hundred thousand it is not. The fix is a `dict[(floor, size)] → list[Spot]` of free spots maintained on assign and release, turning allocation into a `pop`. I would not build that at a thousand spots, and I would name the threshold — say ten thousand spots or fifty arrivals a second.

One global lock serialises all entries. At four events a second that is free. At a thousand it is the bottleneck, and the fix is a lock per floor, since two floors never contend for the same spot. The cost is that `SpreadAcrossFloors` reads several floors and would then need to take several locks — which is where deadlocks come from, so you would take them in a fixed order.

Everything is in memory, so a restart loses the car park. For a real installation the spot state and open tickets must be persisted, and then the concurrency answer changes from a lock to a conditional update: `UPDATE spots SET plate = ? WHERE id = ? AND plate IS NULL`, with zero rows updated meaning somebody else won. The class design does not change; only the claim step does.

The size hierarchy is a total order, and real car parks are not. Motorcycle < car < bus works until you meet a spot that is wide but low — fine for a car, not for a van — or an EV bay that is physically a car spot but only useful to some cars. The moment two dimensions matter, `Size` should become a **set of features** on the spot and a **set of requirements** on the vehicle, and the comparison becomes a subset test. I would not build that up front, and I would say exactly what would make me.

No reservations. Everything here assumes vehicles arrive and take what is free. Reservations turn the inventory into a calendar — a spot is not “free” but “free between these times” — and that is a genuinely different system, closer to BookMyShow than to this.

Lost tickets have no story. `unpark` needs a ticket id. In reality people lose them, and the answer is a lookup by number plate plus a flat lost-ticket fee, which needs a plate index and a policy. Worth naming even if you do not build it.

“I would change this design if...”

- **...spots exceed about ten thousand, or arrivals about fifty a second.** Then free-spot lists instead of scans, and a lock per floor.
- **...it must survive a restart.** Then persistence, and the claim becomes a conditional update.
- **...spot suitability stops being one ordered dimension.** Then features and requirements as sets.
- **...reservations are required.** Then availability becomes time-ranged and this is a booking system.
- **...there is one floor, one vehicle size and a fixed rate.** Then delete both interfaces and put the fifteen lines back inside `ParkingLot`. Two interfaces for a car park with no variation is exactly the over-engineering this course keeps warning about.

The honest concession

The two interfaces are the whole design, and they are also the whole risk. If allocation genuinely never changes, `SpotAllocation` costs every reader a hop for nothing. The reason to build it anyway is that I **can name two implementations that someone would really want** — nearest-to-entrance and spread-across-floors are both real rules in real car parks — and that gate is what separates a design decision from a reflex.

8 In the interview

How it gets asked

- The standard: *“Design a parking lot.”* No further detail, deliberately.
- The narrowed version: *“Design just the part that decides where a car goes.”* The interesting part has been named for you — go straight there and go deep.
- The extension, at minute thirty-five, guaranteed: *“Now add electric-vehicle charging bays”, or “monthly pass holders”, or “a second entrance”.*
- The concurrency probe: *“Two cars arrive at two gates at the same instant.”*
- The scale probe: *“Now it is an airport with fifty thousand spots.”*

The timed script

Minutes 0–5 · Requirements and scope. Four questions whose answers change the design. State the assumptions. State the exclusions: no payment gateway, no plate recognition, no app.

Minutes 5–10 · Estimation, out loud. “A thousand spots, three-hour average stay, sixteen hours open — that is about 5,300 vehicles a day and under one event every five seconds. So a single lock is adequate and I do not need anything clever. And the whole model is about 440 kilobytes, so it fits in memory.”

Minutes 10–15 · The nouns and their responsibilities. Seven classes, one line each. Say which two are interfaces and why.

Minutes 15–25 · The class diagram, drawn while narrating. The containment first, then the two dashed arrows for the delegated decisions.

Minutes 25–35 · The deep dive on allocation. Version one, what breaks it, the interface, two implementations written properly, and the size-fallback rule with the reason it lives in the policy and not in `Floor`.

Minutes 35–40 · Failure modes and what you would do next. Raise the two-gate race yourself with the number — about three collisions a day — and give both fixes. Then pre-empt the extension.

The follow-ups

“Two cars arrive at two gates at the same instant.” “They can both be given the same spot, because finding a free spot and claiming it are two separate steps about a hundred and fifty microseconds apart. At peak that works out to roughly three double-allocations a day, which is a real incident, not a theoretical one. In one process I hold a lock across find-and-claim — not around each separately — and at four events a second that lock

costs nothing. I would also keep the check inside `Spot.assign`, so that even if the lock discipline is broken somewhere, the second claim raises rather than silently overwriting. Across processes it becomes a conditional update on the spot row, and zero rows updated means I lost the race and should retry with the next candidate.”

“Now add electric-vehicle charging bays.” “One class changes. A spot gains a set of features — `{'ev_charger'}` — and a vehicle gains a set of requirements, and the allocation implementation filters on `requirements ≤ spot.features` before it considers size. `ParkingLot.park` does not change at all, because it holds no rules. That is the entire reason allocation is behind an interface. The follow-on question I would raise is whether an ordinary car may occupy a charging bay when nothing else is free — that is a policy decision, and it belongs in the same class.”

“Now it is an airport with fifty thousand spots and fifty arrivals a second.” “Two things change and neither is the class design. First, allocation stops being a scan: I keep a dictionary from (floor, size) to a list of free spots, maintained on assign and release, so finding a spot is a `pop` rather than fifty thousand checks — the scan would be about seven milliseconds at that size, which at fifty a second is a third of a core doing nothing useful. Second, one global lock serialises fifty arrivals a second, so I shard it per floor, since two floors never contend. If I shard the lock, `SpreadAcrossFloors` needs several locks and I would take them in a fixed order to avoid deadlock — or accept a slightly stale free count.”

“How do you handle a lost ticket?” “An index from number plate to the open ticket, and a lost-ticket policy — typically a flat maximum fee, since you cannot prove the entry time. I would keep that fee in `PricingPolicy` rather than in `ParkingLot`, because it is a pricing rule. And I would note that plate lookup is the point where you need plate uniqueness, which is not guaranteed across states, so in practice it is plate plus a verification question.”

“Why is Ticket frozen?” “Because it records something that already happened. If it were mutable, the exit path could adjust `entered_at` and change the fee, and that bug is invisible in review. There is also a rupee argument: at 5,300 vehicles a day, a one-minute error on every ticket is about eighty-eight unbilled hours a day. Payment is a separate frozen record rather than a flag on the ticket, so the audit trail exists by construction.”

“Is this over-engineered?” “Two interfaces and five plain classes, and I applied the same gate to each: can I name a second implementation somebody would actually want? For allocation, nearest-to-entrance and spread-across-floors — both are real. For pricing, flat hourly and weekend surge — both are real. For everything else I could not, so they are plain classes. If you told me the allocation rule will never change, I would delete that interface and put fifteen lines back inside `ParkingLot`.”

A model answer

Asked: *design a parking lot. Multiple floors, multiple vehicle types.*

“Let me suggest the shape first: a few minutes on requirements, a quick estimate, then the classes, then I want to spend the biggest block on the part I think actually decides this design, which is how you choose which spot a vehicle gets. Concurrency at the end.

Four questions. Multiple floors — yes. Three vehicle sizes, and can a car use a bus bay when nothing else is free — yes, but only then. Payment on exit, hourly, different rate per size. And about a thousand spots. I will assume a ticket at entry, settlement at exit, one vehicle per spot, and that we are designing the object model rather than a schema. I am not designing the payment gateway, plate recognition, or the app.

A quick estimate, because it changes what I build. A thousand spots, a three-hour average stay, sixteen hours open — about 5,300 vehicles a day, so roughly one event every five seconds, and maybe four a second at peak. And the whole model — spots, floors, tickets, vehicles — is about four hundred and forty kilobytes. So everything fits in memory and a single lock will be more than adequate. I say that now because it stops me designing something clever that this problem does not need.

The nouns. `Vehicle` is a plate and a size. `Spot` is an id, a floor, a size, a distance from the entrance, and the vehicle in it if any. `Floor` holds spots and reports what is free — bookkeeping only, no rules. `Ticket` links a plate to a spot with an entry time, and it is frozen, because it records something that happened and nothing downstream should be able to edit the entry time and change the fee. `ParkingLot` is the entry point.

Now the part that matters. The simple version is `park` scanning the floors for a free spot of the right size — about fifteen lines, correct, and fine for a long time. It stops being fine the first time someone says ‘spread cars across floors so the lift queue is shorter’, or ‘reserve floor one for electric vehicles’, or ‘monthly pass holders get the near bays’. Each of those adds a branch to the one method every single car goes through.

So I pull that decision out behind an interface — `SpotAllocation`, one method, takes the floors and a vehicle, returns a spot or nothing — and I would write two implementations, because one implementation is just a class. `NearestToEntrance` tries the exact size first and only then larger sizes, and within a size picks the nearest to the gate. `SpreadAcrossFloors` picks the emptiest floor. Pricing gets the same treatment for the same reason: flat hourly and weekend surge are both real rules.

One rule that is easy to put in the wrong place: a car may use a bus bay only when no car bays are free. That is a policy — another car park might forbid it — so it lives in the allocation implementation, not in `Floor`. `Floor` answers a factual question: which spots of exactly this size are free. Keeping bookkeeping and policy apart is what makes the rest of this cheap.

With that, `ParkingLot.park` is about nine lines and contains no rules at all: ask where, claim it, issue a ticket, record it.

Concurrency, before you ask. Finding a spot and claiming it are two steps about a hundred and fifty microseconds apart, and with two gates at four arrivals a second that is roughly three double-allocations a day — a real incident. So the lock goes around find-and-claim together, not around each. And `Spot.assign` raises if the spot is already occupied, so a broken lock discipline anywhere gives me an exception rather than two cars in one bay. Across processes it becomes a conditional update — claim the spot only if it is still free — and zero rows updated means retry.

One thing about the display board, because it is a nice trap: ‘free spots’ and ‘usable spots’ are different numbers. If three motorcycle bays are free and a bus arrives, the bus is turned away while the board says three. So the board reports per floor and per size, not a single total.

And if you added charging bays now: `Spot` gains a set of features, `Vehicle` a set of requirements, and the allocation policy filters on them. One class. That is the only reason the interface is there.”

9 Recall card

- **The interesting part is spot allocation, and everything else is bookkeeping.** Version one is a scan inside `ParkingLot.park`; it breaks on the first policy change (spread across floors · reserve a floor · pass holders). Pull it behind `SpotAllocation` and show **two** implementations — *one implementation is a class, two is a demonstration*. Same for `PricingPolicy`. **Seven classes, two interfaces.**
- **Keep policy out of bookkeeping.** `Floor.free_spots(size)` answers a *factual* question — spots of exactly this size. The **fallback rule** (a car may take a bus bay only if no car bays are free) is *policy* and lives in the allocation implementation. `Size` as an `IntEnum` makes the fallback a comparison.
- **The numbers that shape the design.** 1,000 spots $\approx$ **441 KB** $\rightarrow$ *the whole model fits in memory, so do not design around a database*. 5,300 vehicles/day $\approx$ **0.18 events/s**, $\sim$ 4/s at peak $\rightarrow$ *one lock is adequate; being clever here would be a mistake*. Change it at **$\sim$ 10,000 spots or $\sim$ 50 arrivals/s**: free-spot lists instead of scans, and a lock per floor.

- **Raise the race yourself: find-then-claim is two steps $\sim 151 \mu\text{s}$ apart, ≈ 3 double-allocations a day.** Hold one lock **across both steps**, and keep the guard inside `Spot.assign` as the last line of defence. Across processes: `UPDATE ... WHERE id = ? AND plate IS NULL`, **0 rows updated = you lost, retry.** `Ticket` is **frozen** and `unpark` uses `pop`, so it cannot be settled twice.
- **“Free” and “usable” are different numbers** — a bus is turned away while three motorcycle bays stand empty, so the board reports **per floor and per size**. The guaranteed extension, **EV charging bays**, costs **one class**: spot **features**, vehicle **requirements**, a subset test in the policy — and `ParkingLot` does not change, which is the entire payoff.

Practice — Nodes and links: pointers without pointers

DSA topic: Nodes and links: pointers without pointers **System design topic:** Design a parking lot

Code these, in this order

One rule for the whole set: **draw the pointers before you write the assignment.** Two boxes, an arrow, and what the arrow should point at afterwards. Every bug in this phase is an assignment made before its picture existed.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Delete Node in a Linked List	LeetCode 237 (Medium)	Whether you notice the tail is impossible, and whether you say so before being asked.
2	Remove Linked List Elements	LeetCode 203 (Easy)	The head as a special case — the exact ugliness day 080 removes.
3	Design Linked List	LeetCode 707 (Medium)	Every operation in one class, including the index arithmetic that everybody gets wrong once.
4	Convert Binary Number in a Linked List to Integer	LeetCode 1290 (Easy)	The traversal loop, clean, with an accumulator. Warm-up for the whole phase.

On problem 1, say the precondition out loud

Before writing anything: “this is impossible for the tail, because there is no successor whose value I can steal.” Then write it. An interviewer who has to point that out has learned something different about you than one who hears it first.

On problem 2, count your special cases

Write it without a dummy head and count how many `if` s exist purely because the first element is different. Then write down what those `if` s have in common. You will delete all of them on [day 080](#), and it lands much harder if you have felt them.

On problem 3, test the boundaries first

`addAtIndex(0, x)` on an empty list. `addAtIndex(size, x)` — legal, appends. `addAtIndex(size+1, x)` — illegal, does nothing. `deleteAtIndex` on the head, the tail, and out of range. Write those six tests before the implementation.

On problem 4, keep it to one variable

`total = total * 2 + node.value`. No list, no string, no reversal. If you built an intermediate list, say out loud what you traded — $O(n)$ space for a line of clarity — and whether it was worth it.

The memory-picture drill

Answer without looking:

1. How does a runtime find `numbers[500]` in a Python list? Give the arithmetic.
2. How does it find the 500th node of a linked list? Say why the arithmetic is unavailable.
3. What does a Python variable actually hold?
4. Show two names referring to the same node, then mutate through one and read through the other.
5. Explain cache locality in three sentences, without the word “fast”.
6. Why is `Node` with `__slots__` 48 bytes and without it over 300?

The trade-off drill

For each, say array or linked list, and give the one-sentence reason:

1. Access the 10,000th element.
2. Insert at the front, a million times.
3. Delete a node you were handed by a hash map.
4. Sum every element.
5. Store ten million small integers in as little memory as possible.
6. A queue where both ends are used.
7. The recency order inside an LRU cache.
8. Binary search over sorted data.

Two of the eight are the *only* honest wins for a linked list. Name them.

The qualifier drill

Say the full, honest complexity for each, with the qualifier included:

1. Insert into a linked list.
2. Delete from a linked list.
3. Insert into an array.
4. Delete from an array at a known position.
5. Find the length.

6. Access by index.

For 1 and 2, any answer that does not contain the words “given the node” is wrong.

The break-it drill

Trigger each and record the exact output or error text:

1. Walk the list using the `head` parameter itself, then try to use `head` afterwards.
2. Write `node.next = new_node` before `new_node.next = node.next`. Then traverse. Describe what happens and how long it takes to happen.
3. Write `while node.next.value  $\neq$  target` on a list where the target is absent. Quote the error.
4. Delete the first element with a loop that starts at `head.next`. Print the list.
5. Call `push_front(head, 5)` and discard the return value. Print the list.
6. Set `last.next = head` and call `to_values`. Describe the failure.
7. Call `delete_this_node` on the tail. Quote the error, then say why the operation is impossible in principle rather than merely unimplemented.

The measurement drill

Reproduce all four on your own machine and write the numbers down:

1. Summing a million-element list against a million-node chain. State the ratio.
2. 100,000 `list.insert(0, x)` against 100,000 front pushes onto a chain. State the ratio.
3. `sys.getsizeof` on a `Node` with and without `__slots__`.
4. Bytes per element in a Python list of a million integers.

Then say which of the four is an asymptotic difference and which three are constant factors.

The build-it drill

Write, from an empty file and without looking:

1. `Node`.
2. `from_values` and `to_values`.
3. `length`, `value_at`, `find`.
4. `push_front`, `insert_after`, `delete_after`.
5. `delete_value`, including the head case.

Then run every one of them on the empty list, a one-element list, and a two-element list. Three of the nine functions have a bug on at least one of those. Find them.

The script drill

Run the six moves on the parking lot, out loud, in forty minutes, with a timer. Then check yourself:

1. Did you announce the shape in the first thirty seconds?
2. Were all four clarifying questions ones whose answers change the design?
3. Did you state scope exclusions?
4. Did you do the estimate *before* the class diagram, and did it change any decision?
5. Did you say version one and what breaks it, before showing the interface?
6. Did you write two implementations, or one?
7. Did you raise concurrency yourself, with a number?

The estimation drill

Compute each from scratch, showing the multiplication:

1. Memory for 1,000 spots, 5 floors, 1,000 tickets and 1,000 vehicles.
2. Vehicles per day at a three-hour average stay over a sixteen-hour day.
3. Events per second, average and at a peak of twenty times average.
4. The worst-case allocation scan in microseconds, at 1,000 spots and at 50,000.
5. The collision rate per day from a 151-microsecond window at four arrivals a second.
6. The rupee cost of a one-minute billing error on every ticket, per year.

Then say which single number told you “one lock is enough” and which told you “at fifty thousand spots, stop scanning”.

The policy-versus-bookkeeping drill

For each, say whether it belongs in `Floor`, in `Spot`, in `ParkingLot`, or in a policy class:

1. Which spots of exactly this size are free.
2. Whether a car may use a bus bay.
3. Whether this spot currently holds a vehicle.
4. How much a three-hour stay costs.
5. Whether the first fifteen minutes are free.
6. Which floor a vehicle should be sent to.
7. Whether a spot is already occupied when you try to claim it.
8. What the display board shows.

Two of the eight are the classic mistakes — rules put into bookkeeping classes. Name them.

The extension drill

For each new requirement, say which classes change and how many:

1. Electric-vehicle charging bays.
2. Monthly pass holders who get reserved bays.
3. A second entrance on the other side of the building.
4. Sunday rates that are double.
5. Reservations made a day in advance.
6. Spots that are wide but low, suitable for cars but not vans.
7. A lost ticket.

Two of the seven cannot be absorbed by this design without changing its shape. Name them and say what shape they need instead.

The concurrency drill

1. Write the two-step operation that races, with both steps named.
2. State the window in microseconds and the collisions per day.
3. Write the single-process fix, and say where the lock goes and why not around each step.
4. Write the cross-process fix as SQL.
5. Say what `Spot.assign` raising achieves that the lock does not.
6. Say what changes if you shard the lock per floor, and what new hazard that introduces.

The over-engineering drill

Add `ParkingLotFactory`, `AbstractTicketBuilder`, an observer per event, and a `SpotAllocationStrategyRegistryProvider` to your design. Then:

1. Count classes, files and hops from “a car arrives” to “a spot is chosen”.
 2. Apply the interface gate to every interface you now have and delete the ones that fail.
 3. Write the one sentence you would say if an interviewer asked “is this over-engineered?”
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What is a linked list, and when is it better than an array?* The contiguous-versus-scattered difference and the arithmetic it enables, both consequences, the “given the node” qualifier said explicitly, the four honest wins with the LRU cache first, and the two costs with real numbers — six times the memory, twice the traversal time, and cache locality as the reason.
 2. *Design a parking lot. Multiple floors, multiple vehicle types.* The shape announced, four clarifying questions, the estimate that justifies one lock and no database, seven classes with one-line responsibilities, version one and what breaks it, the allocation interface with two implementations, the fallback rule placed in policy not bookkeeping, and the race raised with a number.
 3. *Delete a node given only that node.* Copy the successor’s value, unlink the successor, say that it is impossible for the tail and why, and note that it breaks any outside reference to the successor.
-

Before you move on

- ☐ I can explain how an array finds element `i`, and why a linked list cannot.
- ☐ I can say what a Python variable holds, and demonstrate two names for one node.
- ☐ I never state linked-list insertion complexity without the words “given the node”.
- ☐ I can name the four honest cases for a linked list, LRU cache first.
- ☐ I reproduced all four measurements myself and know which are constant factors.
- ☐ I triggered the wrong-order relink and watched a node point at itself.
- ☐ I discarded a `push_front` return value and saw nothing happen, with no error.
- ☐ I wrote all nine functions from scratch and tested them on lists of length 0, 1 and 2.
- ☐ I counted the head special cases in `delete_value` and can say what they have in common.
- ☐ I ran the full six-move script on the parking lot against a timer.
- ☐ I did the estimate before the diagram and can say which decisions it changed.
- ☐ I can sort eight rules into bookkeeping and policy without hesitating.
- ☐ I can say which two extensions this design cannot absorb, and what they need instead.
- ☐ I can state the race, the window, the collisions per day, and both fixes.
- ☐ I answered all three questions above out loud.

Traversal, insertion, and deletion

DATA STRUCTURES AND ALGORITHMS

Traversal, insertion, and deletion

You can insert and delete at any position without losing the rest of the list.

SYSTEM DESIGN

Design an elevator system

You can model requests, scheduling and direction without hand-waving the algorithm.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Delete the node at position k . What if it is the head?*
- *Design an elevator system for a ten-storey building with three lifts.*

1 What this is, and why they ask it

Yesterday you met the node and the chain. Today you learn the three operations that everything else in this phase is made of: **walk to a position, insert there, and delete there.**

There is one idea underneath all three, and it is worth stating before anything else: **to change the list at position k , you must be standing at position $k - 1$.** You cannot remove a node while holding it, because the node before it is still pointing at it and you cannot reach backwards. So every operation is really “walk to the node before, then do one or two assignments”.

They ask it because the code is six lines and the boundaries are where people fall over. Position zero is different, because there is no node before the head. Position `size` is different, because there is nothing after it. An empty list is different again. A candidate who writes the loop confidently and then cannot say what happens when `k` is 0 has shown the interviewer exactly what they wanted to see. It is also the setup for tomorrow: every one of those special cases disappears with the dummy-head trick on [day 080](#), and that trick only lands if you have felt the pain first.

2 The story

The shunting yard at the goods station is behind the water tank, and Ganesh has worked it for nineteen years.

His job on a Tuesday morning is to take the train that came in overnight and change it. Two wagons out, one new one in, and the rest goes on at eleven. The wagons are joined by couplings — a heavy hook and a screw at each joint — and one man with a bar can do a coupling in about two minutes.

What Ganesh does, and what the new boys do not do, is that he never walks to the wagon he wants. He walks to the **joint before it.**

If the third wagon has to come out, he does not go and stand at the third wagon. He counts couplings from the engine end — one, two — and stops at the joint between the second and the third. That is the joint he has to open. Then he goes one further along,

opens the joint between the third and the fourth, and then he has a loose wagon that can be pulled aside. Then he brings the second and the fourth together and makes one new joint. Three pieces of work, and all of them happen at the joints, never at the wagon.

Putting one in is the same in reverse. Open one joint, and make two.

He is very particular about the counting, because he has seen what happens when somebody is off by one. Twenty years ago a boy opened the joint on the wrong side and they took out the wrong wagon, and nobody noticed until Belgaum.

The one job that is genuinely different is the first wagon. There is no joint before it — there is the engine. Taking the first wagon out is not a shunting job at all, it is a job for the driver, and it is done differently, by different people, with the engine moving. Ganesh will tell you this without being asked, because every new boy tries to do it his own way once.

And there is a small thing he does at the end that the new boys skip. He walks the whole train to the back and checks the last wagon, because the yard's board has to say how many wagons and which one is last, and if that board is wrong at eleven o'clock the whole thing goes to the wrong platform.

3 The idea in plain English

Ganesh's rule is the rule. **You work at the joint before the wagon**, which in code is: to change the list at position `k`, hold the node at position `k - 1`.

Walking to a position

```
node = head
for _ in range(k):          # take exactly k steps
    node = node.next
```

After `k` steps you are standing on the node at index `k`. Ganesh counting couplings from the engine end. There is no shortcut — this is $O(k)$, and it is the price of the structure from [day 078](#).

Take `k - 1` steps instead and you are standing on the node *before* the one you want, which is where the work happens.

Insertion at position k

You want the new node to end up at index `k`, so the node currently at `k - 1` must point at it, and it must point at whatever used to be at `k`.

```
previous = walk_to(head, k - 1)
previous.next = Node(value, previous.next)    # build, then relink
```

One line of work, once you are standing in the right place. Notice the order inside it: the new node is fully built — already pointing at `previous.next` — *before* `previous.next` is reassigned. Written as two statements the wrong way round, you lose the rest of the list.

Deletion at position k

Open two joints and make one.

```
previous = walk_to(head, k - 1)
previous.next = previous.next.next          # close the gap
```

The removed node still points forward, but nothing points at it, so Python collects it. One assignment.

The three boundaries, and what makes each one different

`k = 0` — **the head**. There is no node at `k - 1`. This is Ganesh's first wagon: not a harder version of the same job, a *different* job, done by whoever holds the reference to the head.

```
if k == 0:
    return head.next          # a NEW head, which the caller must accept
```

And that “which the caller must accept” is the second half of the trap. In Python, rebinding a parameter does not change the caller's variable, so any operation that can change the first node must **return the new head**.

`k = size` — **appending**. Legal for insertion, illegal for deletion. Inserting at `size` means walking to the last node and attaching. It is the one insertion position that needs no gap-closing.

`k > size` — **out of range**. Must raise, and must raise *before* you dereference `None`. The naive loop hits `None.next` and gives you the phase's favourite error.

Size and tail: the board at the end of the yard

Ganesh walking to the back to check the last wagon is bookkeeping that a bare chain of nodes cannot do for itself. Two things are worth storing in a wrapper class:

- `size` — otherwise `len()` is $O(n)$ every time. Maintained on every insert and delete.
- `tail` — a reference to the last node, which makes `append` **$O(1)$** instead of $O(n)$.

Both are pure profit and both are a maintenance burden: **every operation must update them**, and the subtle one is that deleting the last node leaves `tail` pointing at a node no longer in the list. That is the bug in this lesson that produces no error at all — the list looks fine until the next `append` attaches to a node nobody can reach.

Note what a `tail` reference does *not* buy you: deleting the last node is still $O(n)$ in a singly linked list, because you need the node *before* the tail and you cannot walk backwards. That is exactly the argument for a doubly linked list, on [day 085](#).

The pattern that covers deletion by value

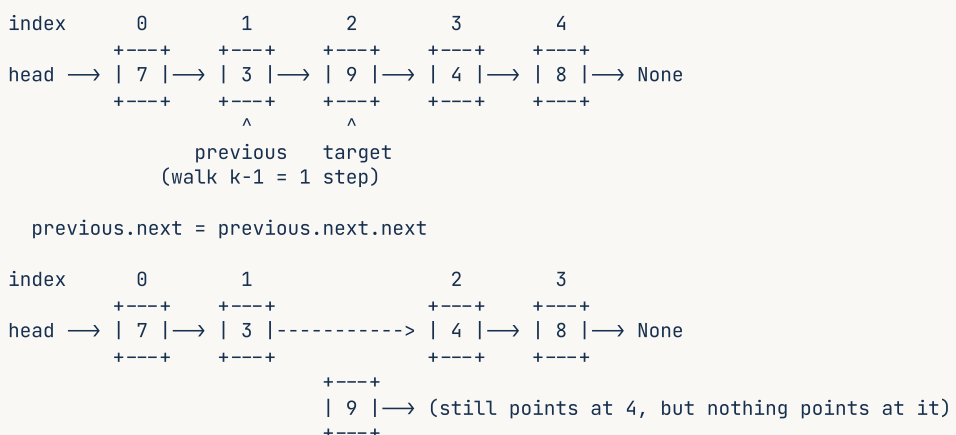
Deleting “the first node whose value is 7” is the same idea. You walk with `previous` and look at `previous.next`, not at `previous`:

```
while previous.next is not None:
    if previous.next.value == target:
        previous.next = previous.next.next
        return
    previous = previous.next
```

Look one ahead, act on the joint. Every delete-by-condition problem in this phase is this loop, plus a special case for the head that tomorrow’s trick removes.

4 The picture

Deleting index 2 from a five-element list. Positions above, values below.



What to notice: the walk is `k - 1 = 1` step, not `k`. Standing on the node you want to delete is useless, because the node before it is still pointing at you.

Now the head case, which is a different shape entirely:

```

head →  $\begin{array}{c} +---+ \\ | 7 | \\ +---+ \end{array} \rightarrow \begin{array}{c} +---+ \\ | 3 | \\ +---+ \end{array} \rightarrow \begin{array}{c} +---+ \\ | 9 | \\ +---+ \end{array} \rightarrow \dots$ 
      ^

```

[illegible]

```

      +---+   +---+
head → | 3 | → | 9 | → ...
      +---+   +---+

```

```

insert 5 at index 2

before:  [3]---->[9]
         prev

RIGHT:   new = Node(5, prev.next)  → [5]---->[9]      (new node built, 9 still reachable)
        prev.next = new          → [3]---->[5]---->[9]

WRONG:   prev.next = new          → [3]---->[5]      (9 is now unreachable)
        new.next = prev.next      → [5]---->[5]      (points at itself: infinite list)

```

```
def _node_at(self, index: int) → Node:
    if index < 0 or index ≥ self._size:
        raise IndexError(f"index {index} out of range for length {self._size}")
    node = self._head
    for _ in range(index):
        node = node.next
    return node
```

```
def insert_at(self, index: int, value: int) → None:
    if index < 0 or index > self._size:          # note: > not ≥, size is legal
        raise IndexError(f"cannot insert at {index} in a list of length {self._size}")
    if index == 0:
        self._head = Node(value, self._head)    # the head case
    if self._size == 0:
        self._tail = self._head                # first ever node is also the tail
```

`index > self._size` rather than `≥`, because inserting *at* `size` is appending and is legal. That one character is the most common off-by-one in this problem.

```
else:
    previous = self._node_at(index - 1)          # stand at the joint before
    previous.next = Node(value, previous.next)
    if previous is self._tail:
        self._tail = previous.next              # appended: tail moved
    self._size += 1
```

Three lines of work and two lines of bookkeeping. The `if previous is self._tail` is what keeps the tail honest, and forgetting it produces a list that is correct until the next `append`.

Step 3 — deletion, and the head case

```
def delete_at(self, index: int) → int:
    if index < 0 or index ≥ self._size:
        raise IndexError(f"index {index} out of range for length {self._size}")
    if index == 0:
        removed = self._head
        self._head = removed.next                # a NEW head
        if self._size == 1:
            self._tail = None                    # the list is now empty
```

The `size == 1` branch is the one that bites. Deleting the only node must clear the tail as well, or the list reports itself empty while `tail` still refers to the departed node.

```
else:
    previous = self._node_at(index - 1)
    removed = previous.next
    previous.next = removed.next
    if removed is self._tail:
        self._tail = previous                  # deleted the last: tail moves back
    removed.next = None                        # do not leave a stale link
    self._size -= 1
    return removed.value
```

`removed.next = None` is not required for correctness, but it stops a caller who kept a reference to the removed node from walking back into a list it is no longer part of. That is a real source of confusing bugs.

Step 4 — append in $O(1)$, which is the whole reason for the tail

```
def append(self, value: int) → None:
    node = Node(value)
    if self._tail is None:
        self._head = self._tail = node        # empty list
    else:
        self._tail.next = node
        self._tail = node
    self._size += 1
```

Without the tail reference this would walk the whole list every time, making `n` appends $O(n^2)$. With it, each append is two assignments. Say the $O(n^2)$ out loud — building a list by repeatedly appending without a tail is a classic accidental quadratic.

Step 5 — delete by value, the look-one-ahead loop

```
def remove_value(self, value: int) → bool:
    if self._head is None:
        return False
    if self._head.value == value:
        self.delete_at(0)                # reuse, do not duplicate
        return True

    previous = self._head
    while previous.next is not None:
        if previous.next.value == value:
            ...
```

Calling `delete_at(0)` rather than re-writing the head logic is worth doing and worth saying: the head case is fiddly enough that it should exist in exactly one place.

The complete solution

```
class Node:
    __slots__ = ("value", "next")

    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value
        self.next = next

    def __repr__(self) → str:
        return f"Node({self.value})"

class LinkedList:
    """A singly linked list with O(1) length and O(1) append.

    The rule underneath every operation: to change the list at position k, you
    must be standing at position k-1, because you cannot reach backwards.

    Two pieces of bookkeeping, and both must be updated by EVERY mutation:
    _size - otherwise len() is O(n)
    _tail - otherwise append() is O(n) and building a list is O(n^2)

    The bug with no error message: deleting the last node without moving _tail
    back leaves it pointing at a node that is no longer in the list.
    """

    def __init__(self, values: list[int] | None = None) → None:
        self._head: Node | None = None
        self._tail: Node | None = None
        self._size = 0
        for value in values or []:
            self.append(value)

    # ---- reading -----

    def __len__(self) → int:
        return self._size
        # O(1), because we store it

    def __iter__(self):
        node = self._head
        while node is not None:
            yield node.value
            node = node.next

    def to_list(self) → list[int]:
        return list(self)

    def __repr__(self) → str:
        return f"LinkedList({self.to_list()})"

    def value_at(self, index: int) → int:
        return self._node_at(index).value

    def index_of(self, value: int) → int:
        for position, item in enumerate(self):
            if item == value:
                return position
        return -1

    # ---- writing -----

    def append(self, value: int) → None:
        """O(1), thanks to the tail reference."""
        node = Node(value)
        if self._tail is None:
            self._head = self._tail = node
        else:
```

```

        self._tail.next = node
        self._tail = node
        self._size += 1

def prepend(self, value: int) → None:
    """O(1) always – the one thing a linked list is unambiguously good at."""
    self._head = Node(value, self._head)
    if self._tail is None:
        self._tail = self._head
    self._size += 1

def insert_at(self, index: int, value: int) → None:
    """O(index). Inserting AT size is legal and means append."""
    if index < 0 or index > self._size:
        # > not ≥
        raise IndexError(f"cannot insert at {index} in a list of length {self._size}")
    if index == 0:
        self.prepend(value)
        return
    if index == self._size:
        self.append(value)
        return
    previous = self._node_at(index - 1)
    previous.next = Node(value, previous.next)
    self._size += 1
    # stand at the joint before
    # build, THEN relink

def delete_at(self, index: int) → int:
    """O(index). Returns the removed value."""
    if index < 0 or index ≥ self._size:
        raise IndexError(f"index {index} out of range for length {self._size}")

    if index == 0:
        removed = self._head
        self._head = removed.next
        if removed is self._tail:
            self._tail = None
        # the list had exactly one node
    else:
        previous = self._node_at(index - 1)
        removed = previous.next
        previous.next = removed.next
        if removed is self._tail:
            self._tail = previous
        # deleted the last node

    removed.next = None
    self._size -= 1
    return removed.value
    # no stale link out of a dead node

def remove_value(self, value: int) → bool:
    """Remove the first node with this value. Look ONE AHEAD and act on the joint."""
    if self._head is None:
        return False
    if self._head.value == value:
        self.delete_at(0)
        return True
    # one home for the head case

    previous = self._head
    while previous.next is not None:
        if previous.next.value == value:
            if previous.next is self._tail:
                self._tail = previous
            previous.next = previous.next.next
            self._size -= 1
            return True
        previous = previous.next
    return False

# ---- internals -----

def _node_at(self, index: int) → Node:
    if index < 0 or index ≥ self._size:

```

```

        raise IndexError(f"index {index} out of range for length {self._size}")
    node = self._head
    for _ in range(index):
        node = node.next
    return node

def _check(self) → None:
    """Assert the invariants. Run this in tests; it catches the stale tail."""
    count, node, last = 0, self._head, None
    while node is not None:
        count, last, node = count + 1, node, node.next
    assert count == self._size, f"size says {self._size}, chain has {count}"
    assert last is self._tail, "tail does not point at the last node"
    assert self._tail is None or self._tail.next is None, "tail is not the end"

if __name__ == "__main__":
    lst = LinkedList([7, 3, 9, 4, 8])
    print(lst)                                # LinkedList([7, 3, 9, 4, 8])
    print(len(lst), lst.value_at(2))          # 5 9

    print(lst.delete_at(2), lst)              # 9 LinkedList([7, 3, 4, 8])
    print(lst.delete_at(0), lst)              # 7 LinkedList([3, 4, 8])
    print(lst.delete_at(2), lst)              # 8 LinkedList([3, 4])
    lst._check()                              # the tail moved back correctly

    lst.insert_at(0, 1)
    lst.insert_at(3, 99)                      # index == size, so this appends
    lst.insert_at(2, 50)
    print(lst, len(lst))                      # LinkedList([1, 3, 50, 4, 99]) 5
    lst._check()

    lst.append(6)
    print(lst, lst.index_of(99))              # LinkedList([1, 3, 50, 4, 99, 6]) 4

    print(lst.remove_value(50), lst)           # True LinkedList([1, 3, 4, 99, 6])
    print(lst.remove_value(404), lst)          # False LinkedList([1, 3, 4, 99, 6])

    single = LinkedList([42])
    print(single.delete_at(0), single, len(single)) # 42 LinkedList([]) 0
    single._check()
    single.append(1)                          # would break if the tail were stale
    print(single)                             # LinkedList([1])

    empty = LinkedList()
    try:
        empty.delete_at(0)
    except IndexError as error:
        print(f"IndexError: {error}")          # index 0 out of range for length 0
    try:
        empty.insert_at(1, 5)
    except IndexError as error:
        print(f"IndexError: {error}")          # cannot insert at 1 in a list of length 0

```

`_check` is worth writing in an interview if there is a minute spare. It catches the stale tail, the wrong size and a broken chain in three assertions, and running it after every operation in a test loop finds bugs that eyeballing never will.

6 What it costs

The operations

walk to index k	$O(k)$	
<code>value_at(k)</code>	$O(k)$	
<code>prepend</code>	$O(1)$	
<code>append</code> (with a tail reference)	$O(1)$	
<code>append</code> (without one)	$O(n)$	
<code>insert_at(k)</code>	$O(k)$	
<code>delete_at(k)</code>	$O(k)$	
<code>delete_at(0)</code>	$O(1)$	
<code>delete</code> the LAST node	$O(n)$	even with a tail reference
<code>remove_value</code>	$O(n)$	
<code>len</code>	$O(1)$	because we store it

Two of those need saying out loud.

`append` without a tail is $O(n)$, so building a list of n elements by appending is $O(n^2)$. At $n = 100,000$ that is five billion steps. With a tail it is 100,000 assignments. This is the single most common accidental quadratic in linked-list code.

Deleting the last node is $O(n)$ even with a tail reference, because you need the node *before* the tail and a singly linked list cannot walk backwards. Say this when the tail comes up — it is the cleanest possible motivation for the doubly linked list.

Against a Python list

	LinkedList	Python list
<code>insert at 0</code>	$O(1)$	$O(n)$ shift everything
<code>delete at 0</code>	$O(1)$	$O(n)$ shift everything
<code>insert at the end</code>	$O(1)$ with a tail	$O(1)$ amortised
<code>delete at the end</code>	$O(n)$	$O(1)$
<code>insert at the middle</code>	$O(k)$ walk	$O(n)$ shift
<code>access index k</code>	$O(k)$	$O(1)$

The honest summary: **the linked list wins at the front and loses everywhere else**. Even in the middle, where the folklore says linked lists win, both are linear — one pays for the walk, the other for the shift — and the array's shift is a fast contiguous memory move while the walk is a chain of scattered dereferences.

The measurement

Building a 100,000-element list, from [day 078](#)'s numbers:

<code>100,000 × list.insert(0, x)</code>	1.443 s	
<code>100,000 × prepend on nodes</code>	0.0246 s	about 59x

And the quadratic append, if you forget the tail:

```
n appends without a tail: 1 + 2 + ... + n = n(n+1)/2 steps
n = 100,000:                about 5 x 10^9 steps - tens of seconds
n appends with a tail:      100,000 x 2 assignments
```

Space

```
the list:      0(n) nodes, 48 bytes each with __slots__
bookkeeping:   0(1) - one size integer and one tail reference
every operation: 0(1) extra - one `previous` variable
```

Every algorithm in this phase should be $O(1)$ extra space. If yours copies to a Python list first, say so and say what you traded.

7 The traps

Trap 1 — walking `k` steps instead of `k - 1`

```
previous = self._node_at(index)          # should be index - 1
previous.next = previous.next.next
```

Deletes the wrong node — the one *after* the target — and does it silently. `delete_at(2)` on `[7, 3, 9, 4, 8]` removes 4 instead of 9. Ganesh's boy opening the joint on the wrong side.

The fix is to say the rule out loud before writing the line: **stand at `k - 1`, act on `k`**.

Trap 2 — the head, unhandled

```
previous = self._node_at(index - 1)      # index 0 → _node_at(-1)
```

```
IndexError: index -1 out of range for length 5
```

if you guarded, and this if you did not:

```
node = self._head
for _ in range(-1):                # loop body never runs
    node = node.next
node.next = node.next.next        # deletes index 1, not index 0
```

Silently wrong, which is worse. **Position zero is a different operation**, not a harder version of the same one.

Trap 3 — not returning the new head

```
def delete_first(head):  
    head = head.next          # rebinds the local name only  
    return None
```

```
delete_first(head)  
print(to_values(head))      # unchanged, no error
```

Python passes the reference by value, so rebinding a parameter changes nothing for the caller. **Any function that can change which node is first must return the new head**, and the caller must assign it. Inside a class this disappears, because `self._head` is shared state — which is one good reason to wrap the chain in a class.

Trap 4 — the stale tail

```
previous.next = removed.next  
self._size -= 1          # forgot: if removed is self._tail
```

No error. The list is correct, `len` is correct, iteration is correct. And then:

```
lst.append(6)              # attaches to the REMOVED node  
print(lst.to_list())      # the 6 is nowhere to be seen
```

The append succeeds and the value vanishes. This is the hardest bug in the lesson to find by reading, and it is why `_check` exists.

The same trap in its other form: deleting the only node must set `_tail = None`, or an empty list holds a reference to a departed node and the next `append` attaches to nothing reachable.

Trap 5 — `>` versus `>=` on the insert bound

```
if index < 0 or index ≥ self._size:    # WRONG for insertion  
    raise IndexError(...)
```

Now `insert_at(size, value)` — appending — raises. Inserting *at* `size` is legal; deleting *at* `size` is not. One character, two different bounds, and the reason is that insertion has `size + 1` valid positions and deletion has `size`.

Trap 6 — `previous.next.next` with no guard

```
while previous.next.value ≠ target:    # two dereferences, one check
```

```
AttributeError: 'NoneType' object has no attribute 'value'
```

Fires as soon as the target is absent. The loop condition must be `while previous.next is not None and previous.next.value != target`, and the `is not None` must come first because `and` short-circuits.

Trap 7 — forgetting to update `_size`

Silent, and it corrupts everything downstream: `_node_at` uses `_size` for its bounds check, so a size that is one too large lets a walk run off the end into `None`, and a size one too small makes the last element unreachable. **Every mutation updates size**, and the assertion in `_check` catches it in one line.

Trap 8 — building a list with `append` and no tail

```
for value in range(100_000):  
    lst.append(value)                                # each append walks the whole list
```

No error, correct output, and about five billion steps. The tail reference turns $O(n^2)$ into $O(n)$ and costs one attribute.

8 In the interview

How it gets asked

- The direct version: “Delete the node at position k .” Then, immediately: “What if k is 0?” — the follow-up is the question.
- The class version: “Design a linked list with `get`, `addAtHead`, `addAtTail`, `addAtIndex` and `deleteAtIndex`.” LeetCode 707, and it is entirely a boundary-conditions exercise.
- The by-value version: “Remove all elements equal to `val`.” LeetCode 203, which is the strongest possible motivation for tomorrow’s dummy head.
- The trap version: “What is the complexity of appending n elements?” — they want to hear “ $O(n)$ with a tail reference, $O(n^2)$ without”.

What to say out loud, in the first ninety seconds

1. **State the rule that generates all three operations.** “To change the list at position k I have to be standing at $k - 1$, because I cannot reach backwards from a node to the one before it.”
2. **Name the boundaries before writing.** “So there are three cases: k equals zero, where there is no previous node and the head itself changes; k in the middle, which is the general case; and k out of range, which must raise before I dereference anything.”

3. **Say the order-of-assignment rule for insertion.** “I build the new node pointing at the rest of the list first, then attach it. The other order loses the tail.”
4. **Say what you will store, and why.** “I would keep a size and a tail reference. Size makes `len` $O(1)$, and the tail makes `append` $O(1)$ — without it, building a list of n elements by appending is $O(n^2)$.”
5. **Name the maintenance cost yourself.** “The price is that every mutation has to maintain both, and the bug that has no error message is deleting the last node without moving the tail back.”

The follow-ups

“**What if k is 0?**” “Then there is no node at $k - 1$ to work from, so it is a different operation rather than a harder one: the head reference itself changes. In a class, `self._head = self._head.next` and I also have to clear the tail if that was the only node. In a bare function, the important part is that I must *return* the new head, because rebinding the parameter does not change the caller’s variable — that is a bug that produces no error at all. Tomorrow’s dummy-head trick removes the case entirely.”

“**What is the complexity of appending n elements?**” “ $O(n)$ if I keep a tail reference, because each append is two assignments. $O(n^2)$ without one, because each append walks the whole list — that is $n(n+1)/2$ steps, about five billion at a hundred thousand elements. It is the classic accidental quadratic in this structure.”

“**You have a tail reference. Can you delete the last node in $O(1)$?**” “No, and that is the cleanest argument for a doubly linked list. To unlink the last node I need the node *before* it so I can set its `next` to `None`, and a singly linked list cannot walk backwards, so I have to traverse from the head. A tail reference makes append $O(1)$ and does nothing for delete-last.”

“**How do you know your list is not corrupt?**” “I would write a small invariant check: walk the chain counting nodes, assert the count equals the stored size, assert the last node reached is the tail, and assert the tail’s `next` is `None`. Three assertions, and they catch the stale tail, the wrong size and a broken chain. In a test I would run it after every operation on a random sequence.”

“**Delete every node with a given value.**” “The same look-one-ahead loop: I hold `previous` and test `previous.next.value`, so that when it matches I can unlink it. Two subtleties. After a deletion I must *not* advance `previous`, or two equal values in a row leave one behind. And the head needs its own loop first, because several leading nodes may all match. Both of those disappear with a dummy node before the head, which is what I would actually write.”

“Why wrap the nodes in a class at all?” “Three reasons. It gives `size` and `tail` a home, so `len` and `append` are $O(1)$. It puts the head case in exactly one place instead of in every function. And it removes the return-the-new-head problem, because `self._head` is shared state rather than a parameter that gets rebound.”

A model answer

Asked: *delete the node at position k. What if it is the head?*

“The rule that generates this whole family: to change the list at position k , I have to be standing at position $k-1$. A node cannot reach the node before it, so I cannot unlink a node I am holding — I have to be holding its predecessor.

So the general case is: walk $k-1$ steps from the head, then `previous.next = previous.next.next`. That is one assignment. The walk is $O(k)$ and the unlink is $O(1)$, so the operation is $O(k)$ overall. The removed node still points forward, but nothing points at it, so it is collected.

Now, k equals zero. There is no node at position minus one, so this is not a harder version of the same operation — it is a different operation. What changes is the head reference itself, not a link inside the list. In a class that is `self._head = self._head.next`, and I also have to clear the tail if that was the only node, or I am left with a reference to a node that is no longer in the list.

If it is a bare function rather than a class, there is a second half to that answer and it is the part people miss: Python passes the reference by value, so rebinding the `head` parameter inside the function does not change the caller’s variable. The function has to *return* the new head and the caller has to assign it. That bug produces no error — the list just still has its first element.

The other boundaries: k greater than or equal to the size must raise, and it must raise *before* I dereference anything, or I get `AttributeError: 'NoneType' object has no attribute 'next'` instead of a useful message. And note that insertion and deletion have different bounds — inserting *at* size is legal and means append, so insertion allows size + 1 positions and deletion allows size.

If I am writing this as a class rather than a one-off function, I would store two extra things. A size, so `len` is $O(1)$ rather than a walk. And a tail reference, so `append` is $O(1)$ — without one, building a list of n elements by appending is $O(n^2)$, about five billion steps at a hundred thousand elements. The cost is that every mutation has to maintain both, and the nastiest bug in this class is deleting the last node and forgetting to move the tail back: the list looks completely correct until the next `append` attaches to a node nobody can reach, and the value silently vanishes. So I would write a three-line invariant check — count the chain, compare to size, confirm the tail is the last node — and run it after every operation in tests.

One thing a tail reference does *not* buy me: deleting the last node is still $O(n)$, because I need the node before the tail and I cannot walk backwards. That is the argument for a doubly linked list.”

9 Recall card

- One rule generates all three operations: to change the list at position `k`, stand at `k - 1`. You cannot reach backwards, so you can never unlink a node you are holding. Deletion is `previous.next = previous.next.next`; insertion is `previous.next = Node(value, previous.next)` — **build first, then relink**, or the node points at itself and the rest of the list is gone.
- `k = 0` is a different operation, not a harder one — the head *reference* changes, not a link. In a bare function you must **return the new head**, because rebinding a parameter changes nothing for the caller and raises no error. Tomorrow's **dummy head** deletes this case entirely.
- Insertion and deletion have different bounds. Inserting at `size` is legal (it appends), so insertion has `size + 1` valid positions and deletion has `size: index > size` versus `index ≥ size`. Check bounds **before** any dereference.
- Store `size` and `tail`, and maintain them in every mutation. `size` makes `len` $O(1)$; `tail` makes `append` $O(1)$ — **without it, n appends is $O(n^2)$** , about 5×10^9 steps at $n = 100,000$. The bug with **no error message** is deleting the last node without moving the tail back: the next `append` attaches to an unreachable node and the value vanishes. A three-assertion `_check` catches it.
- A tail reference does not make delete-last $O(1)$ — you need the node *before* the tail and a singly linked list cannot walk backwards. That is the argument for the **doubly linked list**. And the honest summary against a Python list: **the linked list wins at the front and loses everywhere else** (100,000 front inserts: **1.443 s vs 0.0246 s**, $\sim 59\times$).

1 What this is, and why they ask it

You are given a building, some number of floors, and some number of lift cars. People press buttons in two different places — outside the lift, where they say which *direction* they want, and inside it, where they say which *floor* they want — and your design has to decide which car goes where, and in what order.

The bookkeeping is easy. `Elevator`, `Floor`, `Button`, `Request` fall out in three minutes. The question is entirely the **scheduling**, and it has two halves that candidates routinely merge into one: *which car answers this hall call*, and, separately, *in what order does one car serve the stops it has accepted*. Getting those apart is most of the interview.

They ask it because there is a real algorithm underneath, it has a name, and the obvious answers are demonstrably bad. Serving requests in the order they arrive makes a car cross the building repeatedly. Always going to the nearest request starves the top floor. The correct answer — keep going in one direction while anything ahead of you wants that direction, then turn — is called the **elevator algorithm**, and disk schedulers use the same one under the name **SCAN**. Expect this prompt at Amazon, Uber, Flipkart and anywhere that runs a low-level design round.

2 The story

Shanta does the week's shopping on Thursday morning, in the lane that runs from the bus stop at one end to the temple at the other. It is about three hundred metres, and there are shops down both sides.

She has eight things to get. Her husband, when he does it, buys them in the order they are listed on his phone — flour, then coriander, then the thing from the shop near the temple, then something back near the bus stop again. He walks that lane four times and comes home after an hour and a half complaining about his knees.

Shanta walks up one side and down the other.

Going up, she buys everything that is on that side, in the order the shops come. She does not care what order the list is in. If the flour is nearer the temple than the dal, she gets the dal first, because she is passing it anyway. At the temple end she crosses over, and comes back down the other side, picking up everything on that side. One pass each way, about twenty-five minutes, and she is at the bus stop again with everything.

The part she is strict about is what happens when her daughter rings.

Her daughter rings most Thursdays and adds one thing. If it is at a shop further up, that is easy — Shanta is going that way anyway, and it costs her nothing. If it is at a shop she has already passed on this side, she does **not** turn round. She notes it, and she gets it on the way down, or next week. Turning round for it would mean walking the stretch three times instead of once, and the whole point is that she walks each stretch once.

She learned that from the year her daughter used to ring four or five times in a morning. Shanta kept turning back for each one, and one Thursday she got to half past twelve having covered the same hundred metres six times and still not reached the temple end at all.

Now she is quite firm about it. She keeps going until there is nothing left ahead of her on this side. Then, and only then, she turns.

3 The idea in plain English

Shanta's lane is the lift shaft. Her two passes are the two directions. And her rule — *keep going while there is anything ahead of you that wants this direction, then turn* — is the **elevator algorithm**, also called **LOOK**, and it is the answer to this whole question.

Two kinds of request, and why the difference matters

This is the distinction that separates a good answer from a vague one.

- A **hall call** is pressed *outside* the lift, on a floor. It carries a floor **and a direction** — “I am on 4 and I want to go up”. It carries no destination, because the person has not said where they are going yet. Any car may serve it, so **the system** must choose one.
- A **car request** is pressed *inside* the lift. It carries a floor and no direction, and it belongs to **that car only**. Nobody else can serve it.

So there are two scheduling decisions, not one:

1. **Dispatch** — which car answers a hall call. A system-level decision.
2. **Stop ordering** — given the stops a car has accepted, in what order does it visit them.
A per-car decision.

Merging these is the most common mistake in this question. Say them separately, in that order.

The three stop-ordering algorithms, and why two of them are wrong

Take a car on floor 1 with stops pending at floors 2, 9, 3, 8 and 4, in that order of arrival.

First come, first served. Serve them in arrival order: 1→2→9→3→8→4.

```
travel: 1 + 7 + 6 + 5 + 4 = 23 floors
```

Correct, fair, and terrible. The car crosses the building four times. Nobody builds this.

Shortest seek first. Always go to the nearest pending stop: 1→2→3→4→8→9.

```
travel: 1 + 1 + 1 + 4 + 1 = 8 floors
```

Efficient — and it **starves**. If people keep pressing buttons near the car, the request on floor 9 is never the nearest one and waits for ever. In a building this shows up as “the tenth floor complains about the lifts”, and it is a genuine, historical complaint.

LOOK, the elevator algorithm. Keep going in the current direction while anything ahead of you wants a stop; when nothing is ahead, reverse.

```
going up from 1: 2, 3, 4, 8, 9 = 8 floors
then reverse and serve anything below
```

Same travel as nearest-first on this input, and **no starvation**, because a request above you is guaranteed to be served before the car turns around. Shanta noting the coriander and getting it on the way down.

The name for the strict version that runs to the very top before turning is **SCAN**. The version that turns as soon as nothing is ahead — which is what real lifts do — is **LOOK**. Knowing both names, and that disk head schedulers use the same algorithm, is worth ten seconds.

The state a single car holds

```
current_floor    : int
direction        : UP | DOWN | IDLE
door             : OPEN | CLOSED
up_stops         : sorted set of floors to serve while going up
down_stops       : sorted set of floors to serve while going down
```

Two sets rather than one is the trick that makes LOOK simple. A stop at floor 7 means two different things depending on whether the person there wants to go up or down, and keeping them apart means the “what is next” question is a `min` or a `max`:

```
if direction is UP:
    above = [f for f in up_stops if f > current_floor]
    next_stop = min(above) if above else ...      # nothing above: turn
```

Dispatch: which car answers a hall call

The simple, defensible rule, and the one to give first:

Prefer a car that is already moving toward the call and in the same direction. Otherwise prefer an idle car. Break ties by distance.

A car on floor 2 going up is a *better* answer to “floor 5, going up” than an idle car on floor 6, even though the idle car is closer — because the moving car passes floor 5 anyway and the idle car would have to come down and then go back up.

Scoring it makes this concrete:

same direction and ahead of the call:	$ car_floor - call_floor $	
idle:	$ car_floor - call_floor + 5$	penalty
wrong direction or behind:	$ car_floor - call_floor + 20$	penalty

The penalties are made up, and you should say so: they are a tuning knob, and the honest sentence is “I would tune those against measured waiting times, not guess them once.”

The interesting part, and where the interface goes

Dispatch is the decision that will change. Real buildings change it constantly — up-peak mode in the morning, a zoned scheme where cars 1 and 2 serve floors 1–5 and car 3 serves 6–10, destination dispatch in modern towers where you enter your floor in the lobby. So:

```
class Dispatcher(Protocol):
    def choose(self, cars: list[Elevator], call: HallCall) → Elevator: ...
```

And, as always, **two implementations**: `NearestSuitable` as above, and `ZonedDispatcher` where each car owns a band of floors. One implementation is a class; two is a demonstration that the design survives a changed requirement.

Stop *ordering*, by contrast, gets no interface. LOOK is not going to be replaced — every lift in the world uses it — and an interface there would be a hierarchy built for a future that will not arrive. Saying why one decision gets an interface and the other does not is exactly the judgement being tested.

4 The picture

One car, running LOOK. Floors on the left, the car’s stop sets on the right.

```

floor
10 |
9  | ← stop (someone on 9 wants DOWN)      up_stops   = {4, 9}
8  |                                         down_stops = {9, 2}
7  |
6  |
5  |
4  | ← stop (someone on 4 wants UP)
3  |
2  | ← stop (someone on 2 wants DOWN)
1  | [CAR] direction = UP
    +-----

going UP from 1:
  up_stops above me: {4}           → go to 4, open, close
  up_stops above me: {9}           → go to 9, open, close
  nothing above wants UP           → TURN
going DOWN from 9:
  down_stops below me: {2}         → go to 2, open, close
  nothing below                     → IDLE

travel: 1→4→9→2 = 3 + 5 + 7 = 15 floors, every stop served, nobody starved

```

What to notice: floor 9 appears in **both** sets, because somebody there pressed “down”. The car serves it as an up-stop when it arrives going up (people who wanted to go up get in) and again — or instead — as a down-stop. Keeping the two sets separate is what makes that expressible at all.

The car as a state machine:

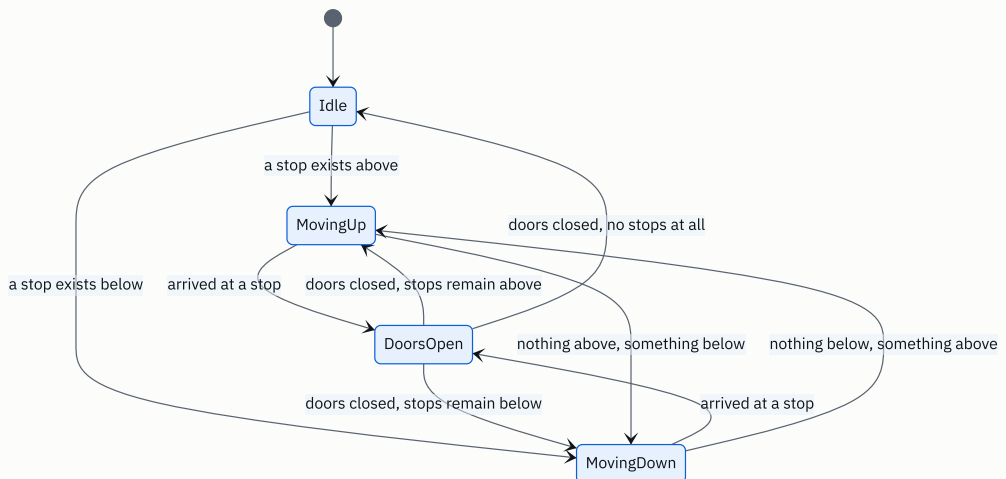


FIGURE 79.1

What to notice: **there is no transition from MovingUp straight to MovingUp at a different floor** — every arrival goes through `DoorsOpen`, and every direction change goes through a state where the doors are shut. That is not decoration; it is the safety property, and drawing it that way is how you show you thought about it.

The classes:

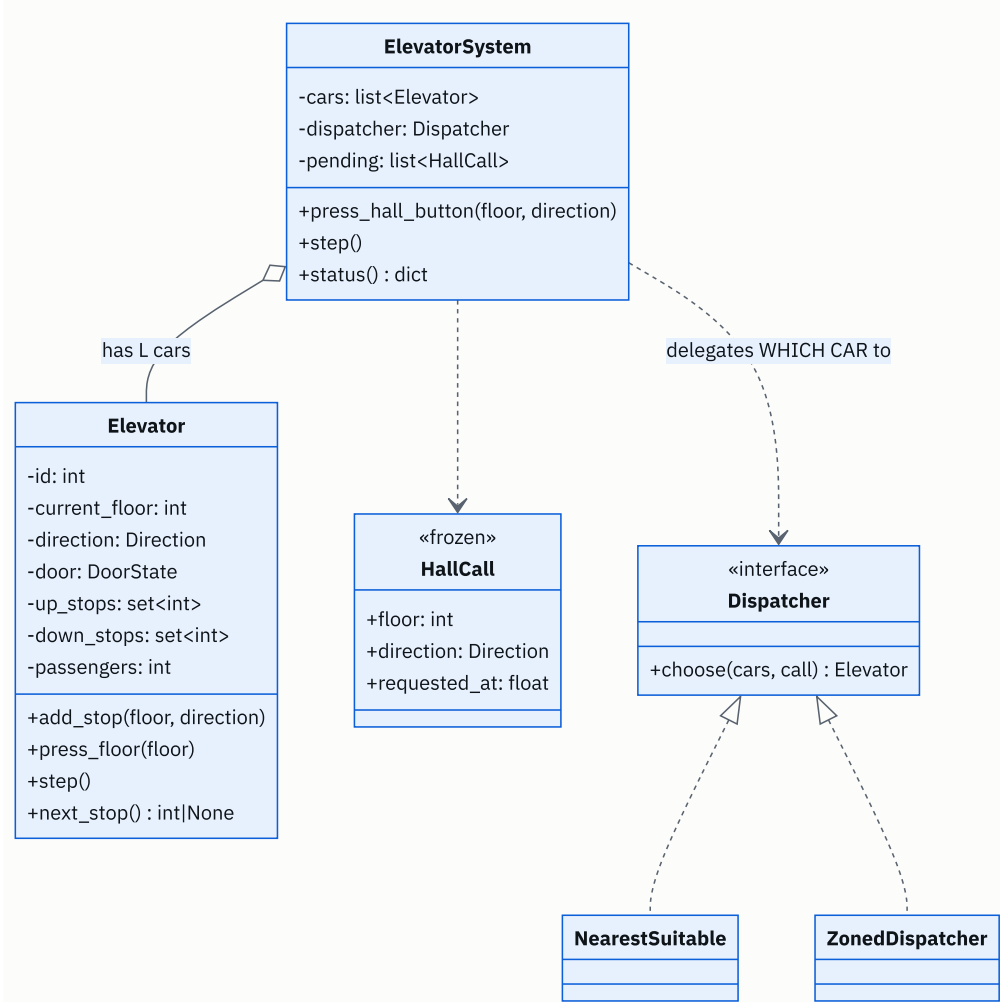


FIGURE 79.2

5 How it actually works

Move 1 · Clarify (minutes 0–5)

“How many floors and how many cars?” — Ten floors, three cars. **“Do the outside buttons have separate up and down, or one call button?”** — Separate. This is the question that most changes the design, because a single button loses the direction and forces the car to guess. **“Is there a capacity limit, and should a full car skip a hall call?”** — Thirteen people, and yes. **“Am I designing the control logic, or also the motor and door hardware?”** — Control logic; assume `move_one_floor()` and `open_doors()` exist.

“I will assume one car per shaft, no express floors, and that people board in the direction the car is going. I am not designing fire-service mode or the maintenance interface, though I will say where they would attach.”

Move 2 · The nouns (minutes 5–10)

- `Direction` — `UP`, `DOWN`, `IDLE`. An enum, because a boolean cannot express idle.
- `HallCall` — a floor plus a direction plus the time it was pressed. Frozen; it is a record of an event.
- `Elevator` — one car: where it is, which way it is going, the doors, and its two stop sets.
- `ElevatorSystem` — holds the cars, receives hall calls, and delegates the choice.
- `Dispatcher` (*interface*) — chooses a car for a hall call.

Five, one of them an interface. Note what is **not** here: no `Button` class, because a button is an input event and modelling it as an object buys nothing; no `Person`, because passengers only exist as a count for the capacity rule; no `Floor`, because a floor is an integer and giving it a class is ceremony. Saying why you left things out is as valuable as what you put in.

Move 3 · The car, which is where the algorithm lives

```
class Direction(Enum):
    UP = 1
    DOWN = -1
    IDLE = 0

class Elevator:
    def __init__(self, id: int, floor: int = 1, capacity: int = 13) → None:
        self.id = id
        self.current_floor = floor
        self.direction = Direction.IDLE
        self.doors_open = False
        self.passengers = 0
        self.capacity = capacity
        self.up_stops: set[int] = set()      # serve these while going UP
        self.down_stops: set[int] = set()    # serve these while going DOWN
```

Two sets, and the reason is worth saying while you type it: a stop at floor 7 means something different depending on which way the person there wants to travel, and one set cannot hold that.

```
def add_stop(self, floor: int, direction: Direction) → None:
    """A hall call assigned to this car."""
    if direction is Direction.UP:
        self.up_stops.add(floor)
    else:
        self.down_stops.add(floor)

def press_floor(self, floor: int) → None:
    """A car request from inside. Which set it goes in depends on where we are."""
    if floor > self.current_floor:
        self.up_stops.add(floor)
    elif floor < self.current_floor:
        self.down_stops.add(floor)
```

The asymmetry there is real and worth pointing at: a hall call *tells* you its direction, and a car request does not, so you infer it from where the car is now.

```
def next_stop(self) → int | None:
    """LOOK: keep going while anything ahead wants this direction, then turn."""
    if self.direction is Direction.UP:
        above = [f for f in self.up_stops if f > self.current_floor]
        if above:
            return min(above)
        below = self.down_stops | {f for f in self.up_stops if f < self.current_floor}
        return max(below) if below else None      # turn: highest thing below
    ...
```

Four lines that are the entire algorithm. Read them out loud as Shanta: *is there anything ahead of me on this side? Then go to the nearest one. Nothing ahead? Then turn, and go to the furthest thing on the other side.*

Move 4 • Dispatch, the interface

```
class Dispatcher(Protocol):
    def choose(self, cars: list[Elevator], call: HallCall) → Elevator | None: ...

class NearestSuitable:
    """Prefer a car already coming this way; then an idle car; then anything.

    The penalties are a tuning knob, not a law – I would fit them to measured
    waiting times rather than guess once.
    """

    SAME_DIRECTION = 0
    IDLE_PENALTY = 5
    WRONG_WAY_PENALTY = 20

    def choose(self, cars, call):
        available = [c for c in cars if c.passengers < c.capacity]
        if not available:
            return None # every car is full: queue it
        return min(available, key=lambda car: self._score(car, call))

    def _score(self, car: Elevator, call: HallCall) → int:
        distance = abs(car.current_floor - call.floor)
        if car.direction is Direction.IDLE:
            return distance + self.IDLE_PENALTY
        approaching = (
            car.direction is call.direction
            and (call.floor - car.current_floor) * car.direction.value > 0
        )
        return distance if approaching else distance + self.WRONG_WAY_PENALTY
```

The `approaching` expression is the one to explain rather than let them read: *the car is going the same way the caller wants, and the caller is ahead of it in that direction*. Multiplying by `direction.value` — which is +1 or −1 — makes “ahead” one comparison for both directions.

```
class ZonedDispatcher:
    """Each car owns a band of floors. Used in tall buildings and at peak."""

    def __init__(self, zones: dict[int, range]) → None:
        self._zones = zones # car id → floors it serves

    def choose(self, cars, call):
        candidates = [c for c in cars
                       if call.floor in self._zones[c.id] and c.passengers < c.capacity]
        if not candidates:
            candidates = [c for c in cars if c.passengers < c.capacity] # fall back
        return min(candidates, key=lambda c: abs(c.current_floor - call.floor), default=None)
```

Two implementations. Zoning reduces the average number of stops per trip, which is why real towers use it above about fifteen floors — and the fallback line matters, because a design that returns nothing when a zone’s car is full is worse than one that leaks across zones.

Move 5 · The system, which holds no algorithm

```
class ElevatorSystem:
    def __init__(self, cars: list[Elevator], dispatcher: Dispatcher) → None:
        self._cars = cars
        self._dispatcher = dispatcher
        self._unassigned: list[HallCall] = []
        self._assigned: dict[tuple[int, Direction], int] = {} # call → car id

    def press_hall_button(self, floor: int, direction: Direction, now: float) → None:
        key = (floor, direction)
        if key in self._assigned:
            return # already answered; ignore
        call = HallCall(floor, direction, now)
        car = self._dispatcher.choose(self._cars, call)
        if car is None:
            self._unassigned.append(call) # all full: retry next tick
            return
        car.add_stop(floor, direction)
        self._assigned[key] = car.id
```

The `if key in self._assigned: return` is the idempotence rule, and it is a real requirement: five people on floor 4 pressing “up” is one call, not five. Without it, five cars are dispatched to one floor — which is a bug real systems have shipped.

Real systems

- **Otis, Kone, Schindler and Mitsubishi** all run variants of LOOK with a dispatch layer on top. Kone’s marketing name for their dispatcher is *destination control*; Otis’s is *Compass*.
 - **Destination dispatch** is the modern change: you enter your destination floor on a lobby keypad *before* boarding, and the system tells you which car to take. That turns a hall call into a full origin-and-destination request, so cars can be grouped by destination and the number of stops per trip drops sharply — typically a 25–30 percent reduction in journey time in tall buildings.
 - **Disk schedulers** use the same algorithm and the same names: Linux’s `elevator=` boot parameter chose between `noop`, `deadline` and `cfq`, and the SCAN/C-SCAN family is textbook for spinning disks. Mentioning that the algorithm is shared between lifts and disks is a genuinely good five-second aside.
 - **Fire-service mode** is a legal requirement, not a feature: on alarm, all cars cancel every stop, return to the ground floor, open, and stay there. Worth naming as a state that pre-empts everything, because it shows you know the domain has rules that are not about efficiency.
-

6 The numbers

The basic physics

floor-to-floor travel	2 s
stop overhead (decelerate, doors open, dwell, close)	8 s
capacity	13 people (1,000 kg)
building	10 floors, 3 cars, ~600 occupants

Why LOOK, in seconds rather than in principle

The five pending stops from §3, from floor 1:

first come, first served:	23 floors × 2 s	= 46 s of travel
LOOK:	8 floors × 2 s	= 16 s of travel

saving		30 s per sweep, ~2.9x

Add the stop overhead — five stops at 8 s each is 40 s under both — and the round becomes 86 s versus 56 s. **A 35 percent reduction in journey time from choosing the order of stops**, with no extra hardware. That is the number that justifies the algorithm.

Can three cars handle the morning?

```
up-peak: 400 people arrive at the lobby over 30 minutes
arrival rate: 400 / 1,800 s = 0.22 people per second

one round trip, up-peak:
  10 floors up      10 × 2 s = 20 s
  6 stops           6 × 8 s = 48 s
  express return down 10 × 2 s = 20 s
  -----
  round trip                88 s, carrying about 8 people

system capacity: 3 cars × 8 people / 88 s = 0.27 people per second
                  = 982 people per hour
demand:          800 people per hour
```

Capacity 982 against demand 800 — it fits, with about 20 percent headroom. That is the arithmetic the question is really asking for, and it is the moment to say what you would do if it did not fit: zoning, or destination dispatch, or a fourth car, in that order of cost.

Waiting time, which is what people actually complain about

```
average wait ≈ round trip time / (2 × number of cars)
              = 88 / 6 ≈ 15 s   at peak
off-peak, one car idle nearby: ≈ 6 s
```

Under 20 seconds is generally considered good in a ten-storey building; over 40 seconds generates complaints. Quoting a target rather than a formula is what makes this sound like engineering.

Memory and events

3 Elevator objects × ~250 B	=	750 B
pending hall calls: at most 10 floors × 2 dirs	=	20 calls × ~100 B = 2 KB

total state	≈	3 KB

Three kilobytes. **The entire system state fits in a processor cache line budget**, which is worth saying because it kills every question about databases and scaling — there is nothing to scale. The constraint here is physics, not computing.

events: ~800 hall calls per hour at peak = 0.22 per second

Under one event every four seconds. A single control loop ticking every 100 ms is enormously more than enough, and proposing anything concurrent would be a mistake.

7 The trade-offs

What this design gives up

LOOK is not optimal, and cannot be. The optimal schedule needs to know future requests, and you do not have them. LOOK is a *greedy* rule that is provably starvation-free and close to optimal in practice, which is the honest claim. Do not say “optimal”; say “no starvation, and within a few percent of the best possible schedule for realistic traffic”.

Dispatch decisions are final here, and real systems re-evaluate. If car 1 is assigned a call on floor 5 and then car 2 finishes early on floor 4, car 2 is now the better answer. Reassignment improves waiting times and introduces a new failure: a call that gets passed between cars repeatedly and is never served. Real controllers reassign but pin the decision once a car is within one or two floors. I would build the simple version first and name this as the next improvement.

Direction is inferred for car requests. `press_floor` decides up or down from the car’s current position, which is wrong in one case: someone presses floor 3 while the car is at 3 with the doors open. Guard it, or the stop is silently dropped.

Capacity is a count, not a weight. Real lifts weigh the car, and thirteen people is a legal proxy. A full car should decline hall calls but must still serve its own car requests, and the distinction matters: skipping your own passengers’ floors is much worse than skipping a hall call.

No express zones or up-peak mode. In the morning, most traffic is lobby-to-everywhere, and a system that parks idle cars at their current floor wastes them. Real systems park idle cars at the lobby during up-peak. That is a policy and it belongs in the dispatcher, which is exactly why the dispatcher is an interface.

Nothing here handles failure. A stuck car, a door that will not close, a floor sensor that lies. A real controller has a watchdog per car and takes a non-responding car out of dispatch. Naming that is better than designing it.

“I would change this design if...”

- **...the building is over about fifteen floors.** Then zoning, or an express bank serving 1 and 20–40 only. The dispatcher changes; nothing else does.
- **...traffic is heavily up-peak.** Then park idle cars at the lobby and consider destination dispatch, which typically cuts journey time by a quarter to a third.
- **...waiting times are measured and are over 40 seconds.** Then reassignment of pending calls, and only then a fourth car — hardware is the expensive answer.
- **...there is one call button per floor rather than up and down.** Then the direction is unknown at dispatch time, the car has to guess, and average waits get materially worse. I would push back on that requirement rather than design around it.

The honest concession

The dispatcher is behind an interface and the stop ordering is not, and that asymmetry is a deliberate judgement rather than an oversight. I can name two dispatch policies that real buildings genuinely use — nearest-suitable and zoned — so the interface earns itself. I cannot name a second stop-ordering algorithm anybody would ship, because LOOK is what every lift in the world does. An interface there would be a hierarchy built for a future that will not arrive.

8 In the interview

How it gets asked

- The standard: *“Design an elevator system for a ten-storey building with three lifts.”*
- The narrowed version, which is better for you: *“How do you decide which lift answers a call?”* — the interesting part has been named; go straight there.
- The algorithm probe: *“What is wrong with always sending the nearest lift?”* — they want the word *starvation*.
- The capacity question: *“Four hundred people arrive between 9:00 and 9:30. Is three lifts enough?”*

- The curve ball: *“Now the building is forty floors”, or “now there is a fire alarm”.*

The timed script

Minutes 0–5 · Clarify. Floors and cars. Separate up and down buttons outside? Capacity, and does a full car skip calls? Control logic only? State the assumptions and the exclusions.

Minutes 5–10 · Estimation. Round-trip time, system capacity in people per hour, demand at peak, average wait. This is unusual for a low-level design round and it is exactly what makes this prompt different — say the 982-against-800 number early, because it justifies every later decision.

Minutes 10–15 · The two decisions, separated. “There are two scheduling problems here and I want to keep them apart: which car answers a hall call, and in what order one car serves the stops it has. Different scopes, different answers.”

Minutes 15–25 · The car and LOOK. The two stop sets, the state machine, `next_stop` in four lines. Compare FCFS, nearest-first and LOOK with the floor counts and name starvation.

Minutes 25–35 · Dispatch behind an interface, with two implementations and the scoring function explained rather than read.

Minutes 35–40 · Failure modes and extension. Idempotent hall calls. A full car. Reassignment as the next improvement, with its risk. Fire mode as a state that pre-empts everything. Then the forty-floor answer: zoning.

The follow-ups

“What is wrong with always sending the nearest lift?” “It starves. Nearest-first is greedy on distance, so if people keep pressing buttons near a car, a request at the far end is never the nearest one and waits indefinitely — in a real building that is the top floor complaining about the lifts, and it is a historical complaint, not a hypothetical one. LOOK fixes it structurally rather than with a timeout: because the car serves everything ahead of it before turning, a request above the car is guaranteed to be served on this sweep.”

“How do you decide which lift answers a hall call?” “A score, and the ordering matters more than the numbers. Best is a car already moving in the caller’s direction with the caller ahead of it, because it passes that floor anyway and the marginal cost is one stop. Next is an idle car, scored by distance plus a penalty. Worst is a car going the wrong way, which has to finish its sweep first. I would keep those penalties as constants and say plainly that they are a tuning knob to fit against measured waiting times, not physics.”

“Four hundred people arrive in half an hour. Is three lifts enough?” “Let me work it out. That is 800 people an hour. A round trip in up-peak is ten floors up at two seconds a floor, six stops at eight seconds each, and an express return — 88 seconds, carrying about eight people. Three cars gives about 982 people an hour against 800 demanded, so it fits with roughly twenty percent headroom, and average wait is round-trip over twice the number of cars, about fifteen seconds. If it had not fitted, the order of fixes by cost is: park idle cars at the lobby during up-peak, then zone the cars, then destination dispatch, then a fourth car.”

“Five people on floor 4 press ‘up’. What happens?” “One call. The system keys pending hall calls by floor and direction and ignores a repeat, so five presses dispatch one car. Without that, five cars converge on floor 4 — which is a bug real systems have shipped. The button light being on is the state, and it clears when a car arrives at that floor going that way.”

“Now the building is forty floors.” “The car logic does not change at all — LOOK is LOOK. The dispatcher does. At forty floors a single pool means every car serves every floor and the average trip has too many stops, so I would zone: cars 1–2 serve 1–20, cars 3–4 serve 1 and 21–40 as an express bank. That is a different `Dispatcher` implementation and nothing else in the design moves, which is why that decision is behind an interface. Above that scale, destination dispatch, where people enter their floor in the lobby, cuts journey time by roughly a quarter to a third by grouping passengers with the same destination.”

“What about a fire alarm?” “Fire-service mode, and it is a legal requirement rather than a feature. On alarm, every car cancels every pending stop, ignores all buttons, travels to the designated floor — usually the ground — opens its doors and stays there until a keyed switch releases it. In the design it is a state that pre-empts everything else, so I would model it as a mode flag checked at the top of the control loop rather than as another entry in the stop sets, because it must not be schedulable.”

A model answer

Asked: design an elevator system for a ten-storey building with three lifts.

“A few clarifying questions first, because two of the answers change the design. Are the buttons outside separate up and down, or one call button? Separate — good, because a single button loses the direction and forces the car to guess. Is there a capacity limit and should a full car skip hall calls? Thirteen and yes. And am I designing control logic or hardware? Control logic. I will assume one car per shaft and no express floors, and I will note where fire mode attaches rather than design it.

Let me do the arithmetic before the classes, because it decides how hard this has to be. Two seconds a floor, about eight seconds for a stop with the doors. In the morning peak, four hundred people arrive over half an hour, so 800 an hour. A round trip is ten floors up, six stops, an express return — about 88 seconds carrying eight people. Three cars is roughly 982 people an hour against 800 demanded, so it fits with about twenty percent headroom, and the average wait is round trip over twice the car count, about fifteen seconds. Also worth saying: the entire system state is a few kilobytes. There is nothing to scale here; the constraint is physics.

Now the design, and the first thing I want to do is separate two scheduling problems that get merged. There is *which car answers a hall call*, which is a system-level decision, and there is *in what order does one car serve the stops it has accepted*, which is per-car. Different scopes, different answers.

Start with the car. It holds its floor, its direction, its door state, and — this is the part that matters — **two** sets of stops rather than one: floors to serve while going up, and floors to serve while going down. A stop at floor seven means something different depending on which way the person there wants to travel, and one set cannot express that.

The ordering algorithm is LOOK, the elevator algorithm: keep going in the current direction while anything ahead of you wants that direction, and only when there is nothing ahead do you turn. Let me justify it against the two obvious alternatives with a concrete case — a car on floor one with stops at 2, 9, 3, 8 and 4. First-come-first-served travels 23 floors and crosses the building four times. Nearest-first travels 8 floors and **starves**: if people keep pressing buttons near the car, floor nine waits for ever, which in a real building is the top floor complaining about the lifts. LOOK also travels 8 floors and cannot starve, because everything ahead is served before the car turns. In seconds that is 46 against 16 of travel time, so about a 35 percent shorter journey with no extra hardware.

Then dispatch, and this is where I would put an interface, because it is the decision that actually changes between buildings. The default rule: prefer a car already travelling the caller's direction with the caller ahead of it, because that car passes the floor anyway; then an idle car, scored by distance plus a penalty; then a car

going the wrong way, with a bigger penalty. I would say plainly that those penalties are a tuning knob to fit against measured waiting times, not physics. And I would write a second implementation — a zoned dispatcher where each car owns a band of floors — because one implementation is a class and two is a demonstration that the design survives a change.

I would deliberately *not* put an interface around the stop ordering. I cannot name a second algorithm anyone would ship; every lift in the world uses LOOK. An interface there would be a hierarchy for a future that will not arrive.

Two details I would raise before you ask. First, hall calls must be idempotent: five people on floor four pressing ‘up’ is one call, so I key pending calls by floor and direction and ignore repeats — otherwise five cars converge on one floor, and that bug has shipped in real systems. Second, a full car should decline hall calls but must still serve its own passengers’ floor requests, because skipping your own passengers is far worse than skipping a waiting person.

And what I would do next, in order: reassign pending calls when a nearer car frees up — which improves waiting times and introduces a call that can be passed around for ever, so I would pin the decision once a car is within a floor or two — then park idle cars at the lobby during up-peak, then zone the cars if the building grows. Fire mode sits outside all of it: a mode flag checked at the top of the control loop that cancels every stop and sends every car to the ground floor, because it must not be schedulable.”

9 Recall card

- **Separate the two scheduling problems before designing anything: *which car answers a hall call* (system-level, dispatch) and *in what order does one car serve its stops* (per-car, ordering).** Merging them is the standard mistake. A **hall call** carries a floor **and** a **direction** and any car may take it; a **car request** carries only a floor and belongs to that car alone.
- **The per-car answer is LOOK, the elevator algorithm: keep going while anything ahead wants this direction, then turn.** Against stops at 2, 9, 3, 8, 4 from floor 1: **FCFS 23 floors · nearest-first 8 but it STARVES · LOOK 8 and cannot starve.** In seconds, 46 vs 16 of travel — about **35 percent shorter journeys**. The strict-to-the-end variant is **SCAN**; disk schedulers use the same algorithm.
- **Keep TWO stop sets per car, `up_stops` and `down_stops`,** because floor 7 means different things to someone going up and someone going down. Then `next_stop` is `min(above)` while going up, and the turn is `max(below)`. Every arrival goes through

a **doors-open** state and every direction change through closed doors — that is the safety property, drawn.

- **The estimate is the part that makes this prompt different — do it before the classes.** 2 s/floor, 8 s/stop, 13 capacity → round trip **88 s** carrying 8 → 3 cars $\approx$ **982 people/hour against 800 demanded**, average wait $\approx$ **15 s**. Total system state $\approx$ **3 KB**; *there is nothing to scale here; the constraint is physics.*
- **The interface goes on Dispatcher and NOT on the stop ordering** — two real dispatch policies exist (nearest-suitable, zoned) and no second ordering algorithm does. **Hall calls must be idempotent** (five presses on floor 4 = one call, or five cars converge). A **full car declines hall calls but still serves its own passengers**. Next improvements in cost order: reassignment · lobby parking at up-peak · zoning · **destination dispatch** (25–30% faster) · a fourth car. **Fire mode pre-empt everything and must not be schedulable.**

DSA topic: Traversal, insertion, and deletion **System design topic:** Design an elevator system

Code these, in this order

One rule for the whole set: **say “stand at $k-1$, act on k ” before every single operation.** If your walk loop runs `range(index)` when it should run `range(index - 1)`, you will delete the wrong node and the tests will pass on three inputs out of four.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Middle of the Linked List	LeetCode 876 (Easy)	The plain traversal loop, and whether you can state the two-answer ambiguity for even lengths before coding.
2	Design Linked List	LeetCode 707 (Medium)	Every boundary at once: index 0, index size, index out of range, empty list, single element.
3	Remove Linked List Elements	LeetCode 203 (Easy)	Look-one-ahead, consecutive matches, and the leading run that makes the head case a <i>loop</i> , not an <code>if</code> .
4	Insert into a Sorted Circular Linked List	LeetCode 708 (Medium)	The walk that has no <code>None</code> to stop it, and the all-equal-values case that hangs.

On problem 2, write the six boundary tests first

`addAtIndex(0, x)` on an empty list. `addAtIndex(size, x)`. `addAtIndex(size + 1, x)`. `deleteAtIndex(0)` on a one-element list. `deleteAtIndex(size - 1)`. `deleteAtIndex(size)`. Write those, watch them fail, then implement.

On problem 3, break it on `[7, 7, 7]`

Write the loop that advances `previous` after every iteration, including after a deletion. Run it on `[7, 7, 7]` with `val = 7` and write down what comes out. Then say in one sentence why you must not advance after removing.

On problem 4, find the input that hangs

A circular list has no `None`, so your loop needs a different stopping condition. Run it on a list where every value is the same and the insert value is different. If it hangs, you have found the case the problem exists for.

The rule drill

Answer each without looking:

1. State the rule that generates traversal, insertion and deletion, in one sentence.
2. Why can you not unlink a node you are holding?
3. How many steps do you walk to delete index `k`?
4. Write the deletion in one assignment.
5. Write the insertion in one assignment, and say which half is built first and why.
6. Name the three boundary cases and say what makes each one different.

The bounds drill

1. How many valid positions does insertion have on a list of length `n`?
2. How many does deletion have?
3. Write both bounds checks and say which uses `>` and which uses `≥`.
4. What must happen before any dereference, and what is the error text if it does not?
5. Which single character separates a working `insert_at(size, x)` from one that raises?

The head-case drill

1. Write `delete_at(0)` for a bare function taking a head and returning something.
2. Call it and discard the return value. Print the list. Explain the result in terms of how Python passes arguments.
3. Now write it as a method on a class. Say what changed and why the bug is no longer available.
4. Write `delete_at(0)` on a one-element list and say which two pieces of bookkeeping must change.

The bookkeeping drill

1. Add `_size` to a bare linked list and list every method that must update it.
2. Add `_tail` and list every method that must update it.
3. Name the one deletion that must move the tail *backwards*, and say why it is $O(n)$ anyway.
4. Write `_check` with three assertions. Say what each one catches.
5. Delete the tail update from `delete_at`, then append and print. Describe what you see, and note that there is no error.

The break-it drill

Trigger each and record the exact output or error text:

1. Walk `range(index)` instead of `range(index - 1)` in `delete_at`. Run it on `[7, 3, 9, 4, 8]` with `k = 2` and say which value disappeared.
2. Call `_node_at(index - 1)` with `index = 0` and no head case. Quote the error, or describe the silent wrong behaviour if you did not guard.
3. Use `>=` instead of `>` on the insert bound and call `insert_at(size, x)`. Quote the error.
4. Write `while previous.next.value != target` with the target absent. Quote the error.
5. Assign `previous.next = new` before `new.next = previous.next`. Traverse and describe the failure.
6. Delete the only node without clearing `_tail`, then append twice. Print the list.
7. Forget `self._size -= 1` in `delete_at`, then call `value_at(size - 1)`. Describe what happens.

The quadratic drill

1. Write `append` without a tail reference.
2. Build a list of 100,000 elements with it and time it.
3. Add the tail reference and time it again.
4. State both complexities and the arithmetic for the slow one.
5. Say which other operation in this class has the same shape of hidden cost.

The comparison drill

For each operation, give the complexity for a linked list and a Python list, and say which wins:

1. Insert at the front.
2. Delete at the front.
3. Insert at the end.
4. Delete at the end.
5. Insert in the middle at a known index.
6. Access by index.
7. Length.

For number 5, both are $O(n)$. Say what each one is actually spending its time on, and which is faster in practice.

The two-decisions drill

1. State the two scheduling decisions in an elevator system, and say which is system-level and which is per-car.
2. Say what a hall call carries that a car request does not.
3. Say why that difference means the system, not the car, chooses for a hall call.
4. Give one sentence for why merging the two decisions is the standard mistake.

The algorithm drill

Take a car on floor 1 with pending stops at 2, 9, 3, 8, 4 in that arrival order.

1. Compute the total floors travelled under first-come-first-served.
2. Compute it under nearest-first.
3. Compute it under LOOK.
4. Convert all three to seconds at two seconds a floor.
5. Construct a request stream where nearest-first starves floor 9, and say how long it waits.
6. Say why LOOK cannot starve, structurally rather than with a timeout.
7. Name the strict variant that runs to the very top before turning, and one other system that uses the same algorithm.

The two-sets drill

1. Say why a car keeps `up_stops` and `down_stops` rather than one set.
2. Give a concrete floor that appears in both and explain what it means.
3. Write `next_stop` for the going-up case in four lines.
4. Write the turn condition, and say what floor you go to when you turn.
5. Say how `press_floor` decides which set to use, and name the one case where that inference is wrong.

The estimation drill

Compute each from scratch, showing the multiplication:

1. Round-trip time in up-peak: ten floors, six stops, express return.
2. People per hour for three cars.
3. Demand in people per hour for 400 arrivals in 30 minutes.
4. Headroom, as a percentage.
5. Average wait, and whether it is acceptable.
6. Total system state in bytes.

Then say which number tells you “there is nothing to scale here” and which tells you whether to buy a fourth lift.

The dispatch drill

1. Write the score function with its three cases.
2. Explain the `approaching` expression in one sentence, including why multiplying by the direction value works for both directions.
3. Say what the penalties are and, honestly, where their values come from.
4. Write the zoned dispatcher and say why it needs a fallback.
5. Say why the dispatcher gets an interface and the stop ordering does not.

The failure drill

For each, say what happens and what you would add:

1. Five people on floor 4 press “up”.
2. A car is full and a hall call arrives on a floor it is passing.
3. A car is full and one of its own passengers wants floor 7.
4. Car 1 is assigned a call on floor 5, then car 2 becomes idle on floor 4.
5. A car stops responding entirely.
6. The fire alarm goes off.
7. Someone presses floor 3 from inside while the car is at floor 3 with the doors open.

Two of the seven are legal or safety requirements rather than optimisations. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Delete the node at position k . What if it is the head?* The stand-at- $k-1$ rule and why you cannot reach backwards, the one-assignment unlink, the head as a *different* operation rather than a harder one, returning the new head and why the bug is silent, the different bounds for insertion and deletion, and the size and tail bookkeeping with the stale tail named.
2. *Design an elevator system for a ten-storey building with three lifts.* The two scheduling decisions separated first, the estimate before the classes, the two stop sets, LOOK justified against FCFS and nearest-first with floor counts and the word starvation, dispatch behind an interface with two implementations, and idempotent hall calls raised unprompted.

3. *What is wrong with always sending the nearest lift?* Starvation, with the concrete request stream that causes it, why it shows up as the top floor complaining, and how LOOK removes it structurally rather than with a timeout.

Before you move on

- [] I can state the stand-at- $k - 1$ rule and say why reaching backwards is impossible.
- [] I can write deletion and insertion each as one assignment, in the right order.
- [] I know the three boundary cases and what makes each different.
- [] I can say why insertion allows `size + 1` positions and deletion allows `size`.
- [] I discarded a return value from a head-changing function and saw nothing happen.
- [] I wrote `_check` and used it to catch a stale tail.
- [] I deleted the tail update and watched an appended value vanish with no error.
- [] I timed 100,000 appends with and without a tail reference.
- [] I can give the linked-list-versus-array table from memory.
- [] I can separate the two elevator scheduling decisions in one sentence each.
- [] I computed FCFS, nearest-first and LOOK travel for the same five stops.
- [] I can construct the starvation stream and explain why LOOK is immune.
- [] I can explain why the car keeps two stop sets, with a floor that is in both.
- [] I did the up-peak estimate and can say whether three lifts are enough and by how much.
- [] I can say why the dispatcher has an interface and the stop ordering does not.
- [] I answered all three questions above out loud.

The dummy head trick

DATA STRUCTURES AND ALGORITHMS

The dummy head trick

You stop writing a special case for the head, in every linked list problem.

SYSTEM DESIGN

Design an ATM

You can model the state machine, the cash dispenser, and the failure cases.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Remove all nodes with a given value, including at the head.*
- *Design an ATM. What happens if the network dies mid-withdrawal?*

1 What this is, and why they ask it

Yesterday every operation had a special case for position zero, because the first node has no predecessor and you cannot unlink a node without one. Today you delete that special case for ever, with one line:

```
dummy = Node(0, head)
```

You put a fake node in front of the list. Now the real first node *has* a predecessor — the fake one — so it is an ordinary node like every other, and the loop that handles the middle handles the front too. At the end you return `dummy.next`, which is whatever the first node turned out to be.

They ask it because it turns a fifteen-line function with three branches into an eight-line function with none, and because the interviewer can see immediately whether you have written linked-list code before. “Remove all elements equal to `val`” is LeetCode 203 and it is the purest test there is: the input `[7, 7, 7, 1, 7]` breaks every solution that treats the head with an `if` instead of a loop. The same trick then reappears in merge-two-sorted-lists, partition, remove-duplicates, and remove-Nth-from-end — half the problems in this phase.

2 The story

The lunch queue at Amina’s school forms at twelve twenty, outside the hall, along the wall with the handwash taps.

The rule the children are given on their first day is one sentence: **stand behind the person in front of you, and move when they move**. It works. Nobody has to be told anything else. Forty children, one rule.

Except for the child at the front, for whom the rule means nothing. There is nobody in front of them. So the front child gets a different rule — watch the hall door, and when Sister opens it, go — and because it is a different rule, it goes wrong in exactly the ways different rules go wrong. The front child goes when the door opens for someone else. The front child does not notice the door at all because he is talking. Two children argue about

who is actually the front child. And when the front child leaves, somebody has to work out who has become the front child now and tell them, which means somebody has to be watching for that.

Sister Mary fixed it four years ago and it took her about a minute.

She stands at the front of the queue. She is not in the queue — she is not getting any lunch, she is not counted, she does not move up — she just stands there, at the head, every day, so that there is always somebody in front of the first child.

Now there is one rule and it is the same rule for everybody, including the child at the front. Stand behind the person in front of you. Move when they move. The front child watches Sister's back like everybody else watches the back in front of them. Nobody needs to be told they have become the front child, because being the front child stopped being a special job.

There is one thing the teachers have to remember, and a student teacher got it wrong last term. When you count the queue, you do not count Sister. She is standing in the line and she is not in the line. The student teacher reported forty-one children waiting for thirty-nine lunches and there was a short, unnecessary panic in the kitchen.

3 The idea in plain English

Sister Mary is a **dummy node** — sometimes called a sentinel head. She sits in front of the real list, holds nothing anybody wants, and exists for one reason: **so that the first real node has a predecessor**.

The line, and what it removes

```
dummy = Node(0, head)
```

That is it. The `0` is never read; the value does not matter and by convention people write `0`. What matters is `dummy.next = head`, because now:

- the first real node is reachable as `dummy.next`, exactly the way the third node is reachable as `something.next`;
- deleting the first node is `dummy.next = dummy.next.next`, which is the *same line* as deleting any other node;
- and the answer at the end is `dummy.next`, which correctly reflects whatever the first node became — including `None` if you deleted everything.

`return dummy.next`, **not** `return head`. This is the student teacher counting Sister. `head` still refers to the node it always referred to, which may have been deleted, so returning it returns a deleted node or a list with the removed element still at the front. It is the single most common bug in this technique and it produces no error at all.

The problem it is for

Remove every node whose value is `7` from `[7, 7, 7, 1, 7]`.

Without a dummy, the head needs a **loop**, not an `if`, because several leading nodes may match:

```
while head is not None and head.value == 7:      # a loop, not an if
    head = head.next
if head is None:
    return None
previous = head
while previous.next is not None:
    if previous.next.value == 7:
        previous.next = previous.next.next
    else:
        previous = previous.next
return head
```

Eleven lines, two loops, one `if`, and the top three lines exist purely because the head is special. Now with a dummy:

```
dummy = Node(0, head)
previous = dummy
while previous.next is not None:
    if previous.next.value == 7:
        previous.next = previous.next.next
    else:
        previous = previous.next
return dummy.next
```

Six lines, one loop, and the head is handled by the same code as everything else. The special case did not get shorter — it stopped existing.

The second rule, which is not about the dummy at all

Look at the `else` in that loop.

```
if previous.next.value == 7:
    previous.next = previous.next.next      # do NOT advance
else:
    previous = previous.next                # only advance when you keep
```

After a deletion, `previous.next` is a **new** node that you have not looked at yet. Advancing would skip it, so two consecutive 7s would leave one behind. On `[1, 7, 7, 2]` an unconditional advance returns `[1, 7, 2]`.

Advance only when you keep. Say it as you write it — it is the second half of this problem and it has nothing to do with the dummy node.

The other use: a dummy as a builder

The dummy has a second job that is just as common, and it looks different enough that people miss the connection.

When you are **building** a new list — merging two sorted lists, partitioning, copying — the first node is special again, for the mirror reason: you have nowhere to attach it to. So the code fills up with “is this the first one?”.

```
dummy = Node(0)          # holds nothing; exists so `tail` has somewhere to start
tail = dummy
while ...:
    tail.next = chosen_node
    tail = tail.next
return dummy.next
```

`tail` is a moving cursor that always points at the last node built so far, and `dummy` is where it starts. There is no “first node” case, because the first node is attached exactly like the fortieth.

Two uses, one idea: **a node that holds nothing, so that “the first one” stops being different.**

Where it shows up

- **Remove Linked List Elements** (LeetCode 203) — the pure case.
- **Merge Two Sorted Lists** (21) — the builder form.
- **Remove Nth Node From End of List** (19) — where the node to remove may be the head, and the dummy also makes the two-pointer gap arithmetic come out right.
- **Remove Duplicates from Sorted List II** (82) — delete *all* copies, so even the head may be part of a run.
- **Partition List** (86) — two dummies, one per output list, then join them.
- **Reverse Nodes in k-Group** (25) — the hardest of them, and unwritable without a dummy.
- **Swap Nodes in Pairs** (24), **Add Two Numbers** (2), **Odd Even Linked List** (328).

If a problem can change the first node, use a dummy. That is the whole recognition rule.

What it costs

One node — about 48 bytes — and one line. It does not change the complexity of anything. It is the cheapest simplification in this course, and the reason to know it is that the branches it removes are where the bugs are.

4 The picture

Removing every 7 from `[7, 7, 7, 1, 7]`.

WITHOUT a dummy — the head needs its own loop first

```
head
|
v
[7]→[7]→[7]→[1]→[7]→ None
^^^^^^^^^^^^^^^^^^
handled by loop 1      handled by loop 2
(three iterations)
```

WITH a dummy — one loop, and `previous` starts on a node that is not in the list

```
dummy (not part of the list, value never read)
|
v
[0]→[7]→[7]→[7]→[1]→[7]→ None
  ^      ^
previous previous.next ← the same shape as every other step

step 1: previous.next is 7 → previous.next = its next (do NOT advance)
[0]→[7]→[7]→[7]→[1]→[7]
step 2: previous.next is 7 → unlink again
[0]→[7]→[1]→[7]
step 3: previous.next is 7 → unlink again
[0]→[1]→[7]
step 4: previous.next is 1 → KEEP, so advance
[0]→[1]→[7]
           ^ previous
step 5: previous.next is 7 → unlink
[0]→[1]→ None

return dummy.next → [1]
return head       → [7]→... the node `head` still refers to. WRONG.
```

What to notice: `previous` never moves during steps 1 to 3. Three deletions in a row happen from the same standing position, which is exactly what an `if` on the head cannot do and a loop can.

And the builder form, merging `[1, 3]` and `[2, 4]`:

```

dummy
|
v
[0]                                tail = dummy
|
v
[0]→[1]                            take 1, tail = the 1
|    ^tail
v
[0]→[1]→[2]                        take 2, tail = the 2
|    ^tail
v
[0]→[1]→[2]→[3]→[4]
|
return dummy.next → [1, 2, 3, 4]

```

Not once did the code ask "is this the first node?"

5 The code, built step by step

Step 1 — the line, and what it means

```

dummy = Node(0, head)
previous = dummy

```

Two lines. `dummy` never moves and is never returned; `previous` is the cursor that walks. Keeping them as separate names matters — if you walk `dummy` itself forward, you have lost your handle on the front of the list and cannot return the answer.

Step 2 — the loop, with the advance rule

```

while previous.next is not None:
    if previous.next.value == target:
        previous.next = previous.next.next    # unlink; stay where you are
    else:
        previous = previous.next              # keep; move on

```

The condition looks at `previous.next`, never at `previous`, because `previous` is the joint and `previous.next` is the wagon. Every delete-by-condition problem in this phase is this loop.

Step 3 — the return

```

return dummy.next

```

Not `head`. If the original head was deleted, `head` still refers to a node that is no longer in the list.

Step 4 — the builder form

```
dummy = Node(0)
tail = dummy
while a is not None and b is not None:
    if a.value ≤ b.value:
        tail.next, a = a, a.next           # attach a, advance a
    else:
        tail.next, b = b, b.next
    tail = tail.next
```

`tail.next, a = a, a.next` evaluates the right side first, so `a.next` is read before `a` is reassigned. Written as two statements in the wrong order it loses the rest of `a`. If tuple assignment makes you nervous under pressure, write it as three explicit lines — clarity beats cleverness in an interview.

```
tail.next = a if a is not None else b           # one list is empty; attach the rest
return dummy.next
```

The last line is why merging is $O(n + m)$ and not more: whatever remains is already sorted and already linked, so you attach it whole rather than copying node by node.

Step 5 — two dummies, for problems that build two lists

Partition List (LeetCode 86) splits a list into “less than x ” and “at least x ”, preserving order.

```
less_dummy, more_dummy = Node(0), Node(0)
less, more = less_dummy, more_dummy
node = head
while node is not None:
    target = less if node.value < x else more
    target.next = node
    target = ...                               # see the full version below
```

Two dummies, two cursors, then join. `more.next = None` at the end is mandatory — the last node of the “more” list still points into the original list, and forgetting it builds a cycle.

The complete solution

```
class Node:
    __slots__ = ("value", "next")

    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value
        self.next = next

def from_values(values: list[int]) → Node | None:
    head: Node | None = None
    for value in reversed(values):
        head = Node(value, head)
    return head

def to_values(head: Node | None) → list[int]:
    out, node = [], head
    while node is not None:
        out.append(node.value)
        node = node.next
    return out

def remove_elements(head: Node | None, target: int) → Node | None:
    """Remove EVERY node equal to target, including at the head.

    The dummy gives the first real node a predecessor, so the head stops being
    a special case. Two rules:
    - return dummy.next, never head (head may have been deleted)
    - advance `previous` ONLY when you keep a node, or consecutive matches
      leave one behind
    """
    dummy = Node(0, head)
    previous = dummy

    while previous.next is not None:
        if previous.next.value == target:
            previous.next = previous.next.next    # unlink, stay put
        else:
            previous = previous.next              # keep, move on

    return dummy.next

def merge_two_sorted(a: Node | None, b: Node | None) → Node | None:
    """Merge two sorted lists into one. The BUILDER form of the dummy: `tail`
    needs somewhere to start, so the first node is attached like every other.

    O(n + m) time, O(1) extra space – no nodes are created, only relinked.
    """
    dummy = Node(0)
    tail = dummy

    while a is not None and b is not None:
        if a.value ≤ b.value:                      # ≤ keeps the merge STABLE
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next

    tail.next = a if a is not None else b          # attach the rest wholesale
    return dummy.next
```

```

def remove_nth_from_end(head: Node | None, n: int) → Node | None:
    """Remove the nth node from the end in one pass.

    The dummy earns itself twice here: removing the last remaining node is the
    head case, and starting `fast` at the dummy makes the gap arithmetic land
    on the node BEFORE the target rather than on the target.
    """
    dummy = Node(0, head)
    fast = slow = dummy

    for _ in range(n):
        # open a gap of n
        if fast.next is None:
            raise ValueError(f"list has fewer than {n} nodes")
        fast = fast.next

    while fast.next is not None:
        # walk both until fast is last
        fast = fast.next
        slow = slow.next

    slow.next = slow.next.next
    return dummy.next

def remove_all_duplicates_sorted(head: Node | None) → Node | None:
    """From a SORTED list, remove every value that appears more than once.
    [1,2,3,3,4,5] → [1,2,5]. Even the head may be part of a run.
    """
    dummy = Node(0, head)
    previous = dummy
    node = head

    while node is not None:
        if node.next is not None and node.value == node.next.value:
            duplicate = node.value
            while node is not None and node.value == duplicate:
                node = node.next
                # skip the whole run
            previous.next = node
            # unlink all of it at once
        else:
            previous = node
            node = node.next

    return dummy.next

def partition(head: Node | None, x: int) → Node | None:
    """Reorder so that every node < x comes before every node ≥ x, keeping the
    relative order within each group. Two dummies, two cursors, then join.
    """
    less_dummy, more_dummy = Node(0), Node(0)
    less, more = less_dummy, more_dummy

    node = head
    while node is not None:
        if node.value < x:
            less.next = node
            less = less.next
        else:
            more.next = node
            more = more.next
        node = node.next

    more.next = None
    less.next = more_dummy.next
    return less_dummy.next

```

```

if __name__ == "__main__":
    print(to_values(remove_elements(from_values([7, 7, 7, 1, 7]), 7))) # [1]
    print(to_values(remove_elements(from_values([1, 7, 7, 2]), 7))) # [1, 2]
    print(to_values(remove_elements(from_values([7, 7, 7]), 7))) # []
    print(to_values(remove_elements(from_values([]), 7))) # []
    print(to_values(remove_elements(from_values([1, 2, 3]), 7))) # [1, 2, 3]

    print(to_values(merge_two_sorted(from_values([1, 3, 5]),
                                         from_values([2, 4, 6])))) # [1,2,3,4,5,6]
    print(to_values(merge_two_sorted(from_values([]), from_values([2])))) # [2]
    print(to_values(merge_two_sorted(None, None))) # []

    print(to_values(remove_nth_from_end(from_values([1, 2, 3, 4, 5]), 2))) # [1,2,3,5]
    print(to_values(remove_nth_from_end(from_values([1, 2, 3, 4, 5]), 5))) # [2,3,4,5]
    print(to_values(remove_nth_from_end(from_values([9]), 1))) # []

    print(to_values(remove_all_duplicates_sorted(
        from_values([1, 2, 3, 3, 4, 4, 5])))) # [1, 2, 5]
    print(to_values(remove_all_duplicates_sorted(
        from_values([1, 1, 1, 2, 3])))) # [2, 3]

    print(to_values(partition(from_values([1, 4, 3, 2, 5, 2]), 3))) # [1,2,2,4,3,5]

    # the bug this whole lesson is about
    original = from_values([7, 7, 1])
    result = remove_elements(original, 7)
    print(to_values(result)) # [1] ← dummy.next
    print(to_values(original)) # [7, 7, 1] ← what `return head` gives

```

The last three lines are worth running. `original` still refers to the node it always referred to, and returning it hands the caller the list with the deleted elements still at the front — no exception, no warning, just a wrong answer.

6 What it costs

Time

<code>remove_elements</code>	$O(n)$	one pass, one comparison per node
<code>merge_two_sorted</code>	$O(n+m)$	each node visited once
<code>remove_nth_from_end</code>	$O(n)$	one pass, two cursors
<code>remove_all_duplicates_sorted</code>	$O(n)$	the inner while advances `node`, never restarts
<code>partition</code>	$O(n)$	one pass

The inner `while` in `remove_all_duplicates_sorted` deserves a sentence, because it looks quadratic and is not: it advances the *same* `node` cursor that the outer loop uses, so each node is visited exactly once across both loops. Same counting argument as the monotonic stack of [day 071](#).

Space

the dummy node	1 node, about 48 bytes
cursors	2 or 3 references

extra space	0(1)

One node, once. Not one per element, not one per iteration. The most common misunderstanding is creating a dummy *inside* a loop, which allocates n nodes and does nothing useful.

What it saves, counted

The version without a dummy, for `remove_elements` :

lines	11	
loops	2	
branches for the head	2	(the leading run, and the all-deleted case)
places the head is referenced	4	

With a dummy:

lines	6	
loops	1	
branches for the head	0	
places the head is referenced	1	(the dummy's constructor)

Two loops become one, and two head branches become none. That is not a style preference. Each of those branches is a place where `[7, 7, 7]` or `[]` or `[7]` breaks the function, and the empty and all-deleted cases are the two an interviewer will try.

Where the dummy does more than tidy up

In `remove_nth_from_end`, starting `fast` and `slow` at the dummy rather than at the head does two things at once. It makes “remove the first node” an ordinary case. And it shifts `slow` back by one so that when `fast` reaches the last node, `slow` is on the **predecessor** of the target rather than on the target itself — which is exactly where you need to be. Without the dummy you need a gap of `n + 1` and a separate check for the head, and getting that arithmetic right under pressure is unpleasant.

7 The traps

Trap 1 — `return head` instead of `return dummy.next`

```
dummy = Node(0, head)
...
return head                                # WRONG
```

`remove_elements(from_values([7, 7, 1]), 7)` returns `[7, 7, 1]`. The function did all the work correctly and then handed back a reference to a node that is no longer part of the list. **No error, wrong answer.** The student teacher counting Sister.

Make it a habit: the moment you write `dummy = Node(0, head)`, write `return dummy.next` as well, before you write the loop.

Trap 2 — advancing after a deletion

```
if previous.next.value == target:
    previous.next = previous.next.next
previous = previous.next                # advances in BOTH cases
```

On `[1, 7, 7, 2]` this returns `[1, 7, 2]`. After unlinking, `previous.next` is a node you have not inspected, and advancing skips it. **Advance only in the `else`.**

This bug is independent of the dummy and survives every other fix, which is why it deserves its own sentence in the interview.

Trap 3 — walking the dummy itself

```
dummy = Node(0, head)
while dummy.next is not None:
    ...
    dummy = dummy.next                # lost the handle
return dummy.next                    # returns the tail, or None
```

`dummy` must not move. Use a second name for the cursor. This one usually produces a nearly-empty result and is obvious the first time you see it — and completely invisible while you are writing it.

Trap 4 — creating the dummy inside the loop

```
while node is not None:
    dummy = Node(0, node)                # a new one every iteration
```

Allocates `n` nodes, achieves nothing, and often produces correct output, so it survives review. One dummy, before the loop.

Trap 5 — forgetting to terminate a built list

```
less.next = more_dummy.next
return less_dummy.next                                # forgot: more.next = None
```

The last node of the “more” chain still points at whatever followed it in the *original* list. Since that region now precedes it, the result contains a **cycle** and printing it never returns:

```
KeyboardInterrupt
```

after the output list has grown to fill memory. **Any time you build a new list out of existing nodes, terminate it explicitly.**

Trap 6 — using a dummy where the list cannot change at the front

Not a bug, just noise. Counting nodes, finding the middle, checking for a cycle — none of these can change the first node, so a dummy adds a line and buys nothing. **The recognition rule: use a dummy when the operation can change which node is first, or when you are building a new list.** Otherwise do not.

Trap 7 — `dummy = Node(0)` when you meant `Node(0, head)`

```
dummy = Node(0)                                     # forgot to link the head
previous = dummy
while previous.next is not None:                     # immediately false
    ...
return dummy.next                                    # None
```

Returns an empty list for every input. Silent, and easy to stare past because the loop looks right. The guard form is `Node(0, head)`; the builder form is `Node(0)` with nothing attached yet. Know which one you are writing.

8 In the interview

How it gets asked

- The pure case: “Remove all nodes with a given value.” LeetCode 203. The interviewer will hand you `[7, 7, 7, 1, 7]` on purpose.
- The builder case: “Merge two sorted linked lists.” LeetCode 21, and the follow-up is “now merge k lists”.
- The one where the dummy does real work: “Remove the *n*th node from the end, in one pass.” LeetCode 19.

- The hardest: “Remove all values that appear more than once from a sorted list.” LeetCode 82 — even the head can be part of a duplicate run, so an `if` cannot save you.
- The meta version: “You wrote a dummy node there. Why?”

What to say out loud, in the first ninety seconds

1. **Name the problem before the trick.** “The awkward part here is that the node I need to remove might be the head, and the head has no predecessor — I cannot unlink a node without holding the one before it.”
2. **Introduce the dummy as the fix, in one sentence.** “So I put a dummy node in front of the list. Now the real head has a predecessor and it is an ordinary node, so one loop handles everything.”
3. **Say the return rule immediately.** “And I return `dummy.next`, not `head` — the original head may have been deleted, and returning it is a silent wrong answer.”
4. **Say the advance rule, as a separate point.** “The other rule in this loop has nothing to do with the dummy: after unlinking, I do *not* advance, because `previous.next` is now a node I have not looked at. On `[1, 7, 7, 2]`, advancing unconditionally leaves a 7 behind.”
5. **State the cost.** “One extra node and $O(1)$ space. It does not change the complexity of anything; it removes branches.”
6. **Then write it**, which is six lines.

The follow-ups

“Why not just handle the head with an `if`?” “Because it is not one node — it is a *run*. On `[7, 7, 7, 1]` the first three all have to go, so the head case is a `while`, not an `if`, and then there is a second special case when everything is deleted and the head becomes `None`. That is two extra branches and two extra tests. The dummy makes the count of special cases zero rather than smaller, which is a different kind of win.”

“What is the dummy’s value?” “It does not matter and it is never read — the loop only ever inspects `previous.next`. I write `0` by convention. The important part is `dummy.next = head`, and the second important part is that `dummy` never moves: I walk with a separate cursor, or I lose the handle I need for the return.”

“You used a dummy in merge as well, but nothing is being deleted there.” “Different job, same idea. When I am *building* a list, the first node is special because there is nothing to attach it to yet, so the code fills up with ‘is this the first?’. A dummy gives the tail cursor somewhere to start, and then the first node is attached exactly like the fortieth. Delete-form and build-form are the two uses, and both are ‘a node that holds nothing, so that the first one stops being different’.”

“Show me the input that breaks the naive solution.” “`[7, 7, 7, 1, 7]` with target 7, for the leading run. `[7, 7, 7]` with target 7, because everything is deleted and the result must be `None`. `[1, 7, 7, 2]`, which catches the advance-after-delete bug rather than the head bug. And the empty list. Those four cover it, and I would run them before saying I was finished.”

“When would you *not* use a dummy?” “When the operation cannot change which node is first. Counting the nodes, finding the middle, detecting a cycle, summing the values — a dummy there is a line that buys nothing and one more thing for a reader to wonder about. The recognition rule I use is: can the first node change, or am I building a new list? If neither, no dummy.”

“How does it help in remove-Nth-from-end?” “Two ways at once. Removing the last remaining node is the head case, so it is handled for free. And because both pointers start at the dummy rather than the head, when the fast pointer reaches the last node the slow pointer is on the *predecessor* of the target instead of on the target — which is exactly where I need to be to unlink it. Without the dummy I need a gap of $n + 1$ and a separate head check, and that arithmetic is easy to get wrong under time pressure.”

A model answer

Asked: *remove all nodes with a given value, including at the head.*

“The awkward part of this problem is not the removal, it is the head. To unlink a node I need the node before it, and the head does not have one. And it is worse than a single special case, because the head might be part of a *run* — on `[7, 7, 7, 1]` with target seven, the first three all have to go, so handling the head with an `if` is not enough; it would have to be a `while`. Then there is a second case when every node is deleted and the answer is an empty list.

So instead I put a dummy node in front. One line: `dummy = Node(0, head)`. The value is never read. What it buys me is that the real first node now has a predecessor — `dummy` — so it is an ordinary node and the single loop that handles the middle handles the front as well. The number of head special cases goes from two to zero, which is better than making them shorter.

Two rules come with it and I would state both before writing.

First, I return `dummy.next`, never `head`. The original head may have been deleted, and `head` still refers to it, so returning it hands back the list with the removed elements still at the front — no exception, just a wrong answer. I write the `return dummy.next` line at the same moment I write the `dummy`, before the loop, so I cannot forget.

Second, and this one is not about the dummy at all: after unlinking a node I must **not** advance the cursor. `previous.next` is now a node I have not inspected, so advancing skips it and two consecutive matches leave one behind — on `[1, 7, 7, 2]` you get `[1, 7, 2]`. So the advance happens only in the branch where I keep a node.

The loop is then four lines: while `previous.next` is not None, if its value matches, unlink and stay; otherwise advance.

Complexity is $O(n)$ time — one comparison per node, one pass — and $O(1)$ extra space: exactly one dummy node, created once before the loop, not one per iteration.

The inputs I would run before calling it done: `[7, 7, 7, 1, 7]` for the leading run, `[7, 7, 7]` where everything goes and the answer is empty, `[1, 7, 7, 2]` for the advance bug, `[1, 2, 3]` where nothing matches, and the empty list.

And it is worth saying that the same trick has a second form. When I am *building* a list rather than pruning one — merging two sorted lists, say — the first node is special for the mirror reason: there is nothing to attach it to. A dummy gives the tail cursor somewhere to start, so the first node is attached exactly like every other one, and I return `dummy.next` again. Half the problems in this phase are one of those two shapes.”

9 Recall card

- **One line deletes the head special case:** `dummy = Node(0, head)`. The real first node now has a predecessor, so it is an ordinary node and **one loop handles everything**. The head case is not just an `if` — on `[7, 7, 7, 1]` it is a *run*, plus a second case when everything is deleted. The dummy takes the count of special cases to **zero**, not to fewer.
- `return dummy.next`, **never** `return head`. The original head may have been deleted, and returning it is a **silent wrong answer** — `remove_elements([7,7,1], 7)` gives back `[7,7,1]`. Write the return line at the same moment you write the dummy. And **dummy must never move** — walk with a separate cursor.
- **The second rule has nothing to do with the dummy: advance ONLY when you keep.** After unlinking, `previous.next` is a node you have not inspected, so an unconditional advance leaves consecutive matches behind — `[1, 7, 7, 2]` becomes `[1, 7, 2]`.
- **Two forms, one idea — a node that holds nothing so the first one stops being different.** The **guard** form is `Node(0, head)`, for pruning. The **builder** form is `Node(0)` plus a moving `tail`, for merging, partitioning and copying — and any list you build out of existing nodes must be **terminated explicitly** (`more.next = None`) or you construct a cycle and the print never returns.
- **Cost: one node, O(1) space, no complexity change — it removes branches, and branches are where the bugs are.** Recognition rule: **use a dummy when the operation can change which node is first, or when you are building a new list.** Not for counting, finding the middle, or cycle detection. Test inputs that matter: `[7,7,7,1,7]` · `[7,7,7]` · `[1,7,7,2]` · `[]`.

1 What this is, and why they ask it

An ATM is a box with a card reader, a keypad, a screen, four cash cassettes and a network connection. The design has three parts: a **state machine** for what the machine is doing, a **dispenser** that turns an amount into a set of notes, and a **protocol** for the moment when the money leaves the machine and the account is debited — which are two separate events that can fail between.

The first two are ordinary work. The third is the question. “*The network dies after the bank has debited the account and before the notes come out*” has a right answer, and it is not “use a transaction”. You cannot roll back cash that has already been counted into somebody’s hand, and you cannot hold a database transaction open across a physical motor.

They ask it because it is the cleanest real example of a distributed failure that a beginner can hold in their head. Everyone understands the stakes. And it separates candidates instantly: one group says “wrap it in a transaction”, and the other says “decide which side takes the risk, log every step locally, and reconcile”. Expect it at Amazon, at every fintech, and in any round that wants to see whether you think about failure before you are asked to.

2 The story

The milk booth by the park opens at six and Anand has been going for years.

It works in two halves. You pay at the first window, where the man gives you a small brass token with a number stamped on it. You carry the token four steps to the second window, hand it over, and the other man fills your can from the tank.

Two windows, because one man cannot both take money and pour milk without a queue forming halfway down the road.

One Tuesday in November the tank ran dry at ten past seven, with four people still holding tokens.

Anand was the third of the four. And what he remembers is that nothing dramatic happened. The man at the second window did not shrug, and he did not go and argue with the first window. He asked for the four tokens back, said the numbers out loud to himself as he took them, and put them in a separate tin on the shelf rather than back in the drawer with the used ones.

Then he told the four of them to come back at eleven.

At eleven the two men do the accounts, which they do every day whatever happens. The money in the box at the first window has to match the milk gone from the tank. On a normal day it does. That Tuesday it did not, and the difference was exactly four tokens' worth — which is why the four tokens went in their own tin instead of into the drawer. The men counted the tin, matched the numbers, and paid the four of them back from the box.

Anand asked once why they do not simply take the money at the second window, when the milk is actually in the can. The man said two things. First, that the queue would be twice as slow, because counting change while pouring is how you spill. And second — and he was quite certain about this — that people who have already paid come back, and people who have not paid do not. If the milk goes out first and something goes wrong, he is chasing somebody down the road. This way the money is already in the box and the worst case is that he owes it, which he can settle at eleven.

3 The idea in plain English

The two windows are the two halves of a withdrawal: **the account is debited** and **the notes are dispensed**. They are separate acts, and something can happen in between. The milk booth's answer — take the money first, keep a record of what was owed, and settle it at the daily count — is very close to what real ATMs do.

The three parts of the design

One: the state machine. An ATM is doing exactly one thing at a time and the legal moves are strictly limited. You cannot dispense before authenticating. You cannot accept a new card while the previous one is still in the slot. This is the State pattern from [day 073](#), and it is the right shape here because each state genuinely carries different behaviour.

Two: the dispenser. Given ₹3,700, decide which notes to hand out from the cassettes you actually have. This looks like a triviality and is not, because the cassettes run out unevenly.

Three: the failure protocol. The part the question is really about.

The state machine

```
IDLE → CARD_INSERTED → AUTHENTICATED → AMOUNT_ENTERED  
      → AUTHORISING (talking to the bank) → DISPENSING → RETURNING_CARD → IDLE
```

Two properties are worth stating out loud.

Every path returns the card. Failure at any state must lead back to a state that ejects the card before going idle, or the machine eats cards — which is the single most common real ATM complaint.

DISPENSING cannot be interrupted. Once the motor starts counting notes, there is no cancellation. That is a physical fact and the design has to respect it: the last cancellable moment is *before* the dispense begins, and the interface should stop offering cancel after that.

The dispenser, and why greedy is not obviously right

Denominations in India are ₹100, ₹200, ₹500 and ₹2000. The obvious algorithm is greedy: take as many of the largest note as you can, then the next, and so on.

For a *canonical* currency system — and every real one is — greedy gives the fewest notes when supply is unlimited. But **supply is not unlimited**, and that is what breaks it:

```
amount wanted: ₹600
cassettes:      ₹500 × 1 remaining, ₹200 × 3 remaining

greedy:  take the ₹500. Now ₹100 remains and there are no ₹100 notes.
          FAIL — and it fails after having already committed a note.
correct: ₹200 × 3.
```

So the real algorithm is a small **bounded** problem: minimise notes subject to the count available in each cassette. With four denominations and a limit of, say, forty notes, a plain search over the counts is tiny — a few hundred combinations — and you can also do it as dynamic programming over the amount in ₹100 units, which is 400 cells for ₹40,000.

The practical answer to give: **try greedy first because it is right almost always and costs nothing; fall back to the bounded search when greedy cannot complete.** Then say the number: greedy fails only when a cassette is nearly empty, which is a few percent of transactions late in a replenishment cycle.

And one more rule that is not an algorithm: **decide the notes before you dispense any of them.** If you dispense as you go and discover at note four that you cannot finish, you have handed out ₹1,500 of a ₹1,700 withdrawal and there is no way to take it back.

The failure protocol, which is the question

Three things happen, in order, and any of them can fail:

1. the bank authorises and DEBITS the account
2. the ATM DISPENSES the notes
3. the ATM CONFIRMS to the bank that the cash was actually handed over

Why debit first? The same reason as the milk booth: **if the money leaves the machine and the debit has not happened, the bank is chasing a person. If the debit has happened and the money did not leave, the bank owes a person.** The second failure is recoverable by an internal process; the first is not. So the design deliberately puts the risk on the side the bank can fix.

Now the failure cases, and each has a different answer:

FAILURE	WHAT HAPPENED	WHAT MUST HAPPEN
Network dies before the debit	Nothing. No debit, no cash.	Show an error, return the card.
Network dies after the debit, before dispensing	Money is gone from the account; no cash.	ATM logs a <i>failed</i> transaction locally; the bank auto-reverses when no confirmation arrives.
Notes jam during dispensing	Partial cash out.	Log exactly how many notes were purged versus presented; reverse the difference.
Confirmation lost after a good dispense	Cash out, bank does not know it completed.	The bank must not reverse — this is why the ATM's local journal is the source of truth at reconciliation.
Customer walks away without taking the notes	Cash presented, not taken.	Retract after ~30 s, log it, reverse the debit.

The two mechanisms that make all of that work:

The electronic journal. Every ATM writes every step to local storage before doing it, and the journal survives a power cut. It records the transaction id, the amount authorised, the notes counted, and whether they were presented and taken. This is the tin on the shelf.

Auto-reversal and reconciliation. If the bank debits and never receives a confirmation, it issues a reversal — typically after a timeout of minutes, and definitively at end-of-day reconciliation, which is the eleven o'clock count. In India this is why a failed ATM withdrawal is credited back within a fixed number of working days, and why the rules mandate compensation if it is not.

Idempotency. Every transaction carries a unique id — in the ISO 8583 protocol that ATMs speak, the **system trace audit number**, or STAN. If the ATM retries because it did not hear an answer, the bank must recognise the retry and not debit twice. This is the same idempotency-key idea as [day 018's](#) API design, and it is what makes retrying safe at all.

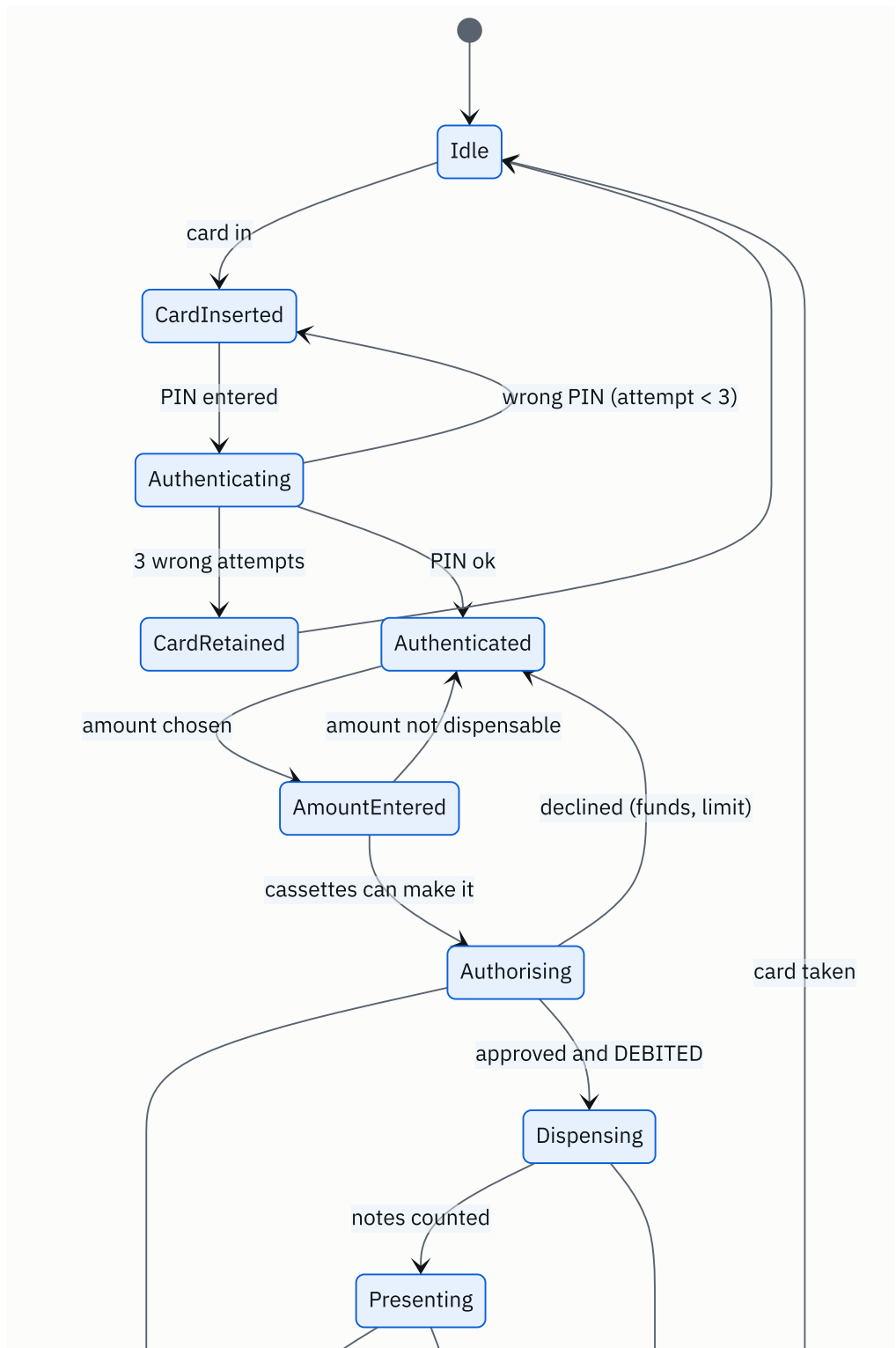
What the ATM must never do

Never store or verify the PIN. The keypad is an encrypting device; the PIN is encrypted at the keypad into a PIN block and verified by a hardware security module at the bank. The ATM software never sees it in the clear. Saying this unprompted is a strong signal, and getting it wrong the other way — “the ATM checks the PIN against the card” — is a serious mark against you.

Never hold a lock across the dispense. The motor takes seconds. Nothing on the bank’s side may be waiting on it.

4 The picture

The state machine, with the failure paths drawn, because those are the design:



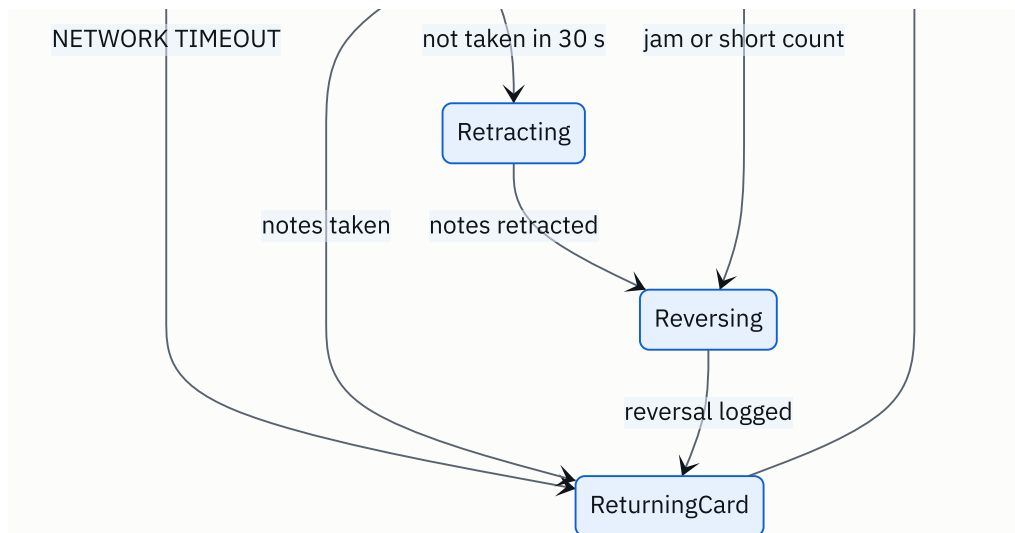


FIGURE 80.1

What to notice: **every state has a path back to ReturningCard**. A machine that can reach a dead end eats the card, and that is the failure customers actually experience. Also notice that **Dispensing** has no cancel arrow — once the motor runs, the only exits are “counted” and “jammed”.

The physical layout, so the cassette problem is visible:

CASSETTES		notes remaining	
+-----+	₹2000	0	(removed; not loaded since 2018)
+-----+	₹500	2,500	
+-----+	₹200	2,500	
+-----+	₹100	2,500	
+-----+	REJECT bin	14	(notes the counter refused)
+-----+	RETRACT bin	3	(presented but not taken)

request ₹3,700 → greedy: $500 \times 7 + 200 \times 1 = 8$ notes
 request ₹600 with only ₹500×1 and ₹200×3 left:
 greedy takes the ₹500, then needs ₹100 and cannot → FAIL
 correct answer is ₹200×3

What to notice: there are two bins that are not cassettes. **The reject bin and the retract bin are part of the design**, not an afterthought — they are how the machine’s cash count still balances when notes go somewhere other than a customer’s hand.

The classes:

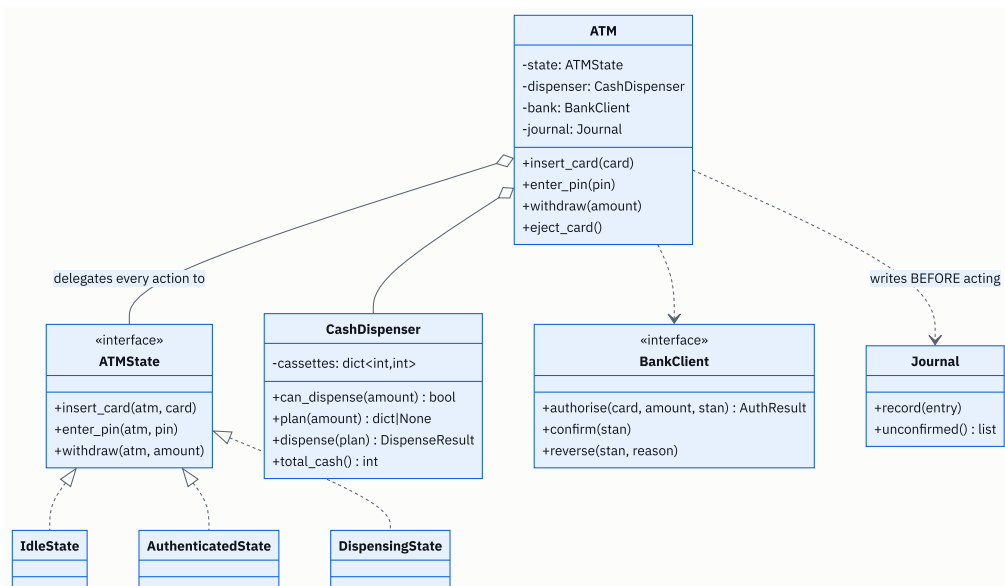


FIGURE 80.2

5 How it actually works

Move 1 · Clarify (minutes 0–5)

“Withdrawal only, or deposits and balance too?” — Withdrawal, balance, and PIN change; no deposits. **“Is the ATM online for every transaction, or does it ever operate offline?”** — Online. This matters enormously; offline ATMs need floor limits and stand-in processing. **“Which denominations, and how many cassettes?”** — ₹100, ₹200, ₹500; four cassettes. **“Am I designing the ATM software, or the bank’s side too?”** — The ATM, with the bank behind an interface.

“I will assume the card is a chip card, that PIN verification happens at the bank and never on the ATM, and that there is a daily withdrawal limit enforced by the bank. I am not designing the hardware drivers or the card network protocol itself.”

Move 2 · The nouns (minutes 5–10)

- **ATM** — the machine. Holds the current state and delegates everything to it.
- **ATMState** (*interface*) — one class per state; each implements only the actions legal in it.
- **Card** — the number and the account it maps to. No PIN, ever.
- **CashDispenser** — owns the cassettes; plans and dispenses.

- `BankClient` (*interface*) — authorise, confirm, reverse. Behind an interface because it must be faked in tests and because failure injection is the whole point.
- `Journal` — an append-only local record, written *before* each irreversible act.
- `Transaction` — a frozen record with a unique id, the amount and the outcome.

Seven, two interfaces. The gate as always: can you name a second implementation? For `BankClient`, yes — the real one and a simulator that times out on demand, which is not a hypothetical because you cannot test this design without it.

Move 3 · The state machine

```
class ATMState:
    """Every action refuses by default; each state enables only what it allows."""
    name: str

    def insert_card(self, atm: "ATM", card: Card) → None:
        self._refuse("insert a card")

    def enter_pin(self, atm: "ATM", pin: str) → None:
        self._refuse("enter a PIN")

    def withdraw(self, atm: "ATM", amount: int) → None:
        self._refuse("withdraw")

    def _refuse(self, action: str) → None:
        raise IllegalOperation(f"cannot {action} while {self.name}")
```

Refusal by default, exactly as on [day 073](#). A new state written in a hurry is maximally restrictive until somebody deliberately opens it up — which in a machine that hands out money is the only acceptable direction to fail in.

```
class AuthenticatedState(ATMState):
    name = "authenticated"

    def withdraw(self, atm: "ATM", amount: int) → None:
        if amount % 100 != 0:
            raise InvalidAmount("amount must be a multiple of 100")
        plan = atm.dispenser.plan(amount)          # decide the notes FIRST
        if plan is None:
            raise CannotDispense("this machine cannot make that amount")
        atm.state = AuthorisingState(amount, plan)
        atm.run_authorisation()
```

Two guards before anything irreversible: the amount must be dispensable *by this machine's current cassettes*, and the plan is computed **before** the bank is asked. Asking the bank first and then discovering you cannot make the amount means a debit you have to reverse for no reason.

Move 4 · The dispenser

```
class CashDispenser:
    def __init__(self, cassettes: dict[int, int]) → None:
        self._cassettes = dict(cassettes)          # denomination → notes remaining

    def plan(self, amount: int) → dict[int, int] | None:
        """Choose the notes. Greedy first; bounded search when greedy cannot finish."""
        greedy = self._greedy(amount)
        if greedy is not None:
            return greedy
        return self._search(amount)
```

Two algorithms, and the reason for both said in one line of docstring. Greedy is right almost always and costs nothing; the search exists for the end of a replenishment cycle.

```
def _greedy(self, amount: int) → dict[int, int] | None:
    plan: dict[int, int] = {}
    remaining = amount
    for note in sorted(self._cassettes, reverse=True):
        take = min(remaining // note, self._cassettes[note])
        if take:
            plan[note] = take
            remaining -= note * take
    return plan if remaining == 0 else None        # None means "greedy could not"
```

```
def _search(self, amount: int) → dict[int, int] | None:
    """Fewest notes, subject to the count available. Tiny: 3-4 denominations."""
    notes = sorted(self._cassettes, reverse=True)

    def best(index: int, remaining: int) → dict[int, int] | None:
        if remaining == 0:
            return {}
        if index == len(notes):
            return None
        note = notes[index]
        for take in range(min(remaining // note, self._cassettes[note]), -1, -1):
            rest = best(index + 1, remaining - note * take)
            if rest is not None:
                return {note: take, **rest} if take else rest
        return None

    return best(0, amount)
```

Exhaustive, and small enough not to care: four denominations and at most forty notes each is a few thousand combinations in the worst case, microseconds. **Say the size before someone asks whether it is expensive.**

```
def dispense(self, plan: dict[int, int]) → None:
    for note, count in plan.items():
        if self._cassettes[note] < count:
            raise CassetteEmpty(note)              # someone else drained it: abort
    for note, count in plan.items():
        self._cassettes[note] -= count              # commit only after all checks
```

Check everything, then commit everything. Half-dispensing is the failure that cannot be undone, so the code is written so it cannot happen.

Move 5 · The withdrawal protocol – the actual answer

```
def run_withdrawal(self, amount: int, plan: dict[int, int]) → None:
    stan = new_stan()                # unique id; makes retries safe
    self.journal.record("AUTH_REQUEST", stan=stan, amount=amount)

    try:
        result = self.bank.authorise(self.card, amount, stan)
    except NetworkTimeout:
        # We do not know whether the bank debited. Assume it might have.
        self.journal.record("AUTH_UNKNOWN", stan=stan, amount=amount)
        self.bank.reverse_later(stan, reason="no response")    # queued, retried
        self.state = ReturningCardState("Unable to complete. Please try again.")
    return
```

The comment is the answer to the interview question. **On a timeout you do not know whether the debit happened**, so you must behave as if it might have: write the unknown to the journal and queue a reversal that will be retried until the bank acknowledges it. Silence is not “no”.

```
if not result.approved:
    self.journal.record("DECLINED", stan=stan, reason=result.reason)
    self.state = AuthenticatedState()    # let them try a smaller amount
    return

# The account is now debited. From here on, the bank is owed an answer.
self.journal.record("DISPENSE_START", stan=stan, plan=plan)
try:
    self.dispenser.dispense(plan)
except DispenserFault as fault:
    self.journal.record("DISPENSE_FAILED", stan=stan, error=str(fault))
    self.bank.reverse(stan, reason="dispenser fault")
    self.state = OutOfServiceState()
    return

self.journal.record("DISPENSED", stan=stan, plan=plan)
self.bank.confirm(stan)                # may fail; the journal is the truth
self.state = PresentingState(plan)
```

Three journal writes around one physical act. Each is written **before** the act it describes where possible, so that a power cut at any instant leaves a record that reconciliation can interpret.

And the last line matters: `confirm` may itself fail, and that is *tolerable*, because the journal records that the notes were dispensed and the daily reconciliation will match it against the bank’s record. **The journal, not the network call, is the source of truth.**

Real systems

- **ISO 8583** is the message format ATMs and card networks actually speak, and the **STAN** (system trace audit number) is its idempotency key. Message type `0200` is a request, `0210` a response, `0420` a reversal advice.
- **NCR and Diebold Nixdorf** make most of the world's ATMs; the software layer is usually **CEN/XFS**, a standard interface to the card reader, cassettes and keypad — the reason ATM software is not written against motors directly.
- **The encrypting PIN pad and the HSM.** The PIN never exists in the clear outside the keypad and the bank's hardware security module. This is a certification requirement, not a design choice.
- **NPCI's rules in India** require a failed ATM withdrawal to be auto-reversed within a fixed number of working days, with compensation per day beyond it. That regulation exists precisely because this failure mode is common enough to need one.
- **The electronic journal** is a real, named component in every ATM, and it is the first thing a dispute investigation reads.

6 The numbers

Cash, and how long it lasts

```
4 cassettes × 2,500 notes = 10,000 notes
loaded as: ₹500 × 5,000 = ₹25,00,000
           ₹200 × 2,500 = ₹5,00,000
           ₹100 × 2,500 = ₹2,50,000
-----
total capacity                ₹32,50,000
```

```
200 withdrawals a day × ₹3,000 average = ₹6,00,000 a day
replenishment interval: 32,50,000 / 6,00,000 ≈ 5.4 days
```

A cash-out every five days. That is the number that decides the cash-in-transit schedule, and it is why the machine must report per-cassette levels rather than a total — the ₹100 cassette empties first in proportion and takes the machine out of service for odd amounts long before the money runs out.

Notes per transaction, which is why the denomination mix matters

```
₹3,000 from {500, 200, 100}: 500×6           = 6 notes
₹3,700:                        500×7 + 200×1     = 8 notes
₹3,000 with no ₹500 left:      200×15           = 15 notes
```

Dispensing runs at roughly 4 notes a second, so 8 notes is 2 seconds and 15 notes is nearly 4. Worse, most machines cap a single dispense at 40 notes. **When the ₹500 cassette empties, the maximum withdrawal effectively drops** — from ₹20,000 to ₹8,000 in the mix above. That is a real operational consequence of a design detail, and it is a good thing to notice out loud.

The failure arithmetic

```
network timeout rate: 0.5% of transactions
200 transactions/day/ATM × 0.5% = 1 incomplete transaction per ATM per day
a bank with 5,000 ATMs:         = 5,000 reversals per day
                                = 1.8 million a year
```

Five thousand a day. That is why auto-reversal has to be an automated pipeline with a queue and retries, not a support ticket. If you build the “happy path plus a phone number”, you have designed a call centre.

And the cost of getting the *order* wrong:

```
dispense-then-debit, with the same 0.5% failure rate:
  5,000 cases a day where cash left and no debit was recorded
  × ₹3,000 average = ₹1.5 crore a day at risk, unrecoverable
debit-then-dispense:
  5,000 cases a day where the customer is owed money
  → automatically reversed, cost is the float and the goodwill
```

₹1.5 crore a day of unrecoverable loss against a reversal pipeline. That single comparison is the whole justification for debiting first, and it is far more convincing than “the bank should take the risk”.

Reconciliation

```
end of day:
  notes counted out of cassettes      (physical count)
  + reject bin + retract bin
  = notes loaded at replenishment     (must balance)

  journal DISPENSED entries × amounts
  = bank's settled transactions       (must balance)

a mismatch of one transaction:  investigate the journal by STAN
```

The dispenser algorithm's cost

```
greedy:          4 denominations, one pass          ~1 μs
bounded search:  worst case ~40^4 = 2.5M combinations, pruned to a few thousand
                  in practice                        < 1 ms
frequency of falling back to search: a few percent, late in the cycle
```

Under a millisecond, a few percent of the time, on a machine whose dispense takes two seconds. **The algorithm is free**, so choose the correct one rather than the fast one — and say that, because it is the right way to weigh an optimisation.

7 The trade-offs

What this design gives up

The customer can be owed money for days. Debit-then-dispense means a timeout leaves the account short until the reversal lands. That is a deliberate choice — the alternative is unrecoverable loss — but it is a genuine cost borne by the customer, and it is why the reversal timer and the regulator's compensation rule exist. Say this plainly; pretending the failure is invisible is worse than owning it.

The journal is a single local file on a machine in a wall. If the ATM's storage fails at the same moment as the network, the evidence is gone and the dispute becomes the bank's word against the customer's. Real machines mirror the journal to the host as soon as connectivity returns, and the gap between “written locally” and “mirrored” is a window of real exposure.

Cash count can drift. Notes stick together, the counter miscounts, a note is torn. That is what the reject bin is for, and it is why the physical count at replenishment is authoritative rather than the software's counter. A design that trusts its own counter absolutely will eventually be wrong by one note and unable to explain it.

Everything is serialised on one customer. One card at a time, one transaction at a time. That is correct here and it is worth noticing that it makes the concurrency question almost trivial *inside* the machine — the concurrency that matters is between the ATM and the bank, and between two ATMs used by the same account at the same moment, which is the bank's problem and is solved by the daily limit and by the account's own locking.

Retries need care in exactly one place. `authorise` may be retried on timeout only because the STAN makes it idempotent. `dispense` must **never** be retried automatically, because the physical act may have partially happened. Distinguishing which operations are safe to retry is the general lesson, and getting it backwards here hands out money twice.

“I would change this design if...”

- **...the ATM must work offline.** Then it needs a floor limit — approve small amounts locally without the bank, up to a cap, and settle later — plus a hot-card list downloaded in advance. That is a different risk model and materially more design.

- **...it accepts deposits.** Cash deposits need a note validator, an escrow area where the notes wait until the customer confirms, and a rule for what happens to the escrow on a power cut. That is another whole state machine.
- **...the same account can be used at two ATMs at once.** Already possible, and the answer is not in the ATM: the daily limit and the account balance are enforced by the bank, which is the only place that sees both machines.
- **...notes could be dispensed before the debit.** Only for a trusted, offline, low-value case, and I would want the ₹1.5 crore number in front of whoever asked for it.

The honest concession

There is no way to make this atomic and there never will be. A database transaction cannot span a motor that pushes paper into a person's hand. So the design does not try: it **picks which side takes the risk**, makes that side the recoverable one, writes down every step before doing it, and settles the difference at a count. That is not a workaround for a missing transaction — it is what a transaction is, implemented in the only way available when one of the participants is physical.

8 In the interview

How it gets asked

- The standard: *“Design an ATM.”* Then, within five minutes: *“What happens if the network dies in the middle of a withdrawal?”*
- The algorithm probe: *“How do you decide which notes to give out?”* — they want you to notice that greedy breaks when a cassette is empty.
- The security probe: *“Where is the PIN checked?”* — a one-line question with a serious wrong answer.
- The state probe: *“What happens if someone yanks the card out halfway?”*
- The scale probe: *“How many of these failures do you expect, and what do you build for them?”*

The timed script

Minutes 0–5 · Clarify. Withdrawal only? Always online? Which denominations and how many cassettes? ATM software or the bank too? State the assumptions, especially that the PIN is verified at the bank.

Minutes 5–10 · Estimation. Cassette capacity, transactions a day, replenishment interval, and — most importantly — the failure rate arithmetic: 0.5 percent of 200 a day is one per ATM per day, five thousand a day for a bank. Say that early, because it justifies building a reversal pipeline rather than an error message.

Minutes 10–18 · The state machine. Draw it. Point at two things: every path returns the card, and `Dispensing` has no cancel.

Minutes 18–25 · The classes. Seven, two interfaces, and say why `BankClient` is one — you cannot test this design without a fake that times out on demand.

Minutes 25–35 · The deep dive: the failure protocol. The three steps, why debit comes first with the crore-a-day comparison, the five failure cases with a different answer each, the journal, the STAN, and auto-reversal.

Minutes 35–40 · The dispenser. Greedy, the counterexample where a cassette is empty, the bounded search, and the rule that the plan is decided before any note moves.

The follow-ups

“The network dies after the bank has debited, before the cash comes out. What happens?” “On a timeout I do not know whether the debit happened, so I have to assume it might have. The ATM writes an ‘authorisation outcome unknown’ entry to its local journal, queues a reversal keyed by the transaction id, and returns the card with a message. The reversal is retried until the bank acknowledges it, and independently the bank auto-reverses any authorisation it never receives a confirmation for. At end of day, reconciliation matches the journal against the bank’s records and the physical cash count, and anything still unmatched is investigated by transaction id. The customer is temporarily out of pocket, which is a real cost, and it is the deliberate trade — the other ordering loses the money permanently.”

“Why not dispense first and debit afterwards?” “Because the two failures are not symmetric. If cash leaves and the debit fails, the bank is chasing a person who already has the money — unrecoverable. If the debit lands and the cash does not, the bank owes a person, and that is fixable by an automated reversal. At a 0.5 percent failure rate, 200 transactions a day and 5,000 ATMs, that is 5,000 cases a day either way — about ₹1.5 crore a day unrecoverable under the wrong ordering, against a reversal pipeline under the right one.”

“How do you decide which notes to dispense?” “Greedy from the largest denomination down, which for a canonical currency gives the fewest notes when supply is unlimited. But supply is not unlimited, and greedy fails when a cassette runs low — asking for ₹600 with one ₹500 note and three ₹200s left, greedy takes the ₹500 and then cannot make ₹100, while the answer is three ₹200s. So greedy first, and a small bounded search over the available counts when greedy cannot complete. With four denominations that search is under a millisecond, on a machine whose dispense takes two seconds, so it costs nothing. The rule that matters more than the algorithm: compute the whole plan before dispensing any note, because a half-completed dispense cannot be undone.”

“Where is the PIN verified?” “Never on the ATM. The keypad is an encrypting device — it turns the PIN into an encrypted PIN block in hardware — and verification happens in a hardware security module at the bank. The ATM software has no access to the PIN in the clear and stores nothing. That is a certification requirement rather than a design preference. What the ATM *does* track is the attempt count, so it can retain the card after three failures.”

“Someone pulls their card out halfway through.” “Physically the card is held by the reader during a transaction, so that is largely prevented in hardware. Logically, any interruption before the authorisation is harmless — cancel and eject. After the debit, it changes nothing about the protocol: the transaction completes or reverses on its own schedule regardless of where the card is. The case I would actually design for is the customer walking away *after* the notes are presented: the machine retracts them after about thirty seconds into a retract bin, logs it, and reverses the debit. The retract bin exists so that the cash count still balances when notes end up somewhere other than a hand.”

“How would you test this?” “That is the reason `BankClient` is an interface. I would write a fake that can be told to time out, to decline, to approve and then lose the confirmation, and to approve twice for the same transaction id. Then a test per row of the failure table, asserting the journal contents and the cassette counts afterwards. The dispenser gets its own tests with cassettes deliberately drained, including the ₹600 case where greedy fails. Testing the happy path here is nearly worthless — every interesting behaviour is a failure path.”

“What if the same account is used at two ATMs at the same moment?” “The ATM cannot solve that, and it should not try — it has no visibility of the other machine. The bank enforces the balance and the daily limit, and it is the only participant that sees both authorisations. What the ATM must do is make its own request idempotent with a transaction id, so its own retries never turn into two debits.”

A model answer

Asked: *design an ATM. What happens if the network dies mid-withdrawal?*

“Let me split this into three parts, because the third is the interesting one. There is a state machine for what the machine is doing, a dispenser that turns an amount into notes, and a protocol for the moment when the account is debited and the cash comes out — which are two separate events with a gap between them.

Before that, one estimate, because it changes what I build. Say 200 withdrawals a day per machine and a network failure rate of half a percent. That is one incomplete transaction per ATM per day, and for a bank with five thousand ATMs, five thousand a day — nearly two million a year. So the failure path is not an edge case with a phone number attached; it is a pipeline with a queue and retries, and I will design it as one.

The state machine first, quickly. Idle, card inserted, authenticating, authenticated, amount entered, authorising, dispensing, presenting, returning card. I would implement it as one class per state with a base that refuses every action, so each state enables only what it allows — a machine that hands out money should fail closed. Two properties I would point at on the diagram: every path has a route back to returning the card, because a machine that can reach a dead end eats cards; and dispensing has no cancel transition, because once the motor is counting notes there is no taking them back.

Now the question. Three things happen in order: the bank authorises and debits, the ATM dispenses, and the ATM confirms that the cash actually went out.

Why that order? Because the two failures are not symmetric. If cash leaves and the debit has not happened, the bank is chasing a person who already has the money, and that is unrecoverable. If the debit happens and the cash does not come out, the bank owes a person, and that is fixable by an automated reversal. At five thousand failures a day and three thousand rupees average, that is about a crore and a half a day of unrecoverable loss under the wrong ordering. So the design deliberately puts the risk on the side that can be fixed — which is exactly what a shop does when it takes your money at one window and gives you the goods at another.

So: on a network timeout during authorisation, I do not know whether the debit happened. Silence is not ‘no’. The ATM writes an ‘outcome unknown’ entry to its local electronic journal, queues a reversal keyed by the transaction id, returns the card and shows an error. The reversal is retried until the bank acknowledges it; independently, the bank auto-reverses any authorisation it never receives a confirmation for. Every step is written to the journal *before* it is attempted, so a power cut at any instant leaves something reconciliation can interpret.

Two mechanisms make that safe. Every transaction carries a unique id — in the ISO 8583 messages ATMs actually speak that is the system trace audit number — so a retry is recognised as a retry and not a second debit. And at end of day, reconciliation matches three things: the physical notes counted out of the cassettes plus the reject and retract bins, the journal's dispensed entries, and the bank's settled transactions. Anything that does not match is investigated by transaction id.

One thing that must never be automatically retried is the dispense itself, because the physical act may have partially happened. Authorise is idempotent and safe to retry; dispense is not. Knowing which operations are safe to retry is the general version of this whole answer.

On the dispenser: greedy from the largest note down is right almost always, and it breaks when a cassette runs low — six hundred rupees with one five-hundred note and three two-hundreds left, greedy takes the five hundred and then cannot make a hundred, while the answer is three two-hundreds. So greedy first and a small bounded search over the available counts as a fallback; with four denominations that is well under a millisecond on a machine whose dispense takes two seconds. And the rule that matters more than the algorithm: decide the entire plan before moving a single note, because a half-finished dispense cannot be undone.

Last thing, unprompted: the PIN is never verified on the ATM. The keypad encrypts it in hardware and a security module at the bank checks it. The ATM tracks the attempt count so it can retain the card after three failures, and that is all it knows about the PIN.”

9 Recall card

- **Three parts: a state machine, a dispenser, and a failure protocol — and only the third is the question.** State machine with **refusal by default** (fail closed), **every path returns the card**, and **Dispensing** has no cancel because a motor cannot be rolled back.
- **Debit first, then dispense, then confirm — and the reason is asymmetry, not convention.** Cash out with no debit is *unrecoverable*; a debit with no cash is *reversible*. At 0.5% of 200/day across 5,000 ATMs that is **5,000 failures a day** either way — about **₹1.5 crore a day unrecoverable** under the wrong ordering, against an automated reversal pipeline under the right one.
- **On a timeout you do not know whether the debit happened — silence is not “no”.** Write `AUTH_UNKNOWN` to the **electronic journal** *before* acting, queue a **reversal**, return the card. Every transaction carries a unique id (**STAN**, in ISO 8583) so retries

are idempotent. **authorise** is safe to retry; **dispense** never is. The journal, not the network call, is the source of truth at reconciliation.

- **Greedy note selection breaks when a cassette runs low:** ₹600 with ₹500×1 and ₹200×3 — greedy takes the ₹500 and strands ₹100; the answer is ₹200×3. **Greedy first, bounded search as fallback** (<1 ms, a few percent of the time). **Decide the whole plan before moving one note** — a half-dispense cannot be undone. The **reject and retract bins** exist so the cash count still balances.
- **The PIN is never seen by the ATM** — encrypting keypad, verified by an HSM at the bank; the ATM tracks only the attempt count. **Nothing here is atomic and nothing can be:** a transaction cannot span a motor. The design *picks which side takes the risk*, **writes every step before doing it**, and **settles at a daily count** — capacity ₹32.5 lakh, ~200 withdrawals/day, replenished every **~5.4 days**.

DSA topic: The dummy head trick **System design topic:** Design an ATM

Code these, in this order

One rule for the whole set: **the moment you type** `dummy = Node(0, head)`, **type** `return dummy.next` **as well** — before the loop, before anything. Half the bugs in this technique are a `return head` that was written on autopilot at the end.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Remove Linked List Elements	LeetCode 203 (Easy)	The guard form. <code>[7,7,7,1,7]</code> for the leading run, <code>[1,7,7,2]</code> for the advance rule.
2	Merge Two Sorted Lists	LeetCode 21 (Easy)	The builder form, and attaching the remainder wholesale instead of node by node.
3	Remove Nth Node From End of List	LeetCode 19 (Medium)	Where the dummy does real work: the head case <i>and</i> landing on the predecessor.
4	Remove Duplicates from Sorted List II	LeetCode 82 (Medium)	A head that is part of a duplicate run, so no <code>if</code> can save you.

On problem 1, run the four inputs before you submit

`[7,7,7,1,7]`, `[7,7,7]`, `[1,7,7,2]`, `[]`. The first tests the leading run, the second tests returning an empty list, the third tests the advance rule and has nothing to do with the head, and the fourth is the one people skip. Any solution that passes all four is almost certainly right.

On problem 2, attach the remainder in one line

When one list runs out, do not keep looping. `tail.next = a if a is not None else b` attaches the whole remaining chain in one assignment, because it is already sorted and already linked. Say out loud why that is correct.

On problem 3, do it without the dummy first

Write the version with a gap of `n + 1` and an explicit head check. Count the lines and the branches. Then write the dummy version and count again. Say what the dummy did *twice* here.

On problem 4, find the input that beats an `if`

Construct an input where the head, the second node and the third node are all part of the same duplicate run. Then say why a single `if` on the head cannot handle it and a `while` on the head is still worse than a dummy.

The two-forms drill

1. Write the guard form and say which constructor argument matters.
2. Write the builder form and say which cursor exists only because of it.
3. Say the single sentence that describes what both forms have in common.
4. Give three problems that need the guard form and three that need the builder form.
5. Give two problems where a dummy would be pure noise, and say what they have in common.

The return drill

1. Write `remove_elements` returning `head` instead of `dummy.next`.
2. Run it on `[7, 7, 1]` with target 7 and write down what comes back.
3. Explain, in terms of what `head` refers to, why nothing raised.
4. Say the habit that prevents it.

The advance drill

1. Write the loop that advances `previous` in both branches.
2. Run it on `[1, 7, 7, 2]` and write down the output.
3. Explain what `previous.next` is immediately after an unlink.
4. Say why this bug survives adding a dummy node.
5. Construct the shortest input that exposes it.

The break-it drill

Trigger each and record the exact output or error text:

1. `return head` instead of `return dummy.next`, on `[7, 7, 1]`.
2. Advance `previous` unconditionally, on `[1, 7, 7, 2]`.
3. Walk `dummy` itself instead of a separate cursor. Print the result.
4. Create the dummy inside the loop. Say what it costs and why the output is still right.
5. Write `dummy = Node(0)` when you meant `Node(0, head)`. Run any input.
6. In `partition`, omit `more.next = None`. Print the result and describe what happens.
7. In `merge_two_sorted`, use `<` instead of `≤`. Say what property you lost, and construct an input where it is visible.

The counting drill

For `remove_elements`, both versions:

1. Count the lines, the loops, the head-specific branches, and the places `head` is referenced.
2. Do the same for the dummy version.
3. Say which of the two head branches in the naive version handles `[7,7,7,1]` and which handles `[7,7,7]`.
4. State the extra space of the dummy version, precisely.

The recognition drill

For each, say whether you would use a dummy, which form, and why:

1. Remove every node equal to a value.
2. Count the nodes.
3. Merge two sorted lists.
4. Find the middle node.
5. Remove the *n*th node from the end.
6. Detect a cycle.
7. Partition around a value.
8. Reverse the list.
9. Swap every pair of adjacent nodes.
10. Sum all the values.

Four of the ten need no dummy. Say the one-sentence rule that rejects all four.

The build-it drill

From an empty file, without looking, write `remove_elements`, `merge_two_sorted`, `remove_nth_from_end` and `partition`. Then run each on: the empty list, a one-element list, a list where every element matches, and a list where none does. Any function that fails one of those four is the one to rewrite.

The state-machine drill

Draw the ATM state machine from memory, then answer:

1. Which state has no cancel transition, and why is that physical rather than a design choice?
2. Which state does every failure path have to be able to reach?
3. What happens after three wrong PIN attempts, and which state is that?

4. Name the state that exists only because a customer may walk away.
5. Why does the base state class refuse everything by default?

The ordering drill

1. Write the three steps of a withdrawal in order.
2. Say what fails, and what is lost, if the order is dispense-then-debit.
3. Compute the daily unrecoverable loss under that ordering at 5,000 ATMs, 200 transactions a day, 0.5 percent failure and ₹3,000 average.
4. Say what the customer experiences under the correct ordering, and how long it lasts.
5. Say which regulator rule exists because of exactly this failure.

The failure-table drill

For each, say what the machine does, what the bank does, and what reconciliation sees:

1. The network dies before authorisation.
2. The network dies after the debit, before dispensing.
3. The notes jam halfway through dispensing.
4. The confirmation message is lost after a successful dispense.
5. The customer does not take the presented notes.
6. The power fails between the journal write and the dispense.
7. The ATM retries an authorisation it already got an answer to.

One of the seven is the case where the bank must **not** reverse. Name it and say what stops it.

The dispenser drill

1. Write the greedy planner.
2. Find an amount and a cassette state where greedy fails but a solution exists. Prove it by hand.
3. Write the bounded search.
4. State its worst-case size for four denominations and forty notes each, and how long it actually takes.
5. Say the rule about when the plan is computed, and what goes wrong if you dispense as you go.
6. Compute the notes needed for ₹3,000 with the ₹500 cassette empty, and say what that does to the maximum withdrawal at a 40-note cap.

The estimation drill

Compute each, showing the multiplication:

1. Total cash capacity for the loadout in the lesson.
2. Days between replenishments at 200 withdrawals of ₹3,000.
3. Incomplete transactions per ATM per day, and for a bank with 5,000 ATMs.
4. Time to dispense 8 notes and 15 notes at 4 notes a second.
5. The end-of-day reconciliation identity: what three quantities must balance?

The security drill

1. Where is the PIN verified, and where is it encrypted?
2. What does the ATM store about the PIN?
3. What does the ATM track that is PIN-related, and why?
4. Say what is wrong with the answer “the ATM checks the PIN against the chip on the card”.

The testability drill

1. Say why `BankClient` is an interface, in terms of what you cannot otherwise test.
 2. List the five behaviours your fake bank must be able to produce on demand.
 3. Write the assertions for the “timeout after debit” test: what should the journal contain, what should the cassettes contain, what should be queued?
 4. Say why testing the happy path here is nearly worthless.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Remove all nodes with a given value, including at the head.* The head having no predecessor and being a *run* rather than a single case, the dummy in one line, `return dummy.next` with the silent failure named, the advance-only-when-you-keep rule stated as separate from the dummy, $O(n)$ and $O(1)$, and the four inputs you would run.
2. *Design an ATM. What happens if the network dies mid-withdrawal?* The three parts, the failure-rate estimate that justifies a pipeline, debit-then-dispense with the asymmetry and the crore-a-day comparison, timeout meaning unknown rather than no, the journal written before acting, the STAN making retries idempotent, auto-reversal and end-of-day reconciliation, and the fact that dispense must never be retried.
3. *Why not just handle the head with an `if`?* Because it is a run, not a single node, so it is a `while`; plus a second branch for the everything-deleted case; and the dummy takes the count to zero rather than making it smaller.

Before you move on

- [] I write `return dummy.next` at the same moment I write the dummy.
- [] I can state the advance-only-when-you-keep rule and why it is independent of the dummy.
- [] I ran all four inputs on `remove_elements` and know what each one tests.
- [] I returned `head` on purpose and saw the silent wrong answer.
- [] I can write both the guard form and the builder form from memory.
- [] I omitted `more.next = None` in `partition` and saw the cycle.
- [] I wrote `remove_nth_from_end` both with and without a dummy and counted the branches.
- [] I can name four problems where a dummy would be noise, and the rule that rejects them.
- [] I can draw the ATM state machine and say why `Dispensing` has no cancel.
- [] I can state the three withdrawal steps and defend the order with the loss comparison.
- [] I can say what a timeout means and what the machine does in response.
- [] I can name the one failure case where the bank must not reverse.
- [] I found a cassette state where greedy fails and a solution exists.
- [] I can say where the PIN is verified and what the ATM stores.
- [] I can list the five behaviours a fake bank must produce for testing.
- [] I answered all three questions above out loud.

Reversing a linked list

DATA STRUCTURES AND ALGORITHMS

Reversing a linked list

You can reverse a list iteratively and recursively and draw the pointer moves.

SYSTEM DESIGN

Design a vending machine

You can model the coin-and-selection state machine cleanly.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Reverse a linked list. Now do it in groups of k .*
- *Design a vending machine. Handle exact change and refunds.*

1 What this is, and why they ask it

Reversing a linked list means turning every arrow round. `1 → 2 → 3 → None` becomes `None ← 1 ← 2 ← 3`, so the old tail is the new head. No node moves, no node is created, no value is copied. You change `next` on each node exactly once.

The whole difficulty is one line long. The moment you overwrite `current.next`, **you have destroyed your only way of reaching the rest of the list** — so you must save it first. Three variables and four lines, in a fixed order, and the order is not negotiable.

They ask it constantly. It is the most-asked linked list question there is, it is asked as a warm-up before something harder, and it is asked because a candidate either has the three-pointer dance in their fingers or does not. The follow-ups are predictable and they escalate: do it recursively, reverse only the middle section, reverse in groups of k . Every one of those is this loop wrapped in something, so the ten minutes you spend making this automatic pay for the rest of the phase.

2 The story

The trip to the caves was Nalini's class, thirty-one children, and the path in was narrow enough that they had to go single file.

She lined them up the way she always does. Each child takes hold of the bag strap of the child in front. Nobody holds anybody's hand — hands get pulled away — but a bag strap you can hold for an hour. Anil at the front, holding nothing, walking beside her.

Two hundred metres in, the path stopped. There was a gate across it, chained, and a board that had been there long enough to be unreadable. The path did not go through. They had to go back out the way they had come.

There was no room to turn around as a group. The path was a metre wide with rock on one side.

So Nalini turned the line inside out, and she did it from the front, one child at a time.

She started with Anil, who was now at the wrong end. She told him: look behind you, see who is there — that is Farida — let go of nothing, because you are holding nothing, and take hold of Farida's strap. Then Farida: look behind you first, that is Kiran, then let

go of Anil's strap, then take hold of Kiran's. Then Kiran, the same. Look back, let go, take hold.

The order was the whole thing, and she said it in that order every single time, thirty-one times, because of what happened on a trip four years ago. A boy let go first and then looked behind him, and in the two seconds in between, the child behind him had stopped to look at something and there was a gap, and the boy could not see who he was supposed to take hold of. The back two-thirds of the line — nineteen children — simply stood there, holding each other, no longer attached to anything at the front. Nalini did not notice for four minutes.

So now it is always: **look behind you first. Then let go. Then take hold.**

At the end, the child who had been at the back was at the front, and she was holding nothing, which is how you know the line is finished. Nalini walked to that end and led them out.

3 The idea in plain English

Nalini's three-step instruction is the algorithm. **Look behind you, let go, take hold** — in that order, and the reason for the order is the boy who lost nineteen children.

The three variables

```
previous = None      # the child behind me, whose strap I will take
current = head       # me
next_node = None     # the child in front of me, saved before I let go
```

`previous` starts as `None` because the first node has nobody behind it, and in the reversed list the old head has nothing after it — it becomes the tail. That single `None` is doing two jobs and it is worth noticing.

The four lines, in the only order that works

```
while current is not None:
    next_node = current.next      # 1. look behind you (save the rest of the list)
    current.next = previous       # 2. let go, take hold (turn the arrow round)
    previous = current            # 3. previous moves up
    current = next_node           # 4. current moves up
return previous                  # previous is the new head
```

Line 1 is the one that matters. `current.next` is overwritten by line 2, so if you have not saved it, the entire remainder of the list is unreachable — the nineteen children standing in the path. Every other bug in this problem is small; this one loses everything.

Return `previous`, not `head`. When the loop ends, `current` is `None` and `previous` is the last node visited, which is the old tail and the new head. `head` still refers to the old first node, which is now the last one. Returning it gives you a one-element list.

Why it is $O(1)$ space

Nothing is copied and nothing is allocated. Three references, whatever the length. Contrast with the tempting alternative — read all the values into a Python list, reverse it, and rebuild — which is $O(n)$ extra space and is a legitimate answer only if you say out loud that you are trading space for simplicity.

The recursive version, and the two lines that look like magic

```
def reverse_recursive(head):
    if head is None or head.next is None:
        return head          # base case: empty, or a single node
    new_head = reverse_recursive(head.next)
    head.next.next = head     # the node after me now points AT me
    head.next = None         # and I point at nothing
    return new_head
```

The trick to reading it is to trust the recursive call, which is exactly the habit [day 087](#) will make explicit. Assume `reverse_recursive(head.next)` has already reversed everything after `head`, and returned the new head of that reversed part.

Now you are standing on `head`, and `head.next` still points at what was the second node — which is now the **last** node of the reversed part. So:

- `head.next.next = head` makes that last node point back at you. You are attached to the end.
- `head.next = None` makes you the new tail.
- `new_head` is unchanged all the way up, because the new head is the old tail and it was found at the very bottom of the recursion.

Do not skip `head.next = None`. Without it, the old head still points forward at the node that now points back at it, and you have built a two-node cycle. Printing the result never returns.

Say the cost honestly: this is $O(n)$ stack space, and Python's recursion limit is 1000, so a list of ten thousand nodes raises. The iterative version is strictly better, and the recursive one is worth knowing because interviewers ask for it and because it is the shape of every tree problem later.

Reversing only part of the list

Reverse the nodes between positions m and n . This is where the dummy head from [day 080](#) earns itself again, because m may be 1 and then the head changes.

The method is: walk to the node just before position *m*, hold it, reverse the sublist with the same four lines, then reattach both ends.

```
before: 1 → 2 → 3 → 4 → 5, reverse positions 2..4
         ^          (this is `before`, the node at position 1)

after:   1 → 4 → 3 → 2 → 5
```

The two reattachments are where it goes wrong: `before.next` must become the last node of the reversed section, and the **first** node of the original section — which is now the last — must point at whatever followed the section. Hold that first node in a variable before you start, because after the reversal you cannot find it.

Reversing in groups of *k*

*Reverse every consecutive group of *k* nodes; leave a final group of fewer than *k* alone.*

Three parts, and the third is the one people forget:

1. **Check there are *k* nodes left**, by walking ahead. If not, stop and leave the rest as it is.
2. Reverse those *k* with the same four lines, stopping after exactly *k* steps.
3. **Reattach**: the node before the group points at the group's new first node, and the group's new last node points at whatever comes next.

The check must happen *before* the reversal, not after. If you reverse first and then discover there were only three nodes when *k* is four, you have to reverse them back — which is legal, and is embarrassing, and is not what anyone wants to watch.

4 The picture

Reversing `1 → 2 → 3`. Watch `next_node`, which exists only to survive line 2.

```

start
  previous = None
  current = [1] → [2] → [3] → None

--- iteration 1 ---
  1. next_node = current.next           next_node = [2]
  2. current.next = previous             [1] → None
  3. previous = current                  previous = [1]
  4. current = next_node                 current = [2]

  None ← [1]           [2] → [3] → None
         ^             ^
       previous      current

--- iteration 2 ---
  next_node = [3]; [2] → [1]; previous = [2]; current = [3]

  None ← [1] ← [2]           [3] → None
         ^             ^
       previous      current

--- iteration 3 ---
  next_node = None; [3] → [2]; previous = [3]; current = None

  None ← [1] ← [2] ← [3]
                   ^
                 previous      current is None: stop

return previous → [3] → [2] → [1] → None

```

What to notice: after line 2 of the first iteration, **nothing in the world points at node 2 except `next_node`**. Node 1's arrow has already been turned round. That one variable is the only thing standing between you and losing the list, which is why it is written first and why Nalini says “look behind you” first every time.

And the recursive version, drawn at the moment it does its work:

```

the call stack has unwound to head = [1], and everything after is already reversed

[1] -----> [2] ← [3]           head.next is still [2]
  ^             ^             and [2] is now the LAST node
 head      head.next  new_head

head.next.next = head   →   [2] → [1]
head.next = None       →   [1] → None

result: [3] → [2] → [1] → None

forget `head.next = None` and you get:

[1] → [2] → [1] → [2] → ...      a two-node cycle, printing never returns

```

5 The code, built step by step

Step 1 — the three variables, and what each is for

```
previous: Node | None = None    # becomes the new head; None makes the old head a tail
current: Node | None = head    # the node whose arrow we are about to turn
```

Two, not three, at the start — `next_node` is created inside the loop because it only has to survive one iteration.

Step 2 — the loop, in the fixed order

```
while current is not None:
    next_node = current.next    # SAVE FIRST. Everything else depends on this.
    current.next = previous
    previous = current
    current = next_node
```

Read them out loud as you type: *save, turn, previous up, current up*. Four lines, always the same, and if you write them in any other order something is lost.

Step 3 — the return

```
return previous
```

`current` is `None` and `previous` is the last node visited. That is the old tail, and the old tail is the new head.

Step 4 — reversing a sublist, with a dummy

```
dummy = Node(0, head)
before = dummy
for _ in range(left - 1):    # walk to the node just before position `left`
    before = before.next
```

The dummy means `left = 1` needs no special case: `before` is simply the dummy itself.

```
tail_of_reversed = before.next    # this node ends up LAST — hold it now
previous, current = None, before.next
for _ in range(right - left + 1): # reverse exactly this many nodes
    next_node = current.next
    current.next = previous
    previous = current
    current = next_node
```

`tail_of_reversed` must be captured **before** the reversal, because afterwards there is no way to find it. This is the line people leave out and then spend five minutes hunting for.

```
before.next = previous          # attach the reversed section
tail_of_reversed.next = current # and its far end to the remainder
return dummy.next
```

Two reattachments, in either order, and both are required.

Step 5 — groups of k, with the check first

```
def has_k_nodes(node, k):
    for _ in range(k):
        if node is None:
            return False
        node = node.next
    return True
```

A separate walk, before touching anything. It costs another $O(n)$ across the whole run — so the algorithm is $O(2n)$, still linear — and it buys you never having to undo a reversal.

The complete solution

```
class Node:
    __slots__ = ("value", "next")

    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value
        self.next = next

def from_values(values: list[int]) → Node | None:
    head: Node | None = None
    for value in reversed(values):
        head = Node(value, head)
    return head

def to_values(head: Node | None) → list[int]:
    out, node = [], head
    while node is not None:
        out.append(node.value)
        node = node.next
    return out

def reverse(head: Node | None) → Node | None:
    """Reverse the list in place. O(n) time, O(1) space.

    The order of the four lines is not negotiable: current.next is overwritten
    on line 2, so the rest of the list must be saved on line 1 or it is lost.

    Returns `previous`, not `head` – the old tail is the new head.
    """
    previous: Node | None = None
    current = head

    while current is not None:
        next_node = current.next      # 1. save the rest of the list
        current.next = previous       # 2. turn this arrow round
        previous = current            # 3. step previous forward
        current = next_node           # 4. step current forward

    return previous

def reverse_recursive(head: Node | None) → Node | None:
    """The same thing recursively. O(n) time, O(n) STACK space.

    Trust the recursive call: assume everything after `head` is already
    reversed. Then head.next is the last node of that reversed part, so point
    it back at head, and make head the new tail.

    Python's recursion limit is 1000, so this raises on a long list.
    """
    if head is None or head.next is None:
        return head                # empty or single node: already reversed

    new_head = reverse_recursive(head.next)
    head.next.next = head           # the node after me now points at me
    head.next = None                # and I point at nothing – MANDATORY
    return new_head                 # unchanged all the way up

def reverse_between(head: Node | None, left: int, right: int) → Node | None:
    """Reverse positions left..right (1-indexed), leaving the rest alone.

    The dummy removes the left == 1 case. `tail_of_reversed` must be captured
    BEFORE the reversal – afterwards there is no way to find it.
    """
    if head is None or left ≥ right:
```



```

        return head

    dummy = Node(0, head)
    before = dummy
    for _ in range(left - 1):
        before = before.next

    tail_of_reversed = before.next    # ends up last; hold it now
    previous, current = None, before.next
    for _ in range(right - left + 1):
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node

    before.next = previous            # front joint
    tail_of_reversed.next = current   # back joint
    return dummy.next

def reverse_k_group(head: Node | None, k: int) → Node | None:
    """Reverse every consecutive group of k nodes. A final group of fewer than
    k is left as it is.

    The count check happens BEFORE the reversal, so a short final group is
    never reversed and then un-reversed.
    """
    if k ≤ 1:
        return head

    dummy = Node(0, head)
    group_before = dummy

    while _has_k_nodes(group_before.next, k):
        group_start = group_before.next    # becomes the group's LAST node
        previous, current = None, group_start
        for _ in range(k):
            next_node = current.next
            current.next = previous
            previous = current
            current = next_node

        group_before.next = previous        # attach the reversed group
        group_start.next = current          # and its far end to the rest
        group_before = group_start          # next group starts after this

    return dummy.next

def _has_k_nodes(node: Node | None, k: int) → bool:
    for _ in range(k):
        if node is None:
            return False
        node = node.next
    return True

def reverse_with_a_list(head: Node | None) → Node | None:
    """The honest O(n)-space alternative. Say out loud that you are trading
    space for simplicity if you write this one."""
    values = to_values(head)
    return from_values(values[::-1])

if __name__ == "__main__":
    print(to_values(reverse(from_values([1, 2, 3, 4, 5]))))    # [5, 4, 3, 2, 1]
    print(to_values(reverse(from_values([1]))))                # [1]
    print(to_values(reverse(from_values([]))))                 # []
    print(to_values(reverse(from_values([1, 2]))))             # [2, 1]

    print(to_values(reverse_recursive(from_values([1, 2, 3, 4])))) # [4, 3, 2, 1]

```

```

print(to_values(reverse_recursive(from_values([])))) # []

print(to_values(reverse_between(from_values([1, 2, 3, 4, 5]), 2, 4))) # [1,4,3,2,5]
print(to_values(reverse_between(from_values([1, 2, 3, 4, 5]), 1, 5))) # [5,4,3,2,1]
print(to_values(reverse_between(from_values([1, 2, 3]), 2, 2))) # [1,2,3]
print(to_values(reverse_between(from_values([5]), 1, 1))) # [5]

print(to_values(reverse_k_group(from_values([1, 2, 3, 4, 5]), 2))) # [2,1,4,3,5]
print(to_values(reverse_k_group(from_values([1, 2, 3, 4, 5]), 3))) # [3,2,1,4,5]
print(to_values(reverse_k_group(from_values([1, 2, 3, 4]), 4))) # [4,3,2,1]
print(to_values(reverse_k_group(from_values([1, 2, 3]), 5))) # [1,2,3]

# the recursion limit is real
long_list = from_values(list(range(5000)))
try:
    reverse_recursive(long_list)
except RecursionError as error:
    print(f"RecursionError: {error}")
print(to_values(reverse(from_values(list(range(5000))))[:3]) # [4999, 4998, 4997]

```

6 What it costs

The iterative version

time:	the loop runs once per node	$O(n)$
	each iteration is 4 assignments	constant
space:	previous, current, next_node	$O(1)$

Count the assignments out loud: $4n$ writes for a list of n nodes, and no allocation at all. That is about as cheap as an algorithm gets, and the $O(1)$ space is the reason this is the version to write.

The recursive version

time:	$O(n)$	one call per node
space:	$O(n)$	one stack frame per node – NOT $O(1)$

Python's default recursion limit is 1000, so:

```
RecursionError: maximum recursion depth exceeded
```

fires at about a thousand nodes. That is not a theoretical limit, it is a crash on any realistic input, and it is the reason the iterative version is the right answer even when the interviewer asks for both. Say it rather than waiting to be told — “recursive is $O(n)$ stack and raises past a thousand nodes in Python, so I would ship the iterative one.”

Reversing a sublist and reversing in groups

```
reverse_between:  O(n) time, O(1) space
                  the walk to `left` is O(left), the reversal is O(right - left)
reverse_k_group:  O(2n) time → O(n), O(1) space
                  each node is visited once by the counting walk and once by the reversal
```

The `2n` is worth stating precisely rather than hiding: **each node is walked over twice, once to check the group is full and once to reverse it**. Still linear, and the alternative — reverse first, then undo if the group was short — costs the same in the bad case and is much harder to get right.

Against the $O(n)$ -space alternative

```
values into a Python list, reverse, rebuild:
    time    O(n), three passes
    space   O(n) for the values list plus n new nodes
    nodes allocated: n
in place:
    time    O(n), one pass
    space   O(1)
    nodes allocated: 0
```

At a million nodes that is roughly 48 MB of new nodes against three variables. The list version is not *wrong*, and if you write it you should say what you traded — but no interviewer asking “reverse a linked list” wants it.

7 The traps

Trap 1 — overwriting `current.next` before saving it

```
while current is not None:
    current.next = previous    # WRONG ORDER — the rest of the list is gone
    next_node = current.next  # this is now `previous`, not the next node
```

The loop runs twice and stops. `reverse(from_values([1, 2, 3]))` returns `[1]`. No exception — the list simply ends. This is Nalini’s boy letting go before looking back, and it is the single defining mistake of this problem.

Trap 2 — returning `head`

```
return head    # WRONG
```

After the loop, `head` refers to the old first node, which is now the **last** node and points at `None`. So you return a one-element list. `previous` is the new head.

Trap 3 — the missing `head.next = None` in the recursive version

```
new_head = reverse_recursive(head.next)
head.next.next = head
return new_head                                # forgot: head.next = None
```

The old head still points forward at the node that now points back at it. Printing the result:

```
KeyboardInterrupt
```

after the output list has grown until memory ran out. A two-node cycle, and the only symptom is that nothing returns.

Trap 4 — capturing the sublist's tail too late

```
before.next = previous
tail_of_reversed = before.next                # WRONG — this is now the FIRST node
```

After the reversal, the node that used to be first is now last, and `before.next` points at what used to be last. Capture it before you start, when it is still `before.next` and still first.

Trap 5 — reversing a short final group

```
while group_before.next is not None:          # no k-count check
```

On `[1, 2, 3]` with `k = 5`, this reverses the three nodes it finds and returns `[3, 2, 1]`, when the specification says a group of fewer than `k` is left alone. Check first, with a separate walk.

Trap 6 — `RecursionError` treated as an edge case

```
RecursionError: maximum recursion depth exceeded
```

It is not an edge case. A list of a few thousand nodes is ordinary, and the recursive solution simply does not work on it in Python. Raising `sys.setrecursionlimit` is not a fix — it moves the failure from a clean exception to a segmentation fault when the C stack runs out.

Trap 7 — reversing values instead of pointers

```
values = to_values(head)
node, i = head, len(values) - 1
while node is not None:
    node.value = values[i]          # copies values back in reverse
    ...
```

It produces the right output and it is $O(n)$ space, and it is a different algorithm from the one you were asked for. Worse, it is wrong the moment the nodes carry anything other than a value — an identity, a reference held elsewhere, extra fields. The question is about pointers. Move the pointers.

Trap 8 — `while current.next is not None`

```
while current.next is not None:    # stops one node early
```

The last node never gets its arrow turned, so it still points at `None` and ends up detached. `reverse([1, 2, 3])` returns `[2, 1]`. The loop condition is `current is not None`, because the node itself is what you are processing, not the link out of it.

8 In the interview

How it gets asked

- The warm-up: “Reverse a linked list.” LeetCode 206. Often the first five minutes of a longer round.
- Immediately after: “Now do it recursively.” They want to see whether you can, and whether you will volunteer the stack cost.
- The escalation: “Reverse only the nodes between positions m and n .” LeetCode 92.
- The hard one: “Reverse the nodes in groups of k .” LeetCode 25, Hard, and it is this loop plus bookkeeping.
- The applied version: “Check whether a linked list is a palindrome, in $O(1)$ space.” — find the middle, reverse the second half, compare, and ideally restore.

What to say out loud, in the first ninety seconds

1. **State what is actually changing.** “No node moves and nothing is allocated — I am turning each `next` pointer round exactly once, so this is $O(n)$ time and $O(1)$ space.”
2. **Name the hazard before writing.** “The danger is that the moment I overwrite `current.next`, I lose my only reference to the rest of the list. So the first line of the loop saves it.”

3. **Say the four lines as a phrase.** “Save the next node, turn the arrow round, move previous forward, move current forward — in that order.”
4. **Say what `previous` starts as, and why.** “`previous` starts as `None`, which does two jobs: the old head correctly becomes the tail pointing at nothing, and it is the value I return for an empty list.”
5. **Say the return.** “I return `previous`, not `head`. When the loop ends, `previous` is the old tail, which is the new head.”
6. **Then offer the recursive version with its cost**, rather than waiting to be asked to compare.

The follow-ups

“Now do it recursively.” “The base case is an empty list or a single node — both are already reversed. Otherwise I reverse everything after `head` and trust that call to have done its job. Then `head.next` is still pointing at what was the second node, which is now the last node of the reversed part, so I make it point back at `head`, and I set `head.next = None` to make `head` the new tail. That last line is mandatory — without it the two nodes point at each other and the list has a cycle. I return the new head unchanged all the way up, because the new head is the old tail and it was found at the very bottom. The cost is $O(n)$ stack, and in Python it raises `RecursionError` past about a thousand nodes, so I would ship the iterative version.”

“Reverse only positions m to n .” “Walk to the node just before position m and hold it. Capture the node *at* position m as well, before doing anything, because after the reversal it will be the last node of the section and there is no way to find it again. Reverse exactly $n - m + 1$ nodes with the same four lines. Then two reattachments: the node before points at the new first node of the section, and the captured node points at whatever followed the section. I would use a dummy head, because if m is 1 the head itself changes.”

“Reverse in groups of k , leaving a short final group alone.” “Three parts. First, check there are k nodes remaining, with a separate walk — before touching anything, so I never reverse a short group and then have to undo it. Second, reverse exactly k nodes. Third, reattach: the node before the group points at the group’s new first node, and the group’s old first node — which is now its last — points at whatever comes next. Then the node before the next group is that old first node. It is $O(2n)$, so still linear, and $O(1)$ space.”

“Can you do it in $O(1)$ space recursively?” “No. Recursion uses stack space proportional to the depth by definition, and the depth here is n . Tail-call elimination would fix it in a language that does that, and Python deliberately does not. So if the constraint is $O(1)$ space, the answer is the iterative version.”

“Where would you actually need this?” “Two places. Checking whether a list is a palindrome in $O(1)$ space: find the middle with fast and slow pointers, reverse the second half, compare, and reverse it back so I do not leave the caller’s data mangled — leaving it reversed is a real bug and worth mentioning. And adding two numbers stored as linked lists with the most significant digit first, where reversing both makes the carry propagate in the natural direction.”

“Your loop condition — why `current` and not `current.next`?” “Because `current` is the node I am processing, not the link. With `current.next is not None` the loop stops one node early, the last node never gets its arrow turned, and it ends up detached — reversing `[1, 2, 3]` gives `[2, 1]`.”

A model answer

Asked: *reverse a linked list.*

“Let me say what is actually changing first, because it decides the approach. No node moves, nothing is allocated, no value is copied — every node’s `next` pointer is turned round exactly once. So this should be one pass, $O(n)$ time and $O(1)$ extra space.

The hazard is a single line. The moment I overwrite `current.next`, I have destroyed my only way of reaching the rest of the list — nothing else points at it. So the first thing inside the loop is to save it.

Three variables. `previous`, which starts as `None`. `current`, which starts at the head. And `next_node`, created inside the loop because it only has to survive one iteration. `previous` starting as `None` is doing two jobs, which is worth noticing: it makes the old head correctly become the new tail pointing at nothing, and it is the correct return value when the list is empty.

The loop is four lines and the order is not negotiable. Save the next node. Turn this node’s arrow round to point at `previous`. Move `previous` up to `current`. Move `current` up to the saved node. Save, turn, previous up, current up.

When the loop ends, `current` is `None` and `previous` is the last node I visited — which is the old tail, and the old tail is the new head. So I return `previous`, not `head`. `head` now refers to the node that ended up last, so returning it hands back a one-element list, and there is no error to tell you.

Complexity: four assignments per node, so $4n$ writes, one pass, and three references of extra space regardless of length. There is an $O(n)$ -space version — read the values into an array, reverse it, rebuild — and I would only write that if simplicity mattered more than space, and I would say so rather than let it pass as the intended answer.

If you want it recursively: the base case is an empty list or a single node. Otherwise reverse everything after `head`, and then, standing on `head`, note that `head.next` still points at what was the second node, which is now the *last* node of the reversed part. So `head.next.next = head` makes it point back at me, and `head.next = None` makes me the new tail. That second line is mandatory — leave it out and those two nodes point at each other and printing the list never returns. The recursive version is $O(n)$ stack and raises `RecursionError` past about a thousand nodes in Python, so I would ship the iterative one and mention the recursive one as the shape that generalises to trees.”

9 Recall card

- **Nothing moves and nothing is allocated — you turn each `next` round exactly once.** Three variables, four lines, one fixed order: **save the next node · turn the arrow · previous up · current up.** *Look behind you, let go, take hold.*
- **Line 1 is the whole problem.** `current.next` is overwritten on line 2, so failing to save it first loses the entire rest of the list — no exception, `[1,2,3]` just returns `[1]`.
- **Return `previous`, not `head`.** At the end, `current` is `None` and `previous` is the old tail, which is the new head; `head` now refers to the last node, so returning it gives a one-element list. And loop on `while current is not None`, never `current.next` — that stops one node early and detaches the tail.
- **Recursive: trust the call, then `head.next.next = head` and `head.next = None`.** The second line is **mandatory** — without it you build a two-node cycle and printing never returns. It is **$O(n)$ stack**, and `RecursionError: maximum recursion depth exceeded` fires at about **1000** nodes in Python, so ship the iterative one.
- **The two escalations are this loop plus bookkeeping.** *Sublist* (LeetCode 92): use a **dummy**, and **capture the node at position `left` before reversing** — afterwards it is the last node and unfindable; then two reattachments. *Groups of k* (LeetCode 25): **count k nodes first**, before touching anything, so a short final group is never reversed and undone; $O(2n) \rightarrow O(n)$, $O(1)$ space.

Design a vending machine

1 What this is, and why they ask it

A vending machine takes money, then a selection, then gives out an item and any change. The design has three parts: a **state machine** for where the customer is in that sequence, an **inventory** of slots with prices and counts, and a **coin box** that has to be able to make change out of the coins it happens to be holding.

The third part is what makes it interesting, and it is the mirror image of yesterday's ATM. In an ATM money goes *out* and the account is debited first. Here money comes *in* first, and the machine holds it — so the failure that matters is not “cash left without a debit”, it is “**I have taken your fifty rupees, the item costs thirty-five, and I have no coins to give you fifteen back.**”

They ask it because it is the standard interview state machine — small enough to draw completely in five minutes, with genuinely non-obvious behaviour at the edges. Cancel at any point. Change that cannot be made. An item that jams after the money is taken. Every one of those has a right answer, and the right answers are not symmetric with each other. Expect it in low-level design rounds everywhere, often as a shorter alternative to the ATM.

2 The story

Devi's shop is the one just before the bus stop, and she opens the shutter at half past five in the morning, which is when the first shift goes past.

If you go at six and buy a ten-rupee packet of biscuits with a five-hundred-rupee note, she will look at you the way she has looked at people for twenty-six years and say no. Not rudely. Just no. Come back later, or bring change.

She is not being difficult. At six in the morning her tin has almost nothing in it — a few coins she put in the night before so she can start at all. Every note she takes early goes in and does not come out, because there is nothing to give back with it.

By about eleven it has fixed itself. Enough small purchases have come through that the tin is full of tens and twenties and coins, and now she can break a five hundred without thinking about it. By six in the evening she has more change than she needs. The same customer, the same note, the same packet of biscuits: refused at six in the morning, fine at six in the evening. Nothing about the sale changed. Only what was in the tin.

There is a second tin, a smaller one, at the back of the shelf. That one has about four hundred rupees of coins in it and she does not spend it. It is what she starts tomorrow with. Her son once used it to give change on a busy evening and she found out the next morning at half past five, and he has not done it since.

The thing she is most careful about is the order she does things in. She works out the change first, in her head, before she hands anything over. If she cannot make it, she says so while the packet is still on her side of the counter.

She got that from a bad afternoon years ago. A man bought a bottle of something cold, she gave him the bottle, took his note, and then found she was eight rupees short of the change. He had already opened it. She could not take it back and he was not going to accept eight rupees less. In the end she gave him a toffee and both of them were annoyed.

Now the change is counted before the packet leaves her hand.

3 The idea in plain English

Devi's tin is the coin box. Her rule about counting the change first is the single most important line in this design. And "the same sale is fine at six in the evening and refused at six in the morning" is the **exact change only** light, which is not a fault condition but an ordinary state.

The three parts

One: the state machine. The customer inserts money, selects, and receives. Between those, the machine is in a definite state and only certain things are legal. This is the State pattern from [day 073](#).

Two: the inventory. Slots, each with a code (`A3`), a product, a price and a count. Simple, and worth keeping simple.

Three: the coin box. A count per denomination, and the ability to answer "can I make ₹15 out of what I am holding?" — which is the same bounded change-making problem as yesterday's cash dispenser, in miniature.

The states

IDLE	nothing inserted
COLLECTING	money is in, more may come, nothing selected yet
DISPENSING	committed: item is being pushed out
RETURNING	giving change or a refund

Four states, and the transitions that matter are the ones that go *backwards*:

- From `COLLECTING`, **cancel** must always work and must return everything inserted. This is not optional and it is not a feature — a machine that can swallow money without giving an item is a machine that gets kicked.
- From `COLLECTING`, selecting an item that is **sold out** returns to `COLLECTING`, not to an error state. The customer's money is still in the machine and they may want something else.
- From `COLLECTING`, selecting an item you **cannot make change for** also returns to `COLLECTING`, with a message. The money stays in; the customer can add a coin or choose something else.

That last one is the design decision people miss. **Not being able to make change is not a failure of the transaction — it is a refusal to start it.**

The rule that carries the whole design

Verify the change before dispensing the item.

Devi counting before the packet leaves her hand. In code:

```
if not self.coin_box.can_make(inserted - price):
    return "Exact change required for this item"    # nothing has happened yet
self.inventory.dispense(code)                      # now commit
self.coin_box.pay_out(change)
```

The order is the design. Dispense first and you may find yourself owing change you cannot pay, with the item already gone and no way to reverse it. That is the man with the opened bottle.

Compare with the ATM on [day 080](#), where the correct order was *debit first, then dispense*. It looks like the opposite rule and it is the same rule: **do the reversible thing first, and the irreversible thing last**. In an ATM the debit is reversible and the cash is not. Here the money is already held by the machine, so the *item* is the irreversible act. Saying that connection out loud is worth a lot — it shows you have a principle rather than two memorised answers.

The exact-change problem, concretely

The customer inserts ₹50 for a ₹35 item. The machine owes ₹15.

```
coin box: ₹10 × 0, ₹5 × 2, ₹2 × 3, ₹1 × 0
can it make 15? 5 + 5 + 2 + 2 + ... = 5+5+2+2 = 14, then nothing. NO.
with one ₹1:    5 + 5 + 2 + 2 + 1 = 15. YES.
```

Greedy alone is not enough for the same reason as the ATM: it can strand you when a denomination runs out. Use greedy first — it is right almost always — and fall back to a bounded search over the counts you hold. With four denominations and a change amount under ₹100 this is trivially small.

And the **exact change only** light is simply: “is there any amount I might owe that I cannot make?” The cheap and correct approximation used by real machines is to check whether the machine can make change for the most common overpayments, and to light the indicator when it cannot.

The coin float, which is a real design element

Devi’s second tin is a **float**: coins reserved so tomorrow can start. In a machine this shows up as a decision about the coins the customer just inserted:

- **Escrow model** — hold the inserted coins separately until the sale completes, then move them into the box. This means a cancel returns *the customer’s own coins*, which is fairer and simpler.
- **Immediate model** — drop the coins into the box at once, and pay change from the pooled box. This gives you *more* coins to make change with, at the cost of a cancel returning different coins than were inserted, which customers do not care about and auditors sometimes do.

Real machines use escrow, and the reason is worth knowing: it makes a cancel provably correct even if the machine loses power, because the customer’s money was never mixed in.

What makes this simpler than the ATM

One customer at a time, physically. There is no concurrency inside the machine at all — no two people can insert coins into one slot simultaneously. Say this early, because it removes a whole category of discussion and shows you know which problems this system does *not* have. The interesting failures here are physical and sequential, not concurrent.

4 The picture

The state machine, with the backward transitions drawn, because those are the design:

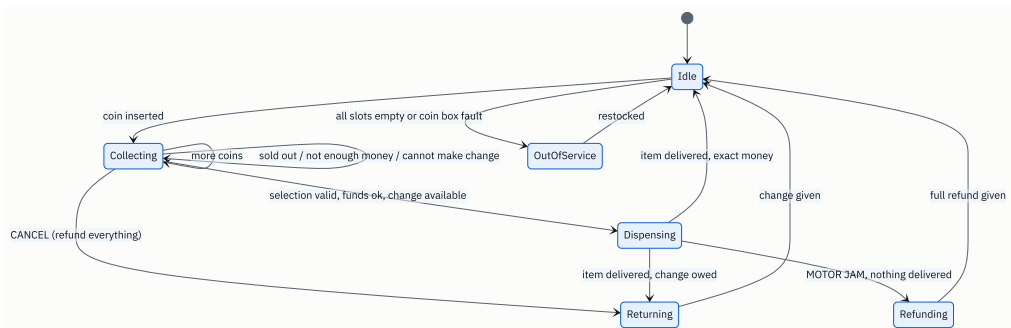


FIGURE 81.1

What to notice: **three different arrows leave Collecting and come straight back to it**. Sold out, insufficient funds, and cannot-make-change are all *refusals to start*, not failures — the money stays in the machine and the customer keeps choosing. A design that sends any of those to an error state or refunds automatically is worse for the customer and more code.

Also notice **Dispensing → Refunding**. The motor is the only thing here that can fail after commitment, and it needs its own path.

The coin box, so the change problem is visible:

```

COIN BOX                                ESCROW (this customer's coins)
+-----+ ₹10 × 0                        +-----+
+-----+ ₹5 × 2                        | ₹20 note |
+-----+ ₹2 × 3                        | ₹10 coin |
+-----+ ₹1 × 0                        +-----+
                                         inserted: ₹30

item A3 costs ₹22 → change owed ₹8
  greedy: ₹5 + ₹2 = ₹7, then needs ₹1 and has none      → greedy FAILS
  search: ₹2 × 3 = ₹6... no combination makes 8        → genuinely impossible
  result: refuse the selection, keep the money in escrow,
           show "exact change required", stay in Collecting

item B1 costs ₹25 → change owed ₹5
  greedy: ₹5 × 1                                         → OK, sell it
  
```

What to notice: the machine can sell B1 and cannot sell A3, **for the same customer with the same money in the escrow**. Availability is per-item and depends on the coin box, which is why the “exact change” indicator is a machine-level approximation of a per-item truth.

The classes:

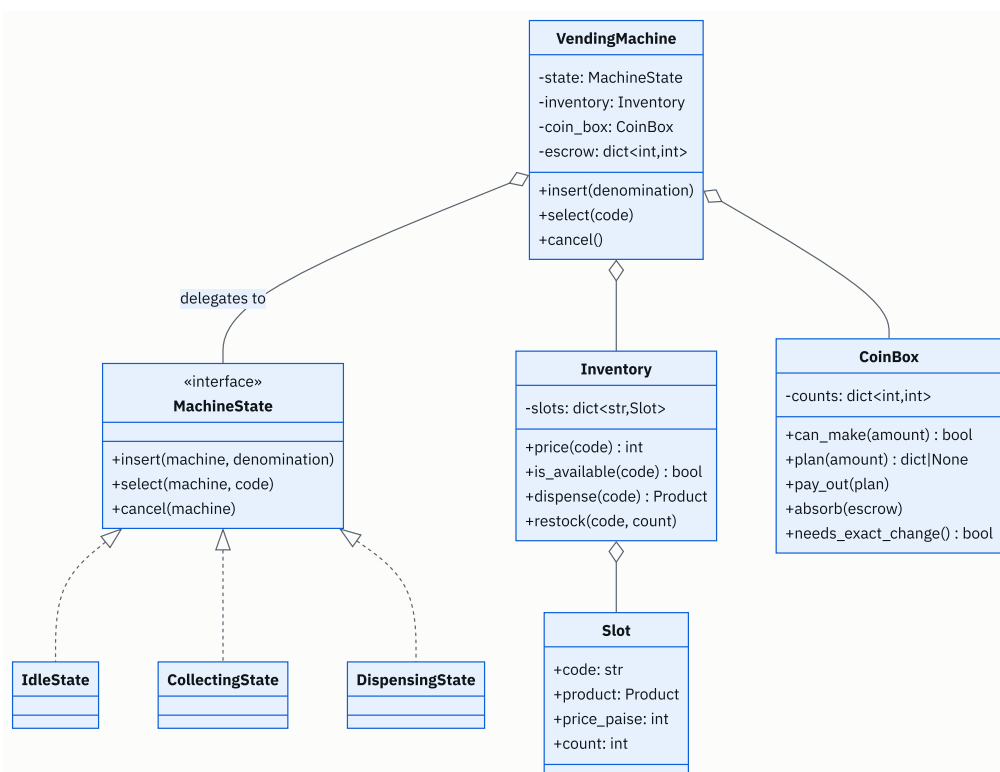


FIGURE 81.2

5 How it actually works

Move 1 · Clarify (minutes 0–5)

“Coins only, or notes and cashless too?” — Coins and small notes; assume no card for now, but ask, because cashless changes the design fundamentally. **“Can the customer cancel after inserting money?”** — Yes, at any time before the item is dispensed. **“Do all items cost the same?”** — No, per-slot pricing. **“What happens if the machine cannot make change?”** — That is the interesting question, and I would propose: refuse the selection, keep the money in, and tell the customer.

“I will assume one customer at a time, which is physically true, so there is no concurrency inside the machine. I am not designing the coin validator hardware or the network telemetry, though I will say where they attach.”

Move 2 · The nouns (minutes 5–10)

- **VendingMachine** — the front door. Holds the state and delegates.

- **MachineState** (*interface*) — one class per state; each allows only what is legal in it.
- **Slot** — a code, a product, a price and a count.
- **Inventory** — the slots, and the availability question.
- **CoinBox** — counts per denomination, and the change-making question.
- **Escrow** — the coins this customer has inserted, held apart until the sale completes.

Six. One interface. Note what is missing: no **Customer** class, because a customer is not a thing the machine stores; no **Transaction** object, because there is only ever one in flight and it lives in the state. Saying why you left things out is worth as much as the classes you include.

Move 3 · The state machine

```
class MachineState:
    """Refuse everything by default; each state enables only what it permits."""
    name: str

    def insert(self, machine: "VendingMachine", denomination: int) → str:
        return self._refuse("insert money")

    def select(self, machine: "VendingMachine", code: str) → str:
        return self._refuse("make a selection")

    def cancel(self, machine: "VendingMachine") → str:
        return self._refuse("cancel")

    def _refuse(self, action: str) → str:
        raise IllegalOperation(f"cannot {action} while {self.name}")
```

Fail closed. In a machine holding customers' money, a state written in a hurry should refuse rather than allow.

```
class CollectingState(MachineState):
    name = "collecting money"

    def insert(self, machine, denomination):
        machine.escrow[denomination] = machine.escrow.get(denomination, 0) + 1
        return f"Credit: {machine.credit()}"

    def cancel(self, machine):
        refund = machine.escrow # the customer's OWN coins
        machine.escrow = {}
        machine.state = IdleState()
        return f"Refunded {sum(d * c for d, c in refund.items())}"
```

Cancel returns the escrow, not coins from the box. That is the escrow model paying off: the refund is exactly what went in, and it is correct even if the box is empty.

Move 4 • The selection, which is where the rule lives

```
def select(self, machine, code):
    if not machine.inventory.is_available(code):
        return "Sold out – choose something else"          # stay in Collecting

    price = machine.inventory.price(code)
    credit = machine.credit()
    if credit < price:
        return f"Insert {price - credit} more"             # stay in Collecting

    change = credit - price
    if change and not machine.coin_box.can_make(change, extra=machine.escrow):
        return "Exact change required for this item"       # stay in Collecting
```

Three refusals, all of which **keep the money in the machine and stay in the same state**. None of them is an error. This block is the answer to “handle exact change”, and it is four lines.

The `extra=machine.escrow` matters: the coins this customer just inserted are available to give back as change, so a ₹5 coin they put in can be part of their own ₹15. Forgetting that makes the machine refuse sales it could complete.

```
# every check has passed – commit, in the irreversible-last order
plan = machine.coin_box.plan(change, extra=machine.escrow)
machine.coin_box.absorb(machine.escrow)          # escrow joins the box
machine.escrow = {}
product = machine.inventory.dispense(code)       # the irreversible act
machine.coin_box.pay_out(plan)
machine.state = IdleState()
return f"Dispensed {product}, change {change}"
```

The plan is computed **before** anything moves, exactly as with the ATM’s cash dispenser. Half a transaction is the failure that cannot be undone, so the code is arranged so it cannot occur.

Move 5 • The coin box

```
class CoinBox:
    def __init__(self, counts: dict[int, int]) → None:
        self._counts = dict(counts)                # denomination → how many

    def plan(self, amount: int, extra: dict[int, int] | None = None) → dict[int, int] | None:
        available = self._merged(extra)
        greedy = self._greedy(amount, available)
        return greedy if greedy is not None else self._search(amount, available)

    def can_make(self, amount: int, extra=None) → bool:
        return amount == 0 or self.plan(amount, extra) is not None
```

Same two-algorithm shape as the ATM: greedy first because it is right almost always and free, a bounded search as the fallback. With four denominations and change under ₹100 the search is a few hundred combinations.

```
def needs_exact_change(self) → bool:
    """Light the indicator when a common change amount cannot be made."""
    return any(self.plan(amount) is None for amount in (1, 2, 5, 10, 15, 20))
```

An approximation, and you should say so: the true answer is per-item and per-customer, and a single lamp on the front of the machine cannot express that. Checking a handful of typical amounts is what real machines do.

Move 6 · The failures that remain

```
class DispensingState(MachineState):
    name = "dispensing"

    def on_motor_jam(self, machine, code, credit):
        machine.inventory.mark_faulty(code)      # do not sell this slot again
        machine.coin_box.refund(credit)          # full refund from the box
        machine.state = IdleState()
        return "Sorry – refunded. That slot is now disabled."
```

The motor is the one component that can fail *after* commitment, and the answer is a full refund plus disabling the slot so the next customer does not hit the same jam. Disabling the slot is the part people forget, and it is what turns one annoyed customer into one annoyed customer rather than fifteen.

Real systems

- **MDB (Multi-Drop Bus)** is the actual serial protocol between a vending controller and its coin mechanism, bill validator and cashless reader. The coin mechanism reports what it accepted and is commanded to pay out — the “coin box” in a real machine is a device you talk to, not a dictionary.
 - **Escrow is a physical component**, not a metaphor: a real coin mech holds the inserted coins in an escrow tube and either returns them on cancel or drops them into the cash box on a completed sale.
 - **Cashless readers** (Nayax, Cantaloupe, UPI QR in India) change the design more than anything else in this list: with a card or a UPI payment, the machine authorises first, dispenses, and *then* captures — which is exactly the ATM’s ordering, and the exact-change problem disappears entirely. That is the single biggest simplification available and it is worth naming.
 - **Telemetry** — modern machines report stock and coin levels over the network so the route is planned by need rather than by schedule, which is where the numbers in §6 come from.
-

6 The numbers

Inventory and how long it lasts

40 slots $\times$ 8 items per slot = 320 items
average price ₹25
sales: 120 per day at a busy site
restock interval: $320 / 120 \approx 2.7$ days

But the average hides the shape:

the top 3 slots take about 40% of sales: $120 \times 0.40 / 3 \approx 16$ sales/day each
8 items in a slot / 16 sales per day = half a day

The popular slots empty in half a day while the machine as a whole lasts nearly three. So a restock schedule based on total stock is wrong, and the machine should report per-slot levels. That is a real conclusion from arithmetic, and it is exactly the kind of thing this section is for.

The coin box

daily takings: $120 \text{ sales} \times ₹25 = ₹3,000$
assume 60% of customers overpay and need change, averaging ₹8 back:
 $72 \times ₹8 = ₹576$ of change paid out per day
coins received: whatever customers insert — mostly ₹10 and ₹20 notes at a busy site

The imbalance is the whole problem:

coins IN from customers: mostly ₹10 coins and notes
coins OUT as change: ₹1, ₹2 and ₹5
net drain on small coins: ~ 70 small coins per day
float loaded at restock: $₹5 \times 100, ₹2 \times 100, ₹1 \times 100 = ₹800$
time until "exact change only" lights: $800 / 576 \approx 1.4$ days

The coin float runs out in under a day and a half, while the stock lasts nearly three. That single comparison explains why the light is on so often on real machines, and it is the argument for either a bigger float, prices chosen to need less change, or cashless payment.

Prices chosen to reduce change

prices ending in ₹5 or ₹0: change is always a multiple of ₹5 $\rightarrow$ only ₹5 coins needed
prices like ₹22 or ₹37: change can need ₹1 and ₹2 coins $\rightarrow$ three denominations to keep

Rounding every price to a multiple of five removes two denominations from the float entirely. **Pricing is a design lever, not just a business decision** — noticing that is a good moment in the interview.

The change algorithm's cost

```
change amounts: 0 to about ₹95
denominations: 4
greedy:                4 steps                ~1 μs
bounded search worst case: ~20^4 pruned        < 1 ms
frequency of the fallback: a few percent, late in the float's life
```

Under a millisecond on a machine whose motor takes four seconds. **The algorithm is free** — so choose the correct one, not the fast one.

State, and the absence of scale

```
40 Slot objects × ~150 B          = 6 KB
CoinBox                          = a few hundred bytes
escrow                           = tens of bytes
-----
total                            ≈ 7 KB
```

Seven kilobytes. **There is nothing to scale here, and no concurrency inside the machine** — one customer, one hand, one slot at a time. The interesting problems are physical and sequential. Saying that early keeps the conversation on the parts that matter.

7 The trade-offs

What this design gives up

“Exact change only” is a machine-level lamp for a per-item truth. The machine may be able to sell you a ₹25 item and not a ₹22 one with the same coins in the box. A single indicator cannot express that, so it is an approximation — check a handful of common change amounts and light it if any fail. The honest alternative is per-item availability shown on a screen, which needs a screen.

Escrow means a cancel returns the customer's own coins, and that reduces change-making ability. If the coins went straight into the box, the box would be fuller and more sales would be possible. Escrow trades a little sales capacity for a refund that is provably correct even after a power cut. Real machines choose escrow, and it is worth saying *why* rather than just which.

A jammed motor is unrecoverable in software. The only honest answers are a full refund and disabling the slot. Retrying the motor risks dispensing two items for one payment, which is worse than a refund. **Never retry a physical dispense** — the same rule as the ATM, for the same reason.

Nothing here handles two customers, and nothing needs to. But it also means nothing here scales to a bank of six machines sharing a cashless reader, which is a real deployment. That would need a transaction id per machine and a shared payment session.

The float is loaded by a human on a schedule and runs out faster than the stock. That is an operational failure the software can only report, not fix. Telemetry that reports coin levels is therefore not a nice-to-have; it is the difference between a machine that is sellable for three days and one that is sellable for one and a half.

No audit trail. A real machine logs every sale, every refund and every coin movement, because the route operator has to reconcile the cash box against the sales. Adding that is one append-only log and it is the same electronic-journal idea as the ATM.

“I would change this design if...”

- **...cashless payment is added.** Then the ordering flips to authorise → dispense → capture, exactly like the ATM, and the entire exact-change problem disappears. This is the biggest available simplification and I would push for it.
- **...the machine sells items of different sizes from one spiral.** Then a slot is not one product and the inventory model changes.
- **...prices are dynamic** — happy-hour pricing, or a discount card. Then price becomes a policy rather than a field on the slot, and I would put an interface there.
- **...one payment session spans several machines.** Then there is a transaction id and a shared session, and the “no concurrency” claim stops being true.

The honest concession

This design has exactly one interesting decision — the order in which money and goods move — and everything else is bookkeeping. It would be easy to dress it up with a `DispenseStrategy` and a `PricingFactory`, and both would be hierarchies built for futures that have not arrived. The state classes earn themselves because each state genuinely allows different actions and refusal-by-default is a safety property. Nothing else does, so nothing else gets one.

8 In the interview

How it gets asked

- The standard: “Design a vending machine.” Then: “What if it cannot make change?”
- The state probe: “What happens if the customer presses cancel after inserting forty rupees?”

- The ordering probe, which is the good one: *“Do you dispense the item first or the change first? Why?”*
- The failure probe: *“The motor jams and the item does not come out. Now what?”*
- The comparison, if the ATM came up earlier: *“Yesterday you debited before dispensing. Here you check change before dispensing. Which is it?”*

The timed script

Minutes 0–5 · Clarify. Coins, notes, cashless? Cancel allowed? Per-slot prices? And ask what should happen when change cannot be made — proposing the answer as you ask is fine and shows you have thought about it.

Minutes 5–10 · Estimation and framing. 40 slots, 120 sales a day, 320 items — stock lasts 2.7 days and the coin float lasts 1.4. Say that gap out loud; it frames the whole design. And say the thing that removes a category of discussion: **one customer at a time, no concurrency inside the machine.**

Minutes 10–20 · The state machine. Four states. Draw the three arrows that leave `Collecting` and come straight back, and say why sold-out, insufficient-funds and cannot-make-change are refusals rather than failures.

Minutes 20–30 · The deep dive: money and goods. The verify-change-before-dispensing rule, the escrow, the greedy-then-search change algorithm, and the connection to the ATM: *do the reversible thing first and the irreversible thing last.*

Minutes 30–40 · Failures and extensions. The motor jam and why you never retry it. The float running out. Then cashless, and how it collapses the hardest part of the problem.

The follow-ups

“What if the machine cannot make change?” “It refuses the *selection*, not the transaction. The money stays in escrow, the machine stays in the collecting state, and the display says exact change is required for that item. The customer can add a coin, choose a cheaper or differently-priced item, or cancel and get their own coins back. What it must not do is take the money and give a partial refund, or dispense and owe change. And I would note that availability is per-item — the same coins might buy a ₹25 item and not a ₹22 one — so the ‘exact change only’ lamp is an approximation of a per-item truth.”

“Do you dispense the item or the change first?” “Neither, first — I *verify* the change before anything moves. Compute the coin plan, confirm it is possible, and only then dispense. If I dispense first and then find I am short, the item is gone and I cannot reverse it. The general rule, which is the same one as the ATM: **do the reversible thing first and**

the irreversible thing last. In an ATM the debit is reversible and the cash is not, so you debit first. Here the machine is already holding the money, so the item is the irreversible act and it goes last.”

“The customer presses cancel after inserting forty rupees.” “Full refund of exactly what they inserted, because the coins are held in escrow rather than mixed into the box. That is the reason for escrow: the refund is provably correct even if the coin box is empty or the machine loses power mid-cancel. Cancel must be legal at any point in the collecting state — a machine that can swallow money without giving an item is a machine that gets kicked.”

“The motor jams and the item does not come out.” “Full refund from the coin box, and mark that slot as faulty so nobody else hits the same jam — that second part is what turns one annoyed customer into one rather than fifteen. And I would never retry the motor automatically: a retry risks dispensing two items for one payment, which is worse than a refund. Same rule as the ATM’s dispense — a physical act that may have partially happened is never safe to repeat.”

“How do you decide which coins to give?” “Greedy from the largest coin down, falling back to a small bounded search over the counts I actually hold — because greedy strands you when a denomination runs out, exactly like the ATM cassettes. And importantly, the coins this customer just inserted are available as change, since they are sitting in escrow. Forgetting that makes the machine refuse sales it could complete. The whole computation is under a millisecond on a machine whose motor takes four seconds, so I would choose the correct algorithm rather than the fast one.”

“How often will the exact-change light be on?” “More often than the machine is out of stock, which surprises people. At 120 sales a day with maybe 60 percent needing change averaging ₹8, that is about ₹576 of small coins paid out daily, against a float of roughly ₹800 loaded at restock — so about a day and a half, while the stock lasts nearly three. Two levers help: a bigger float, and pricing everything in multiples of five, which removes the ₹1 and ₹2 denominations from the problem entirely. And the real fix is cashless.”

“Add card and UPI payment.” “It simplifies the hardest part of this design out of existence. With cashless the machine authorises the exact price, dispenses, and then captures — there is no change to make, so the coin box, the escrow and the exact-change lamp all stop mattering for those sales. The ordering becomes the ATM’s: authorise first because it is reversible, dispense last because it is not. What I gain in exchange is a network dependency and the same failure I had yesterday — authorised but not dispensed — which needs a reversal and a local log.”

A model answer

Asked: *design a vending machine. Handle exact change and refunds.*

“Three parts: a state machine for where the customer is, an inventory of slots with prices and counts, and a coin box that has to make change from whatever it happens to be holding. The third is where the design actually lives.

One framing point first, because it removes a whole category of discussion: there is exactly one customer at a time, physically. No concurrency inside the machine. The interesting failures here are physical and sequential.

The states are idle, collecting, dispensing and returning. I would implement them as one class per state with a base that refuses every action, so each state enables only what it permits — a machine holding customers’ money should fail closed.

The transitions that matter are the ones that go backwards. From collecting, there are three different refusals that all come straight back to collecting: the item is sold out, there is not enough money in yet, and — the interesting one — the machine cannot make change for that item. None of those is a failure. The money stays in, and the customer picks something else or adds a coin. That is the answer to ‘handle exact change’: it is a refusal to *start* the transaction, not a failure in the middle of one.

And cancel must work at any point in collecting, returning everything. I would hold the inserted coins in escrow — separate from the coin box — until the sale completes, so a refund gives back exactly what went in and is correct even if the box is empty or the power cuts. Real coin mechanisms do this in hardware for the same reason.

Now the rule that carries the design: **verify the change before dispensing the item**. Compute the coin plan, confirm it is possible, and only then move anything. If I dispense and then discover I am eight rupees short, the item is gone and there is no reversing it.

That is the same principle as the ATM yesterday, even though it looks like the opposite order: do the reversible thing first and the irreversible thing last. In an ATM the debit is reversible and the cash is not, so you debit first. Here the machine already holds the money, so the item is the irreversible act and it goes last.

For the change itself: greedy from the largest coin down, with a small bounded search as a fallback, because greedy strands you when a denomination runs out. And the coins in escrow count as available change — a five-rupee coin the customer just inserted can be part of their own fifteen rupees back. Forgetting that makes the machine refuse sales it could complete.

Two failures I would design for explicitly. A jammed motor: full refund, and mark the slot faulty so the next fifteen customers do not hit it, and never retry the motor, because a retry risks two items for one payment. And the coin float running out, which is not a bug — some arithmetic: 120 sales a day, maybe sixty percent needing change averaging eight rupees, is about five hundred and seventy-six rupees of small coins out per day against a float of eight hundred. So the exact-change light comes on in about a day and a half, while the stock lasts nearly three. The levers are a bigger float, pricing everything in multiples of five so only five-rupee coins are needed, and telemetry so the route is planned by coin level rather than by schedule.

If I could change one requirement, it would be to add cashless payment, because it deletes the hardest part of this design: no change to make, no float to run out, no exact-change lamp. What it costs is a network dependency and the ATM's failure mode — authorised but not dispensed — which needs a reversal and a local log.”

9 Recall card

- **The rule that carries the design: verify the change BEFORE dispensing the item.** Same principle as the ATM even though the order looks opposite — **do the reversible thing first, the irreversible thing last.** ATM: the debit is reversible, cash is not → debit first. Vending: the money is already held, the item is irreversible → item last.
- **Cannot-make-change is a REFUSAL TO START, not a failure.** Sold out · not enough money · cannot make change are three arrows that leave **Collecting** and come **straight back to it**: the money stays in, the customer chooses again. The “exact change only” lamp is a machine-level approximation of a **per-item** truth — the same coins may buy a ₹25 item and not a ₹22 one.
- **Hold inserted coins in ESCROW, not in the box.** Then cancel returns *exactly what went in* and is correct even after a power cut — and the escrow coins still count as available change, which real designs forget. **Cancel must be legal at any point** before dispensing.
- **Greedy change, bounded search as fallback** (greedy strands you when a denomination empties); **compute the whole plan before anything moves; never retry a jammed motor** — refund fully and **disable the slot**, or fifteen more customers hit the same jam.
- **The float runs out before the stock does: ~₹576/day of change against a ~₹800 float ≈ 1.4 days, while 320 items at 120 sales/day ≈ 2.7 days.** Levers: bigger float · prices in multiples of five (removes ₹1 and ₹2 from the problem) · per-slot telemetry

(the top 3 slots empty in **half a day**) · and the real fix, **cashless**, which deletes change-making entirely and moves the ordering back to authorise → dispense → capture. Total state $\approx$ **7 KB**, **one customer at a time**, **no concurrency**.

DSA topic: Reversing a linked list **System design topic:** Design a vending machine

Code these, in this order

One rule for the whole set: **say “save, turn, previous up, current up” before you write the loop, every time.** Until those four words arrive without thinking, you are not finished with this day.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Reverse Linked List	LeetCode 206 (Easy)	The three-pointer dance, iteratively and recursively, with the stack cost volunteered.
2	Palindrome Linked List	LeetCode 234 (Easy)	Reversal as a <i>tool</i> : find the middle, reverse half, compare — and restore it afterwards.
3	Reverse Linked List II	LeetCode 92 (Medium)	Capturing the sublist’s future tail before you start, plus both reattachments.
4	Reverse Nodes in k-Group	LeetCode 25 (Hard)	Counting before reversing, and the three-way reattachment per group.

On problem 1, write it blind

Close the lesson. Write the iterative version from memory. Then write the recursive one. Then, without running them, predict the output for `[]`, `[1]` and `[1, 2]`. Then run them.

On problem 2, restore what you broke

The $O(1)$ -space solution reverses the second half of the caller’s list. Leaving it reversed is a real bug — you have mutated data you were only asked to inspect. Reverse it back before returning, and say out loud why that matters even though the tests pass either way.

On problem 3, capture before you reverse

Write it once *without* capturing the node at position `left` first. Get to the reattachment and find you cannot name the node you need. That two minutes of being stuck is worth more than reading about it.

On problem 4, count first on purpose

Write the version that reverses first and then discovers the group was short. Watch yourself have to un-reverse it. Then move the count to the front and compare the two.

The four-lines drill

Say each answer out loud, five times:

1. What are the four lines, in order, as a four-word phrase?
2. What exactly is lost if you swap lines 1 and 2? Give the output for `[1, 2, 3]`.
3. Why does `previous` start as `None`, and what two jobs does that do?
4. What do you return, and what would returning `head` give you?
5. Why is the loop condition `current is not None` and not `current.next is not None`?
6. State the time and space, and count the assignments.

The recursion drill

1. Write the base case and say why it covers two inputs.
2. State what you are *trusting* the recursive call to have done.
3. Explain what `head.next` points at when the call returns, in one sentence.
4. Write the two lines that do the work, and say what each achieves.
5. Delete `head.next = None` and run it. Describe the failure precisely.
6. State the space complexity and the exact error text at 5,000 nodes.
7. Say why `sys.setrecursionlimit` is not a fix.

The break-it drill

Trigger each and record the exact output or error text:

1. Overwrite `current.next` before saving it. Run `[1, 2, 3]`.
2. Return `head` instead of `previous`. Run `[1, 2, 3, 4]`.
3. Loop on `current.next is not None`. Run `[1, 2, 3]`.
4. Omit `head.next = None` in the recursive version, then print the result.
5. In `reverse_between`, capture `tail_of_reversed` after the reversal instead of before. Describe what you actually captured.
6. In `reverse_k_group`, drop the count check. Run `[1, 2, 3]` with `k = 5`.
7. In `reverse_k_group`, forget to move `group_before` at the end of a group. Describe the loop.

The escalation drill

1. Reverse the whole list.
2. Reverse positions 2 to 4 of a five-element list, by hand on the diagram, naming every pointer change.
3. Reverse in groups of 2, then 3, then 5, on `[1, 2, 3, 4, 5]`. Write all three outputs before running.
4. For each of the three escalations, say which part is “the same four lines” and which part is new.

The application drill

1. Write `is_palindrome` in $O(1)$ space using reversal.
2. Say which two other techniques it needs, and which day each came from.
3. Restore the list before returning, and say why.
4. Write “add two numbers given most-significant-digit-first” using reversal, and say what the alternative would be if you were forbidden to mutate the inputs.

The space drill

1. Write the $O(n)$ -space version using a Python list.
 2. State exactly what it allocates for a million nodes, in megabytes.
 3. State what the in-place version allocates.
 4. Say the sentence you would use if you chose the $O(n)$ version deliberately.
-

The ordering drill

1. State the rule that decides the order of money and goods, in one sentence.
2. Apply it to the ATM and say which act goes first, and why.
3. Apply it to the vending machine and say which act goes last, and why.
4. Say why those two answers are the same principle rather than two memorised cases.
5. Describe what goes wrong in a vending machine that dispenses first, with a concrete amount.

The state-machine drill

Draw the four states from memory, then answer:

1. Name the three transitions that leave `Collecting` and return to it, and say why none is a failure.
2. Which transition must be legal at any moment before dispensing, and what does it return?

3. Which state can fail *after* commitment, and what are the two things you do?
4. Why does the base state class refuse everything?

The change drill

Given a coin box of ₹10 × 0, ₹5 × 2, ₹2 × 3, ₹1 × 0 and an escrow of one ₹20 note and one ₹10 coin:

1. Can the machine sell a ₹22 item? Show the working.
2. Can it sell a ₹25 item? Show the working.
3. What does that pair prove about the “exact change only” indicator?
4. Find a change amount where greedy fails and a solution exists.
5. Say why the escrow coins must be counted as available change, and what is lost if they are not.

The escrow drill

1. Describe the escrow model and the immediate model in one sentence each.
2. Say what a cancel returns under each.
3. Say which one gives more change-making ability, and which one real machines use.
4. Give the reason real machines choose the one they do, in terms of power failure.

The estimation drill

Compute each, showing the multiplication:

1. Items in the machine and days of stock at 120 sales a day.
2. Days of stock for a top-3 slot at 40 percent of sales across three slots.
3. Change paid out per day at 60 percent of customers averaging ₹8.
4. Days until the float runs out, given ₹800 loaded.
5. The gap between those two lifetimes, and what you would do about it.
6. Total machine state in kilobytes.

Then say which number tells you to report per-slot levels and which tells you to change the pricing.

The failure drill

For each, say what the machine does and what the customer sees:

1. Sold out, money already inserted.
2. Not enough money inserted.
3. Cannot make change for the selected item.
4. Cancel pressed with ₹40 inserted.

5. The motor jams after commitment.
6. Power fails while coins are in escrow.
7. The same slot jams a second time.

Two of the seven leave the machine in `Collecting`. Name them and say why that is right.

The cashless drill

1. Say what changes in the ordering when payment is by card or UPI.
 2. Say which three components stop mattering.
 3. Say which failure mode you have just imported, and from which lesson.
 4. Say what you would add to handle it.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Reverse a linked list. Now do it in groups of k.* What is actually changing, the hazard on line 2 stated before writing, the four lines as a phrase, why `previous` starts as `None`, returning `previous`, the complexities — then k-groups: count first, reverse exactly k, three reattachments, $O(2n)$ and $O(1)$.
 2. *Design a vending machine. Handle exact change and refunds.* Three parts and one customer at a time, the four states with the three refusals that stay in `Collecting`, escrow and what cancel returns, verify-change-before-dispensing with the reversible-first principle and the ATM connection, greedy-then-search, the motor jam, and the float arithmetic.
 3. *Do you dispense the item or the change first?* Neither — verify first. Then the general rule, the ATM comparison, and the concrete failure if you get it backwards.
-

Before you move on

- [] I can say the four lines as a four-word phrase without thinking.
- [] I can state what is lost by swapping lines 1 and 2, with the output.
- [] I know why `previous` starts as `None` and what two jobs that does.
- [] I returned `head` on purpose and saw the one-element list.
- [] I wrote the recursive version blind and can explain both magic lines.
- [] I deleted `head.next = None` and saw the cycle.

- [] I can quote the `RecursionError` and say why raising the limit is not a fix.
- [] I wrote `reverse_between` and got stuck once by not capturing the tail first.
- [] I wrote `reverse_k_group` with the count check in the wrong place, then fixed it.
- [] I used reversal to solve palindrome in $O(1)$ space, and restored the list.
- [] I can state the reversible-first rule and apply it to both the ATM and the vending machine.
- [] I can name the three refusals that stay in `Collecting` and say why none is a failure.
- [] I worked the ₹22-versus-₹25 example and can say what it proves about the lamp.
- [] I can explain escrow and why real machines use it.
- [] I computed the stock lifetime and the float lifetime and can quote the gap.
- [] I answered all three questions above out loud.

Finding the middle, and the runner technique

DATA STRUCTURES AND ALGORITHMS

Finding the middle, and the runner technique

You can find the middle and the Nth from the end in a single pass.

SYSTEM DESIGN

Design a library management system

You can model members, catalogues, loans and fines with the right responsibilities.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Remove the Nth node from the end in one pass.*
- *Design a library system. Who owns the rule about late fees?*

Finding the middle, and the runner technique

1 What this is, and why they ask it

A linked list does not know its own length, and you cannot jump to position $n/2$. So the obvious way to find the middle is to walk the whole list counting, then walk again to the halfway point. Two passes, and correct.

The **runner technique** does it in one. You send two pointers along the list at once. Either one moves twice as fast as the other — so when the fast one reaches the end, the slow one is at the middle — or they move at the same speed with a **fixed gap** between them, so when the front one reaches the end, the back one is exactly that many nodes from it.

Those two variants — different **speeds**, or a different **starting point** — solve a surprising number of problems: the middle, the Nth from the end, whether the list is a palindrome, splitting the list in half for a merge sort, and tomorrow's cycle detection. They ask it because the two-pass solution is obvious and correct, so the question is really *“can you see the one-pass version, and can you get the off-by-one right?”* — and the off-by-one is genuinely fiddly. `while fast and fast.next` and `while fast.next and fast.next.next` differ by one character group and give you different middles.

2 The story

The school is putting up a compound wall along the back of the field and Krishnappa has the job of painting it.

It is a long wall. It runs from the corner by the water tank, past the field, and curves out of sight behind the trees, and you cannot see one end from the other. Nobody has ever measured it and the tape in the storeroom is thirty metres.

He needs two marks on it before he starts.

The first is the exact middle, because the school wants a gate there and the gate has to be centred or it will look wrong for the next forty years.

The second is a point exactly twenty paces in from the far end, where a board is going up.

His nephew Shivu suggested walking it once to count the paces, then walking it again to the halfway number. Krishnappa did that once, years ago, on a different wall. It is an hour, and by the second walk you have stopped trusting your own count.

What he does instead takes one walk.

For the middle, the two of them start together at the water-tank corner. Krishnappa walks normally. Shivu takes two paces for every one of his uncle's — he has done it since he was small and it annoys his uncle for the first hundred metres and then stops mattering. They keep together, one pace to two, all the way. When Shivu reaches the far corner, Krishnappa stops where he is and puts a chalk mark on the stone at his shoulder. That is the middle. He does not know how long the wall is and he does not need to.

For the second mark it is a different trick, and Shivu likes this one better. They both walk at the same speed, but they do not start together. Shivu goes twenty paces ahead first and waits. Then they walk in step, keeping exactly that gap, neither closing it nor opening it. When Shivu reaches the far corner, Krishnappa stops. He is twenty paces from the end, exactly, and again neither of them knows how long the wall is.

The one thing that goes wrong, and it went wrong on the wall behind the market, is when the wall is shorter than the gap. Shivu walks his twenty paces and finds he is already past the end, and there is nothing to measure from. Krishnappa says you have to check that first, before either of you starts walking.

3 The idea in plain English

Two walkers, one wall, one pass. **Different speeds** finds the middle. A **fixed gap** finds a point a known distance from the end. Neither needs to know the length.

The speed variant: finding the middle

```
slow = head
fast = head
while fast is not None and fast.next is not None:
    slow = slow.next          # one step
    fast = fast.next.next     # two steps
return slow                  # slow is at the middle
```

Why it works: `fast` covers exactly twice the distance of `slow` at every moment. When `fast` has travelled `n` steps and stopped, `slow` has travelled `n/2`. That is the whole argument, and it is one sentence.

The loop condition is doing two jobs. `fast is not None` protects against an even-length list where `fast` lands exactly on `None`. `fast.next is not None` protects against an odd-length list where `fast` lands on the last node and `fast.next.next` would explode. **Both checks are required, and `fast is not None` must come first**, because `and` short-circuits.

The even-length ambiguity, which you must raise

A list of four nodes has **two** middles. `[1, 2, 3, 4]` — is the middle 2 or 3?

```
while fast and fast.next:      → slow ends on 3   (the SECOND middle)
while fast.next and fast.next.next: → slow ends on 2   (the FIRST middle)
```

Both are one line. Which you want depends on the problem:

- **Splitting for a merge sort** wants the *first* middle, so that `[1,2]` and `[3,4]` are equal halves and the recursion always shrinks.
- **“Return the middle node”** (LeetCode 876) wants the *second* middle, because that is what the problem statement says.
- **Palindrome checking** wants the second middle, so the second half is the shorter or equal one.

Ask which one before writing. An interviewer who has to tell you there are two middles has learned something; one who hears you ask has learned something better.

The gap variant: the Nth from the end

```
fast = head
for _ in range(n):          # open the gap FIRST
    fast = fast.next
slow = head
while fast is not None:     # now walk in step
    fast = fast.next
    slow = slow.next
return slow                 # slow is the nth node from the end
```

Shivu walking his twenty paces before either of them starts. The gap never changes after that, so when `fast` falls off the end, `slow` is exactly `n` nodes behind it.

And Krishnappa’s warning is the edge case: **if the list is shorter than `n`, the opening walk runs off the end.**

```
AttributeError: 'NoneType' object has no attribute 'next'
```

Check inside the opening loop, and decide what to do — raise, or return the head unchanged — and say which you chose.

Removing the Nth from the end, and why the dummy comes back

To *remove* a node you need the node **before** it, from [day 079](#). So `slow` must stop one node earlier than it does above. Two ways to arrange that, and the good one is the dummy head from [day 080](#):

```
dummy = Node(0, head)
fast = slow = dummy           # both start at the dummy
for _ in range(n):
    fast = fast.next
while fast.next is not None:   # stop when fast is ON the last node
    fast = fast.next
    slow = slow.next
slow.next = slow.next.next
return dummy.next
```

The dummy does two jobs at once here, and it is worth pointing at both. It shifts `slow` back by one so it lands on the **predecessor** of the target. And it makes “remove the first node” — `n` equal to the length — an ordinary case instead of a special one.

Without the dummy you need a gap of `n + 1` and a separate check for whether the head is the target. That works and it is harder to get right under pressure.

The other things the runner technique gives you

Split the list in half, which is the first step of a merge sort on a linked list ([day 084](#)). Find the *first* middle, then cut: `second = middle.next; middle.next = None`. Forgetting to cut is the bug — you get two lists that share a tail, and the recursion never terminates.

Palindrome in O(1) space (LeetCode 234). Find the middle, reverse the second half with [day 081](#)’s four lines, walk both halves comparing, then reverse the second half back so you do not hand the caller a mangled list.

Reorder the list (LeetCode 143): `1→2→3→4→5` becomes `1→5→2→4→3`. Split, reverse the second half, interleave. Three techniques from three days, in one function.

Cycle detection, which is tomorrow. If the list loops, `fast` never reaches the end — and it eventually laps `slow`. Same two pointers, entirely different conclusion.

The honest comparison with two passes

The two-pass version is: count the nodes, then walk `length // 2`. It is O(n) time and O(1) space — **the same complexity as the one-pass version**. So the runner technique is not asymptotically better; it is one pass instead of two, which halves the constant.

Say that plainly rather than overclaiming. The reasons to prefer it are that it works on a stream you can only read once, and that the interviewer asked for one pass. If neither applies, counting is easier to read and easier to get right, and there is no shame in saying so.

4 The picture

Finding the middle of [1, 2, 3, 4, 5]. s is slow, f is fast.

```
start      [1] [2] [3] [4] [5] None
            s,f

step 1      [1] [2] [3] [4] [5] None
            s      f

step 2      [1] [2] [3] [4] [5] None
            s      f

fast.next is None → stop. slow is on 3, the middle of five. Correct.
```

And on an even length, [1, 2, 3, 4], where the two conditions part company:

```
while fast and fast.next:
    start s,f at [1]
    step 1 s=[2] f=[3]
    step 2 s=[3] f=None → stop. slow = 3 (SECOND middle)

while fast.next and fast.next.next:
    start s,f at [1]
    step 1 s=[2] f=[3]
                                → fast.next is [4], fast.next.next is None: stop
                                slow = 2 (FIRST middle)
```

What to notice: **the same walk, two different stopping rules, two different answers.** Neither is wrong. The problem statement decides.

The gap variant, removing the 2nd from the end of [1, 2, 3, 4, 5]:

```
dummy → [1] [2] [3] [4] [5] None
s,f
        open a gap of n = 2

dummy → [1] [2] [3] [4] [5] None
s      f

        now walk together until f is ON the last node

dummy → [1] [2] [3] [4] [5] None
            s      f      ← f.next is None: stop

slow is on [3], which is the node BEFORE the target [4].
slow.next = slow.next.next → [1] [2] [3] [5]
```

What to notice: `slow` started at the **dummy**, not at the head. That one decision is what puts it on the predecessor instead of on the target, and it is why `n` equal to the list length — removing the head — needs no special case.

5 The code, built step by step

Step 1 — the middle, with the condition explained

```
slow = fast = head
while fast is not None and fast.next is not None:
    slow = slow.next
    fast = fast.next.next
```

Two guards, and both are needed. On an even-length list `fast` lands on `None`; on an odd-length list it lands on the last node and `fast.next.next` would raise. `fast is not None` must be first, because `and` short-circuits and the second check dereferences.

Step 2 — the other middle, one line different

```
slow = head
while fast.next is not None and fast.next.next is not None:
    slow = slow.next
    fast = fast.next.next
```

Now the checks look one node further ahead, so the walk stops one step earlier and `slow` lands on the first middle. Requires `head` to be non-`None`, so guard the empty list separately.

Step 3 — the gap, with the length check

```
fast = head
for _ in range(n):
    if fast is None:
        raise ValueError(f"list has fewer than {n} nodes")
    fast = fast.next
```

Krishnappa's warning, in code. Check *inside* the loop, before dereferencing.

Step 4 — walking in step

```
slow = head
while fast is not None:
    fast = fast.next
    slow = slow.next
```


The gap is fixed the moment the first loop ends and nothing changes it. When `fast` becomes `None`, `slow` is `n` nodes from the end.

Step 5 — splitting, and the cut that people forget

```
middle = first_middle(head)
second = middle.next
middle.next = None          # THE CUT. Without it, the halves share a tail.
return head, second
```

One line, and leaving it out gives you two lists that are really one list, so a merge sort recurses for ever. Write the cut immediately after finding the middle, in the same breath.

The complete solution

```
class Node:
    __slots__ = ("value", "next")

    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value
        self.next = next

def from_values(values: list[int]) → Node | None:
    head: Node | None = None
    for value in reversed(values):
        head = Node(value, head)
    return head

def to_values(head: Node | None) → list[int]:
    out, node = [], head
    while node is not None:
        out.append(node.value)
        node = node.next
    return out

def middle_second(head: Node | None) → Node | None:
    """The SECOND middle on an even-length list. [1,2,3,4] → 3.
    This is what LeetCode 876 asks for. O(n) time, O(1) space."""
    slow = fast = head
    while fast is not None and fast.next is not None: # order matters: `and` short-circuits
        slow = slow.next
        fast = fast.next.next
    return slow

def middle_first(head: Node | None) → Node | None:
    """The FIRST middle on an even-length list. [1,2,3,4] → 2.
    This is the one a merge sort needs, so both halves shrink."""
    if head is None:
        return None
    slow = fast = head
    while fast.next is not None and fast.next.next is not None:
        slow = slow.next
        fast = fast.next.next
    return slow

def nth_from_end(head: Node | None, n: int) → Node:
    """The nth node counting from the end (n = 1 is the last node).

    Open a gap of n, then walk both in step. When `fast` falls off the end,
    `slow` is exactly n nodes behind it.
    """
    if n < 1:
        raise ValueError("n must be at least 1")

    fast = head
    for _ in range(n):
        if fast is None:
            raise ValueError(f"list has fewer than {n} nodes")
        fast = fast.next

    slow = head
    while fast is not None:
        fast = fast.next
        slow = slow.next
    return slow

def remove_nth_from_end(head: Node | None, n: int) → Node | None:
```

```

"""Remove the nth node from the end, in one pass.

The dummy does two jobs: it shifts `slow` back one so it lands on the
PREDECESSOR of the target, and it makes removing the head an ordinary case.
"""
dummy = Node(0, head)
fast = slow = dummy

for _ in range(n):
    if fast.next is None:
        raise ValueError(f"list has fewer than {n} nodes")
    fast = fast.next

while fast.next is not None:      # stop with fast ON the last node
    fast = fast.next
    slow = slow.next

slow.next = slow.next.next
return dummy.next

def split_in_half(head: Node | None) → tuple[Node | None, Node | None]:
    """Cut the list into two halves at the FIRST middle.

    The cut is mandatory: without `middle.next = None` the two halves share a
    tail, and a merge sort built on this never terminates.
    """
    if head is None or head.next is None:
        return head, None
    middle = middle_first(head)
    second = middle.next
    middle.next = None              # THE CUT
    return head, second

def reverse(head: Node | None) → Node | None:
    """Day 081's four lines, needed by the two functions below."""
    previous, current = None, head
    while current is not None:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node
    return previous

def is_palindrome(head: Node | None) → bool:
    """O(n) time, O(1) space: find the middle, reverse the second half,
    compare, then put the list back the way you found it.

    Restoring matters – leaving the caller's list half-reversed is a real bug
    even though the tests pass without it.
    """
    if head is None or head.next is None:
        return True

    middle = middle_first(head)
    second = reverse(middle.next)
    middle.next = None

    left, right = head, second
    answer = True
    while right is not None:      # the second half is the shorter one
        if left.value != right.value:
            answer = False
            break
        left, right = left.next, right.next

    middle.next = reverse(second) # restore
    return answer

```

```

def reorder(head: Node | None) → Node | None:
    """[1,2,3,4,5] → [1,5,2,4,3]. Split, reverse the second half, interleave.
    Three techniques from three days in one function."""
    if head is None or head.next is None:
        return head

    first, second = split_in_half(head)
    second = reverse(second)

    a, b = first, second
    while b is not None:
        a_next, b_next = a.next, b.next
        a.next = b
        b.next = a_next if a_next is not None else b_next
        a, b = a_next, b_next
    return first

if __name__ == "__main__":
    print(middle_second(from_values([1, 2, 3, 4, 5])).value) # 3
    print(middle_second(from_values([1, 2, 3, 4])).value) # 3 (second)
    print(middle_first(from_values([1, 2, 3, 4])).value) # 2 (first)
    print(middle_second(from_values([1])).value) # 1
    print(middle_second(from_values([]))).value # None

    print(nth_from_end(from_values([1, 2, 3, 4, 5]), 1).value) # 5
    print(nth_from_end(from_values([1, 2, 3, 4, 5]), 5).value) # 1
    try:
        nth_from_end(from_values([1, 2]), 3)
    except ValueError as error:
        print(f"ValueError: {error}") # list has fewer than 3 nodes

    print(to_values(remove_nth_from_end(from_values([1, 2, 3, 4, 5]), 2))) # [1,2,3,5]
    print(to_values(remove_nth_from_end(from_values([1, 2, 3, 4, 5]), 5))) # [2,3,4,5]
    print(to_values(remove_nth_from_end(from_values([7]), 1))) # []

    left, right = split_in_half(from_values([1, 2, 3, 4, 5]))
    print(to_values(left), to_values(right)) # [1, 2, 3] [4, 5]
    left, right = split_in_half(from_values([1, 2, 3, 4]))
    print(to_values(left), to_values(right)) # [1, 2] [3, 4]

    for values in ([1, 2, 2, 1], [1, 2, 3, 2, 1], [1, 2], [1], []):
        original = from_values(values)
        result = is_palindrome(original)
        print(values, result, to_values(original)) # list must be UNCHANGED

    print(to_values(reorder(from_values([1, 2, 3, 4, 5])))) # [1, 5, 2, 4, 3]
    print(to_values(reorder(from_values([1, 2, 3, 4])))) # [1, 4, 2, 3]

```

The palindrome loop printing the original list afterwards is the test that matters. A version that forgets to restore still returns the right boolean, and hands the caller a list that has been cut in half and reversed.

6 What it costs

Time and space

middle	0(n) time, 0(1) space	– one pass, two references
nth_from_end	0(n) time, 0(1) space	
remove_nth_from_end	0(n) time, 0(1) space	
split_in_half	0(n) time, 0(1) space	
is_palindrome	0(n) time, 0(1) space	– three walks, still linear
reorder	0(n) time, 0(1) space	

Count the middle out loud: `slow` takes $n/2$ steps and `fast` takes $n/2$ iterations of two steps each, so about $1.5n$ pointer moves in one pass. The two-pass version is n moves to count plus $n/2$ to walk, which is the same $1.5n$ — **so the runner technique is not doing less work, it is doing it in one traversal instead of two.**

That distinction matters when the data can only be read once, and it matters much less otherwise. Say it honestly.

Against the alternatives

middle, two passes:	count n , then walk $n/2$	0(n) time, 0(1) space
middle, runner:	one pass	0(n) time, 0(1) space
middle, into a list:	copy all values, index $n/2$	0(n) time, 0(n) SPACE

The array version is the one to reject explicitly: at a million nodes it allocates a million-element list — about 8 MB of pointers plus the values — to answer a question that needs two variables.

Palindrome, three ways

copy values into a list and compare with its reverse:	0(n) time, 0(n) space
use a stack of the first half:	0(n) time, 0(n/2) space
runner + reverse in place:	0(n) time, 0(1) space

The last one is what the follow-up asks for, and it is the reason today and yesterday are next to each other in the syllabus.

The $1.5n$ versus $2n$ question, which does get asked

For `remove_nth_from_end`, the two-pass version walks n to count and then $n - k$ to reach the predecessor — about $2n$ pointer moves. The one-pass version walks n with `fast` and $n - k$ with `slow`, which is also about $2n$ moves, but in a **single traversal**.

So the honest answer to “is one pass faster?” is: **the same number of pointer moves, half the number of traversals.** On a linked list where each node is a fresh cache miss, one traversal is genuinely better than two — but it is a constant factor, not a complexity

difference, and claiming otherwise will get you corrected.

7 The traps

Trap 1 — the loop condition, in the wrong order

```
while fast.next is not None and fast is not None:    # WRONG ORDER
```

```
AttributeError: 'NoneType' object has no attribute 'next'
```

`and` evaluates left to right, so `fast.next` is dereferenced before `fast` is checked. Fires on any even-length list. **`fast is not None` first, always.**

Trap 2 — the wrong middle

```
while fast is not None and fast.next is not None:    # gives the SECOND middle
```

used where the first middle was wanted. In a merge sort this is fatal rather than merely wrong: on a two-element list `[1, 2]`, the second middle is node 2, so the split gives `[1, 2]` and `[]`, the recursion does not shrink, and you get:

```
RecursionError: maximum recursion depth exceeded
```

For splitting, always the first middle. Say which one you are using and why, every time.

Trap 3 — forgetting the cut

```
middle = middle_first(head)
second = middle.next
return head, second                                # forgot: middle.next = None
```

Both “halves” still share every node from the middle onwards. `to_values(left)` prints the whole list. A merge sort built on this recurses on inputs that never get smaller. No error until the recursion limit.

Trap 4 — the gap running off a short list

```
for _ in range(n):
    fast = fast.next                                # no check
```

```
AttributeError: 'NoneType' object has no attribute 'next'
```

when `n` exceeds the length. Check inside the loop and decide the behaviour — raise, or treat it as “remove nothing” — and say which. Interviewers ask “what if `n` is bigger than the list?” precisely because the naive code crashes.

Trap 5 — `slow` starting at the head when you need the predecessor

```
fast = head
for _ in range(n): fast = fast.next
slow = head
while fast is not None:
    fast, slow = fast.next, slow.next
    slow.next = slow.next.next          # slow is ON the target, not before it
```

This removes the node *after* the one you wanted. Start both at a dummy and stop when `fast.next` is `None`, which puts `slow` on the predecessor and handles the head case for free.

Trap 6 — not restoring the list in the palindrome solution

```
middle.next = None
second = reverse(middle.next)
...
return answer          # list left cut in half and reversed
```

The function returns the correct boolean. It also hands back a mutated list, and the caller has no idea. Every test that only checks the return value passes. **A read-only question must leave the data as it found it**, and saying that unprompted is worth more than the algorithm.

Trap 7 — assuming one pass is asymptotically better

It is not. Both are $O(n)$ time and $O(1)$ space, and both do about the same number of pointer moves. One pass is better because it works on read-once data and because it touches memory once instead of twice. Overclaiming here is an easy way to be corrected in an interview.

Trap 8 — the empty list and the single node

`middle_first` dereferences `head.next` in its loop condition, so `head is None` must be handled before it. `[1]` is its own middle and its own palindrome. `remove_nth_from_end(from_values([7]), 1)` must return an empty list, not raise. Run all three before you say you are done.

8 In the interview

How it gets asked

- The base: *“Find the middle node of a linked list.”* LeetCode 876. Almost always followed by “in one pass”.
- The main event: *“Remove the Nth node from the end of the list, in one pass.”* LeetCode 19.
- The application: *“Is this linked list a palindrome? Now do it in $O(1)$ space.”* LeetCode 234.
- The composite: *“Reorder the list so it goes first, last, second, second-last...”* LeetCode 143 — three techniques in one problem.
- Tomorrow’s version: *“Does this list have a cycle?”* Same two pointers, different question.

What to say out loud, in the first ninety seconds

1. **State the two-pass version and why you are not writing it.** “I could count the nodes and then walk halfway. That is $O(n)$ and correct. You asked for one pass, so I will use two pointers.”
2. **Name which variant you need.** “Two pointers at different speeds finds the middle. Two pointers at the same speed with a fixed gap finds a fixed distance from the end. This problem is the second one.”
3. **Say the invariant in one sentence.** “I open a gap of `n` first, then move both together, so the gap never changes — when the front pointer falls off the end, the back one is exactly `n` nodes from it.”
4. **Raise the ambiguity yourself, if it is the middle.** “On an even-length list there are two middles. Which do you want? For splitting I would take the first, so both halves shrink.”
5. **Name the edge case before writing.** “If `n` is bigger than the list, the opening walk runs off the end, so I check inside that loop.”
6. **If removing, say why the dummy.** “I need the node *before* the target, so both pointers start at a dummy — that also makes removing the head an ordinary case.”

The follow-ups

“What if the list has an even number of nodes?” “Then there are two middles, and the loop condition decides which you get. `while fast and fast.next` lands on the second; `while fast.next and fast.next.next` lands on the first. Neither is more correct — it depends what you are doing. If I am splitting for a merge sort I need the first, because with the second, a two-element list splits into itself and an empty list and the recursion never shrinks.”

“What if n is larger than the length of the list?” “The opening walk runs off the end and you get `AttributeError: 'NoneType' object has no attribute 'next'`. So I check inside that loop. What to do then is a product decision I would ask about: raise, or treat it as ‘remove nothing’ and return the list unchanged. LeetCode guarantees n is valid, but real code does not.”

“Why does `slow` start at the dummy rather than the head?” “Because I need the node *before* the one being removed, and starting `slow` one node earlier is the cheapest way to get it — the alternative is a gap of $n + 1$, which is easy to get wrong. The dummy also means that removing the head, when n equals the length, needs no special case at all.”

“Is one pass actually faster than two?” “Not asymptotically — both are $O(n)$ time and $O(1)$ space, and both do roughly the same number of pointer moves. What one pass buys is a single traversal instead of two, which matters when each node is a separate cache miss, and it is required if the data can only be read once, like a stream. I would not claim a complexity improvement, because there is not one.”

“Now check whether the list is a palindrome in $O(1)$ space.” “Three steps, all from this week. Find the first middle with the runner. Reverse the second half in place with the three-pointer loop. Walk the two halves comparing values, stopping when the second half runs out, since it is the shorter one on an odd length. Then reverse the second half back and reattach, because leaving the caller’s list cut in half and reversed is a real bug even though the boolean is right.”

“Where else does this technique turn up?” “Splitting a list for a merge sort, which is the first half of merge sort on linked lists. Reordering a list into first-last-second-second-last, which is split, reverse, interleave. And cycle detection, where the fast pointer never reaches the end and instead laps the slow one — same two pointers, completely different conclusion.”

A model answer

Asked: remove the N th node from the end of the list, in one pass.

“The two-pass version is: walk the list counting to get the length, then walk again to position length minus n . That is $O(n)$ and perfectly correct, and I mention it because it is the baseline you are asking me to beat.

The one-pass version uses two pointers with a **fixed gap**. I move one pointer forward n steps first, so the two are exactly n apart. Then I move both together, one step each, so the gap never changes. When the front pointer falls off the end, the back one is exactly n nodes from the end — and I never had to know the length.

Two adjustments turn ‘find it’ into ‘remove it’.

First, to remove a node I need the node *before* it, because I cannot reach backwards. So the back pointer has to stop one node earlier. I do that by starting both pointers at a dummy node placed in front of the head, rather than at the head itself.

Second, that same dummy handles the case where n equals the length — removing the head. Without it, that is a special case with its own branch, and the alternative arrangement is a gap of $n + 1$, which is one of those off-by-ones that is very easy to get wrong when someone is watching.

The edge case I would check before writing the loop: if n is larger than the list, the opening walk dereferences `None` and you get an `AttributeError`. So the check goes inside that first loop. Whether to raise or to return the list unchanged is a decision I would ask you about.

Then the second loop walks both pointers until the fast one is *on* the last node — not past it — which leaves the slow one on the predecessor of the target. One assignment removes the node, and I return `dummy.next` rather than `head`, because the head may be the node I just removed.

Complexity: $O(n)$ time, $O(1)$ space, one traversal. I would not claim it is asymptotically faster than counting first — both are linear and both do about the same number of pointer moves. What it buys is one pass over memory instead of two, which matters on a linked list where every node is a separate cache miss, and it is the only option if the data can be read only once.

Before I say I am done: I would run it on a single-node list with n equal to one, which must return an empty list; on n equal to the length, which removes the head; and on n equal to one, which removes the tail.”

9 Recall card

- **Two variants of one idea, and naming which you need is the first move.** Different speeds (`slow += 1` , `fast += 2`) finds the **middle**; a fixed **gap** (`fast` starts `n` ahead, then both move together) finds a **fixed distance from the end**. Neither needs the length.
- **The middle's loop condition is doing two jobs and the order matters:** `while fast is not None and fast.next is not None` — the first guard protects even lengths, the second odd, and `and` short-circuits so the null check must come first.
- **On an even length there are TWO middles — ask which.** `while fast and fast.next` → the **second** (LeetCode 876). `while fast.next and fast.next.next` → the **first**, which is what **splitting for a merge sort** needs: with the second, `[1,2]` splits into itself and nothing and you get `RecursionError`. And **the cut is mandatory** — `middle.next = None` , or the halves share a tail.
- **To remove the Nth from the end, start BOTH pointers at a dummy** — it lands `slow` on the **predecessor** and makes removing the head an ordinary case. Guard the opening walk, or a short list gives `AttributeError: 'NoneType' object has no attribute 'next'`.
- **One pass is not asymptotically better — both are $O(n)$ time and $O(1)$ space and do about the same pointer moves.** What it buys is **one traversal instead of two** (and read-once data). Applications: **palindrome in $O(1)$ space** (middle → reverse → compare → **reverse back**, because a read-only question must leave the data as it found it) · **reorder** (split, reverse, interleave) · and tomorrow, **cycle detection** — the same two pointers, a completely different conclusion.

1 What this is, and why they ask it

A library lends physical copies of books to members, takes them back, charges for late returns, and lets people reserve something that is currently out. Four verbs, and the whole design turns on two questions that look small and are not.

The first is a **modelling** question: is “*Ponniyin Selvan*” one thing or six? The library owns six physical copies, each with its own barcode, its own condition, and its own borrower. But a member searching the catalogue is looking for the *title*, and a reservation queue is a queue for the title, not for copy number four. Merging those two ideas into one class is the single most common mistake in this prompt, and it breaks reservations, availability and reporting all at once.

The second is a **responsibility** question, and it is the one in the prompt: **who owns the rule about late fees?** Not the book. Not the member. Arguably not even the loan. Getting that answer right — and being able to say *why* the obvious homes are wrong — is what this round is scored on.

They ask it because it is a domain everyone understands, so no knowledge separates candidates, only judgement. And because the two questions above are exactly the two questions every real business system asks: *what are the entities really*, and *where does the rule live*.

2 The story

Shakuntala’s shop is two rooms behind the temple and it has been a lending library since 1979, when her father-in-law started it with about two hundred Kannada novels.

There are about four thousand now. People pay thirty rupees a month and take two at a time.

She has two problems and she has had both of them for forty years.

The first is the word “book”. A woman comes in and asks whether Shakuntala has *Mookajjiya Kanasugalu*, and Shakuntala says yes — because she does, it is on the list, they have had it since before the shop had electricity. Then she has to say the other thing, which is that both copies are out, one with the schoolmaster who has had it since August

and one with a woman in the next lane. So the answer is yes and also no, and the woman is annoyed, and Shakuntala has explained this ten thousand times. The book exists. The books you can take home do not.

She has three copies of one Bhyrappa novel and one of them has a torn spine and she will not lend it to anyone she does not know. So even “we have three” is not a simple sentence.

The second problem is the late fee, and this one her daughter finally made her look at properly.

There is a rule. It is a rupee a day after two weeks. Shakuntala will tell you that rule without hesitating.

But she does not charge the schoolmaster, because he is seventy-nine and he brings the books back eventually and once he gave her father-in-law money when the shop flooded. She does not charge anybody in the week of a festival. She charges the college boys the full amount and sometimes more than the full amount, because otherwise they keep the books for months. She once charged a woman fourteen rupees and then took eight of it off when the woman said the child had been ill.

Her daughter, who does the accounts on a laptop now, asked her one evening what the rule actually is, so she could put it in.

Shakuntala said, a rupee a day after two weeks.

Her daughter said, but that is not what you do.

And Shakuntala said the thing that ended the conversation, which was: the rule is not in the book and it is not in the person, it is in my head, and I change it. Write down what happened — who took what, when they took it, when they brought it back. What they owe is my business, and I will tell you.

3 The idea in plain English

Shakuntala’s two problems are the two design questions. And her last sentence is the answer to the second one, almost word for word.

Problem one: a book is two things

There are two entirely different concepts sharing a word, and they need two classes.

- **Book** — the *title*. ISBN, title, author, publisher, subject. There is exactly one of these for *Mookajjiya Kanasugalu*, however many copies exist. This is what the catalogue searches and what a reservation queues for.

- **BookItem** — a *physical copy*. Its own barcode, its own rack location, its own condition, its own status (on the shelf, lent out, lost, being repaired), its own purchase date and price. Six of these can point at one **Book**.

Once you split them, three things that were awkward become obvious:

```
"Do you have X?"      → a Book lookup. Answer: yes.
"Can I take X home?"  → is any BookItem of that Book available? Answer: no.
"I want X when it comes" → a reservation on the Book, not on a copy.
"This copy has a torn spine, do not lend it" → a condition on ONE BookItem.
```

Reservations queue for the title; loans are of a copy. That single sentence is the pay-off, and it is impossible to say if **Book** and **BookItem** are the same class. This is the same title-versus-instance split you see everywhere once you look for it: a film versus a showing, a flight number versus today's flight, a product versus a serial number.

Problem two: who owns the late-fee rule

Consider each candidate home and say why it is wrong. Doing this out loud is the answer to the question.

Not Book. A fine is not a property of *Ponniyin Selvan*. The book does not change when the library changes its rates.

Not BookItem. Same, and worse — the fine would then vary by copy for no reason.

Not Member. A member is a person with a name and a membership. They do not know the daily rate, and putting it there means every member object carries pricing logic.

Not Loan either, and this is the interesting one, because **Loan** is the tempting answer. A loan knows who borrowed what and when it was due, so `loan.fine()` looks natural. But to compute a fine correctly it would need: the daily rate, which varies by membership tier; the maximum cap; the holiday calendar, because most libraries do not charge for days they were closed; the current date; and whatever waiver applies. All of those change without any loan changing, and none of them is something a *record of a borrowing* should know.

So: a FinePolicy. The loan holds the **facts** — who, what, when it went out, when it was due, when it came back. The policy **interprets** those facts into money.

```
class FinePolicy(Protocol):
    def amount_paise(self, loan: Loan, on: date) → int: ...
```

That is Shakuntala's sentence: *write down what happened; what they owe is my business*.

And now the rule is a thing you can change, test, and have several of at once — a student rate, a staff rate, a festival amnesty — without touching a single `Loan`.

The line between the record and the rule

This does not mean `Loan` is a bag of data with no behaviour. On [day 044](#) you met the **anaemic model** — an object that is only fields, with all the logic elsewhere — and it is a real smell, not something to aim for.

The line to draw is this:

- `Loan` **answers questions about itself**. Is it returned? Is it overdue on a given date? How many days late? Can it be renewed? Those are facts about the loan and nothing else is needed to answer them.
- `FinePolicy` **answers questions about money**. Because money needs rates, tiers, calendars and waivers, and those are not facts about the loan.

`days_overdue` lives on the loan. `rupees_owed` does not. That one-line boundary is the most reusable idea in this lesson, and it generalises far beyond libraries.

The second policy, which people forget

There is a matching rule on the way *out*: how many books may this member take, and for how long?

```
class LendingPolicy(Protocol):
    def max_loans(self, member: Member) → int: ...
    def loan_period_days(self, member: Member) → int: ...
    def max_renewals(self, member: Member) → int: ...
```

A student may take 3 for 14 days; a staff member 10 for 30; a member with an outstanding fine over a threshold may take none. Same argument as the fine: it varies by tier and by policy changes, and it does not belong on `Member`.

Two policies, and the gate for both is the one from [day 076](#): **can you name a second implementation someone would actually want?** For fines: standard, and a festival amnesty. For lending: student and staff. Both pass easily. Nothing else in this design does, so nothing else gets an interface.

Reservations, which are a queue on the title

When every copy is out, a member joins a **first-in-first-out queue** on the `Book`. When any copy comes back:

1. If the queue is empty, the copy goes back on the shelf, status available.
2. Otherwise the copy is **held** for the member at the front — not lent, held — for a fixed window, say three days.

3. If they do not collect it in that window, the hold expires, they lose their place, and the next member in the queue is offered it.

The hold window is the part that makes it a real design rather than a queue. Without an expiry, one member who never comes in blocks the copy for ever. Say that unprompted.

4 The picture

The class diagram. Notice the two arrows out of `LendingService` — the two delegated decisions.

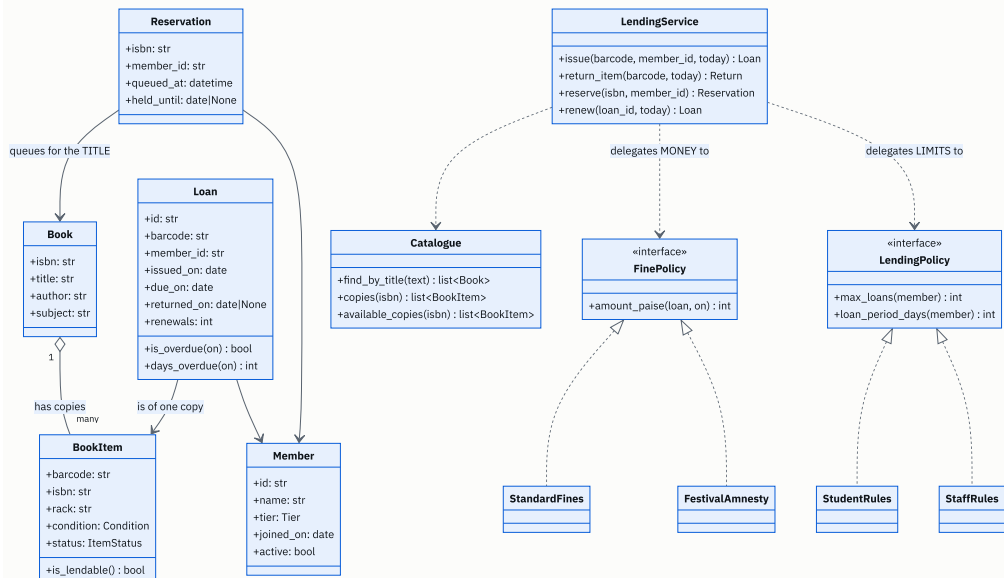


FIGURE 82.1

What to notice: **Loan** points at a **BookItem** and **Reservation** points at a **Book**. Those two arrows going to different classes is the answer to problem one, and if both pointed at the same class the design would be wrong in a way that only shows up when someone reserves a title with six copies.

The lifecycle of a single physical copy:

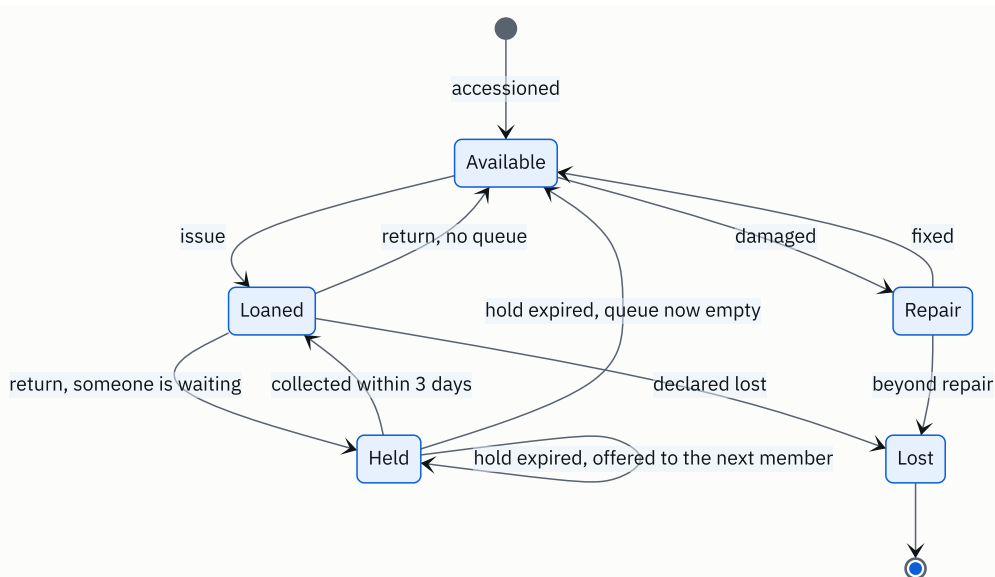


FIGURE 82.2

What to notice: `Held` is a real state, distinct from both `Available` and `Loaned`. A copy on hold is in the building and cannot be lent to a walk-in. Designs that skip it either lend the copy to the wrong person or lose it from the shelf count.

And the two ideas of “a book”, drawn:

Book (title)	BookItem (copies)
+-----+	+-----+
ISBN 978-81-...	BC-1041 BC-1042 BC-1043
Mookajjiya Kanasugalu	rack 3A rack 3A rack 7B
Kota Shivarama Karanth	good good TORN
+-----+	+-----+
^	
Reservation queue	Loans point HERE
points HERE (the title)	(one specific copy)
"Do you have it?"	→ yes, the Book exists
"Can I take it home?"	→ no, zero lendable copies

5 How it actually works

Move 1 · Clarify (minutes 0–5)

“Physical books only, or e-books too?” — Physical. E-books have no copies and no fines, so they would change the model. **“Can members reserve a book that is currently out?”** — Yes, with a queue. **“Do different member types have different limits and rates?”** — Yes: students and staff. **“Are fines charged for days the library is closed?”** — Good question to ask, because the answer decides whether the fine calculation needs a calendar. Assume no charge on closed days.

“I will assume a single branch, that a loan is of one copy, and that payment of fines is out of scope beyond recording that they were paid. I am not designing the search engine internals or the membership billing.”

Move 2 · The nouns (minutes 5–12)

- **Book** — the title. One per ISBN.
- **BookItem** — one physical copy. Barcode, rack, condition, status.
- **Member** — a person, a tier, and whether the membership is active.
- **Loan** — one copy out to one member, with dates. Answers questions about *itself*.
- **Reservation** — one member queued for a *title*, with a hold window when offered.
- **Catalogue** — search and availability lookups.
- **LendingService** — issue, return, renew, reserve. The entry point; holds no rules.
- **FinePolicy** (*interface*) — facts to money.
- **LendingPolicy** (*interface*) — limits and periods per member.

Nine, two of them interfaces. Say what is deliberately absent: no **Librarian** class, because a librarian is a user of the system rather than a thing in it; no **Rack** class, because a rack is a string on the copy until someone asks for shelf management.

Move 3 · The records, written first

```
@dataclass(frozen=True)
class Book:
    isbn: str
    title: str
    author: str
    subject: str

class BookItem:
    def __init__(self, barcode: str, isbn: str, rack: str) → None:
        self.barcode = barcode
        self.isbn = isbn                # which title this is a copy of
        self.rack = rack
        self.condition = Condition.GOOD
        self.status = ItemStatus.AVAILABLE

    def is_lendable(self) → bool:
        return self.status is ItemStatus.AVAILABLE and self.condition is not Condition.TORN
```

`is_lendable` is one line and it earns its place: “available” and “lendable” are different things, and Shakuntala’s torn Bhyrappa is why.

```
@dataclass
class Loan:
    id: str
    barcode: str
    member_id: str
    issued_on: date
    due_on: date
    returned_on: date | None = None
    renewals: int = 0

    def is_returned(self) → bool:
        return self.returned_on is not None

    def is_overdue(self, on: date) → bool:
        return not self.is_returned() and on > self.due_on

    def days_overdue(self, on: date) → int:
        end = self.returned_on or on
        return max(0, (end - self.due_on).days)
```

Three methods, and not one of them mentions money. This is the boundary from §3, in code: the loan answers questions about itself using only what it holds. `days_overdue` needs no rate, no calendar and no tier, so it belongs here.

Move 4 • The policies — the interesting part

```
class FinePolicy(Protocol):
    def amount_paise(self, loan: Loan, on: date) → int: ...

class StandardFines:
    """A rate per day per tier, a cap, and no charge for days the library
    was closed. All four of those change without any Loan changing, which is
    exactly why this is not a method on Loan."""

    RATE_PAISE = {Tier.STUDENT: 100, Tier.PUBLIC: 200, Tier.STAFF: 0}
    CAP_PAISE = 50_000 # never more than the book is worth

    def __init__(self, members: "MemberDirectory", closed_days: set[date]) → None:
        self._members = members
        self._closed = closed_days

    def amount_paise(self, loan: Loan, on: date) → int:
        if not loan.is_overdue(on) and not loan.is_returned():
            return 0
        chargeable = self._chargeable_days(loan, on)
        tier = self._members.get(loan.member_id).tier
        return min(chargeable * self.RATE_PAISE[tier], self.CAP_PAISE)
```

Look at the constructor. The policy holds a member directory and a closed-day calendar — two dependencies that a `Loan` should never have. That is the concrete argument for the split, and it is more convincing than any principle.

```
    def _chargeable_days(self, loan: Loan, on: date) → int:
        end = loan.returned_on or on
        days = 0
        day = loan.due_on + timedelta(days=1)
        while day ≤ end:
            if day not in self._closed:
                days += 1
            day += timedelta(days=1)
        return days

class FestivalAmnesty:
    """The second implementation. Every year, for one week, fines are waived –
    and this is a one-line class rather than a branch inside the standard one."""

    def __init__(self, wrapped: FinePolicy, from_day: date, to_day: date) → None:
        self._wrapped, self._from, self._to = wrapped, from_day, to_day

    def amount_paise(self, loan: Loan, on: date) → int:
        if self._from ≤ on ≤ self._to:
            return 0
        return self._wrapped.amount_paise(loan, on)
```

Worth pointing at: `FestivalAmnesty` **wraps** the standard policy rather than replacing it, which makes it a decorator ([day 069](#)) as well as a second implementation of the interface. Shakuntala not charging anyone in festival week, expressed in six lines instead of in her head.

Move 5 • The service, which holds no rules

```
class LendingService:
    def issue(self, barcode: str, member_id: str, today: date) → Loan:
        member = self._members.get(member_id)
        if not member.active:
            raise NotAllowed("membership is not active")

        open_loans = self._loans.open_for(member_id)
        if len(open_loans) ≥ self._lending.max_loans(member):
            raise NotAllowed(f"limit of {self._lending.max_loans(member)} books reached")

        item = self._catalogue.item(barcode)
        if not item.is_lendable():
            raise NotAllowed(f"copy {barcode} is {item.status.name.lower()}")
```

Three checks, and each one asks somebody else the question. `max_loans` comes from the policy, not from a constant here. That is what “the service holds no rules” means in practice.

```
held = self._reservations.hold_on(item.barcode)
if held is not None and held.member_id ≠ member_id:
    raise NotAllowed(f"this copy is held for another member until {held.held_until}")

period = self._lending.loan_period_days(member)
loan = Loan(new_id(), barcode, member_id, today, today + timedelta(days=period))
item.status = ItemStatus.LOANED
self._loans.add(loan)
if held is not None:
    self._reservations.fulfil(held)
return loan
```

The held-copy check is the one people miss: a copy sitting on the hold shelf must not be handed to a walk-in, and the error message should say why.

```
def return_item(self, barcode: str, today: date) → "Return":
    loan = self._loans.open_for_barcode(barcode)
    loan.returned_on = today
    fine = self._fines.amount_paise(loan, today) # the policy decides the money

    next_in_queue = self._reservations.next_for(self._catalogue.item(barcode).isbn)
    item = self._catalogue.item(barcode)
    if next_in_queue is None:
        item.status = ItemStatus.AVAILABLE
    else:
        item.status = ItemStatus.HELD # not available – held
        next_in_queue.held_until = today + timedelta(days=3)
        self._notify(next_in_queue.member_id, item)
    return Return(loan, fine_paise=fine)
```

Two things at once, and both matter. The fine is computed by asking the policy. And the returned copy goes to `HELD` rather than `AVAILABLE` if anyone is waiting, with an expiry — because a hold with no expiry blocks the copy for ever.

Real systems

- **Koha** and **Evergreen** are the two big open-source library systems, and both have exactly this split: a *bibliographic record* (the title) and *items* (the copies), which in library standards are the MARC bibliographic record and the holdings record.
- **The circulation rules matrix** in Koha is literally a table of (member category × item type) → loan period, renewals, fine rate. That is `LendingPolicy` and `FinePolicy` as configuration rather than code — and it is what a mature version of this design turns into, because librarians change rules and cannot deploy software.
- **RFID and barcodes** are per-copy, which is another argument for `BookItem`: the physical world already has the distinction, and a design that lacks it cannot represent what the scanner reads.
- **Overdrive / Libby** for e-books deliberately *simulates* copies — a library buys a number of simultaneous licences and members queue for them — precisely so that the same circulation model keeps working. The industry chose to keep the copy concept even where it is artificial.

6 The numbers

Size, which decides the storage design

titles	50,000	
copies	80,000	(1.6 copies per title on average)
members	20,000	
active loans	12,000	(60% of members holding 1-2 books)
loans per year	500 per day × 300 days = 150,000	

Book	~400 B × 50,000	= 20 MB
BookItem	~200 B × 80,000	= 16 MB
Member	~300 B × 20,000	= 6 MB
Loan	~200 B × 150,000/yr	= 30 MB per year of history

current working set	≈ 42 MB, plus 30 MB per year of loans	

Everything except the loan history fits comfortably in memory. So the interesting storage question is not the catalogue, it is the loan table, which grows for ever — ten years is 1.5 million rows and 300 MB, still small, but it is the only thing that grows and that is worth saying.

Traffic

```
500 loans/day + 500 returns/day + ~2,000 searches/day
= 3,000 operations over a 10-hour day
= 0.08 operations per second
```

One operation every twelve seconds. This is not a scale problem, and saying so early stops the conversation drifting into caching and sharding. The design questions here are about *responsibility*, not throughput.

Fines, which is what the prompt is about

```
15% of loans returned late, averaging 4 days over
150,000 loans/year × 0.15 = 22,500 late returns
22,500 × 4 days × ₹2/day = ₹1,80,000 per year
```

And the reason the cap exists:

```
one loan forgotten for 3 years at ₹2/day = ₹2,190
the book cost ₹450
→ without a cap, the fine is 5x the value of the book, and nobody ever pays it
→ cap at the replacement cost, ₹500, and the debt is collectable
```

An uncapped fine is an uncollectable fine. That is a business rule with an arithmetic justification, and it is exactly the kind of thing that belongs in `FinePolicy` where it can be changed.

Closed days, which is why the calendar is a dependency

```
library closed: 52 Sundays + 15 public holidays = 67 days a year
67 / 365 = 18% of days
a 4-day overdue period spanning a weekend: 4 days charged vs 2 chargeable
→ the naive calculation overcharges by up to 50% on short overdues
```

Half of a typical fine, wrong, if you subtract dates naively. **That is the single strongest argument for the fine rule not living on `Loan`:** a loan has no idea when the library was shut.

Reservations

```
popular title: 6 copies, 40 members in the queue
average loan: 14 days
copies returning: 6 / 14 ≈ 0.43 per day
wait for position 40: 40 / 0.43 ≈ 93 days
```

Three months for the fortieth person. Which is why the hold window matters:

```
hold window 3 days, 20% of members do not collect  
wasted shelf time per copy cycle:  $0.20 \times 3 \text{ days} = 0.6 \text{ days}$   
across 6 copies over a year:  $6 \times (365/14) \times 0.6 \approx 94 \text{ copy-days lost}$ 
```

Ninety-four copy-days a year lost to uncollected holds, which is about a quarter of one extra copy. That is the number that justifies shortening the window or charging for a missed hold.

7 The trade-offs

What this design gives up

Two policies is two indirections. Answering “what does this member owe?” now means finding the policy implementation rather than reading a method. That is the standard cost of putting a rule behind an interface, and it is worth it here only because the rule genuinely varies — by tier, by calendar, by amnesty — and changes without any entity changing.

A policy in code still needs a deploy. Real libraries change fine rates by decree, on a Monday. The mature version of this design makes the policy **configuration** — a rules table keyed by member category and item type, which is exactly what Koha does — and the code becomes an interpreter of that table. I would build the interface first and move to a table when the second rate change arrives.

Fines are computed, not stored, and that is a decision with consequences. Computing on demand means a rate change silently re-prices historical debts. Storing the amount at return time means the number never moves but a bug in the policy is frozen into the record. The usual answer is: compute at the moment of return, **store the result on the return record**, and keep the policy for the calculation. Say which you chose.

The reservation queue is per title and ignores everything else. Strict FIFO is easy to explain and easy to defend, and it means a member who reserves six popular books ties up six queue positions. Real systems cap simultaneous reservations per member, which is another line in `LendingPolicy`.

No branches, no inter-library loans, no e-books. Multiple branches turn `BookItem` into something with a home branch and a current branch, and turn availability into a per-branch question. E-books have no copies at all, which either breaks the model or forces the licence-simulation trick real vendors use.

`Loan mutating returned_on` is convenient and slightly dishonest. A loan is a record of something that happened; mutating it means the object means different things at different times. The purer alternative is an immutable `Loan` plus a separate immut-

able `Return` record, which gives a clean audit trail at the cost of two lookups everywhere. For a library, mutation is fine; for anything financial, it would not be.

“I would change this design if...”

- **...there are multiple branches.** `BookItem` gains a branch, availability becomes per-branch, and transfers become a first-class operation with their own state.
- **...fine rules change more than about twice a year.** Then policies become a configuration table that a librarian edits, not classes an engineer deploys.
- **...e-books are added.** No copies, no fines, automatic expiry. Either a separate lending path, or the licence-count simulation.
- **...members can be in more than one queue for the same title,** or queues need priority — research staff before undergraduates. Then FIFO becomes a priority queue and I would want the rule written down before building it.

The honest concession

Two interfaces in a system with nine classes is a lot of ceremony, and I would want to defend each one. `FinePolicy` earns it because the fine depends on four things that are not on the loan — tier, rate, cap, calendar — and because a second implementation, the amnesty, is real rather than hypothetical. `LendingPolicy` earns it for the same reason with student and staff. If the library had one member type and one flat rate, both would be a method on the service and I would say so.

8 In the interview

How it gets asked

- The standard: “*Design a library management system.*” Then, within a few minutes: “*Who owns the rule about late fees?*”
- The modelling probe, which is the good one: “*You have six copies of the same book. Model that.*”
- The reservation probe: “*A member wants a book that is out. What happens when a copy comes back?*”
- The responsibility probe: “*Why not just put `calculate_fine()` on the `Loan` class?*”
- The concurrency probe: “*Two members try to borrow the last copy at the same moment.*”

The timed script

Minutes 0–5 · Clarify. Physical only? Reservations? Member tiers? Fines on closed days? State the assumptions and the exclusions.

Minutes 5–10 · The modelling insight, early. “The first thing I want to settle is that a book is two things — the title and the physical copy — because reservations queue for the title and loans are of a copy, and merging them breaks both.” Getting this out early sets the tone.

Minutes 10–18 · The classes. Nine, with one-line responsibilities. Draw the two arrows that go to different classes: `Loan → BookItem`, `Reservation → Book`.

Minutes 18–30 · The deep dive: who owns the fine rule. Walk through each candidate home and reject it with a reason. Land on `FinePolicy`, and show the constructor with the member directory and the holiday calendar — those two dependencies are the argument. Then draw the line: `days_overdue` on the loan, `rupees_owed` in the policy.

Minutes 30–36 · Reservations and the hold state. The queue on the title, the hold window, and what happens when it expires.

Minutes 36–40 · Numbers and failure. One operation every twelve seconds, so this is not a scale problem. The 18-percent-closed-days figure. The fine cap argument. Then the last-copy race.

The follow-ups

“Why not put `calculate_fine()` on `Loan`?” “Because to do it correctly the loan would need four things it has no business knowing: the daily rate, which varies by member tier; the cap; the holiday calendar, because we do not charge for days we were shut; and today’s date. All four change without any loan changing. So the loan holds the *facts* — who, what, when it was due, when it came back — and it answers questions about itself, like `days_overdue`. A `FinePolicy` interprets those facts into money. The tell is in the constructor: the policy needs a member directory and a calendar, and a `Loan` that held those would be doing somebody else’s job.”

“Is that not an anaemic model?” “It would be if `Loan` were only fields. It is not — it answers `is_overdue`, `days_overdue`, `is_returned` and `can_renew`, which are all facts about itself that need nothing external. The line I draw is: *the loan owns questions about the loan; the policy owns questions about money*, because money needs rates and calendars. `days_overdue` on the loan, `rupees_owed` in the policy.”

“You have six copies of the same book. Model that.” “Two classes. `Book` is the title — ISBN, author, subject — and there is one per title. `BookItem` is a physical copy with its own barcode, rack, condition and status, and six of those point at one `Book`. The payoff is one sentence: reservations queue for the title, loans are of a copy. You also get honest answers to ‘do you have it’ versus ‘can I take it home’, and you can mark one copy as damaged without affecting the others. Merging them is the standard mistake here and it breaks reservations, availability and reporting at the same time.”

“A member wants a book that is out. What happens?” “They join a FIFO queue on the title. When any copy of that title is returned, the copy does not go back to available — it goes to a distinct `HELD` state and is held for the member at the front of the queue for a fixed window, say three days, and they are notified. If they do not collect it, the hold expires, they lose their place, and it is offered to the next person. The expiry is the part that matters: without it, one member who never comes in blocks the copy indefinitely. And `HELD` has to be its own state, or a walk-in gets handed a copy that was promised to someone else.”

“Two members try to borrow the last copy at the same moment.” “Inside one process, the check-then-issue is two steps and needs a lock around them. But the real answer is at the storage layer, and it is a conditional update on the copy: set status to loaned where the barcode matches *and* the status is still available. Zero rows updated means somebody else won and that member gets ‘sorry, just gone — shall I reserve it?’. That is the same pattern as the parking spot and the shipped order: **the two-step operation is find-then-claim, and the claim must be conditional**. At one operation every twelve seconds this will almost never fire, and it must still be correct.”

“How would you handle a rate change?” “Two questions hidden in that. For future loans, it is a new `FinePolicy` implementation or a new row in a rules table, and I would move to the table version the second time a rate changes, because librarians change rules and cannot deploy software — that is what Koha’s circulation rules matrix is. For loans already returned, I would not want a rate change to silently re-price history, so I compute the fine at the moment of return and *store the amount* on the return record. The policy computes; the record remembers.”

“How big does this get?” “Small. Fifty thousand titles, eighty thousand copies, twenty thousand members is about forty megabytes — the whole catalogue fits in memory. The only thing that grows without bound is the loan history, at about thirty megabytes a year. And the traffic is roughly one operation every twelve seconds, so there is no scale problem here at all. The design questions are about responsibility, not throughput, and I would say that early so we spend the time on the right thing.”

A model answer

Asked: design a library system. Who owns the rule about late fees?

“Let me settle one modelling question first, because everything else depends on it. A ‘book’ is two different things and they need two classes. There is the *title* — ISBN, author, subject — and there is the *physical copy*, with its own barcode, its own rack, its own condition, its own borrower. Six copies point at one title.

The reason that matters, in one sentence: **reservations queue for the title, and loans are of a copy**. If those are one class, you cannot express either properly. You also get honest answers to the two questions a member is really asking — do you have it, which is about the title, and can I take it home, which is about the copies — and you can mark one copy as damaged without touching the other five.

From there the nouns fall out. `Book`, `BookItem`, `Member`, `Loan`, `Reservation`, a `Catalogue` for lookups, and a `LendingService` as the entry point.

Now the question you actually asked. Where does the late-fee rule live?

Not on `Book` — a fine is not a property of a novel. Not on `BookItem`, for the same reason and worse. Not on `Member` — a person does not know the daily rate.

And, I would argue, not on `Loan` either, even though that is the tempting answer. To compute a fine correctly you need four things: the daily rate, which varies by member tier; the maximum cap; the holiday calendar, because we do not charge for days the library was shut; and today’s date. Every one of those changes without any loan changing, and none is something a record of a borrowing should know. The concrete tell is the constructor — the thing that computes a fine needs a member directory and a calendar, and a `Loan` holding those is doing somebody else’s job.

So there is a `FinePolicy`, one method: given a loan and a date, return an amount. The loan holds the facts — who, what, when it went out, when it was due, when it came back — and answers questions about *itself*: is it overdue, how many days late, can it be renewed. The policy turns facts into money.

The line I would state explicitly, because it generalises: **`days_overdue` belongs on the loan; `rupees_owed` does not**. That also answers the anaemic-model objection — the loan is not just fields, it just does not own pricing.

Two numbers that support this. The library is closed 52 Sundays and about 15 holidays, so 18 percent of days, and a naive date subtraction overcharges a typical four-day overdue by up to half. And without a cap, a book forgotten for three years accrues about two thousand rupees of fine on a four-hundred-rupee book, which nobody ever pays — so the cap is set at replacement cost, and an uncapped fine is an uncollectable one. Both of those rules live in the policy, which is exactly where a rule that changes belongs.

There is a matching policy on the way out — how many books and for how long, by member tier — and both pass the same gate: I can name a real second implementation. For fines, a festival amnesty, which I would write as a wrapper around the standard policy rather than a branch inside it. For lending, student and staff. Nothing else in this design gets an interface.

Two more things I would raise unprompted. Reservations are a FIFO queue on the title, and when a copy comes back it goes into a distinct `HELD` state — not available — for three days, and the hold expires if nobody collects it, or one member who never comes in blocks a copy for ever. And the last copy is a find-then-claim race: two members can both see it as available, so the claim must be a conditional update — set it to loaned only if it is still available — and zero rows updated means the other person won.

On size: the whole catalogue is about forty megabytes and the traffic is roughly one operation every twelve seconds. This is not a scale problem, and I would rather spend the time on responsibilities, which is where the actual difficulty is.”

9 Recall card

- A “book” is two classes, and this is the modelling insight the round is testing. `Book` is the **title** (one per ISBN); `BookItem` is a **physical copy** (barcode, rack, condition, status). The payoff sentence: **reservations queue for the title, loans are of a copy** — and on the diagram those two arrows go to *different classes*.
- The fine rule lives in a `FinePolicy`, and you win the question by rejecting each wrong home with a reason. Not `Book / BookItem` (a fine is not a property of a novel) · not `Member` (a person does not know rates) · **not** `Loan` **either**, because computing a fine needs the **tier rate, the cap, the holiday calendar and today’s date** — four things that change without any loan changing. The tell is the constructor: the policy needs a member directory and a calendar.
- Draw the line: `days_overdue` **belongs on the loan**; `rupees_owed` **does not**. The loan is not anaemic — it answers `is_overdue`, `days_overdue`, `can_renew` from its own fields. It just does not own money. Matching policy on the way out: `LendingPolicy` for limits and periods per tier.
- `HELD` is a real state, distinct from available and loaned, and holds must **EXPIRE**. Return a copy → if the queue is empty it goes available, otherwise it is **held for the front of the queue for ~3 days** and then offered to the next member. No expiry means one absent member blocks a copy for ever; no `HELD` state means a walk-in gets a promised copy.

- **The numbers that justify the design rather than decorate it.** 50k titles + 80k copies + 20k members $\approx$ **42 MB, all in memory**; traffic $\approx$ **one operation every 12 seconds — not a scale problem.** The library is shut **18% of days**, so naive date subtraction overcharges a short overdue by up to **50%.** **An uncapped fine is uncollectable** — ₹2,190 of fine on a ₹450 book — so cap at replacement cost. The last copy is a **find-then-claim** race: the claim must be a **conditional update.**

PRACTICE

Practice — Finding the middle, and the runner technique

DSA topic: Finding the middle, and the runner technique **System design topic:** Design a library management system

Code these, in this order

One rule for the whole set: **before writing the loop, say which runner variant you need — different speeds, or a fixed gap.** They solve different problems and mixing them up wastes five minutes you do not have.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Middle of the Linked List	LeetCode 876 (Easy)	The two-guard loop condition, and whether you raise the even-length ambiguity before being told.
2	Remove Nth Node From End of List	LeetCode 19 (Medium)	The fixed gap, the dummy putting <code>slow</code> on the predecessor, and <code>n</code> larger than the list.
3	Palindrome Linked List	LeetCode 234 (Easy)	Three techniques in one, and restoring the caller's list afterwards.
4	Reorder List	LeetCode 143 (Medium)	Split, reverse, interleave — and the interleave loop's terminating condition.

On problem 1, ask the question out loud first

“On an even-length list there are two middles — which do you want?” Then write the version they asked for, and afterwards write the other one and confirm both by running `[1,2,3,4]`.

On problem 2, do the arithmetic without the dummy once

Write it with `slow = head` and a gap of `n + 1`, and handle the head separately. Count the branches. Then write the dummy version. Say the two distinct jobs the dummy did.

On problem 3, print the list after returning

Every test that only checks the boolean passes whether or not you restored the list. Add a print of the original list after the call and make it match the input. That is the actual requirement.

On problem 4, write the `interleave` slowly

The loop that weaves two half-lists together is four assignments and it is easy to build a cycle. Draw the pointers for `[1, 2, 3]` and `[5, 4]` before writing, and check the last node points at `None`.

The two-variants drill

1. State both runner variants in one sentence each.
2. For each, say what is true when the fast pointer stops.
3. Give two problems solved by the speed variant and two by the gap variant.
4. Say which variant tomorrow's cycle detection uses, and what changes about the conclusion.

The condition drill

1. Write the middle loop condition and say what each half protects against.
2. Swap the two halves and run it on `[1, 2, 3, 4]`. Quote the error.
3. Write the other condition and say which middle it gives.
4. Run both on `[1, 2, 3, 4]` and on `[1, 2, 3, 4, 5]`. Write all four answers down.
5. Say which one a merge sort needs, and what happens if you use the other.

The gap drill

1. Write the opening walk with its bounds check.
2. Remove the check and run it with `n` larger than the list. Quote the error.
3. Say the two reasonable behaviours for that case and pick one.
4. Explain why the gap never changes once the second loop starts.
5. Say where `slow` must start if you want the predecessor, and why.

The break-it drill

Trigger each and record the exact output or error text:

1. Loop condition in the wrong order. Run `[1, 2, 3, 4]`.
2. Use the second middle to split for a merge sort. Run `[1, 2]` and describe the failure.
3. Forget `middle.next = None` in the split. Print both halves.
4. Start `slow` at the head instead of the dummy in `remove_nth_from_end`. Run `[1, 2, 3, 4, 5]` with `n = 2` and say which node actually disappeared.
5. Skip the restore in `is_palindrome`. Print the original list afterwards.
6. In `reorder`, forget to terminate the woven list. Describe the failure.

The complexity-honesty drill

1. Count the pointer moves for the one-pass middle.
2. Count them for the two-pass version.
3. State both complexities.
4. Say, in one sentence, what one pass actually buys — and what it does not.
5. Say what makes one traversal genuinely better than two on a linked list specifically.

The composition drill

Write `reorder` from scratch using only functions you already have, then answer:

1. Which day did each of the three pieces come from?
 2. Which middle does the split need, and why?
 3. What is the terminating condition of the interleave, and why is it the shorter half?
 4. Confirm the final node points at `None` and say how you checked.
-

The modelling drill

1. Say, in one sentence, why “a book” needs two classes.
2. List four fields that belong on the title and four that belong on the copy.
3. Say which class a reservation points at, and which a loan points at.
4. Give three questions the split answers cleanly that a merged class cannot.
5. Name three other domains with the same title-versus-instance split.

The responsibility drill

For each candidate home for the late-fee rule, give the one-sentence rejection:

1. `Book`
2. `BookItem`
3. `Member`
4. `Loan`
5. `LendingService`

Then state where it does belong, and name the two constructor dependencies that prove it.

The anaemic-model drill

1. List four methods that belong on `Loan`.
2. List two that do not, and say why.
3. State the boundary as a single sentence containing `days_overdue` and `rupees_owed`.

4. Say what would make `Loan` genuinely anaemic, and why this version is not.

The policy drill

1. Write the `FinePolicy` interface.
2. Write the standard implementation, including the closed-days handling.
3. Write the festival amnesty as a *wrapper* rather than a branch. Name the pattern.
4. Apply the interface gate to both `FinePolicy` and `LendingPolicy` and say what a second implementation is for each.
5. Say when you would move these from classes to a configuration table, and why.

The reservation drill

1. Say which class the queue is attached to and why.
2. Walk through what happens when a copy is returned and someone is waiting.
3. Say why `HELD` must be a separate state from both available and loaned.
4. Say what goes wrong with no hold expiry.
5. Compute the wait for position 40 in a queue for a title with 6 copies and a 14-day loan.
6. Compute the copy-days lost per year to uncollected holds at a 20 percent no-show rate.

The estimation drill

Compute each, showing the multiplication:

1. Total memory for titles, copies and members.
2. Loan history growth per year.
3. Operations per second across loans, returns and searches.
4. Annual fine revenue at 15 percent late, 4 days average, ₹2 a day.
5. The fine on a book forgotten for three years, uncapped, against the book's cost.
6. The percentage of days the library is closed, and the overcharge on a four-day overdue.

Then say which two numbers you would quote in the first ten minutes of the interview, and why.

The concurrency drill

1. Name the two-step operation that races when two members want the last copy.
2. Write the conditional update that fixes it.
3. Say what zero rows updated means and what the member sees.

4. Say why this is the same pattern as the parking spot and the shipped order.
 5. Say roughly how often it will fire at this system's traffic, and why you build it anyway.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Remove the Nth node from the end in one pass.* The two-pass baseline named and set aside, the gap variant with the invariant in one sentence, the dummy doing two jobs, the bounds check on the opening walk, the honest complexity claim, and the three inputs you would run.
 2. *Design a library system. Who owns the rule about late fees?* The title-versus-copy split stated first with the one-sentence payoff, each wrong home for the fine rule rejected with a reason, `FinePolicy` with its two telling dependencies, the `days_overdue / rupees_owed` boundary, the closed-days and uncapped-fine numbers, and the hold state with its expiry.
 3. *Why not just put `calculate_fine()` on the `Loan` class?* The four things it would need that are not on the loan, all of which change without any loan changing, and the constructor test that makes it concrete.
-

Before you move on

- [] I can state both runner variants and what is true when the fast pointer stops.
- [] I can write the middle's loop condition and say what each half protects against.
- [] I ran both conditions on `[1,2,3,4]` and know which middle each gives.
- [] I know which middle a merge sort needs and what happens with the other one.
- [] I forgot the cut on purpose and saw two halves that were really one list.
- [] I wrote `remove_nth_from_end` without the dummy and counted the extra branches.
- [] I can say what one pass actually buys, without overclaiming.
- [] I wrote the palindrome check and confirmed the list was unchanged afterwards.
- [] I can say in one sentence why a book needs two classes.
- [] I can reject all four wrong homes for the fine rule, each with a reason.
- [] I can name the two constructor dependencies that prove the policy split.
- [] I can state the `days_overdue / rupees_owed` boundary.
- [] I wrote the amnesty as a wrapper and named the pattern.

- ☐ I can explain the `HELD` state and what breaks without a hold expiry.
- ☐ I can quote the closed-days percentage and the uncapped-fine arithmetic.
- ☐ I answered all three questions above out loud.

Cycle detection, and why Floyd's algorithm works

DATA STRUCTURES AND ALGORITHMS

Cycle detection, and why Floyd's algorithm works

You can detect a cycle, find where it starts, and explain the two-pointer proof simply.

SYSTEM DESIGN

Design tic-tac-toe, and then chess

You can design a board game so the rules live in one replaceable place.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Detect a cycle in a linked list. Now find the node where it begins.*
- *Design tic-tac-toe. Now make it work for an N by N board.*

Cycle detection, and why Floyd's algorithm works

1 What this is, and why they ask it

A linked list has a **cycle** if some node's `next` points back at a node you have already visited. Then the list has no end: walking it never reaches `None`, and any loop written the ordinary way runs for ever.

Floyd's cycle detection — the tortoise and the hare — finds out in $O(n)$ time and $O(1)$ space. Send one pointer forward one step at a time and another two steps at a time. If the list ends, the fast one falls off. If it loops, the fast one comes round and catches the slow one from behind, and **that meeting is the proof**.

Then comes the part that makes this a real interview question rather than a trick: *find the node where the cycle begins*. The answer is three lines and looks like magic — put one pointer back at the head, move both one step at a time, and they meet exactly at the entrance. You will be asked why it works, and “I memorised it” is a visible answer. The arithmetic is four lines and worth having.

They ask it because the hash-set solution is obvious and the $O(1)$ -space solution is not, because the proof separates people who understand from people who recall, and because the same algorithm answers questions that are not about linked lists at all — *find the duplicate number in an array* (LeetCode 287) is this algorithm in disguise.

2 The story

The park behind the water tank has one path in and one loop, and Ravi has walked it most mornings for eleven years.

From the gate a straight path runs past the swings to a junction by the big tree — three hundred metres, give or take. At the junction the loop begins, and the loop is about eight hundred metres and comes back to that same tree. So “doing the track” means walking out along the path and then round, and round, and round.

Manju joined him about two years ago, and Manju does not walk. He jogs, at roughly twice Ravi's pace, which they established once by counting steps and arguing.

They set off together from the gate and they do not stay together. Manju pulls away, disappears round the loop, and then — this is the part Ravi still finds slightly annoying — comes up behind him. Not towards him. **Behind him**. Somewhere on the far side of

the loop, most mornings near the broken bench, Manju taps his shoulder on the way past.

Ravi's daughter pointed out what that actually means, one Sunday when she came along.

If the path had no loop — if it just ended at a wall — Manju would reach the wall and that would be that. He would be standing there waiting. He could never come up behind Ravi, because there is no behind. The only way a faster person catches a slower one **from behind** is if the route comes back on itself.

So the tap on the shoulder is not a coincidence of pace. It is a fact about the shape of the park.

The second thing they worked out took them three attempts and they still cannot explain why it works, though Ravi's daughter did the numbers on the back of her hand and said it was obvious.

They wanted to know exactly where the loop begins — which tree, precisely, because there are two big trees near each other and they had been arguing about it for a year.

What she told them to do was this. Manju stays exactly where he tapped Ravi's shoulder. Ravi walks back to the gate. Then both of them set off at the **same** speed — Manju continuing round the loop, Ravi coming in along the path from the gate.

They meet at the junction. Every time. The correct tree, settled.

3 The idea in plain English

The park is the shape every list with a cycle has: **a straight bit, then a loop**. People draw it as the Greek letter rho, ρ — a tail and a circle.

Phase one: is there a loop at all?

```
slow = fast = head
while fast is not None and fast.next is not None:
    slow = slow.next
    fast = fast.next.next
    if slow is fast:
        return True           # the tap on the shoulder
return False                 # fast fell off the end: no loop
```

The same two runners as [day 082](#), asking a different question. There are exactly two possible outcomes:

- **fast reaches None**. The route ends. No cycle. This is the wall.

- **fast catches slow**. They are on the same node. There is a cycle — and there must be, because a faster runner can only come up *behind* a slower one on a route that returns to itself.

Compare with `is`, not `=`. You are asking whether they are on the same *node*, not whether two nodes hold the same value. `=` on two different nodes with equal values would report a cycle that does not exist.

Why they must meet, rather than skip past each other

The one gap in the obvious argument: could the fast runner jump *over* the slow one and keep missing?

No, and the reason is simple. Once both are inside the loop, look at the distance from **fast** to **slow** going forwards round the loop. Each step, **slow** advances one and **fast** advances two, so that gap **shrinks by exactly one every step**. A gap that decreases by exactly one can never jump from 1 to -1 ; it must pass through 0. So they land on the same node.

That is the whole argument and it fits in two sentences. It is also why the speed ratio must be 2 and 1 rather than, say, 3 and 1 — with a gap shrinking by two each step, they could indeed skip past each other on a cycle of odd length.

Phase two: where does the loop begin?

```
finder = head
while finder is not slow:
    finder = finder.next
    slow = slow.next
return finder                # the junction
```

Manju standing still while Ravi walks back to the gate, then both moving at the same pace. Three lines, and the interviewer will ask why.

The proof, in four lines of arithmetic

Name three distances, in nodes:

a = from the head to the start of the cycle	(the path from the gate)
b = from the start of the cycle to the meeting	(round to the broken bench)
c = the length of the cycle	(once round)

When they meet:


```
slow has walked    a + b
fast has walked    a + b + kc           - the same place, having gone k full laps more
fast walked twice as far:
  2(a + b) = a + b + kc
    a + b = kc
      a = kc - b
```

Read that last line as an instruction. **From the meeting point, walking a more steps takes you $b + a = kc$ steps into the cycle — a whole number of laps — which lands you exactly back at the start of the cycle.** And walking a steps from the head also lands you at the start of the cycle, by definition of a .

So two pointers, one at the head and one at the meeting point, both moving one step at a time, arrive at the cycle's start at the same moment. That is the whole proof, and you can say it in thirty seconds.

The common case $k = 1$ makes it concrete: $a = c - b$. The distance from the head to the junction equals the distance from the meeting point round to the junction. Ravi's three hundred metres of path equals the remaining stretch of loop from the broken bench back to the tree.

The length of the cycle, if asked

Stay at the meeting point and walk until you come back to it, counting:

```
length, node = 1, meeting.next
while node is not meeting:
    node = node.next
    length += 1
```

$O(c)$, no extra space.

The obvious solution, named and rejected

```
seen = set()
node = head
while node is not None:
    if node in seen:           # by IDENTITY - nodes are unhashable by value here
        return node
    seen.add(node)
    node = node.next
```

Correct, $O(n)$ time, **$O(n)$ space**, and it gives you the cycle's start for free with no proof required. Say it first, then say why you are not writing it: at a million nodes that is a set holding a million references — roughly 32 MB — to answer a question that needs two variables.

It is not a bad answer. It is the answer you give if you cannot recall Floyd's, and it is far better than nothing. Volunteer it as the baseline, and then improve it.

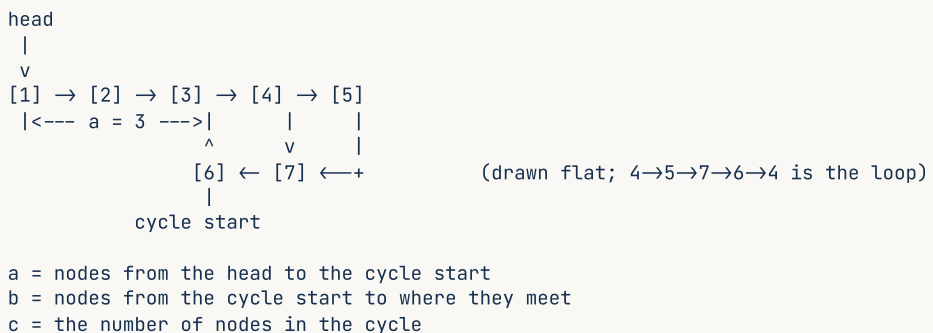
Where this shows up when it is not a linked list

Find the Duplicate Number (LeetCode 287): an array of $n + 1$ integers, each between 1 and n , with exactly one value repeated. Treat the array as a function — from index i , go to index $\text{nums}[i]$. Because values are in $1..n$, following that never leaves the array, so the walk must eventually repeat: it is a rho shape. The duplicate value is the entrance to the cycle, and Floyd's finds it in $O(1)$ space without modifying the array. That reframing is a genuinely hard leap and it is worth knowing it exists.

Happy Number (LeetCode 202) is the same idea over “replace the number with the sum of the squares of its digits”.

4 The picture

The rho shape, with the three distances marked.



Now the walk itself, on a list of six nodes where the cycle starts at node 3 and has length 4:

```

nodes:  1 → 2 → 3 → 4 → 5 → 6 -+
              ^               |
              +-----+

```

step	slow	fast	
0	1	1	
1	2	3	
2	3	5	
3	4	3	fast has lapped
4	5	5	MEET at node 5

a = 2 (nodes 1, 2 before the cycle)
 b = 2 (from node 3 to node 5)
 c = 4 (nodes 3, 4, 5, 6)
 check: a + b = 4 = kc with k = 1. Correct.

phase two:
 finder = head = 1, slow stays at 5
 step 1: finder = 2, slow = 6
 step 2: finder = 3, slow = 3 MEET at node 3 = the cycle start

What to notice: they meet at node 5, which is **not** the cycle start, and phase two walks exactly `a = 2` more steps to reach it. The meeting point is almost never the entrance, and any solution that returns it is wrong on most inputs.

And the reason a faster runner cannot jump over a slower one:

distance from fast to slow, going forwards round the loop:

step 0:	gap = 3	
step 1:	gap = 2	slow +1, fast +2 → gap shrinks by exactly 1
step 2:	gap = 1	
step 3:	gap = 0	they are on the same node

a gap that decreases by exactly one cannot skip 0.

5 The code, built step by step

Step 1 — detection, and the comparison that matters

```

slow = fast = head
while fast is not None and fast.next is not None:
    slow = slow.next
    fast = fast.next.next
    if slow is fast:
        return True
return False

```

Advance **first**, then compare. Comparing before advancing would report a cycle immediately, since both start on the head. And `is`, not `=`: you are asking about node identity.

Step 2 — the loop condition, which is the same two guards as yesterday

`fast is not None` covers an even-length tail where `fast` lands exactly on `None`; `fast.next is not None` covers an odd one where `fast.next.next` would raise. The null check must come first because `and` short-circuits.

Step 3 — phase two, returned from phase one

```
finder = head
while finder is not slow:
    finder = finder.next
    slow = slow.next
return finder
```

Note there is no `while True` and no length check — the arithmetic guarantees they meet, and they meet within `a` steps. If your phase two has a safety counter in it, you do not believe your own proof.

Step 4 — restructuring so both phases live in one function

```
def find_cycle_start(head):
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            break                # meeting point found; slow is standing on it
    else:
        return None              # the loop ended normally: no cycle
```

The `while ... else` runs the `else` only when the loop finished **without** `break`, which is exactly “no cycle”. It is a real Python feature, it is the right tool here, and using it deliberately with a one-line explanation reads well. If it makes you uncomfortable, a boolean flag is fine.

The complete solution

```
class Node:
    __slots__ = ("value", "next")

    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value
        self.next = next

    def __repr__(self) → str:
        return f"Node({self.value})"

def build_with_cycle(values: List[int], cycle_at: int | None = None) → Node | None:
    """Build a list, optionally joining the tail back to index `cycle_at`."""
    if not values:
        return None
    nodes = [Node(value) for value in values]
    for first, second in zip(nodes, nodes[1:]):
        first.next = second
    if cycle_at is not None:
        nodes[-1].next = nodes[cycle_at]
    return nodes[0]

def has_cycle(head: Node | None) → bool:
    """Floyd's tortoise and hare. O(n) time, O(1) space.

    A faster runner can only catch a slower one FROM BEHIND on a route that
    returns to itself. Once both are inside the loop, the forward gap between
    them shrinks by exactly one each step, so it cannot skip past zero.
    """
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next          # advance first...
        if slow is fast:              # ...then compare, by IDENTITY
            return True
    return False

def find_cycle_start(head: Node | None) → Node | None:
    """The node where the cycle begins, or None.

    Why phase two works, in four lines:
    a = head to cycle start, b = cycle start to meeting, c = cycle length
    slow walked a + b ; fast walked 2(a + b) = a + b + kc
    so a + b = kc, therefore a = kc - b
    ⇒ from the meeting point, a more steps lands exactly on the start
    """
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            break
    else:
        return None                    # fell off the end: no cycle

    finder = head
    while finder is not slow:          # guaranteed to meet within `a` steps
        finder = finder.next
    slow = slow.next
    return finder

def cycle_length(head: Node | None) → int:
    """The number of nodes in the cycle, or 0 if there is none."""
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            length, node = 1, slow.next
```

```

        while node is not slow:
            node = node.next
            length += 1
        return length
    return 0

def has_cycle_with_a_set(head: Node | None) → Node | None:
    """The obvious solution: O(n) time, O(n) SPACE. Name it, then improve on it.

    At a million nodes this set holds a million references – about 32 MB – to
    answer a question that needs two variables.
    """
    seen: set[int] = set()
    node = head
    while node is not None:
        if id(node) in seen:
            return node
        seen.add(id(node))
        node = node.next
    return None

def find_duplicate(numbers: list[int]) → int:
    """LeetCode 287, which is this algorithm in disguise.

    n + 1 values, each in 1..n, exactly one repeated. Read the array as a
    function: from index i, go to index numbers[i]. Values stay in range, so
    the walk can never leave the array and must eventually repeat – a rho
    shape. The repeated VALUE is the entrance to the cycle.

    O(n) time, O(1) space, and the input is never modified.
    """
    slow = fast = numbers[0]
    while True:
        slow = numbers[slow]
        fast = numbers[numbers[fast]]
        if slow == fast:
            break

    finder = numbers[0]
    while finder != slow:
        finder = numbers[finder]
        slow = numbers[slow]
    return finder

if __name__ == "__main__":
    straight = build_with_cycle([1, 2, 3, 4, 5])
    looped = build_with_cycle([1, 2, 3, 4, 5, 6], cycle_at=2) # cycle starts at node 3

    print(has_cycle(straight), has_cycle(looped)) # False True
    print(has_cycle(None), has_cycle(build_with_cycle([1]))) # False False

    print(find_cycle_start(straight)) # None
    print(find_cycle_start(looped)) # Node(3)
    print(cycle_length(looped), cycle_length(straight)) # 4 0

    self_loop = build_with_cycle([1], cycle_at=0) # a node pointing at itself
    print(has_cycle(self_loop), find_cycle_start(self_loop), cycle_length(self_loop))
    # True Node(1) 1

    whole_loop = build_with_cycle([1, 2, 3], cycle_at=0) # the entire list is the cycle
    print(find_cycle_start(whole_loop), cycle_length(whole_loop)) # Node(1) 3

    print(has_cycle_with_a_set(looped)) # Node(3) – same answer, O(n) space

    print(find_duplicate([1, 3, 4, 2, 2])) # 2
    print(find_duplicate([3, 1, 3, 4, 2])) # 3
    print(find_duplicate([2, 2, 2, 2, 2])) # 2

    # the two implementations must agree, on every shape
    import random

```

```

for _ in range(2000):
    size = random.randint(1, 12)
    values = list(range(size))
    at = random.choice([None] + list(range(size)))
    head = build_with_cycle(values, at)
    expected = None if at is None else head
    for _ in range(at or 0):
        expected = expected.next
    assert find_cycle_start(head) is expected
    assert has_cycle_with_a_set(head) is expected
print("Floyd's and the set version agreed on 2000 random shapes")

```

The three edge cases in the middle are the ones interviewers reach for: a single node pointing at itself, and a list where the entire thing is the cycle so `a = 0` and phase two must return the head immediately without moving.

6 What it costs

Time

Phase one. Before either pointer enters the cycle, `slow` walks `a` steps. Once both are in the cycle, the gap between them shrinks by one per step, and the gap is at most `c`, so they meet within `c` more steps. So phase one is at most `a + c` iterations of `slow`, and $a + c \leq n$.

```

phase one:  ≤ a + c steps   →  O(n)
phase two:  exactly a steps →  O(n)
total:      0(n)

```

Say it as “at most `a + c`, which is at most `n`” rather than waving at $O(n)$. The bound on phase one being `c` and not something worse is the part that needs the shrinking-gap argument.

Space

```

slow, fast, finder:  three references →  O(1)

```

That is the entire point of the algorithm, and it is what you are being asked for. Against the set version:

```

n = 1,000,000
set of node references: ~32 MB
Floyd's:               three variables

```

Against the alternatives

hash set of nodes	$O(n)$ time,	$O(n)$ space,	start is free, no proof needed
Floyd's	$O(n)$ time,	$O(1)$ space,	start needs 3 more lines and a proof
mark visited nodes	$O(n)$ time,	$O(1)$ space,	but MUTATES the input – usually forbidden

The third one is worth naming and rejecting: adding a `visited` flag to each node, or reversing links as you go, works and destroys the caller's data. If the interviewer says "you may modify the list", it becomes a legitimate answer.

The duplicate-number version

```
array of n+1 values in 1..n
sorting:           0(n log n) time, and it mutates the array
a set of seen values: 0(n) time, 0(n) space
marking with negatives: 0(n) time, 0(1) space, but mutates
Floyd's on index → value: 0(n) time, 0(1) space, no mutation
```

Only the last one satisfies all three constraints the problem states, which is why the problem is stated with all three.

7 The traps

Trap 1 – comparing values instead of identity

```
if slow == fast:                # WRONG for arbitrary node objects
```

`=` on two different `Node` objects falls back to identity in Python *unless* someone has defined `__eq__` — and the moment somebody does, on values, this reports a cycle whenever two nodes hold the same number. **Write** `is`. It is also clearer about what you mean.

Trap 2 – comparing before advancing

```
while fast is not None and fast.next is not None:
    if slow is fast:
        # both start on the head
        return True
    slow, fast = slow.next, fast.next.next
```

Returns `True` for every non-empty list, immediately. Advance first, then compare.

Trap 3 — returning the meeting point as the cycle start

```
if slow is fast:
    return slow                # this is NOT the entrance
```


On the six-node example the meeting is at node 5 and the entrance is node 3. It happens to be right only when `a = 0`, meaning the whole list is one cycle — which is exactly the test case people use, so this bug survives casual testing.

Trap 4 — the loop condition in the wrong order

```
while fast.next is not None and fast is not None:
```

```
AttributeError: 'NoneType' object has no attribute 'next'
```

on any list without a cycle whose length is even. `and` short-circuits left to right, so the null check must be first. Same trap as [day 082](#), and it will keep being the same trap.

Trap 5 — a safety counter in phase two

```
steps = 0
while finder is not slow and steps < 1_000_000:    # do not do this
```

The arithmetic guarantees they meet within `a` steps. Adding a guard says you do not trust the proof, and if phase one was correct the guard can never fire. If you *want* a guard, put an `assert` on the result instead — that documents the invariant rather than hiding a bug.

Trap 6 — assuming the fast pointer cannot jump over the slow one

The most common gap in an otherwise good explanation. It cannot, and the reason is that the forward gap shrinks by **exactly one** each step, so it must pass through zero. Note this is specific to the 2:1 ratio: at 3:1 the gap shrinks by two and could indeed step from 1 to -1 on an odd-length cycle. Being able to say that is a genuine sign of understanding.

Trap 7 — modifying the list to detect the cycle

Marking nodes as visited, or reversing links as you walk, both work and both destroy the caller's data. If the list is shared, or the caller uses it afterwards, this is a serious bug and it is invisible in a test that only checks the boolean. Ask before mutating.

Trap 8 — the two degenerate shapes

A single node pointing at itself: `a = 0`, `b = 0`, `c = 1`, and phase two must return the head without moving. A list where the entire thing is the cycle: `a = 0` again. Both are the cases where “return the meeting point” accidentally works, so run them last to catch that bug rather than first.

8 In the interview

How it gets asked

- The base: *“Does this linked list have a cycle?”* LeetCode 141. Usually five minutes.
- The real question: *“Now return the node where the cycle begins.”* LeetCode 142 — and then *“why does that work?”*
- The constraint version: *“Can you do it without extra space?”* — which is the hint that the set solution is not what they want.
- The disguised version, which is genuinely hard: *“Given an array of $n + 1$ integers each between 1 and n , find the duplicate, without modifying the array and in $O(1)$ space.”* LeetCode 287.
- The proof probe: *“Why must they meet? Could the fast one jump over the slow one?”*

What to say out loud, in the first ninety seconds

1. **Give the $O(n)$ -space answer first, as the baseline.** “The obvious solution is a set of visited nodes — $O(n)$ time and $O(n)$ space, and it gives me the entrance for free. Let me improve the space.”
2. **State the shape.** “A list with a cycle is a tail followed by a loop — a rho shape. So either a walk falls off the end, or it goes round for ever.”
3. **State the detection rule and why, not just the mechanics.** “Two pointers, one at single speed and one at double. If the list ends, the fast one falls off. If it loops, the fast one comes up *behind* the slow one — and a faster runner can only catch a slower one from behind on a route that returns to itself.”
4. **Pre-empt the skipping objection.** “They cannot pass each other: once both are in the loop, the forward gap shrinks by exactly one per step, so it has to hit zero.”
5. **Then the entrance, with the arithmetic offered.** “For the start of the cycle, put one pointer back at the head and move both at single speed — they meet at the entrance. I can show you why in about four lines if you want.”
6. **State the costs.** “ $O(n)$ time, $O(1)$ space, and I do not modify the list.”

The follow-ups

“Why does phase two work?” “Call a the distance from the head to the cycle start, b the distance from the cycle start to the meeting point, and c the cycle length. When they meet, slow has walked $a + b$ and fast has walked twice that, and fast is in the same place having gone some whole number of extra laps — so $2(a + b) = a + b + kc$, which gives $a + b = kc$, so $a = kc - b$. That last line says: from the meeting point, walking a more steps takes me $a + b = kc$ steps into the cycle, a whole number of laps, which is exactly the cycle start. And walking a steps from the head is the cycle

start by definition. So both pointers arrive there together. In the common case where k is one it reads even more simply: the distance from the head to the entrance equals the distance from the meeting point round to the entrance.”

“Could the fast pointer skip over the slow one?” “No. Once both are inside the loop, look at the gap from fast to slow measured forwards round the loop. Each step slow gains one and fast gains two, so that gap decreases by exactly one. A quantity that decreases by exactly one cannot go from 1 to -1 without being 0, so they land on the same node. That argument depends on the 2:1 ratio — at 3:1 the gap would shrink by two and could step over zero on an odd-length cycle.”

“Why not just use a hash set?” “You can, and I would say so first — $O(n)$ time, $O(n)$ space, and it hands you the entrance with no proof required. I would not ship it if space matters: at a million nodes the set holds a million references, about thirty-two megabytes, to answer something two variables can answer. And it is usually exactly the improvement the question is asking for.”

“What is the time complexity, precisely?” “Phase one is at most $a + c$ steps of the slow pointer: a to enter the cycle, then at most c more because the gap shrinks by one each step and starts below c . Phase two is exactly a . So at most $2a + c$, which is at most $2n$ — linear, and I would rather give the bound that way than just say $O(n)$, because the c bound is the part that needs the shrinking-gap argument.”

“Find the duplicate number in an array of $n+1$ values from 1 to n , without modifying it, in $O(1)$ space.” “That is this algorithm wearing a costume. Read the array as a function: from index i , go to index $nums[i]$. Every value is between 1 and n , so the walk can never leave the array, and since it is infinite and the space is finite it must eventually repeat — a rho shape. Two values map to the same index precisely at the duplicate, so the duplicate is the *entrance* to the cycle, and Floyd’s finds entrances. Start both at index 0, run the tortoise and hare on the mapping, then run phase two. $O(n)$ time, $O(1)$ space, and the array is untouched — which is why the problem states all three constraints.”

“Can you find the cycle’s length too?” “Yes, and it is nearly free once you have the meeting point: stay there and walk forward counting until you return to it. That is $O(c)$ and no extra space. If you want the length before the entrance, that also gives an alternative phase two — advance one pointer c steps ahead of the other and then move both together, which lands them at the entrance for the same reason.”

A model answer

Asked: *detect a cycle in a linked list, then find the node where it begins.*

“Let me give you the obvious answer first so we agree on the baseline. Walk the list, keep a set of nodes you have seen, and the first node you meet twice is the entrance. That is $O(n)$ time, $O(n)$ space, and it needs no cleverness. If space is not a constraint, I would ship it.

To do it in constant space: a list with a cycle has a definite shape — a straight tail, then a loop — so a walk either falls off the end or goes round for ever. I send two pointers, one moving one step at a time and one moving two.

If the list ends, the fast pointer reaches `None` and there is no cycle. If it loops, the fast pointer comes up *behind* the slow one — and that is the whole insight. A faster runner can only catch a slower one from behind on a route that returns to itself. On a route that ends, the fast one arrives and waits; there is no behind.

The objection to pre-empt is whether the fast one could jump over the slow one. It cannot. Once both are inside the loop, the forward gap from fast to slow shrinks by exactly one every step — slow gains one, fast gains two — and a quantity that decreases by exactly one has to pass through zero. That depends on the two-to-one ratio, incidentally: at three to one the gap shrinks by two and could step over zero on an odd-length cycle.

Two details in the code. I compare with `is`, not `=`, because the question is whether they are on the same node, not whether two nodes hold the same value. And I advance both before comparing — comparing first would report a cycle immediately, since they start together.

Now the entrance, which is the actual question. Leave the slow pointer at the meeting point, put a new pointer at the head, and move both one step at a time. They meet exactly at the cycle start.

Here is why, and it is four lines. Call `a` the distance from the head to the cycle start, `b` the distance from the start round to the meeting point, and `c` the cycle length. Slow has walked `a + b`. Fast has walked twice that, and ended up in the same place, so it did some whole number of extra laps: $2(a + b) = a + b + kc$. Therefore $a + b = kc$, therefore $a = kc - b$. Read the last one as an instruction: from the meeting point, `a` more steps takes me `a + b` steps into the cycle, which is `kc`, a whole number of laps, landing exactly on the entrance. And `a` steps from the head is the entrance by definition. So they arrive together.

With `k = 1` it is easier to picture: the distance from the head to the entrance equals the distance from the meeting point round to the entrance.

Complexity: phase one is at most $a + c$ steps, because the gap shrinks by one and starts below the cycle length; phase two is exactly a . So at most about $2n$ — linear time, three variables of space, and I never modify the list, which matters because marking nodes as visited would also work and would destroy the caller's data.

Two cases I would run before saying I am done: a single node pointing at itself, and a list where the whole thing is one cycle. In both, a is zero and phase two must return the head without moving — and those are the two cases where the wrong answer, returning the meeting point, happens to look correct.”

9 Recall card

- **A list with a cycle is a tail plus a loop — a rho shape.** Two runners, **one step and two steps**. Fast falls off the end $\rightarrow$ no cycle. Fast catches slow **from behind** $\rightarrow$ cycle, because *a faster runner can only catch a slower one from behind on a route that returns to itself*. Compare with **is**, not **=**, and **advance before comparing**.
- **They cannot skip past each other: inside the loop the forward gap shrinks by exactly one per step**, so it must hit zero. That argument needs the **2:1 ratio** — at 3:1 the gap falls by two and could step over zero on an odd cycle.
- **Phase two, and the proof is four lines.** a = head $\rightarrow$ start, b = start $\rightarrow$ meeting, c = cycle length. slow walked $a + b$; fast walked $2(a+b) = a+b+kc$; so $a + b = kc$, so $a = kc - b$. Hence from the meeting point, a more steps = a whole number of laps = the entrance. Put one pointer back at the **head**, move both at **single speed**, they meet at the start. **No safety counter** — the arithmetic guarantees it.
- **The meeting point is NOT the entrance** (it only looks right when $a = 0$, which is exactly the test case people try). Run the two degenerate shapes last: **one node pointing at itself**, and **a list that is entirely one cycle**.
- **Costs: phase one $\leq a + c$, phase two exactly a , so $\leq 2n$ — $O(n)$ time, $O(1)$ space, no mutation.** Name the **hash-set** baseline first ($O(n)$ space, ~ 32 MB at a million nodes, entrance for free) and then improve it; reject **marking nodes** because it destroys the caller's data. The disguise to recognise: **Find the Duplicate Number** — read the array as $i \rightarrow \text{nums}[i]$, the walk can never leave the array, and the **duplicate is the entrance**.

Design tic-tac-toe, and then chess

1 What this is, and why they ask it

Tic-tac-toe is the smallest possible board game, and that is exactly why it is asked: there is nowhere to hide. Nine squares, two players, one win condition. Anybody can make it work. The interview is about what happens next.

Two things happen next, and they are the whole lesson. First: “*now make it N by N* ”, where the naive win check — scan every row, column and diagonal after every move — becomes $O(N^2)$ per move, and the good answer is $O(1)$ using running counters. Second: “*now design chess*”, where the board is the same grid and everything else is different, and the question becomes **where do the rules live so that changing the game does not mean rewriting the board?**

They ask it because a candidate who writes `if board[0][0] == board[0][1] == board[0][2]` nine times has told you something, and a candidate who writes four counters has told you something else. And because chess is the cleanest example of a rule that cannot live on any single object: whether a bishop may move is not a property of the bishop.

2 The story

Every evening in the courtyard behind the flats the children play the chalk game, and Shivanna, who is eighty-one and sits on the step by the meter box, is the umpire, because nobody trusts anybody else.

The game is old and the children have made it bigger over the years. It used to be three across. Then somebody drew four. Now they draw ten by ten with a bit of broken tile, which takes about ten minutes and always ends in an argument about whether the lines are straight.

Two teams. They take turns filling in one square each. The first team to fill a whole line — across, down, or corner to corner — wins.

The children check for a winner the same way every time. When a square is filled, four or five of them crouch over the grid and read every line, one after another, out loud, while everybody else waits. On the ten-by-ten grid that takes the better part of a minute. It happens after every single square. By the end of a game they have read that grid two hundred times.

Shivanna does not look at the grid at all.

He keeps numbers. One for each row across, one for each line going down, and one for each of the two corner-to-corner lines. Twenty-two numbers, in his head. When a child fills a square he adds one to that row's number and one to that column's number, and if the square happens to sit on a corner-to-corner line, to that number too. If any number reaches ten, he says so. He does not look at the other ninety-nine squares, because nothing that just happened could have changed them.

He has been doing it since the game was three across and there were only eight numbers.

The children changed the game again last winter — now it is five in a row rather than a whole line, on the same ten-by-ten grid. Shivanna adapted in about four seconds. The counting is the same. Only the number he is waiting for changed.

And when the older boys play their own game on the same chalk grid, with completely different rules about who may go where, Shivanna will not umpire. He says the grid is the same grid and the marks are the same marks, but the rules are somebody else's business, and he has not learned them.

3 The idea in plain English

Shivanna's twenty-two numbers are the $O(1)$ win check. And his last sentence — *the grid is the same, the rules are somebody else's business* — is the answer to the chess half.

The naive win check, and why it stops being acceptable

After a move, scan every row, every column and both diagonals:

3 x 3:	3 rows + 3 cols + 2 diagonals = 8 lines	× 3 cells	= 24 checks per move
N x N:	2N + 2 lines	× N cells	= $O(N^2)$ per move

At $N = 3$ that is 24 checks and nobody cares. At $N = 1000$ it is about two million checks **per move**, and a full game is a million moves, so 2×10^{12} operations. The naive version is not slow because it is badly written; it is slow because it re-derives everything from scratch each time.

The counter trick

A move can only affect the lines that pass through the square it was played on — one row, one column, and at most two diagonals. Nothing else on the board changed, so nothing else needs checking.

So keep a running count per line. The neat encoding is **+1** for player X and **-1** for player O:

```

delta = 1 if player is Player.X else -1
self._rows[row] += delta
self._cols[col] += delta
if row == col:
    self._diagonal += delta
if row + col == self._n - 1:
    self._anti_diagonal += delta

```

Then a line is won when its count reaches `+n` or `-n`, because that can only happen if every cell in it belongs to one player:

```

if abs(self._rows[row]) == n or abs(self._cols[col]) == n \
    or abs(self._diagonal) == n or abs(self._anti_diagonal) == n:
    return player

```

Four increments and four comparisons. $O(1)$ per move, whatever N is. The `+1/-1` encoding is what makes it one number per line instead of two counts; `abs(count) == n` is unambiguous because you can only reach `n` by having all `n` cells and `-n` by having none of them for the other player.

Space is $O(N)$: two arrays of length N and two integers, against the $O(N^2)$ board itself. So the counters are free.

The diagonal conditions, which are the two lines people get wrong

main diagonal	<code>row == col</code>	<code>(0,0)</code> <code>(1,1)</code> <code>(2,2)</code>
anti-diagonal	<code>row + col == n - 1</code>	<code>(0,2)</code> <code>(1,1)</code> <code>(2,0)</code>

Both, not one — a cell can be on both, and the centre of an odd-sized board always is. Write them as two separate `if` s, never an `elif`.

Making the win condition replaceable

Shivanna adapting to “five in a row” in four seconds is the design point. The board did not change; the *condition* did.

That earns an interface, by the usual gate from [day 076](#) — **can you name a second implementation somebody actually wants?**

```

class WinCondition(Protocol):
    def check(self, board: Board, row: int, col: int, player: Player) → Player | None: ...

```

- `FullLine` — the whole row, column or diagonal. Classic tic-tac-toe.
- `KInARow` — any run of k , anywhere. That is Gomoku at $k = 5$ on 15×15 , and Connect Four at $k = 4$ with gravity.

Two real implementations, so the interface is justified. Nothing else in tic-tac-toe is, and inventing a `MoveValidatorFactory` for a nine-square game is the drill through the wall from [day 076](#).

Note that `KInARow` cannot use the whole-line counters — a run of five inside a row of fifteen is a different question. It scans outward from the played cell in four directions, which is $O(k)$ per move, still independent of N . Say that rather than pretending one trick covers both.

Now chess: what changes, and what does not

The board barely changes. It is still a grid of squares, each holding a piece or nothing, and it still knows nothing about rules.

What changes is that “**is this move legal?**” is **no longer a property of one object**. That is the whole difficulty and it is worth stating flatly, because a candidate who puts `can_move` on `Piece` and stops has missed it.

Three reasons a piece cannot answer alone:

One: the path. A rook moving from a1 to a8 is geometrically fine and illegal if anything stands between. So the piece needs the board.

Two: check. A move that is perfectly legal for the piece is illegal if it leaves your own king attacked. A pinned bishop may not move at all, even though nothing about the bishop has changed. This is not a piece rule; it is a *position* rule.

Three: moves that involve more than one piece or the past. Castling moves two pieces and depends on whether either has moved before, and on whether the king passes through an attacked square. En passant depends on what the *opponent did on the previous move*. Promotion replaces the piece with a different one. None of these can live inside a single piece object.

The structure that survives all three

Two stages, and naming them is what shows you have thought about chess rather than about class diagrams:

- | | |
|----------------------------|--|
| 1. PSEUDO-LEGAL generation | the piece + the board:
"where could this piece go, ignoring check?" |
| 2. LEGALITY filtering | the game:
"make the move, is my king attacked? if so, discard it" |

```

class Piece:
    def pseudo_legal_moves(self, board: Board, at: Square) → list[Move]: ...

class Game:
    def legal_moves(self, at: Square) → list[Move]:
        moves = self._board[at].pseudo_legal_moves(self._board, at)
        legal = []
        for move in moves:
            undo = self._board.apply(move)
            if not self._board.is_attacked(self._board.king_square(self._turn), by=other(self._turn)):
                legal.append(move)
            self._board.undo(undo)          # make, test, unmake
        return legal

```

Make, test, unmake. That is how every real chess engine does it, and the `undo` is not an optimisation — it is what makes legality checking possible at all without copying the board thirty-five times per position.

Where the rules actually live

- **Piece** — geometry only. How this kind of piece moves on an empty-ish board. One class per type, or one class holding a movement rule; either is defensible, and the second composes better because a fairy-chess variant can add a piece without a new subclass.
- **Board** — squares, what is on them, `apply`, `undo`, and `is_attacked`. Mechanics, no rules.
- **Game** — turn order, history, the check filter, castling rights, the en passant square, the fifty-move counter, and the end conditions. **The rules that are about the position rather than about a piece live here.**

That split is the answer to “so the rules live in one replaceable place”. The replaceable place is `Game` plus the piece movement rules — and a variant like Chess960 changes only castling rights inside `Game`, while a variant with a new piece changes only a movement rule.

4 The picture

The counters, on a 3×3 after four moves:

	col 0	col 1	col 2	row counts
r0	X	0		0 (+1 -1)
r1		X		+1
r2	0			-1
col	0	0	0	
	(+1-1)	(-1+1)		

diagonal (0,0)(1,1)(2,2) = X, X, _ → +2
anti-diagonal (0,2)(1,1)(2,0) = _, X, 0 → 0

X plays (2,2):
rows[2] += 1 → 0
cols[2] += 1 → +1
row = col → diagonal += 1 → +3
abs(+3) = 3 → X WINS, and we touched four numbers, not nine squares

What to notice: the only numbers that changed are the ones for lines through (2,2). Shivan not looking at the other squares, because nothing that just happened could have changed them.

The two designs, side by side:

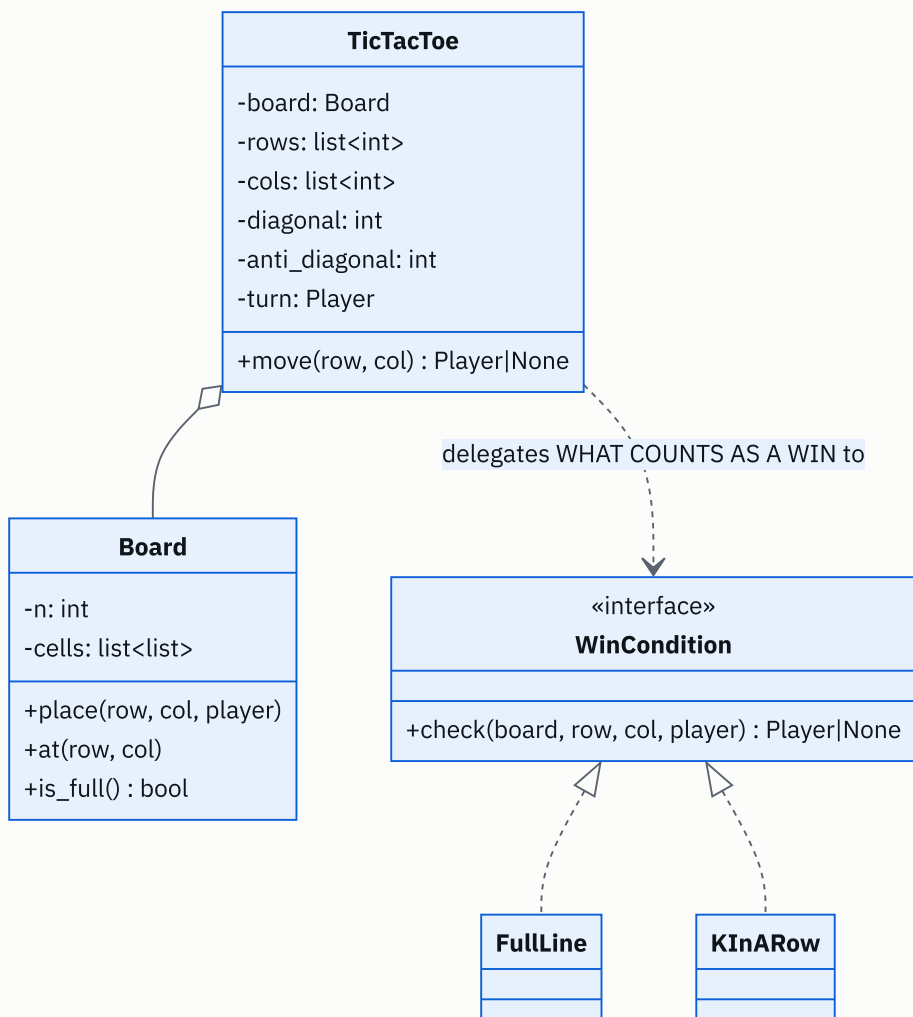


FIGURE 83.1

And chess, where the split is the point:

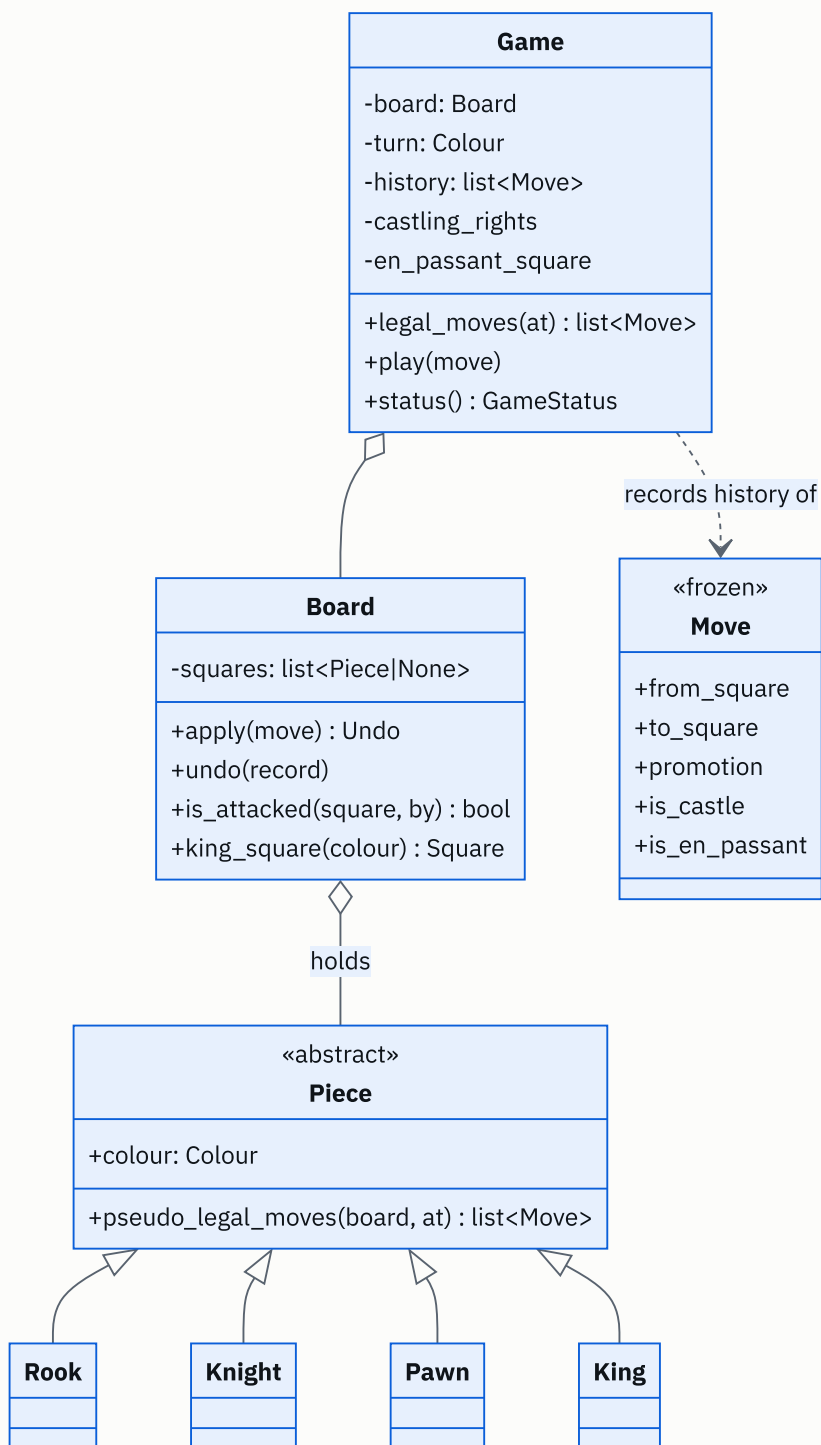


FIGURE 83.2

What to notice: `Piece` has `pseudo_legal_moves`, not `can_move`. That one name change carries the whole design — it says out loud that the piece cannot decide legality, only geometry, and that somebody else finishes the job.

The two-stage filter, drawn:

```
Piece.pseudo_legal_moves()      Game.legal_moves()
"where could I go?"            "which of those are actually allowed?"

Bishop c1 → [d2, e3, f4, g5, ...]  for each move:
                                     board.apply(move)
                                     if my king is attacked → discard
                                     board.undo()

result: [] if the bishop is pinned – the bishop's geometry did not change,
          the POSITION made it illegal.
```

5 How it actually works

Move 1 • Clarify (minutes 0–5)

“Fixed 3×3 , or general $N \times N$?” — Design for N , since it costs nothing. **“Win is a full line, or k in a row?”** — Start with a full line; ask whether k -in-a-row is coming, because it changes the win check. **“Two players only?”** — Yes, but the counter trick generalises to more with a count per player. **“Do I need undo, or a computer opponent?”** — Ask, because both change the design: undo needs a move history, and an opponent needs move generation and evaluation.

Move 2 • The nouns for tic-tac-toe (minutes 5–10)

- `Player` — an enum, X and O. Not a string, so a typo is a failure at the boundary.
- `Board` — the grid and what is on each square. **No rules.**
- `WinCondition` (*interface*) — what counts as a win.
- `Game` — turn order, move validation, the counters, and the result.

Four. One interface, and only because k -in-a-row is a real second implementation.

Move 3 · The game, with the counters

```
class TicTacToe:
    def __init__(self, n: int = 3) → None:
        self._n = n
        self._cells: list[list[Player | None]] = [[None] * n for _ in range(n)]
        self._rows = [0] * n
        self._cols = [0] * n
        self._diagonal = 0
        self._anti_diagonal = 0
        self._turn = Player.X
        self._moves_played = 0
```

$O(N)$ of bookkeeping against an $O(N^2)$ board. Say that ratio out loud — it is why the optimisation is unarguable rather than a trade.

```
def move(self, row: int, col: int) → Player | None:
    if not (0 ≤ row < self._n and 0 ≤ col < self._n):
        raise ValueError(f"({row}, {col}) is off the board")
    if self._cells[row][col] is not None:
        raise ValueError(f"({row}, {col}) is already taken")
    if self._winner is not None:
        raise ValueError("the game is over")
```

Three guards before touching anything. The third is the one people leave out, and it lets a finished game keep accepting moves.

```
player = self._turn
self._cells[row][col] = player
self._moves_played += 1
delta = 1 if player is Player.X else -1

self._rows[row] += delta
self._cols[col] += delta
if row == col:
    self._diagonal += delta
if row + col == self._n - 1:
    self._anti_diagonal += delta # a separate `if`, never `elif`
```

Two separate `if`s. The centre of an odd board is on both diagonals, and an `elif` silently drops one of them — a bug that only shows up on odd boards with a diagonal win through the middle.

```
n = self._n
if (abs(self._rows[row]) == n or abs(self._cols[col]) == n
    or abs(self._diagonal) == n or abs(self._anti_diagonal) == n):
    self._winner = player
    return player

self._turn = Player.O if player is Player.X else Player.X
return None
```

Only the four lines through this square are checked. That is the entire optimisation and it is five lines of code.

Move 4 · The k-in-a-row condition, which needs a different method

```
class KInARow:
    """A run of k anywhere. Cannot use whole-line counters – scan outward from
    the played cell in four directions. O(k) per move, still independent of N."""

    def __init__(self, k: int) → None:
        self._k = k

    DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))      # → ↓ ↘ ↙

    def check(self, cells, row, col, player, n):
        for dr, dc in self.DIRECTIONS:
            run = 1
            run += self._count(cells, row, col, dr, dc, player, n)
            run += self._count(cells, row, col, -dr, -dc, player, n)
            if run ≥ self._k:
                return player
        return None
```

Four directions, not eight, because each direction and its opposite are counted together. Getting that wrong doubles the work and is the usual first version.

Move 5 · Chess – the classes, and the one that matters

```
@dataclass(frozen=True)
class Move:
    from_square: int
    to_square: int
    promotion: str | None = None
    is_castle: bool = False
    is_en_passant: bool = False
```

Frozen, and it carries the three special cases as flags rather than as separate types. Those three flags are exactly the moves that no piece can describe on its own, and having them on `Move` is what lets `Board.apply` do the right thing.


```

class Rook(Piece):
    DIRECTIONS = ((0, 1), (0, -1), (1, 0), (-1, 0))

    def pseudo_legal_moves(self, board, at):
        moves = []
        for dr, dc in self.DIRECTIONS:
            square = at
            while True:
                square = step(square, dr, dc)
                if square is None:                    # off the board
                    break
                occupant = board[square]
                if occupant is None:
                    moves.append(Move(at, square))
                    continue
                if occupant.colour is not self.colour:
                    moves.append(Move(at, square))    # capture
                break                                # blocked either way
        return moves

```

The sliding loop, which the rook, bishop and queen all share — so it belongs in a shared helper with a direction list, and the three pieces become three tuples. Knight and king use the same helper with a single step instead of a slide.

```

class Board:
    def apply(self, move: Move) → "Undo":
        """Make the move and return everything needed to take it back."""
        captured = self._squares[move.to_square]
        record = Undo(move, captured, self._en_passant, self._castling)
        self._squares[move.to_square] = self._squares[move.from_square]
        self._squares[move.from_square] = None
        ...
        return record

    def undo(self, record: "Undo") → None:
        """Restore exactly. This is what makes legality checking affordable."""

```

`apply` returning an `Undo` rather than the board copying itself is the single most important performance decision in a chess program. Copying a board per candidate move is about thirty-five copies per position; make-and-unmake is two pointer writes.

```

class Game:
    def legal_moves(self, at: int) → list[Move]:
        piece = self._board[at]
        if piece is None or piece.colour is not self._turn:
            return []
        legal = []
        for move in piece.pseudo_legal_moves(self._board, at):
            undo = self._board.apply(move)
            in_check = self._board.is_attacked(
                self._board.king_square(self._turn), by=opposite(self._turn))
            self._board.undo(undo)
            if not in_check:
                legal.append(move)
        return legal

```

Nine lines, and they are the answer to “where do the rules live”. The piece contributed geometry; the game contributed the rule that you may not leave your own king attacked. **Neither could have done it alone**, which is why `can_move` on a piece is the wrong shape.

```

def status(self) → GameState:
    moves = [m for sq in self._board.squares_of(self._turn) for m in self.legal_moves(sq)]
    if moves:
        return GameState.IN_PLAY
    in_check = self._board.is_attacked(
        self._board.king_square(self._turn), by=opposite(self._turn))
    return GameState.CHECKMATE if in_check else GameState.STALEMATE

```

Checkmate and stalemate are the same test with one extra question. No legal moves plus in check is checkmate; no legal moves and not in check is a draw. Getting that in one method is a small, real demonstration of understanding the domain.

Real systems

- **Stockfish and every serious engine use bitboards:** one 64-bit integer per piece type per colour, where each bit is a square. Move generation becomes bit shifts and masks, and “all squares a rook attacks” is a table lookup. That is the production answer for performance, and mentioning it — while noting you would not write it in an interview — shows you know where this goes.
- **Make/unmake with an undo record** is universal in engines, for exactly the reason above: a search visits millions of positions and cannot copy the board at each one.
- **FEN and PGN** are the standard formats for a position and a game, and a design that can produce FEN has, by construction, remembered castling rights, the en passant square, the half-move clock and the side to move — the four bits of state people forget.
- **python-chess** is the reference library, and its API is exactly this shape: a `Board` that generates legal moves, `push / pop` for make and unmake.

6 The numbers

Why the counters are not a micro-optimisation

```
naive check per move: (2N + 2) lines × N cells = 2N2 + 2N ≈ O(N2)
counter check:       4 increments + 4 comparisons = O(1)
```

```
N = 3:      24 checks   vs 8   – nobody cares
N = 100:    20,200     vs 8   – 2,500x
N = 1000:   2,002,000  vs 8   – 250,000x
```

And over a whole game, where the number of moves is itself $O(N^2)$:

```
N = 1000, a full board = 106 moves
naive:    106 moves × 2 × 106 checks = 2 × 1012 operations
counters: 106 moves × 8               = 8 × 106 operations
```

Two trillion against eight million. That is the difference between a program that finishes and one that does not, and it comes from one observation: only the lines through the played square can have changed.

Memory

board	N^2 cells × 8 B (a reference)	$N = 1000 \rightarrow 8$ MB
counters	$2N$ integers + 2	$N = 1000 \rightarrow 16$ KB
ratio		0.2% overhead

The optimisation costs **one fifth of one percent** of the board's memory. There is no trade-off to discuss, which is worth saying — most optimisations cost something and this one does not.

Chess, in numbers

squares	64	
board as an array	64 references × 8 B	= 512 B
board as bitboards	12 pieces × 8 B	= 96 B
average legal moves	~35 per position	
average game length	~40 moves per side	= 80 plies

```
legality filtering, per position:
  35 pseudo-legal moves × (apply + is_attacked + undo)
  is_attacked ≈ 8 directions × up to 7 squares ≈ 50 checks
  → 35 × 50 ≈ 1,750 operations per position
```

```
with board COPYING instead of undo:
  35 copies × 512 B = 18 KB of copying per position
  at 1,000,000 positions/second in a search: 18 GB/s of memcpy – impossible
```

That is the argument for make-and-unmake, in a number rather than as a principle. It is also why a naive object-per-square design with `deepcopy` cannot be the basis of an engine.

The state people forget

side to move	1 bit	
castling rights	4 bits	(each side, each rook)
en passant target square	6 bits	
half-move clock (50-move)	~7 bits	
full-move number	~10 bits	

Under four bytes, and every one of them is required for the rules to be correct. A design that stores only “which piece is on which square” **cannot** answer whether castling is legal or whether the game is a draw by the fifty-move rule. Listing them is a quick way to show you know the domain rather than the diagram.

7 The trade-offs

What this design gives up

The counters only work for whole-line wins. They are exactly wrong for k-in-a-row, where a run of five inside a row of fifteen is not visible in a row total. That is why `WinCondition` is an interface rather than a flag — the two conditions do not share an algorithm, only a question. If I had bolted k-in-a-row on as a parameter to the counter version, it would have needed rewriting.

Counters have to be maintained by every mutation, so undo becomes a hazard. If undo is added later, it must reverse the counters as well as the board, and forgetting that gives a board and a set of counters that disagree — with no error. If undo is a requirement, I would either store deltas with each move or drop back to scanning, and I would decide that *before* building the counters, not after.

Chess with a class per piece is inheritance, with all that implies. A variant that adds a piece needs a new subclass; a variant that changes how an existing piece moves needs an override. Composing a movement rule into a single `Piece` class — the Strategy shape from [day 071](#) — handles both without new types, at the cost of one more indirection. For an interview I would write subclasses and *name* the composition alternative, because subclasses read faster on a whiteboard.

Make-and-unmake makes the board mutable and stateful. That is fast and it is a genuine source of bugs: any early return between `apply` and `undo` leaves the board corrupt. In production this is a context manager or a rigid try/finally, and the bug it pre-

vents — a search that silently explores an impossible position — is very hard to find.

Nothing here handles the draw rules properly. Threefold repetition needs a hash of every position seen, insufficient material needs a piece-count table, and the fifty-move rule needs the half-move clock. Each is small and each is easy to leave out, and a chess design that cannot detect a draw is not a chess design.

“I would change this design if...”

- **...the board is large and the win is k-in-a-row.** Different win condition, different algorithm — scan outward from the played cell, and drop the whole-line counters entirely.
- **...undo or replay is required.** Then a move history is the primary structure and the board becomes a projection of it, which also gives you replay and analysis for free.
- **...there is a computer opponent.** Then move generation performance dominates everything, and I would move to bitboards and accept that the code stops being readable.
- **...variants must be pluggable** — Chess960, fairy pieces, three-check. Then movement becomes composed rules rather than subclasses, and `Game` gets a rule set rather than hard-coded castling.

The honest concession

Tic-tac-toe does not need any of this. Nine squares and eight lines is twenty-four comparisons and a first-year student can write it correctly in ten minutes. Everything in this lesson is a response to the *second* question — “now make it N by N”, “now design chess” — and the reason to know it is that the second question is always asked. Building the counters for a 3×3 game and calling it engineering would be the drill through the wall; building them because the interviewer said N is judgement.

8 In the interview

How it gets asked

- The opener: “*Design tic-tac-toe.*” Ten minutes, and it is a warm-up.
- The real question, always: “*Now make it work for an N by N board. What is the cost of checking for a winner?*”
- The variation probe: “*Now the win condition is five in a row on a large board.*”
- The escalation: “*Now design chess.*” Sometimes the whole round is this.
- The chess probe that separates people: “*Where does the rule live that you cannot move into check?*”

The timed script

Minutes 0–5 · Clarify. $N \times N$ or 3×3 ? Full line or k-in-a-row? Undo or an opponent? Those three answers change the design and none of them is guessable.

Minutes 5–10 · The naive version, said and costed. “The obvious check scans every line after every move — $2N + 2$ lines of N cells, so $O(N^2)$ per move. At $N = 1000$ that is two million checks per move and two trillion over a game.”

Minutes 10–18 · The counters. The observation first — only lines through the played square can have changed — then the `+1/-1` encoding, then the two separate diagonal `if` s.

Minutes 18–25 · The classes, with `WinCondition` as an interface and the honest note that k-in-a-row does not share the counters’ algorithm.

Minutes 25–35 · Chess. The three reasons a piece cannot decide legality, then pseudo-legal generation plus the check filter, then make-test-unmake with the copying number.

Minutes 35–40 · The state people forget — castling rights, en passant square, half-move clock — and the draw conditions.

The follow-ups

“How do you check for a winner on an $N \times N$ board?” “Not by scanning. A move can only change the lines that pass through the square just played — one row, one column, and at most two diagonals — so I keep a running count per line and update four of them. I encode X as `+1` and O as `-1`, so a line is won when its absolute value reaches N . That is $O(1)$ per move against $O(N^2)$ for scanning: at $N = 1000$, eight operations instead of two million. And the counters are $O(N)$ memory against an $O(N^2)$ board, so it costs about a fifth of a percent — there is no trade to discuss.”

“Two diagonals — is one `if` enough?” “No, they must be two separate `if` s, not an `if/elif`. The centre of any odd-sized board is on both diagonals, so an `elif` silently drops the update to the second one, and the bug only appears on an odd board with a diagonal win through the middle — which is precisely the classic 3×3 game.”

“Now the win is five in a row on a 15×15 board.” “Different algorithm, and I would say that rather than parameterise the counters, because a run of five inside a row of fifteen is not visible in a row total. I scan outward from the played cell in four directions — right, down, and the two diagonals — counting the run in both directions along each. That is $O(k)$ per move, still independent of N . It is why the win condition is behind an interface: the two rules share a question, not an implementation.”

“Now design chess. Where does ‘you cannot move into check’ live?” “Not on the piece, and that is the key point. A piece can only answer geometry — where could I go given what is on the board — and it cannot know that moving would expose its own

king. So I split it in two: the piece generates *pseudo-legal* moves, and the game filters them for legality by making each move, asking whether its own king is attacked, and unmaking it. The name `pseudo_legal_moves` rather than `can_move` is deliberate — it says out loud that the piece does not finish the job.”

“Why make and unmake rather than copying the board?” “Cost. There are about thirty-five legal moves in a typical position, so filtering means thirty-five trial moves. Copying a 64-square board each time is about 18 KB per position, and a search visiting a million positions a second would need 18 GB a second of copying, which is not possible. Make-and-unmake is two writes and an undo record. The price is that the board is mutable and any early return between apply and undo corrupts it, so in real code that pairing is a context manager or a try/finally.”

“What about castling and en passant?” “Those are the moves that prove a piece cannot own the rules. Castling moves two pieces and depends on history — whether the king or that rook has ever moved — and on whether the king passes through an attacked square. En passant depends on what the opponent did on the *previous* move. Promotion replaces the piece entirely. So `Move` carries flags for those cases, `Board.apply` knows how to perform them, and the *rights* — castling availability and the en passant target square — live on the game state. If I can produce a FEN string, I have by construction remembered all of it: side to move, castling rights, en passant square, half-move clock, move number. Under four bytes, and every bit is needed for correctness.”

“How do you detect checkmate?” “Checkmate and stalemate are the same test with one extra question: generate all legal moves for the side to move; if there are none, ask whether that side is currently in check. In check and no moves is checkmate; not in check and no moves is stalemate, a draw. And a complete implementation needs the other draws too — threefold repetition, which needs a hash of every position seen; insufficient material; and the fifty-move rule, which is why the half-move clock is part of the state.”

A model answer

Asked: *design tic-tac-toe, then make it N by N.*

“Let me start with what I would not do, because it is the thing the N by N question is aimed at.

The obvious win check is to scan every row, every column and both diagonals after each move. That is $2N + 2$ lines of N cells, so $O(N^2)$ per move. At three by three it is twenty-four comparisons and nobody cares. At a thousand by a thousand it is two million checks per move, and a full board is a million moves, so two trillion operations over a game.

The observation that fixes it: **a move can only change the lines that pass through the square it was played on.** One row, one column, and at most two diagonals. Everything else on the board is exactly as it was, so there is nothing to re-check.

So I keep a running count per line — one per row, one per column, and two for the diagonals — and I encode one player as plus one and the other as minus one. A move updates at most four numbers. A line is won when its absolute value reaches N , because you can only get there by owning all N cells. Four increments and four comparisons: $O(1)$ per move, whatever N is. At $N=1000$ that is eight operations instead of two million.

The counters are two arrays of length N against an N -squared board, so about a fifth of a percent of the memory. There is no trade-off to weigh, which is unusual and worth saying.

One detail that catches people: the two diagonal updates must be two separate `if`s, not an `if/elif`. The centre square of any odd board is on both diagonals, and an `elif` silently drops one — and the bug only shows up on a diagonal win through the middle, which is the most common way an actual three-by-three game ends.

For the classes: `Player` as an enum so a typo fails at the boundary; a `Board` that holds the grid and knows no rules; and a `Game` that owns turn order, validation, the counters and the result. I would put the win condition behind an interface, but only because I can name a second implementation somebody actually wants — five in a row on a large board, which is Gomoku. And I would say plainly that the k-in-a-row version cannot reuse the counters: a run of five inside a row of fifteen is not visible in a row total, so it scans outward from the played cell in four directions instead. The two conditions share a question, not an algorithm, which is exactly what an interface is for.

Three guards before every move: on the board, not already taken, and the game is not already over — the third is the one people forget, and it lets a finished game keep accepting moves.

If you want chess after this, the thing that changes is not the board — it is that legality stops being a property of any single object. A rook's move depends on what is in the way, so the piece needs the board. And a perfectly legal rook move is illegal if it leaves your own king attacked, which is a fact about the position and not about the rook at all. So the piece generates *pseudo-legal* moves and the game filters them by making each move, testing whether its own king is attacked, and unmaking it.”

9 Recall card

- The $N \times N$ question is the real question, and the answer is one observation: a move can only change the lines through the square it was played on. Keep a running count per row, per column, and for the two diagonals; encode one player $+1$ and the other -1 ; a line is won when `abs(count) == n`. **$O(1)$ per move** against $O(N^2)$ — **8 operations vs 2,000,000 at $N = 1000$** — for $O(N)$ memory, about **0.2%** of the board.
- The two diagonal updates are two separate `if` s, never `if/elif` — the centre of an odd board is on both, and an `elif` drops one, which is invisible except on a diagonal win through the middle. Guard three things before every move: **on the board · not taken · game not already over**.
- `WinCondition` earns an interface because **k-in-a-row is a real second implementation** — and it does NOT share the counters' algorithm. A run of 5 inside a row of 15 is invisible in a row total, so it scans outward from the played cell in **four** directions (each direction plus its opposite), $O(k)$ per move. *The two rules share a question, not an implementation.*
- **Chess: legality is not a property of a piece**. Three reasons — the **path** (needs the board), **check** (a pinned piece may not move though nothing about it changed), and moves involving **two pieces or the past** (castling, en passant, promotion). So: `Piece.pseudo_legal_moves` for geometry, and `Game` filters by **make** → **test is_attacked** → **unmake**. The method name `pseudo_legal_moves` rather than `can_move` carries the design.
- **Make-and-unmake instead of copying, and the reason is a number**: ~ 35 moves per position $\times 512$ B is **18 KB of copying per position**, so a million-position search would need **18 GB/s**. And **remember the state people forget** — side to move, **castling rights**, **en passant square**, **half-move clock**, **move number**: under 4 bytes, and without them castling and the fifty-move draw are unimplementable. **Checkmate and stalemate are one test plus one question**: no legal moves, and are you in check.

PRACTICE

Practice — Cycle detection, and why Floyd's algorithm works

DSA topic: Cycle detection, and why Floyd's algorithm works **System design topic:** Design tic-tac-toe, and then chess

Code these, in this order

One rule for the whole set: **say the hash-set solution out loud first, then improve on it.** Naming the $O(n)$ -space baseline before writing the clever one is worth more than jumping straight to Floyd's, and it is what you should do in a real interview.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Linked List Cycle	LeetCode 141 (Easy)	The two runners, <code>is</code> versus <code>=</code> , and advancing before comparing.
2	Linked List Cycle II	LeetCode 142 (Medium)	Phase two, and whether you can prove it rather than recite it.
3	Happy Number	LeetCode 202 (Easy)	Recognising a cycle question that has no linked list in it.
4	Find the Duplicate Number	LeetCode 287 (Medium)	The hardest reframing: an array read as a function, with the duplicate as the cycle entrance.

On problem 2, do the arithmetic before you code

Write out `a`, `b` and `c`, derive `a = kc - b`, and say the sentence that turns it into an instruction. Only then write the three lines. If you write the code first you will not be able to defend it.

On problem 3, find the cycle without a list

There are no nodes here — the “next” of a number is the sum of the squares of its digits. Write the `next` function first, then run exactly the same two-runner loop on it. Say out loud why the walk must eventually repeat.

On problem 4, say why each obvious solution is banned

Sorting mutates. A set costs $O(n)$ space. Marking with negatives mutates. Only then explain the index-to-value mapping and why the duplicate is the entrance. If you cannot state all three rejections, you do not yet know why the problem is stated the way it is.

The why-they-meet drill

Say each answer out loud, five times:

1. Why can a faster runner only catch a slower one from behind on a route that loops?
2. What happens instead if the route ends?
3. What is the forward gap doing on every step once both are inside the loop?
4. Why can that gap not skip over zero?
5. What would break if the ratio were 3:1 instead of 2:1? Give a concrete cycle length.

The proof drill

1. Define `a`, `b` and `c` in words.
2. Write the two distances walked when they meet.
3. Derive `a + b = kc` and then `a = kc - b`.
4. Turn the last line into an instruction about where to walk from the meeting point.
5. State the `k = 1` version in one sentence about distances.
6. Deliver the whole thing in thirty seconds, without notes.

The break-it drill

Trigger each and record the exact output or error text:

1. Use `=` instead of `is`, on a `Node` class with `__eq__` defined on the value.
2. Compare before advancing. Run any non-empty list.
3. Return the meeting point as the cycle start. Run the six-node example, then run a list that is entirely one cycle, and say why the second one hides the bug.
4. Put the loop condition in the wrong order. Run an even-length list with no cycle.
5. Add a safety counter to phase two. Say what it means about your own proof.
6. Detect the cycle by marking nodes as visited. Say what you have destroyed.

The edge-case drill

Build and run each shape:

1. An empty list.
2. A single node with `next = None`.
3. A single node pointing at itself.
4. A two-node list with no cycle.
5. A list where the entire thing is one cycle.
6. A long tail with a two-node cycle at the end.

For each, state `a`, `b` and `c`, and check that phase two returns what you expect. Two of the six are where “return the meeting point” accidentally looks correct.

The complexity drill

1. Bound phase one, in terms of `a` and `c`, and justify the `c` part.
2. Bound phase two exactly.
3. State the total in terms of `n`.
4. State the space, and the memory of the set version at a million nodes.
5. Say which of the three approaches — set, Floyd’s, marking — fails which constraint.

The reframing drill

For each, say whether Floyd’s applies, and if so what plays the part of “next”:

1. A linked list.
2. Replacing a number with the sum of the squares of its digits.
3. An array of $n + 1$ values in $1..n$, read as `i → nums[i]`.
4. A directed graph where every node has exactly one outgoing edge.
5. A directed graph where nodes have several outgoing edges.
6. A random number generator’s sequence of states.

One of the six does not work. Name it and say precisely why the algorithm needs what it lacks.

The counter drill

1. State the observation that makes the $O(1)$ win check possible, in one sentence.
2. Write the four updates for a move at `(row, col)`.
3. Say why `+1` and `-1` beats two separate counts per line.
4. Write the win test and say why `abs(count) == n` is unambiguous.
5. Compute the naive and counter costs at $N = 3$, $N = 100$ and $N = 1000$.
6. Compute the total operations over a full game at $N = 1000$, both ways.
7. State the memory overhead as a percentage of the board.

The diagonal drill

1. Write both diagonal conditions.
2. Say which cells satisfy both on a 3×3 , a 4×4 and a 5×5 .
3. Change the second `if` to an `elif` and construct the game that now reports no winner.

4. Say why this bug survives most testing.

The win-condition drill

1. Apply the interface gate to `WinCondition` and name the second implementation.
2. Say why k-in-a-row cannot use the counters.
3. Write the four directions it scans and say why four and not eight.
4. State its cost per move and say what it does *not* depend on.
5. Say what would have gone wrong if you had added k-in-a-row as a parameter to the counter version.

The chess-responsibility drill

For each rule, say whether it belongs to `Piece`, `Board`, or `Game`, and why:

1. A knight moves in an L.
2. A rook cannot jump over a piece.
3. You may not leave your own king attacked.
4. White moves first.
5. Castling requires that neither piece has moved.
6. En passant is only available immediately after a two-square pawn advance.
7. A pawn reaching the last rank becomes another piece.
8. The game is a draw after fifty moves with no capture or pawn move.

Three of the eight cannot belong to any single piece. Name them and say what they have in common.

The two-stage drill

1. Say why `pseudo_legal_moves` is a better method name than `can_move`.
2. Write the legality filter in nine lines.
3. Say what `apply` must return and why.
4. Compute the cost of copying the board instead, per position and at a million positions a second.
5. Name the bug that make-and-unmake makes possible, and the language feature that prevents it.

The forgotten-state drill

1. List the five pieces of state beyond “which piece is on which square”.
2. For each, name a rule that is unimplementable without it.
3. Say roughly how many bytes all five take.

4. Say which standard format forces you to have remembered all of them.

The endgame drill

1. Write the test that distinguishes checkmate from stalemate.
 2. Name the three draw conditions this design does not yet handle.
 3. Say what data structure each one needs.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Detect a cycle in a linked list. Now find the node where it begins.* The hash-set baseline named first, the rho shape, the two runners with *why* a faster runner can only catch from behind, the shrinking-gap answer to “could it skip past”, phase two, and the four-line proof delivered without notes.
 2. *Design tic-tac-toe. Now make it work for an N by N board.* The naive $O(N^2)$ check costed with a number, the observation about lines through the played square, the ± 1 counters with the two separate diagonal `if`s, the memory overhead as a percentage, and the win condition behind an interface with the honest note that k-in-a-row does not share the algorithm.
 3. *Where does “you cannot move into check” live?* Not on the piece, and the three reasons a piece cannot decide legality; pseudo-legal generation plus the game’s filter; make, test, unmake, with the copying number that justifies it.
-

Before you move on

- [] I state the hash-set baseline before writing Floyd’s.
- [] I can say why a faster runner can only catch a slower one from behind.
- [] I can answer “could the fast pointer skip past” with the shrinking-gap argument.
- [] I can say what breaks at a 3:1 ratio, with a concrete cycle length.
- [] I can derive `a = kc - b` and turn it into an instruction, in thirty seconds.
- [] I returned the meeting point on purpose and found the shape where it looks correct.
- [] I ran all six edge shapes and checked `a`, `b` and `c` for each.
- [] I solved Find the Duplicate Number and can state all three rejected approaches.
- [] I can state the observation behind the $O(1)$ win check in one sentence.

- [] I computed naive versus counter cost at $N = 1000$, per move and per game.
- [] I turned the second diagonal `if` into an `elif` and built the game that breaks.
- [] I can say why k-in-a-row needs a different algorithm, not a parameter.
- [] I can sort eight chess rules into piece, board and game.
- [] I can name the three rules no single piece can own, and what they have in common.
- [] I can justify make-and-unmake with the copying arithmetic.
- [] I can list the five pieces of forgotten chess state and a rule that needs each.
- [] I answered all three questions above out loud.

Merging and sorting linked lists

DATA STRUCTURES AND ALGORITHMS

Merging and sorting linked lists

You can merge two sorted lists and sort a list in $O(n \log n)$ without extra arrays.

SYSTEM DESIGN

Design a deck of cards and a card game

You can model shuffling, dealing and hand comparison cleanly.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Merge two sorted linked lists. Now sort an unsorted one.*
- *Design a deck of cards. Now build blackjack on top of it.*

1 What this is, and why they ask it

Merging two sorted linked lists is the easiest hard-sounding problem there is: walk both, always take the smaller front node, and when one runs out attach the rest of the other in one assignment. No nodes are created and nothing is copied — you relink what already exists.

Then the follow-up turns it into the real question: **sort an unsorted linked list**. Split it in half, sort each half, merge. That is **merge sort**, and on a linked list it is not merely one option among several — it is the *right* one, for a reason worth being able to state: quicksort needs random access to partition efficiently, and a linked list has none, while merge sort needs only sequential access and, unlike on an array, needs no scratch space to merge into.

They ask it because it composes three days in a row. The dummy-head builder from [day 080](#) does the merge, the runner from [day 082](#) does the split, and the whole thing is [day 053](#)'s merge sort with pointers instead of slices. It is LeetCode 21, 148 and 23, and “merge k sorted lists” is one of the most-asked hard questions at Amazon and Google.

2 The story

The tomato section of the mandi opens at four in the morning and Basavaraj has graded there for twenty-two years.

The buyers want them in a line on the tray, smallest at one end and largest at the other, because a hotel man walking past can then take his eighty from the middle without picking through anything.

Two farmers bring tomatoes on a Tuesday. Both of them have already graded their own — each basket is in order, small to large, because they do it at home while the lorry is loading. What Basavaraj has to do is make one line out of two.

He does not tip both baskets out and start again. He has watched people do that and it takes forty minutes.

He puts a hand over each basket, takes whichever of the two front tomatoes is smaller, and lays it down. Then again. And again. He is not comparing anything except two tomatoes at a time, and he never looks back at what he has already laid down, because

whatever he laid down last was the smallest of everything remaining and nothing smaller can turn up now.

When one basket runs out, he stops comparing entirely. He tips the rest of the other basket straight onto the end of the line, in the order it is already in, because it is already in order and there is nothing left to compare it against. That part takes four seconds and it is most of the second basket.

His nephew asked him once how the farmers get their baskets sorted at home in the first place, since they are dealing with three hundred tomatoes and no line.

Basavaraj said nobody sorts three hundred tomatoes. What his brother-in-law does is put them in pairs — just two, decide which is smaller, that is a sorted pair. Then he pushes two pairs together the same way Basavaraj pushes two baskets together, and gets a sorted four. Then two fours into an eight. He keeps doubling. By the time he has been at it half an hour there are two big graded rows and one last push, and the basket is done.

The only skill in the whole thing, Basavaraj says, is the two hands over the two baskets. Everything else is doing that again with bigger baskets.

3 The idea in plain English

Basavaraj's two hands are the merge. His brother-in-law doubling from pairs is the sort. That is the entire lesson, and the second is built from the first.

Merging two sorted lists

Walk both, always take the smaller head:

```
dummy = Node(0)
tail = dummy
while a is not None and b is not None:
    if a.value ≤ b.value:
        tail.next = a
        a = a.next
    else:
        tail.next = b
        b = b.next
    tail = tail.next
tail.next = a if a is not None else b      # the rest, in one assignment
return dummy.next
```

Three things are worth saying about those nine lines.

The dummy is the builder form from [day 080](#): `tail` needs somewhere to start, so the first node is attached exactly like the fortieth and there is no “is this the first one?” branch.

The last line is Basavaraj tipping the basket in. Whatever remains is already sorted and already linked, so you attach the whole chain with one assignment. Looping to move it node by node is correct and pointless, and noticing that is the difference between having understood the merge and having copied it.

`≤` **not** `<` **keeps the merge stable.** When two values are equal, taking from `a` first preserves the original relative order. Stability is invisible on integers and matters the moment the nodes carry anything else, and it is what makes merge sort stable. Say it.

Nothing is allocated. No new nodes, no arrays, no copies of values — only `next` pointers are rewritten. That is the property that makes this $O(1)$ extra space, and it is exactly what an array merge cannot do.

Sorting: split, sort, merge

```
def sort(head):
    if head is None or head.next is None:
        return head          # 0 or 1 nodes is already sorted
    left, right = split_in_half(head)
    return merge(sort(left), sort(right))
```

Five lines, and every one of them is a day you have already done. The split uses the runner technique from [day 082](#), and it must find the **first** middle:

```
slow = fast = head
while fast.next is not None and fast.next.next is not None:
    slow, fast = slow.next, fast.next.next
second = slow.next
slow.next = None          # THE CUT – mandatory
return head, second
```

Two details, both fatal if missed.

The first middle, not the second. With the second middle, a two-node list splits into itself and an empty list, the recursion never shrinks, and you get `RecursionError`.

The cut. Without `slow.next = None`, the two “halves” share every node from the middle onwards, so both halves are really the whole list and the recursion again never terminates.

Why merge sort and not quicksort

This is the question the interviewer is heading for, and the answer is about **access**, not about worst cases.

Quicksort on an array partitions in place by swapping elements at two indices moving toward each other. That needs random access — `arr[i]` and `arr[j]` in constant time — and a linked list has none. You *can* write a linked-list quicksort by building three lists (less, equal, greater) in one pass and concatenating, and it works, but it is $O(n^2)$ in the worst case and its pivot choice is worse because you cannot cheaply sample the middle element.

Merge sort on a linked list needs only sequential access, which is all a list offers. And here is the part that is *better* on a list than on an array: merging two arrays needs a third array to merge into, so array merge sort is $O(n)$ extra space. Merging two linked lists needs **no space at all** — you relink existing nodes.

	array	linked list
quicksort	$O(n \log n)$ avg	awkward: no random access, $O(n^2)$ worst
merge sort	$O(n \log n)$	$O(n \log n)$
merge sort extra	$O(n)$ scratch	$O(1)$ — pointers only

Merge sort is the linked list's best case and the array's worst case for space. That inversion is the answer, and it is why `sort()` on a linked list is a merge sort in every standard library that has one.

The recursion's space, and how to remove it

The recursive version uses $O(\log n)$ stack frames — depth $\log_2 n$, so about 20 at a million nodes. That is small, and it is not $O(1)$, and an interviewer asking for strictly constant space wants the **bottom-up** version:

```
pass 1: merge sublists of size 1 → sorted runs of 2
pass 2: merge sublists of size 2 → sorted runs of 4
pass 3: merge sublists of size 4 → sorted runs of 8
...     log n passes
```

Basavaraj's brother-in-law making pairs, then fours, then eights. No recursion, no stack, $O(1)$ extra space — at the cost of considerably fiddlier code, because each pass has to cut out two runs of the right size, merge them, and stitch the result back into a growing output.

Write the recursive one first. Mention the bottom-up one and its trade honestly: *log n stack frames against about twenty lines of extra bookkeeping.*

Merging k sorted lists

The escalation, and it has three answers of increasing quality.

One at a time. Merge list 1 with list 2, then that with list 3, and so on. Each merge walks everything merged so far, so the total is $n + 2n + 3n + \dots = O(k^2 n/k)$ — for k lists of n/k nodes each, that is $O(nk)$. On 10,000 lists this is hopeless.

A min-heap of the k current heads. Pop the smallest, attach it, push its successor. Each of the n nodes goes through a heap of size k , so $O(n \log k)$. This is the standard answer.

Divide and conquer. Merge lists in pairs, then merge the results in pairs, and so on — $\log k$ rounds, each touching all n nodes, so also $O(n \log k)$, with no heap and no extra data structure at all. Slightly faster in practice and much easier to get right.

Give the heap answer and then say you would probably write the pairwise one, because it reuses the two-list merge you have already written and has nothing to go wrong.

4 The picture

Merging $[1, 3, 5]$ and $[2, 4]$.

```
a: [1] → [3] → [5] → None
b: [2] → [4] → None

dummy → ?                                tail = dummy

1 ≤ 2   take a       dummy → [1]          a = [3], tail = [1]
3 > 2   take b       dummy → [1] → [2]     b = [4], tail = [2]
3 ≤ 4   take a       dummy → [1] → [2] → [3] a = [5], tail = [3]
5 > 4   take b       ... → [4]             b = None, tail = [4]

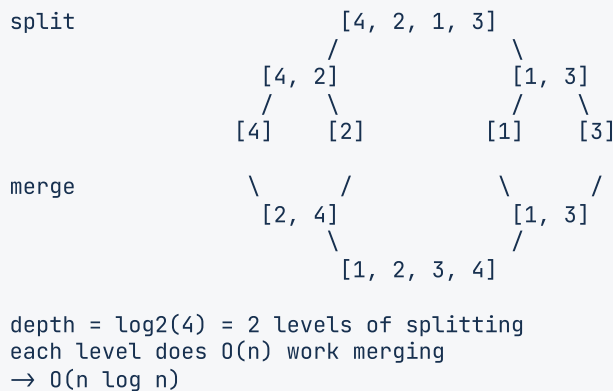
b is exhausted → tail.next = a      ONE assignment attaches [5]

result: [1] → [2] → [3] → [4] → [5]

nodes created: 0.  values copied: 0.  only `next` pointers were rewritten.
```

What to notice: the final step attached the whole of what was left of `a` with one assignment. Basavaraj tipping the basket in. On a merge where one list is much longer than the other, that single line does most of the work.

The sort, drawn as the recursion tree on $[4, 2, 1, 3]$:



What to notice: **every level does exactly n units of work**, because every node is looked at once per level, and there are $\log_2 n$ levels. That is the whole complexity argument, in a picture.

And the split, where both mistakes live:

`[1] → [2] → [3] → [4]`

FIRST middle (while fast.next and fast.next.next) slow ends on [2]
`second = [3]; slow.next = None`
`→ [1, 2] and [3, 4]` both halves shrink. Correct.

SECOND middle (while fast and fast.next) slow ends on [3]
`→ [1, 2, 3] and [4]` still shrinks here, BUT on [1, 2]:
 slow ends on [2], `second = None`
 `→ [1, 2] and []` – no shrink, infinite recursion

NO CUT (forgetting `slow.next = None`)
`→ "[1, 2]" is really [1, 2, 3, 4] because it still points onward`
 both halves are the whole list. Infinite recursion.

5 The code, built step by step

Step 1 – the merge, and the line that does the most work

```
tail.next = a if a is not None else b
```

Written before the loop in your head, not after. It is the reason the merge is $O(n + m)$ rather than something worse, and it is the line that shows you understood that the remainder is already sorted.

Step 2 — the split, with the cut written immediately

```
second = slow.next
slow.next = None
```

Two lines, always together. Write the cut in the same keystroke as the split or you will forget it, and forgetting it does not raise — it hangs.

Step 3 — the recursion, and its base case

```
if head is None or head.next is None:
    return head
```

Both conditions. `head is None` for an empty list; `head.next is None` for a single node. Without the second, `split_in_half` on one node returns the node and `None`, and `sort(None)` returns `None`, and the merge is fine — but the recursion has not shrunk on the way in, and some formulations spin. Write both.

Step 4 — merging k lists, pairwise

```
while len(lists) > 1:
    merged = []
    for i in range(0, len(lists), 2):
        first = lists[i]
        second = lists[i + 1] if i + 1 < len(lists) else None
        merged.append(merge_two(first, second))
    lists = merged
return lists[0] if lists else None
```

Six lines, no heap, and it reuses `merge_two` unchanged. The `if i + 1 < len(lists)` handles an odd count by merging the last list with `None`, which `merge_two` already handles correctly — so there is no special case.

Step 5 — bottom-up, for strict $O(1)$ space

```
size = 1
while size < length:
    dummy = Node(0)
    tail, current = dummy, head
    while current is not None:
        left = current
        right = _split_off(left, size)          # cut `size` nodes, return the rest
        current = _split_off(right, size)
        tail = _append_merged(tail, merge_two(left, right))
    head = dummy.next
    size *= 2
return head
```

Two nested loops: the outer doubles the run length, the inner walks the list cutting out pairs of runs. Every cut is a `next = None`, and every stitch is one assignment. It is genuinely fiddlier, and it is the honest answer to “can you do it in $O(1)$ space”.

The complete solution

```
class Node:
    __slots__ = ("value", "next")

    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value
        self.next = next

def from_values(values: list[int]) → Node | None:
    head: Node | None = None
    for value in reversed(values):
        head = Node(value, head)
    return head

def to_values(head: Node | None) → list[int]:
    out, node = [], head
    while node is not None:
        out.append(node.value)
        node = node.next
    return out

def merge_two(a: Node | None, b: Node | None) → Node | None:
    """Merge two sorted lists into one sorted list.

    Nothing is created and nothing is copied – only `next` pointers move, so
    this is O(1) extra space, which an array merge cannot manage.

    `≤` rather than `<` keeps the merge STABLE: equal values keep their
    original relative order, which is what makes merge sort stable.
    """
    dummy = Node(0)
    tail = dummy

    while a is not None and b is not None:
        if a.value ≤ b.value:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next

    tail.next = a if a is not None else b    # the rest is already sorted: one assignment
    return dummy.next

def split_in_half(head: Node) → tuple[Node, Node | None]:
    """Cut at the FIRST middle, so both halves are strictly smaller.

    The second middle would leave [1,2] as [1,2] and [], and the recursion
    would never terminate. The cut is mandatory for the same reason.
    """
    slow = fast = head
    while fast.next is not None and fast.next.next is not None:
        slow = slow.next
        fast = fast.next.next
    second = slow.next
    slow.next = None    # THE CUT
    return head, second

def sort(head: Node | None) → Node | None:
    """Merge sort. O(n log n) time, O(log n) stack space.

    Merge sort rather than quicksort because a linked list has no random
    access – and because merging lists needs no scratch space at all, where
```

```

merging arrays needs a second array.
"""
if head is None or head.next is None:    # both conditions
    return head
left, right = split_in_half(head)
return merge_two(sort(left), sort(right))

def merge_k_pairwise(lists: list[Node | None]) → Node | None:
    """Merge k sorted lists.  $O(n \log k)$  time, and it reuses merge_two unchanged.

    Merging them one at a time would be  $O(nk)$ : each merge re-walks everything
    merged so far. Pairwise rounds halve the number of lists each time, so
    every node is touched  $\log k$  times.
    """
    lists = [item for item in lists if item is not None]
    if not lists:
        return None

    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
            second = lists[i + 1] if i + 1 < len(lists) else None
            merged.append(merge_two(lists[i], second))    # merge_two handles None
        lists = merged
    return lists[0]

def merge_k_heap(lists: list[Node | None]) → Node | None:
    """The same thing with a min-heap of the current heads. Also  $O(n \log k)$ .

    The `index` in the tuple is a tie-breaker: without it, Python compares the
    Node objects when two values are equal and raises TypeError.
    """
    import heapq

    heap: list[tuple[int, int, Node]] = []
    for index, node in enumerate(lists):
        if node is not None:
            heapq.heappush(heap, (node.value, index, node))

    dummy = Node(0)
    tail = dummy
    while heap:
        _, index, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next is not None:
            heapq.heappush(heap, (node.next.value, index, node.next))
    tail.next = None    # the last node still points into its old list
    return dummy.next

def sort_bottom_up(head: Node | None) → Node | None:
    """Merge sort with no recursion:  $O(n \log n)$  time,  $O(1)$  space.

    Pass 1 merges runs of 1 into runs of 2, pass 2 merges 2s into 4s, and so
    on for  $\log n$  passes. Fiddlier, and it is the honest answer to "strictly
    constant space".
    """
    if head is None or head.next is None:
        return head

    length = 0
    node = head
    while node is not None:
        length += 1
        node = node.next

    dummy = Node(0, head)
    size = 1

```

```

while size < length:
    previous, current = dummy, dummy.next
    while current is not None:
        left = current
        right = _cut(left, size)          # left keeps `size` nodes; right is the rest
        current = _cut(right, size)       # right keeps `size` nodes; current is the rest
        previous.next = merge_two(left, right)
        while previous.next is not None:
            previous = previous.next      # walk to the end of what we just merged
        size *= 2
    return dummy.next

def _cut(head: Node | None, size: int) → Node | None:
    """Keep `size` nodes on `head`, cut them off, and return the remainder."""
    node = head
    while size > 1 and node is not None:
        node = node.next
        size -= 1
    if node is None:
        return None
    rest = node.next
    node.next = None
    return rest

if __name__ == "__main__":
    print(to_values(merge_two(from_values([1, 3, 5]), from_values([2, 4])))) # [1,2,3,4,5]
    print(to_values(merge_two(from_values([]), from_values([2]))))           # [2]
    print(to_values(merge_two(None, None)))                                 # []
    print(to_values(merge_two(from_values([1, 1]), from_values([1]))))       # [1, 1, 1]

    left, right = split_in_half(from_values([1, 2, 3, 4]))
    print(to_values(left), to_values(right))                               # [1, 2] [3, 4]
    left, right = split_in_half(from_values([1, 2, 3]))
    print(to_values(left), to_values(right))                               # [1, 2] [3]
    left, right = split_in_half(from_values([1, 2]))
    print(to_values(left), to_values(right))                               # [1] [2]

    print(to_values(sort(from_values([4, 2, 1, 3]))))                      # [1, 2, 3, 4]
    print(to_values(sort(from_values([-1, 5, 3, 4, 0]))))                  # [-1, 0, 3, 4, 5]
    print(to_values(sort(from_values([2, 2, 1, 1]))))                      # [1, 1, 2, 2]
    print(to_values(sort(from_values([1]))))                               # [1]
    print(to_values(sort(None)))                                           # []

    print(to_values(sort_bottom_up(from_values([4, 2, 1, 3, 9, 7]))))      # [1,2,3,4,7,9]

    ks = [from_values([1, 4, 5]), from_values([1, 3, 4]), from_values([2, 6])]
    print(to_values(merge_k_pairwise(ks)))                                # [1,1,2,3,4,4,5,6]
    ks = [from_values([1, 4, 5]), from_values([1, 3, 4]), from_values([2, 6])]
    print(to_values(merge_k_heap(ks)))                                    # [1,1,2,3,4,4,5,6]
    print(to_values(merge_k_pairwise([]), to_values(merge_k_pairwise([None])))) # [] []

    import random
    for _ in range(2000):
        sample = [random.randint(-20, 20) for _ in range(random.randint(0, 15))]
        assert to_values(sort(from_values(sample))) == sorted(sample), sample
        assert to_values(sort_bottom_up(from_values(sample))) == sorted(sample), sample
    print("both sorts agreed with sorted() on 2000 random inputs")

```

6 What it costs

Merging two lists

```
time:    every node from each list is attached exactly once →  $O(n + m)$ 
space:   one dummy node and one tail reference →  $O(1)$ 
```

Compare with merging two **arrays**, which is also $O(n + m)$ time but needs an $O(n + m)$ output array, because you cannot write into either input without destroying what you have not read yet. **The linked list version needs no scratch space**, and that single difference is why merge sort behaves better on lists than on arrays.

Sorting

```
splitting:  $O(n)$  per level, walking to find the middle
merging:    $O(n)$  per level, every node attached once
levels:     $\log_2(n)$ 
-----
time:       $O(n \log n)$ 
space:      $O(\log n)$  recursion stack ( $O(1)$  bottom-up)
```

Count it out: at $n = 1,000,000$, $\log_2 n$ is about 20, so twenty levels of n work each — about 2×10^7 node visits, and twenty stack frames.

Against the alternatives:

```
copy values to a list, sort, write back:  $O(n \log n)$  time,  $O(n)$  SPACE,  $n$  writes
merge sort in place:                     $O(n \log n)$  time,  $O(\log n)$  space, 0 allocations
bubble/insertion sort on the list:       $O(n^2) - 10^{12}$  at  $n = 10^6$ 
```

The first is a legitimate answer if you say what you traded. It is also usually what the interviewer is testing you *not* to reach for.

Merging k lists

```
one at a time: merge 1+2, then that + 3, ...
                total node visits =  $n/k * (1 + 2 + \dots + k) \approx n \cdot k/2$  →  $O(nk)$ 
heap of heads:  each of  $n$  nodes pushed and popped once, heap size  $k$  →  $O(n \log k)$ 
pairwise:        $\log_2(k)$  rounds, each touching all  $n$  nodes →  $O(n \log k)$ 
```

```
k = 10,000 lists, n = 1,000,000 nodes:
one at a time:  ~5 × 109 operations
O(n log k):    106 × 13 ≈ 1.3 × 107
ratio:         about 380x
```

And between the two good answers: the heap does n pushes and n pops with a comparison-heavy constant, while pairwise does plain merges. **Pairwise is usually faster in practice and has nothing to go wrong**, which is why it is worth naming even though

the heap is the “expected” answer.

Stability

Merge sort is stable when the merge uses $\leq$, and that is free — one character. Quicksort is not stable without extra work. On a list of records sorted by one key and then another, stability is the difference between a correct result and a subtly wrong one, and it is worth one sentence.

7 The traps

Trap 1 — splitting at the second middle

```
while fast is not None and fast.next is not None:    # the SECOND middle
```

On `[1, 2]`, `slow` ends on node 2, so `second` is `None` and the “halves” are `[1, 2]` and `[]`. The recursion does not shrink:

```
RecursionError: maximum recursion depth exceeded
```

For splitting, always the first middle. This is the same distinction as [day 082](#), and this is the problem where getting it wrong is fatal rather than merely different.

Trap 2 — forgetting the cut

```
second = slow.next
return head, second    # forgot: slow.next = None
```

Both halves are the whole list, because the first still points onward into the second. Same `RecursionError`, different cause, and it is harder to see because the split *looks* right.

Trap 3 — looping to attach the remainder

```
while a is not None:    # instead of tail.next = a
    tail.next = a
    a = a.next
    tail = tail.next
```

Correct, and it does node-by-node work for something one assignment achieves. Worse, it signals that you have not noticed the remainder is already sorted and already linked — which is the one insight the merge is testing.

Trap 4 — `<` instead of `<=`

Both produce a correctly sorted list of integers. `<` takes from `b` when values are equal, which reverses the relative order of equal elements and makes the sort **unstable**. Invisible on numbers, wrong on records. One character, and worth saying out loud.

Trap 5 — the heap comparing Node objects

```
heapq.heappush(heap, (node.value, node))           # no tie-breaker
```

```
TypeError: '<' not supported between instances of 'Node' and 'Node'
```

Fires the moment two nodes have equal values, because Python falls through to comparing the second element of the tuple. Add an index or a counter as a tie-breaker. This is the single most common bug in “merge k sorted lists”.

Trap 6 — the heap version not terminating the output

```
while heap:
    ...
    tail = node
    return dummy.next                               # forgot: tail.next = None
```

The final node still points into whatever followed it in its original list, so the result silently includes nodes that were already merged — or loops. Any list you build out of existing nodes must be terminated explicitly, exactly as in [day 080](#)'s partition.

Trap 7 — the base case with only one condition

```
if head is None:
    return head                                     # missing: or head.next is None
```

A single-node list is split into itself and `None`, and `sort` is called on the same single node again. Depending on how the split is written this either recurses for ever or does redundant work. Both conditions, always.

Trap 8 — copying values into a Python list “just to sort it”

```
values = sorted(to_values(head))
return from_values(values)
```

$O(n)$ extra space and `n` new nodes. It is a legitimate answer if you say what you traded — and it is almost always the thing the question exists to test you *not* to do. If you write it, write it as the baseline and then improve it.

Trap 9 — reaching for quicksort

Partitioning a linked list by swapping at two moving indices is not possible without random access. The three-list version (less, equal, greater) does work, but it is $O(n^2)$ in the worst case and its pivot selection is worse because sampling the middle costs $O(n)$. **Merge sort is the correct choice here and you should be able to say why in one sentence**, rather than because you happened to remember it.

8 In the interview

How it gets asked

- The warm-up: “Merge two sorted linked lists.” LeetCode 21, five minutes.
- The main question: “Sort a linked list in $O(n \log n)$ time.” LeetCode 148, and then “... and $O(1)$ space”, which is asking for bottom-up.
- The hard one: “Merge k sorted linked lists.” LeetCode 23.
- The reasoning probe, which is the real test: “Why merge sort rather than quicksort?”
- The stability probe: “Is your sort stable? Does it matter?”

What to say out loud, in the first ninety seconds

1. **For the merge, name what you are not doing.** “I am not creating any nodes or copying any values — I am relinking the existing ones, so this is $O(1)$ extra space.”
2. **Say the dummy’s job.** “A dummy head so the first node is attached like every other one, and I return `dummy.next`.”
3. **Say the remainder line before you write it.** “When one list runs out, the other is already sorted and already linked, so I attach the whole thing with one assignment rather than looping.”
4. **For the sort, name the three pieces.** “Split with the fast and slow pointers, sort each half recursively, merge. That is merge sort, and it is five lines because I already have the merge.”
5. **Pre-empt the quicksort question.** “Merge sort rather than quicksort, because quicksort’s partition wants random access and a list has none — and because merging lists needs no scratch space, whereas merging arrays needs a second array. Merge sort is the list’s best case.”
6. **Flag the two fatal split details.** “Two things I have to get right: the *first* middle, so both halves shrink, and the cut, so they are actually separate lists.”

The follow-ups

“Why merge sort and not quicksort?” “Two reasons, and the second is the interesting one. First, quicksort partitions by moving two indices toward each other and swapping, which needs random access — a linked list has none, and walking to an index is $O(n)$. You can write a list quicksort by building three sublists, but it is $O(n^2)$ in the worst case and you cannot cheaply sample a good pivot. Second, and this is what makes merge sort actively *good* here rather than merely possible: merging two arrays needs a third array to write into, so array merge sort costs $O(n)$ scratch space. Merging two linked lists costs nothing, because you relink existing nodes. Merge sort is the array’s worst case for space and the list’s best.”

“Can you do it in $O(1)$ space?” “The recursive version is $O(\log n)$ stack — about twenty frames at a million nodes — so strictly it is not constant. For genuinely $O(1)$ I would go bottom-up: pass one merges runs of size 1 into runs of 2, pass two merges 2s into 4s, and so on for $\log n$ passes, with no recursion. Same $O(n \log n)$ time, no stack, and about twenty lines more code because each pass has to cut out two runs of the right size and stitch the result back in. I would write the recursive one first and offer this if constant space is a hard requirement.”

“Is your sort stable?” “Yes, because the merge takes from the first list when values are equal — $\leq$ rather than $<$. That is one character and it is free. It is invisible when the nodes hold integers, and it is the difference between right and wrong when they hold records that were already ordered by another key. Quicksort is not stable without extra work, which is another reason not to reach for it here.”

“Now merge k sorted lists.” “Three answers. Merging one at a time is $O(nk)$, because each merge re-walks everything merged so far — at ten thousand lists and a million nodes that is about five billion operations. A min-heap of the current heads is $O(n \log k)$: each node is pushed and popped once from a heap of size k . Or divide and conquer — merge them in pairs, then merge the results in pairs, for $\log k$ rounds, which is also $O(n \log k)$. I would write the pairwise one, because it reuses the two-list merge unchanged and has nothing to go wrong. The heap version has a trap: you must include a tie-breaker in the tuple, or Python compares two Node objects when values are equal and raises `TypeError`.”

“What is the complexity, and where does the $\log n$ come from?” “Each level of the recursion does $O(n)$ work — every node is walked once during splitting and attached once during merging — and the depth is $\log_2 n$, because the list halves each time. So $O(n \log n)$. At a million nodes that is twenty levels of a million, about two times ten to the seven node visits. Space is $O(\log n)$ for the stack, not $O(n)$, because nothing is allocated per level.”

“Why not just copy the values into an array and sort that?” “You can, and it is $O(n \log n)$ too. What it costs is $O(n)$ extra space plus rebuilding the list, and it throws away node identity — anything holding a reference to a specific node now points into the old structure. I would say it as the baseline and then give the in-place version, because the in-place one is what the question is asking for.”

A model answer

Asked: *merge two sorted linked lists, then sort an unsorted one.*

“The merge first. I walk both lists and always take whichever front node is smaller, attaching it to the end of the result. The important thing to say is what I am *not* doing: I am not creating any nodes and not copying any values. Only `next` pointers move, so this is $O(1)$ extra space — which an array merge cannot manage, and that turns out to matter for the second half of the question.

Two details in the loop. I use a dummy head so the tail cursor has somewhere to start and the first node is attached exactly like every other, and I return `dummy.next`. And when one list runs out, I attach *the whole remainder* with a single assignment rather than looping — whatever is left is already sorted and already linked, so there is nothing to do to it.

One more: I compare with `≤`, not `<`, so that when values are equal I take from the first list. That keeps the merge stable, which is invisible on integers and matters as soon as the nodes carry records.

Now sorting. Split the list in half, sort each half, merge the two results. That is merge sort, and it is five lines because the merge already exists.

Two things in the split are fatal if I get them wrong. I must take the *first* middle, not the second — with the second, a two-node list splits into itself and an empty list, the recursion never shrinks, and it blows the stack. And I must cut: set the first half's last node's `next` to `None`, or the two halves are really the same list and it never terminates either. I write the cut in the same breath as the split for that reason.

Why merge sort and not quicksort — I think that is the actual question. Quicksort partitions by moving two indices toward each other and swapping, which needs random access, and a linked list has none. You can write a list quicksort with three sub-lists, and it is $O(n^2)$ in the worst case with a pivot you cannot choose well. But the better reason is the other way round: merging two arrays needs a third array to write into, so array merge sort costs $O(n)$ scratch space. Merging two lists costs nothing. Merge sort is the array's worst case for space and the linked list's best.

Complexity: every level does $O(n)$ work and there are $\log_2 n$ levels, so $O(n \log n)$. Space is $O(\log n)$ for the recursion — about twenty frames at a million nodes. If you want strictly $O(1)$, the bottom-up version merges runs of one into runs of two, then twos into fours, with no recursion; same time, no stack, and about twenty more lines of bookkeeping.

Before I called it done I would run: an empty list, a single node, two nodes, all-equal values, and already-sorted input — and I would cross-check against Python's `sorted` on a few thousand random inputs, because the split is exactly the kind of code that is right on the cases you think of and wrong on the ones you do not.”

9 Recall card

- The merge creates nothing and copies nothing — only `next` pointers move, so it is $O(1)$ extra space. Use a **dummy builder** so the first node attaches like every other, `≤ not <` so the merge is **stable**, and attach the remainder **in one assignment** (`tail.next = a if a else b`), because whatever is left is already sorted and already linked.
- **Sort = split, sort each half, merge — five lines, all of them days you have already done.** Two fatal details in the split: the **FIRST middle** (`while fast.next and fast.next.next`), or `[1,2]` splits into itself and nothing and you get `RecursionError`; and the **cut** (`slow.next = None`), or both halves are the whole list. Base case needs **both** `head is None` **and** `head.next is None`.
- **Why merge sort, in one sentence each.** Quicksort's partition needs **random access**, which a list does not have. And merging **arrays** needs a scratch array while merging **lists** needs nothing — so **merge sort is the array's worst case for space and the list's best.**
- $O(n \log n)$ time — every level does `n` work, and there are `log2 n` levels (≈ 20 at a million nodes). Space is $O(\log n)$ stack, not $O(1)$; the **bottom-up** version (merge runs of $1 \rightarrow 2 \rightarrow 4 \dots$) is genuinely $O(1)$ at the cost of ~ 20 fiddlier lines. Copying values into a Python list is $O(n)$ space and throws away node identity.
- **Merge k lists: one at a time is $O(nk)$ ($\sim 5 \times 10^9$ at $k = 10^4$, $n = 10^6$); a heap of the k heads is $O(n \log k)$; pairwise rounds are also $O(n \log k)$ and reuse `merge_two` unchanged — prefer pairwise.** The heap's trap: **include a tie-breaker in the tuple** or `TypeError: '<' not supported between instances of 'Node'`, and **terminate the output with** `tail.next = None`.

1 What this is, and why they ask it

A deck of cards is fifty-two things with two attributes each. It is the smallest design prompt there is, and it is asked precisely because there is nothing to hide behind — the whole interview is in three decisions.

One: a card is a value object, not an entity. The seven of hearts has no identity beyond its suit and rank; two of them are the same card. That means immutable, with equality and hashing by value.

Two: a card has no value. This is the one candidates get wrong. An ace is worth 1 or 11 in blackjack, is the highest card in poker and the lowest in some rummy variants. `Card.value` is a *game* rule wearing a card's clothes, and putting it on `Card` means the deck can only ever play one game.

Three: shuffling is an algorithm with a wrong version that looks right. “For each card, swap it with a random card” is the natural thing to write and it is measurably biased. The correct one is Fisher–Yates, it is three lines, and knowing why the other one is wrong is the part being tested.

They ask it as a warm-up before a longer round, and because the follow-up — *now build blackjack* — immediately exposes whether your deck knew about games.

2 The story

Sunday afternoons at Prakash's house have been cards for as long as anyone can remember, and the same pack has been in the drawer under the television for about nine years.

They play three different games depending on who has turned up.

In the first one the queen is the highest card in the pack and the ace is nothing much. In the second the ace is worth one, the picture cards are worth ten each, and you are trying to reach a number. In the third the ace beats everything, including the queen, and the twos are wild.

It is the same fifty-two cards. Nobody has ever suggested buying a different pack for a different game, and if you asked Prakash what the ace of spades is *worth* he would say it depends what we are playing, in the tone of somebody being asked something silly.

The argument, which happens about twice a year, is about Vasu's shuffling.

Vasu shuffles the way he has always shuffled. He cuts the pack roughly in half, pushes the two halves together, squares them up, and does it three times. It looks like shuffling. It sounds like shuffling.

Prakash's nephew, who is at engineering college and is insufferable about it, sat down last Deepavali and wrote down every hand for two hours. Then he showed them: the three cards that had been together at the end of one hand were still together, in the same order, four hands later. Not always. Often enough that he had a page of them.

Vasu said that was coincidence. The nephew said it was not, and that a proper shuffle means every one of the orders the pack could be in is equally likely, and three quick riffles does not come close.

Nobody changed their mind that evening. But Prakash's daughter, who plays for money with people from her office, does it differently now. She spreads the whole pack face down on the table and pushes them around with both hands for a good thirty seconds before gathering them up, and she does that every single hand, and she will not start until she has.

3 The idea in plain English

Prakash's answer — *it depends what we are playing* — is the design. And the nephew's page of repeated sequences is the shuffling half.

A card is a value object

```
class Suit(Enum):
    CLUBS = "♣"
    DIAMONDS = "♦"
    HEARTS = "♥"
    SPADES = "♠"

class Rank(Enum):
    TWO = "2"
    ...
    ACE = "A"

@dataclass(frozen=True)
class Card:
    rank: Rank
    suit: Suit
```

`frozen=True` gives you three things at once, and they are exactly the three from [day 065](#). Immutability, so a card cannot be edited after it is dealt. `__eq__` by value, so two seven-of-hearts objects are equal. And `__hash__`, so cards can go in sets and be dictionary keys — which you need the moment you want “which cards have been played”.

No **value field**. That is the decision, and §3 continues with why.

A card has no value — the game decides

This is the same question as [day 082](#)’s late fee: *where does the rule live?* And the answer has the same shape.

ace, in blackjack:	1 or 11, whichever helps	→ a rule about a hand, not a card
ace, in poker:	high, or low in a wheel	→ a rule about a hand
ace, in rummy:	1, or sometimes high	→ depends on the variant
king, in blackjack:	10	
king, in a trick game:	beats a queen, worth 0	→ ordering, not arithmetic

Not one of those is a property of the piece of card. Put `value = 11` on the ace and your deck can play exactly one game, and the second game forces either a second `Card` class or a flag.

So `Rank` is an **identity**, not a number. The game supplies a mapping:

```
class HandEvaluator(Protocol):
    def score(self, cards: list[Card]) → int: ...
```

- `BlackjackEvaluator` — face cards are 10, an ace is 11 unless that busts, then 1.
- `PokerEvaluator` — returns a category and tie-breakers, not a single number at all.

Two implementations that do not even agree on the *return type* of “score”, which is the strongest possible evidence that this does not belong on `Card`.

If ranks need an ordering for a trick game, give `Rank` an ordinal — `Rank.KING.order = 13` — and let the *game* decide whether higher wins, whether the ace wraps, and whether trumps override it.

Shuffling: the wrong version, measured

The natural thing to write:

```
for i in range(n):
    j = random.randint(0, n - 1)          # ANY position, including ones already done
    cards[i], cards[j] = cards[j], cards[i]
```

It looks fine and it is biased. The reason is counting: this makes `n` independent choices from `n` options, so there are `nn` equally likely execution paths — but only `n!` possible orderings. `nn` is not divisible by `n!` for `n > 2`, so the orderings **cannot** come out

equally likely.

Enumerated exhaustively for three cards:

27 equally likely paths, 6 possible orders

```
(0,1,2) 4/27 = 14.81%
(0,2,1) 5/27 = 18.52%
(1,0,2) 5/27 = 18.52%
(1,2,0) 5/27 = 18.52%
(2,0,1) 4/27 = 14.81%
(2,1,0) 4/27 = 14.81%
```

uniform would be 16.67%. Some orders are 25% more likely than others.

That is not a rounding artefact; it is exact arithmetic. And it is exactly the kind of bias a player who watches for two hours will find — Prakash’s nephew with his page of repeats.

The correct algorithm, **Fisher–Yates**, walks from the end and picks only from the *unshuffled* prefix:

```
for i in range(n - 1, 0, -1):
    j = random.randint(0, i)          # 0..i, NOT 0..n-1
    cards[i], cards[j] = cards[j], cards[i]
```

One character group different, and now the number of paths is $n \times (n-1) \times \dots \times 2 = n!$ — exactly one path per ordering, so every ordering is equally likely. For three cards: six paths, six orders, one each.

In Python you write `random.shuffle(cards)`, which is Fisher–Yates. **Write the loop in the interview to show you know it, then say you would call the library.**

The randomness itself, if money is involved

```
52! = 8.07 × 10^67 possible orderings, which needs about 226 bits to index
Python's Mersenne Twister has 19,937 bits of state – enough
but if it is seeded from a 32-bit value, only 2^32 ≈ 4.3 × 10^9 shuffles are reachable
→ 4 billion out of 8 × 10^67. Effectively none of them.
```

And Mersenne Twister is **not** cryptographically secure: observe enough outputs and the internal state can be reconstructed, after which every future “shuffle” is predictable. For a game with money on it you use `secrets` or `random.SystemRandom`, which draw from the operating system’s entropy source. Saying this unprompted is one of the highest-value sentences available in this prompt.

Dealing, and the shoe

A `Deck` deals from one end and knows how many remain. Casinos use a **shoe** of six or eight decks and do not deal it to the end — they place a cut card at about 75 percent, so about a quarter is never seen. That is specifically to defeat card counting, and it is a design constraint that comes from the domain rather than from engineering.

```
class Shoe:
    def __init__(self, decks: int = 6, penetration: float = 0.75) → None:
        self._cards = [c for _ in range(decks) for c in standard_deck()]
        random.shuffle(self._cards)
        self._cut = int(len(self._cards) * penetration)
        self._dealt = 0
```

Blackjack, and the only interesting rule in it

A hand's value depends on the aces, and the standard mistake is to try to decide each ace as it arrives. You cannot — an ace's value depends on the cards that come *after* it.

The clean way is two lines:

```
total = sum(10 if c.rank in FACES else 11 if c.rank is Rank.ACE else int(c.rank.value)
            for c in cards)
aces = sum(1 for c in cards if c.rank is Rank.ACE)
while total > 21 and aces:                # downgrade aces from 11 to 1, one at a time
    total -= 10
    aces -= 1
```

Count every ace as 11, then downgrade while you are bust. No branching per card, no ordering problem, and it handles four aces correctly ($11 + 1 + 1 + 1 = 14$). A hand that still counts an ace as 11 is called **soft**, and that matters because the dealer's rule is often “stand on hard 17, hit on soft 17” — so the evaluator has to report softness, not just a number.

4 The picture

The classes, with the line that matters marked:

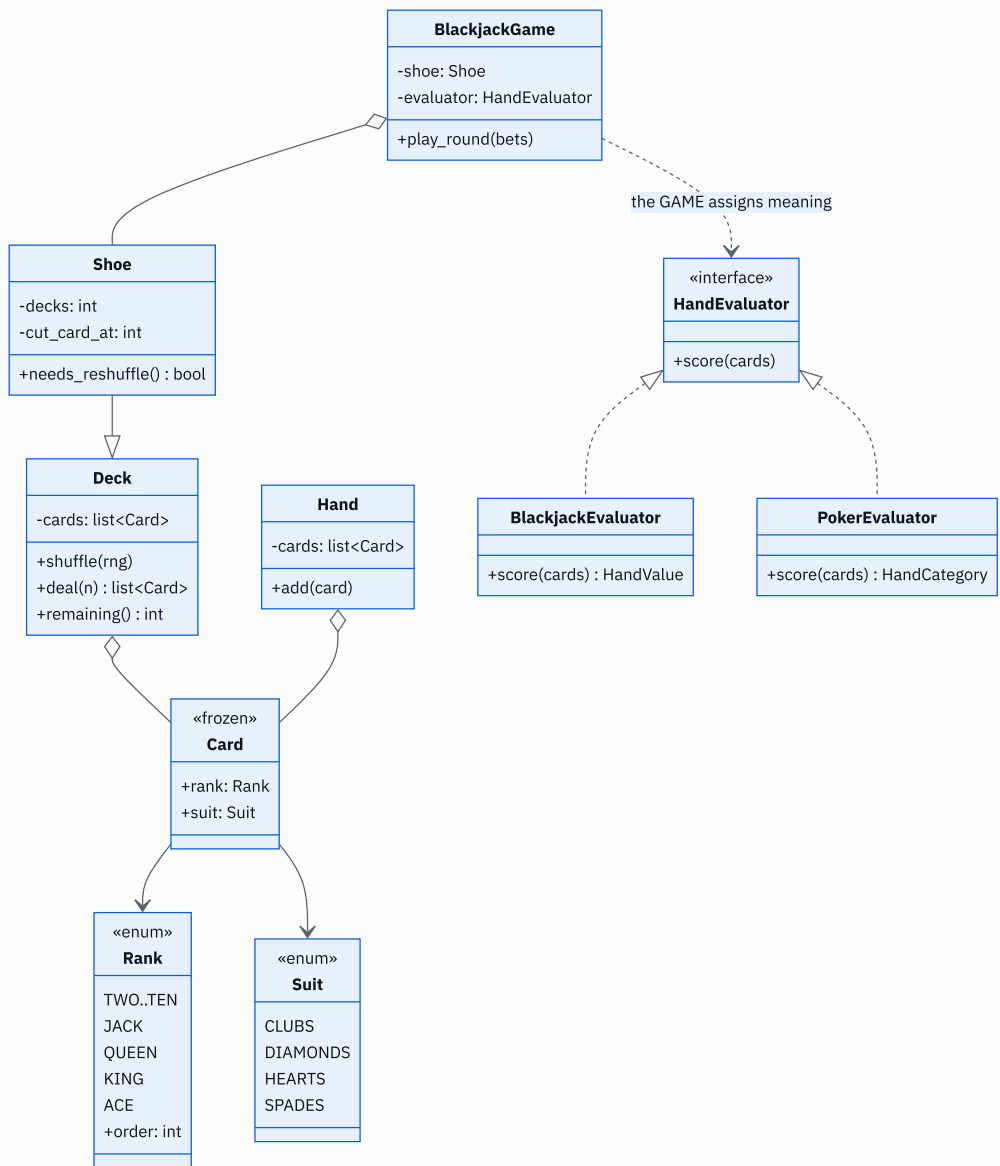


FIGURE 84.1

What to notice: **there is no arrow from Card to anything about value or scoring.** Card points at Rank and Suit and stops. The two evaluators do not even return the same type — one returns a number and softness, the other returns a hand category with tie-breakers — which is the proof that “the value of a card” was never a single idea.

The shuffle bias, drawn as counts rather than described:

THREE CARDS

naive: swap *i* with a random position in 0..*n*-1
27 equally likely execution paths, 6 possible orders

order	paths	probability	
(0,1,2)	4	14.81%	← under
(0,2,1)	5	18.52%	← over
(1,0,2)	5	18.52%	← over
(1,2,0)	5	18.52%	← over
(2,0,1)	4	14.81%	← under
(2,1,0)	4	14.81%	← under

uniform = 16.67%

27 / 6 = 4.5, which is not a whole number, so equal probability is IMPOSSIBLE.

Fisher-Yates: swap *i* with a random position in 0..*i*
3 x 2 x 1 = 6 paths, 6 orders, exactly one path each. Uniform, provably.

And the shoe:

6 decks = 312 cards

```
|<----- 234 dealt (75%) ----->|← 78 never seen →|
[##### | cut card | .....]
                        ^
                    reshuffle when reached
```

the last quarter is deliberately never dealt – that is what defeats counting

5 How it actually works

Move 1 • Clarify (minutes 0–5)

“One deck or a shoe?” — Ask, because it changes Deck into Shoe with a cut card. “Which game, or several?” — This is the question that decides the whole design. If it is several, the deck must know nothing about scoring. “Is money involved?” — Because that changes the random number generator from random to secrets, and it is not a detail. “Do I need to support undo, replay or an audit trail?” — A dealt-cards log rather than mutation.

“I will assume a standard 52-card deck with no jokers, that a card is never modified once created, and that we start with blackjack but should be able to add another game without touching the deck.”

Move 2 · The nouns (minutes 5–10)

- `Suit`, `Rank` — enums. Not strings, so a typo fails at the boundary rather than silently.
- `Card` — frozen dataclass of a rank and a suit. **No value.**
- `Deck` — the cards, shuffling and dealing. Knows no game.
- `Shoe` — several decks plus a cut card. Extends or wraps `Deck`.
- `Hand` — the cards one player holds.
- `HandEvaluator` (*interface*) — turns cards into whatever this game means by a score.
- `BlackjackGame` — turn order, dealer rules, betting, and the round.

Seven, one interface. The gate: can you name a second implementation? Blackjack and poker, and they do not even share a return type. Easily justified.

Move 3 · The card and the deck

```
@dataclass(frozen=True, order=False)
class Card:
    rank: Rank
    suit: Suit

    def __str__(self) → str:
        return f"{self.rank.value}-{self.suit.value}"
```

`order=False` deliberately: cards are **not** orderable, because “is the king greater than the ace” is a question with no answer until a game is named. Making them orderable would invite exactly the bug this design exists to prevent.

```
class Deck:
    def __init__(self, cards: list[Card] | None = None, rng: random.Random | None = None) → None:
        self._cards = cards if cards is not None else standard_deck()
        self._rng = rng or random.SystemRandom()      # injected: testable AND secure
        self._dealt = 0
```

The random number generator is **injected**, and that is worth a sentence: it makes the deck testable with a seeded `random.Random(42)`, and it lets production use `SystemRandom`. A deck that calls `random.shuffle` directly can be neither tested deterministically nor made secure.

```
def shuffle(self) → None:
    """Fisher-Yates. Written out to show the algorithm; in production this
    is exactly what random.shuffle does."""
    cards = self._cards
    for i in range(len(cards) - 1, 0, -1):
        j = self._rng.randint(0, i)          # 0..i, NOT 0..n-1
        cards[i], cards[j] = cards[j], cards[i]
    self._dealt = 0
```

The comment on `randint(0, i)` is the whole lesson. Everything else about this method is bookkeeping.

```
def deal(self, count: int = 1) → list[Card]:
    if count > len(self._cards) - self._dealt:
        raise OutOfCards(f"{count} requested, {self.remaining()} left")
    dealt = self._cards[self._dealt:self._dealt + count]
    self._dealt += count
    return dealt
```

Dealing moves an index rather than popping, which keeps the shuffled order intact for an audit trail — the same head-marker idea as the queue on [day 073](#), and for a similar reason.

Move 4 • The evaluator, which is the interesting part

```
@dataclass(frozen=True)
class HandValue:
    total: int
    soft: bool
    # is an ace still counting as 11?

    @property
    def is_bust(self) → bool:
        return self.total > 21

class BlackjackEvaluator:
    TEN_VALUED = {Rank.TEN, Rank.JACK, Rank.QUEEN, Rank.KING}

    def score(self, cards: list[Card]) → HandValue:
        total = 0
        aces = 0
        for card in cards:
            if card.rank is Rank.ACE:
                aces += 1
                total += 11
            elif card.rank in self.TEN_VALUED:
                total += 10
            else:
                total += int(card.rank.value)

        while total > 21 and aces > 0:
            total -= 10
            aces -= 1
            # then downgrade, one at a time

        return HandValue(total, soft=aces > 0)
```

Count high, then downgrade. Say why out loud: an ace's value depends on the cards that come *after* it, so deciding per card as it arrives is impossible. This handles `A A A A` correctly — 44, then 34, 24, 14 — which is the case people's per-card version gets wrong.

`soft` is returned because the dealer's rule needs it: *stand on hard 17, hit on soft 17* is a real casino variant, and an evaluator returning only a number cannot express it.

Move 5 • The game, which owns the rules

```
class BlackjackGame:
    def deal_round(self, players: list[Player]) → None:
        if self._shoe.needs_resuffle():
            self._shoe.resuffle()                # at the cut card, not when empty
        for _ in range(2):
            for player in players:
                player.hand.add(self._shoe.deal()[0])
            self._dealer.hand.add(self._shoe.deal()[0])

    def dealer_plays(self) → None:
        while True:
            value = self._evaluator.score(self._dealer.hand.cards)
            if value.total > 17:
                break
            if value.total == 17 and not (self.HITS_SOFT_17 and value.soft):
                break
            self._dealer.hand.add(self._shoe.deal()[0])
```

The dealer's rule is a **constant on the game**, not on the evaluator and certainly not on the cards — because it is a house rule that varies between casinos. `HITS_SOFT_17` being a class attribute you can flip is the smallest possible version of “the rules live in one replaceable place”.

Real systems

- `random.shuffle` in Python is Fisher–Yates, and the CPython source says so. So is Java's `Collections.shuffle` and JavaScript's... nothing, which is why every JavaScript project eventually contains someone's hand-rolled shuffle and about half of them are the biased version.
- `random.SystemRandom` and the `secrets` module draw from the OS entropy pool (`/dev/urandom`, `CryptGenRandom`). The default `random` is the **Mersenne Twister**: excellent statistically, and reconstructible from 624 consecutive 32-bit outputs, so useless where an adversary can watch.
- **Regulated gaming** requires certified RNGs — GLI-19 and similar standards — and the certification is about the generator and the seeding, not about the shuffle algorithm, which everybody agrees on.
- **Continuous shuffling machines** in casinos return played cards to the shoe after every hand, making the cut card irrelevant. That is a hardware answer to the same problem the cut card solves.
- **The 2001 online poker exploit** is the canonical story: a site seeded a 32-bit RNG with the clock, which reduced the reachable shuffles to a number small enough that the current deck could be deduced from a few visible cards, in real time.

6 The numbers

The size of the space you are sampling from

```
52! = 8.07 × 1067 orderings  
log2(52!) ≈ 225.6 bits needed to name one
```

For context: that is more orderings than there are atoms in the observable universe, by about twelve orders of magnitude. **A properly shuffled deck has almost certainly never existed before.**

Why the seed matters more than the algorithm

```
Mersenne Twister internal state: 19,937 bits – plenty, in principle  
seeded from a 32-bit integer: 232 = 4.29 × 109 reachable shuffles  
as a fraction of 52!: 4.29e9 / 8.07e67 ≈ 5 × 10-59
```

Five in a hundred billion billion billion billion billion billion. With a 32-bit seed, effectively none of the possible decks can ever be produced — and, worse, an observer who can guess the seed knows the whole deck. That is the arithmetic behind “use secrets if there is money on it”, and it is far more convincing than the phrase “cryptographically secure”.

The bias of the wrong shuffle, exactly

```
n = 3: 27 paths / 6 orders = 4.5 → impossible to be uniform  
actual: three orders at 4/27 (14.81%), three at 5/27 (18.52%)  
the most likely order is 25% more likely than the least  
n = 4: 256 paths / 24 orders ≈ 10.67  
n = 52: 5252 ≈ 1.7 × 1089 paths / 8.07 × 1067 orders – never divisible
```

The bias does not go away with more cards; the counting argument shows it cannot. And the practical consequence is exactly Prakash’s nephew’s page: sequences that were together stay together more often than chance.

The shoe

```
6 decks × 52 = 312 cards  
penetration 75%: deal 234, never deal the last 78  
blackjack round: ~5.5 cards per player-round with 4 players + dealer ≈ 12 cards  
rounds per shoe: 234 / 12 ≈ 19 rounds before reshuffling
```

```
a counter's edge with 100% penetration: ~1.5% over the house  
with 75% penetration: ~0.5%  
with a continuous shuffling machine: 0%
```

That is why the cut card exists, and it is a nice example of a design constraint that comes from the business rather than from the code.

Memory, so you can dismiss it

```
Card (frozen dataclass, 2 enum refs)    ~64 B
52 cards                                ~3.3 KB
a 6-deck shoe                            ~20 KB
```

Twenty kilobytes. There is nothing to optimise here, and the interesting decisions are all about correctness and responsibility. Say that early so the time goes to the right place.

The evaluator's cost

```
blackjack score: one pass over  $\leq 11$  cards, then  $\leq 4$  downgrades  $\rightarrow$  ~15 operations
poker 5-card evaluation, naive: compare against 9 categories  $\rightarrow$  ~100 operations
poker 7-card best-of-21-combinations, naive:  $21 \times 100 = 2,100$ 
production poker evaluators: a perfect-hash lookup, ~5 operations
```

Worth knowing that real poker evaluators are table lookups rather than logic, because it is the same move as the chess bitboards from [day 083](#): when the domain is small and fixed, precompute it.

7 The trade-offs

What this design gives up

Enums for rank mean arithmetic needs a mapping. `int(Rank.SEVEN.value)` is uglier than `card.value`. That ugliness is the point — it makes the game do the converting, which is where the knowledge belongs — but it is real friction and it will tempt somebody to add `Rank.numeric` “just for convenience”, after which blackjack’s ace problem quietly reappears.

Injecting the RNG makes the deck harder to construct. Every caller now has to supply or default one. The payoff is deterministic tests and a swappable secure generator, and I would take that trade every time, but it is a trade.

Dealing by moving an index means the deck never shrinks. Good for audit — you can reconstruct the entire dealt order — and it means `remaining()` is arithmetic rather than `len()`. If memory mattered you would pop, and it does not.

Shoe extending Deck is inheritance for a small gain. Composition — a shoe *holds* a deck-like list — would be more flexible, especially for a continuous shuffling machine, which is not a deck with a cut card but a fundamentally different dealing rule. If CSMs

were a requirement I would make `CardSource` an interface and have `Deck`, `Shoe` and `ContinuousShuffler` implement it.

Nothing here is thread-safe or multi-table. One shoe, one game, one thread. A real casino backend has thousands of concurrent tables, and the deck becomes per-table state in a service — at which point the interesting problems are session management and auditability, not shuffling.

No audit trail. Regulated gaming requires that every shuffle and every deal be logged and reproducible. That means recording the seed and the deal order, which is easy if the RNG is injected and impossible if it is not — another argument for the injection.

“I would change this design if...”

- **...money is involved.** `random.SystemRandom` or `secrets`, a logged seed, and a certified generator. Not optional and not a detail.
- **...there is a continuous shuffling machine.** Then `CardSource` is an interface, because a CSM has no concept of a shoe running out.
- **...jokers, or a 32-card piquet deck, or two-suit spider solitaire.** `standard_deck()` becomes a parameter, and this is easy precisely because nothing else knows what 52 means.
- **...poker is the target rather than blackjack.** The evaluator returns a category and tie-breakers rather than a number, and I would use a lookup table rather than logic, which changes the shape of that class entirely — and nothing else.

The honest concession

Almost all of this design is one decision repeated: **the deck knows nothing about the game**. The frozen card, the absence of a value field, the injected evaluator and the game-owned dealer rule are four expressions of the same idea. If the requirement were genuinely “blackjack, for ever, and nothing else”, a `Card` with a `blackjack_value` property and a `Deck` that scored hands would be shorter and perfectly defensible — and it would be rewritten from scratch the day somebody asks for rummy. The choice is a bet on the second game arriving, and in this prompt the interviewer has already told you it will.

8 In the interview

How it gets asked

- The opener: “*Design a deck of cards.*” Ten minutes.
- The follow-up that is the actual question: “*Now build blackjack on top of it.*” — and the score is how little of the deck you have to change.

- The algorithm probe: “How do you shuffle it?” and then “why is that correct?”
- The security probe: “This is for a real-money site. Does anything change?”
- The modelling probe: “Should `Card` have a `value`?” — sometimes asked as a leading question to see whether you agree.

The timed script

Minutes 0–5 · Clarify. One deck or a shoe? Which game, or several? Is money involved? Do you need an audit trail? The second and third answers change the design materially.

Minutes 5–12 · The card and the deck. `Suit` and `Rank` as enums, `Card` as a frozen dataclass, and **say the no-value decision explicitly** with the ace-in-three-games justification. Then `Deck` with an injected RNG.

Minutes 12–20 · Shuffling. Write the naive version, say why it is biased with the counting argument, then write Fisher–Yates and point at the one changed range. Then the security note about seeding.

Minutes 20–30 · Blackjack. The evaluator behind an interface, the count-high-then-downgrade algorithm, and the `soft` flag with the dealer rule that needs it.

Minutes 30–40 · The shoe, the cut card and its business reason, then failure and extension — what changes for poker, for a CSM, for real money.

The follow-ups

“Should `Card` have a `value`?” “No, and this is the decision I would defend hardest. An ace is 1 or 11 in blackjack, the highest card in poker, and something else again in rummy. A king is worth 10 in blackjack and worth nothing in a trick game where it merely beats a queen. So ‘value’ is a rule about a *game*, not a property of a card. If I put it on `Card`, the deck can play exactly one game. Instead `Rank` is an identity, and the game supplies an evaluator. The strongest evidence is that a blackjack evaluator returns a number and a poker evaluator returns a category with tie-breakers — they do not even share a return type, so ‘the value of a card’ was never one idea.”

“How do you shuffle?” “Fisher–Yates: walk from the last position down, and for each position swap it with a random position between zero and that index *inclusive* — not zero to $n-1$. That produces exactly $n!$ execution paths for $n!$ orderings, so every ordering is equally likely, and that is provable rather than empirical. In production I would call `random.shuffle`, which is this algorithm; I write it out to show I know why it is correct.”

“Why is the obvious version wrong?” “The obvious version swaps each position with a random position anywhere in the deck. That makes n independent choices from n options, so there are n -to-the- n equally likely paths but only $n!$ possible orderings — and

n-to-the-n is not divisible by $n!$ for n above two, so equal probability is arithmetically impossible. For three cards it is 27 paths over 6 orderings: three orderings come out at $4/27$, about 14.8 percent, and three at $5/27$, about 18.5 percent, against a uniform 16.7. The most likely ordering is 25 percent more likely than the least, and that is exact, not simulated.”

“It is a real-money site. What changes?” “The generator, and it matters more than the algorithm. The default `random` is a Mersenne Twister, which is statistically excellent and completely predictable once you have seen 624 outputs — so an adversary who can watch enough hands knows the deck. And the seed is the bigger problem: 52 factorial is about 8 times 10 to the 67, but a 32-bit seed reaches only about 4 billion distinct shuffles, which is effectively none of them. I would use `secrets` or `random.SystemRandom`, log the seed for audit, and for a regulated market use a certified generator. That is also why I inject the RNG rather than calling `random` directly — it makes the deck both testable and swappable.”

“How do you score a blackjack hand with aces?” “Count every ace as 11 first, then subtract 10 for each ace while the total is over 21. Two lines, and it is correct for four aces — 44 down to 14. The reason not to decide each ace as it arrives is that an ace’s value depends on the cards that come *after* it, so a per-card decision is impossible in principle. I also return whether an ace is still counting as 11 — a ‘soft’ hand — because the dealer rule is often ‘stand on hard 17, hit on soft 17’, and an evaluator that returns only a number cannot express that.”

“Why not deal all the way to the end of the shoe?” “Because of card counting. A counter’s edge grows with how much of the shoe has been seen — roughly 1.5 percent with full penetration against about half a percent at 75 percent. So the cut card is placed about three quarters of the way in and the last quarter is never dealt. It is a business constraint expressed in the design, and the modern hardware answer is a continuous shuffling machine, which returns played cards to the shoe and removes the concept entirely.”

“Now build poker instead.” “Only the evaluator changes, which is the test this design was built to pass. The deck, the card, the shuffle and the dealing are all untouched. What changes shape is the evaluator’s return type: poker needs a category plus tie-breakers rather than a single number, and a production one is a perfect-hash lookup table rather than logic, because evaluating the best five of seven cards is 21 combinations per hand and table lookups are about twenty times faster than branching.”

A model answer

Asked: *design a deck of cards, then build blackjack on top of it.*

“Three decisions carry this, and I will take them in order.

First, a card is a **value object**. The seven of hearts has no identity beyond its rank and suit — two of them are the same card — so I make it a frozen dataclass over two enums. Frozen gives me immutability, equality by value, and hashing, which I need the moment I want a set of cards already played. Enums rather than strings so that a typo fails at construction.

Second, and this is the one I would defend hardest: **a card has no value**. An ace is 1 or 11 in blackjack, the highest card in poker, and something else in rummy. A king is worth ten in blackjack and worth nothing in a trick game where it merely beats a queen. ‘Value’ is a rule about a game wearing a card’s clothes. So `Rank` is an identity, and the *game* supplies an evaluator. The evidence that this is right: a blackjack evaluator returns a number and a poker evaluator returns a category with tie-breakers. They do not share a return type, so there was never one idea called ‘the value of a card’.

Third, **shuffling has a wrong version that looks right**. The natural thing to write is: for each position, swap it with a random position anywhere in the deck. That is biased, and the argument is counting, not statistics — it makes n independent choices from n options, so n -to-the- n equally likely paths, but only n factorial orderings, and n -to-the- n is not divisible by n factorial. For three cards it is 27 paths over 6 orderings, and exactly three orderings come out at $4/27$ and three at $5/27$, so the most likely is 25 percent more likely than the least.

The correct one is Fisher–Yates: walk from the end, and swap each position with a random position from zero to *that index*, not to $n - 1$. That gives exactly n factorial paths for n factorial orderings. One changed range. In production I would call `random.shuffle`, which is exactly this.

I would also inject the random number generator rather than calling `random` directly, for two reasons: tests can seed it deterministically, and production can swap in `SystemRandom`. Which matters, because if there is money involved the default Mersenne Twister is predictable from 624 outputs, and a 32-bit seed reaches about four billion shuffles out of the 8 times 10-to-the-67 that exist — effectively none of them.

Now blackjack, and the point is how little of the above changes: nothing. A `BlackjackEvaluator` implements the evaluator interface. Its only interesting rule is aces: count every ace as eleven first, then subtract ten for each ace while the total is over twenty-one. That is two lines and it is correct for four aces. Deciding each ace as it arrives is impossible in principle, because an ace’s value depends on the cards dealt after it.

The evaluator returns the total *and* whether an ace is still counting as eleven — a soft hand — because the dealer’s rule is commonly ‘stand on hard seventeen, hit on soft seventeen’, and that rule lives on the game as a house constant, not on the evaluator and certainly not on the cards.

Two domain details I would add: a shoe of six decks rather than one deck, and a cut card at about 75 percent so the last quarter is never dealt — which exists specifically because a counter’s edge grows with penetration, roughly 1.5 percent at full against half a percent at three quarters.

If you asked for poker next, only the evaluator changes, and that is the whole reason for the shape.”

9 Recall card

- **A card is a value object: frozen dataclass over `Rank` and `Suit` enums** — immutability, equality by value and hashing all come free, and a typo fails at construction rather than silently. Deliberately **not `orderable`**, because “is the king above the ace” has no answer until a game is named.
- **A card has NO value, and this is the decision the prompt is testing.** Ace = 1 or 11 in blackjack, high in poker, variable in rummy; a king is 10 in one game and merely beats a queen in another. So `Rank` is an **identity** and the **game** supplies a `HandEvaluator` — and the proof is that a blackjack evaluator returns a *number* while a poker evaluator returns a *category*, so “value” was never one idea.
- **The naive shuffle is biased and the argument is counting, not statistics.** Swapping each position with a random position in `0..n-1` gives **n^n paths for $n!$ orderings**, and n^n is not divisible by $n!$ — for $n = 3$ that is $27/6 = 4.5$, so three orderings land at $4/27$ (14.8%) and three at $5/27$ (18.5%) against a uniform 16.7%. **Fisher-Yates** picks from `0..i` instead: **exactly $n!$ paths, one per ordering**.
- **If money is involved the seed matters more than the algorithm.** $52! \approx 8.07 \times 10^{67}$ (~226 bits), but a **32-bit seed reaches only 4.3×10^9 shuffles** — 5×10^{-59} of them — and Mersenne Twister is reconstructible from **624 outputs**. Use `secrets / SystemRandom`, log the seed, and **inject the RNG** so the deck is both testable and swappable.
- **Blackjack’s only interesting rule: count every ace as 11, then subtract 10 while bust** — correct for `A A A A` ($44 \rightarrow 14$), and a per-card decision is impossible because an ace depends on cards dealt *after* it. Return `soft` too, because the dealer rule is

“stand on hard 17, hit on soft 17” — and that rule lives on the **game**, not the evaluator. The **cut card at ~75%** exists because a counter’s edge is ~1.5% at full penetration and ~0.5% at three quarters.

DSA topic: Merging and sorting linked lists **System design topic:** Design a deck of cards and a card game

Code these, in this order

One rule for the whole set: **write the cut in the same keystroke as the split.** `second = slow.next` and `slow.next = None` are one action, not two, and separating them is how this day's only fatal bug gets in.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Merge Two Sorted Lists	LeetCode 21 (Easy)	The builder dummy, $\leq$ for stability, and attaching the remainder in one assignment.
2	Sort List	LeetCode 148 (Medium)	The first middle, the cut, and whether you can say why merge sort and not quicksort.
3	Merge k Sorted Lists	LeetCode 23 (Hard)	$O(nk)$ versus $O(n \log k)$, the heap tie-breaker, and terminating the output.
4	Insertion Sort List	LeetCode 147 (Medium)	The $O(n^2)$ baseline, so you can say what merge sort bought you and at what n it matters.

On problem 1, delete the last line and see what happens

Replace `tail.next = a if a is not None else b` with a loop that moves the remainder node by node. Confirm it still works. Then say, in one sentence, what the one-line version knows that the loop does not.

On problem 2, break the split twice on purpose

First use the second middle and run `[1, 2]`. Quote the error. Then restore the first middle, remove the cut, and run `[1, 2, 3, 4]`. Quote that error too. They are the same error from two different causes, and you should be able to tell them apart from the code alone.

On problem 3, hit the TypeError

Write the heap version with `(node.value, node)` and no tie-breaker. Feed it two lists whose heads are equal. Quote the exact error. Then fix it and say why the fix works.

On problem 4, get the number

Time insertion sort and merge sort on 10,000 random nodes. Write both numbers down. Then say at what size you would stop caring, and why an interviewer still asks for $O(n \log n)$ at $n = 100$.

The merge drill

1. Say what the merge creates and what it copies.
2. Say what the dummy is for, in the builder sense.
3. Write the remainder line and say why one assignment is enough.
4. Say what $\leq$ buys over $<$, and construct an input where it is visible.
5. State the time and space, and say what the array version would need that this does not.

The split drill

1. Write the first-middle loop and the cut.
2. Run the second-middle version on `[1, 2]` and describe the recursion.
3. Remove the cut and describe what the two “halves” actually are.
4. Say why both mistakes produce the same error message.
5. State the base case with both its conditions and say what each one covers.

The why-merge-sort drill

Answer each in one sentence:

1. What does quicksort's partition need that a linked list does not have?
2. What does merging two arrays need that merging two lists does not?
3. Which sort is the array's worst case for space and the list's best?
4. Is your sort stable, and what one character decides it?
5. What is the recursion's space, and what would make it $O(1)$?

The break-it drill

Trigger each and record the exact output or error text:

1. Split at the second middle. Run `[1, 2]`.
2. Omit the cut. Run `[1, 2, 3, 4]`.
3. Use $<$ instead of $\leq$ and sort a list of records already ordered by a second key. Describe what moved.
4. Loop to attach the remainder instead of one assignment. Confirm it works, then say what it signals.

5. Heap merge with no tie-breaker, two equal heads. Quote the error.
6. Heap merge without `tail.next = None`. Print the result and describe the failure.
7. Base case with only `head is None`. Run a single-node list.

The k-lists drill

1. Compute the cost of merging one at a time, for k lists of n/k nodes.
2. Compute it at k = 10,000 and n = 1,000,000.
3. Compute the $O(n \log k)$ cost for the same inputs and state the ratio.
4. Write the pairwise version and say why it needs no special case for an odd count.
5. Write the heap version and name its two traps.
6. Say which you would submit and why.

The bottom-up drill

1. Describe the passes in one sentence.
2. Say how many passes there are for a million nodes.
3. State its time and space.
4. Write `_cut` and say exactly what it returns.
5. Say honestly what it costs you in code, and when you would write it.

The composition drill

Write `sort` from scratch using only functions you already have, and then name, for each piece, the day it came from:

1. The dummy in the merge.
 2. The runner in the split.
 3. The divide-and-conquer shape.
 4. The stability argument.
-

The value-object drill

1. Say why a card is a value object rather than an entity.
2. Name the three things `frozen=True` gives you, and which day introduced them.
3. Say why `Suit` and `Rank` are enums rather than strings.
4. Say why you would set `order=False`, and what bug that prevents.

The no-value drill

1. Give the value of an ace in three different games.

2. Give the value of a king in two games where the answer differs in kind, not just in number.
3. Say what breaks the day you put `value` on `Card`.
4. Say what the return types of a blackjack evaluator and a poker evaluator are, and what that proves.
5. Name the day this same “who owns the rule” question was asked about something else.

The shuffle drill

1. Write the naive shuffle.
2. Enumerate all 27 paths for three cards by hand or by code, and tabulate the six orderings.
3. State the counting argument for why uniformity is impossible.
4. Write Fisher–Yates and point at the one changed range.
5. Count its paths for three cards and say why that proves uniformity.
6. Say what Python function does this for you.

The randomness drill

1. State $52!$ and roughly how many bits it takes to name one ordering.
2. State Mersenne Twister’s state size and why that is not the problem.
3. Compute the fraction of shuffles reachable from a 32-bit seed.
4. Say what an adversary can do with 624 consecutive outputs.
5. Name the two Python facilities you would use instead, and one more reason to inject the RNG.

The blackjack drill

1. Write the hand evaluator in the count-high-then-downgrade form.
2. Run it on `A A A A` and show the arithmetic.
3. Say why a per-card decision about an ace is impossible in principle.
4. Say what `soft` is and which rule needs it.
5. Say where the dealer’s hit-or-stand rule lives, and why not on the evaluator.

The shoe drill

1. Compute the cards in a six-deck shoe and the number dealt at 75 percent penetration.
2. Compute roughly how many rounds that is with four players.
3. State a counter’s edge at full penetration and at three quarters.

4. Say what a continuous shuffling machine changes, and which class would become an interface.

The extension drill

For each, say which classes change and how many:

1. Build poker instead of blackjack.
2. Add jokers.
3. Use a 32-card piquet deck.
4. Move to a real-money site.
5. Support a continuous shuffling machine.
6. Add an audit trail that can reproduce any past shuffle.

One of the six changes nothing but a function call. Name it, and say why that is the pay-off for the whole design.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Merge two sorted linked lists. Now sort an unsorted one.* What the merge does not do, the builder dummy, $\leq$ for stability, the remainder in one assignment, then split-sort-merge with both fatal split details, the quicksort comparison in two sentences, and the complexity with where the log comes from.
 2. *Design a deck of cards. Now build blackjack on top of it.* Card as a frozen value object, the no-value decision with the ace in three games, the naive shuffle killed by the counting argument with the 27-over-6 numbers, Fisher-Yates and the one changed range, the seeding arithmetic if money is involved, and blackjack changing nothing but the evaluator.
 3. *Why merge sort rather than quicksort, for a linked list?* No random access for the partition, and merging lists needs no scratch space where merging arrays does — so merge sort is the array's worst case for space and the list's best.
-

Before you move on

- [] I write the cut in the same keystroke as the split.
- [] I can say what the merge creates and copies, and the answer is nothing.
- [] I can state what the one-line remainder attachment knows that a loop does not.

- [] I can name the character that makes the sort stable and construct a visible case.
- [] I broke the split both ways and can tell the two causes apart from the code.
- [] I can give the quicksort comparison in two sentences without hedging.
- [] I can say where the `log n` comes from and quote the level count at a million nodes.
- [] I computed $O(nk)$ versus $O(n \log k)$ at $k = 10,000$ and quoted the ratio.
- [] I hit the heap `TypeError` on purpose and know the fix.
- [] I can describe bottom-up merge sort and say what it costs in code.
- [] I can say why a card is a value object and what `frozen=True` gives me.
- [] I can give the ace's value in three games and say what that proves.
- [] I enumerated the 27 paths and can quote 14.81 and 18.52 percent.
- [] I can state the counting argument for the bias in one sentence.
- [] I can compute the fraction of shuffles reachable from a 32-bit seed.
- [] I can score `A A A A` and explain why a per-card ace decision is impossible.
- [] I answered all three questions above out loud.

Doubly and circular linked lists

DATA STRUCTURES AND ALGORITHMS

Doubly and circular linked lists

You can splice a node out in $O(1)$ and say what the extra pointer costs.

SYSTEM DESIGN

Design Splitwise

You can model groups, expenses, splits and settlement, with the simplification algorithm.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Why does an LRU cache use a doubly linked list?*
- *Design Splitwise. How do you minimise the number of transactions?*

1 What this is, and why they ask it

A **doubly linked list** gives every node a second reference: `prev`, pointing back at the node before it. That one extra field changes exactly one thing, and it is the thing that matters: **given a node, you can remove it in $O(1)$** , because you can reach both of its neighbours and join them to each other.

In a singly linked list you cannot. From [day 079](#), removing a node requires the node *before* it, and finding that means walking from the head — $O(n)$. The extra pointer buys you the predecessor for free.

A **circular** list is a different, independent change: the last node points at the first instead of at `None`. There is no end, so “keep going round” is the natural traversal, and there is no first or last node to be a special case. The two ideas combine — a circular doubly linked list with one sentinel is the cleanest list structure there is, and it is what Python’s `collections.deque` and the Linux kernel’s `list_head` both are.

They ask it because you built an LRU cache on [day 076](#) and used one without being able to justify it in one sentence yet. The interviewer wants that sentence, plus an honest account of what the second pointer costs — eight bytes a node, twice the pointer updates, and twice as many chances to leave the list inconsistent.

2 The story

The courtyard game starts at about half past six, once it is cool enough, and it needs at least ten children to be worth playing.

They stand in a ring holding hands and one of them, usually Divya because she is loudest, walks round the outside counting. Every seventh child is out. The child who is out lets go, steps back, and sits on the low wall to watch, and the two children either side of the gap take each other’s hands. The ring closes up. Divya carries on counting from wherever she had got to.

That last part is what makes the game work. Nobody stops. Nobody goes back to the beginning. There is no beginning to go back to — it is a ring, so wherever Divya is standing is as good as anywhere else, and the counting simply continues.

It takes about four seconds for the gap to close and it does not matter at all whether there are nineteen children left or four.

Last year the younger ones tried to play it in a straight line instead, because there were only six of them and a ring of six looked silly.

It went badly in a way that took them a while to understand. When a child in the middle was out and stepped away, the child behind had nobody in front of her — she could not simply take the next hand, because she did not know whose it was. She was holding the shoulder of somebody who had gone. The only way to sort it out was to go to the front of the line and count along until they reached the gap, and by then the counting had been lost and three of them were arguing about whose turn it was.

Divya watched this for one evening and told them to stand in a ring like everybody else.

Her small brother asked her afterwards why the line was harder, since it was the same children.

She said: in the ring you are holding two hands. You know who is on both sides of you, so when you go, the two of them just take each other. In the line each of you was only holding the one in front. Nobody knew who was behind them.

3 The idea in plain English

The children joining hands are the two pointer updates. The ring having no front is the circular list. And the fact that the game never restarts is why round-robin problems want one.

The node, with one more field

```
class Node:
    __slots__ = ("value", "prev", "next")

    def __init__(self, value: int) → None:
        self.value = value
        self.prev: "Node | None" = None
        self.next: "Node | None" = None
```

One extra reference, eight more bytes. Everything below follows from it.

Removal in $O(1)$, which is the whole point

```
node.prev.next = node.next
node.next.prev = node.prev
```

Two assignments. The child stepping back, and the two either side taking hands. **No walk, no length check, no dependence on where the node is** — the same two lines whether the list holds three nodes or three million.

Compare with the singly linked version, which needs the predecessor and therefore a walk:

```
singly linked, given the node:  O(n)  - walk from the head to find `previous`
doubly linked, given the node:  O(1)  - two assignments
```

That is the LRU cache's entire reason. The hash map hands you the node for a key in $O(1)$; if removing it were $O(\text{capacity})$ you would have gained nothing. The map answers *where*, the doubly linked list answers *cheaply*, and neither works without the other.

Insertion, and the order of the four assignments

```
def insert_after(node: Node, fresh: Node) → None:
    fresh.prev = node
    fresh.next = node.next
    node.next.prev = fresh          # fix the neighbour BEFORE overwriting node.next
    node.next = fresh
```

Four assignments, and the order is not free. Set `node.next = fresh` first and you have lost your reference to the old successor, so line three has nothing to fix. **Point the new node at both neighbours first, then rewire the neighbours** — the same rule as the singly linked insertion on [day 078](#), now with twice as many links to get wrong.

The reliable habit: write the two assignments *into* the new node, then the two assignments *out of* the neighbours.

Sentinels, which stop being optional

With one direction, a missing `prev` was not a problem. With two, every operation touches both neighbours, so an operation at either end has a `None` where a node should be — and you get the phase's favourite error:

```
AttributeError: 'NoneType' object has no attribute 'prev'
```

The fix is the sentinel from [day 080](#), used at **both** ends:

```
head ↔ A ↔ B ↔ C ↔ tail
(fake)                               (fake)
```

Now every real node has a real `prev` and a real `next`, `_unlink` and `_push_front` have no branches, and inserting into an empty list is the same code as inserting into a full one. On [day 076](#) this was presented as a convenience. In a doubly linked list it is closer to

a requirement — the number of `None` checks it removes roughly doubles.

Circular lists: no end, and no first

Make the last node point at the first and the first point back at the last, and the structure has no boundary at all. Two consequences.

Traversal must stop by counting or by comparison, never by `None`.

```
node = start
while True:
    visit(node)
    node = node.next
    if node is start:           # the only way to know you have been round
        break
```

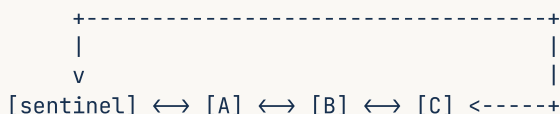
`while node is not None` never terminates on a circular list. Every loop needs either a “have I come back to where I started” test or a known count, and forgetting that is the standard way to hang a program with no error message.

There is no head to be special. Round-robin scheduling, a music playlist on repeat, the players in a card game, the turn order in a board game — all of them are “the next one, for ever”, and a circular list expresses that without a wrap-around branch.

The classic problem is the **Josephus problem**: `n` people in a ring, every `k`-th is removed until one remains. It is the counting-out game, and with a circular doubly linked list it is a direct simulation — walk `k - 1`, unlink in $O(1)$, continue from where you are, no restart. That “continue from where you are” is the property an array simulation has to fake with modular arithmetic.

The combination, which is what real code uses

A **circular doubly linked list with one sentinel** is the standard industrial structure:



The sentinel is both the head and the tail. `sentinel.next` is the first real node, `sentinel.prev` is the last, an empty list is `sentinel.next is sentinel`, and **there is not a single `None` in the structure**. Every insertion and removal is the same four or two assignments with no branches at all.

This is what the Linux kernel’s `list_head` is, what Python’s `collections.deque` is internally (a circular doubly linked list of fixed-size blocks), and what a well-written LRU cache uses.

What the second pointer costs, honestly

memory:	+8 bytes per node (about +17% on a 48-byte node)
writes:	2 per removal instead of 1; 4 per insertion instead of 2
bugs:	every operation must maintain BOTH directions, and a list that is correct forwards and broken backwards looks completely fine until something walks the other way

That last one is the real cost. A singly linked list is either right or obviously wrong. A doubly linked list can be **half right**, and the symptom appears far from the cause. Write an invariant check — walk forward collecting nodes, walk backward collecting nodes, assert the reverse of one equals the other — and run it after every operation in tests.

When you do *not* want one

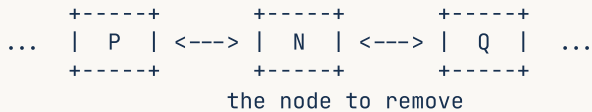
- **You only ever walk forwards.** Then `prev` is 17% more memory and twice the writes for nothing.
- **You only ever insert and remove at the front.** A singly linked list is a perfect stack.
- **Memory is tight and n is large.** Ten million nodes is 80 MB of `prev` pointers alone.

The recognition rule: **use a doubly linked list when something else hands you a node and you must remove it in $O(1)$.** That is the LRU cache, a scheduler removing a task it holds a handle to, and an editor's undo list. If you are always searching for the node first, the search is $O(n)$ anyway and the second pointer buys nothing.

4 The picture

Removal, drawn at the pointer level — this is the diagram to reproduce in an interview.

before:

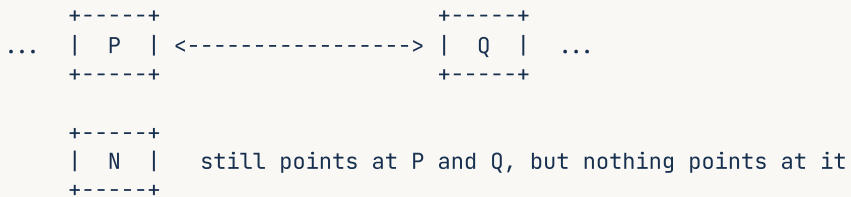


```

node.prev.next = node.next    → P.next = Q
node.next.prev = node.prev    → Q.prev = P

```

after:



Two assignments. No loop. No length. Divya's two children taking hands.

Insertion, with the ordering that matters:

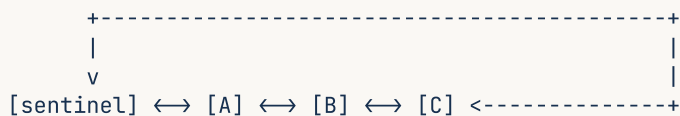
insert F between P and Q

RIGHT: 1. F.prev = P (F now knows both neighbours)
 2. F.next = Q
 3. P.next.prev = F → Q.prev = F
 4. P.next = F

WRONG: 1. P.next = F (Q is now unreachable from P)
 2. F.next = P.next → F.next = F – points at itself
 3. ... the rest of the list is gone

Rule: write INTO the new node first, then rewire the neighbours.

The circular list with one sentinel, which is the shape to remember:



```

sentinel.next → the first real node
sentinel.prev → the last real node
empty list   → sentinel.next is sentinel
no None anywhere, so no branch anywhere

```

And the traversal difference, because it is where circular lists bite:

```

linear:      while node is not None:      ... terminates at the end
circular:    while node is not None:      ... NEVER terminates

circular, correct:
    start = node
    while True:
        visit(node)
        node = node.next
        if node is start: break

```

5 The code, built step by step

Step 1 — the two sentinels

```

self._head = Node()           # fake
self._tail = Node()          # fake
self._head.next = self._tail
self._tail.prev = self._head

```

Four lines that delete every `None` check in the class. Write them before anything else, exactly as on [day 076](#).

Step 2 — unlink, in two assignments

```

def _unlink(self, node: Node) → None:
    node.prev.next = node.next
    node.next.prev = node.prev
    node.prev = node.next = None      # do not leave a dead node pointing in

```

The third line is optional for correctness and worth writing: a removed node that still points into the list lets a caller who kept a reference walk back into a structure it is no longer part of, which is a genuinely confusing bug.

Step 3 — insert, into the new node first

```

def _insert_between(self, fresh: Node, left: Node, right: Node) → None:
    fresh.prev, fresh.next = left, right      # into the new node
    left.next = fresh                          # then the neighbours
    right.prev = fresh

```

Taking `left` and `right` as parameters rather than computing them inside removes the ordering trap entirely: both neighbours are captured before anything is overwritten. Every public insertion — front, back, before, after — becomes a one-line call to this.

Step 4 — the invariant check, which you should actually write

```
def _check(self) → None:
    forwards, node = [], self._head.next
    while node is not self._tail:
        forwards.append(node.value)
        node = node.next
    backwards, node = [], self._tail.prev
    while node is not self._head:
        backwards.append(node.value)
        node = node.prev
    assert forwards == backwards[::-1], f"{forwards} vs {backwards[::-1]}"
    assert len(forwards) == self._size
```

Walk both ways and compare. This is the check that catches a list which is correct forwards and broken backwards — the failure mode a singly linked list cannot have and this one can. Three lines of assertion, and running it after every operation in a random test finds every pointer bug in minutes.

Step 5 — the circular traversal

```
node = start
while True:
    yield node
    node = node.next
    if node is start:
        break
```

Never `while node is not None`. On a circular list there is no `None` and that loop hangs with no error at all.

The complete solution

```
class Node:
    __slots__ = ("value", "prev", "next")

    def __init__(self, value: int = 0) → None:
        self.value = value
        self.prev: "Node | None" = None
        self.next: "Node | None" = None

    def __repr__(self) → str:
        return f"Node({self.value})"

class DoublyLinkedList:
    """A doubly linked list with sentinels at both ends.

    The one thing the extra pointer buys: given a node, remove it in O(1),
    because both neighbours are reachable from it. That is why an LRU cache
    uses one – the hash map answers WHERE, the list answers CHEAPLY.

    Two sentinels mean there is no None anywhere among the real nodes, so
    every operation is the same handful of assignments with no branches.
    """

    def __init__(self, values: list[int] | None = None) → None:
        self._head = Node()          # fake front
        self._tail = Node()          # fake back
        self._head.next = self._tail
        self._tail.prev = self._head
        self._size = 0
        for value in values or []:
            self.push_back(value)

    # ---- the two primitives -----

    def _insert_between(self, fresh: Node, left: Node, right: Node) → Node:
        """Both neighbours are parameters, so nothing can be overwritten early."""
        fresh.prev, fresh.next = left, right
        left.next = fresh
        right.prev = fresh
        self._size += 1
        return fresh

    def _unlink(self, node: Node) → int:
        """The whole reason for the second pointer. Two assignments, O(1)."""
        node.prev.next = node.next
        node.next.prev = node.prev
        node.prev = node.next = None          # no dangling links out of a dead node
        self._size -= 1
        return node.value

    # ---- the public operations -----

    def push_front(self, value: int) → Node:
        return self._insert_between(Node(value), self._head, self._head.next)

    def push_back(self, value: int) → Node:
        return self._insert_between(Node(value), self._tail.prev, self._tail)

    def insert_after(self, node: Node, value: int) → Node:
        return self._insert_between(Node(value), node, node.next)

    def insert_before(self, node: Node, value: int) → Node:
        return self._insert_between(Node(value), node.prev, node)

    def remove(self, node: Node) → int:
        """O(1) – this is the operation a singly linked list cannot do."""
```

```

        return self._unlink(node)

def pop_front(self) → int:
    if not self._size:
        raise IndexError("pop from an empty list")
    return self._unlink(self._head.next)

def pop_back(self) → int:
    if not self._size:
        raise IndexError("pop from an empty list")
    return self._unlink(self._tail.prev)    # 0(1), unlike a singly linked list

def move_to_front(self, node: Node) → None:
    """Unlink and re-insert. The LRU cache's `get`, in two calls."""
    node.prev.next = node.next
    node.next.prev = node.prev
    node.prev, node.next = self._head, self._head.next
    self._head.next.prev = node
    self._head.next = node

def __len__(self) → int:
    return self._size

def __iter__(self):
    node = self._head.next
    while node is not self._tail:
        yield node.value
        node = node.next

def reversed_values(self) → list[int]:
    """Proof that the backward links are maintained – walk the other way."""
    out, node = [], self._tail.prev
    while node is not self._head:
        out.append(node.value)
        node = node.prev
    return out

def __repr__(self) → str:
    return f"DoublyLinkedList({list(self)})"

def _check(self) → None:
    """Walk both ways and compare. Catches a list that is correct forwards
    and broken backwards – the failure a singly linked list cannot have."""
    forwards = list(self)
    backwards = self.reversed_values()
    assert forwards == backwards[::-1], f"{forwards} vs {list(reversed(backwards))}"
    assert len(forwards) == self._size, f"size {self._size}, chain {len(forwards)}"

class CircularList:
    """A circular doubly linked list with ONE sentinel, which is both head and
    tail. There is no None anywhere, so there are no branches anywhere.

    This is what the Linux kernel's list_head is, and what collections.deque
    is internally (over blocks rather than single elements).
    """

    def __init__(self, values: list[int] | None = None) → None:
        self._sentinel = Node()
        self._sentinel.next = self._sentinel.prev = self._sentinel    # points at itself
        self._size = 0
        for value in values or []:
            self.push_back(value)

    def push_back(self, value: int) → Node:
        fresh = Node(value)
        last = self._sentinel.prev
        fresh.prev, fresh.next = last, self._sentinel
        last.next = fresh

```

```

        self._sentinel.prev = fresh
        self._size += 1
        return fresh

def remove(self, node: Node) → int:
    node.prev.next = node.next
    node.next.prev = node.prev
    self._size -= 1
    return node.value

def is_empty(self) → bool:
    return self._sentinel.next is self._sentinel    # the emptiness test

def __len__(self) → int:
    return self._size

def __iter__(self):
    node = self._sentinel.next
    while node is not self._sentinel:
        yield node.value
        node = node.next

def walk_from(self, start: Node, steps: int) → Node:
    """Move `steps` forward, skipping the sentinel so it is invisible."""
    node = start
    for _ in range(steps):
        node = node.next
        if node is self._sentinel:
            node = node.next
    return node

def josephus(n: int, k: int) → int:
    """n people in a ring, every k-th is eliminated. Return the survivor (1-based).

    A direct simulation: the ring closes in O(1) after each removal and the
    counting continues from where it stopped – no restart, no wrap-around
    arithmetic. Divya's game.

    O(n*k) time, O(n) space.
    """
    if n < 1 or k < 1:
        raise ValueError("n and k must be at least 1")

    ring = CircularList(list(range(1, n + 1)))
    current = ring._sentinel.next
    while len(ring) > 1:
        current = ring.walk_from(current, k - 1)    # step to the k-th
        following = current.next
        if following is ring._sentinel:
            following = following.next
        ring.remove(current)
        current = following                        # continue from where we are
    return next(iter(ring))

if __name__ == "__main__":
    lst = DoublyLinkedList([1, 2, 3, 4])
    print(lst, lst.reversed_values())            # [1,2,3,4]  [4,3,2,1]
    lst._check()

    node = lst.push_back(5)
    lst.push_front(0)
    print(lst, len(lst))                        # [0,1,2,3,4,5] 6
    lst._check()

    print(lst.remove(node))                      # 5 - O(1), given the node
    print(lst.pop_front(), lst.pop_back())       # 0 4
    print(lst)                                   # [1, 2, 3]

```

```

lst._check()

middle = lst._head.next.next          # the node holding 2
lst.move_to_front(middle)
print(lst, lst.reversed_values())      # [2,1,3] [3,1,2]
lst._check()

empty = DoublyLinkedList()
print(len(empty), list(empty))        # 0 []
try:
    empty.pop_front()
except IndexError as error:
    print(f"IndexError: {error}")      # pop from an empty list

ring = CircularList([1, 2, 3])
print(list(ring), len(ring), ring.is_empty()) # [1,2,3] 3 False

print(josephus(7, 3))                  # 4
print(josephus(5, 2))                  # 3
print(josephus(1, 5))                  # 1
print(josephus(10, 1))                 # 10

# the invariant check earns itself on random sequences
import random
for _ in range(2000):
    test = DoublyLinkedList()
    handles: list[Node] = []
    for _ in range(30):
        action = random.random()
        if action < 0.4:
            handles.append(test.push_back(random.randint(0, 99)))
        elif action < 0.7:
            handles.append(test.push_front(random.randint(0, 99)))
        elif handles:
            victim = handles.pop(random.randrange(len(handles)))
            test.remove(victim)
    test._check()
print("forwards and backwards agreed on 2000 random sequences")

```

6 What it costs

The operations, against a singly linked list

	singly	doubly	
remove, GIVEN the node	$O(n)$	$O(1)$	← the whole point
remove at the front	$O(1)$	$O(1)$	
remove at the back	$O(n)$	$O(1)$	← with a tail reference
insert before a given node	$O(n)$	$O(1)$	
insert after a given node	$O(1)$	$O(1)$	
walk forwards	$O(n)$	$O(n)$	
walk backwards	impossible	$O(n)$	

Three rows changed from $O(n)$ to $O(1)$, and every one of them is “you already have the node”. That is the pattern: **the second pointer converts *having* a node into *being able to act on it*.**

The price

```
memory per node:
  singly, __slots__: 48 B    (value, next)
  doubly, __slots__: 56 B    (value, prev, next)
  → +8 B, about +17%

at 10,000,000 nodes: +80 MB of prev pointers alone

writes:
  removal: 1 assignment → 2
  insertion: 2 assignments → 4
```

Seventeen percent more memory and twice the pointer writes. Neither is a reason to avoid it when you need $O(1)$ removal, and both are reasons not to use one when you do not.

The cost that is not in the table

A doubly linked list can be half correct. Forward links right, backward links wrong. Iterating forwards gives the right answer, `len` gives the right answer, every test passes — and then something walks backwards, or removes a node, and the structure falls apart far from where the bug was introduced.

A singly linked list has no such state: it is right, or it is visibly broken. That is the real cost of the second pointer, and the mitigation is the `_check` above — walk both ways, compare, assert. Say this in an interview; it is the kind of thing that comes from having debugged one.

The LRU cache, priced

From [day 076](#), with capacity `c`:

```
with a doubly linked list: get = map lookup + 2 writes + 4 writes →  $O(1)$ 
with a singly linked list: get = map lookup +  $O(c)$  walk to find prev
  c = 100,000: ~100,000 steps per get instead of about 6 writes
```

Roughly ten thousand times more work per read, and it would make the cache slower than the database it was put in front of. That is the one-sentence answer to “why doubly linked”.

Circular lists

```
memory:          identical to a linear doubly linked list
traversal:        $O(n)$  per lap, and there is no natural stopping point
Josephus:         $O(n*k)$  by direct simulation,  $O(n)$  space
                  (the closed-form recurrence is  $O(n)$  time and  $O(1)$  space, and is
                   a different solution rather than a better implementation)
```

The circular structure buys **no asymptotic improvement anywhere**. What it buys is the absence of special cases: no first, no last, no wrap-around branch, no `None`.

7 The traps

Trap 1 — updating only one direction

```
node.prev.next = node.next          # forgot: node.next.prev = node.prev
```

The list is now correct forwards and broken backwards. Iteration works. `len` works. Printing works. And then a later `pop_back` or a backward walk finds a `prev` pointing at a node that was removed, and the removed node comes back from the dead. **No error, and the symptom is nowhere near the cause.**

This is the defining bug of doubly linked lists and it is why `_check` exists.

Trap 2 — insertion in the wrong order

```
left.next = fresh                    # WRONG FIRST
fresh.next = left.next               # this is now `fresh` — points at itself
```

A node pointing at itself, and the rest of the list unreachable. The next traversal either hangs or returns one element. **Write into the new node first, then rewire the neighbours** — or take both neighbours as parameters, which makes it impossible.

Trap 3 — `None` at the ends, without sentinels

```
node.prev.next = node.next          # node.prev is None for the first node
```

```
AttributeError: 'NoneType' object has no attribute 'next'
```

With two directions there are twice as many ends and twice as many branches to forget. Two sentinels remove all of them, and in a doubly linked list that is closer to a requirement than a convenience.

Trap 4 — `while node is not None` on a circular list

```
while node is not None:              # there is no None. This never ends.
    visit(node)
    node = node.next
```

No exception, no output, no end — the program simply stops responding, and if `visit` appends to a list, it eventually dies of memory exhaustion instead. Circular traversal must compare against the starting node or count.

Trap 5 — the sentinel appearing in the output

```
for value in ring:                # yields the sentinel's value too
```

The sentinel is not data. Every traversal must skip it, and every `walk_from` must step over it, or your Josephus simulation eliminates a child who does not exist. Wrapping the traversal in `__iter__` once — so no caller ever walks manually — is the fix.

Trap 6 — forgetting that `remove` needs the node, not the value

```
lst.remove(5)                    # 5 is a value, not a node
```

The $O(1)$ guarantee is *given the node*. If the caller has a value, finding the node is an $O(n)$ search and the whole advantage is gone. This is why the LRU cache keeps a map from key to node, and why the API here takes a `Node` — making the caller hold handles is a deliberate design decision, not an oversight.

Trap 7 — a removed node still pointing into the list

```
node.prev.next = node.next
node.next.prev = node.prev
# node.prev and node.next still set
```

A caller who kept a reference can now walk from a removed node back into the live list, and in an LRU cache a stale node can be moved to the front, silently resurrecting an evicted entry. One line — `node.prev = node.next = None` — closes it.

Trap 8 — using one when you only ever walk forwards

Seventeen percent more memory, twice the writes, and a whole class of half-correct states, in exchange for nothing. If you never remove a node you already hold, and never walk backwards, a singly linked list is strictly better. Say the recognition rule rather than reaching for the fancier structure.

8 In the interview

How it gets asked

- The direct version: “Why does an LRU cache use a doubly linked list?” — a one-sentence answer with a number attached.

- The comparison: “What does the extra pointer cost you?”
- The implementation: “Implement a doubly linked list with insert, delete and reverse traversal.”
- The circular version: “ n people stand in a circle and every k -th is eliminated. Who survives?” — the Josephus problem.
- The real-world probe: “Where have you seen one of these?” — `collections.deque`, the Linux kernel, a browser’s history, a music playlist.

What to say out loud, in the first ninety seconds

1. **Name the one thing it buys.** “The second pointer buys exactly one operation: given a node, remove it in $O(1)$, because I can reach both neighbours and join them to each other.”
2. **Say what it replaces.** “In a singly linked list I would need the node before it, which means walking from the head — $O(n)$.”
3. **Give the LRU sentence with the number.** “That is why the LRU cache uses one. The map gives me the node in $O(1)$; with a singly linked list, removing it would be $O(\text{capacity})$, so at a hundred thousand entries a read would be a hundred thousand steps instead of about six writes.”
4. **Say the sentinels are not optional here.** “I use fake head and tail nodes, because with two directions there are twice as many ends and twice as many `None` checks to forget.”
5. **State the cost honestly.** “Eight bytes a node, about seventeen percent, and twice the pointer writes. And the real cost is that the list can be half correct — right forwards and broken backwards — which a singly linked list cannot be. I would write an invariant check that walks both ways and compares.”

The follow-ups

“**Why does an LRU cache need a doubly linked list?**” “Because the hash map hands me the node directly, and unlinking it must be $O(1)$ or the map’s $O(1)$ lookup is wasted. Unlinking needs both neighbours joined to each other, and reaching the predecessor from a node is exactly what the `prev` pointer gives me. With a singly linked list I would have to walk from the head to find the predecessor — $O(\text{capacity})$ — so at a hundred thousand entries every read becomes a hundred-thousand-step walk. The map answers *where*; the doubly linked list answers *cheaply*.”

“**What does the extra pointer cost?**” “Eight bytes per node, which is about seventeen percent on a small node, and eighty megabytes at ten million nodes. Twice the pointer writes: two per removal instead of one, four per insertion instead of two. And the cost that does not show up in a table — the list can be correct in one direction and broken in

the other, so it passes every forward test and falls apart later, somewhere else. A singly linked list cannot be half right. That is why I would write a check that walks both ways and asserts the reverse of one equals the other, and run it after every operation in tests.”

“What is a circular list good for?” “Anything that is ‘the next one, for ever’ with no natural end — round-robin scheduling, a playlist on repeat, turn order in a game, a ring buffer. The structural benefit is the absence of special cases: no first node, no last node, no wrap-around branch, and with a single sentinel there is no `None` anywhere, so insertion and removal have no branches at all. That is exactly what the Linux kernel’s `list_head` is. The trap is that `while node is not None` never terminates, so every traversal has to compare against the starting node or count.”

“n people in a circle, every k-th eliminated. Who survives?” “Direct simulation with a circular list: walk `k-1`, unlink in $O(1)$, and continue from where I am — no restart and no modular arithmetic, because the ring closes itself. That is $O(n \cdot k)$ time and $O(n)$ space. There is also a closed-form recurrence — the survivor for `n` people is $(\text{survivor}(n-1) + k) \bmod n$, with one person surviving trivially — which is $O(n)$ time and $O(1)$ space, and I would mention it as the better answer if `k` is large. The list version is the one that shows you understand the structure; the recurrence is the one that is fast.”

“Where would you not use one?” “When I only ever walk forwards and never remove a node I already hold. Then `prev` is pure cost — more memory, more writes, and a class of bugs I did not have. The recognition rule I use is: a doubly linked list is for when something *else* hands me the node and I must act on it in constant time. If I have to search for the node first, the search is $O(n)$ anyway and the second pointer buys nothing.”

“Have you used one without knowing?” “`collections.deque` is a circular doubly linked list of fixed-size blocks — that is how it gets $O(1)$ at both ends without shifting. `OrderedDict` is a dict plus a doubly linked list, which is why `move_to_end` is $O(1)$ and why it is an LRU cache. The Linux kernel’s `list_head` is a circular doubly linked list with a sentinel, embedded in the structs themselves. And a browser’s back and forward history is the obvious user-facing one.”

A model answer

Asked: *why does an LRU cache use a doubly linked list?*

“Because of exactly one operation, and it is the one the cache does on every single read.

An LRU cache needs two things at once: find the entry for a key in constant time, and know which entry has been untouched the longest. The hash map does the first. A list in recency order does the second. And the moment a key is read, that entry has to move to the front — which means unlinking it from wherever it currently is.

Unlinking a node means joining its predecessor to its successor. Two assignments, if I can reach both. In a singly linked list I cannot reach the predecessor from the node — I would have to walk from the head until I found the node whose `next` is my node, and that is $O(\text{capacity})$. So the map’s constant-time lookup would be immediately thrown away by a linear walk. At a hundred thousand entries, every read would be about a hundred thousand steps instead of half a dozen pointer writes — roughly ten thousand times more work, which would make the cache slower than the thing it was put in front of.

The `prev` pointer is what makes it two assignments. The general statement is: **the second pointer converts *having a node into being able to act on it in constant time***. That is also why the map maps to nodes rather than to values, and why the node stores its own key — when I evict the node before the tail, I need its key to delete the map entry.

I would use sentinels at both ends — fake head and tail nodes that always exist. In a singly linked list that is a convenience; here it is close to a requirement, because with two directions there are twice as many ends and every operation touches both neighbours, so without them half the code is `None` checks.

The cost, honestly: eight bytes a node, about seventeen percent, and twice the pointer writes — two per removal, four per insertion. And the cost people do not mention: a doubly linked list can be *half* correct. If I update the forward links and forget the backward ones, iteration works, `len` works, every test passes, and then a later removal walks backwards into a node that was deleted. A singly linked list cannot be in that state. So I would write an invariant check that walks forwards and backwards and asserts they are reverses of each other, and run it after every operation in a randomised test — that catches every pointer bug in this structure in about a minute.

One more thing worth saying: I would not reach for a doubly linked list by default. The rule is that it is for when something else hands me the node. If I have to search for the node first, the search is $O(n)$ anyway and `prev` has bought me nothing but memory and bugs.”

9 Recall card

- **The second pointer buys exactly one thing: given a node, remove it in $O(1)$** — `node.prev.next = node.next; node.next.prev = node.prev`. Two assignments, no walk, no length. In a singly linked list the same operation is $O(n)$, because reaching the predecessor means walking from the head. *The second pointer converts having a node into being able to act on it.*
- **That is the LRU cache's whole reason, with a number:** the map hands you the node in $O(1)$, so unlinking must be $O(1)$ too — at capacity 100,000 a singly linked list would make every **read** a 100,000-step walk instead of ~ 6 pointer writes.
- **Write into the new node first, then rewire the neighbours** — or take both neighbours as parameters, which makes the ordering bug impossible. **Use sentinels at BOTH ends**; with two directions there are twice as many `None` checks to forget, so they stop being a convenience.
- **The cost that is not memory: a doubly linked list can be HALF correct** — right forwards, broken backwards — and it passes every forward test before failing somewhere else entirely. A singly linked list cannot be in that state. **Write `_check`: walk both ways, assert one is the reverse of the other.** Price: +8 B per node ($\sim 17\%$), 2 writes per removal instead of 1, 4 per insertion instead of 2.
- **Circular = the last node points at the first: no head, no tail, no wrap-around branch, and with one sentinel no `None` anywhere.** `while node is not None` **never terminates** — compare against the starting node or count. Good for round-robin, playlists, turn order, ring buffers, and **Josephus** (simulate in $O(n \cdot k)$, or use `survivor(n) = (survivor(n-1) + k) mod n` in $O(n)$ time and $O(1)$ space). Real ones: `collections.deque`, `OrderedDict`, the Linux kernel's `list_head`.

1 What this is, and why they ask it

Splitwise records who paid for what and works out who owes whom. Six friends go on a trip; one pays for the hotel, another for petrol, a third for dinner twice. At the end, everybody wants one number: what do I owe, and to whom.

There are three real problems in that, and the prompt names the third.

Splitting an expense is not one rule but four — equally, by exact amounts, by percentage, by shares — and the totals have to come out exact in a currency that does not divide by three.

Tracking balances means deciding whether “what Ravi owes Priya” is a number you store or a number you compute, and both answers have consequences.

Minimising transactions is the interesting part. Six friends can owe each other across fifteen different pairs, and the answer is that at most five payments settle everything. Getting from fifteen to five is one clean idea, and knowing that the *true* minimum is NP-hard — so what you are writing is a good heuristic and not an optimal algorithm — is what separates a strong answer from a confident one.

They ask it because everybody has been in the group chat where this goes wrong, because the money arithmetic has a real correctness trap in it, and because the simplification question has a satisfying answer that most candidates get most of the way to and then overclaim about.

2 The story

Six of them went to Gokarna for four days, and the trouble started on the bus home.

Sandeep had paid for the rooms, because he booked them, and that was eleven thousand four hundred for two rooms for three nights. Meera paid for the petrol, both ways, which came to about three thousand two hundred with the tolls. Ravi paid for dinner on the first night and again on the third. Anjali bought all the water and the snacks and a beach umbrella that nobody used. The other two had not paid for anything and were slightly embarrassed about it.

Nobody had been keeping track, so the conversation on the bus was six people scrolling through their phones reading out numbers, and every number changed the answer for everybody else.

Then it got worse, because they started settling in pairs. Ravi worked out that he owed Sandeep one thousand nine hundred. Sandeep pointed out that he owed Ravi eight hundred and fifty for the dinners. So they sat there for a while trying to send each other money in both directions before Meera said, quite sharply, that Ravi should send eleven hundred and fifty and they should stop.

That was the first thing they learned. Two people owing each other is one payment, not two.

The second thing took Anjali until the next morning. She wrote out every pair — Ravi to Sandeep, Ravi to Meera, Ravi to Anjali, Sandeep to Meera, and so on — and there were fifteen pairs, and about nine of them had money moving. Nine payments to settle one weekend.

What she did instead was work out one number per person. Not what each pair owed each other — what each *person* was up or down overall. Sandeep was up by a lot. Meera was up a bit. Ravi was roughly even. The other three were down.

Then it was easy, and slightly obvious once she saw it. The three who were down pay the two who are up, and you only need five payments, because each payment either finishes off somebody who is down or finishes off somebody who is up, and there are only six people.

The one complaint came from Vinod, who was down four hundred and was told to send it to Meera. He said, reasonably enough, that he had never taken anything from Meera in his life — he had eaten Ravi's dinners. Anjali said it did not matter and the money comes out the same. Vinod said it mattered to him.

3 The idea in plain English

Anjali's move — **stop tracking pairs and track one number per person** — is the whole algorithm. And Vinod's complaint is the trade-off, which is real and which Splitwise takes seriously enough to make simplification optional.

Money is integers, and the remainder has to go somewhere

Before anything else. **Never use floating point for money.** Store paise as an integer; $0.1 + 0.2$ is not 0.3 in binary floating point, and a system that adds a hundred thousand of those is wrong in a way nobody can trace.

Then the problem that integers do not solve for you:

```
₹1,000 split equally between 3 people
= 100,000 paise / 3 = 33,333.33...
```

```
33,333 × 3 = 99,999. One paise is unaccounted for.
```

You cannot leave it. If the shares do not sum to the total, the group's balances do not sum to zero, and the error compounds with every expense. So the rule is: **compute the shares, then assign the remainder deterministically.**

```
base, remainder = divmod(total_paise, len(participants))
shares = {person: base for person in participants}
for person in sorted(participants)[:remainder]:    # deterministic, not arbitrary
    shares[person] += 1
```

Sorted, so it is reproducible: the same expense recalculated gives the same shares, which matters when an expense is edited and the balances are recomputed. Giving the remainder to the payer is also fine. Giving it to “whoever comes first in a set” is not, because sets have no order and the answer changes between runs.

Assert the invariant. `sum(shares.values()) == total_paise`, every time. It is one line and it makes an entire class of money bugs impossible.

Four ways to split, which is a genuine interface

```
class SplitStrategy(Protocol):
    def shares(self, total_paise: int, participants: list[UserId], config: dict) → dict[UserId, int]:
        ...
```

- **Equal** — divide by the count, distribute the remainder.
- **Exact** — the caller supplies an amount per person; validate that they sum to the total.
- **Percentage** — the caller supplies percentages; validate they sum to 100, then apply the same remainder rule.
- **Shares** — “Ravi eats twice as much”: weights of 2, 1, 1 mean sixths.

Four real implementations, none hypothetical, so the interface passes the gate from [day 076](#) comfortably. And every one of them ends with the same assertion that the parts sum to the whole.

Balances: derive them, and then cache the derivation

An expense produces a set of **ledger entries**: the payer is owed, each participant owes their share.

```
Sandeep pays ₹11,400 for a room shared equally by 6:
Sandeep:  +11,400 - 1,900 = +9,500
each of the other five:    -1,900
```

Now, does “Ravi owes Sandeep ₹1,900” get stored, or computed?

Store the expenses; derive the balances. The expense list is the truth — it is what actually happened, it is what the user can see and edit, and it is auditable. A balance is a *derived* quantity, and deriving it means summing every ledger entry for that user, which is $O(\text{expenses})$.

That is too slow to do on every screen, so you keep a **materialised balance** per (group, user), updated in the same database transaction as the expense insert, and you run a periodic job that recomputes balances from the ledger and alerts on any mismatch.

That combination is the honest answer: **the ledger is the source of truth, the balance is a cache, and there is a reconciliation job because caches drift.** A design that stores only balances cannot answer “why do I owe this?”, and one that only derives cannot render a screen.

Now the actual question: minimising transactions

Step one: net out each person. For each user in the group, add up everything they are owed and subtract everything they owe. One number each.

```
Sandeep +9,500
Meera   +1,800
Ravi     +50
Anjali  -2,300
Vinod   -4,600
Kiran   -4,450
-----
sum      0      ← always, and a useful assertion
```

The sum of all net balances is **always zero**, because every rupee that one person is owed is a rupee somebody else owes. Assert it.

This step alone does most of the work. Ravi and Sandeep owing each other in both directions collapses into one number — Meera’s sharp remark on the bus.

Step two: match the biggest creditor with the biggest debtor, repeatedly.

```
while creditors and debtors:
    creditor, owed = max(creditors)          # a max-heap in practice
    debtor, owes = max(debtors)
    amount = min(owed, -owes)
    record(debtor → creditor, amount)
    # whichever is now zero drops out; the other keeps its remainder
```

Each payment **zeroes at least one person**, so with k people having a non-zero balance you need at most $k - 1$ payments. Six friends: at most five. Anjali’s answer.

Compare with settling pairwise: $n(n-1)/2$ possible pairs, and in this group nine of the fifteen had money moving. **Nine payments become five**, and in a group of twenty it is up to 190 pairs against at most 19 payments.

The part most candidates overclaim

The greedy algorithm gives **at most** $k - 1$ transactions. It does **not** always give the minimum.

The true minimum is smaller whenever a subset of people happens to net to exactly zero among themselves — then that subset can be settled internally and separated from the rest. Finding the best such partition is the **subset-sum / set-partition problem**, which is NP-hard. So the exact minimum cannot be computed efficiently for large groups.

```
balances: A +5, B +5, C -5, D -5
greedy:   C pays A 5, D pays B 5 → 2 transactions
optimal:                                     2 transactions (same here)

balances: A +3, B +2, C -5
greedy:   C pays A 3, C pays B 2 → 2 transactions = k - 1 = 2, optimal

the gap appears when zero-sum subsets exist and greedy splits them:
balances: A +4, B +1, C -1, D -4
greedy:   D pays A 4, C pays B 1 → 2 (greedy happens to find it)
a worse ordering could give 3.
```

Say this. “Greedy gives at most $k-1$ and is not guaranteed optimal, because the exact problem is NP-hard” is a sentence that changes how the rest of the interview goes. Claiming the greedy is optimal is the single most common mistake in this question.

And Vinod’s complaint, which is a design decision not a bug

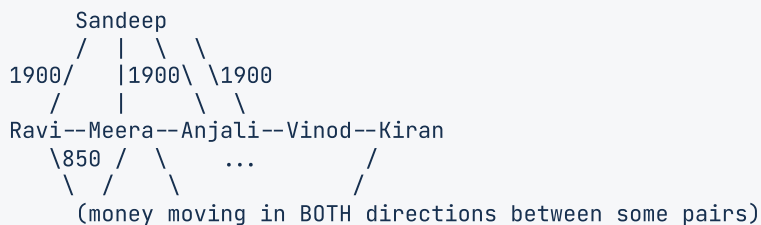
Simplification **destroys provenance**. After it, Vinod owes Meera money for dinners he ate that Ravi paid for. The arithmetic is right and the story is gone, and people mind — because a payment request from someone you never transacted with looks like a mistake.

Real Splitwise makes simplification an **opt-in setting per group**, and that is the right answer: it is a preference, not an optimisation, and the correct design keeps both views available because the underlying ledger is still there.

4 The picture

The collapse from pairs to nets, which is the algorithm in one image:

PAIRWISE – up to $n(n-1)/2 = 15$ edges for 6 people, 9 with money on them



NET – one number per person, and the sum is always zero

Sandeep	+9,500		Anjali	-2,300
Meera	+1,800		Vinod	-4,600
Ravi	+50		Kiran	-4,450

creditors	+11,350		debtors	-11,350

SETTLE – biggest creditor against biggest debtor, repeatedly

Vinod	→ Sandeep	4,600	(Vinod is done)
Kiran	→ Sandeep	4,450	(Kiran is done)
Anjali	→ Sandeep	450	(Sandeep is done: 9,500 settled)
Anjali	→ Meera	1,800	(Meera is done)
Anjali	→ Ravi	50	(Anjali and Ravi are done)

5 transactions for 6 people. At most $k - 1$, always.			

What to notice: **every single payment finishes somebody off**. That is the whole reason the bound is $k - 1$ rather than something larger, and it is the sentence to say when asked why.

The classes:

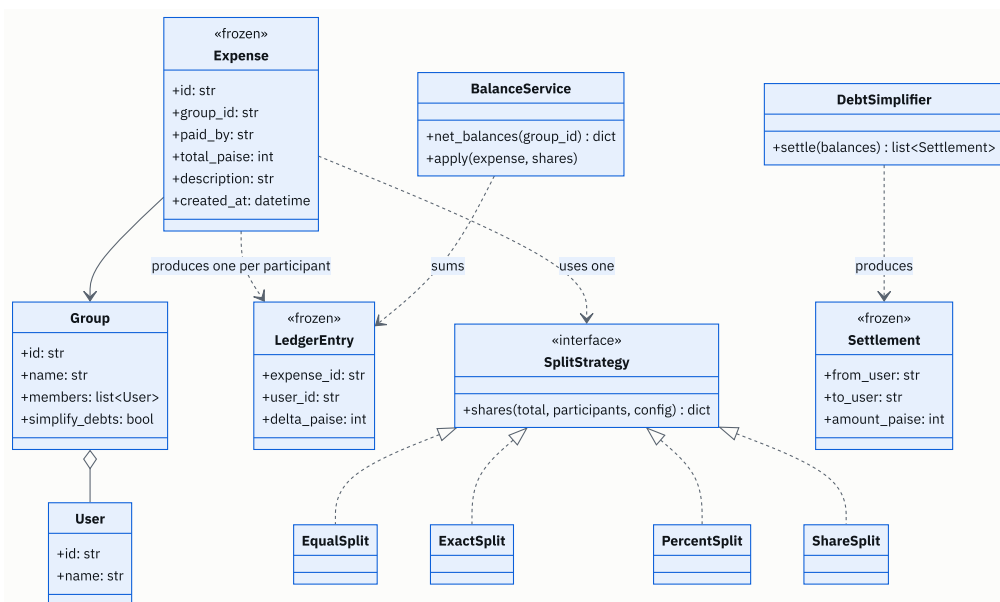


FIGURE 85.1

What to notice: `Expense` and `LedgerEntry` are **frozen**, and the balance is a *service* rather than a field on `User`. The ledger is what happened; the balance is a conclusion drawn from it. That is the same records-versus-rules line as the library’s fines on [day 082](#).

5 How it actually works

Move 1 · Clarify (minutes 0–5)

“Groups only, or also one-to-one expenses between friends?” — Both; a one-to-one expense is a group of two, which keeps the model uniform. “Which split types?” — Equal, exact, percentage and shares. “Multiple currencies?” — Ask. If yes, every amount needs a currency and settlement across currencies needs a rate at a point in time, which is materially more design. “Should the app simplify debts automatically?” — The right answer is “make it a per-group setting”, and saying why is worth more than either yes or no.

“I will assume all amounts are in one currency stored as integer paise, that expenses can be edited and deleted, and that settlements are recorded rather than actually moving money — no payment gateway in scope.”

Move 2 • The nouns (minutes 5–12)

- `User`, `Group` — obvious, and `Group` carries the `simplify_debts` flag.
- `Expense` — frozen: who paid, how much, when, in which group.
- `LedgerEntry` — frozen: one per participant per expense, a signed delta. This is the truth.
- `SplitStrategy` (*interface*) — four implementations.
- `BalanceService` — nets the ledger, and maintains the materialised balances.
- `DebtSimplifier` — balances to a list of settlements.
- `Settlement` — frozen: from, to, amount.

Seven, one interface with four implementations. `DebtSimplifier` is a plain class, not an interface — there is only one algorithm and I cannot name a second implementation anyone wants.

Move 3 • Money, before anything else

```
def equal_shares(total_paise: int, participants: list[str]) → dict[str, int]:
    """Split as evenly as integers allow, then assign the remainder DETERMINISTICALLY."""
    if not participants:
        raise ValueError("no participants")
    base, remainder = divmod(total_paise, len(participants))
    shares = {person: base for person in participants}
    for person in sorted(participants)[:remainder]:
        shares[person] += 1
    assert sum(shares.values()) == total_paise    # the invariant, every time
    return shares
```

Three things worth narrating as you write them: **integers, never floats**; the remainder goes somewhere **deterministic** so an edited expense recomputes identically; and the assertion, which turns a whole family of silent money bugs into a loud failure.

Move 4 • The ledger

```
def record(expense: Expense, shares: dict[str, int]) → list[LedgerEntry]:
    """One signed entry per participant. The payer is credited the whole amount
    and debited their own share, so the entries always sum to zero."""
    entries = [LedgerEntry(expense.id, expense.paid_by, +expense.total_paise)]
    entries += [LedgerEntry(expense.id, person, -share) for person, share in shares.items()]
    assert sum(e.delta_paise for e in entries) == 0
    return entries
```

The entries for one expense always sum to zero, which is the double-entry idea in one assertion. It also means the sum of *all* balances in a group is always zero, which is the assertion the simplifier starts from.

Move 5 • The simplifier — the interesting part

```
def simplify(balances: dict[str, int]) → list[Settlement]:
    """Fewest-ish transactions to settle a group.

    Net each person first, then repeatedly match the largest creditor with the
    largest debtor. Each payment zeroes at least one person, so this produces at
    most k-1 settlements for k people with a non-zero balance.

    NOT guaranteed optimal: the true minimum requires finding zero-sum subsets,
    which is the set-partition problem and is NP-hard.
    """
    assert sum(balances.values()) == 0, "balances must net to zero"

    creditors = [(-amount, user) for user, amount in balances.items() if amount > 0]
    debtors = [(amount, user) for user, amount in balances.items() if amount < 0]
    heapq.heapify(creditors)          # min-heap of negatives = max-heap
    heapq.heapify(debtors)

    settlements: list[Settlement] = []
    while creditors and debtors:
        credit, creditor = heapq.heappop(creditors)
        debit, debtor = heapq.heappop(debtors)
        amount = min(-credit, -debit)
        settlements.append(Settlement(debtor, creditor, amount))

        if -credit > amount:
            heapq.heappush(creditors, (credit + amount, creditor))
        if -debit > amount:
            heapq.heappush(debtors, (debit + amount, debtor))

    return settlements
```

Two heaps, because “largest creditor” and “largest debtor” have to be found repeatedly. Python’s `heapq` is a min-heap, so amounts are negated — worth saying, because it looks like a bug otherwise.

Say the two claims separately. At most `k - 1` settlements, because each payment zeroes somebody. And not necessarily the minimum, because the exact problem is NP-hard.

Move 6 • The concurrency question

Two people add an expense to the same group at the same moment.

```
BEGIN;
INSERT INTO expenses (...);
INSERT INTO ledger_entries (...);           -- several rows
UPDATE balances SET net_paise = net_paise + $delta
WHERE group_id = $g AND user_id = $u;      -- read-modify-write, in SQL
COMMIT;
```

The important detail is `net_paise = net_paise + $delta` **inside the UPDATE**, not read-then-write in application code. The database applies the delta atomically under a row lock, so two concurrent expenses both land. Reading the balance into Python, adding, and writing it back is the classic lost update, and it loses money.

Real systems

- **Splitwise** itself makes “simplify debts” a per-group toggle, exactly because of Vinod’s complaint. Their published description of the feature emphasises that the totals are unchanged.
- **Double-entry bookkeeping** is the model here and is four hundred years old: every transaction produces entries that sum to zero, and balances are derived. Ledger systems — **Stripe**, **Modern Treasury**, **TigerBeetle** — are all built on it, and none of them store a balance as the primary truth.
- **Integer minor units** are universal for money: Stripe’s API is in the smallest currency unit, `java.math.BigDecimal` and Python’s `decimal` exist for the cases where fractions of a paisa genuinely matter, and floating point is used by nobody who has been burned once.
- **The banker’s-rounding question** appears in payroll and tax systems for the same reason as the remainder here, and the answer is the same: pick a deterministic rule and assert the total.

6 The numbers

Pairwise against netted

```
n people, worst case pairwise debts: n(n-1)/2
n = 6:    15 possible pairs    (9 with money moving, in the trip)
n = 20:   190 possible pairs
n = 50:  1,225 possible pairs

after netting and greedy settlement: at most k - 1  (k = people with non-zero balance)
n = 6:      5 transactions
n = 20:     19 transactions
n = 50:     49 transactions
```

```
n = 20:  190 → 19,  a 10x reduction
n = 50:  1,225 → 49, a 25x reduction
```

The saving grows with the group, because pairs grow quadratically and settlements grow linearly. That is the number to lead with.

The rounding leak, which is the correctness argument

```
one paisa dropped per 3-way split
50,000,000 expenses on the platform
→ 500,000 rupees unaccounted for, and every group's balances fail to sum to zero
```

Five lakh, from one missing line. And the damage is not the money — it is that balances stop summing to zero, so the simplifier's assertion fires, or worse, does not and produces settlements that leave residue. **Assign the remainder and assert the total.**

Scale

```
users          10,000,000
groups         3,000,000
expenses       50,000,000 total, ~40,000/day
average group size      5
ledger entries  50,000,000 × 5 = 250,000,000

Expense row    ~120 B × 50M = 6 GB
LedgerEntry row ~48 B × 250M = 12 GB
Balance row    ~40 B × 15M = 600 MB
-----
total                      ≈ 19 GB
```

Nineteen gigabytes fits on one database server comfortably. **This is not a scale problem**, and saying so keeps the interview on the parts that matter — the money arithmetic and the simplification.

Deriving a balance, and why it needs a cache

```
a heavy group: 8 members, 400 expenses over two years
deriving one user's balance: sum 400 × 8 = 3,200 ledger rows
at ~1 µs per row from a warm index: ~3 ms
the home screen shows balances for ALL of a user's groups, say 12
→ 36 ms of pure summation per screen load, per user
```

Thirty-six milliseconds is not fatal and it is not free, and it is entirely wasted work because the answer changes only when an expense changes. Hence a materialised balance updated in the same transaction, and a nightly reconciliation that recomputes from the ledger and alerts on drift.

The simplifier's cost

```
k people with a non-zero balance
heapify:          O(k)
each settlement:  2 pops + up to 2 pushes = O(log k)
settlements:      at most k - 1
-----
total:           O(k log k)

k = 50: ~300 heap operations, microseconds
```

Nothing. The algorithm is cheap and the interesting question about it is correctness, not speed — which is worth saying, because it steers the follow-up to the NP-hardness point rather than to optimisation.

7 The trade-offs

What this design gives up

Simplification destroys provenance, and users notice. After it, Vinod owes Meera for dinners Ravi paid for. The totals are right; the story is gone. That is why it must be a per-group setting rather than always-on, and why the ledger has to survive underneath so the unsimplified view is still available. A design that simplifies destructively — overwriting the ledger with the settlements — cannot answer “why?” and cannot be undone.

Greedy is not optimal and I would not claim it is. The exact minimum requires finding subsets that net to zero, which is set-partition and NP-hard. For a group of six the difference is usually zero or one transaction; for fifty it could be more. The honest position: $k - 1$ is a good bound, it is achieved in one pass, and the exact optimum is not worth exponential time to save one payment.

Materialised balances can drift. Any bug in the update path, any expense edited outside the transaction, any partial failure, and the balance disagrees with the ledger. That is why the reconciliation job is part of the design rather than an operational afterthought — and why the ledger, not the balance, is what the app shows when they disagree.

Editing and deleting expenses is harder than it looks. An edit is not an update; it should be a reversing set of ledger entries plus a new set, so history is preserved and the balance moves by the difference. Overwriting the original entries makes the audit trail lie, and makes a concurrent edit lose data.

One currency. Multi-currency turns every amount into an amount-plus-currency, and settlement across currencies needs a rate *at a point in time* which then has to be stored with the settlement, or the same debt is worth different amounts on different days. That is a genuinely bigger design and I would scope it out explicitly rather than hand-wave it.

Partial settlements and disputes. Someone pays half. Someone says they already paid in cash. Both are just more ledger entries — a settlement is an expense with one payer and one participant — which is a nice property of the model, and worth pointing at.

“I would change this design if...”

- **...groups get very large** — a shared house of 50, or an office lunch group. Then deriving is too slow to fall back on and the reconciliation job needs to be incremental rather than a full recompute.
- **...money actually moves.** A payment gateway makes settlements real transactions, which brings idempotency keys, reversals and the whole failure model from the ATM on [day 080](#).
- **...multiple currencies are required.** Store the currency with every amount, never convert on write, and freeze the rate onto each settlement.
- **...users want the story preserved.** Then simplification is a *view* rendered on demand rather than a stored set of settlements, and the pairwise debts stay derivable.

The honest concession

The design’s centre is a very old idea — **double-entry bookkeeping** — and almost everything good about it follows from one decision: the ledger is the truth and the balance is derived. The simplification algorithm is a small, cheap piece of graph arithmetic on top, and it is the part everybody talks about because it is the part that looks clever. If I had to defend one decision, it would not be the simplifier; it would be that expenses and ledger entries are immutable, because that is what makes edits, disputes, reconciliation and “why do I owe this?” all answerable.

8 In the interview

How it gets asked

- The standard: *“Design Splitwise.”* Then, quickly: *“How do you minimise the number of transactions?”*
- The arithmetic probe, which is a trap: *“Three people split a thousand rupees. What do you store?”*
- The modelling probe: *“Do you store balances, or compute them?”*
- The correctness probe: *“Two people add an expense to the same group at the same moment.”*
- The one that separates people: *“Is your simplification optimal?”*

The timed script

Minutes 0–5 · Clarify. Groups and one-to-one? Which split types? Multiple currencies? Should simplification be automatic — and answer that one yourself as “a per-group setting, and here is why”.

Minutes 5–10 · Money first. Integer paise, never floats. The three-way split remainder, the deterministic assignment, and the assertion. This takes two minutes and it is the correctness foundation for everything after.

Minutes 10–18 · The model. Expense and LedgerEntry, both immutable, entries summing to zero per expense. `SplitStrategy` with its four implementations. Balances derived, materialised, reconciled.

Minutes 18–30 · The deep dive: simplification. Net each person, assert the sum is zero, greedy match by two heaps, the $k - 1$ bound with *why*, and the NP-hardness caveat.

Minutes 30–36 · Provenance and the setting. Vinod's complaint, and why it makes simplification a preference rather than an optimisation.

Minutes 36–40 · Failure and scale. The concurrent-expense update, the reconciliation job, and the 19 GB figure that says this is not a scale problem.

The follow-ups

“Three people split a thousand rupees. What do you store?” “Integer paise — 100,000 — never a float, because binary floating point cannot represent a tenth and a system that adds a hundred thousand of those is wrong in a way nobody can trace. Then 100,000 divided by three is 33,333 with a remainder of one paisa, and that paisa has to go somewhere deterministic — I sort the participant ids and give it to the first, or give it to the payer. Deterministic matters because an edited expense gets recomputed, and the same input must give the same shares. And I assert that the shares sum to the total, every time. Without that, balances stop summing to zero and every downstream calculation is quietly wrong.”

“Do you store balances or compute them?” “Both, and the order matters. The expenses and their ledger entries are the source of truth — they are what happened, they are what the user can see and edit, and they are immutable. A balance is derived by summing ledger entries, which for a heavy group is a few thousand rows and a few milliseconds; the home screen needs a dozen of those, so I materialise the balance per group and user and update it in the same database transaction as the expense. And then a reconciliation job recomputes from the ledger periodically and alerts on any mismatch, because a cache that nobody checks eventually lies. If they ever disagree, the ledger wins.”

“How do you minimise the number of transactions?” “Two steps. First, net everybody out: one number per person instead of a debt per pair. That alone collapses two people owing each other in both directions into one payment, and it turns up to n -times- n -minus-one-over-two pairs into n numbers. Second, repeatedly match the largest creditor with the largest debtor and settle the smaller of the two amounts, using two

heaps. Every payment zeroes at least one person, so for k people with a non-zero balance you need at most $k - 1$ payments. Six friends: at most five, against fifteen possible pairs. Twenty people: nineteen against a hundred and ninety.”

“Is that optimal?” “No, and I would not claim it is. It gives at most $k - 1$, which is a good bound and achieved in one pass. But the true minimum can be lower whenever a subset of people happens to net to zero among themselves — that subset can be settled internally and cut out of the rest. Finding the best such partition is the set-partition problem, which is NP-hard, so there is no efficient exact algorithm for large groups. In practice the greedy is within a transaction or two, and I would not spend exponential time to save one payment.”

“A user complains they are being asked to pay someone they never borrowed from.” “That is the real cost of simplification and it is not a bug — the arithmetic is right and the story is gone. Which is why it should be a per-group setting rather than always on, and why the ledger must survive underneath so the unsimplified view is still available. Splitwise does exactly this. The design consequence is that simplification produces a *view* or a set of suggested settlements, and never overwrites the ledger.”

“Two people add an expense to the same group at the same instant.” “The expense inserts are independent and both land. The risk is the materialised balance, and the fix is to write the update as `net = net + delta` inside the SQL rather than reading the balance into application code, adding, and writing it back — that is the classic lost update and it loses money. With the arithmetic in the UPDATE, the database applies both deltas atomically under a row lock. And because the ledger is the truth, the reconciliation job will catch it even if something goes wrong.”

“What if someone edits or deletes an expense?” “Not an update — a reversal plus a new expense. I would write reversing ledger entries for the original, then entries for the new version, so history is preserved, the audit trail is honest, and the balance moves by the difference rather than being recomputed and hoped for. It also makes concurrent edits safe, because two reversals and two new sets of entries all commute, whereas two overwrites do not.”

A model answer

Asked: *design Splitwise. How do you minimise the number of transactions?*

“Let me do the money arithmetic first, because it is the correctness foundation and it takes two minutes.

Everything is integer paise. Never floating point — binary floating point cannot represent a tenth, and a system that adds a hundred thousand of those is wrong in a way that is untraceable. And then the problem integers do not solve: a thousand rupees between three people is 100,000 paise divided by three, which is 33,333 with one paisa left over. That paisa has to be assigned somewhere deterministic — sorted participant order, or the payer — because an edited expense will be recomputed and must give the same answer. And I assert that the shares sum to the total. Without that assertion, balances stop summing to zero, and at fifty million expenses a dropped paisa per three-way split is about five lakh unaccounted for.

The model is double-entry bookkeeping. An `Expense` is immutable — who paid, how much, when, in which group. It produces one immutable `LedgerEntry` per person: the payer credited the full amount, each participant debited their share. The entries for one expense always sum to zero, which I assert too, and that is what makes every group’s balances sum to zero.

How the shares are computed varies four ways — equal, exact amounts, percentages, and shares like ‘Ravi eats twice as much’ — so that is an interface with four real implementations, and each ends with the same total assertion.

Balances are *derived* by summing ledger entries. The ledger is the truth; the balance is a conclusion. But deriving is a few thousand rows for a heavy group, and the home screen wants a dozen groups at once, so I materialise the balance per group and user and update it in the same transaction as the expense — with the arithmetic written as `net = net + delta` inside the SQL, not read-modify-write in application code, which is the classic lost update. And a reconciliation job recomputes from the ledger periodically, because a cache nobody checks eventually lies.

Now the question. Six people can owe each other across fifteen pairs, and on a real trip about nine of those have money moving. Two steps take it to five.

Step one: net everybody out. One number per person instead of a number per pair. This alone collapses two people owing each other in both directions into a single payment.

Step two: repeatedly match the largest creditor with the largest debtor and settle the smaller of the two amounts — two heaps, $O(k \log k)$ overall, microseconds. **Every payment zeroes at least one person, so for k people with a non-zero balance you need at most $k - 1$ payments.** Six people, five payments. Twenty people, nineteen instead of up to a hundred and ninety.

One thing I want to be precise about: that is at most $k - 1$, and it is **not guaranteed optimal**. The true minimum is smaller whenever some subset of people nets to zero among themselves, because that subset can be settled internally and separated. Finding the best partition is the set-partition problem, which is NP-hard, so there is no efficient exact algorithm. The greedy is a good heuristic and I would not spend exponential time to save one payment.

And the cost that is not arithmetic: simplification destroys provenance. After it, someone is asked to pay a person they never transacted with, and users mind, because it looks like a mistake. So simplification should be a per-group setting, and it must produce suggested settlements rather than overwriting the ledger — which the immutable ledger makes easy.

On scale: ten million users, fifty million expenses, a quarter of a billion ledger entries is about nineteen gigabytes. That fits on one server. This is not a scale problem, and I would rather spend the remaining time on the money arithmetic and the edit path, because that is where the bugs are.”

9 Recall card

- **Money is integer paise, never floats — and the remainder must be assigned deterministically.** $\text{₹}1,000 \div 3 = 33,333$ paise each with **one paisa left over**; sort the participants and give it to the first (or to the payer), then **assert** `sum(shares) = total`. A dropped paisa per 3-way split across 50M expenses is **₹5 lakh** and, worse, balances stop summing to zero.
- **Double entry: the ledger is the truth, the balance is derived.** `Expense` and `LedgerEntry` are **immutable**, the entries for one expense **sum to zero**, and every group's balances therefore sum to zero. Materialise balances for speed, update them with `net = net + delta` **inside the SQL** (not read-modify-write), and run a **reconciliation job** — if they disagree, the ledger wins. An edit is a **reversal plus a new expense**, never an overwrite.
- **Minimising transactions is two steps.** *Net each person* — one number instead of a debt per pair, which alone collapses mutual debts. Then *repeatedly match the largest creditor with the largest debtor* with two heaps, $O(k \log k)$. **Every payment zeroes at least one person, so at most $k - 1$ payments:** 6 people $\rightarrow$ 5, against 15 possible pairs; 20 people $\rightarrow$ 19 against 190.
- **Say it is NOT optimal.** The exact minimum needs zero-sum subsets found and separated, which is the **set-partition problem — NP-hard**. “At most $k - 1$, in one pass, not guaranteed minimal” is the strong answer; claiming optimality is the most common mistake in this prompt.

- **Simplification destroys provenance, and that is a design decision not a bug** — you end up owing someone you never transacted with. Make it a **per-group setting**, produce **suggested settlements** rather than overwriting the ledger, and keep the unsimplified view derivable. Scale is a non-issue: 10M users and 250M ledger entries $\approx$ **19 GB**, so spend the interview on the arithmetic and the edit path.

DSA topic: Doubly and circular linked lists **System design topic:** Design Splitwise

Code these, in this order

One rule for the whole set: **after every operation, walk the list both ways and compare.** A doubly linked list can be correct forwards and broken backwards, and nothing else will tell you.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Design Linked List	LeetCode 707 (Medium)	Do it doubly this time. Every boundary, twice as many pointers.
2	Design Browser History	LeetCode 1472 (Medium)	A cursor in a doubly linked list, and truncating the forward chain on a new visit.
3	Flatten a Multilevel Doubly Linked List	LeetCode 430 (Medium)	Splicing a whole sublist in, with both directions maintained at three joints.
4	LRU Cache	LeetCode 146 (Medium)	Build it again, and this time say the one-sentence justification before writing a line.

On problem 1, write `_check` before anything else

Walk forwards collecting values, walk backwards collecting values, assert one is the reverse of the other and that the length matches the stored size. Then call it after every single operation in your tests. It will find bugs you would otherwise spend an hour on.

On problem 2, notice what the `prev` pointer is for

Back and forward are the two directions. Then answer: when the user visits a new page after going back, what happens to everything ahead of the cursor, and which pointer has to change?

On problem 3, count the joints

Splicing a child list in means three places where links must be fixed, and each needs both directions. Draw all six assignments before writing any of them.

On problem 4, say the sentence first

“The map gives me the node in $O(1)$; unlinking must also be $O(1)$, and reaching the predecessor from a node is exactly what `prev` buys.” Say it, then write the code.

The one-thing drill

1. State, in one sentence, the only operation the second pointer buys.
2. Write the two assignments that remove a node.
3. State the same operation's cost in a singly linked list, and why.
4. Give the LRU number: capacity 100,000, cost per read, both ways.
5. State the general rule about when a doubly linked list is worth it.

The ordering drill

1. Write the four insertion assignments in the correct order.
2. Write them in the wrong order and trace what `fresh.next` ends up being.
3. Say the habit that makes the ordering bug impossible.
4. Rewrite `insert` to take both neighbours as parameters and say why that helps.

The half-correct drill

1. Write a removal that updates only the forward links.
2. Iterate forwards and print. Confirm it looks right.
3. Print `len`. Confirm it looks right.
4. Now walk backwards and describe what you find.
5. Say why this failure mode does not exist in a singly linked list.
6. Write `_check` and confirm it catches it.

The break-it drill

Trigger each and record the exact output or error text:

1. Remove a node with only one direction updated, then `pop_back`.
2. Insert with `left.next = fresh` written first.
3. Remove the first node with no sentinels. Quote the error.
4. Traverse a circular list with `while node is not None`.
5. Iterate a circular list without skipping the sentinel. Describe the extra element.
6. Remove a node and leave its `prev` and `next` set, then walk from it.
7. Call `remove(5)` where 5 is a value rather than a node. Say what the $O(1)$ guarantee actually requires.

The circular drill

1. Draw a circular doubly linked list with one sentinel.
2. Write the emptiness test.
3. Write `sentinel.next` and `sentinel.prev` in words.

4. Say how many `None`s appear in the structure, and what that buys.
5. Write a correct traversal and say what makes it terminate.

The Josephus drill

1. Simulate $n = 7$, $k = 3$ by hand and write the elimination order.
2. Run your implementation and confirm the survivor.
3. State the simulation's time and space.
4. Write the closed-form recurrence and say its time and space.
5. Say which you would give in an interview and why you would mention both.

The cost drill

1. Bytes per node, singly and doubly, with `__slots__`.
2. The percentage overhead, and the absolute at 10 million nodes.
3. Writes per removal and per insertion, both structures.
4. Name the cost that does not appear in any table.
5. Name three cases where you would *not* use a doubly linked list.

The real-world drill

For each, say what structure it is and what the second pointer buys:

1. `collections.deque`
 2. `OrderedDict`
 3. The Linux kernel's `list_head`
 4. A browser's back and forward buttons
 5. A music player on repeat
 6. An LRU cache
-

The money drill

1. Say why money is never a float, in one sentence about binary.
2. Split ₹1,000 three ways in paise and show the remainder.
3. Write the deterministic assignment and say why determinism matters.
4. Write the assertion and say what it prevents.
5. Compute the platform-wide leak at 50 million expenses.

The ledger drill

1. Write the ledger entries for "Sandeep pays ₹11,400 for a room shared equally by 6".
2. Confirm they sum to zero and say why that matters.

3. Say what is stored and what is derived, and which one wins in a disagreement.
4. Say why the balance is materialised anyway, with the millisecond figure.
5. Describe the reconciliation job in one sentence.

The simplification drill

Given balances Sandeep +9,500, Meera +1,800, Ravi +50, Anjali -2,300, Vinod -4,600, Kiran -4,450:

1. Confirm they sum to zero.
2. Run the greedy by hand and write out every settlement.
3. Count them and compare with $k - 1$.
4. Count the possible pairs for six people.
5. Say why every payment zeroes at least one person.
6. Construct a balance set where a zero-sum subset exists, and say what the optimal is.
7. State what problem finding the true minimum reduces to, and its complexity.

The provenance drill

1. State Vinod's complaint in one sentence.
2. Say whether it is a bug, and defend the answer.
3. Say what the design consequence is — what simplification must produce, and what it must not touch.
4. Say why real Splitwise makes it a setting.

The concurrency drill

1. Write the wrong balance update as read-modify-write in application code.
2. Describe what two concurrent expenses do to it.
3. Write the correct SQL.
4. Say why the ledger being the source of truth limits the damage.
5. Describe how you would handle an *edit* to an existing expense, and why not an update.

The estimation drill

Compute each, showing the multiplication:

1. Possible pairs for 6, 20 and 50 people, and the settlement bound for each.
2. Total storage for 10M users, 50M expenses and 250M ledger entries.
3. Time to derive one balance for a group with 8 members and 400 expenses.
4. The same for a home screen showing 12 groups.

5. The simplifier's complexity, and the operation count at $k = 50$.

Then say which single number tells you this is not a scale problem.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why does an LRU cache use a doubly linked list?* The one operation the second pointer buys, the two assignments, the singly linked cost and why, the capacity-100,000 comparison, sentinels at both ends, and the honest cost including the half-correct failure mode.
 2. *Design Splitwise. How do you minimise the number of transactions?* Integer paise and the remainder rule with its assertion, the immutable ledger with entries summing to zero, balances derived then materialised then reconciled, netting each person, the greedy with two heaps, the `k - 1` bound with *why*, the NP-hardness caveat, and provenance as a setting.
 3. *Is your debt simplification optimal?* At most $k - 1$ and achieved in one pass; not minimal, because zero-sum subsets could be separated; that is set-partition and NP-hard; and the practical position on why you would not pay exponential time for one payment.
-

Before you move on

- ☐ I can state the one operation the second pointer buys, in one sentence.
- ☐ I wrote `_check` and used it to catch a one-direction removal.
- ☐ I can write the four insertion assignments in the right order and say why.
- ☐ I traversed a circular list with `while node is not None` and saw it hang.
- ☐ I can state the LRU justification with the capacity-100,000 numbers.
- ☐ I can name three cases where I would not use a doubly linked list.
- ☐ I implemented Josephus by simulation and know the closed form exists.
- ☐ I can say why money is never a float, in terms of binary.
- ☐ I can split ₹1,000 three ways and say where the last paisa goes and why deterministically.
- ☐ I can write the assertion and say what it prevents platform-wide.
- ☐ I can write ledger entries that sum to zero and say why that matters.
- ☐ I ran the greedy by hand on the six-person example and got five settlements.
- ☐ I can say why every payment zeroes at least one person.

- ☐ I can name the NP-hard problem the true minimum reduces to.
- ☐ I can defend “provenance loss is a setting, not a bug”.
- ☐ I can write the correct concurrent balance update in SQL.
- ☐ I answered all three questions above out loud.

Linked lists revision and mock round

DATA STRUCTURES AND ALGORITHMS

Linked lists revision and mock round

You can solve two unseen pointer-surgery problems cold.

SYSTEM DESIGN

Design BookMyShow

You can model shows, seats and the booking lock that stops double-selling a seat.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two problems, no hints, talk as you go.*
- *Design BookMyShow. Two users click the same seat at the same moment.*

1 What this is, and why they ask it

Eight days ago a node was two fields. Since then you have inserted and deleted at any position, deleted the head special case with a dummy node, reversed a list three ways, found the middle and the Nth from the end in one pass, detected a cycle and proved where it starts, merged and sorted, and added a second pointer to get $O(1)$ removal.

Today is none of those. Today is **recognition**: looking at an unfamiliar linked-list problem and knowing, within thirty seconds, which two or three of those techniques it is made of.

That is the actual skill, because linked-list problems are almost never one technique. “Reorder the list” is the runner plus a reversal plus a merge. “Copy a list with random pointers” is a hash map or an interleaving trick. “Is it a palindrome” is the runner plus a reversal plus a comparison, and then a reversal back. A candidate who has memorised eight solutions freezes on the ninth. A candidate with six trigger questions does not.

They run this round because the phase’s techniques are small and combinable, which makes it very easy to set a problem nobody has seen. The defence is not more problems. It is being able to say *what this is made of* before writing anything.

2 The story

Ismail’s cycle shop is the one under the peepal tree, and it is about the width of two cupboards.

People bring him bicycles in every state. He does not, as far as anyone has ever seen, take a bicycle apart to find out what is wrong with it. He asks two questions and then he knows.

The first question is: does it happen when you pedal, or all the time? The second is: does it get worse when you turn, or on a straight road?

That is it. Between those two answers he has, he says, about seven problems, and he has been fixing those same seven for thirty-one years. Chain. Bearings. Brake rubbing. Wheel out of true. Bent mudguard catching. Tyre. Something loose. He does not have a list written down anywhere; he just knows that a noise when you pedal and not when

you coast is one small family of things, and a noise that gets worse on a turn is a completely different family, and once you know which family, the actual repair is the easy part.

The boy who works with him now, Sameer, is quick and careful and much better with the newer bikes with gears. And for the first six months he took things apart. A man would bring in a bicycle making a noise and Sameer would have the chain off and the wheel out inside four minutes, and then he would sit there in front of a bicycle in pieces, still not knowing what was wrong, and Ismail would come over, lift the front end, spin the wheel, listen, and say bearings.

What Ismail told him, eventually, was not about bicycles. He said: you are very good at the repairs. You are doing them in the wrong order. Find out which of the seven it is first, and then you will only ever do one repair. If you start repairing before you know, you will do three.

Sameer says the change took about a month and that he now diagnoses more than he repairs, and that customers think he has got faster at fixing bicycles, which is not what happened at all.

3 The idea in plain English

Ismail's two questions are the point. **Diagnose first, then repair** — and for linked lists there are six questions, and every problem in this phase is one or two of them.

The six trigger questions

Run these on any linked-list problem, in under thirty seconds.

1. **Can the operation change which node is first?** → **Dummy head** (day 080). Remove-by-value, remove-Nth-from-end, merge, partition, reverse-a-sublist. Write `dummy = Node(0, head)` and `return dummy.next` together, before the loop.

2. **Do I need a position I cannot count to — the middle, or `n` from the end — in one pass?** → **The runner** (day 082). Different *speeds* for the middle; a fixed *gap* for a distance from the end.

3. **Might this list not end?** → **Floyd's** (day 083). Two runners; if the fast one catches the slow one from behind, there is a cycle; then head-plus-meeting-point at equal speed finds the entrance.

4. **Am I turning the arrows around?** → **The three-pointer reverse** (day 081). *Save, turn, previous up, current up*. Return `previous`.

5. Am I combining two ordered things, or imposing order? → Merge, and merge sort (day 084). Builder dummy, $\leq$ for stability, attach the remainder in one assignment.

6. Do I already hold the node I must remove? → Doubly linked (day 085). Two assignments, $O(1)$. If I have to *search* for the node first, the search is $O(n)$ and the second pointer buys nothing.

The five sentences that carry the phase

1. “To change the list at position k , I must be standing at $k - 1$.” You cannot reach backwards, so every insertion and deletion is really “walk to the node before”.
2. “Save the next node before you overwrite the pointer.” The defining mistake of the phase: overwrite `current.next` first and the rest of the list is gone, with no error.
3. “Return `dummy.next`, never `head`.” And its cousin: a bare function that changes the first node must *return* the new head, because rebinding a parameter changes nothing for the caller.
4. “Insertion is $O(1)$ given the node.” Given a position, both a list and an array are $O(n)$ — one pays for the walk, the other for the shift.
5. “Advance only when you keep.” After unlinking, `previous.next` is a node you have not inspected.

The one-page comparison

TECHNIQUE	COST	SPACE	USE IT WHEN
Traverse with <code>previous</code>	$O(n)$	$O(1)$	Delete or insert by position or value
Dummy head	—	$O(1)$	The first node can change, or you are building a list
Reverse (iterative)	$O(n)$	$O(1)$	Turning arrows; also the $O(1)$ -space half of palindrome
Reverse (recursive)	$O(n)$	$O(n)$ stack	Never in Python past ~1000 nodes
Runner, different speeds	$O(n)$	$O(1)$	The middle, in one pass
Runner, fixed gap	$O(n)$	$O(1)$	The Nth from the end, in one pass
Floyd's	$O(n)$	$O(1)$	Cycle detection, and the entrance
Merge two sorted	$O(n + m)$	$O(1)$	Combining; also the inner loop of a sort
Merge sort	$O(n \log n)$	$O(\log n)$	Sorting a list; $O(1)$ if bottom-up
Doubly linked	$O(1)$ removal	+8 B/node	You already hold the node (LRU)
Hash map of nodes	$O(n)$	$O(n)$	The honest baseline for cycles and for copying

What the round is actually scoring

Four things, in this order of weight:

- 1. Did you say what the problem is made of, before coding?** “This is the runner to find the middle, then a reverse of the second half, then an interleave.” That sentence is the answer.
- 2. Did you draw the pointers before assigning them?** Every bug in this phase is an assignment made before its picture existed.
- 3. Did you handle the empty list, one node, and two nodes?** All three, out loud, before you claim to be finished.
- 4. Did you leave the caller's data as you found it,** or say that you did not? A function asked to *inspect* a list must not return it reversed.

4 The picture

The whole phase, grouped by the question that triggers it.

"can the first node change?"	→ DUMMY HEAD
+-- remove all nodes equal to v	guard form: Node(0, head)
+-- remove the nth from the end	builder form: Node(0) + tail
+-- merge two sorted lists	
+-- partition around a value	
+-- reverse a sublist	
"a position I cannot count to, in one pass?"	→ RUNNER
+-- the middle	different SPEEDS (1 and 2)
+-- first middle → splitting for a sort	
+-- second middle → "return the middle"	
+-- the nth from the end	fixed GAP
"might it not end?"	→ FLOYD'S
+-- has a cycle	meet ⇒ cycle
+-- where does the cycle start	head + meeting, equal speed
+-- find the duplicate number	read the array as i → nums[i]
"turning the arrows around?"	→ THREE-POINTER REVERSE
+-- reverse the whole list	save, turn, previous up, current up
+-- reverse positions m..n	capture the future tail FIRST
+-- reverse in groups of k	COUNT before reversing
"combining, or imposing order?"	→ MERGE
+-- merge two sorted	attach the remainder in ONE line
+-- sort a list	split (first middle + CUT), sort, merge
+-- merge k sorted	pairwise rounds, or a heap
"do I already hold the node?"	→ DOUBLY LINKED
+-- LRU cache	map answers WHERE, list answers CHEAPLY
+-- browser history, playlists	circular: no first, no last, no None

Six questions. **Every problem in eight days is one or two of them**, and that compression is what you are trying to walk into the room with.

And the three error messages, which between them account for most of your debugging:

```

AttributeError: 'NoneType' object has no attribute 'next'
→ a `.next.next` where only one node was guaranteed. Guard, or use a sentinel.

RecursionError: maximum recursion depth exceeded
→ recursive reverse past ~1000 nodes, OR a split that does not shrink
(second middle, or a missing cut)

(no error at all – the program hangs, or memory fills)
→ a cycle you built by accident, or `while node is not None` on a circular list

```

The third is the dangerous one, because there is nothing to read.

5 The code, built step by step

Two problems, worked the way you would work them in the room: the diagnosis out loud, then the code.

Problem one: “Reorder the list $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ into $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$, in place.”

Diagnose first, out loud.

“Let me say what this is made of before I write anything. I need the second half in reverse order, interleaved with the first half. So: find the middle — that is the runner. Reverse the second half — that is the three-pointer loop. Then weave the two together, one node from each. Three techniques I already have, and no new idea.”

“One thing to settle: which middle. $[1,2,3,4]$ should become $[1,4,2,3]$, so the first half is $[1,2]$ and the second is $[3,4]$ — the **first** middle, and I cut there so the halves are genuinely separate.”

Then the code, in fragments.

```
slow = fast = head
while fast.next is not None and fast.next.next is not None:
    slow, fast = slow.next, fast.next.next
second = slow.next
slow.next = None                                # the cut, in the same breath
```

```
previous = None
while second is not None:
    second.next, previous, second = previous, second, second.next
second = previous                                # the reversed second half
```

The tuple assignment is the four lines compressed; if it makes you nervous under pressure, write them out. Say which you are doing and why.

```
first = head
while second is not None:
    first_next, second_next = first.next, second.next
    first.next = second
    second.next = first_next
    first, second = first_next, second_next
```

Say the loop’s invariant while writing it: the second half is the same length or one shorter, so looping while `second` is not `None` terminates correctly and the last node ends up pointing at `None` on its own.

Then the edge cases, out loud. “Empty and single-node lists return unchanged, so I guard at the top. $[1,2]$ becomes $[1,2]$, which is correct. $[1,2,3]$ becomes $[1,3,2]$.”

Then the complexity. “Three passes, each $O(n)$, so $O(n)$ time and $O(1)$ space — nothing is allocated, only relinked.”

Problem two: “Each node has `next` and an extra `random` pointer to any node or `None`. Make a deep copy.”

Diagnose first.

“The difficulty is that `random` can point *forward* to a node I have not created yet. So I cannot build the copy in one pass unless I have a way to map an original node to its copy.”

“The obvious solution is a hash map from original node to copy: one pass to create all the copies, a second to wire up both pointers. $O(n)$ time, $O(n)$ space. I would say that first because it is clean and correct.”

“There is an $O(1)$ -space trick, and it is worth knowing: interleave each copy directly after its original, so $A \rightarrow A' \rightarrow B \rightarrow B' \rightarrow \dots$. Then the copy of any node is simply `node.next`, which is the map, stored in the list itself. Three passes: interleave, set the random pointers, then unweave.”

The map version.

```
mapping = {None: None}
node = head
while node is not None:
    mapping[node] = Node(node.value)    # create every copy first
    node = node.next

node = head
while node is not None:
    mapping[node].next = mapping[node.next]
    mapping[node].random = mapping[node.random]
    node = node.next
return mapping[head]
```

`mapping[None] = None` at the start removes every “is this pointer null” branch from the second loop — the same trick as a sentinel, applied to a dictionary.

The interleaving version, which is the one that gets remembered:

```
node = head                                # pass 1: A → A' → B → B'
while node is not None:
    copy = Node(node.value)
    copy.next, node.next = node.next, copy
    node = copy.next
```

```
node = head                                # pass 2: the random pointers
while node is not None:
    if node.random is not None:
        node.next.random = node.random.next    # the copy of X is X.next
    node = node.next.next
```

`node.random.next` is the copy of `node.random`. That single expression is the whole trick, and it is worth pointing at rather than letting the interviewer decode it.

```
node, copy_head = head, head.next          # pass 3: unweave, restoring the original
while node is not None:
    copy = node.next
    node.next = copy.next
    copy.next = copy.next.next if copy.next else None
    node = node.next
return copy_head
```

Restoring the original list is mandatory, not optional tidiness. The caller handed you a list to copy, not to mangle, and a version that leaves it interleaved passes every test that only checks the copy.

The reference implementations, for revision

```
class Node:
    __slots__ = ("value", "next", "random")

    def __init__(self, value: int, next: "Node | None" = None) → None:
        self.value = value
        self.next = next
        self.random: "Node | None" = None

def from_values(values: list[int]) → Node | None:
    head: Node | None = None
    for value in reversed(values):
        head = Node(value, head)
    return head

def to_values(head: Node | None) → list[int]:
    out, node = [], head
    while node is not None:
        out.append(node.value)
        node = node.next
    return out

def reverse(head: Node | None) → Node | None:
    """Save, turn, previous up, current up. Return `previous`."""
    previous, current = None, head
    while current is not None:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node
    return previous

def first_middle(head: Node) → Node:
    """The FIRST middle, which is what splitting needs so both halves shrink."""
    slow = fast = head
    while fast.next is not None and fast.next.next is not None:
        slow, fast = slow.next, fast.next.next
    return slow

def reorder(head: Node | None) → Node | None:
    """L0 → Ln → L1 → Ln-1 → ... in place.

    Three techniques: the runner finds the middle, the three-pointer loop
    reverses the second half, and an interleave weaves them together.
    O(n) time, O(1) space – nothing is allocated, only relinked.
    """
    if head is None or head.next is None:
        return head

    middle = first_middle(head)
    second = reverse(middle.next)
    middle.next = None # the cut

    first = head
    while second is not None:
        first_next, second_next = first.next, second.next
        first.next = second
        second.next = first_next
        first, second = first_next, second_next
    return head
```

```

def copy_with_random_map(head: Node | None) → Node | None:
    """Deep copy with an extra `random` pointer. O(n) time, O(n) space.

    The honest baseline: a map from original node to copy, so a forward
    `random` pointer can be resolved in the second pass.
    mapping[None] = None removes every null check from that pass.
    """
    mapping: dict[Node | None, Node | None] = {None: None}
    node = head
    while node is not None:
        mapping[node] = Node(node.value)
        node = node.next

    node = head
    while node is not None:
        mapping[node].next = mapping[node.next]
        mapping[node].random = mapping[node.random]
        node = node.next
    return mapping[head]

def copy_with_random_interleaved(head: Node | None) → Node | None:
    """The same thing in O(1) extra space, by storing the map IN the list.

    Interleave each copy after its original, so the copy of X is X.next.
    Then the random pointers are node.next.random = node.random.next.
    Three passes, and the third RESTORES the caller's list – mandatory.
    """
    if head is None:
        return None

    node = head
    # 1. A → A' → B → B'
    while node is not None:
        copy = Node(node.value)
        copy.next = node.next
        node.next = copy
        node = copy.next

    node = head
    # 2. the copy of X is X.next
    while node is not None:
        if node.random is not None:
            node.next.random = node.random.next
        node = node.next.next

    node = head
    # 3. unweave, restoring the original
    copy_head = head.next
    while node is not None:
        copy = node.next
        node.next = copy.next
        copy.next = copy.next.next if copy.next is not None else None
        node = node.next
    return copy_head

def has_cycle(head: Node | None) → bool:
    """Two runners. Advance first, compare by IDENTITY."""
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False

def merge_two(a: Node | None, b: Node | None) → Node | None:

```

```

"""Builder dummy, `≤` for stability, remainder in one assignment."""
dummy = Node(0)
tail = dummy
while a is not None and b is not None:
    if a.value ≤ b.value:
        tail.next, a = a, a.next
    else:
        tail.next, b = b, b.next
    tail = tail.next
tail.next = a if a is not None else b
return dummy.next

def remove_nth_from_end(head: Node | None, n: int) → Node | None:
    """Fixed gap, both pointers starting at the dummy so `slow` lands on the
    PREDECESSOR and removing the head needs no special case."""
    dummy = Node(0, head)
    fast = slow = dummy
    for _ in range(n):
        if fast.next is None:
            raise ValueError(f"list has fewer than {n} nodes")
        fast = fast.next
    while fast.next is not None:
        fast, slow = fast.next, slow.next
    slow.next = slow.next.next
    return dummy.next

if __name__ == "__main__":
    print(to_values(reorder(from_values([1, 2, 3, 4, 5])))) # [1, 5, 2, 4, 3]
    print(to_values(reorder(from_values([1, 2, 3, 4])))) # [1, 4, 2, 3]
    print(to_values(reorder(from_values([1, 2])))) # [1, 2]
    print(to_values(reorder(from_values([1])))) # [1]
    print(to_values(reorder(None))) # []

    # a list with random pointers: 1 → 2 → 3, randoms 1→3, 2→1, 3→None
    a, b, c = Node(1), Node(2), Node(3)
    a.next, b.next = b, c
    a.random, b.random, c.random = c, a, None

    for copier in (copy_with_random_map, copy_with_random_interleaved):
        copy = copier(a)
        print(to_values(copy),
              copy.random.value, copy.next.random.value, copy.next.next.random)
        assert copy is not a and copy.random is not c # a real deep copy
        assert to_values(a) == [1, 2, 3] and a.random is c # original UNCHANGED
    print("both copies are deep, and the original survived")

    print(to_values(merge_two(from_values([1, 3]), from_values([2, 4])))) # [1,2,3,4]
    print(to_values(remove_nth_from_end(from_values([1, 2, 3, 4, 5]), 2))) # [1,2,3,5]
    print(has_cycle(from_values([1, 2, 3]))) # False

```

6 What it costs

The whole phase, priced

traverse / search	$O(n)$ time, $O(1)$ space
insert or delete at position k	$O(k)$ time, $O(1)$ space
insert or delete GIVEN a node	$O(1)$ singly (after), $O(1)$ doubly (either side)
reverse, iterative	$O(n)$ time, $O(1)$ space
reverse, recursive	$O(n)$ time, $O(n)$ STACK
middle / n th from end	$O(n)$ time, $O(1)$ space, ONE pass
cycle detection	$O(n)$ time, $O(1)$ space
cycle start	$\leq a + c$ then exactly a , so $O(n)$, $O(1)$
merge two sorted	$O(n+m)$ time, $O(1)$ space
merge sort	$O(n \log n)$ time, $O(\log n)$ stack ($O(1)$ bottom-up)
merge k sorted	$O(n \log k)$ time
doubly linked removal	$O(1)$ time, +8 B per node

Every single one is $O(1)$ space except the recursive reversal and the merge sort's stack. That is the shape of the phase: linked-list algorithms move pointers, they do not allocate. If your solution allocates, say what you traded.

The measurements to quote

1. traversal, 1,000,000 elements
python list 0.0621 s | linked list 0.1232 s 2.0x (cache locality)
2. 100,000 insertions at the FRONT
list.insert(0,x) 1.443 s | node push_front 0.0246 s ~59x
3. memory per element
list slot 8 B | Node with __slots__ 48 B | without __slots__ 336 B
4. LRU cache read at capacity 100,000
doubly linked ~6 pointer writes | singly linked ~100,000 steps ~10⁴ x

One real measurement beats three complexity classes. Number 2 is the linked list's only unambiguous win; number 1 is the honest cost.

Space, side by side

in-place pointer surgery	$O(1)$	– the default, and what interviews want
hash map of nodes	$O(n)$	– the baseline for cycles and for deep copy
copy values into a list	$O(n)$	– legitimate, but say what you traded
recursion	$O(n)$	– and RecursionError past ~1000 in Python

The complexity claims that get probed

- “Insertion in a linked list is $O(1)$.” → only given the node. Given a position, both structures are $O(n)$.
- “Your sort is $O(n \log n)$.” → every level does n work, and there are $\log_2 n$ levels, about 20 at a million nodes.
- “Floyd’s is $O(n)$.” → at most $a + c$ for phase one and exactly a for phase two, so at most $2n$.

- “One pass is faster than two.” → **same asymptotics and about the same pointer moves**; what it buys is one traversal instead of two, which matters when each node is a fresh cache miss.

7 The traps

The complete list for the phase. Each one has cost somebody an interview.

Overwriting a pointer before saving what it pointed at

The defining mistake. `current.next = previous` before `next_node = current.next`, and the rest of the list is unreachable. `[1,2,3]` reverses to `[1]`, with no error.

Returning `head` instead of `previous` or `dummy.next`

After a reversal, `head` is the last node. After a removal, `head` may be a deleted node. Both return a wrong answer silently.

Rebinding a parameter and expecting the caller to see it

```
push_front(head, 5)           # return value discarded – nothing happens
```

Python passes the reference by value. Any function that can change the first node must **return the new head**, and the caller must assign it.

`.next.next` with only one node guaranteed

```
AttributeError: 'NoneType' object has no attribute 'next'
```

Every double dereference is a promise that two nodes exist. Guard it, and put the null check first because `and` short-circuits.

The head as a special case, handled with an `if` instead of a dummy

On `[7,7,7,1]` the head is a *run*, so an `if` is not enough — and then everything-deleted is a second branch. A dummy takes the count to zero.

Advancing after a deletion

`[1,7,7,2]` becomes `[1,7,2]`. After unlinking, `previous.next` is a node you have not inspected. **Advance only in the `else`**.

The wrong middle when splitting

The second middle leaves `[1,2]` as `[1,2]` and `[]`, so a merge sort never terminates. **Splitting needs the first middle.**

Forgetting the cut

`slow.next = None` . Without it both halves are the whole list. Same `RecursionError` , different cause.

The off-by-one in the runner's expiry or gap

`window[0] ≤ index - k` , derived not memorised. And the opening walk must check for a short list, or it dereferences `None` .

Returning the meeting point as the cycle's start

It is only right when `a = 0` — a list that is entirely one cycle, which is exactly the case people test with.

Comparing nodes with `==` instead of `is`

Fine until someone defines `__eq__` on the value, at which point two distinct nodes holding equal values report a cycle that does not exist.

`while node is not None` on a circular list

There is no `None` . The program hangs with no error.

Updating only one direction in a doubly linked list

Correct forwards, broken backwards. Every forward test passes and it falls apart later, elsewhere. Write the both-ways `_check` .

Not terminating a list built from existing nodes

`more.next = None` in partition, `tail.next = None` after a heap merge. Without it you build a cycle and printing never returns.

The heap in “merge k lists” with no tie-breaker

```
TypeError: '<' not supported between instances of 'Node' and 'Node'
```

Fires the moment two heads are equal. Add an index to the tuple.

Mutating the caller's list in a read-only operation

The palindrome check that leaves the second half reversed; the deep copy that leaves the list interleaved. Both return the right answer and hand back damaged data.

Recursion in Python

```
RecursionError: maximum recursion depth exceeded
```

at about a thousand nodes. Not an edge case — an ordinary input. And raising the limit turns a clean exception into a segmentation fault.

8 In the interview

How it gets asked

- The plain mock: *“Two problems, forty-five minutes, talk as you go.”*
- The composite, which is the standard hard linked-list question: *“Reorder the list”, “is it a palindrome, in $O(1)$ space”, “copy a list with random pointers”, “reverse in groups of k ”.*
- The design question wearing a list costume: *“design an LRU cache”, “design browser history”.*
- The reasoning probe: *“why a linked list rather than an array?”* — and the answer must contain the words “given the node”.
- The trap: *“just use `sorted()`”* — say what you traded.

The script, minute by minute, for a 45-minute round

Minutes 0–3 — restate and clarify. Say the problem back. Ask the three things that always matter here: can the list be empty; can the first node change; and may I modify the input list. That third question is specific to this phase and asking it is a strong signal.

Minutes 3–6 — diagnose out loud. Ismail’s two questions. “This is the runner to find the middle, then a reversal, then an interleave.” Name the pieces before writing any of them. If you cannot name them, say what the baseline $O(n)$ -space solution is and improve from there.

Minutes 6–10 — draw the pointers. Two or three boxes and arrows, and what each arrow should point at afterwards. Every bug in this phase is an assignment made before its picture existed.

Minutes 10–25 — code, narrating. Write the dummy and its `return dummy.next` together. Say “save first” as you write the save. Say “advance only when I keep” as you write the `else`.

Minutes 25–30 — trace the degenerate case out loud. Empty, one node, two nodes. Finding your own off-by-one reads far better than having it pointed out.

Minutes 30–35 — complexity, with the counting, and say the space explicitly, including whether you mutated the input.

Minutes 35–45 — the second problem, faster, because the diagnosis is now warm.

The follow-ups

“Why a linked list rather than an array?” “Only for $O(1)$ insertion and deletion **given the node** — and that qualifier is the whole answer. If I have a position rather than a node, both are $O(n)$: the array pays for the shift and the list pays for the walk. The genuine wins are when something else hands me the node, like a hash map in an LRU cache; when I only ever work at the front, where a hundred thousand front insertions were 1.4 seconds against 0.02; and when the structure itself is the point, since a tree is a node with two links. Otherwise an array wins on memory — 8 bytes a slot against 48 — and on traversal, about twice as fast, because of cache locality.”

“Can you do it in $O(1)$ space?” “Usually yes, and that is what these problems are for. The pattern is: replace a hash map of nodes with information stored *in* the list. Cycle detection replaces a set of visited nodes with two runners. Copying a list with random pointers replaces the map with interleaving each copy after its original, so the copy of X is $X.next$. Reversal is already $O(1)$ if it is iterative. The one place I cannot is a recursive solution, which is $O(n)$ stack by definition.”

“May I modify the input?” “That is the question I would ask *you*, and it changes the answer. If yes, the palindrome check can reverse the second half in place and be $O(1)$ space. If no, I would either copy the values into an array — $O(n)$ space, and I would say so — or reverse and then reverse back, restoring the list before returning. Silently handing back a mangled list is the bug in this phase that no test catches.”

“Is that not quadratic?” “No, and let me count rather than assert. In a merge sort, every level does n work and there are $\log_2 n$ levels. In Floyd’s, phase one is at most $a + c$ and phase two is exactly a , so at most $2n$. In a merge, every node is attached exactly once. The one place it *is* quadratic is appending to a list with no tail reference — n appends is $n(n+1)/2$ steps, about five billion at a hundred thousand — which is the classic accidental quadratic here.”

“What is the most common bug in your code?” A good answer to have ready: “Assignment order. Overwriting `current.next` before saving what it pointed at, or attaching a new node to a neighbour before the new node knows both its neighbours. Both lose part of the list and neither raises. So I write the save first, and I draw the two boxes before I type the assignments.”

A model answer

Asked: *reorder the list so it goes first, last, second, second-last, and so on — in place.*

“Before writing anything, let me say what this is made of, because it is three things I already have rather than one new idea.

I need the second half of the list, reversed, interleaved with the first half. So: find the middle with two runners, one moving one step and one moving two. Reverse the second half with the three-pointer loop. Then weave them together, one node from each. Nothing gets allocated — it is all relinking — so this should be $O(n)$ time and $O(1)$ space.

One decision to settle first: which middle. On `[1,2,3,4]` the answer is `[1,4,2,3]`, so the halves are `[1,2]` and `[3,4]` — that is the **first** middle, the one where the loop condition looks two nodes ahead. And I cut immediately: set the first half's last node's `next` to `None`, in the same breath as finding it, or the two halves are still one list and the interleave will loop.

The reversal is the standard four lines: save the next node, turn this arrow round, move `previous` up, move `current` up. Save first, always — the moment I overwrite `current.next` I have lost my only reference to the rest, and that failure produces no error, just a shorter list.

The interleave is four assignments per step. I hold `first_next` and `second_next` before touching anything, because both are about to be overwritten. Then `first` points at `second`, `second` points at `first_next`, and both cursors advance. The second half is the same length or one shorter, so looping while `second` is not `None` terminates correctly, and the final node ends up pointing at `None` without a special case.

Edge cases before I say I am done: empty and one node return unchanged, which I guard at the top. `[1,2]` should come out as `[1,2]`, and it does. `[1,2,3]` becomes `[1,3,2]`.

Complexity: three passes over the list, each $O(n)$, so $O(n)$ time. $O(1)$ space — three or four references, and nothing allocated.

One thing I would confirm with you: this modifies the caller's list in place, which is what 'in place' asks for. If it had been a read-only question I would either work on a copy or restore the list before returning, because handing back a rearranged list that the caller did not expect is the kind of bug that passes every test.”

9 Recall card

- **Six trigger questions cover eight days.** *Can the first node change?* → **dummy head**. *A position I cannot count to, in one pass?* → **runner** (speeds for the middle, a gap for the end). *Might it not end?* → **Floyd's**. *Turning the arrows?* → **three-pointer reverse**. *Combining or ordering?* → **merge**. *Do I already hold the node?* → **doubly linked**. Diagnose first, then repair.
- **The five sentences.** *Stand at $k-1$.* · *Save the next node before you overwrite the pointer.* · *Return `dummy.next`, never `head` — and a bare function must return the new head.* · *Insertion is $O(1)$ given the node.* · *Advance only when you keep.*
- **Composites are the real questions, and they are two or three techniques each.** Reorder = runner + reverse + interleave. Palindrome in $O(1)$ = runner + reverse + compare + **reverse back**. Copy with random pointers = a node map, or **interleave so the copy of X is `X.next`** and then unweave to **restore the caller's list**. Sort = split (first middle + **cut**) + merge.
- **Almost everything here is $O(1)$ space — if yours allocates, say what you traded.** The exceptions are the recursive reverse (**$O(n)$ stack**, `RecursionError` at ~ 1000) and merge sort's $O(\log n)$. The measurements: traversal **$2\times$ slower** than a list, **$\sim 59\times$ faster** at front insertion, **48 B vs 8 B** per element, and the LRU read **$\sim 10^4\times$ worse** with a singly linked list.
- **The traps that end rounds:** overwriting before saving · `return head` · discarding the returned head · `.next.next` unguarded · the head as an `if` instead of a dummy · advancing after a deletion · the **second** middle when splitting · the missing **cut** · returning the meeting point as the cycle start · `=` instead of `is` · `while node is not None` on a circular list · one direction updated · an unterminated built list · a heap with no tie-breaker · and **mutating the caller's list in a read-only operation**.

1 What this is, and why they ask it

BookMyShow lists films, cinemas and showtimes, draws a seat map, and lets you book seats. The catalogue half is straightforward. The prompt names the half that is not: **two users click seat H7 for the same show at the same instant, and exactly one of them must get it.**

Two ideas carry the design.

A seat is two things. H7 is a physical seat in screen 3, and it is also “H7 for the 9:30 show on Friday” — which is available or not, independently of H7 at every other show. Those are different objects, and merging them makes the whole problem unexpressible. It is the same title-versus-instance split as the library’s `Book` and `BookItem` on [day 082](#).

Selecting a seat is not booking it. Between choosing a seat and paying there is a gap of minutes, and during that gap the seat must be held for you and must not be held for ever. That is a **lock with an expiry**, and getting its two halves right — the claim being atomic, and the expiry being automatic — is the whole interview.

They ask it because everyone has used it, because the race is easy to state and easy to get subtly wrong, and because the follow-ups are unusually good: what if the payment fails, what if the user closes the tab, what happens when a hundred thousand people open the same seat map in the same second.

2 The story

The Alankar has been running four shows a day since 1978, and until about ten years ago the whole thing was managed on one board behind the counter.

It is a big black board with two hundred and forty little squares chalked on it — twelve rows of twenty — and it is wiped and redrawn between every show. When somebody buys a seat, the man at the counter puts a cross in that square.

There are two windows, because on a Friday one window is not enough. And for years those two windows shared that one board, which worked until it did not.

What went wrong was always the same thing, maybe once a month. Two people, one at each window, ask for the same two seats at almost the same moment. Both clerks look at the board. Both squares are empty. Both clerks take the money, and both put a cross in

the same square, and the second cross goes on top of the first and looks exactly like one cross. Nobody finds out until eight o'clock, when two families are standing in row H arguing in front of everybody.

Ratnamma, who has run the place since her husband died, fixed it with a duster.

There is one duster. Whoever is holding it may mark the board, and the other one waits. It takes four seconds and it has not happened since.

The second thing took her longer to see. People would ask for seats and then say wait, I will get the money from the car, and go. The clerk would leave those squares empty, because the sale was not done, and somebody else would buy them, and then the first man would come back with his money and there would be a row about it.

So now the clerk draws a small circle instead of a cross. A circle means somebody is coming back. And the rule — this is the part Ratnamma is strict about — is that a circle gets rubbed out at the interval bell of the previous show. If he is not back by then, the seat is empty again and anybody may have it.

She says people always understand the circle. What they will not accept is being told a seat is gone when nobody is sitting in it.

3 The idea in plain English

The duster is the atomic claim. The circle is the lock. The interval bell is the expiry. Ratnamma solved this in the 1990s and the design has not changed.

A seat is two things

- **Seat** — a physical seat in a screen. Row H, number 7, screen 3, seat type (recliner, regular), and that is all. It exists whether or not there is a show.
- **ShowSeat** — that seat, for that show. It has a **status** — available, locked, booked — and a price, because the same seat costs more on a Saturday night.

One **Seat** produces one **ShowSeat** per show. A 200-seat screen with 5 shows a day produces 1,000 **ShowSeat** rows a day.

The payoff, in one sentence: **the seat map you draw is a **ShowSeat** query, and the thing you lock is a **ShowSeat** row.** If you tried to lock a **Seat**, you would block that seat for every show in the building.

Selecting is not booking

The naive flow is: user picks seats, user pays, seats are marked booked. It has an obvious hole — the seats are unprotected during the payment, which takes minutes — and the fix is a middle state:

```
AVAILABLE --select→ LOCKED (with an expiry) --payment ok→ BOOKED
                        |
                        +--payment fails, or the expiry passes→ AVAILABLE
```

Three states, and both arrows out of `LOCKED` are required. Ratnamma's circle, and the interval bell.

The expiry is not a nicety. Without it, every abandoned checkout removes seats from sale for ever, and on a busy Friday that is most of the auditorium. **A lock with no expiry is a leak.**

The race, stated precisely

Two users, one seat. Both browsers loaded the seat map ten seconds ago; both show H7 as available. Both click. Both requests arrive at the server within a millisecond of each other.

The mistake is to check and then act:

```
seat = db.get(show_id, "H7")
if seat.status == "AVAILABLE":      # both requests see AVAILABLE here
    seat.status = "LOCKED"          # and both write it
    db.save(seat)
```

Both succeed. Two people own H7. **Check-then-act is not atomic**, and it is the same shape as the parking spot on [day 078](#) and the shipped order on [day 073](#).

The fix is one statement that checks *and* claims:

```
UPDATE show_seats
SET status = 'LOCKED', locked_by = :user, locked_until = :now_plus_10_min
WHERE show_id = :show
AND seat_id IN (:seats)
AND (status = 'AVAILABLE'
     OR (status = 'LOCKED' AND locked_until < :now)); -- expired locks count as free
```

Then **check the affected row count**. If you asked for four seats and it reports four, you have them. If it reports three, somebody took one and you must roll back and tell the user. That row count is the whole mechanism.

All four seats, or none

A family booking four seats must get four or nothing. Three seats plus an apology is a worse product and, worse, two users each holding a partial set can block each other indefinitely.

Two rules make it safe:

One statement, one transaction. The `UPDATE ... WHERE seat_id IN (...)` above claims all of them or you roll back. Not four separate updates.

A fixed order, if you must take row locks. If the implementation ends up locking rows one at a time — `SELECT ... FOR UPDATE` — always order by seat id. Two transactions taking the same rows in opposite orders is a textbook deadlock, and the database will resolve it by killing one of them, which the user sees as a random failure.

Expiry: lazy, not scheduled

There are two ways to make locks expire.

A background job that sweeps expired locks back to available every minute. It works, and it means correctness depends on a scheduler running — if it stops, seats silently stay locked.

Lazily, by treating an expired lock as available *in the query itself* — the `OR (status = 'LOCKED' AND locked_until < now)` clause above. Nothing has to run. A lock that expired an hour ago is just as claimable as one that expired a second ago.

Prefer lazy. The sweeper then becomes an optimisation for the seat *map* — so the display does not show stale locks — rather than something correctness depends on. Saying which one is load-bearing is the kind of distinction that reads as experience.

Where the lock lives: the database or Redis

Both are defensible and the trade is worth stating.

In the database, as a status column on `ShowSeat`. One source of truth, the lock and the booking are in the same transaction, and nothing can drift. Slower — every claim is a write to the primary — and hot rows for a blockbuster show all sit in one place.

In Redis, as `SET show:123:H7 user42 NX EX 600` — set if not exists, expiring in ten minutes. Extremely fast, and expiry is free. But now there are two systems, and a Redis failure or a lock-expiring-mid-payment leaves them disagreeing: Redis says free, the database says booked, or the reverse.

Use the database for the seat state, and Redis only if you measure that you need it — and if you do, the database remains the authority and the final booking is still a conditional update. The sentence to say: *the lock may live in Redis, but the truth must live where the booking lives.*

Payment, and the two failures that matter

```
lock 4 seats (10 minute expiry)
→ user pays
  → payment succeeds → UPDATE ... SET status='BOOKED' WHERE locked_by = :user
  → payment fails    → release: back to AVAILABLE
  → payment times out → do nothing; the lock expires on its own
```

Two subtleties.

The confirming update must also be conditional — `WHERE status = 'LOCKED' AND Locked_by = :user`. Otherwise a payment that arrives after the lock expired, and after somebody else booked the seat, overwrites their booking. That is the worst failure in the system: a customer with a valid ticket loses their seat.

If the payment succeeds and the lock has expired, you owe a refund, and it must be automatic. This is the ATM's problem from [day 080](#) in a different costume: the money moved and the goods did not, so reverse it. Keeping the lock window comfortably longer than the payment timeout makes it rare; it does not make it impossible.

4 The picture

The two ideas of “a seat”, drawn:

Seat (physical, in a screen)	ShowSeat (that seat, for that show)
+-----+	+-----+
screen 3, row H, no. 7 ----->	show 9:30 Fri AVAILABLE ₹250
type: RECLINER	+-----+
+-----+	+----> show 6:30 Fri BOOKED ₹250
	+-----+
	+----> show 9:30 Sat LOCKED ₹350
	+-----+

the seat map you DRAW is a ShowSeat query
the thing you LOCK is a ShowSeat row
locking the Seat would block it for every show in the building

The state machine for one `ShowSeat`, which is the design:

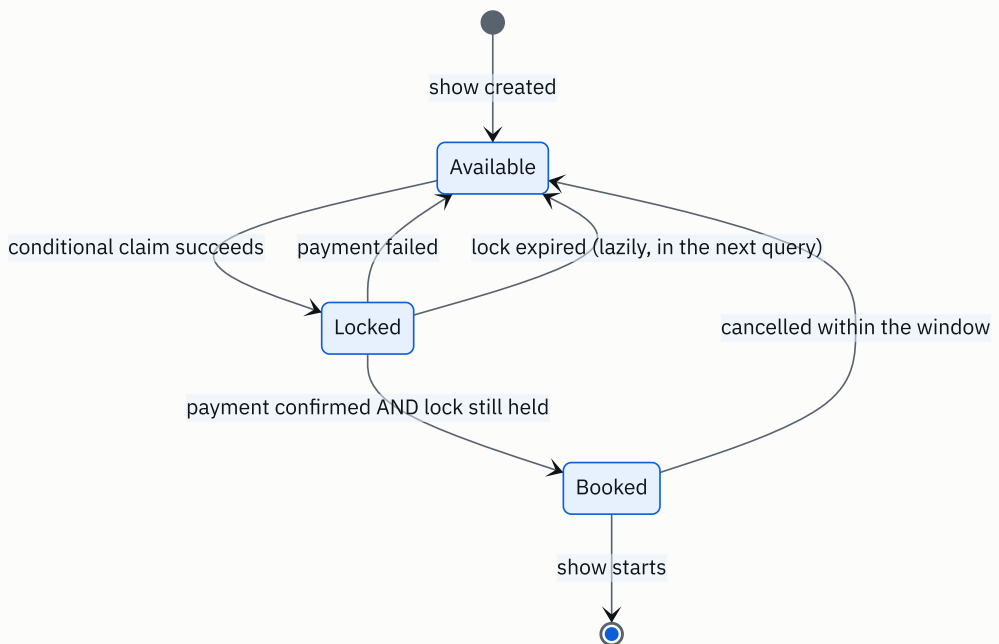


FIGURE 86.1

What to notice: **two different arrows go from Locked back to Available**, and only one of them is an action anybody takes. The other happens by the passage of time and is evaluated lazily, which means nothing has to be running for it to be correct.

The race, and the fix:

WRONG – check, then act

user A	database	user B
SELECT H7	-----> AVAILABLE	
		<----- SELECT H7
	AVAILABLE	----->
UPDATE → LOCKED (A)	→	
		<----- UPDATE → LOCKED (B)
	LOCKED by B	both users think they hold H7

RIGHT – one statement that checks and claims

user A	database	user B
UPDATE ... WHERE status='AVAILABLE' →	1 row (A wins)	
	<----- UPDATE ... WHERE status='AVAILABLE'	
	0 rows (B loses, and KNOWS it)	

the affected row count IS the answer

And the whole booking flow:

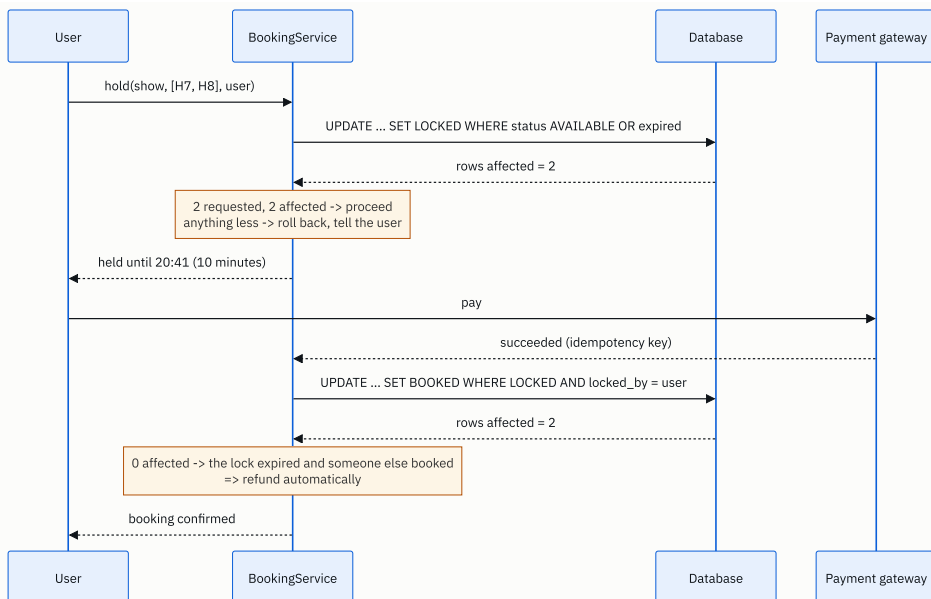


FIGURE 86.2

5 How it actually works

Move 1 • Clarify (minutes 0–5)

“**Just the booking flow, or the catalogue and search too?**” — Focus on booking; the catalogue is ordinary CRUD. “**Can a user book several seats in one go?**” — Yes, and it must be all-or-nothing. “**How long do we hold seats during payment?**” — Ask, and propose ten minutes. This number is the whole design’s main tuning knob. “**Is cancellation allowed?**” — Ask; it adds a transition back from `BOOKED` .

“I will assume one seat is one ticket, that prices are per show-and-seat-type, and that payment is an external gateway behind an interface. I am not designing search, recommendations, or the seat-map rendering.”

Move 2 • The nouns (minutes 5–12)

- `Cinema` , `Screen` — a building and an auditorium.
- `Seat` — physical: screen, row, number, type. No status.
- `Show` — a film, on a screen, at a time.
- `ShowSeat` — the seat for that show: status, price, `locked_by` , `locked_until` . **This is the row everything happens to.**

- **Booking** — a user, a show, a set of seats, an amount, a state.
- **Payment** — an attempt with an idempotency key and an outcome.
- **BookingService** — hold, confirm, release. Holds no rules beyond sequencing.
- **PricingPolicy** (*interface*) — seat type $\times$ show time $\rightarrow$ amount. Weekend surge and matinee discounts are real second implementations.

Eight, one interface. Note what is absent: no **SeatLock** class, because the lock is a *state* of a **ShowSeat** rather than a separate object — and inventing a lock entity is how people end up with two sources of truth.

Move 3 • The claim, which is the whole design

```
def hold(self, show_id: str, seat_ids: list[str], user_id: str, now: datetime) → Hold:
    """Claim seats atomically. Either all of them, or none.

    The WHERE clause both checks and claims, so there is no window between
    them. The affected row count is the answer: fewer rows than requested
    means somebody else won a seat, and we roll back the whole thing.
    """
    expires = now + self.HOLD_DURATION # 10 minutes
    with self._db.transaction():
        affected = self._db.execute(
            """
            UPDATE show_seats
            SET status = 'LOCKED', locked_by = %s, locked_until = %s
            WHERE show_id = %s
              AND seat_id = ANY(%s)
              AND (status = 'AVAILABLE'
                   OR (status = 'LOCKED' AND locked_until < %s))
            """
            ,
            user_id, expires, show_id, seat_ids, now,
        )
        if affected != len(seat_ids):
            raise SeatsUnavailable(f"{len(seat_ids) - affected} of your seats were just taken")
        return Hold(show_id, seat_ids, user_id, expires)
```

Three things to narrate while writing this, because each is a decision:

One statement, not a loop. Four separate updates can partially succeed and leave two users each holding two seats of a four-seat block, blocking each other until both expire.

Expired locks are claimable in the **WHERE clause.** No sweeper is required for correctness.

The exception rolls the transaction back, so a partial claim never persists. The user sees “those seats have just gone”, which is the honest message.

Move 4 • Confirming, and the conditional that prevents the worst bug

```
def confirm(self, hold: Hold, payment: Payment, now: datetime) → Booking:
    with self._db.transaction():
        affected = self._db.execute(
            """
            UPDATE show_seats
              SET status = 'BOOKED', booking_id = %s
             WHERE show_id = %s AND seat_id = ANY(%s)
             AND status = 'LOCKED' AND locked_by = %s
            """,
            payment.booking_id, hold.show_id, hold.seat_ids, hold.user_id,
        )
        if affected ≠ len(hold.seat_ids):
            self._payments.refund(payment, reason="hold expired before confirmation")
            raise HoldExpired("your hold expired; you have been refunded")
        ...
```

`AND status = 'LOCKED' AND locked_by = user` is the line that stops the worst failure in the whole system: a payment that arrives late, after the lock expired and somebody else booked, must not overwrite their booking. Without that clause, a customer with a valid ticket silently loses their seat, and they find out in row H at eight o'clock.

And when it does fire, the refund is **automatic**, not a support ticket — the same money-moved-goods-did-not shape as the ATM.

Move 5 • Reading the seat map

```
def seat_map(self, show_id: str, now: datetime) → list[SeatView]:
    """Expired locks are displayed as available, because they ARE available."""
    return self._db.query(
        """
        SELECT seat_id, seat_type, price_paise,
               CASE WHEN status = 'BOOKED' THEN 'BOOKED'
                    WHEN status = 'LOCKED' AND locked_until ≥ %s THEN 'LOCKED'
                    ELSE 'AVAILABLE' END AS display_status
        FROM show_seats WHERE show_id = %s
        """,
        now, show_id,
    )
```

The same lazy-expiry rule applied to reads, so the display and the claim agree. And this query is **cacheable for a few seconds** — the map is read enormously more often than it is written, and a five-second stale map is fine because the claim is authoritative anyway. **Being wrong on the display is cheap; being wrong on the claim is not.** That asymmetry is what makes this system scalable.

Real systems

- **BookMyShow, Ticketmaster and airline seat maps** all work this way: a temporary hold with a visible countdown, then payment, then confirmation. The visible timer is deliberate — it makes the expiry a feature the user understands rather than a surprise.

- `SELECT ... FOR UPDATE` is the row-lock version, and it is why lock ordering matters: two transactions taking the same rows in different orders deadlock, and Postgres resolves it by killing one with `deadlock detected`.
- **Redis** `SET key value NX EX seconds` is the distributed-lock primitive, and **Redlock** is the multi-node version — which its own author notes is not safe for correctness-critical locks without fencing tokens. Worth naming, and worth not depending on for money.
- **Idempotency keys** on the payment, exactly as in the ATM: the gateway must recognise a retry rather than charging twice.
- **Virtual waiting rooms** — the queue page you see for a big concert — exist because the seat map itself falls over when a hundred thousand people arrive at once. That is an admission-control answer, not a locking answer.

6 The numbers

The size of the inventory

```

cinemas                8,000
screens per cinema      3      → 24,000 screens
seats per screen        200    → 4.8 M physical seats
shows per screen/day    5      → 120,000 shows/day
ShowSeat rows/day = 120,000 × 200 = 24,000,000 rows per day

```

```

ShowSeat row: show_id 16 B + seat_id 8 B + status 1 B + price 4 B
               + locked_by 16 B + locked_until 8 B + overhead ≈ 80 B
24 M × 80 B = 1.9 GB per day of seat rows
kept for 90 days: ~170 GB

```

Twenty-four million rows a day, which is why `ShowSeat` rows are generated per show and archived after the show — not kept for ever. That is a real design conclusion from arithmetic: the physical seats are five million and permanent; the show-seats are twenty-four million a day and disposable.

The traffic, and where it actually hurts

```

average: 500,000 bookings/day = ~6 bookings/second
seat-map views: 30 views per booking = 180 views/second

blockbuster opening, first minute:
  100,000 users on ONE show's seat map
  → 100,000 reads/second on ~200 rows
  → and perhaps 3,000 claim attempts/second on those same rows

```

The asymmetry is the whole scaling story:

```
reads: 100,000/s, on data that may be a few seconds stale    → CACHE
writes: 3,000/s, on data that must be exactly right          → the database
```

Reads outnumber writes by about thirty to one normally and thirty thousand to one at a peak. So cache the seat map aggressively with a short TTL and never cache the claim. A stale map shows a seat as free that has just gone, and the user's claim fails with an honest message — which is exactly the failure you can afford.

The race window, quantified

```
without a conditional claim:
time between SELECT and UPDATE in application code ≈ 2 ms
3,000 claim attempts/second on one hot show
→ collisions per second ≈ 3,000 × 3,000 × 0.002 / 200 seats ≈ 90/second
```

Ninety double-sold seats a second on an opening night. Not a theoretical risk — a catastrophe. With the conditional update the window is zero, because the check and the claim are the same statement.

The hold duration, which is the main tuning knob

```
hold = 10 minutes, abandonment rate ≈ 20%
on a hot show: 3,000 holds in the first minute, 600 abandoned
→ 600 seats unsellable for 10 minutes

hold = 5 minutes: seats return twice as fast
                  but users on slow connections lose their seats mid-payment
hold = 15 minutes: fewer failed payments, ~50% more dead inventory at peak
```

Too long and the auditorium is full of nobody; too short and paying customers lose seats. Ten minutes is the industry norm because it comfortably exceeds a card-plus-OTP flow, which is typically 60–120 seconds. Say the number *and* say what it is trading.

Storage and cost of the rest

```
Booking row ~200 B × 500,000/day = 100 MB/day = 36 GB/year (keep for ever)
Payment row ~150 B × 500,000/day = 27 GB/year
seat map cache: 200 rows × 80 B = 16 KB per show
120,000 shows × 16 KB ≈ 2 GB — the entire day's seat maps fit in one Redis node
```

That last line is worth saying: **every seat map for every show in the country fits in two gigabytes of cache.** It makes the read-scaling answer obvious rather than hand-waved.

7 The trade-offs

What this design gives up

A hold is dead inventory. Every held seat is a seat nobody can buy, and 20 percent of holds are abandoned. At peak that is hundreds of seats unsellable for ten minutes on exactly the show where demand is highest. Shortening the window recovers them and costs paying customers their seats mid-OTP. There is no setting that is right for everybody, which is why the timer is shown to the user — it turns a constraint into an expectation.

A stale seat map means avoidable failures. Caching for five seconds means a user can click a seat that went two seconds ago and get an error. That is a deliberate trade: the alternative is 100,000 uncached reads a second against the primary during exactly the minute it can least afford it. The mitigation is a good message — “those seats have just gone, here are the nearest available” — rather than a stale-free map.

Everything hot is in one place. All the contention for a blockbuster is on 200 rows of one show, and no amount of sharding by show helps, because it is *one* show. The honest answers are: keep the transaction as short as possible, put the whole claim in one state-machine, and use admission control — a virtual waiting room — when demand exceeds what the rows can serve. Sharding is not the answer here and claiming it is would be wrong.

A lock in Redis is faster and adds a second source of truth. If Redis and the database disagree you can double-sell or dead-lock inventory, and the failure is hard to reason about. Keeping the state in the database is slower and always consistent, and I would start there and move only with a measurement.

Cancellation reopens every question. A cancelled seat must return to `AVAILABLE`, which means the seat map changes after the show is “sold out”, refunds have their own state machine, and partial cancellation of a four-seat booking needs a rule. It is a whole second design and I would scope it explicitly.

Nothing here handles seat *selection* quality. Real systems will not let you leave a single orphaned seat between two bookings, because a lone seat rarely sells. That is a placement rule on top of availability, and it interacts with locking in ways worth mentioning and not building in forty minutes.

“I would change this design if...”

- **...one show can attract a hundred thousand simultaneous users.** Then a virtual waiting room in front of the seat map, so the number of people who can attempt a claim is controlled rather than hoped about.

- **...holds must survive a database failover instantly.** Then Redis with fencing tokens, and the database still authoritative for the booking.
- **...there are unreserved or general-admission events.** Then there are no seats to lock at all — just a counter, decremented conditionally, which is a much simpler problem and worth noticing.
- **...seats have adjacency rules.** Then holding is not per-seat but per-block, and the claim has to reason about what it leaves behind.

The honest concession

Almost every difficulty in this system comes from one gap: the minutes between choosing a seat and paying for it. Remove that gap — pay instantly, or hold nothing — and the design collapses to a conditional update and nothing else. The lock, the expiry, the refund path, the dead inventory and the stale map are all consequences of the fact that a human being needs two minutes and a one-time password. That is worth saying out loud, because it frames every trade-off as a variation on the same question: *how long are we willing to hold a seat for somebody who may not come back?*

8 In the interview

How it gets asked

- The standard: *“Design BookMyShow.”* Then within five minutes: *“Two users click the same seat at the same moment.”*
- The modelling probe: *“Where does the seat’s availability live?”* — they are checking for the `Seat` / `ShowSeat` split.
- The failure probe: *“The user pays and the hold has expired. Now what?”*
- The scale probe: *“A blockbuster opens and a hundred thousand people load the same seat map.”*
- The distributed probe: *“Would you use Redis for the lock?”* — a question about trade-offs, not about Redis.

The timed script

Minutes 0–5 · Clarify. Booking only? Multiple seats, all-or-nothing? How long is the hold? Is cancellation in scope? Propose ten minutes and say it is the main tuning knob.

Minutes 5–10 · The modelling split, early. “A seat is two things: the physical seat in the screen, and that seat for that show. Availability belongs to the second. Locking the first would block it for every show in the building.”

Minutes 10–15 · Estimation. 24 million ShowSeat rows a day, and reads outnumbering writes thirty to one. Say the conclusion: *rows are per-show and disposable, and the seat map is cacheable while the claim is not.*

Minutes 15–25 · The deep dive: the lock. The three states, both arrows out of `LOCKED`, the conditional update as one statement, the affected-row-count check, all-or-nothing, and lazy expiry.

Minutes 25–33 · The payment path, including the conditional confirm and the automatic refund when the hold expired.

Minutes 33–40 · Scale and failure. The blockbuster case, cache the map and never the claim, the virtual waiting room, and the honest note that sharding does not help when the contention is one show.

The follow-ups

“Two users click the same seat at the same moment.” “Exactly one gets it, and the mechanism is that the check and the claim have to be the same statement. The mistake is to read the seat, see it is available, and then write — both requests see available and both write, so both users own it. Instead: one `UPDATE` that sets the status to locked `WHERE` the status is still available, and then check the affected row count. One row means I won; zero means I lost and I tell the user honestly. Without that, at three thousand claim attempts a second on a hot show and a two-millisecond gap between the read and the write, I make it roughly ninety double-sold seats per second on an opening night.”

“A user wants four seats. What if only three are free?” “All or nothing. One `UPDATE` with `seat_id IN (...)` and a check that the affected count equals four — anything less rolls the transaction back and the user is told the seats have just gone. Four separate updates would be the bug: two users each end up holding two seats of the same block, and neither can complete until both expire. And if I ever ended up taking row locks one at a time, I would order them by seat id, because two transactions taking the same rows in different orders is a textbook deadlock.”

“How do the holds expire?” “Lazily, and that is a deliberate choice. Each locked row carries a `locked_until` timestamp, and every claim query treats an expired lock as available in its `WHERE` clause. That means correctness does not depend on any scheduler running — if a sweeper job dies, nothing breaks. I would still run a sweeper, but only so the *displayed* map is not misleading, which is a cosmetic concern rather than a correctness one. Ten minutes is the usual window, because a card-plus-OTP flow is one to two minutes and you want comfortable headroom.”

“The payment succeeds but the hold has expired.” “Two things must happen. First, the confirming update has to be conditional — set booked `WHERE` the status is still locked *and* locked by this user. Without that clause, a late payment overwrites whoever

booked the seat in the meantime, and a customer with a valid ticket silently loses their seat. That is the worst failure in the system. Second, when the confirm affects zero rows, I refund automatically — the money moved and the goods did not, exactly like a failed ATM dispense, and it has to be a pipeline and not a support ticket. Making the hold window comfortably longer than the payment timeout makes this rare, not impossible.”

“A hundred thousand people load the same seat map in one second.” “Two different problems, and they need opposite answers. The reads — a hundred thousand a second on two hundred rows — go to a cache with a short TTL, a few seconds, and I accept that a user may click a seat that has just gone, because the claim is authoritative and the failure message is cheap. The writes cannot be cached at all, and they are all on the same two hundred rows, so sharding does not help — there is one show. What actually helps is keeping the transaction as short as possible, doing the claim in one statement, and admission control: a virtual waiting room, which is what every ticketing site does for a big release. It is worth noting the whole country’s seat maps for a day are about two gigabytes, so caching them is easy.”

“Would you use Redis for the lock?” “I would start with the database, because then the lock and the booking are in the same transaction and cannot disagree. Redis is genuinely faster and gets expiry for free with `SET NX EX`, and I would move if I measured the need — but the sentence I would hold onto is that *the lock may live in Redis, but the truth must live where the booking lives*. With two systems, a Redis failover or a lock expiring mid-payment gives you a state where one says free and the other says booked, and resolving that is harder than the performance problem I was solving. And Redlock specifically is not something I would depend on for money without fencing tokens.”

“Where does the seat’s availability live?” “On the `ShowSeat`, not on the `Seat`. A seat is a physical thing in a screen — row H, number 7, recliner — and it exists whether or not a film is playing. Availability and price belong to that seat *for a particular show*, because H7 can be booked at 6:30 and free at 9:30 and cost more on Saturday. It is the same distinction as a book title versus a physical copy. The practical payoff: the seat map I draw is a `ShowSeat` query and the row I lock is a `ShowSeat` row — if I locked the `Seat` I would block it for every show in the building.”

A model answer

Asked: *design BookMyShow. Two users click the same seat at the same moment.*

“Let me settle one modelling question first, because the rest depends on it. A seat is two things. There is the physical seat — screen three, row H, number seven, recliner — which exists whether or not anything is showing. And there is that seat *for a particular show*, which has a status and a price, because H7 can be booked at six thirty and free at nine thirty and cost more on a Saturday. Those are two classes. The seat map I draw is a query over the second, and the row I lock is a row of the second. If I locked the physical seat I would block it for every show in the building.

Some quick arithmetic, because it changes a decision. Eight thousand cinemas, three screens each, two hundred seats, five shows a day is about twenty-four million show-seat rows a day, roughly two gigabytes. So those rows are generated per show and archived after it — they are disposable, unlike the five million physical seats. And reads beat writes about thirty to one normally and far more at a peak, which tells me the seat map is cacheable and the claim is not.

Now the actual question. The gap that causes all the difficulty is that choosing a seat and paying for it are minutes apart. So there are three states: available, locked, booked. Selecting a seat locks it with an expiry — ten minutes — and there are two ways out of locked: the payment confirms it, or the expiry returns it. Both are required. A lock with no expiry means every abandoned checkout removes seats from sale for ever.

Two users clicking the same seat: exactly one must win, and the mechanism is that the check and the claim have to be the same statement. The bug is to select the seat, see it is available, and then update — both requests read available, both write, both users own H7. Instead: one `UPDATE` that sets the status to locked *where* the status is still available, and then look at the affected row count. One row means I won. Zero means somebody beat me and I say so. With a two-millisecond gap between a read and a write, and a few thousand attempts a second on an opening night, I make the naive version about ninety double-sold seats a second — so this is not a theoretical concern.

For a family booking four seats it is all or nothing: one statement with the four seat ids and a check that four rows were affected, otherwise roll back. Four separate updates would let two users each hold half of the same block and block each other until both expire.

The expiry I would do lazily rather than with a sweeper: each locked row carries a `locked_until`, and the claim query treats an expired lock as available. That way correctness does not depend on a scheduled job running. I would still run a sweeper, but only so the displayed map is not misleading — which is cosmetic.

On confirmation, the update must also be conditional — booked *where* it is still locked by this user. Without that, a payment that arrives after the hold expired overwrites whoever booked the seat in the meantime, and a customer with a valid ticket loses it. When that check fails I refund automatically, because the money moved and the seat did not.

And for a blockbuster: the reads go to a cache with a few seconds' TTL, and I accept that a user may click a seat that has just gone, because the claim is authoritative and that error is cheap. The writes are all on two hundred rows of one show, so sharding does not help — there is one show. The real answer there is admission control, a virtual waiting room, which is what every ticketing site does.”

9 Recall card

- A seat is two classes: **Seat** (physical, in a screen) and **ShowSeat** (that seat, for that show). Availability, price, `locked_by` and `locked_until` live on the second. The map you draw is a **ShowSeat** query and the row you lock is a **ShowSeat** row — locking the physical seat would block it for every show in the building. Same split as `Book` / `BookItem`.
- **Selecting is not booking:** **AVAILABLE** → **LOCKED** (with an expiry) → **BOOKED**, and **both** arrows out of **LOCKED** are required — payment failure, and the expiry. A lock with no expiry is a leak.
- **The race is fixed by making the check and the claim one statement.** `UPDATE ... SET LOCKED WHERE status='AVAILABLE' OR (LOCKED AND locked_until < now)`, then **check the affected row count** — fewer than requested means roll back and tell the user. Check-then-act at a 2 ms gap and ~3,000 attempts/s gives roughly **90 double-sold seats per second**. Multiple seats: **one statement, all-or-nothing**; if you ever take row locks, **order them by seat id** or you deadlock.
- **Expire locks LAZILY, in the WHERE clause** — then correctness needs no scheduler; a sweeper is only cosmetic. And **the confirming update must also be conditional** (`AND status='LOCKED' AND locked_by = user`), or a late payment overwrites somebody else's booking — the worst failure here — and when it affects zero rows, **refund automatically**.
- **Reads and writes need opposite answers.** ~100,000 reads/s on a hot show's 200 rows, on data that may be seconds stale → **cache** (every seat map in the country is ~2 GB); ~3,000 claims/s that must be exact → **never cache**. Sharding does not help — *there is one show* — so the real lever is **admission control (a virtual waiting room)**. The hold duration is the main knob: **10 minutes**, because a card-plus-OTP flow is 1–2 minutes, and 20% abandonment means hundreds of seats are dead inventory at peak.

DSA topic: Linked lists revision and mock round **System design topic:** Design BookMyShow

Code these, in this order

One rule for the whole set: **diagnose before you repair**. Say which two or three techniques the problem is made of, out loud, before typing anything. If you cannot name them, name the $O(n)$ -space baseline instead and improve from there.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Odd Even Linked List	LeetCode 328 (Medium)	Two builder chains at once, and terminating both — the simplest composite.
2	Copy List with Random Pointer	LeetCode 138 (Medium)	The map baseline, then the interleaving trick, and restoring the original.
3	Reorder List	LeetCode 143 (Medium)	Runner + reverse + interleave. Three techniques, no new idea.
4	Sort List	LeetCode 148 (Medium)	First middle, the cut, and the quicksort question you will be asked.

On every one, run the six questions first

Can the first node change? Do I need a position I cannot count to? Might it not end? Am I turning arrows around? Am I combining or ordering? Do I already hold the node? Thirty seconds, out loud, before the first keystroke.

On problem 1, count your terminations

Two chains being built means two `next = None`s at the end, and forgetting either one builds a cycle. Print the result; if it never returns, you found it.

On problem 2, do both versions

The map version first, because it is the honest baseline and you should always say it. Then the interleaving version, and then check that the *original* list still has its `next` and `random` pointers exactly as they were.

On problem 4, prepare the answer before you are asked

“Why merge sort and not quicksort?” is coming. Two sentences: no random access for the partition, and merging lists needs no scratch space where merging arrays does.

The six-questions drill

For each problem, name the technique or techniques in under ten seconds:

1. Remove every node with value 7.
2. Return the middle node.
3. Is this list a palindrome, in $O(1)$ space?
4. Does this list have a cycle, and where does it start?
5. Reverse the nodes in groups of three.
6. Merge k sorted lists.
7. Design an LRU cache.
8. Remove the 3rd node from the end.
9. Reorder the list into first, last, second, second-last.
10. Copy a list with random pointers.
11. Flatten a multilevel doubly linked list.
12. Sort a linked list.

Four of the twelve need two or more techniques. Name them and their parts.

The five-sentences drill

Say each aloud, five times, then use each in a sentence about a specific problem:

1. To change the list at position k , stand at $k - 1$.
2. Save the next node before you overwrite the pointer.
3. Return `dummy.next`, never `head`.
4. Insertion is $O(1)$ *given the node*.
5. Advance only when you keep.

The error-message drill

For each error, name at least two distinct causes from this phase:

1. `AttributeError: 'NoneType' object has no attribute 'next'`
2. `RecursionError: maximum recursion depth exceeded`
3. `TypeError: '<' not supported between instances of 'Node' and 'Node'`
4. No error at all — the program hangs.

The fourth has three causes. Name all three.

The break-it drill, phase-wide

Trigger each and record the exact output or error text:

1. Overwrite `current.next` before saving it, in a reversal. Run `[1,2,3]` .
2. Return `head` after a reversal.
3. Discard the return value of a function that changes the head.
4. Handle a run of leading matches with an `if` instead of a dummy. Run `[7,7,7,1]` .
5. Advance after a deletion. Run `[1,7,7,2]` .
6. Split at the second middle in a merge sort. Run `[1,2]` .
7. Omit the cut in a split. Run anything.
8. Return the meeting point as a cycle start. Run a list that is entirely one cycle, then one with a tail.
9. Update one direction only in a doubly linked list, then walk backwards.
10. Leave a built list unterminated. Print it.

The mock round

Set a timer for forty-five minutes. Pick two problems you have never seen from the linked-list tag. Solve both out loud. Then mark every place where you:

- wrote code before naming the techniques,
- assigned a pointer before drawing what it should point at,
- skipped the empty, one-node or two-node case,
- mutated the caller's list without saying so.

Those four marks are the practice list. The problems were the excuse.

The read-only drill

For each, say whether the operation is allowed to modify the input, and what you would do if not:

1. Reverse the list.
2. Check whether it is a palindrome.
3. Detect a cycle.
4. Find the middle.
5. Copy a list with random pointers.
6. Sort the list.

Two of the six are commonly implemented in a way that damages the input while returning the right answer. Name them.

The two-things drill

1. Say why a seat is two classes, in one sentence.
2. List three fields for each class.
3. Say which one the seat map queries and which one you lock.
4. Say what goes wrong if you lock the other one.
5. Name two other domains from this course with the same split.

The race drill

1. Write the check-then-act version and describe the interleaving that breaks it.
2. Write the conditional `UPDATE` that fixes it.
3. Say what the affected row count means, in both cases.
4. Compute the collisions per second at 3,000 attempts, a 2 ms window and 200 seats.
5. Say why this is the same pattern as the parking spot and the shipped order.

The all-or-nothing drill

1. Write the four-seat claim as one statement.
2. Write it as four statements and describe the failure with two concurrent users.
3. Say what the affected count must equal, and what happens otherwise.
4. Say when lock ordering matters and what it prevents.

The expiry drill

1. Describe lazy expiry and where the condition lives.
2. Describe the sweeper approach.
3. Say which one correctness depends on, and why that is the better choice.
4. Say what the sweeper is still for.
5. Choose a hold duration and defend it with two numbers.

The payment drill

1. Write the confirming update with its conditional clause.
2. Delete the clause and describe the worst-case failure in the whole system.
3. Say what happens when the confirm affects zero rows.
4. Say which earlier lesson this failure is a copy of.

The scale drill

Compute each, showing the multiplication:

1. ShowSeat rows per day, and bytes.

2. Reads and writes per second on a blockbuster's seat map.
3. The read-to-write ratio, normal and at peak.
4. The size of every seat map in the country, in cache.
5. Dead inventory at a 20 percent abandonment rate with a ten-minute hold.

Then say which number tells you to cache the map, and which tells you that sharding will not help.

The trade-off drill

For each, give the trade in one sentence and say which side you would pick:

1. Lock in the database versus lock in Redis.
2. Five-minute hold versus fifteen-minute hold.
3. Cached seat map versus always-fresh seat map.
4. Sweeper-based expiry versus lazy expiry.
5. Sharding by show versus a virtual waiting room.

One of the five has an answer that is wrong in both directions until you add a qualifier. Name it.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two problems, no hints, talk as you go.* Run the six trigger questions on an unseen problem, name the techniques it is made of, draw the pointers, code narrating the two rules that matter, trace the degenerate case, and give the complexity with the counting and the space including whether you mutated the input.
 2. *Design BookMyShow. Two users click the same seat at the same moment.* The `Seat / ShowSeat` split first with the one-sentence payoff, the three states with both arrows out of LOCKED, the conditional claim as one statement with the row-count check and the collisions number, all-or-nothing, lazy expiry, the conditional confirm with the automatic refund, and cache the map but never the claim.
 3. *Why a linked list rather than an array?* $O(1)$ insertion and deletion **given the node**, the LRU case where a hash map hands you the node, the front-insertion measurement, and the honest costs — six times the memory and twice the traversal time.
-

Before you move on

- ☐ I can name all six trigger questions and apply them in ten seconds.
- ☐ I can say the five sentences without hesitating.
- ☐ I can give two causes for each of the four failure signatures.
- ☐ I triggered all ten phase traps and recorded the exact output.
- ☐ I ran a timed forty-five-minute mock on two unseen problems, out loud.
- ☐ I marked the four failure types in my own recording.
- ☐ I always ask whether I may modify the input list.
- ☐ I wrote both versions of the random-pointer copy and confirmed the original survived.
- ☐ I can say why a seat is two classes and what breaks if it is one.
- ☐ I can write the conditional claim and say what the row count means.
- ☐ I can compute the double-sold seats per second for the naive version.
- ☐ I can explain lazy expiry and why correctness should not depend on a scheduler.
- ☐ I can write the conditional confirm and name the failure it prevents.
- ☐ I can say which number justifies caching the map and which says sharding will not help.
- ☐ I can defend a ten-minute hold with two numbers.
- ☐ I answered all three questions above out loud.

PART XI

Recursion and backtracking

Days 87 to 97 · 11 chapters

Alongside, on the system design track:

Low-level design case studies, Scaling fundamentals

087 Recursion: the leap of faith

088 The call stack, drawn

089 Writing a recursive function that terminates

090 Recursion on arrays and strings

091 Subsets: the include-or-exclude tree

092 Permutations

093 Combinations and combination sum

094 Backtracking: the undo step

095 N-Queens and constraint grids

096 Sudoku, word search, and grid backtracking

097 Recursion and backtracking revision and mock round

Recursion: the leap of faith

DATA STRUCTURES AND ALGORITHMS

Recursion: the leap of faith

You can write a recursive function by trusting it on the smaller input.

SYSTEM DESIGN

Design a food delivery order flow

You can model restaurants, carts, orders and the delivery state machine.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Solve this recursively. What is the base case?*
- *Design the order flow for a food delivery app.*

1 What this is, and why they ask it

A **recursive** function is one that calls itself. It solves a big problem by asking a smaller version of itself for an answer and then doing one small piece of work with what comes back.

The mechanics are two lines. There is a **base case** — an input so small you can answer it without asking anybody — and a **recursive case**, which asks the same function about something smaller and combines the answer.

The difficulty is not the mechanics. It is that most people try to *trace* the recursion in their head: this call makes that call, which makes that one, and by the fourth level they are lost. That is the wrong way to think about it, and it is why recursion feels hard.

The right way is the **leap of faith**: assume the recursive call already works, on the smaller input, and write the one line that uses its answer. You do not check. You do not trace. You trust it, exactly the way you trust a library function you did not write.

They ask it because everything from here to the end of the course is recursive — backtracking, trees, graphs, divide and conquer, dynamic programming — and because “what is the base case?” is a question you will be asked in every one of those rounds. Getting comfortable with the leap of faith this week is what makes the next sixty days possible.

2 The story

The ration shop opens at nine and by ten past eight the line already goes round the corner and along the side of the school wall, and from where Murali is standing he cannot see the shop at all.

He wants to know how many people are ahead of him, because if it is more than about forty he will come back after lunch.

He cannot count them. The line bends twice, there are people sitting on the step who may or may not be in it, and walking to the front to count and coming back would lose him his place.

So he taps the shoulder of the woman in front of him and asks how many people are ahead of *her*.

She does not know either. She is in exactly the same position — she cannot see the front, and the line bends. So she taps the man in front of her and asks him the same question.

He asks the person in front of him. And so on, all the way up the line and round the corner, one shoulder at a time.

At the very front, right at the shutter, is an old man who has been there since half past seven. When he is asked how many people are ahead of him, he does not have to ask anybody. He can see the shutter. He says: none.

Then it comes back. The person behind him hears “none” and says “one” to the person behind her. She hears nothing else and needs nothing else — she does not know or care how the number was arrived at. She adds herself and passes it back. One becomes two, two becomes three, and about ninety seconds later the woman in front of Murali turns round and says thirty-seven.

Murali adds one for her and knows he is thirty-eighth. He stays.

What he did not do, and this is the whole thing, is check her answer. He did not walk up the line to verify that there really were thirty-seven people. He asked one person one question and trusted what came back, and every single person in that line did exactly the same, and the answer was right.

3 The idea in plain English

Murali’s question is the recursive call. The old man at the shutter is the base case. And nobody checking anybody else’s answer is the leap of faith.

The three questions

Every recursive function you will ever write comes from answering these three, **in this order**:

1. What is the smallest input I can answer without asking anybody? That is the base case. The old man who can see the shutter. If you cannot name it, you cannot write the function.

2. What is a smaller version of the same problem? Not a different problem — the *same* problem, on less. “How many are ahead of the person in front of me” is the same question, one person shorter.

3. Given the answer to that smaller problem, what one thing do I do to get mine? Add one. That is all Murali’s neighbour did.

```
def count_ahead(person):
    if person.is_at_the_front():          # 1. the base case
        return 0
    return 1 + count_ahead(person.in_front) # 2. smaller, and 3. one small step
```

Three lines, and they are the three questions in order.

The leap of faith

Here is the habit that makes recursion easy, and it is genuinely a habit rather than an insight.

When you write the recursive call, assume it already works. Do not trace it. Do not wonder what happens three levels down. Treat `count_ahead(person.in_front)` as if it were a finished library function written by somebody competent, which returns the correct answer for a shorter line.

Then your only job is the one line that uses it.

People fail at recursion because they try to hold the whole tree of calls in their head. Nobody can do that past three levels, and you do not need to: **if the base case is right, and each call is on a strictly smaller input, and your one line is right, then the whole thing is right.** Those three conditions are all you have to check.

Worked, four times, so the shape becomes familiar

Sum a list.

- Smallest input I can answer directly: the empty list. Sum is 0.
- Smaller version: the list without its first element.
- One small step: add the first element.

```
def total(numbers: list[int]) → int:
    if not numbers:
        return 0
    return numbers[0] + total(numbers[1:])
```

Reverse a string.

- Base case: a string of length 0 or 1 is its own reverse.
- Smaller: everything after the first character.
- One step: put the first character on the *end*.

```
def reverse(text: str) → str:
    if len(text) ≤ 1:
        return text
    return reverse(text[1:]) + text[0]
```

Count the nodes in a linked list, which ties this to the last eight days:

```
def length(node) → int:
    if node is None:                # the base case is the end of the list
        return 0
    return 1 + length(node.next)    # exactly Murali's question
```

Raise a number to a power.

- Base case: anything to the power 0 is 1.
- Smaller: one less in the exponent.
- One step: multiply by the base.

```
def power(base: int, exponent: int) → int:
    if exponent == 0:
        return 1
    return base * power(base, exponent - 1)
```

Four functions, one shape. **Base case, smaller call, one small step.** If you can see that shape, you can write the function without understanding what happens in the middle — and you are not supposed to understand what happens in the middle.

The two ways the base case goes wrong

Missing. The function never stops:

```
RecursionError: maximum recursion depth exceeded
```

Unreachable, which is worse because it looks correct. `total(numbers[2:])` skips a step, so an odd-length list steps over the empty case and goes negative into an infinite loop of empty slices — or, more commonly, `power(base, exponent - 2)` never lands on exactly 0 for an odd exponent.

Every recursive call must make the input strictly smaller, and it must be able to reach the base case. Say both halves out loud when you write it. This is [day 089](#)'s whole subject.

Recursion is not free

Every call in progress occupies a **stack frame** — a small block of memory holding its arguments and where to return to. From [day 068](#), that is a real stack, and Python's limit is about a thousand frames:

```
RecursionError: maximum recursion depth exceeded
```

So `total` on a list of ten thousand numbers **crashes**, while the loop version does not. That is not an edge case; ten thousand is a small list.

Recursion costs $O(\text{depth})$ space, always. A loop costs $O(1)$. When the depth is `log n` — binary search, merge sort — that is twenty frames at a million elements and nobody cares. When the depth is `n`, it is a real limit.

The one where recursion is a trap

```
def fib(n: int) → int:
    if n ≤ 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

Beautiful, correct, and **exponential**. Each call makes two more, so the number of calls roughly doubles with every increase in `n`. `fib(40)` makes about 331 million calls and takes about thirteen seconds. `fib(50)` would take about half an hour.

The reason is that the same subproblems are computed again and again — `fib(35)` is recomputed millions of times. Two fixes, and both are worth knowing today:

```
from functools import cache

@cache
def fib(n: int) → int:
    return n if n ≤ 1 else fib(n - 1) + fib(n - 2)
```

remember answers; now each n is computed once

One decorator turns 331 million calls into 41. That is **memoisation**, and it is the entire foundation of dynamic programming later in the course.

Or write the loop, which is $O(n)$ time and $O(1)$ space and needs no stack at all.

Fibonacci is the standard example of recursion and it is a bad one, because it teaches the shape while hiding the cost. Notice the difference: in `total` and `length`, each call makes **one** further call — so the work is linear. In `fib`, each call makes **two** — so the work explodes. Counting the calls per level is how you tell them apart, and it is [day 088's](#) material.

Recursion and iteration are interchangeable

Anything recursive can be written with an explicit stack, and anything with a stack can be written recursively — because the call stack *is* a stack. From day 068: a recursion that would blow the thousand-frame limit becomes a loop with a list, limited only by memory.

So when do you choose recursion? **When the problem is defined recursively.** A tree is a node with subtrees. A directory contains directories. Merge sort sorts two halves. Writing those iteratively means managing a stack by hand, and the code is longer and

worse. Writing `total` recursively, on the other hand, is showing off — the loop is clearer and does not crash.

4 The picture

`total([4, 7, 2])`, drawn as the two journeys — down the line asking, and back up answering.

GOING DOWN (each call asks a smaller question)

```
total([4, 7, 2])
  | 4 + ?
  v
total([7, 2])
  | 7 + ?
  v
total([2])
  | 2 + ?
  v
total([])      ← the base case. The old man at the shutter.
  returns 0
```

COMING BACK UP (each call does its one small step)

```
total([])    → 0
total([2])   → 2 + 0 = 2
total([7,2]) → 7 + 2 = 9
total([4,7,2]) → 4 + 9 = 13
```

Nobody on the way down knew the answer.
Nobody on the way up checked anybody else's.

What to notice: **the work happens on the way back up**. Going down, every call is suspended in the middle of an addition, waiting. That waiting is what a stack frame is, and it is why depth costs memory.

The shape of the three questions, as a template:

```
def solve(problem):
    if <smallest case>:                # 1. What can I answer without asking?
        return <the direct answer>
    smaller = solve(<problem, but smaller>) # 2. Ask, and TRUST the answer
    return <one small step involving `smaller`> # 3. My one piece of work
```

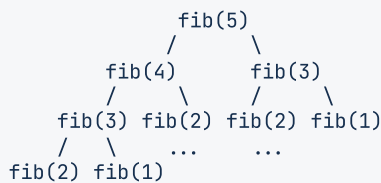
And the difference between linear and exponential recursion, which is the thing to be able to see:

ONE call per level → linear

```
total([4,7,2])
|
total([7,2])
|
total([2])
|
total([])
```

3 calls for 3 elements
n calls for n elements

TWO calls per level → exponential



15 calls for $n = 5$
 $\sim 2^n$ calls
 $\text{fib}(40) \approx 331,000,000$ calls

Count the branches per call. **One branch is a line; two branches is a tree**, and a tree of depth n has about 2^n nodes.

5 The code, built step by step

Step 1 — answer the three questions out loud, before typing

For “sum a list”:

“The smallest input I can answer without asking anybody is the empty list, and the answer is zero. A smaller version of the same problem is the list without its first element. And given the sum of that, my one step is to add the first element.”

Three sentences, and the function is now written. Say them in the interview — it is the same move as “say what the stack holds” in the last phase.

Step 2 — write the base case first

```
if not numbers:
    return 0
```

Always first, always at the top. A function whose base case is buried below the recursive call has already made the call before checking, which is an immediate `RecursionError`.

Step 3 — write the recursive call, and do not think about it

```
return numbers[0] + total(numbers[1:])
```

`total(numbers[1:])` is *correct*. It returns the sum of the rest. You are not going to check that, and looking inside it is the thing that makes recursion feel hard.

Step 4 — check the three conditions, not the trace

1. Does the base case return the right answer? `total([])` is 0. Yes.
2. Is every recursive call on a **strictly smaller** input? `numbers[1:]` is one shorter. Yes.
3. Is the combining step right? Sum of the whole is the first plus the sum of the rest. Yes.

Those three checks replace tracing. If all three hold, the function is correct, and you never had to imagine four levels of calls.

Step 5 — the cost, from the recurrence

`total` makes one call per element, so `n` calls, each doing constant work — $O(n)$ time. But `numbers[1:]` **copies the list**, which is $O(n)$ itself, so this particular version is secretly $O(n^2)$ in time and $O(n^2)$ in total allocation.

The fix is to pass an index instead of slicing:

```
def total(numbers: list[int], start: int = 0) → int:
    if start == len(numbers):
        return 0
    return numbers[start] + total(numbers, start + 1)
```

Slicing in a recursive call is the quiet performance bug of this phase, and it is worth catching now, because it will reappear in every string and array recursion for the next ten days.

The complete solution

```
from functools import cache

def total(numbers: list[int], start: int = 0) → int:
    """Sum a list recursively.

    1. base case      : nothing left, the sum is 0
    2. smaller problem: the list from `start + 1` onwards
    3. one small step : add numbers[start]

    An INDEX, not a slice: `numbers[1:]` copies the list on every call, which
    turns an  $O(n)$  algorithm into  $O(n^2)$  time and allocation.

     $O(n)$  time,  $O(n)$  stack – so it dies past ~1000 elements in Python. The loop
    version is strictly better; this exists to show the shape.
    """
    if start == len(numbers):
        return 0
    return numbers[start] + total(numbers, start + 1)

def reverse_string(text: str) → str:
    """A string of 0 or 1 characters is its own reverse; otherwise put the
    first character behind the reverse of the rest.

     $O(n)$  calls, but each `+` builds a new string, so  $O(n^2)$  total work – the
    same slicing trap in a different costume.
    """
    if len(text) ≤ 1:
        return text
    return reverse_string(text[1:]) + text[0]

def power(base: int, exponent: int) → int:
    """base ** exponent, for exponent ≥ 0.  $O(\text{exponent})$  calls."""
    if exponent < 0:
        raise ValueError("exponent must be non-negative")
    if exponent == 0:
        return 1
    return base * power(base, exponent - 1)

def fast_power(base: int, exponent: int) → int:
    """The same thing in  $O(\log \text{exponent})$  calls, by halving instead of decrementing.

     $x^n = (x^{(n/2)})^2$  for even  $n$ , and  $x * x^{(n-1)}$  for odd  $n$ . The recursive call
    is made ONCE and stored – calling it twice would make this exponential
    again, and that is a real mistake people make here.
    """
    if exponent == 0:
        return 1
    half = fast_power(base, exponent // 2)          # once, not twice
    return half * half if exponent % 2 == 0 else base * half * half

def count_nodes(node) → int:
    """The linked list from the last phase, counted recursively.
    The base case is the end of the list – Murali's old man at the shutter."""
    if node is None:
        return 0
    return 1 + count_nodes(node.next)

def is_palindrome(text: str) → bool:
    """Two characters at a time from the outside in.

    base case      : 0 or 1 characters left is a palindrome
    smaller problem: the string without its two ends
    one small step : the two ends must match
    """
    if len(text) ≤ 1:
```

```

        return True
    if text[0] != text[-1]:
        return False
    return is_palindrome(text[1:-1])

def fib_slow(n: int) → int:
    """The classic example, and a BAD one. TWO recursive calls per level means
    about 2^n calls: fib(40) is roughly 331 million and takes about thirteen seconds."""
    if n ≤ 1:
        return n
    return fib_slow(n - 1) + fib_slow(n - 2)

@cache
def fib_fast(n: int) → int:
    """The same function with one decorator. Each n is now computed once, so
    fib(40) is 41 calls instead of 331 million. This is memoisation, and it is
    the foundation of dynamic programming later in the course."""
    if n ≤ 1:
        return n
    return fib_fast(n - 1) + fib_fast(n - 2)

def fib_loop(n: int) → int:
    """O(n) time, O(1) space, no stack at all. What you would actually ship."""
    previous, current = 0, 1
    for _ in range(n):
        previous, current = current, previous + current
    return previous

def binary_search(numbers: list[int], target: int, low: int = 0, high: int | None = None) → int:
    """Recursion where the DEPTH is log n, so the stack cost is irrelevant:
    about 20 frames at a million elements."""
    if high is None:
        high = len(numbers) - 1
    if low > high:
        return -1 # base case: nothing left
    middle = (low + high) // 2
    if numbers[middle] == target:
        return middle
    if numbers[middle] < target:
        return binary_search(numbers, target, middle + 1, high)
    return binary_search(numbers, target, low, middle - 1)

if __name__ == "__main__":
    import sys
    import time

    print(total([4, 7, 2]), total([1]), total([5])) # 13 0 5
    print(reverse_string("recursion"), reverse_string("")) # noisrucer ''
    print(power(2, 10), power(7, 0)) # 1024 1
    print(fast_power(2, 10), fast_power(3, 13)) # 1024 1594323
    print(is_palindrome("malayalam"), is_palindrome("recursion")) # True False
    print(binary_search([1, 3, 5, 7, 9, 11], 9)) # 4
    print(binary_search([1, 3, 5, 7, 9, 11], 4)) # -1

    start = time.perf_counter()
    print(fib_slow(30), f"{time.perf_counter() - start:.3f}s") # 832040
    start = time.perf_counter()
    print(fib_fast(30), f"{time.perf_counter() - start:.6f}s") # 832040, ~0.000s
    print(fib_loop(30), fib_fast(90)) # 832040 2880067194370816120

    print(sys.getrecursionlimit()) # 1000
    try:
        total(list(range(5000)))
    except RecursionError as error:
        print(f"RecursionError: {error}")

    # the same input, iteratively, with no limit at all
    print(sum(range(5000))) # 12497500

```

6 What it costs

Time, read off the recurrence

One recursive call per level means the number of calls is the depth:

```
total(n):    T(n) = T(n-1) + O(1)    →  n calls        →  O(n)
power(n):    T(n) = T(n-1) + O(1)    →  n calls        →  O(n)
fast_power:  T(n) = T(n/2) + O(1)    →  log2(n) calls →  O(log n)
binary_search: same shape           →  O(log n)
```

Two recursive calls per level means the number of calls doubles each level:

```
fib(n):      T(n) = T(n-1) + T(n-2) + O(1) →  about 2^n calls →  O(2^n)
```

```
fib(30):      about 2.7 million calls      ~0.10 s   (measured)
fib(40):      about 331 million calls      ~13 s    (123x the calls)
fib(50):      about 4 x 10^10 calls        ~30 minutes
fib(40), memoised: 41 calls                ~0.00002 s
```

Count the recursive calls in the body. One is a line, two is a tree, and the difference between them is the difference between a tenth of a second and half an hour.

Space, which is always the depth

```
every call in progress = one stack frame
stack frame in CPython ≈ 100-500 bytes depending on locals

total(n):      depth n          →  O(n) space, crashes past ~1000
fast_power(n): depth log n      →  O(log n), about 20 frames at a million
binary_search: depth log n      →  irrelevant
fib(n):        depth n          →  O(n), even though it makes 2^n calls
```

That last line is worth pausing on: **fib makes 2ⁿ calls but only ever has n of them on the stack at once**, because it finishes the left branch before starting the right. Time and space complexity are answering different questions, and this is the cleanest example of the difference.

The recursion limit is a real limit

```
sys.getrecursionlimit() → 1000
```

```
RecursionError: maximum recursion depth exceeded
```

on a list of five thousand. Not an exotic input — a small list. And `sys.setrecursionlimit(100000)` is **not** a fix: it moves the failure from a clean Python exception to a segmentation fault when the C stack runs out, which crashes the whole process with no traceback.

The rule: recursion whose depth is `log n` is free. Recursion whose depth is `n` is a bug waiting for a big input, unless you know `n` is small.

The slicing trap

```
return numbers[0] + total(numbers[1:])
```

```
calls:      n
work per call: O(n) to build the slice
-----
total:      O(n^2) time AND O(n^2) allocation
```

At $n = 1000$ that is half a million element copies to add up a thousand numbers. Pass an index. The same trap in strings — `text[1:]` — makes `reverse_string` quadratic, and it is the reason real string recursions carry two indices.

Against iteration

	recursive	iterative
sum a list	$O(n)$ time	$O(n)$ time
	$O(n)$ space	$O(1)$ space
	dies at ~ 1000	no limit
fibonacci	$O(2^n)$ naive	$O(n)$ time, $O(1)$ space
	$O(n)$ memoised	
binary search	$O(\log n)$ both, and the space difference is 20 frames	

Iteration wins whenever the depth is proportional to the input, and it does not matter when the depth is logarithmic. Recursion wins when the *problem* is recursive — trees, directories, divide and conquer — where the iterative version means managing a stack by hand.

7 The traps

Trap 1 — no base case

```
def total(numbers):
    return numbers[0] + total(numbers[1:])    # nothing stops it
```

```
RecursionError: maximum recursion depth exceeded
```

The most common recursion bug, and the friendliest, because Python tells you exactly what happened. **Write the base case first, at the top, before the recursive call.**

Trap 2 — an unreachable base case

```
def power(base, exponent):
    if exponent == 0:
        return 1
    return base * power(base, exponent - 2)    # steps over 0 on odd exponents
```

`power(2, 5)` goes 5, 3, 1, -1, -3, ... and never equals 0. Same error, much harder to see, because the base case is *there* — it just cannot be reached from every input.

Say both halves out loud: the base case exists, and every input can reach it. Guarding with `if exponent ≤ 0` instead of `= 0` is a defensive fix that hides the real bug; fixing the step is the right one.

Trap 3 — forgetting to return the recursive call

```
def total(numbers, start=0):
    if start == len(numbers):
        return 0
    numbers[start] + total(numbers, start + 1) # computed, then thrown away
```

Returns `None` for any non-empty list, and then:

```
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

one level up. **Every path through a recursive function must return something.** Reading the function and checking that each branch has a `return` takes five seconds.

Trap 4 — two recursive calls where one would do

```
def fast_power(base, exponent):
    if exponent == 0:
        return 1
    if exponent % 2 == 0:
        return fast_power(base, exponent // 2) * fast_power(base, exponent // 2)
```

Correct, and **exponential** — it recomputes the same value twice at every level, which turns $O(\log n)$ into $O(n)$. Call it once, store it in a variable, use it twice. This is the same mistake as naive Fibonacci and it is easy to make while writing a function specifically designed to be fast.

Trap 5 — slicing in the recursive call

```
return numbers[0] + total(numbers[1:])
```

$O(n^2)$ time and allocation, silently. Pass an index or two indices. This is the quiet one — the function is correct and just mysteriously slow on larger inputs.

Trap 6 — a mutable default argument

```
def collect(node, found=[]):                # created ONCE, at definition time
    ...
```

The list is created once when the function is defined, not per call, so the second call to `collect` sees everything the first one found. Use `None` and create it inside:

```
def collect(node, found=None):
    if found is None:
        found = []
```

This bites hardest in recursion because helper functions with accumulators are so common here.

Trap 7 — sharing mutable state across branches

```
def subsets(items, index=0, current=[]):
    ...
    current.append(items[index])
    subsets(items, index + 1, current)    # the SAME list, mutated
    current.pop()                        # so it must be undone
```

Passing a mutable accumulator down is a real and standard technique — it is what backtracking is — but every branch must leave it exactly as it found it. The missing `pop` is the defining bug of [day 094](#), and it is worth knowing today that it is coming.

Trap 8 — raising the recursion limit

```
sys.setrecursionlimit(1_000_000)          # do not
```

Python's limit exists to convert a C-stack overflow into a catchable exception. Raising it past what the C stack can hold turns a clean `RecursionError` into a segmentation fault that kills the process with no traceback. If the depth is genuinely large, convert to iteration with an explicit stack.

8 In the interview

How it gets asked

- The direct version: *“Solve this recursively.”* Then, immediately: *“What is the base case?”*
- The conversion: *“Here is a loop. Write it recursively.”* Or the reverse, which is more common in production work.
- The classic trap: *“Write Fibonacci recursively.”* Followed by *“what is its complexity?”* and *“can you do better?”*
- The cost question: *“What does recursion cost that a loop does not?”* — they want stack depth.
- The design question: *“When would you choose recursion over a loop?”*

What to say out loud, in the first ninety seconds

1. **Answer the three questions, aloud, before writing.** “The smallest input I can answer directly is X, and the answer is Y. A smaller version of the same problem is Z. And given that answer, my one step is W.”
2. **Name the leap of faith explicitly.** “I am going to assume the recursive call works on the smaller input, and only write the line that uses it.”
3. **Write the base case first,** at the top.
4. **Say the two termination conditions.** “The base case is reachable because every call reduces the input by exactly one, so it lands on zero.”
5. **Give both complexities, and separate them.** “One recursive call per level and n levels, so $O(n)$ time. And $O(n)$ stack space, which matters — this dies at about a thousand elements in Python.”
6. **Offer the iterative version if the depth is linear.** “For a list, I would actually write the loop — the recursion is $O(n)$ stack for no benefit. I would use recursion where the problem is genuinely recursive, like a tree.”

The follow-ups

“What is the base case?” “The smallest input I can answer without making another call. For summing a list it is the empty list, which sums to zero. Two things have to be true about it: it must return the right answer, and every input must be able to *reach* it. The second half is the one people miss — a function that steps by two towards a base case of exactly zero never terminates on odd inputs, and the base case looks perfectly correct while it happens.”

“What does recursion cost that a loop does not?” “Stack space, proportional to the depth. Every call in progress holds a frame with its arguments and its return address, and they all coexist until the deepest one returns. In Python the limit is about a thousand

frames, so a recursion whose depth is the size of the input dies on a five-thousand-element list — that is not an exotic input. When the depth is logarithmic, like binary search or merge sort, it is about twenty frames at a million elements and nobody cares. So the rule I use: log-depth recursion is free, linear-depth recursion is a bug waiting for a big input.”

“Write Fibonacci recursively. What is its complexity?” “It is three lines and it is exponential, and the reason is worth stating precisely: the body makes **two** recursive calls, so the number of calls roughly doubles per level — about 2^n . `fib(40)` is around 331 million calls, about thirteen seconds. Compare with summing a list, where the body makes **one** call and the work is linear. Counting the recursive calls in the body is how you tell those apart. The fix is memoisation — one `@cache` decorator turns 331 million calls into 41, because each `n` is computed once — and that is the foundation of dynamic programming. Or write the loop, which is $O(n)$ time and $O(1)$ space.”

“When would you choose recursion over iteration?” “When the problem is defined recursively and the depth is bounded. A tree is a node with subtrees, a directory contains directories, merge sort sorts two halves — writing those iteratively means managing a stack by hand, and the code is longer and easier to get wrong. For a flat list I would write the loop: the recursive version is $O(n)$ stack for no gain and it crashes on real inputs. And anything recursive *can* be converted, because the call stack is a stack — that is the escape hatch when the depth is too large.”

“Your recursive version is slower than the loop even though both are $O(n)$. Why?” “Two reasons, and one of them is usually a bug. Every call has overhead — building a frame, binding arguments, returning — which is a real constant factor in Python. But the bug is usually slicing: if the recursive call is `f(items[1:])`, each call copies the list, so an $O(n)$ algorithm is secretly $O(n^2)$ in both time and allocation. Passing an index instead of slicing fixes it, and the same trap appears in every string recursion.”

“How would you make this work on a million elements?” “Convert it to a loop with an explicit stack, because the call stack is a stack — that transformation always exists. I would not raise the recursion limit: Python’s limit is there to turn a C-stack overflow into a catchable exception, and raising it past what the C stack can hold gives a segmentation fault instead, which kills the process with no traceback.”

A model answer

Asked: *sum the numbers in a list, recursively. What is the base case?*

“Let me answer three questions before writing anything, because they are the function.

First: what is the smallest input I can answer without asking anybody? The empty list. Its sum is zero. That is the base case.

Second: what is a smaller version of the *same* problem? The list without its first element. Not a different question — the same question, one element shorter.

Third: given the sum of that smaller list, what one thing do I do? Add the first element.

So: if the list is empty, return zero; otherwise return the first element plus the sum of the rest. Three lines.

The habit that makes this easy is the leap of faith. When I write the recursive call, I assume it already works — that it correctly returns the sum of the shorter list — and I only write the line that uses the answer. I do not trace it. Trying to hold four levels of calls in your head is what makes recursion feel hard, and it is unnecessary: if the base case is right, and every call is on a strictly smaller input, and my one combining step is right, then the whole function is right. Three checks instead of a trace.

On the base case specifically, since you asked: two things have to be true, and people usually only say the first. It has to return the correct answer, and every input has to be able to *reach* it. If I stepped by two instead of one, an odd-length list would step straight over the empty case and recurse for ever — and the base case would still look perfectly correct while it happened.

One implementation detail I would flag. I would pass an index rather than slicing, because `numbers[1:]` copies the list on every call, which turns this into $O(n^2)$ time and allocation. It is a quiet bug — the function is correct, just mysteriously slow on bigger inputs — and the same trap appears in every string recursion.

Complexity: one recursive call per element, so n calls of constant work, $O(n)$ time. And $O(n)$ *stack* space, which is the part that matters. Python’s recursion limit is about a thousand frames, so this raises `RecursionError` on a five-thousand-element list, which is not a large list.

So honestly, for a flat list I would write the loop. The recursion is $O(n)$ stack for no benefit. I would reach for recursion when the problem is genuinely recursive — a tree, a directory, merge sort — where the depth is logarithmic or the iterative version means managing a stack by hand.”

9 Recall card

- **Three questions, in order, and they *are* the function.** *What is the smallest input I can answer without asking anybody? (the base case) · What is a smaller version of the same problem? · Given that answer, what one small step gives me mine?* Write the base case **first, at the top.**
- **The leap of faith: assume the recursive call already works, and write only the line that uses it.** Do not trace it — nobody can hold four levels in their head. **Three checks replace tracing:** the base case returns the right answer · every call is on a **strictly smaller** input · the combining step is right.
- **A base case can be present and unreachable, which is the harder bug.** Stepping by two towards `= 0` never terminates on odd inputs. Say both halves: *it exists, and every input can reach it.*
- **Count the recursive calls in the body: one is a line, two is a tree.** `total` makes one $\rightarrow O(n)$. `fib` makes two $\rightarrow \sim 2^n$: `fib(40)` is **~ 331 million calls, ~ 13 s**, and `@cache` turns it into **41 calls** — that is memoisation, and it is the foundation of DP.
- **Recursion always costs $O(\text{depth})$ stack, and Python's limit is ~ 1000 frames** — so a linear-depth recursion dies on a **5,000-element list**, and `sys.setrecursionlimit` converts a clean `RecursionError` into a **segfault**. *Log-depth recursion is free; linear-depth is a bug waiting for a big input.* And **never slice in the recursive call** — `f(items[1:])` copies every time and makes an $O(n)$ algorithm $O(n^2)$; pass an index.

1 What this is, and why they ask it

A customer builds a cart from one restaurant's menu, pays, the restaurant accepts and cooks, a delivery partner collects it, and it arrives. Eight steps, and the design is entirely about those eight steps and what happens when one of them does not occur.

Three things make this different from the state machines you have already built.

The transitions come from different actors. The customer places, the restaurant accepts, the partner picks up. No single party drives the order forward, so no single party can be asked "what state is it in?" and be trusted.

Time is one of the actors. If the restaurant does not accept within ninety seconds, something must happen anyway. A transition with no actor and no timeout is an order stuck for ever, and at two million orders a day even a rare stuck state is thousands of angry customers.

The price must be frozen. Menus change during the day; a dish costing ₹240 at 7:58 and ₹260 at 8:02 must be charged at whatever the customer saw. That is the same immutability rule as the library's loan and the cinema's booking, and it is the source of most real billing disputes.

They ask it because everyone has used one of these apps, because the happy path is boring and the failure paths are not, and because "what happens if the restaurant never responds?" separates a candidate who has drawn a state machine from one who has run a system.

2 The story

Lakshmi has been sending out tiffins from the house since her husband's factory closed, and it is now about ninety boxes a day.

There are three of them. Lakshmi cooks. Her daughter Shruthi takes the calls and writes the orders into the tablet. And her son Kiran does the deliveries on the scooter, two runs, one at half past eleven and one at half past twelve.

Three rules run the whole thing, and each of them came out of something going wrong.

The first is about the price. For a while Shruthi would quote whatever that day's rate was, and then if vegetables had been dear that morning Lakshmi would want to charge ten rupees more, and the customer would say but she told me sixty. It caused a bad argument with a regular customer in the second month. Now the rule is: whatever Shruthi said on the phone is the price. It is written into the tablet when the order is taken and it does not change, even if tomatoes doubled.

The second is about time. There was a day when Shruthi took eleven orders before realising her mother had gone to the hospital with her aunt and nothing was cooking. The customers found out at one o'clock. So now the rule is that Shruthi does not confirm an order to the customer until Lakshmi has said yes to it — and if Lakshmi has not said yes within a few minutes, Shruthi rings the customer back and says sorry, not today. Not at one o'clock. Straight away.

The third is about cancelling, and it is the one Shruthi has had to explain most often. If you ring before the food is made, that is fine, no problem, nothing is lost. If you ring while it is cooking, Lakshmi will usually still say fine, but she has spent the ingredients. And if you ring after Kiran has left the house with the box on the back of the scooter, the answer is no — not because they are being difficult, but because the food is thirty minutes away on a scooter and it is not coming back.

Shruthi says the customers who get upset are almost always the ones who assumed the answer was the same at every stage. Once she explains that it depends where the box is, they understand it immediately, because everybody understands that a box on a scooter is a different situation from a box that does not exist yet.

3 The idea in plain English

Lakshmi's three rules are the three design decisions. The price frozen at order time. A timeout on the step somebody else has to take. And cancellation being a function of *where the order is*, not a setting.

The states, and who moves each one

CART	the customer is still choosing	(no order exists yet)
PAYMENT_PENDING	payment initiated	
PLACED	paid; the restaurant has not seen it yet	
ACCEPTED	the restaurant said yes	
PREPARING	cooking	
READY	waiting for a partner to collect	
PICKED_UP	on the scooter	
DELIVERED	done	
REJECTED	the restaurant said no	
CANCELLED	the customer, or the system, stopped it	

The important column is not the state. It is **who causes the transition, and what happens if they do not**:

TRANSITION	ACTOR	TIMEOUT	WHAT HAPPENS ON TIMEOUT
PAYMENT_PENDING → PLACED	payment gateway	5 min	Cancel, release the cart
PLACED → ACCEPTED	restaurant	90 s	Auto-cancel and refund, or offer to a backup restaurant
ACCEPTED → PREPARING	restaurant	—	Usually merged with ACCEPTED
PREPARING → READY	restaurant	prep estimate + 10 min	Alert operations; notify the customer of a delay
READY → PICKED_UP	partner	10 min	Reassign to another partner
PICKED_UP → DELIVERED	partner	ETA + 20 min	Alert operations; the customer sees a delay

Every row has a timeout except the ones the restaurant does back-to-back. That is the design. A transition whose actor is somebody else, with no timeout, is an order that can sit for ever — and the thing customers actually complain about is not failure, it is silence.

Time as an actor

The cleanest way to model this is that **each state carries a deadline**, and a single sweeper asks “which orders are past their deadline?” and applies that state’s timeout rule.

```
@dataclass(frozen=True)
class StateRule:
    timeout: timedelta | None
    on_timeout: Callable[["Order"], None]

RULES = {
    OrderState.PLACED: StateRule(timedelta(seconds=90), auto_cancel_and_refund),
    OrderState.READY: StateRule(timedelta(minutes=10), reassign_partner),
    OrderState.PICKED_UP: StateRule(None, None),           # no automatic action
}
```

One table, one sweeper, and the alternative — a scheduled job per transition type — is six jobs that can each independently stop running. **One mechanism with a table of rules is much easier to keep correct than six mechanisms.**

Note what this is *not*: it is not the lazy expiry from [day 086](#). A cinema seat lock can expire lazily because nothing has to *happen* when it does — the seat simply becomes claimable again. Here the timeout must trigger a refund and a notification, so something active is required. Knowing which situations allow lazy expiry and which do not is the distinction worth drawing.

The price snapshot

```
@dataclass(frozen=True)
class OrderItem:
    menu_item_id: str
    name_at_order: str          # the menu can be renamed
    price_paise: int            # the price AT THE MOMENT OF ORDERING
    quantity: int
```

`OrderItem` copies the name and the price rather than pointing at the `MenuItem`. That looks like denormalisation and it is deliberate: the order is a **record of what was agreed**, and joining to the live menu to compute a total means an order's price changes after the fact.

The same rule applies to the delivery fee, the taxes, the surge multiplier and the discount. **Freeze everything that was shown to the customer**, and store the total as well, so a rounding-rule change next year cannot alter last year's invoices.

Shruthi writing the price into the tablet when she takes the call.

Cancellation is a function of state

```
def can_cancel(order: Order, by: Actor) → CancelPolicy:
    match order.state:
        case OrderState.PLACED:
            return CancelPolicy(allowed=True, refund_percent=100)
        case OrderState.ACCEPTED | OrderState.PREPARING:
            return CancelPolicy(allowed=True, refund_percent=0 if by is Actor.CUSTOMER else 100)
        case OrderState.READY | OrderState.PICKED_UP:
            return CancelPolicy(allowed=False, reason="the order has left the restaurant")
        case _:
            return CancelPolicy(allowed=False, reason=f"cannot cancel a {order.state.name} order")
```

Not a boolean on the order. A **question asked of the state**, and the answer depends on who is asking — a restaurant cancelling after accepting is a refund to the customer, while a customer cancelling after the food is cooked is not. Shruthi's third rule, and the reason customers accept it once it is explained.

The partner assignment decision, which is the genuinely interesting trade

When do you assign a delivery partner?

At **ACCEPTED** — as soon as the restaurant says yes. The partner arrives during cooking, so the food goes out the instant it is ready. But the partner waits at the restaurant, unpaid, unable to take another job, and if the kitchen is slow that is fifteen wasted minutes.

At **READY** — when the food is actually done. No partner waiting. But now the food sits getting cold while somebody is found and travels there, which is five to ten minutes on average.

In practice: at **ACCEPTED** plus the predicted prep time minus the predicted travel time, so the partner arrives as the food does. That turns the assignment into a prediction problem, and the honest statement is that the *design* is a scheduled assignment with a predicted timestamp, and the *prediction* is a separate system.

Say all three. The interviewer is looking for whether you notice there is a choice at all — most candidates assign at one of the ends without remarking on it.

The one-restaurant rule

A cart holds items from **one** restaurant, and that is not an arbitrary simplification. It means one order has one pickup point, one acceptance, one prep time and one partner. Multi-restaurant orders turn one order into several sub-orders with independent states and a join at the end, which is a materially larger design — and it is exactly why the apps that support it charge extra for it.

Name the restriction and say what relaxing it would cost. That is worth more than silently assuming it.

4 The picture

The state machine, with the actor on every arrow — which is the thing to draw:

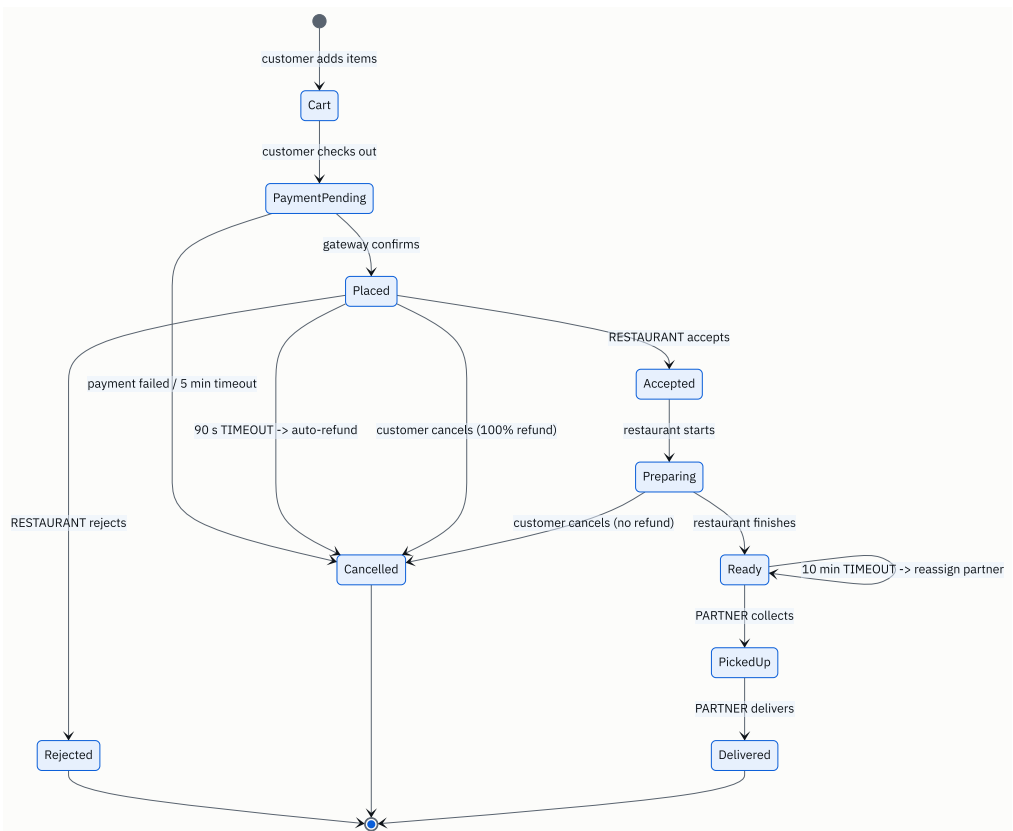


FIGURE 87.1

What to notice: **three different kinds of arrow leave Placed** — the restaurant accepts, the restaurant rejects, or nobody does anything and the clock fires. That third arrow is the one candidates leave off, and it is the one that matters at scale: at two million orders a day and a five percent no-response rate, it fires a hundred thousand times.

The three parties and what each one can do:

	CUSTOMER	RESTAURANT	PARTNER	CLOCK
Cart	add/remove	-	-	-
PaymentPending	-	-	-	cancel @5m
Placed	cancel (100%)	accept / reject	-	cancel @90s
Accepted	cancel (0%)	start preparing	-	-
Preparing	cancel (0%)	mark ready	-	alert @est+10m
Ready	-	-	pick up	reassign @10m
PickedUp	-	-	deliver	alert @eta+20m
Delivered	rate	-	-	-

every blank in the CLOCK column is a decision, not an omission

And the classes:

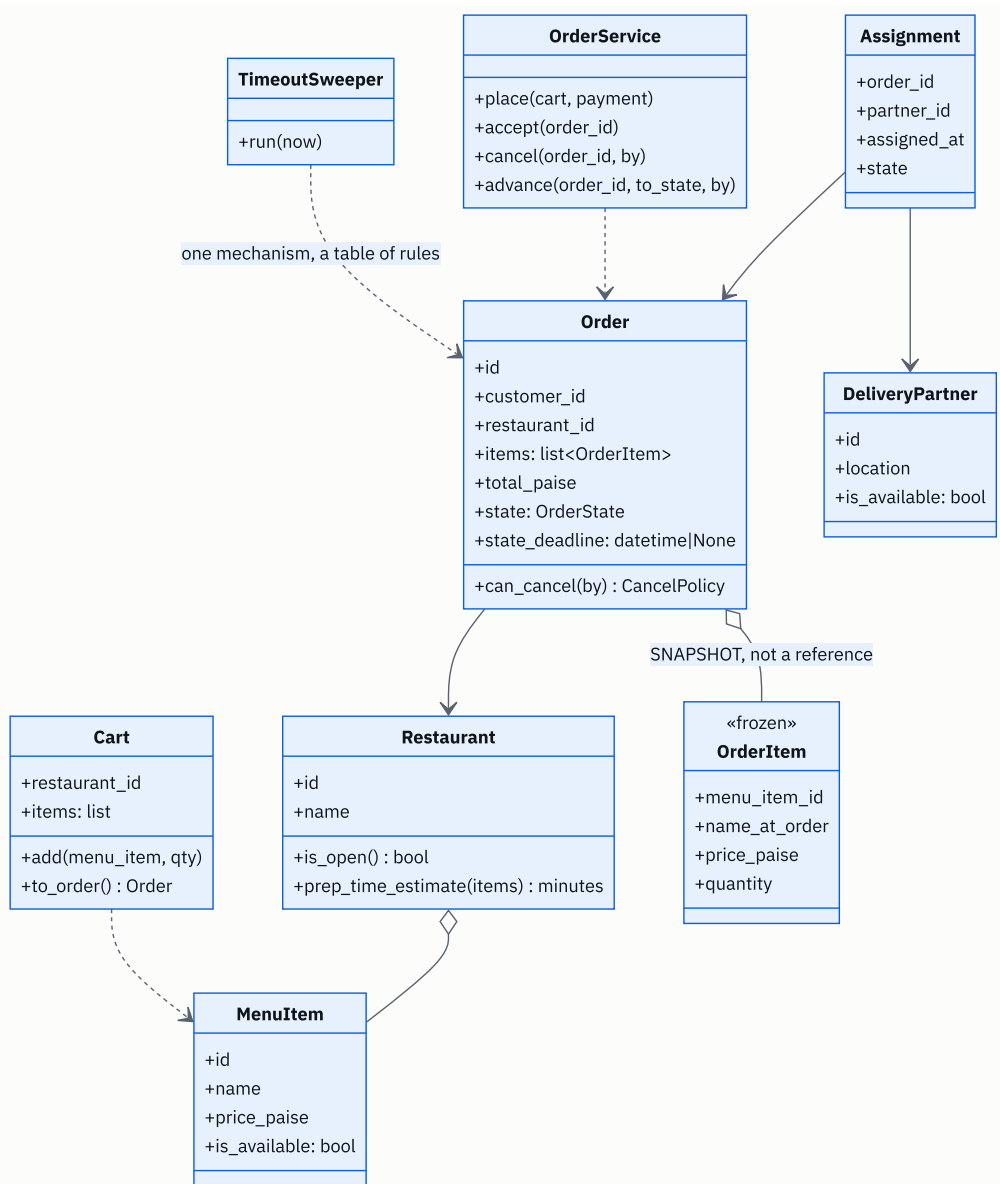


FIGURE 87.2

What to notice: **OrderItem** copies the name and price rather than pointing at **MenuItem**. That dashed comment is the difference between an order that means something a year later and one that silently re-prices itself.

5 How it actually works

Move 1 · Clarify (minutes 0–5)

“One restaurant per order, or several?” — One, and I would say what relaxing it costs. **“What happens if the restaurant does not respond?”** — This is the question I would ask first, because the answer is the design. Propose: auto-cancel and refund after ninety seconds. **“Is the delivery partner assigned before or after the food is ready?”** — The interesting trade, and worth surfacing early. **“Can the customer cancel, and is there a refund?”** — Yes, and it depends on the state.

“I will assume payment is upfront through a gateway behind an interface, that prices are integer paise frozen at order time, and that partner matching and ETA prediction are separate systems I will call into. I am not designing search, recommendations, or the maps.”

Move 2 · The nouns (minutes 5–12)

- `Restaurant` , `MenuItem` — the catalogue. `MenuItem` has a live price and availability.
- `Cart` — the customer’s working set, tied to one restaurant. Not persisted as an order.
- `Order` — the record. State, deadline, totals.
- `OrderItem` (*frozen*) — a **snapshot** of name, price and quantity.
- `DeliveryPartner` , `Assignment` — who is carrying it, and since when.
- `OrderService` — the transitions, and nothing else.
- `TimeoutSweeper` — one mechanism driven by a table of per-state rules.

Seven, and note there is no `OrderStateMachine` class: the states are an enum plus a transition table, because here the states carry very little behaviour of their own — the *actors* and *deadlines* are what differ. That is a deliberate difference from [day 073](#)’s State pattern, where each state carried real work, and saying why you chose differently is worth more than applying the pattern reflexively.

Move 3 • Placing the order — the snapshot

```
def place(self, cart: Cart, customer_id: str, now: datetime) → Order:
    restaurant = self._restaurants.get(cart.restaurant_id)
    if not restaurant.is_open(now):
        raise NotAllowed("the restaurant is closed")

    items = []
    for line in cart.lines:
        menu_item = self._menu.get(line.menu_item_id)
        if not menu_item.is_available:
            raise NotAllowed(f"{menu_item.name} is no longer available")
        items.append(OrderItem(                # SNAPSHOT, not a reference
            menu_item_id=menu_item.id,
            name_at_order=menu_item.name,
            price_paise=menu_item.price_paise,
            quantity=line.quantity,
        ))
```

Validate against the *live* menu, then **copy** what you validated. The order now means the same thing in a year's time as it does today.

```
subtotal = sum(item.price_paise * item.quantity for item in items)
order = Order(
    id=new_id(), customer_id=customer_id, restaurant_id=restaurant.id,
    items=items,
    subtotal_paise=subtotal,
    delivery_fee_paise=self._fees.delivery(restaurant, cart.address, now),
    tax_paise=self._fees.tax(subtotal),
    state=OrderState.PAYMENT_PENDING,
    state_deadline=now + timedelta(minutes=5),
)
```

Every money component is stored, not just the total. When a customer disputes a bill eight months later, “subtotal 480, delivery 35, tax 24, total 539” answers it; a single number does not.

Move 4 • The transition, with the actor and the deadline

```
def advance(self, order_id: str, to_state: OrderState, by: Actor, now: datetime) → Order:
    order = self._orders.get_for_update(order_id)    # row lock: one transition at a time

    allowed = TRANSITIONS.get((order.state, to_state))
    if allowed is None:
        raise IllegalTransition(f"{order.state.name} → {to_state.name} is not a legal move")
    if by not in allowed.actors:
        raise NotAllowed(f"a {by.name.lower()} cannot make this order {to_state.name}")

    order.state = to_state
    rule = RULES.get(to_state)
    order.state_deadline = now + rule.timeout if rule and rule.timeout else None
    self._history.append(Transition(order.id, order.state, to_state, by, now))
    self._events.publish(OrderStateChanged(order.id, to_state))
    return order
```

Four things in one place, and that is the point of routing every change through one method:

The transition table decides legality, so an illegal move is impossible rather than merely discouraged. **The actor check** stops a customer marking their own order delivered. **The deadline** is set from the table, so no state can accidentally be left without one. And **the history row plus the published event** happen for every transition, which gives the audit trail and the notifications for free — the observer hook from [day 072](#).

Move 5 • The sweeper

```
class TimeoutSweeper:
    def run(self, now: datetime) → int:
        overdue = self._orders.query(
            "SELECT * FROM orders WHERE state_deadline IS NOT NULL AND state_deadline < %s "
            "ORDER BY state_deadline LIMIT 500 FOR UPDATE SKIP LOCKED",
            now,
        )
        for order in overdue:
            RULES[order.state].on_timeout(order, now)
        return len(overdue)
```

`FOR UPDATE SKIP LOCKED` is the line worth explaining: it lets several sweeper instances run at once without two of them picking up the same order, and without them blocking each other. That is how you make a sweeper horizontally scalable, and it is one clause.

The rules themselves are small:

```
def auto_cancel_and_refund(order: Order, now: datetime) → None:
    """PLACED for 90 seconds with no response from the restaurant."""
    service.advance(order.id, OrderState.CANCELLED, Actor.SYSTEM, now)
    payments.refund(order.payment_id, reason="restaurant did not respond")
    notify(order.customer_id, "Sorry – the restaurant could not take your order. Refunded.")
    metrics.increment("orders.auto_cancelled.no_restaurant_response")
```

The metric is not decoration. **The rate at which this fires is a restaurant quality signal**, and it is the input to deciding whether that restaurant should stay in search results.

Move 6 • Partner assignment

```
def schedule_assignment(order: Order, now: datetime) → None:
    prep = predictor.prep_minutes(order) # a separate system
    travel = predictor.travel_minutes_to(order.restaurant_id)
    assign_at = now + max(timedelta(0), prep - travel)
    queue.schedule(AssignPartner(order.id), at=assign_at)
```

Assign so the partner **arrives when the food does**. And the honest sentence to say: the design here is “assign at a predicted moment”, and the quality of the prediction is a different system with its own orders and its own failure modes. Do not pretend the prediction is the easy part.

Real systems

- **Swiggy, Zomato, Uber Eats and DoorDash** all show the customer a state machine — a progress bar with named steps — which is the state machine leaking deliberately into the interface, because the main complaint about food delivery is not lateness, it is not knowing.
- **Restaurant acceptance timeouts** are real and short, typically 60–120 seconds, and the auto-rejection rate is a published quality metric for restaurants on these platforms.
- **Order price snapshots** are why your invoice from six months ago still shows the old price. It is also a legal requirement in most tax regimes: the invoice must reflect what was charged.
- `SELECT ... FOR UPDATE SKIP LOCKED` is the standard Postgres pattern for a work queue and is what lets several sweepers share the same table. Many teams reach for Kafka here and a table plus SKIP LOCKED is enough for a long time.
- **Assignment as a scheduled job** rather than an immediate one is the difference between a partner network that idles and one that does not; DoorDash has published on assigning by predicted ready time for exactly this reason.

6 The numbers

Traffic

```
orders per day          2,000,000
peak: 40% in the two dinner hours
      800,000 / 7,200 s      ≈ 111 orders/second
state transitions per order ~8
      → 16,000,000 transitions/day, ~890/second at peak
```

One hundred and eleven orders a second is not large, and it is worth saying so — the difficulty in this system is correctness across three parties, not throughput.

The timeout that actually fires

```
restaurants failing to respond within 90 s: ~5%
2,000,000 × 5% = 100,000 auto-cancellations per day
               = ~5.5 per second at peak
```

A hundred thousand times a day. That is not an edge case; it is a feature with its own refund pipeline, its own notification and its own metric. A design that treats it as an exception has designed for a system that does not exist.

Storage

Order row	$\sim 500 \text{ B} \times 2\text{M/day}$	$= 1.0 \text{ GB/day}$	$= 365 \text{ GB/year}$
OrderItem row	$\sim 120 \text{ B} \times 3 \text{ items} \times 2\text{M}$	$= 720 \text{ MB/day}$	
Transition row	$\sim 100 \text{ B} \times 8 \times 2\text{M}$	$= 1.6 \text{ GB/day}$	

total		$\approx 3.3 \text{ GB/day}$	$\approx 1.2 \text{ TB/year}$

The transition history is **half the storage**, which is the moment to decide deliberately: keep 90 days hot for support and archive the rest to object storage. Deleting it is the wrong answer — it is what answers “why was I charged for this?”.

The price snapshot, priced

menu items changed per day: $\sim 2\%$ of a restaurant's items
orders touching a changed item: $\sim 2\% \times 2,000,000 = 40,000/\text{day}$

without a snapshot: 40,000 orders/day whose price could be recomputed differently
at an average dispute rate of 1%: 400 disputes/day $\rightarrow \sim 146,000/\text{year}$
at a ₹50 support cost each: ₹73 lakh/year

Seventy-three lakh a year, from one denormalisation decision. That is the argument for the snapshot, and it is far more convincing than “orders should be immutable”.

The assignment trade, in minutes

assign at ACCEPTED:
partner waits at the restaurant $\approx 8 \text{ min average}$
at 2M orders/day: 16,000,000 partner-minutes/day wasted $= 266,000 \text{ partner-hours}$

assign at READY:
food waits for a partner $\approx 6 \text{ min average}$
 $\rightarrow 6 \text{ minutes colder, and } 6 \text{ minutes later, on every order}$

assign at (ACCEPTED + prep - travel):
both waits $\approx 1\text{-}2 \text{ min}$, subject to prediction error

Two hundred and sixty-six thousand partner-hours a day is the cost of assigning too early, and six minutes of cold food on every order is the cost of assigning too late. That comparison is the whole argument for predicting.

The sweeper

orders with a live deadline at any moment: $\sim 111/\text{s} \times \sim 5 \text{ min of exposure} \approx 33,000$
sweeper runs every 5 s, batch 500, several instances
 $\rightarrow$ comfortably keeps up; the deadline column needs an index

A single index on `state_deadline` is the difference between a sweeper that scans two million rows and one that touches a few hundred. Worth one sentence.

7 The trade-offs

What this design gives up

The transition table plus an enum is not the State pattern, and that is deliberate. Here the states carry almost no behaviour of their own — what differs is who may act and what the deadline is, and both are data. Using a class per state would put five nearly-empty classes in the codebase. But it means an illegal transition is caught by a lookup rather than by the type system, and a typo in the table is a runtime failure. If states started carrying real work — different pricing while preparing, different refunds by state — I would move to classes.

One sweeper is a single point of failure and a single point of correctness. If it stops, nothing times out and orders pile up in `PLACED` invisibly. So the sweeper needs its own alert on “time since last successful run”, and that alert matters more than most of the application’s own monitoring. The alternative — a scheduled job per transition — spreads the risk across six things that can each fail quietly, which is worse.

Denormalising the order items means the order and the menu can disagree, and reporting has to choose which one it means. “Revenue for dish X” is a different number depending on whether you join to the menu or read the snapshot. That is real and it is the right trade, and the answer is that the snapshot is authoritative for money and the menu for the catalogue.

Cancellation policy in code will end up in configuration. Refund percentages by state, by city, by whether the restaurant has already started — those change by business decision, on a Monday, and no engineer should deploy for them. I would build it as a policy object first and move it to a rules table on the second change.

Nothing here handles partial fulfilment. One item out of four is unavailable after acceptance is extremely common, and the honest options are: cancel the whole order, or substitute, or refund the line and deliver the rest — each of which needs the customer’s consent while the food is cooking. That is a whole conversation flow and I would scope it out explicitly rather than pretend the state machine covers it.

No multi-restaurant orders. One order, one pickup, one acceptance, one prep time. Relaxing that turns one order into several sub-orders with independent states, a join at the end, and a partner routing problem — which is why platforms charge extra for it.

“I would change this design if...”

- **...the restaurant response rate got worse.** Then `PLACED` needs a fallback to a backup restaurant rather than a cancellation, which changes the state machine and needs the customer’s consent.

- **...orders could contain several restaurants.** Sub-orders with their own states, and the parent order becomes a coordinator — closer to a saga than a state machine.
- **...cancellation rules changed more than twice a year.** A rules table a support lead can edit, not classes an engineer deploys.
- **...the volume grew tenfold.** The sweeper would need partitioning by deadline bucket, and the transition history would move to a separate store. Neither changes the model.

The honest concession

The state machine is the easy half and it is what everybody draws. The half that decides whether the system works is the **timeout column** — the fact that every transition performed by somebody else has a deadline and a defined thing that happens when it passes. A design with eight states and no clock is a design where a hundred thousand orders a day sit silently in `PLACED`, and the customer finds out at one o'clock, like Shruthi's eleven customers.

8 In the interview

How it gets asked

- The standard: *“Design the order flow for a food delivery app.”*
- The failure probe, which is the real question: *“What happens if the restaurant never accepts?”*
- The money probe: *“The restaurant changes the price while the order is in the cart. What is charged?”*
- The trade probe: *“When do you assign the delivery partner?”*
- The policy probe: *“Can the customer cancel? When, and is there a refund?”*

The timed script

Minutes 0–5 · Clarify. One restaurant per order? What happens when the restaurant does not respond — ask this first, because the answer is the design. Partner assigned before or after ready? Cancellation and refunds?

Minutes 5–10 · Estimation. Two million orders a day, 111 a second at peak, and — the number that matters — a five percent restaurant no-response rate meaning a hundred thousand automatic cancellations a day. Say that early: it turns the timeout from an edge case into a feature.

Minutes 10–20 · The state machine, drawn with the actor on every arrow. Then the timeout column, which is the part most candidates omit.

Minutes 20–28 · The two immutability decisions. The price snapshot with the dispute arithmetic, and the transition history.

Minutes 28–35 · The interesting trade: partner assignment, with the partner-hours-versus-cold-food comparison and the prediction as a separate system.

Minutes 35–40 · Failure modes. The sweeper as a single point of correctness, partial availability after acceptance, and what would change at ten times the volume.

The follow-ups

“What happens if the restaurant never accepts?” “That is the transition I would design first, because it happens about five percent of the time — a hundred thousand times a day at two million orders. The order sits in `PLACED` with a deadline ninety seconds out. A sweeper picks up anything past its deadline, cancels it, refunds automatically, notifies the customer immediately rather than at dinner time, and increments a metric — because the rate at which this fires is a restaurant quality signal and feeds back into search ranking. The important design point is general: **every transition performed by somebody else needs a deadline and a defined thing that happens when it passes**. A state with an actor and no timeout is an order that can sit for ever, and silence is what customers actually complain about.”

“The restaurant changes the price while the order is in the cart.” “The customer is charged what they saw. `OrderItem` stores a *snapshot* — the name, the price and the quantity at the moment of ordering — rather than a reference to the menu item, and I store every money component separately, not just the total. It looks like denormalisation and it is deliberate: an order is a record of what was agreed, and joining to the live menu means an order silently re-prices itself after the fact. There is an arithmetic argument too — about two percent of menu items change daily, so roughly forty thousand orders a day touch a changed item, and even a one percent dispute rate on those is a few hundred support tickets a day.”

“When do you assign the delivery partner?” “There is a genuine trade here and I would name all three options. At acceptance, the partner arrives during cooking and the food goes out instantly — but the partner waits about eight minutes on average, unable to take another job, which across two million orders a day is roughly two hundred and sixty thousand partner-hours. At ready, no partner waits — but the food sits about six minutes getting cold while somebody is found and travels. So in practice you assign at a *predicted* moment: acceptance plus prep time minus travel time, so the partner arrives as the food does. That makes the design a scheduled assignment and pushes the difficulty into a prediction system, which I would name as a separate component with its own failure modes rather than pretend it is easy.”

“Can the customer cancel?” “It depends on the state, and I would model it as a question you ask the order rather than a flag on it. Before the restaurant accepts, yes, full refund, nothing has been lost. While it is being prepared, allowed but no refund, because ingredients have been spent. Once the partner has collected it, no — the food is on a scooter and it is not coming back. And the answer also depends on *who* is asking: a restaurant cancelling after accepting is a full refund to the customer. Those percentages will end up in configuration rather than code, because they change by business decision, but the *shape* — a policy derived from state and actor — is the design.”

“How do the timeouts actually fire?” “Each order carries a `state_deadline` set from a per-state rules table when it enters that state, and one sweeper queries for orders past their deadline and applies that state’s rule. One mechanism with a table beats six scheduled jobs that can each fail quietly. The query uses `FOR UPDATE SKIP LOCKED` so several sweeper instances can run without picking up the same order or blocking each other, and the deadline column is indexed. And I would alert on time-since-last-successful-sweep, because the sweeper is a single point of *correctness* — if it stops, nothing times out and orders pile up invisibly.”

“Why not a class per state, like the State pattern?” “Because here the states carry very little behaviour of their own. What differs between them is who may act and what the deadline is, and both of those are data — so a transition table plus an enum expresses it directly and a class per state would be five nearly-empty classes. I would switch to classes if states started carrying real work: different refund arithmetic per state, or a state that has to call out to another system on entry. That is the same judgement as any pattern — the gate is whether the implementations differ in *behaviour* or only in *values*.”

“An item becomes unavailable after the restaurant accepts.” “Very common, and the state machine alone does not cover it. The options are to cancel the whole order, to substitute, or to refund that line and deliver the rest — and all three need the customer’s consent while the food is cooking, so it is a conversation flow with its own timeout, not just a transition. I would scope it explicitly rather than pretend it falls out of the design, and I would note that the refund-the-line option needs the per-line prices I already store, which is a second reason for the snapshot.”

A model answer

Asked: *design the order flow for a food delivery app.*

“Let me start with the question I think decides this design, and ask it rather than assume: what happens when the restaurant does not respond? Because the interesting part of this system is not the eight happy states, it is the fact that three different parties move the order forward and any of them can simply not act.

The states are roughly: cart, payment pending, placed, accepted, preparing, ready, picked up, delivered, plus rejected and cancelled. Everybody draws that. The column that matters is next to it: **who causes each transition, and what happens if they do not**. The customer places. The restaurant accepts and cooks. The partner collects and delivers. And the clock is the third actor — placed times out after ninety seconds and auto-cancels with a refund; ready times out after ten minutes and reassigns the partner; picked up times out and alerts operations.

That timeout is not an edge case. At two million orders a day and a five percent no-response rate, it fires a hundred thousand times a day — so it needs a refund pipeline, an immediate customer notification, and a metric, because the rate at which it fires is a restaurant quality signal.

I would implement time as one mechanism rather than six. Each order carries a `state_deadline`, set from a per-state rules table when it enters that state, and a single sweeper picks up anything past its deadline and applies that state’s rule. One thing to keep correct instead of six things that can each fail quietly — and I would alert on how long since the sweeper last succeeded, because if it stops, nothing times out and orders pile up invisibly.

The second decision is immutability. `OrderItem` stores a *snapshot* of the name and price at the moment of ordering, not a reference to the menu item, and I store the subtotal, the delivery fee and the tax separately as well as the total. An order is a record of what was agreed. About two percent of menu items change on a given day, so roughly forty thousand orders a day touch a changed item — without the snapshot, every one of those could be re-priced after the fact, and that is where billing disputes come from.

Third, cancellation is a question you ask the state, not a flag. Before acceptance: allowed, full refund. While preparing: allowed, no refund, because ingredients are spent. After pickup: not allowed — the food is on a scooter. And the answer depends on who is asking, since a restaurant cancelling after accepting refunds the customer fully. Those percentages will move to configuration, but the shape is a policy derived from state and actor.

The one genuinely interesting trade is when to assign the delivery partner. At acceptance, the food leaves instantly but the partner waits about eight minutes — across two million orders that is roughly two hundred and sixty thousand partner-

hours a day. At ready, nobody waits but the food sits six minutes getting cold. So you assign at a predicted moment — acceptance plus prep time minus travel time — which makes it a scheduled job and pushes the difficulty into a prediction system with its own data and its own failure modes. I would name that as a separate component rather than pretend it is easy.

On scale: a hundred and eleven orders a second at peak, about three gigabytes a day of storage, half of it transition history. That is not a throughput problem. The difficulty here is correctness across three parties who can each fall silent, which is why I would spend the design time on the timeout column rather than on the states.”

9 Recall card

- **Everybody draws the eight states; the column that matters is next to them — WHO causes each transition, and WHAT HAPPENS IF THEY DO NOT.** Three parties (customer, restaurant, partner) plus **the clock as a third actor**. *A transition performed by somebody else with no timeout is an order that sits for ever, and silence is what customers actually complain about.*
- **The restaurant timeout is a feature, not an edge case: ~5% of 2M orders/day = 100,000 auto-cancels a day**, each needing a refund, an *immediate* notification, and a metric that feeds restaurant ranking. Implement time as **one sweeper plus a per-state rules table** (`state_deadline`, indexed, `FOR UPDATE SKIP LOCKED` so instances share the work) — one thing to keep correct instead of six that fail quietly. **Not** lazy expiry, because something must actively *happen*.
- **OrderItem** is a **SNAPSHOT** of name and price, not a reference to the menu — and store every money component, not just the total. ~2% of menu items change daily → ~40,000 orders/day touch a changed item, which is where billing disputes come from. Same immutability rule as a loan and a booking.
- **Cancellation is a question you ask the state, and the answer also depends on the actor.** Placed → full refund; preparing → allowed, no refund (ingredients spent); picked up → refused, *the food is on a scooter*. Expect the percentages to move to configuration; the *shape* stays.
- **The one genuinely interesting trade is when to assign the partner.** At ACCEPTED: the partner waits ~8 min → ~266,000 partner-hours/day. At READY: the food waits ~6 min and arrives cold. So assign at **ACCEPTED + prep – travel**, which makes it a scheduled job and moves the difficulty into a *prediction* system — name that as separate rather than pretend it is easy. And at **111 orders/s**, this is a correctness problem, not a throughput one.

DSA topic: Recursion: the leap of faith **System design topic:** Design a food delivery order flow

Code these, in this order

One rule for the whole set: **answer the three questions out loud before typing**. What is the smallest input I can answer without asking anybody? What is a smaller version of the same problem? What one step turns that answer into mine? If you cannot say all three, you are not ready to write the function.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Fibonacci Number	LeetCode 509 (Easy)	The classic, and whether you notice it is exponential before being told.
2	Reverse String	LeetCode 344 (Easy)	Recursion with two indices instead of slicing, in $O(1)$ extra space.
3	Pow(x, n)	LeetCode 50 (Medium)	Halving instead of decrementing, negative exponents, and calling the recursion once .
4	Merge Two Sorted Lists	LeetCode 21 (Easy)	You solved this iteratively on day 084. Now do it recursively and compare the two.

On problem 1, measure before you optimise

Write the naive version. Time `fib(30)`, then `fib(32)`, then `fib(34)`. Write the three numbers down and say what the ratio between them tells you. Then add `@cache` and time `fib(100)`.

On problem 2, do not slice

The obvious recursive reverse allocates a new string per call. Write it with two indices moving toward each other instead, and say what you changed the complexity from and to.

On problem 3, count your recursive calls

Write the version that calls `fast_power` twice for the even case. Time it against the version that calls it once and stores the result. Explain the difference in one sentence about branching.

On problem 4, compare honestly

Write both. Say which you would ship and why, in terms of stack depth on a hundred-thousand-node list.

The three-questions drill

For each, state the base case, the smaller problem and the one step — out loud, before coding:

1. Sum a list of numbers.
2. Count the nodes in a linked list.
3. Reverse a string.
4. Raise a number to a power.
5. Check whether a string is a palindrome.
6. Find the maximum of a list.
7. Count the digits in a positive integer.
8. Flatten a nested list of lists.

Two of the eight have a base case that is *not* “empty”. Name them.

The leap-of-faith drill

1. Write `total` and then, without running it, say why it is correct — using the three checks rather than a trace.
2. Try to trace `total([1,2,3,4])` four levels deep in your head. Notice how it feels.
3. State the three conditions that replace tracing.
4. Say, in one sentence, what treating the recursive call as a library function buys you.

The base-case drill

1. Write a function with no base case. Quote the error.
2. Write one whose base case is present but unreachable — step by two towards `= 0`. Quote the error and say why it is harder to diagnose.
3. Fix it two ways: by changing the guard, and by changing the step. Say which is the real fix.
4. State both halves of the termination condition in one sentence.

The one-call-versus-two drill

1. Draw the call tree for `total([1,2,3])`.
2. Draw the call tree for `fib(5)`.
3. Count the nodes in each.

4. State the complexity of each and say what in the *body* predicts it.
5. Give a third example of each kind.

The break-it drill

Trigger each and record the exact output or error text:

1. Omit the base case.
2. Put the base case *after* the recursive call.
3. Forget to `return` the recursive call. Quote the error one level up.
4. Slice in the recursive call and time it at $n = 2000$ against the index version.
5. Call the recursion twice in `fast_power`'s even branch and time it.
6. Use a mutable default argument and call the function twice.
7. Run a depth-5000 recursion. Quote the error.
8. Raise the recursion limit to a million and run it again. Describe what happens.

The cost drill

1. State the time complexity of `total`, `power`, `fast_power` and `fib`, and where each comes from.
2. State the *space* of each, and explain why `fib` is $O(n)$ space despite 2^n calls.
3. Quote Python's recursion limit and the input size at which a linear-depth recursion dies.
4. Say why `sys.setrecursionlimit` is not a fix.
5. Give the rule about log-depth versus linear-depth recursion in one sentence.

The conversion drill

1. Write `total` as a loop.
 2. Write `fib` as a loop.
 3. Write binary search as a loop.
 4. For each, say whether the loop is better and why.
 5. Say when you would choose recursion, and give two problems where the loop version is clearly worse.
-

The actors drill

1. List the eight order states.
2. For each transition, name the actor.
3. For each transition where the actor is not the platform, give a timeout and what happens when it fires.

4. Say which transitions have no timeout and defend each one.
5. State the general rule in one sentence.

The timeout drill

1. Compute the auto-cancellations per day at 2M orders and a 5 percent no-response rate.
2. Say what must happen on each one — list four things.
3. Say why a metric is one of them.
4. Describe the sweeper: the query, the batch, and the clause that lets several run at once.
5. Say what you would alert on, and why that alert matters more than most.
6. Say why lazy expiry works for a cinema seat and not here.

The snapshot drill

1. Write `OrderItem` and mark the fields that are copies rather than references.
2. Say what goes wrong without the snapshot, concretely.
3. Compute the orders per day touching a changed menu item.
4. Say why the subtotal, delivery fee and tax are stored separately from the total.
5. Name two other lessons in this course with the same immutability rule.

The cancellation drill

For each, say whether cancellation is allowed, the refund, and the reason in the customer's language:

1. Customer, order placed, restaurant has not responded.
2. Customer, order accepted, food not started.
3. Customer, food being prepared.
4. Customer, partner has collected the food.
5. Restaurant, after accepting.
6. System, after ninety seconds with no restaurant response.

Say what shape this logic takes in code, and why it is not a boolean on the order.

The assignment drill

1. State the three options for when to assign a partner.
2. Compute the wasted partner-hours per day for the earliest option.
3. State the cost of the latest option in minutes per order.
4. State the middle option as a formula.

5. Say what that formula turns the problem into, and why you would name it as a separate system.

The estimation drill

Compute each, showing the multiplication:

1. Peak orders per second, given 40 percent of 2M orders in two hours.
2. State transitions per day and per second at peak.
3. Storage per day for orders, items and transitions.
4. The fraction of storage that is transition history, and what you would do about it.
5. The annual support cost of not snapshotting prices.

The pattern-choice drill

1. Say why this design uses a transition table and an enum rather than a class per state.
 2. Say what would make you switch to classes.
 3. Name the lesson where a class per state was the right answer, and what was different there.
 4. State the general gate in one sentence.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Solve this recursively. What is the base case?* The three questions answered in order before any code, the leap of faith named, the base case written first, both halves of termination, the index-not-slice point, and both complexities with the stack limit stated as a real limit.
 2. *Design the order flow for a food delivery app.* Ask the restaurant-timeout question first, the states with an actor on every arrow, the timeout column with the hundred-thousand-a-day number, one sweeper with a rules table, the price snapshot with its dispute arithmetic, cancellation as a question asked of the state, and the assignment trade with both costs.
 3. *Write Fibonacci recursively. What is its complexity?* Two recursive calls per level so about 2^n , the measured numbers, memoisation turning 331 million calls into 41, the loop as what you would ship, and the general rule about counting calls in the body.
-

Before you move on

- [] I answer the three questions out loud before writing any recursive function.
- [] I can state the three checks that replace tracing.
- [] I write the base case first, at the top, every time.
- [] I can state both halves of the termination condition.
- [] I built an unreachable base case on purpose and can say why it is the harder bug.
- [] I can predict a recursion's complexity by counting the calls in the body.
- [] I timed `fib(30)` , `fib(32)` and `fib(34)` and can say what the ratio means.
- [] I added `@cache` and can say what it changed and what it is the foundation of.
- [] I never slice in a recursive call, and I can say what it costs.
- [] I can quote the recursion limit and the input size where it bites.
- [] I can say why raising the limit is not a fix.
- [] I can name every actor on every order transition.
- [] I can give the auto-cancellation number and the four things that must happen.
- [] I can describe the sweeper, including the clause that lets several run at once.
- [] I can explain the price snapshot with its arithmetic.
- [] I can give all three partner-assignment options with their costs.
- [] I answered all three questions above out loud.

The call stack, drawn

DATA STRUCTURES AND ALGORITHMS

The call stack, drawn

You can trace a recursive call by hand and read a `RecursionError` correctly.

SYSTEM DESIGN

Design a ride-hailing booking flow

You can model riders, drivers, matching and trip state at the object level.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Trace this recursion. What is on the stack at the deepest point?*
- *Design the booking flow for a cab app.*

1 What this is, and why they ask it

Yesterday you learned not to trace recursion. Today you learn to trace it anyway — because an interviewer will ask you to, because a bug eventually forces you to, and because understanding what the machine is actually doing is what turns `RecursionError` from a mystery into a diagnosis.

When a function calls another, the machine has to remember three things about the caller: **its arguments and local variables**, and **where to resume** when the call returns. That bundle is a **stack frame**, and the frames pile up in a stack — the same last-in-first-out structure from [day 068](#), except this one is built into the language.

Two ideas do most of the work today. **The recursion tree and the call stack are different things**: the tree is every call that ever happens, and the stack is only the calls that are live at one instant. Fibonacci of twenty makes 21,891 calls and never has more than twenty frames on the stack. And **the depth is the space cost**, which is why a function making a billion calls can be fine and a function making five thousand can crash.

They ask it because “trace this” is a five-minute question that instantly separates people, because reading a traceback correctly is a daily skill, and because “convert this recursion to an iteration” — which is the same question wearing a hat — comes up whenever depth is a problem.

2 The story

Vasant’s workshop is one room off the main road with a shutter that only half opens, and he has been repairing scooters in it since 1994.

The work never goes in a straight line. A man brings a scooter that will not start. To check the plug, Vasant has to get the side panel off. To get the side panel off he has to take out two screws that are behind the footboard, and one of them is rounded, so now he has to find the extractor, and the extractor is in the box under the bench that has the chain tool sitting on top of it.

So at any moment he is not doing one job. He is in the middle of four.

What he does — and he has done it so long that he does not think of it as a system — is the shelf of trays. There are nine metal trays on a shelf behind him. When he has to stop what he is doing to do something else first, everything from the current job goes in a

tray: the screws, the washer, the spanner he was using, all of it. And on the side of the tray he writes one line in chalk. Not what the job is. **What he was in the middle of when he stopped.** “Second screw, footboard, left.” That line is the whole point of the tray.

Then the tray goes on the shelf and he starts the next thing with clean hands.

When that next thing is finished, he takes the top tray down — always the top one, never one from the middle — reads the chalk, and carries on from exactly there. He does not re-read the whole job. He does not work out where he was. The line tells him.

Two things about the shelf. It holds nine trays, and that is a real limit. Twice in thirty years a job has gone so deep that he ran out of shelf, and both times the answer was to stop, put everything on the floor in order, and start again more carefully — not to balance a tenth tray on top of the ninth.

And the trays come down in the opposite order to the way they went up. Always. The last thing he set aside is the first thing he comes back to, and the scooter that started the whole afternoon is the last tray, at the bottom, with a chalk line on it that says “plug”.

3 The idea in plain English

Each tray is a **stack frame**. The chalk line is the **return address**. The shelf is the **call stack**. And the nine trays are Python’s recursion limit.

What is in a frame

When a function is called, the machine creates a frame holding:

- **the arguments** it was called with,
- **the local variables** it creates,
- **the return address** — the exact point in the caller to resume at,
- and a link to the frame below it.


```

+-----+
| total(numbers, start=2) | ← top of stack: the call running right now
|   locals: start=2      |
|   return to: line 4 of  |
|           the frame    |
|           below        |
+-----+
| total(numbers, start=1) | ← suspended, mid-addition
+-----+
| total(numbers, start=0) | ← suspended, mid-addition
+-----+
| main()                  |
+-----+

```

Every frame except the top one is **suspended in the middle of an expression**, waiting for the frame above it to return a value. That is Vasant's tray with the screws still in it and the chalk line saying where he stopped.

Tracing, properly

The method: **draw the stack growing downward, and write what each frame is waiting for.**

`total([4, 7, 2])` where `total` returns `numbers[start] + total(numbers, start + 1)`:

CALL PHASE – frames pile up, each suspended mid-addition

```

total(start=0)   waiting on:  4 + ?
total(start=1)   waiting on:  7 + ?
total(start=2)   waiting on:  2 + ?
total(start=3)   base case → returns 0           ← deepest point: 4 frames

```

RETURN PHASE – frames come off in reverse order, each completing its addition

```

total(start=3) → 0
total(start=2) → 2 + 0 = 2
total(start=1) → 7 + 2 = 9
total(start=0) → 4 + 9 = 13

```

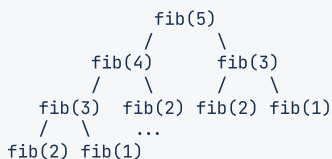
The work happens on the way back up. On the way down nothing is computed — every call just gets suspended. That is why a recursion of depth `n` holds `n` frames of memory at its deepest moment, and why a recursion that does its work on the way *down* and returns nothing useful can sometimes be rewritten as a loop with no stack at all.

The recursion tree is not the call stack

This is the idea that answers most interview questions in this area.

The **recursion tree** shows every call that ever happens, laid out over time. The **call stack** shows only the calls that are alive at one instant — which is a single root-to-current path in that tree.

fib(5) — the TREE has 15 nodes (measured)



the STACK, at its deepest, holds 5 frames: fib(5) → fib(4) → fib(3) → fib(2) → fib(1)

fib(20): 21,891 calls (measured) but the stack never exceeds 20 frames

So:

Total calls determine the time. Maximum depth determines the space.

fib is exponential in time and only linear in space, because it finishes the whole left branch — and pops all those frames — before it starts the right one. Being able to say that sentence is worth more than any amount of tracing.

Reading a `RecursionError`

```
File "shop.py", line 7, in total
    return numbers[start] + total(numbers, start + 1)
File "shop.py", line 7, in total
    return numbers[start] + total(numbers, start + 1)
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

Three things to read out of that.

In Python, the most recent call is at the bottom. The traceback reads oldest first, so the line just above the error is where it actually broke. (C and Java print it the other way round; knowing which you are looking at saves five minutes.)

“Previous line repeated 995 more times” means the same line, which means a genuine recursion rather than deep mutual calls.

And it is a limit, not a crash. Python counts frames and raises at 1,000 — and the depth your own function actually reaches is a few short of that, measured at **996** from inside a script and **998** from a bare prompt, because the interpreter and your harness are already using frames. Nothing has overflowed yet; Python stopped you before it could.

The limit, and why you must not raise it

```
sys.getrecursionlimit()      # 1000
sys.setrecursionlimit(1_000_000) # do NOT
```

The limit exists so that a runaway recursion produces a catchable Python exception instead of exhausting the **C stack** underneath, which is a fixed-size block of memory the operating system gave the thread. Overflow that and the process dies with a segmentation fault: no traceback, no exception, nothing in the log.

So the limit is a guard rail, and moving the guard rail does not move the cliff. Vasant putting a tenth tray on top of a shelf that holds nine.

If the depth is genuinely large, **convert to iteration**.

Converting recursion to iteration

The call stack is a stack, so you can always build your own. The recipe:

1. Make a list to hold the work items — the things that were arguments.
2. Push the initial call.
3. Loop while the list is not empty: pop an item, do the work, and push the sub-calls.

```
def count_nodes_iterative(root) → int:
    stack = [root]                # your own stack instead of the machine's
    count = 0
    while stack:
        node = stack.pop()
        if node is None:
            continue
        count += 1
        stack.append(node.left)
        stack.append(node.right)
    return count
```

Now the depth is limited by memory, not by 1,000 frames.

The catch, and it is a real one: this transformation is easy when the work happens *before* the recursive calls, and genuinely fiddly when it happens *after* — because then you have to remember “which child am I up to”, which is exactly the return address that the machine was storing for you. That is why an iterative in-order tree traversal is harder than a recursive one, and it is worth saying rather than claiming the conversion is always mechanical.

Tail calls, and why Python will not help

A **tail call** is a recursive call whose result is returned directly, with nothing left to do afterwards:

```
def total_tail(numbers, start=0, running=0):
    if start == len(numbers):
        return running
    return total_tail(numbers, start + 1, running + numbers[start]) # nothing after it
```

There is nothing waiting in the caller's frame, so in principle the machine could reuse it rather than stacking a new one — **tail call elimination**, which Scheme and most functional languages do, turning the recursion into a loop for free.

Python deliberately does not, and the reason is the traceback: eliminating frames would mean the error above could not show you where it came from. Guido van Rossum has written about choosing debuggability over the optimisation.

So in Python, a tail-recursive function stacks exactly like any other and dies at the same depth. Know the term, know that it does not help you here, and write the loop.

4 The picture

`factorial(4)`, drawn as the shelf filling and emptying.

```
GOING DOWN                                the shelf, top at the top

factorial(4) = 4 * factorial(3)    →    | factorial(1)  waiting: 1 * ? | ← top
factorial(3) = 3 * factorial(2)    | factorial(2)  waiting: 2 * ? |
factorial(2) = 2 * factorial(1)    | factorial(3)  waiting: 3 * ? |
factorial(1) = 1 (base case)       | factorial(4)  waiting: 4 * ? | ← bottom
                                   +-----+
                                   deepest point: 4 frames

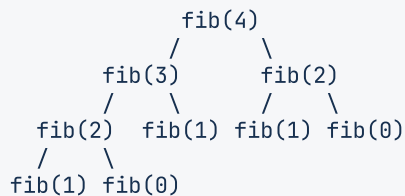
COMING BACK UP — always the top tray, never one from the middle

factorial(1) → 1
factorial(2) → 2 * 1 = 2
factorial(3) → 3 * 2 = 6
factorial(4) → 4 * 6 = 24
```

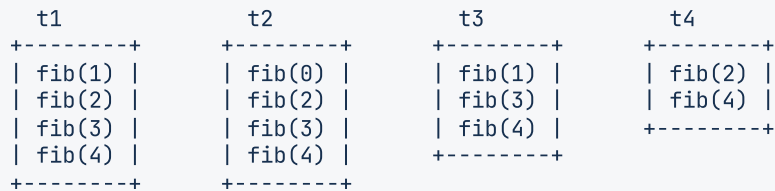
What to notice: at the deepest point, **all four frames exist simultaneously**, each holding its own `n` and each suspended in the middle of a multiplication. That simultaneity is the memory cost.

The tree against the stack, which is the picture to be able to draw:

fib(4) – TREE: every call that ever happens (9 nodes)



fib(4) – STACK at four different moments (never more than 4 frames)



9 calls over time. 4 frames at once.

TIME is the tree. SPACE is the deepest path through it.

And the two shapes of recursion, side by side, because the difference decides everything:

LINEAR	BRANCHING
one call per frame	two calls per frame
depth n, total calls n	depth n, total calls $\sim 2^n$
$O(n)$ time, $O(n)$ space	$O(2^n)$ time, $O(n)$ space
total(), factorial(), length()	fib(), subsets(), permutations()

5 The code, built step by step

Step 1 – make the stack visible

The fastest way to understand a recursion is to print the depth:

```
def total(numbers, start=0, depth=0):
    print(" " * depth + f"→ total(start={start})")
    if start == len(numbers):
        print(" " * depth + "← 0 (base case)")
        return 0
    result = numbers[start] + total(numbers, start + 1, depth + 1)
    print(" " * depth + f"← {result}")
    return result
```

The indentation is the stack depth, and the two arrows are the two phases. This is five minutes of work and it replaces an hour of confusion — do it once for every new recursive shape you meet.

Step 2 — inspect the real stack

```
import inspect

def depth_now() → int:
    return len(inspect.stack())
```

`inspect.stack()` returns the live frames, so `len` is the current depth. Slow, and useful exactly once: to confirm that your mental model matches the machine's.

Step 3 — find the real limit

```
def deepest() → int:
    def go(n=0):
        try:
            return go(n + 1)
        except RecursionError:
            return n
    return go()
```

Measured: **996** from inside a script, **998** from a bare prompt — never 1,000, because the interpreter and whatever called you are already a few frames deep. Worth knowing when you are told “the limit is 1000” and watch it fail at 996.

Step 4 — the mechanical conversion

```
stack = [initial_work]
while stack:
    item = stack.pop()
    ...do the work...
    stack.append(sub_item_1)
    stack.append(sub_item_2)
```

Work first, then push. When the work must happen *after* the children — an in-order traversal, or anything that combines results — you additionally need to record how far through each frame you are, which is the return address by hand.

The complete solution

```
import inspect
import sys

def total(numbers: list[int], start: int = 0) → int:
    """Linear recursion: one call per frame. depth = n, total calls = n."""
    if start == len(numbers):
        return 0
    return numbers[start] + total(numbers, start + 1)

def total_traced(numbers: list[int], start: int = 0, depth: int = 0) → int:
    """The same function with the stack made visible.

    The indentation IS the depth. The two arrows are the two phases: nothing is
    computed on the way down, and everything is computed on the way back up.
    """
    print(" " * depth + f"→ total(start={start})")
    if start == len(numbers):
        print(" " * depth + "← 0 (base case)")
        return 0
    result = numbers[start] + total_traced(numbers, start + 1, depth + 1)
    print(" " * depth + f"← {result}")
    return result

CALLS = 0
MAX_DEPTH = 0

def fib_counted(n: int, depth: int = 1) → int:
    """Counts total calls AND maximum depth, to show they are different things.

    Branching recursion: 2 calls per frame. Total calls ~2^n, but the stack
    never exceeds n frames, because the left branch finishes and pops before
    the right one starts.
    """
    global CALLS, MAX_DEPTH
    CALLS += 1
    MAX_DEPTH = max(MAX_DEPTH, depth)
    if n ≤ 1:
        return n
    return fib_counted(n - 1, depth + 1) + fib_counted(n - 2, depth + 1)

def deepest_reachable() → int:
    """The real limit, which is a little under sys.getrecursionlimit() because
    the interpreter is already a few frames deep."""
    def go(n: int = 0) → int:
        try:
            return go(n + 1)
        except RecursionError:
            return n
    return go()

def depth_now() → int:
    """The live stack depth, straight from the interpreter."""
    return len(inspect.stack())

def total_tail(numbers: list[int], start: int = 0, running: int = 0) → int:
    """A TAIL-recursive version: nothing happens after the recursive call.
```

A language with tail-call elimination would reuse the frame and turn this into a loop. Python deliberately does not – it keeps the frames so that tracebacks stay readable – so this dies at the same depth as any other recursion. Know the term; write the loop.

```
"""
if start == len(numbers):
    return running
return total_tail(numbers, start + 1, running + numbers[start])

def total_iterative(numbers: list[int]) → int:
    """What tail-call elimination would have produced. O(1) space, no limit."""
    running = 0
    for value in numbers:
        running += value
    return running

class TreeNode:
    __slots__ = ("value", "left", "right")

    def __init__(self, value: int, left=None, right=None) → None:
        self.value = value
        self.left = left
        self.right = right

def count_recursive(node: TreeNode | None) → int:
    """Work happens AFTER the recursive calls – the hard shape to convert."""
    if node is None:
        return 0
    return 1 + count_recursive(node.left) + count_recursive(node.right)

def count_iterative(node: TreeNode | None) → int:
    """The mechanical conversion: your own stack instead of the machine's.

    Easy here because the work (counting) does not depend on the children's
    results. When it does – an in-order traversal, or combining returns – you
    also have to record HOW FAR THROUGH each frame you are, which is the return
    address the machine was storing for you.
    """
    stack = [node]
    count = 0
    while stack:
        current = stack.pop()
        if current is None:
            continue
        count += 1
        stack.append(current.left)
        stack.append(current.right)
    return count

def sum_iterative_hard(node: TreeNode | None) → int:
    """A conversion where the work DOES depend on the children, done with an
    explicit stack of (node, visited) – `visited` is the return address."""
    stack: list[tuple[TreeNode | None, bool]] = [(node, False)]
    total_value = 0
    while stack:
        current, visited = stack.pop()
        if current is None:
            continue
        if visited:
            # children are done; do my work
            total_value += current.value
        else:
            stack.append((current, True))
            # come back to me afterwards
```



```

        stack.append((current.left, False))
        stack.append((current.right, False))
    return total_value

if __name__ == "__main__":
    total_traced([4, 7, 2])
    # → total(start=0)
    #   → total(start=1)
    #   → total(start=2)
    #   → total(start=3)
    #   ← 0 (base case)
    #   ← 2
    #   ← 9
    #   ← 13

    for n in (5, 10, 20):
        CALLS = MAX_DEPTH = 0
        value = fib_counted(n)
        print(f"fib({n}) = {value}: {CALLS} calls, max depth {MAX_DEPTH}")
    # fib(5) = 5: 15 calls, max depth 5
    # fib(10) = 55: 177 calls, max depth 10
    # fib(20) = 6765: 21891 calls, max depth 20

    print("limit:", sys.getrecursionlimit(), " deepest reachable:", deepest_reachable())
    print("depth at module level:", depth_now())

    print(total_tail([4, 7, 2]), total_iterative([4, 7, 2]))      # 13 13
    try:
        total_tail(list(range(5000)))
    except RecursionError as error:
        print(f"RecursionError: {error}")    # tail recursion does NOT help in Python
    print(total_iterative(list(range(5000))))      # 12497500

    root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
    print(count_recursive(root), count_iterative(root))          # 5 5
    print(sum_iterative_hard(root))                             # 15

    # a deep tree that recursion cannot handle, and iteration can
    deep = TreeNode(1)
    node = deep
    for _ in range(5000):
        node.left = TreeNode(1)
        node = node.left
    try:
        count_recursive(deep)
    except RecursionError as error:
        print(f"RecursionError: {error}")
    print("iterative count on the same tree:", count_iterative(deep))    # 5001

```

6 What it costs

Frames, and what a frame costs

measured: about 830 KB of Python-level allocation for 20,000 frames
 → roughly 40 bytes each of traced Python objects
 plus: a slice of the thread's C stack per call, which is the thing the recursion limit actually protects

The Python-object part is small. The C stack is the binding constraint, and it is a **fixed-size block the operating system gave the thread** — typically 8 MB on Linux for the main thread. That is why the limit is a count of frames rather than a measure of memory: Python cannot easily know how much C stack each frame consumed, so it counts conservatively.

The two complexities, kept apart

total(n)	calls: n	depth: n	$O(n)$ time,	$O(n)$ space
factorial(n)	calls: n	depth: n	$O(n)$ time,	$O(n)$ space
binary search	calls: $\log n$	depth: $\log n$	$O(\log n)$ time,	$O(\log n)$ space
fib(n)	calls: $\sim 2^n$	depth: n	$O(2^n)$ time,	$O(n)$ space

Measured, so the last line is not hand-waving:

```
fib(5)    = 5 : 15 calls, max depth 5
fib(10)   = 55 : 177 calls, max depth 10
fib(20)   = 6765 : 21,891 calls, max depth 20
```

Twenty-one thousand calls, twenty frames. Time is the whole tree; space is the deepest path through it. This is the single most useful distinction in the lesson.

The limit, exactly

```
sys.getrecursionlimit() → 1000
deepest reachable       → 996    (measured in a script; 998 from a bare prompt)
```

```
linear recursion on a list of 5,000    → RecursionError
log-depth recursion on 1,000,000 items → 20 frames
```

A recursion whose depth is the input size fails on inputs that are not large. Five thousand elements is nothing. That is the practical rule this lesson exists to establish.

Recursion against iteration, priced

	recursive	iterative
sum a list	$O(n)$ space, dies at ~ 1000	$O(1)$ space, no limit
tree traversal	$O(\text{height})$ space dies on a degenerate tree of depth 1000	$O(\text{height})$ space with your own stack works to memory
binary search	$O(\log n) = 20$ frames	$O(1)$ — the difference is irrelevant

The middle row is the one that matters in practice. A balanced tree of a million nodes has a height of about twenty and recursion is completely safe. A **degenerate** tree — one where every node has a single child, which is what an unbalanced binary search tree

becomes when you insert sorted data — has a height of a million, and recursion dies. Same code, same tree class, different data.

The conversion's cost

recursion:	frames managed by the machine, ~40 B of Python objects each
explicit stack:	a Python list of your work items, ~8 B per entry plus the item

The explicit version is usually *cheaper* in memory as well as unlimited in depth — because your work items are smaller than full frames. What it costs is clarity: an in-order traversal is four lines recursively and about twelve with an explicit stack and a `visited` flag.

7 The traps

Trap 1 — confusing total calls with depth

Saying “`fib(30)` will blow the stack” because it makes 2.7 million calls. It will not — the depth is 30. It will take a tenth of a second and finish. Conversely, `total` on a five-thousand-element list makes only five thousand calls and *does* blow the stack, because they are all alive at once.

Time is the tree. Space is the deepest path. Getting these two backwards is the most common misunderstanding in this area.

Trap 2 — reading the traceback from the wrong end

In Python the **most recent call is last**. The line immediately above the error message is where it broke. In C, Java and most debuggers it is the other way round. Reading a Python traceback top-down and concluding the bug is in `main` costs five minutes every time.

Trap 3 — raising the recursion limit

```
sys.setrecursionlimit(1_000_000)
```

Converts a clean, catchable `RecursionError` into a segmentation fault when the C stack runs out:

```
Segmentation fault (core dumped)
```

No traceback, no exception, nothing in the log, and in a server it takes the whole process down including every other request in flight. The limit is a guard rail; moving it does not move the cliff.

Trap 4 — assuming tail recursion helps in Python

```
return total_tail(numbers, start + 1, running + numbers[start])
```

There is nothing after the call, so a language with tail-call elimination would reuse the frame. Python does not, by an explicit design decision to keep tracebacks complete. This function dies at exactly the same depth as the non-tail version. Know the term; do not rely on it.

Trap 5 — recursion on a degenerate structure

A balanced tree of a million nodes has height ~ 20 ; recursion is safe. Insert a million *sorted* values into an unbalanced binary search tree and the height is a million:

```
RecursionError: maximum recursion depth exceeded
```

Same code, same class, different data. **The depth is a property of the input, not of the algorithm**, and that is why “it worked in testing” is such a common preface to this bug.

Trap 6 — a mutable default in a traced helper

```
def walk(node, path=[]): # created ONCE at definition time
```

The list is shared across every top-level call, so the second call sees the first one’s path. Use `None` and create it inside. This bites hardest here because path-tracking helpers are exactly the things people add while debugging a recursion.

Trap 7 — printing inside a hot recursion

Adding a `print` to `fib(30)` turns a tenth of a second into minutes and produces 2.7 million lines. Trace on the smallest input that shows the shape — `fib(5)` — and read fifteen lines instead.

Trap 8 — thinking the conversion to iteration is always mechanical

It is easy when the work happens *before* the recursive calls. When the work happens *after* — an in-order traversal, combining children’s return values — you must record how far through each frame you are, which means pushing `(node, visited)` pairs and handling each twice. That is the return address, by hand, and it is why the iterative version is three times the length.

8 In the interview

How it gets asked

- The direct version: *“Trace this function. What is on the stack when it is deepest?”*
- The distinction: *“How many calls does `fib(20)` make, and how much stack does it use?”*
- The diagnosis: *“Here is a `RecursionError`. What went wrong and how would you fix it?”*
- The conversion: *“Rewrite this recursion iteratively.”*
- The sneaky one: *“This works in tests and crashes in production on the same code path. Why?”* — a degenerate structure.

What to say out loud, in the first ninety seconds

1. **Say what a frame holds.** “Each call gets a frame with its arguments, its locals, and where to resume in the caller. They pile up and come off in reverse order.”
2. **Separate the two costs immediately.** “Total calls determine the time; maximum depth determines the space. Those are different numbers — `fib(20)` makes about twenty-two thousand calls and never exceeds twenty frames.”
3. **When tracing, draw and narrate the two phases.** “Going down, every frame is suspended in the middle of an expression. Coming back up is where all the arithmetic happens.”
4. **Give the limit as a real number.** “Python’s limit is a thousand frames — about 996 reachable in practice — so a recursion whose depth is the input size fails on a five-thousand-element list.”
5. **Offer the conversion, and be honest about it.** “If the depth is a problem I would use my own stack. That is mechanical when the work happens before the recursive calls, and genuinely fiddlier when it happens after.”

The follow-ups

“How many calls does `fib(20)` make, and how deep does the stack go?” “About twenty-two thousand calls — 21,891 exactly — and the stack never exceeds twenty frames. Those are two different questions and the answers diverge enormously. The tree of calls is everything that ever happens, which is what determines the time. The stack is only the calls alive at one instant, which is a single path from the root to wherever you are, and that determines the space. `fib` finishes its entire left branch and pops all those frames before it starts the right one, so it is exponential in time and only linear in space.”

“What is on the stack at the deepest point?” “For a linear recursion like summing a list, the deepest point is one frame per element plus the base case, and every frame except the top is suspended in the middle of an addition, waiting for the value from

above. Concretely, for a three-element list: the base-case call, plus three frames each holding its own index and each about to add its element to whatever comes back. That simultaneity is the memory cost — they all exist at once.”

“I got a `RecursionError`. What do I do?” “First, read it correctly: in Python the most recent call is *last*, so the line just above the message is where it broke, and ‘previous line repeated N times’ tells me it is genuine recursion rather than deep mutual calls. Then it is one of three things. A missing or unreachable base case, which is a bug — fix that. Or genuinely deep data, like a degenerate tree, in which case I convert to an explicit stack. Or the input is just large and the algorithm is linear-depth, which means the algorithm is wrong for the input. What I would *not* do is raise the recursion limit — that converts a catchable exception into a segmentation fault when the C stack runs out, which kills the process with no traceback.”

“Rewrite this recursively-defined traversal as a loop.” “The call stack is a stack, so I build my own: a list of work items, push the initial call, then loop popping items, doing the work and pushing the sub-calls. That is mechanical when the work happens before the recursive calls. When it happens after — combining the children’s results — I also have to record how far through each frame I am, which means pushing `(node, visited)` pairs and handling each node twice: once to schedule its children, once to do its own work. That flag is the return address the machine was storing for me, and it is why the iterative version is about three times the length.”

“Does making it tail-recursive help?” “Not in Python. A tail call is one where nothing happens after the recursive call returns, so in principle the frame could be reused — Scheme and most functional languages do that and turn the recursion into a loop for free. Python deliberately does not, because eliminating frames would make tracebacks incomplete, and that was an explicit trade in favour of debuggability. So a tail-recursive function in Python dies at exactly the same depth. I would know the term and write the loop.”

“It works in tests and crashes in production on the same code path.” “That is usually a degenerate structure. A balanced binary tree of a million nodes has a height of about twenty and recursion is completely safe; the same tree class fed a million *sorted* insertions has a height of a million and dies immediately. The depth is a property of the *input*, not of the algorithm, which is why the test data hid it. Same story for a linked list built from user data, or a deeply nested JSON document from an external source. If the depth can be attacker-controlled, that is a denial-of-service vector and iteration is the fix rather than a bigger limit.”

A model answer

Asked: *trace this recursion. What is on the stack at the deepest point?*

“Let me set up the vocabulary first, because it makes the trace short. Every call gets a stack frame holding three things: the arguments it was called with, its local variables, and the return address — the exact place in the caller to resume. The frames pile up and come off in the opposite order, always the top one.

So for summing `[4, 7, 2]` with `total(numbers, start)`:

Going down, four frames are created. `start=0` computes nothing — it evaluates `numbers[0]`, which is 4, and then suspends in the middle of the addition waiting for the recursive call. Same for `start=1` waiting on `7 + ?`, and `start=2` waiting on `2 + ?`. Then `start=3` hits the base case and returns 0 without calling anything.

That is the deepest point: four frames alive simultaneously, three of them suspended mid-addition, each holding its own `start`. Their simultaneity is the memory cost — that is what $O(n)$ space means here.

Coming back up, each frame completes its addition and pops: 0, then 2, then 9, then 13. **All the arithmetic happens on the way back up.** Nothing was computed on the way down.

The distinction I would want to make explicit, because it answers most questions in this area: the recursion *tree* and the call *stack* are different things. The tree is every call that ever happens and it determines the time. The stack is only the calls alive at one instant — a single path from the root — and it determines the space. `fib(20)` makes 21,891 calls and never has more than twenty frames, because it finishes the left branch and pops all of it before starting the right. Exponential in time, linear in space.

On the practical side: Python’s limit is a thousand frames, and about 996 are actually reachable from inside a script, because the interpreter and the caller are already a few frames deep. So a linear-depth recursion fails on a five-thousand-element list — which is not a large list. That limit exists to turn a C-stack overflow into a catchable Python exception, so raising it does not help; it just turns a `RecursionError` into a segmentation fault with no traceback.

If I needed the depth, I would convert to an explicit stack — the call stack is a stack, so that transformation always exists. It is mechanical when the work happens before the recursive calls, and genuinely fiddlier when it happens after, because then I have to store how far through each frame I am, which is the return address the machine was keeping for me.”

9 Recall card

- A stack frame holds the arguments, the locals, and the return address — where to resume in the caller. Frames pile up and come off in **reverse order**, and every frame except the top is **suspended mid-expression**. **All the work happens on the way back up**; nothing is computed on the way down.
- The recursion **TREE** and the call **STACK** are different things, and this answers most questions here. *Total calls determine the time; maximum depth determines the space.* Measured: `fib(20)` = **21,891 calls**, **max depth 20** — exponential in time, **linear** in space, because the left branch pops before the right one starts.
- To trace, draw the stack downward and write what each frame is waiting for, then read the two phases. The five-minute tool: add a `depth` parameter and indent the prints — the indentation is the depth. Trace the **smallest** input that shows the shape.
- Python's limit is 1000 frames, ~996 reachable from a script, so a linear-depth recursion dies on a 5,000-element list. Read the traceback from the **bottom** (most recent call last). **Never raise the limit** — it exists to turn a C-stack overflow into a catchable exception, so raising it gives a **segmentation fault** with no traceback. **Tail recursion does not help in Python**, deliberately, to keep tracebacks complete.
- The conversion to iteration always exists — build your own stack — but it is only mechanical when the work happens **BEFORE** the recursive calls. When it happens after, you must push `(item, visited)` pairs and handle each item twice; that flag is the return address by hand. And watch for **degenerate structures**: a balanced tree of 10^6 nodes has height ~ 20 , the same class fed sorted insertions has height 10^6 — **the depth is a property of the input, not the algorithm.**

Design a ride-hailing booking flow

1 What this is, and why they ask it

A rider asks for a car from A to B. The system finds a driver, the driver agrees, drives over, picks them up, drives to B, and gets paid. The trip state machine is ordinary — you have built three of them this phase.

The part that is not ordinary is **matching**, and it has a shape you have already met twice this week without noticing. Offering a ride to a driver is a **lock with a short expiry**: while the offer is out, that driver must not be offered anything else, and if they do not answer in fifteen seconds the offer has to move on. It is the cinema seat from [day 086](#) and the restaurant timeout from [day 087](#), applied to a person who is driving.

The second unusual thing is the **write volume from something that is not a booking**. A hundred thousand online drivers sending a location every four seconds is twenty-five thousand writes a second — hundreds of times the booking rate — and it needs a completely different storage answer from the trips. Noticing that the location firehose and the trip records are two different systems is most of the scaling insight.

They ask it because everyone has used it, because “two riders, one driver” is a race that is easy to state, and because the follow-ups are unusually good: what if nobody accepts, what if the driver cancels after accepting, and what does the rider get charged when the fare changes on the way.

2 The story

The auto stand outside the station has about forty autos on a good evening, and it is run by Shivanand, who does not own any of them.

He stands at the head of the rank with a whistle he never uses and a system everybody has agreed to because the alternative was drivers racing each other down the ramp.

The autos are in a line, and the one at the front gets the next passenger. That part is simple and it works for the ordinary fares — station to the market, station to the hospital.

The trouble is the fares nobody wants. Late at night, a long way out to the industrial area where nothing comes back. When one of those comes up, the driver at the front says no.

What Shivanand does then, and he is completely rigid about it, is go to the second auto. Not the third, not whoever is shouting. The second. And while he is standing there asking the second driver, the first driver's refusal is final and the passenger is not offered to anybody else. One driver at a time.

He tried it the other way for about a week, years ago — announcing the fare to the whole rank at once and letting whoever wanted it come forward. Two things happened. Three drivers came forward for the good fares and started an argument, and for the bad ones nobody moved at all and the passenger stood there while everyone looked at the ground.

The other rule is the clock. He gives a driver a few seconds — genuinely a few, and he counts them — and if the man is finishing a phone call or pretending not to hear, Shivanand moves on. He says if you wait for a driver who is not going to say yes, you are not waiting for that driver, you are making the passenger wait for nothing.

And the last thing, which is the one passengers argue about. He tells them the fare before the auto leaves. If it turns out longer because of the flyover being shut, that is the driver's bad luck, and if it is shorter the driver keeps the difference. He fixed that after too many evenings of people shouting at the far end of a trip about a number that had changed on the way.

3 The idea in plain English

Shivanand's three rules are the design. **One driver at a time. Count the seconds and move on. The price is agreed before the vehicle moves.**

An offer is a lock on a driver

The naive matching loop is: find nearby drivers, tell all of them, first to accept wins. That is Shivanand's bad week. Two problems, both real:

- **Several accept at once**, and now you have a race and two drivers who each believe they have the ride. You can resolve it with a conditional claim — first `UPDATE` wins — but the loser has already turned the car around.
- **Nobody accepts**, and there is no mechanism that notices, because nothing was ever owed.

So: **offer to one driver, exclusively, with an expiry.**

```
driver state:  ONLINE  →  OFFERED (locked, 15 s)  →  ASSIGNED
                |
                +-- declines, or 15 s pass → ONLINE, offer the next
```

An `OFFERED` driver is not available to any other request, exactly as a `LOCKED` seat is not available to any other booking. The claim must be conditional for the same reason:

```
UPDATE drivers SET status = 'OFFERED', offered_ride = :ride, offer_expires = :now_plus_15s
WHERE id = :driver
AND (status = 'ONLINE' OR (status = 'OFFERED' AND offer_expires < :now));
```

Zero rows updated means somebody else got that driver first, and you move to the next candidate. Same pattern as the seat, the parking spot and the shipped order — **find-then-claim needs a conditional claim** — and by now it should be automatic.

Sequential, broadcast, or something in between

The honest answer names all three:

	LATENCY	DRIVER EXPERIENCE	RACE
Sequential , one at a time	slow: 15 s per decline	fair, calm	none
Broadcast to all candidates	fast	several accept, all but one disappointed	needs a conditional claim
Batched : offer to the best 3 at once	fast	mild competition	needs a conditional claim

Sequential at a 60 percent acceptance rate averages **1.7 offers**, and if each takes about eight seconds that is roughly **fourteen seconds** to match. That is acceptable in a city and terrible at an airport queue where a hundred requests arrive together.

Batching is the usual production answer: offer to a small number simultaneously, first conditional claim wins, the rest get a “taken” notification quickly. It trades a little driver goodwill for a lot of latency, and saying which you would choose and why is worth more than picking one.

Choosing the candidates: the location problem

To offer at all, you need “drivers within three kilometres, available, right vehicle type”.

A plain database index does not answer that well. Indexing latitude and longitude separately means the database can narrow on one and then scan the other, and “within 3 km” is a circle, not a rectangle. The standard answers turn two dimensions into one:

- **Geohash** — interleave the bits of latitude and longitude into a single string, so nearby points share a prefix. `tdr1v` is a cell of about 5 km; one more character is about 1.2 km. Then “nearby” is a prefix query, which an ordinary index handles.
- **S2 cells** or **H3 hexagons** — the same idea with better-behaved cell shapes; H3’s hexagons have uniform neighbour distances, which matters for “expand the search radius” logic.

The catch that is worth mentioning: a geohash cell has edges, and two points either side of an edge are close in reality and share no prefix. So a real query looks at the cell **and its eight neighbours**, and then filters by true distance. Knowing that is the difference between having read about geohashing and having used it.

And the storage: driver locations are **not** a database table. They are hot, they are overwritten every few seconds, and nobody needs last Tuesday's. That is Redis — a geospatial set per city, or a hash keyed by driver — and the durability requirement is genuinely “none”.

The fare is agreed before the car moves

A rider is quoted a fare at request time. That quote must **survive the trip**:

```
@dataclass(frozen=True)
class FareQuote:
    base_paise: int
    per_km_paise: int
    per_minute_paise: int
    surge_multiplier: float           # frozen at REQUEST time, not at completion
    estimated_paise: int
    quoted_at: datetime
```

The **surge multiplier especially** is frozen. Surge changes minute by minute; a rider who accepts at $1.4\times$ and is charged $2.1\times$ because demand rose during the trip has been cheated, and that is the single most damaging thing this kind of app can do to trust.

Same immutability rule as the food order's price snapshot and the library's fine — and by the third time you meet it, it should be automatic: **anything shown to a customer before they commit is frozen at that moment**.

The *actual* fare is computed at completion from real distance and time, and then reconciled against the quote by whatever policy the business has — usually “charge the lower”, or “charge the quote unless the route changed materially”.

The trip state machine, and who moves it

```
REQUESTED → MATCHING → ASSIGNED → ARRIVING → ARRIVED → IN_PROGRESS → COMPLETED
```

TRANSITION	ACTOR	TIMEOUT	ON TIMEOUT
REQUESTED → MATCHING	system	—	—
MATCHING → ASSIGNED	driver accepts	15 s per offer , ~2 min overall	give up, tell the rider, widen the radius or raise surge
ASSIGNED → ARRIVING	driver starts moving	60 s	nudge, then reassign
ARRIVED → IN_PROGRESS	driver starts the trip	5 min waiting	driver may cancel with a fee to the rider
IN_PROGRESS → COMPLETED	driver ends the trip	—	operations alert on a very long trip

Exactly the same shape as yesterday’s food order: **every transition performed by somebody else carries a deadline**, and the interesting cell is what happens when it passes. Matching failing entirely — no driver in two minutes — is a real outcome with a real product decision behind it, not an error.

Cancellation, which is a policy on state and actor

rider cancels before ASSIGNED	free
rider cancels after ASSIGNED	fee, because a driver is now driving to them
rider cancels after ARRIVED	larger fee plus waiting time
driver cancels after ASSIGNED	no fee to the rider; counts against the driver
system cancels (no match)	free, and an apology

Same shape as the food order, with one addition that matters: **driver cancellations are counted**, because a driver who accepts everything and cancels the bad ones is gaming the offer system. The cancellation *rate* is an input to who gets offered rides — which is a design consequence, not an afterthought.

4 The picture

The offer loop, which is the heart of it:

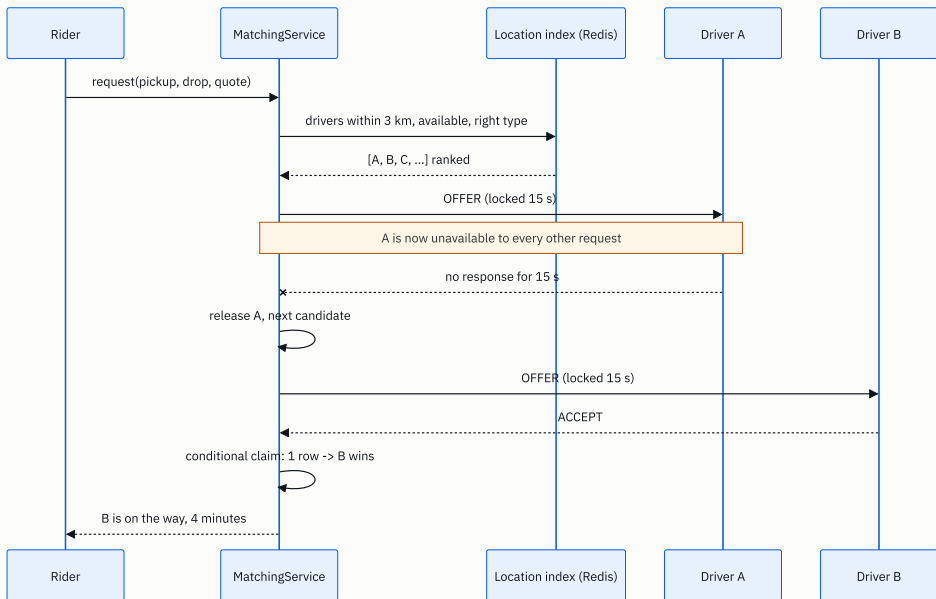


FIGURE 88.1

What to notice: **between the offer and the answer, driver A is locked** — not “preferred”, locked. Any other request that considers A gets zero rows from its conditional claim and moves on. That is the same mechanism as the cinema seat, applied to a person.

The trip state machine with the actors marked:

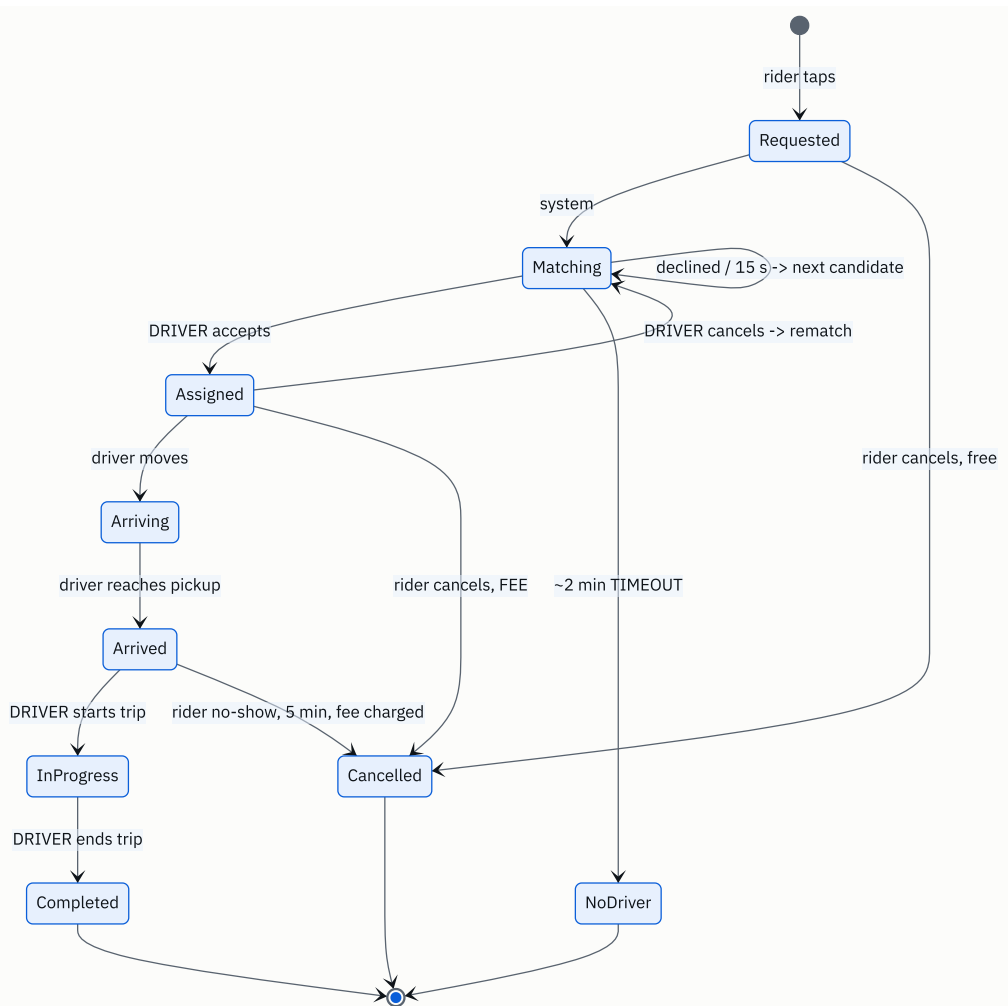


FIGURE 88.2

The two systems that look like one and are not:

LOCATION FIREHOSE

100,000 online drivers
1 ping every 4 s
= 25,000 writes/second

overwritten constantly
durability required: NONE
storage: Redis geo set, in memory
size: 100,000 x ~100 B = 10 MB

625x the write rate, and none of it is worth keeping

TRIP RECORDS

2,000,000 trips/day
~40 trips/second at peak
= 40 writes/second

written once, kept for years
durability required: ABSOLUTE
storage: the database
500 B x 2M/day = 1 GB/day

And the geohash edge problem, which is the detail worth knowing:



so a "nearby" query must read the cell AND its 8 neighbours,
then filter by true distance

5 How it actually works

Move 1 · Clarify (minutes 0–5)

“Just the booking flow, or pricing and driver payouts too?” — Booking and the fare quote; payouts are a separate system. **“Is the fare quoted upfront or metered?”** — Quoted upfront. This decides whether surge is frozen, which is the most user-visible decision here. **“One rider per trip, or pooled rides?”** — One. Pooling changes matching from “find a driver” to “find a driver whose route can absorb this”, which is a much larger problem. **“What should happen if no driver accepts?”** — Ask, because it is a product decision: give up and apologise, widen the radius, or raise the surge and retry.

“I will assume drivers are already onboarded and online, that payment is a gateway behind an interface, and that ETA and surge prediction are separate services I call into. I am not designing maps, routing, or the driver app’s UI.”

Move 2 · The nouns (minutes 5–12)

- **Rider**, **Driver**, **Vehicle** — the participants. **Driver** carries a **status** and, when offered, **offered_ride** and **offer_expires**.
- **RideRequest** — pickup, drop, vehicle type, the frozen **FareQuote**.
- **Trip** — the record: rider, driver, state, timestamps, actual distance and time, final fare.
- **Offer** — a driver, a ride, an expiry, an outcome. Worth being its own record because the *history* of offers is how you measure acceptance rates.
- **LocationIndex** (*interface*) — **update(driver, point)** and **nearby(point, radius, filters)**. Behind an interface because the in-memory test version and the Redis version are both real.
- **MatchingService** — the offer loop.

- **FarePolicy** (*interface*) — quote and final fare; surge and city rules are genuine second implementations.

Eight, two interfaces. Note there is no **Ride** and **Trip**: a request that never matches is a **RideRequest** that ended in **NO_DRIVER**, and one that matches becomes a **Trip**. Two names for the same thing is how state machines get duplicated.

Move 3 • The location index

```
class LocationIndex(Protocol):
    def update(self, driver_id: str, lat: float, lng: float, at: datetime) → None: ...
    def nearby(self, lat: float, lng: float, radius_m: int,
               vehicle_type: str) → list[tuple[str, float]]: ...

class RedisLocationIndex:
    """Driver locations are hot, overwritten every few seconds, and worthless
    once stale — so they live in Redis, not in the database.

    GEOADD/GEOSEARCH do the geohash work; the TTL means a driver whose app
    dies simply falls out of the index instead of being offered rides for ever.
    """

    STALE_AFTER = 30 # seconds

    def update(self, driver_id, lat, lng, at):
        key = f"drivers:{self._city_of(lat, lng)}"
        self._redis.geoadd(key, (lng, lat, driver_id))
        self._redis.setex(f"driver:live:{driver_id}", self.STALE_AFTER, "1")

    def nearby(self, lat, lng, radius_m, vehicle_type):
        found = self._redis.geosearch(
            f"drivers:{self._city_of(lat, lng)}",
            longitude=lng, latitude=lat, radius=radius_m, unit="m",
            withdist=True, sort="ASC", count=30,
        )
        return [(d, dist) for d, dist in found
                if self._redis.exists(f"driver:live:{d}")
                and self._vehicle_type(d) == vehicle_type]
```

Two details worth narrating. **The staleness key** means a driver whose phone dies drops out in thirty seconds rather than being offered rides for ever — a location index without an expiry accumulates ghosts. And **count=30** because you only need enough candidates to make a handful of offers; fetching every driver in three kilometres at an airport would be thousands.

Move 4 · The offer loop — the interesting part

```
class MatchingService:
    OFFER_TIMEOUT = timedelta(seconds=15)
    OVERALL_TIMEOUT = timedelta(minutes=2)
    BATCH = 3                                # offer to 3 at once

    def match(self, request: RideRequest, now: datetime) → Trip | None:
        deadline = now + self.OVERALL_TIMEOUT
        tried: set[str] = set()

        while datetime.now() < deadline:
            candidates = self._rank(
                self._locations.nearby(request.pickup_lat, request.pickup_lng,
                                       radius_m=3000, vehicle_type=request.vehicle_type)
            )
            batch = [d for d, _ in candidates if d not in tried][:self.BATCH]
            if not batch:
                self._widen_or_wait(request)
                continue
```

`tried` matters: without it, a driver who declined is immediately re-offered the same ride, which is maddening for them and wastes the whole window.

```
        offered = [d for d in batch if self._claim(d, request.id, now)]
        tried.update(batch)
        accepted = self._await_first_acceptance(offered, self.OFFER_TIMEOUT)
        for driver_id in offered:
            if driver_id ≠ accepted:
                self._release(driver_id)                # release the losers immediately
        if accepted is not None:
            return self._create_trip(request, accepted, now)

        return None                                # NO_DRIVER: a real outcome
```

Releasing the losers immediately is the line people forget. If three drivers are offered and one accepts, the other two must be returned to `ONLINE` in the same breath — not left to time out — or you have taken two drivers off the market for fifteen seconds for nothing, and at an airport that is most of your supply.

```
def _claim(self, driver_id: str, ride_id: str, now: datetime) → bool:
    """Conditional claim: an OFFERED driver belongs to exactly one ride."""
    affected = self._db.execute(
        """
        UPDATE drivers
        SET status = 'OFFERED', offered_ride = %, offer_expires = %s
        WHERE id = %s
        AND (status = 'ONLINE'
            OR (status = 'OFFERED' AND offer_expires < %s))
        """,
        ride_id, now + self.OFFER_TIMEOUT, driver_id, now,
    )
    return affected == 1
```

The same shape as the seat, the spot and the order. **By the fourth time you meet find-then-claim, the conditional update should be automatic**, and saying “this is the same pattern as the seat lock” out loud is a good use of five seconds.

Move 5 • Ranking, which is not just distance

```
def _rank(self, candidates: list[tuple[str, float]]) → list[tuple[str, float]]:
    """Nearest is a poor proxy for soonest. Rank by predicted ARRIVAL TIME,
    then break ties toward drivers who accept and do not cancel."""
    scored = []
    for driver_id, metres in candidates:
        eta = self._eta.seconds(driver_id, metres)          # a separate service
        reliability = self._stats.acceptance_rate(driver_id)
        scored.append((eta - 30 * reliability, driver_id, metres))
    scored.sort()
    return [(driver_id, metres) for _, driver_id, metres in scored]
```

A driver 500 metres away across a river is further, in time, than one two kilometres down the same road. **Rank by predicted arrival time, not by straight-line distance** — and note that the prediction is somebody else’s system, which is the same honest boundary as the food app’s prep-time estimate.

Move 6 • The fare

```
def quote(self, request: RideRequest, now: datetime) → FareQuote:
    surge = self._surge.multiplier(request.pickup_cell, now) # changes minute to minute
    distance_km, minutes = self._eta.route(request.pickup, request.drop)
    estimate = round((self.BASE + self.PER_KM * distance_km
                     + self.PER_MIN * minutes) * surge)
    return FareQuote(self.BASE, self.PER_KM, self.PER_MIN,
                     surge_multiplier=surge, estimated_paise=estimate, quoted_at=now)
```

Frozen at quote time, stored on the request, and used at completion:

```
def final_fare(self, trip: Trip) → int:
    metered = round((trip.quote.base_paise
                     + trip.quote.per_km_paise * trip.actual_km
                     + trip.quote.per_minute_paise * trip.actual_minutes)
                    * trip.quote.surge_multiplier) # the QUOTED surge
    return min(metered, trip.quote.estimated_paise) if self.HONOUR_QUOTE else metered
```

`trip.quote.surge_multiplier`, not today’s surge. That one attribute access is the difference between a rider who trusts the app and one who does not.

Real systems

- **Uber and Ola** both use hexagonal spatial indexing; Uber open-sourced **H3** for exactly this, and their published reason is that hexagons have uniform neighbour distances, which makes “expand the search” behave sensibly. Google’s **S2** is the square-cell equivalent.
- **Redis** `GEOADD` / `GEOSEARCH` implement geohashing directly, which is why driver-location services so often sit on Redis rather than on a relational database.

- **Batched offers with a short expiry** are the industry norm; the visible countdown in a driver app is the lock's TTL, exposed to the user, exactly as the cinema's booking timer is.
- **Upfront pricing** — quoting before the trip rather than metering — was a deliberate product change by Uber and Ola precisely because end-of-trip surprises destroy trust. The engineering consequence is the frozen quote.
- **Driver acceptance and cancellation rates** are tracked and used in dispatch by every major platform, which is the design consequence of drivers being able to game an offer system.

6 The numbers

Two systems, not one

online drivers at peak	100,000
location ping interval	4 s
→ location writes	25,000 per second
trips per day	2,000,000
peak: 30% in 4 hours → 600,000 / 14,400 s	
→ trip creations	42 per second

$25,000 / 42 \approx 600x$

The location firehose is six hundred times the booking rate, and not one byte of it is worth keeping. That is the single most important number in this design: it says locations go to an in-memory store with an expiry, and trips go to the database, and treating them as one system is how you end up with a database doing twenty-five thousand writes a second of data nobody will ever read.

location state: 100,000 drivers × ~100 B	= 10 MB	— the whole country, in RAM
trip records: ~500 B × 2M/day	= 1 GB/day	= 365 GB/year

The matching loop

```
candidates within 3 km, city centre:    ~30 drivers
driver acceptance rate:                  ~60%
offers needed, sequential: 1 / 0.6      ≈ 1.7
seconds per offer (accept or time out):  ~8 s average
→ time to match, sequential              ≈ 14 s

with a batch of 3:
  probability at least one of 3 accepts = 1 - 0.4^3 = 94%
  → usually one round
  → time to match                        ≈ 6 s
```

Fourteen seconds against six. That is the argument for batching, and the cost is that two drivers out of three get an offer that evaporates — which is why releasing the losers *immediately* rather than letting them time out matters so much.

The cost of not releasing losers

```
batch of 3, 42 matches/second at peak
losers per second: 2 × 42 = 84 drivers

released immediately:    locked for ~3 s each → ~250 drivers locked at any moment
left to time out:        locked for 15 s each → 1,260 drivers locked at any moment
```

A thousand extra drivers off the market at all times, out of a hundred thousand — one percent of supply, permanently, from a missing line of code. At an airport rank where the local supply is thirty, it is catastrophic rather than merely wasteful.

Failing to match

```
requests with no driver in 2 minutes: ~2% in a city, ~15% in outer areas at 2 a.m.
2,000,000 × 2% = 40,000 per day
```

Forty thousand a day, so “no driver found” is a designed outcome with its own screen, its own metric and its own product decision — widen the radius, raise the surge, or apologise. Not an error path.

The geohash query

```
geohash precision 6: cell ≈ 1.2 km × 0.6 km
a 3 km radius: read the cell + 8 neighbours = 9 cells
drivers per cell in a busy area: ~50
→ ~450 candidates fetched, filtered by true distance, take the nearest 30
```

Nine reads of a Redis geo set, sub-millisecond. And the reason for the nine rather than one is the edge problem: two drivers two hundred metres apart on either side of a cell boundary share no prefix.

The fare quote

surge changes: every minute, and can move $1.2x \rightarrow 2.0x$ in five minutes during rain
an average trip: 18 minutes

without freezing the multiplier:

a rider quoted at $1.4x$ could be charged at $2.0x$ = 43% more than agreed
at an average fare of ₹250, that is ₹107 of surprise

Forty-three percent more than agreed is not a pricing decision, it is a trust incident. That arithmetic is the whole argument for the frozen quote, and it is more convincing than “orders should be immutable”.

7 The trade-offs

What this design gives up

Batched offers waste driver attention. Two of every three offers evaporate, and drivers notice and resent it. Sequential is fairer and more than twice as slow. There is no right answer; there is a knob, and the batch size is the knob. I would start at three and tune it against measured match time and driver complaints.

Locking drivers reduces effective supply. At any instant a percentage of the fleet is in `OFFERED` and unavailable, and that percentage rises with batch size and with the offer timeout. It is a real cost that does not show up as an error anywhere — it shows up as slightly worse match times for everyone else — which is exactly why releasing losers immediately matters.

Locations in Redis means locations are lost on failure. That is deliberate — they are worthless after thirty seconds — but it means a Redis failover briefly empties the index and matching stops entirely for that region. The mitigation is replication and the fact that drivers re-ping within four seconds, so recovery is fast. Worth stating rather than pretending the choice is free.

Ranking by predicted ETA depends on a prediction. A bad ETA service produces bad matches that look like a matching bug. And ranking partly by acceptance rate creates a feedback loop: reliable drivers get more offers, get busier, and become less available — which needs damping or it concentrates all the work on a few people.

Freezing the quote transfers route risk to the platform. If the flyover is shut and the trip takes twice as long, somebody absorbs it. Charging the rider breaks the promise; charging the driver is unfair; absorbing it costs money. Most platforms honour the quote unless the *route* changed materially, and that “materially” is a policy with an appeals process behind it — which I would name rather than design in forty minutes.

Nothing here handles pooling. Shared rides change matching from “find a driver” to “find a driver whose current route can absorb this detour within the other rider’s tolerance”, which is a routing optimisation and a different system.

“I would change this design if...”

- **...supply is dense and demand is spiky** — an airport, a stadium at closing time. Then the rank is a genuine queue, first-come-first-served among drivers, and offering by proximity is both unfair and pointless because everybody is fifty metres away.
- **...matching must consider a whole city at once.** Batch matching every few seconds — solving an assignment problem over all pending requests and all free drivers — beats greedy per-request matching on total waiting time, at the cost of adding a few seconds of latency to every request.
- **...pooling is required.** Matching becomes route-aware, and the trip state machine gains riders boarding and alighting mid-trip.
- **...regulators require metered fares.** Then the quote is an estimate only and the surge freeze has no meaning, and the whole trust argument changes shape.

The honest concession

Nothing in this design is new. The offer is the cinema’s seat lock. The timeout table is the food order’s. The frozen quote is the library’s fine snapshot and the menu price snapshot. The conditional claim is the parking spot. What makes ride-hailing feel harder is the **location firehose**, and that turns out to be a separate system with an entirely separate answer — six hundred times the write rate and zero durability requirement. Recognising that the hard-looking part is a different problem, and that the rest is patterns you already have, is the design.

8 In the interview

How it gets asked

- The standard: *“Design the booking flow for a cab app.”*
- The matching probe, which is the real question: *“How do you decide which driver gets the request?”*
- The race probe: *“Two riders request at the same moment and the same driver is nearest to both.”*
- The scale probe: *“How do you store and query driver locations?”*
- The trust probe: *“Surge goes up during the trip. What does the rider pay?”*

The timed script

Minutes 0–5 · Clarify. Booking only? Quoted or metered fare — this decides the surge question. One rider or pooling? And what should happen when nobody accepts, because that is a product decision.

Minutes 5–10 · Split the problem in two, out loud. “There are two systems here with completely different requirements: the location firehose at twenty-five thousand writes a second with no durability requirement, and the trip records at forty a second that must never be lost. I will treat them separately.” That sentence reframes the whole round.

Minutes 10–18 · Matching, the deep dive. The offer as a lock with a TTL, the conditional claim, the sequential-versus-batched trade with both latency numbers, releasing losers immediately, and ranking by ETA rather than distance.

Minutes 18–25 · The location index. Geohash or H3, the cell-plus-neighbours query, the staleness key, and why Redis rather than the database.

Minutes 25–32 · The trip state machine with the actor and timeout on every transition, and cancellation as a policy on state and actor.

Minutes 32–40 · The fare quote and trust, with the 43-percent arithmetic, then failure modes: no-driver as a designed outcome, driver cancellation and its rate, Redis fail-over.

The follow-ups

“How do you decide which driver gets the request?” “An offer is a **lock on a driver with a short expiry** — while it is out, that driver is unavailable to every other request, exactly like a locked cinema seat. I claim the driver with a conditional update — set status to offered where the status is online or the previous offer has expired — and zero rows means somebody else got them first, so I move to the next candidate. On how many at once: sequential is fairest but at a sixty percent acceptance rate it averages 1.7 offers and about fourteen seconds. Offering to three at once gives a ninety-four percent chance one accepts in the first round, so about six seconds. I would batch, and the critical detail is releasing the losers *immediately* rather than letting them time out — at forty matches a second that difference is about a thousand drivers needlessly off the market at any moment.”

“Two riders request at the same moment and the same driver is nearest to both.” “Exactly one gets the offer, and it is the conditional claim that decides — the same find-then-claim pattern as the parking spot, the cinema seat and the shipped order. The other request sees zero rows affected, marks that driver as tried, and immediately offers to its next candidate. What must *not* happen is offering to both and resolving it when they accept, because by then one driver has already turned the car around.”

“How do you store and query driver locations?” “Separately from everything else, because it is a different problem. A hundred thousand online drivers pinging every four seconds is twenty-five thousand writes a second — about six hundred times the booking rate — and none of it is worth keeping for more than thirty seconds. So it goes in Redis with a geospatial index, not in the database. The query ‘drivers within three kilometres’ needs a spatial index because separate latitude and longitude indexes cannot answer a circle: geohash interleaves the two into one string so nearby points share a prefix, and Uber’s H3 and Google’s S2 are the better-behaved cell versions. The detail worth knowing is that cells have edges, so two drivers two hundred metres apart across a boundary share no prefix — a real query reads the cell and its eight neighbours and then filters by true distance. And every driver key has a TTL, so a phone that dies drops out of the index instead of being offered rides for ever.”

“Surge goes up during the trip. What does the rider pay?” “What they were quoted. The surge multiplier is frozen onto the fare quote at request time and stored with the trip, and the final fare uses that multiplier, not today’s. The arithmetic is the argument: surge can move from 1.4 to 2.0 during a rainy eighteen-minute trip, which on a ₹250 fare is about ₹107 more than the rider agreed to — a forty-three percent surprise. That is not a pricing decision, it is a trust incident. It is the same immutability rule as a menu price snapshot or a library fine: anything shown to a customer before they commit is frozen at that moment.”

“Nobody accepts the ride.” “That is a designed outcome, not an error — roughly two percent of requests in a city and much more at two in the morning in outer areas, so about forty thousand a day. After an overall timeout of a couple of minutes I stop offering and tell the rider, and what I do *instead* is a product decision I would ask about: widen the radius, raise the surge to attract supply, or apologise and suggest trying later. Each has a screen and a metric. What I would not do is let the request sit silently, which is the same lesson as the food order’s restaurant timeout.”

“Should you offer to the nearest driver?” “Nearest by straight-line distance is a poor proxy for soonest. A driver five hundred metres away across a river is further in time than one two kilometres down the same road, so I rank by predicted arrival time — which means calling an ETA service, and I would name that as a separate system with its own failure modes rather than pretend it is a distance calculation. I would also break ties toward drivers with a good acceptance record, with the caveat that ranking on reliability creates a feedback loop that concentrates work on a few drivers, so it needs damping.”

“A driver accepts and then cancels.” “The trip goes back to matching and the rider is told immediately, which is the important part — a silent re-match feels like the app has forgotten them. There is no fee to the rider. And the cancellation is *counted*, because a driver who accepts everything and cancels the long ones is gaming the offer system,

and acceptance and cancellation rates feed back into who gets offered rides. That feedback is a design consequence rather than an afterthought: any system that offers scarce work to people has to measure whether they honour it.”

A model answer

Asked: *design the booking flow for a cab app.*

“Before the classes, let me split this into two systems, because they have completely different requirements and merging them is the main mistake available here.

There is the **location firehose**: a hundred thousand online drivers sending a position every four seconds, which is twenty-five thousand writes a second — about six hundred times the booking rate — and none of it matters after thirty seconds. And there are the **trip records**: forty a second at peak, which must never be lost. The first goes in Redis with a geospatial index and a TTL. The second goes in the database. If I put locations in the database I have a primary doing twenty-five thousand writes a second of data nobody will ever read.

Now the interesting part, which is matching.

The key idea is that **an offer to a driver is a lock with a short expiry**. While the offer is out, that driver is unavailable to every other request — exactly like a locked cinema seat. I claim them with a conditional update: set status to offered where they are online, or where a previous offer has already expired. Zero rows means another request got them first and I move on. That is the same find-then-claim pattern as the parking spot and the seat, and by now it should be the reflex.

How many drivers to offer at once is a genuine trade and I would name both ends. Sequential is fairest — one driver at a time, nobody disappointed — but at a sixty percent acceptance rate it takes about 1.7 offers and roughly fourteen seconds. Offering to three simultaneously gives a ninety-four percent chance somebody accepts in the first round, so about six seconds. I would batch at three, and the detail that matters is **releasing the losers the instant somebody accepts** rather than letting them time out — at forty matches a second, that is the difference between about two hundred and fifty drivers locked at any moment and about twelve hundred.

For choosing candidates, ‘drivers within three kilometres’ needs a spatial index — separate latitude and longitude indexes cannot answer a circle. Geohash interleaves the coordinates so nearby points share a prefix; Uber’s H3 hexagons are the better-behaved version. The detail worth knowing is that cells have edges, so the query reads the cell and its eight neighbours and then filters by true distance. And I rank by predicted *arrival time*, not distance, because five hundred metres across a river is further than two kilometres down the same road.

The trip state machine is the same shape as any other in this phase — requested, matching, assigned, arriving, arrived, in progress, completed — and the column that matters is who moves each transition and what happens if they do not. Fifteen

seconds per offer, about two minutes overall, and ‘no driver found’ is a real outcome about two percent of the time, so forty thousand a day. It gets a screen and a metric, not an error log.

The last thing, and it is the most user-visible: **the fare quote is frozen at request time, especially the surge multiplier**. Surge can move from 1.4 to 2.0 during a rainy eighteen-minute trip, which on a ₹250 fare is about a hundred rupees more than the rider agreed to — a forty-three percent surprise. That is a trust incident, not a pricing decision. Same rule as a menu price snapshot: anything shown to a customer before they commit is frozen at that moment.”

9 Recall card

- **Split it in two before designing anything: the location firehose and the trip records.** $100,000 \text{ drivers} \times \text{a ping every } 4 \text{ s} = 25,000 \text{ writes/second}$, $\sim 600 \times$ the booking rate, with zero durability requirement → Redis geo set with a TTL. Trips at $\sim 42/\text{s}$ must never be lost → the database. *The hard-looking part is a different system.*
- **An offer to a driver is a LOCK with a short expiry** — while it is out, that driver is unavailable to every other request, exactly like a cinema seat. Claim with a **conditional update** (`WHERE status='ONLINE' OR offer_expires < now`), **0 rows = you lost**. Same find-then-claim as the parking spot, the seat and the shipped order.
- **Sequential vs batched is a real trade with real numbers.** 60% acceptance → sequential averages **1.7 offers $\approx$ 14 s**; a batch of 3 accepts in the first round **94%** of the time $\approx$ **6 s**. And **release the losers immediately** rather than letting them time out — at 42 matches/s that is ~ 250 drivers locked instead of $\sim 1,260$, a full percent of supply.
- **Spatial queries need a spatial index: geohash / S2 / H3, because separate lat and lng indexes cannot answer a circle.** Cells have edges, so read the cell **and its 8 neighbours**, then filter by true distance. Give every driver key a **staleness TTL**, or a dead phone gets offered rides for ever. And **rank by predicted ETA, not distance** — 500 m across a river beats 2 km down the road.
- **Freeze the fare quote, and the surge multiplier especially.** Surge $1.4 \times \rightarrow 2.0 \times$ over an 18-minute trip is **+43%, about ₹107 on a ₹250 fare** — a trust incident, not a pricing decision. Same immutability rule as the menu price and the library fine. And **“no driver found” is a designed outcome** ($\sim 2\%$ of requests, $\sim 40,000/\text{day}$) with a screen, a metric and a product decision behind it.

DSA topic: The call stack, drawn **System design topic:** Design a ride-hailing booking flow

Code these, in this order

One rule for the whole set: **before optimising or converting anything, say which number you are worried about — total calls or maximum depth.** They are different questions and the fixes are different.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Binary Tree Inorder Traversal	LeetCode 94 (Easy)	Write it recursively, then iteratively. This is the conversion where the work happens <i>after</i> the calls.
2	Maximum Depth of Binary Tree	LeetCode 104 (Easy)	Depth as a value <i>and</i> depth as a cost — and the degenerate tree that kills the recursive version.
3	Climbing Stairs	LeetCode 70 (Easy)	Naive recursion, count the calls, then memoise. Fibonacci wearing a hat.
4	Flatten Nested List Iterator	LeetCode 341 (Medium)	An explicit stack because the depth is attacker-controlled — the practical reason for the conversion.

On problem 1, write both and count the lines

The recursive version is four lines. The iterative one needs a stack of nodes and a way to remember whether you have already scheduled a node's children. Count both, and say what the extra lines are storing.

On problem 2, build the tree that breaks it

Insert 5,000 sorted values into an unbalanced binary search tree, then call the recursive version. Quote the error. Then call the iterative one on the same tree.

On problem 3, instrument before you memoise

Add a global counter. Run `n = 10`, `20`, `30` and write the three call counts down. Then memoise and run the same three. Say what the ratio is at `n = 30`.

On problem 4, say why iteration is not optional here

The nesting depth comes from the input. Say what that means if the input comes from outside your system, and name the class of vulnerability.

The frame drill

1. Name the three things a stack frame holds.
2. Draw the stack for `factorial(4)` at its deepest point, with what each frame is waiting for.
3. Say which frames are suspended and what they are suspended in the middle of.
4. Say in which phase the arithmetic happens, and why that determines the space cost.

The tree-versus-stack drill

1. Draw the recursion tree for `fib(5)` and count the nodes.
2. Draw the stack at three different moments during that run.
3. State the maximum depth.
4. Instrument `fib` to count both and run it for $n = 5, 10, 20$. Write down six numbers.
5. State the sentence that separates the two, in seven words or fewer.
6. Give one function that is cheap in time and expensive in space, and one that is the reverse.

The tracing drill

1. Add a `depth` parameter to a recursive function and indent the prints.
2. Run it on the smallest input that shows the shape.
3. Now run it on `fib(25)` with the prints still in. Describe what happens and what you learned about where to trace.
4. Use `len(inspect.stack())` to confirm the depth matches your drawing.

The limit drill

1. Print `sys.getrecursionlimit()`.
2. Write the function that finds the deepest actually reachable depth. Run it from a script and from a bare prompt and write both numbers.
3. Explain the gap between the two.
4. Run a linear-depth recursion on a 5,000-element list. Quote the error.
5. Raise the limit to a million and run it again. Describe exactly what happens and what you lose.

The traceback drill

1. Trigger a `RecursionError` and read the traceback.
2. Say which end is the most recent call, and how that differs from C or Java.
3. Say what “previous line repeated N times” tells you.
4. Given a traceback, list the three possible causes and how you would tell them apart.

The conversion drill

1. Convert a pre-order traversal to an explicit stack. Count the lines.
2. Convert a post-order traversal — where the work happens after the children — and count again.
3. Name what the extra state is storing, in terms of what the machine was doing for you.
4. Say when the conversion is mechanical and when it is not.
5. Convert a linear recursion to a plain loop and say why no stack was needed at all.

The tail-call drill

1. Rewrite a linear recursion in tail form with an accumulator.
2. Run it on a 5,000-element list. Quote the error.
3. Say what a language with tail-call elimination would have done.
4. Say why Python does not do it, and what was traded for what.

The degenerate-input drill

1. Build a balanced tree of 100,000 nodes and run the recursive count. Note the depth.
 2. Build a degenerate tree of 5,000 nodes and run the same function. Quote the error.
 3. State, in one sentence, whose property the depth is.
 4. Give three real sources of degenerate input.
 5. Say what it means when the depth can be chosen by an attacker.
-

The two-systems drill

1. Compute the location writes per second and the trip writes per second.
2. State the ratio.
3. State the durability requirement of each.
4. Say where each one is stored and why.
5. Compute the total size of the live location state and say what that makes possible.

The offer-lock drill

1. Say what an offer to a driver *is*, in one sentence, and name the two earlier lessons with the same shape.
2. Write the conditional claim.
3. Say what zero rows affected means and what the matcher does next.
4. Say why the `tried` set matters.
5. Say what must happen the instant a driver accepts, and compute what it costs to forget.

The batching drill

1. Compute the expected number of offers for sequential matching at a 60 percent acceptance rate.
2. Compute the time to match at eight seconds per offer.
3. Compute the probability that at least one of three accepts.
4. Compute the expected time to match when batching three.
5. State the cost of batching in the drivers' terms.
6. Say which you would ship and what you would tune it against.

The location drill

1. Say why separate latitude and longitude indexes cannot answer “within 3 km”.
2. Explain geohashing in two sentences.
3. Describe the edge problem and the fix.
4. Compute how many cells a 3 km query reads at precision 6.
5. Say what the staleness TTL is for, and what happens without it.
6. Say why ranking by distance is worse than ranking by ETA, with an example.

The fare drill

1. Write the frozen quote and mark which field matters most.
2. Compute the overcharge when surge moves $1.4\times$ to $2.0\times$ on a ₹250 fare.
3. Say what that number is an argument about — and it is not pricing.
4. Name two earlier lessons with the same immutability rule.
5. Say what freezing the quote transfers to the platform, and how real platforms handle it.

The state-and-timeout drill

For each transition, name the actor, the timeout, and what happens when it passes:

1. REQUESTED → MATCHING
2. MATCHING → ASSIGNED
3. MATCHING → no driver
4. ASSIGNED → ARRIVING
5. ARRIVED → IN_PROGRESS
6. IN_PROGRESS → COMPLETED

Then compute how many “no driver” outcomes there are per day, and say what that number makes it.

The cancellation drill

For each, state the fee and the reason in the user's language:

1. Rider cancels before a driver is assigned.
2. Rider cancels after assignment.
3. Rider does not appear after the driver has arrived.
4. Driver cancels after accepting.
5. The system fails to match.

Say what is *counted* rather than charged, and why that is a design consequence.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Trace this recursion. What is on the stack at the deepest point?* What a frame holds, the two phases with the arithmetic on the way back up, the deepest point drawn with what each frame is waiting for, the tree-versus-stack distinction with the measured `fib(20)` numbers, the real limit, and the conversion with its honest caveat.
 2. *Design the booking flow for a cab app.* Split it into the location firehose and the trip records with the ratio, the offer as a lock with a conditional claim, sequential versus batched with both times, releasing losers immediately with its cost, geohashing with the edge problem, ranking by ETA, and the frozen surge with the 43 percent arithmetic.
 3. *How many calls does `fib(20)` make and how much stack does it use?* 21,891 and 20, why they diverge, and the seven-word sentence that separates time from space.
-

Before you move on

- [] I can name the three things in a stack frame.
- [] I can draw the stack at the deepest point with what each frame is waiting for.
- [] I can say in which phase the work happens and why that is the space cost.
- [] I instrumented `fib` and can quote calls and depth for $n = 5, 10$ and 20 .
- [] I can state the time-versus-space sentence in one line.
- [] I traced a recursion with indentation and know to use the smallest input.
- [] I measured the real reachable depth and can explain why it is not 1,000.
- [] I raised the recursion limit on purpose and saw what it costs.

- [] I can read a Python traceback from the correct end.
- [] I converted both a pre-order and a post-order traversal and can say what the extra state is.
- [] I wrote a tail-recursive function and confirmed it dies at the same depth.
- [] I built a degenerate tree and killed the recursive version on it.
- [] I can compute the location-to-trip write ratio and say where each is stored.
- [] I can write the driver claim and say what zero rows means.
- [] I can compute both matching times and say what releasing losers late costs.
- [] I can explain the geohash edge problem and the nine-cell query.
- [] I can give the surge arithmetic and say what it is an argument about.
- [] I answered all three questions above out loud.

Writing a recursive function that terminates

DATA STRUCTURES AND ALGORITHMS

Writing a recursive function that terminates

You can name the value that shrinks on every call, so your recursion always ends.

SYSTEM DESIGN

Design a rate limiter, at the object level

You can implement fixed window, sliding window and token bucket as classes.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Why does your recursion terminate?*
- *Implement a rate limiter class. Which algorithm, and why?*

Writing a recursive function that terminates

1 What this is, and why they ask it

A recursive function ends because something **gets smaller every time and cannot get smaller for ever**. That is the whole idea, and being able to name that something — out loud, in one phrase — is the difference between a recursion you trust and one you hope about.

The thing that shrinks has a name: the **measure**, or the **variant**. For summing a list it is the number of remaining elements. For binary search it is the width of the interval. For Euclid's algorithm it is the second argument. Three conditions have to hold, and all three are easy to state:

1. The measure **strictly decreases** on every recursive call.
2. It is **bounded below** — it cannot fall for ever.
3. The **base case triggers** at or before that bound.

Miss any one and the function does not terminate. And the interesting failures are not the obvious ones: it is very easy to write a recursion with a perfectly good base case, on inputs that get smaller, that still runs for ever — because “smaller” was not strict, or because the measure you had in mind is not actually the one the code changes.

They ask it because `RecursionError` is the most common failure in this phase, because “why does it terminate?” is a question with a crisp answer that most candidates cannot give, and because the moment you meet graphs — where there is no natural measure at all — the answer becomes a design decision rather than an observation.

2 The story

Kamala borrowed eighteen thousand rupees in 2019 for her daughter's admission, from a man who lends in the lane behind the vegetable market, and she has been paying him nine hundred rupees every Friday since.

That is a hundred and fifty-something Fridays. Well over a lakh, paid.

She still owes eighteen thousand rupees.

Her nephew Prashanth, who does accounts for a transport company, sat with her one Sunday because his mother had asked him to, and it took him about ten minutes. Every Friday she pays nine hundred. Every Friday nine hundred is added on. She is paying, on time, without fail, and the number at the bottom of the page has not moved once in three years.

He said the thing she repeats now to anybody who will listen. It does not matter how many times you pay. It matters whether the number gets smaller.

Her neighbour Zubeida borrowed from the co-operative society instead, and her arrangement is different in one specific way. Her weekly payment covers what is added *and* takes at least five hundred rupees off the balance. Not sometimes. Every week, at least five hundred.

Which means Zubeida can do something Kamala cannot: she can count. Thirty-six thousand, at least five hundred off a week, is at most seventy-two weeks. She knows, on the day she borrows, that there is a last Friday and roughly when it falls. She has it written in her phone.

There is a third thing Prashanth explained, and this is the one Kamala had not thought of. It is not enough that the number goes down. It has to be able to reach the bottom. If a rule said the balance halves every week, it would fall very fast — nine thousand, four and a half, two and a quarter — and it would never actually be zero, and she would be paying small amounts for ever. So the society's rule has a second half: once the balance is under a thousand, you pay it off and it is finished.

Kamala's arrangement fails on the first point. A halving rule would fail on the third. Zubeida's works because it satisfies both.

3 The idea in plain English

Kamala's balance is the **measure** of the recursion, and it does not decrease. Zubeida's does, by at least five hundred every week, and it has a floor with a rule for reaching it.

The three conditions

Every terminating recursion satisfies all three. Say them in this order.

1. Name the measure, and show it strictly decreases. Some number derived from the arguments must be smaller in the recursive call than it was in this one. Not “different”. Not “usually smaller”. **Strictly smaller, every time.**

2. It is bounded below. It cannot go down for ever. Usually it is a non-negative integer, which is why integer measures are so much easier to reason about than floating-point ones.

3. The base case triggers at or before the bound. The measure reaching its floor must actually stop the recursion. Zubeida’s “under a thousand, pay it off” rule.

```
def total(numbers: list[int], start: int = 0) → int:
    if start == len(numbers):          # 3. the base case fires when the measure is 0
        return 0
    return numbers[start] + total(numbers, start + 1)
#                                     ^ 1. the measure is len(numbers) - start,
#                                     and it drops by exactly 1
#                                     2. it is a non-negative integer, so it has a floor
```

The measure is `len(numbers) - start`, and that phrase is what you say when asked why it terminates. Not “because there is a base case” — that is only the third condition.

Naming the measure for things you have already written

FUNCTION	THE MEASURE	WHY IT STRICTLY DECREASES
<code>total(numbers, start)</code>	<code>len(numbers) - start</code>	<code>start</code> goes up by exactly 1
<code>reverse(text)</code>	<code>len(text)</code>	the slice is one character shorter
<code>power(base, n)</code>	<code>n</code>	<code>n - 1</code>
<code>fast_power(base, n)</code>	<code>n</code>	<code>n // 2</code> , and <code>n // 2 < n</code> for <code>n ≥ 1</code>
<code>binary_search(low, high)</code>	<code>high - low</code>	the interval halves
<code>merge sort</code>	the list’s length	each half is strictly shorter
<code>factorial(n)</code>	<code>n</code>	<code>n - 1</code>
<code>count_nodes(node)</code>	nodes remaining ahead	one node consumed per call

Every one of those is a non-negative integer. That is not a coincidence: integers give you all three conditions almost for free, because “strictly smaller non-negative integer” cannot go on for ever.

Euclid’s algorithm, which is the interesting one

```
def gcd(a: int, b: int) → int:
    if b == 0:
        return a
    return gcd(b, a % b)
```

What shrinks? Not `a` — the first argument becomes `b`, which may be larger. The measure is `b`, and it strictly decreases because `a % b` is always in the range `0` to `b - 1`, so the new second argument is strictly less than the old one. It is bounded below by zero, and the base case is exactly `b == 0`.

Two arguments, and only one of them is the measure. That is the shape of most non-obvious termination arguments, and being able to point at which argument is doing the work is exactly what the question is asking.

When the measure is a combination

Sometimes no single argument decreases:

```
def paths(row: int, col: int) → int:
    """How many ways to reach (0,0) moving only up or left."""
    if row == 0 or col == 0:
        return 1
    return paths(row - 1, col) + paths(row, col - 1)
```

Neither `row` nor `col` decreases on every call — the first branch leaves `col` alone. But `row + col` **decreases by exactly one on every call**, and it is bounded below by zero.

The measure does not have to be an argument. It has to be **a non-negative integer function of the arguments that strictly decreases**. Sums, differences, list lengths, “cells not yet filled”, “unvisited nodes” — all legitimate.

The three ways termination fails

Failure one: the measure does not decrease. Kamala’s balance.

```
def total(numbers, start=0):
    if start == len(numbers):
        return 0
    return numbers[start] + total(numbers, start)    # `start` never moves
```

Perfect base case. Never reached.

Failure two: it decreases but steps over the base case.

```
def power(base, n):
    if n == 0:
        return 1
    return base * power(base, n - 2)    # 5, 3, 1, -1, -3, ...
```

The measure decreases beautifully. It is not bounded below by the base case, because on odd `n` it never equals zero. **Bound with $n \leq 0$, or fix the step** — and fixing the step is the real repair, because the guard hides a genuine bug.

Failure three: it decreases but never reaches the floor. Zubeida’s halving rule.

```
def find_root(low: float, high: float) → float:
    middle = (low + high) / 2
    if f(middle) == 0:                # exact equality on a float: almost never
        return middle
    return find_root(low, middle) if f(middle) > 0 else find_root(middle, high)
```

The interval halves every time — strictly decreasing — and with floating point it eventually stops changing at all, so the recursion neither terminates nor progresses. **The fix is an epsilon:** stop when `high - low < 1e-9`. This is exactly the relative-epsilon discussion from [day 048](#), and here it is a termination question rather than a precision one.

Real measures are non-negative integers. When they are not, you have to supply the floor yourself.

Graphs: when there is no measure at all

```
def visit(node):
    for neighbour in node.neighbours:
        visit(neighbour)                # a cycle makes this run for ever
```

There is nothing that shrinks. Walking a graph with a cycle revisits the same nodes, and no argument gets smaller.

The fix **creates** a measure that did not exist:

```
def visit(node, seen: set):
    if node in seen:
        return
    seen.add(node)                    # the measure is |V| - |seen|
    for neighbour in node.neighbours:
        visit(neighbour, seen)
```

Now “the number of unvisited nodes” strictly decreases on every call that does real work, and it is bounded below by zero. **The visited set is not an optimisation; it is the termination argument.** That is worth understanding now, because it is the entire reason depth-first search on a graph looks different from depth-first search on a tree — a tree has no cycles, so the measure already exists.

Mutual recursion

```
def is_even(n: int) → bool:
    return True if n == 0 else is_odd(n - 1)

def is_odd(n: int) → bool:
    return False if n == 0 else is_even(n - 1)
```


Neither function calls itself. The measure is still `n`, and it decreases across the *pair*. The rule generalises: **the measure must decrease around every cycle in the call graph**, not merely within one function. This is where termination bugs hide, because each function looks fine on its own.

And the one nobody can answer

```
def collatz(n: int) → int:
    if n == 1:
        return 0
    return 1 + collatz(n // 2 if n % 2 == 0 else 3 * n + 1)
```

Does it terminate? **Nobody knows**. It has been checked for every starting value up to about 2^{68} and always ends at 1, and there is no proof. `3n + 1` makes the value jump *up*, so there is no obvious measure at all.

Worth knowing for one reason: it is the clearest possible demonstration that “it works on the inputs I tried” is not a termination argument. If you cannot name the measure, you do not know it terminates — you have only observed that it did.

4 The picture

The three conditions, and how each one fails.

WORKS – measure strictly decreases, bounded below, base case at the bound

measure: $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$
^ base case fires here. Terminates.

FAILS 1 – the measure does not decrease (Kamala)

measure: $5 \rightarrow 5 \rightarrow 5 \rightarrow 5 \rightarrow \dots$
the base case at 0 is perfect, and unreachable

FAILS 2 – it decreases but steps over the bound (power(base, n-2) on odd n)

measure: $5 \rightarrow 3 \rightarrow 1 \rightarrow -1 \rightarrow -3 \rightarrow \dots$
^ passed 0 without touching it

FAILS 3 – it decreases and never reaches the floor (float halving)

measure: $1.0 \rightarrow 0.5 \rightarrow 0.25 \rightarrow \dots \rightarrow 1e-17 \rightarrow 5e-18 \rightarrow \dots$
strictly decreasing for ever. Needs an epsilon.

Euclid’s algorithm, where only one argument is the measure:

```

gcd(48, 18)
  a=48 b=18    b decreases: 18
gcd(18, 12)
  a=18 b=12    b decreases: 12    ← `a` went DOWN here, but that is not the point
gcd(12, 6)
  a=12 b=6      b decreases: 6
gcd(6, 0)
  b = 0 → return 6

measure = b. Strictly decreases because a % b is in [0, b-1].
`a` is irrelevant to termination: it can grow, as in gcd(3, 100) → gcd(100, 3).

```

And the graph case, where the measure has to be manufactured:

NO MEASURE

```

A → B → C
^       |
+-----+

```

```

visit(A) → visit(B) → visit(C)
→ visit(A) → ... for ever

```

MEASURE CREATED BY THE VISITED SET

```

A → B → C
^       |
+-----+

```

```

seen = {}          unvisited = 3
seen = {A}         unvisited = 2
seen = {A,B}       unvisited = 1
seen = {A,B,C}     unvisited = 0
visit(A) returns immediately

```

the visited set IS the termination argument, not an optimisation

5 The code, built step by step

Step 1 — write the measure down as a comment, first

Before the base case, before anything:

```

def total(numbers: list[int], start: int = 0) → int:
    # measure: len(numbers) - start, decreases by 1, bounded below by 0

```

Ten seconds, and it forces the question that catches most termination bugs. If you cannot write that comment, you do not yet know whether the function ends.

Step 2 — make the base case fire *at* the bound

```

if start == len(numbers):    # measure == 0
    return 0

```

The condition should be the measure hitting its floor, stated as directly as the code allows. When the base case and the measure are written in different vocabularies — `if not numbers` while the measure is an index — that is where “reachable” quietly stops being true.

Step 3 — check the recursive call actually moves the measure

```
return numbers[start] + total(numbers, start + 1)
#                ^^^^^^^^^^ measure drops by exactly 1
```

Point at it. The failure is nearly always here: `start` instead of `start + 1`, or `n - 2` instead of `n - 1`, or a slice that is empty and therefore not smaller.

Step 4 — for multiple arguments, find the combination

```
def paths(row: int, col: int) → int:
    # measure: row + col, decreases by exactly 1 on every call
    if row == 0 or col == 0:
        return 1
    return paths(row - 1, col) + paths(row, col - 1)
```

Neither argument decreases on every branch; their **sum** does. Write the combination in the comment and the function becomes obviously terminating.

Step 5 — when there is no measure, create one

```
def reachable(start, edges: dict, seen: set | None = None) → set:
    if seen is None:
        seen = set()                # NOT a mutable default
    if start in seen:
        return seen                # the base case, in terms of the measure
    seen.add(start)                # measure: |V| - |seen|, strictly decreases
    for neighbour in edges.get(start, ()):
        reachable(neighbour, edges, seen)
    return seen
```

Two things worth narrating. `seen=None` rather than `seen=set()`, because a mutable default is created once at definition time and shared between calls. And `seen.add(start)` **before** the loop, not after — adding afterwards means a cycle re-enters the node before it was ever marked, and the whole mechanism fails.

The complete solution

```
def total(numbers: list[int], start: int = 0) → int:
    """MEASURE: len(numbers) - start. Decreases by exactly 1. Floor 0."""
    if start == len(numbers):
        return 0
    return numbers[start] + total(numbers, start + 1)

def gcd(a: int, b: int) → int:
    """Euclid. MEASURE: b - NOT a, which can grow: gcd(3, 100) → gcd(100, 3).

    b strictly decreases because a % b is always in [0, b-1]. Floor 0, and the
    base case is exactly b = 0.
    """
    if b == 0:
        return abs(a)
    return gcd(b, a % b)

def paths(row: int, col: int) → int:
    """MEASURE: row + col. Neither argument decreases on every branch; their
    SUM decreases by exactly 1 on every call. Floor 0."""
    if row < 0 or col < 0:
        raise ValueError("grid coordinates must be non-negative")
    if row == 0 or col == 0:
        return 1
    return paths(row - 1, col) + paths(row, col - 1)

def is_even(n: int) → bool:
    """MUTUAL recursion. The measure is n, and it decreases around the CYCLE
    is_even → is_odd → is_even, not within either function."""
    if n < 0:
        n = -n
    return True if n == 0 else is_odd(n - 1)

def is_odd(n: int) → bool:
    if n < 0:
        n = -n
    return False if n == 0 else is_even(n - 1)

def reachable(start: str, edges: dict[str, list[str]], seen: set[str] | None = None) → set[str]:
    """A graph has NO natural measure - a cycle revisits the same nodes for ever.

    The visited set CREATES one: |V| - |seen| strictly decreases on every call
    that does work. It is the termination argument, not an optimisation.

    `seen=None` because a mutable default is created once at definition time.
    `seen.add` BEFORE the loop, or a cycle re-enters before the node is marked.
    """
    if seen is None:
        seen = set()
    if start in seen:
        return seen
    seen.add(start)
    for neighbour in edges.get(start, ()):
        reachable(neighbour, edges, seen)
    return seen

def bisect_root(f, low: float, high: float, epsilon: float = 1e-9, depth: int = 0) → float:
    """MEASURE: high - low, which halves each time - but a float interval never
    reaches zero, so the floor must be supplied: stop below epsilon.

    The depth guard is a second, independent bound: 1e-9 relative to a starting
    width of 1 is about 30 halvings, so 200 is generous and still finite.
    """
    if depth > 200:
        raise RuntimeError("bisection did not converge")
    if high - low < epsilon:
```

```

        return (low + high) / 2
    middle = (low + high) / 2
    if f(low) * f(middle) ≤ 0:
        return bisect_root(f, low, middle, epsilon, depth + 1)
    return bisect_root(f, middle, high, epsilon, depth + 1)

def ackermann(m: int, n: int) → int:
    """The standard example where NO single argument decreases, and the measure
    is the PAIR (m, n) compared lexicographically: either m drops, or m stays
    and n drops. It terminates, and it grows absurdly fast – ackermann(3, 3) is
    61 and ackermann(4, 2) has 19,729 digits. Keep the arguments tiny."""
    if m == 0:
        return n + 1
    if n == 0:
        return ackermann(m - 1, 1)          # m decreases
    return ackermann(m - 1, ackermann(m, n - 1)) # inner: m same, n decreases

def collatz_steps(n: int, limit: int = 1000) → int:
    """NO KNOWN MEASURE. 3n+1 makes the value jump UP, and whether this always
    terminates is an open problem – verified past 2^68, never proved.

    So this one gets an explicit limit, which is the honest thing to do when you
    cannot name a measure.
    """
    steps = 0
    while n ≠ 1:
        if steps > limit:
            raise RuntimeError(f"no termination within {limit} steps from this start")
        n = n // 2 if n % 2 == 0 else 3 * n + 1
        steps += 1
    return steps

# ---- the three failures, written out so you can run them -----

def broken_no_decrease(numbers: list[int], start: int = 0) → int:
    """FAILURE 1: perfect base case, measure never moves."""
    if start == len(numbers):
        return 0
    return numbers[start] + broken_no_decrease(numbers, start)    # start, not start + 1

def broken_steps_over(base: int, n: int) → int:
    """FAILURE 2: measure decreases, but odd n steps over the base case."""
    if n == 0:
        return 1
    return base * broken_steps_over(base, n - 2)

def broken_no_visited(start: str, edges: dict[str, list[str]]) → int:
    """FAILURE 3 for graphs: no measure exists at all, so a cycle runs for ever."""
    return 1 + sum(broken_no_visited(n, edges) for n in edges.get(start, ()))

if __name__ == "__main__":
    print(total([4, 7, 2]))          # 13
    print(gcd(48, 18), gcd(3, 100), gcd(17, 0))    # 6 1 17
    print(paths(3, 3), paths(0, 5))    # 20 1
    print(is_even(10), is_odd(10), is_even(0))    # True False True

    graph = {"A": ["B"], "B": ["C"], "C": ["A", "D"], "D": []}    # A→B→C→A is a cycle
    print(sorted(reachable("A", graph)))    # ['A', 'B', 'C', 'D']

    print(round(bisect_root(lambda x: x * x - 2, 0.0, 2.0), 9))    # 1.414213562
    print(ackermann(2, 3), ackermann(3, 3))    # 9 61
    print(collatz_steps(27))    # 111

    for name, call in (
        ("no decrease", lambda: broken_no_decrease([1, 2, 3])),
        ("steps over", lambda: broken_steps_over(2, 5)),
        ("no visited set", lambda: broken_no_visited("A", graph)),
    ):

```

```
try:
    call()
except RecursionError:
    print(f"{name}: RecursionError: maximum recursion depth exceeded")
```

6 What it costs

Proving termination costs nothing at run time

The measure is a comment and an argument you were passing anyway. **Naming it is free**, and it converts a class of bug that only appears on certain inputs into something you can check by reading. That is a very good trade and it is the reason this lesson exists.

The two cases where termination has a real price

The visited set on a graph is $O(V)$ memory and $O(1)$ per lookup:

```
a graph of 1,000,000 nodes
visited set of ids: ~32 B per entry in a Python set → ~32 MB
without it:         does not terminate at all
```

Thirty-two megabytes to convert “runs for ever” into “runs”. Not a trade-off — a requirement.

The epsilon on a float recursion costs precision:

```
bisection on [0, 2], epsilon = 1e-9
halvings needed: log2(2 / 1e-9) ≈ 31
→ 31 calls, and an answer accurate to about 9 decimal places
epsilon = 1e-15: ~51 calls, and near the limit of double precision
```

Smaller epsilon buys precision at about one extra call per bit. The floor is where doubles stop distinguishing values at all — which is why an absolute epsilon fails on very large numbers and a relative one is safer, as on [day 048](#).

How the measure predicts the complexity

The measure does more than prove termination — **the way it shrinks tells you the running time**.

measure drops by 1	→ n calls	→ $O(n)$	total, factorial
measure halves	→ $\log_2(n)$ calls	→ $O(\log n)$	binary search, fast_power
measure drops by 1 with TWO calls per level	→ $\sim 2^n$ calls	→ $O(2^n)$	naive fib
measure = row + col, two calls per level	→ $C(\text{row}+\text{col}, \text{row})$	→ exponential	paths(n, n)

`paths(3, 3)` is 20 and `paths(10, 10)` is 184,756 — the number of calls is a binomial coefficient. So “what shrinks, and by how much, and how many calls per level” gives you both the termination proof and the complexity, from the same sentence.

Euclid, priced

```
gcd(a, b): the measure b at least HALVES every two calls  
→  $O(\log(\min(a, b)))$  calls  
gcd of two numbers near  $10^{18}$ : about 90 calls
```

The worst case is consecutive Fibonacci numbers, which is a pleasing fact and the reason the bound is logarithmic rather than linear.

Ackermann, as the limiting case

```
ackermann(3, 3) = 61  
ackermann(4, 2) =  $2^{65536} - 3$ , which has 19,729 digits
```

It terminates — the pair `(m, n)` decreases lexicographically on every call — and it is not *computable in practice* past tiny arguments. **Termination and feasibility are different properties**, and Ackermann is the clean example: a proof that a function ends tells you nothing about whether you can wait.

7 The traps

Trap 1 — “it terminates because there is a base case”

The most common wrong answer, and it only covers the third condition. A base case that is never reached is worth nothing — Kamala’s arrangement had a perfectly good rule for what happens at zero. **The answer to “why does it terminate?” starts with “the measure is...”, not with “there is a base case at...”.**

Trap 2 — the measure that does not move

```
return numbers[start] + total(numbers, start)
```

```
RecursionError: maximum recursion depth exceeded
```

Usually a typo — `start` for `start + 1`, or `node` for `node.next`. Point at the recursive call and ask “which argument is smaller here?” every single time.

Trap 3 — stepping over the bound

```
return base * power(base, n - 2)
```

The measure decreases and misses zero on odd inputs. Guarding with `n ≤ 0` stops the crash and leaves the real bug — the function now returns a wrong answer for odd exponents. **Fix the step, not the guard.**

Trap 4 — a float measure with no floor

```
if low == high:                                # equality on floats: essentially never true
    return low
```

The interval halves for ever and the base case never fires. Then, worse, at some point `(low + high) / 2` equals `low` exactly and the recursion stops making progress while still recursing. **Use an epsilon, and add a depth guard as an independent bound.**

Trap 5 — no visited set on a graph

```
for neighbour in node.neighbours:
    visit(neighbour)
```

Terminates on a tree and runs for ever on anything with a cycle — so it passes every test written against sample data and fails on production data with one back-edge. **The visited set is the termination argument.**

And the subtler version: adding to `seen` *after* the recursive loop instead of before. The node is re-entered on a cycle before it was ever marked, and nothing changes.

Trap 6 — mutual recursion where each function looks fine

```
def is_even(n): return True if n == 0 else is_odd(n)      # note: n, not n - 1
def is_odd(n):  return False if n == 0 else is_even(n - 1)
```

`is_even` alone looks correct. `is_odd` alone looks correct. The measure has to decrease **around the cycle**, and here it drops on one leg and not the other, so pairs of calls make no progress. When functions call each other, draw the cycle and check the measure across the whole loop.

Trap 7 — a mutable default as the accumulator

```
def reachable(start, edges, seen=set()):                # created ONCE at definition time
```


The second top-level call inherits the first call's `seen`, so it returns immediately having “already visited” everything. Silent, and the function *looks* like it terminated correctly. Use `None` and create it inside.

Trap 8 — believing an empirical check

Collatz has been verified for every starting value below about 2^{68} and is not proved. Your function working on the fifty inputs you tried is exactly as strong an argument. **If you cannot name the measure, add an explicit limit and raise** — that is the honest thing to do, and it turns an infinite loop into a diagnosable error.

8 In the interview

How it gets asked

- The direct version, usually right after you write a recursive function: “*Why does that terminate?*”
- The debugging version: “*This gives a `RecursionError`. Find the bug.*”
- The graph version, which is the important one: “*Write a depth-first search.*” — and the visited set is the answer to a termination question, not a performance one.
- The trap version: “*Does this terminate?*” on something with a non-obvious measure — Euclid, Ackermann, or Collatz.
- The design version: “*How would you make sure this always finishes?*”

What to say out loud, in the first ninety seconds

1. **Answer with the measure, not with the base case.** “It terminates because `len(numbers) - start` strictly decreases by one on every call, it is a non-negative integer so it has a floor, and the base case fires exactly when it reaches zero.”
2. **Point at the argument that moves.** “This is the one that shrinks” — and if two arguments are involved, say which, or name the combination.
3. **Say all three conditions, briefly.** Strictly decreasing, bounded below, base case at the bound.
4. **For a graph, say why a set is required rather than helpful.** “A graph has no natural measure, so the visited set creates one: the number of unvisited nodes. It is the termination argument.”
5. **For floats, name the floor you supplied.** “A halving interval never reaches zero, so I stop below an epsilon and I would also add a depth guard as an independent bound.”

The follow-ups

“Why does that terminate?” “Because there is a value that strictly decreases on every call and cannot decrease for ever. Here it is `len(numbers) - start`, which drops by exactly one and is bounded below by zero, and the base case fires precisely when it reaches zero. Those are the three conditions: strictly decreasing, bounded below, and a base case that triggers at the bound. Saying ‘there is a base case’ only answers the third — Kamala’s loan had a perfectly good rule for reaching zero and never got there.”

“What about Euclid’s algorithm? a can get bigger.” “It can — `gcd(3, 100)` calls `gcd(100, 3)` — and that is fine, because `a` is not the measure. `b` is. The new second argument is `a % b`, which is always between zero and `b - 1`, so `b` strictly decreases. It is bounded below by zero and the base case is exactly `b = 0`. That is the general shape of a non-obvious termination argument: several arguments, and only one of them, or one combination, is doing the work. Incidentally the same measure gives you the complexity — `b` at least halves every two steps, so it is $O(\log \min(a, b))$, about ninety calls on numbers near 10^{18} .”

“Write a depth-first search on a graph.” “The visited set is the first thing I write, and I would say why: a graph has no natural measure. On a tree, ‘nodes below me’ shrinks automatically; on a graph with a cycle, nothing shrinks and the traversal runs for ever. The set manufactures a measure — the number of unvisited nodes — which strictly decreases on every call that does work. So it is the termination argument, not an optimisation. Two details: mark the node *before* recursing into its neighbours, or a cycle re-enters before it is marked; and pass the set explicitly rather than as a mutable default, which would be shared across calls.”

“This recursion crashes on some inputs and not others.” “That is almost always condition two or three rather than condition one — the measure does decrease, but it steps over the base case, or the base case is stated in different terms from the measure. The classic is decrementing by two towards an equality test at zero: it terminates for even inputs and never for odd. I would look at the recursive call, name the measure, and check that the base-case condition is exactly the measure hitting its floor. And I would fix the step rather than widening the guard to `≤ 0`, because the wider guard turns a crash into a wrong answer.”

“Does Collatz terminate?” “Nobody knows. It has been verified for every starting value up to about 2^{68} and there is no proof — `3n + 1` makes the value jump upward, so there is no obvious measure. I like it as an example because it makes the point that ‘it worked on the inputs I tried’ is not a termination argument. If I had to ship something like that, I would give it an explicit step limit and raise, which turns an infinite loop into a diagnosable error.”

“Give me a recursion where no single argument decreases.” “Counting paths in a grid: from `(row, col)` you recurse to `(row - 1, col)` and `(row, col - 1)`, so neither argument decreases on every branch — but `row + col` decreases by exactly one on every call and is bounded below by zero. The measure does not have to be an argument; it has to be a non-negative integer function of the arguments that strictly decreases. The extreme case is Ackermann, where the measure is the *pair* compared lexicographically: either `m` drops, or `m` stays the same and `n` drops. It terminates and it is still completely infeasible past tiny arguments, which is a nice reminder that termination and practicality are different questions.”

A model answer

Asked: *why does your recursion terminate?*

“Because there is a value that gets strictly smaller on every call and cannot get smaller for ever. That value has a name — the measure — and naming it is the answer.

For this function the measure is `len(numbers) - start`. Three things have to be true about it, and I would check all three explicitly.

First, it strictly decreases. The recursive call passes `start + 1`, so the measure drops by exactly one. Not ‘usually smaller’ — every call, without exception.

Second, it is bounded below. It is a non-negative integer, so it cannot fall for ever. That is why integer measures are so much easier to reason about than floating-point ones: a strictly decreasing non-negative integer has to stop.

Third, the base case fires at that bound. `start == len(numbers)` is exactly the measure reaching zero. This is the condition people give as the whole answer, and on its own it is worth nothing — a base case that is never reached does not help. The classic failure is decrementing by two towards an equality test at zero: it works for even inputs and recurses for ever on odd ones, and the base case looks perfectly correct while it happens.

The measure is not always an argument. In Euclid’s algorithm, `a` can grow — `gcd(3, 100)` becomes `gcd(100, 3)` — and the measure is `b`, which strictly decreases because `a % b` is always less than `b`. In a grid-path count neither coordinate decreases on every branch, but their *sum* does. So the real definition is: a non-negative integer function of the arguments that strictly decreases.

Two cases are worth calling out because they are where this stops being theory.

On a graph, there is no natural measure at all — a cycle brings you back to the same node with the same arguments. The visited set *creates* one: the number of unvisited nodes, which strictly decreases on every call that does work. So the set is the termination argument, not a performance optimisation, and it has to be updated before recursing rather than after.

And with floating point, a halving interval decreases strictly and never reaches zero, so there is no floor unless I supply one. That is what an epsilon is for, and I would also add a depth guard as an independent bound, because two independent reasons to stop is cheap insurance.

The last thing I would say is that the measure gives me the complexity as well as the proof. Drops by one, one call per level: linear. Halves: logarithmic. Drops by one with two calls per level: exponential. Same sentence, two answers.”

9 Recall card

- “Why does it terminate?” is answered with the **MEASURE**, not the base case. Three conditions, all three needed: it **strictly decreases** every call · it is **bounded below** · the **base case fires at that bound**. “There is a base case” only covers the third — and an unreachable one is worth nothing.
- The measure need not be an argument — it is any **non-negative integer function of the arguments** that strictly decreases. Euclid’s is `b`, not `a` (which can grow: `gcd(3,100) → gcd(100,3)`); a grid path’s is `row + col`; Ackermann’s is the **pair, lexicographically**. And in **mutual recursion** it must decrease **around the cycle**, not within one function.
- **Three ways it fails, and only the first is obvious.** The measure never moves (`start` instead of `start + 1`) · it decreases but **steps over** the bound (`n - 2` towards `= 0` on odd inputs — works for even, never for odd) · it decreases and **never reaches** the floor (a halving float interval). **Fix the step, not the guard** — widening to `≤ 0` turns a crash into a wrong answer.
- **A graph has NO natural measure, so the visited set creates one:** `|V| - |seen|`. It is the **termination argument, not an optimisation** — which is the whole reason graph DFS differs from tree DFS. Mark the node **before** recursing, and pass the set explicitly (**never a mutable default**).
- **The measure also gives you the complexity, from the same sentence.** Drops by 1, one call per level → **$O(n)$** . Halves → **$O(\log n)$** (Euclid: ~ 90 calls at 10^{18}). Drops by 1 with two calls per level → **$O(2^n)$** . And **termination $\neq$ feasibility**: Ackermann provably terminates and `ackermann(4,2)` has 19,729 digits. If you cannot name a measure — **Collatz**, verified past 2^{68} and unproved — add an explicit limit and raise.

1 What this is, and why they ask it

A rate limiter answers one question: **may this caller do this now?** A hundred requests a minute per user, five logins an hour per IP address, one password-reset email a day per account.

The interface is one method returning a boolean, which is why this looks like a small problem. It is not, and the reason is that there are five different algorithms with genuinely different behaviour, and the obvious one — count requests per minute, reset the counter each minute — lets a caller send **twice the limit** in a two-second window, at exactly the moment when they are most likely to be attacking you.

The second reason is memory. The exact algorithm keeps a timestamp per request per caller, which at a hundred requests a minute and a million users is nearly a gigabyte of Redis. The approximate ones keep two numbers per caller. Choosing between exactness and eight hundred megabytes is the interesting decision, and it is one you can only make if you know what the approximation actually gives up.

They ask it because it is small enough to write in twenty minutes, because the algorithm comparison is a real engineering trade with numbers on both sides, and because the distributed version has a race condition that most implementations ship with.

2 The story

Sundaram has held the kerosene licence for four streets since 1998, and the rule from the office has always been the same: two litres per card per month.

For years it worked. Then, some time around 2015, he started running out on the second of every month and having nothing at all by the sixth, and he could not understand it, because the total he was issuing had not changed.

It took him about three months to see it, and he saw it standing at the shutter on the first of February with forty people outside and no kerosene.

People were taking their two litres on the last day of one month and their next two litres on the first day of the next. Four litres in two days. Then nothing for eight weeks, and then four litres in two days again.

The month total was exactly right. Every card, two litres a month, no exceptions. And his shop was empty for two days and then idle for four weeks.

What he did — and he had to argue about it at the office for a while — was stop counting by month.

Now it works differently. Every card builds up an allowance, half a litre a week. You can let it build; the most it will ever hold is two litres, so there is no point saving longer than a month. And you can take what you have built up whenever you like, all at once if you want. If you have not come for three weeks, you can take one and a half litres today. If you came yesterday, you have almost nothing today and you come back next week.

The households were suspicious for about two months and then stopped noticing. What changed for Sundaram is that his shop is never empty and never idle. The same total goes out every month. It just goes out at roughly the rate it is used, instead of in a two-day rush against a line on a calendar.

He says the mistake was that the rule was about the calendar and not about the person.

3 The idea in plain English

Sundaram's month is a **fixed window**. The four litres in two days is the **boundary burst**. And what he replaced it with — an allowance that builds up at a steady rate and stops at a cap — is a **token bucket**, which is the answer to almost every rate-limiting question you will be asked.

The interface, before the algorithms

```
@dataclass(frozen=True)
class Decision:
    allowed: bool
    remaining: int           # how many more, right now
    retry_after: float       # seconds until the next one would be allowed

class RateLimiter(Protocol):
    def allow(self, key: str, now: float) → Decision: ...
```

Three things about that signature are worth defending.

It returns a `Decision`, not a `boolean`. The caller needs to send `X-RateLimit-Remaining` and `Retry-After` headers, and a client that knows when to come back is worth far more than one that retries blindly and gets refused again. A `boolean` forces every caller to guess.

now is a parameter. A limiter that calls the clock itself cannot be tested without sleeping, and tests that sleep are tests nobody runs. Injecting time is the single most important testability decision in this class.

The key is a string. Per user, per IP, per API key, per user-and-endpoint — the limiter does not care, and the *choice* of key is the caller's design decision. Say that: the algorithm and the key are separate questions and people conflate them.

Algorithm one: fixed window counter

Divide time into fixed minutes. Count. Reset.

```
window = int(now // self.period)                # which minute are we in
if window != self.current_window:
    self.count, self.current_window = 0, window # reset
if self.count < self.limit:
    self.count += 1
    return Decision(True, ...)
```

Two numbers per key. Trivial to implement, trivial to distribute — Redis `INCR` on a key named after the window, with an expiry.

And it has Sundaram's problem:

```
limit: 100 per minute

10:00:59   100 requests   → all allowed (window "10:00")
10:01:00   100 requests   → all allowed (window "10:01")
-----
200 requests in about one second, against a limit of 100 per minute
```

Twice the limit, in the time it takes to cross a boundary. And the boundary is *predictable* — it is the top of the minute — so anybody attacking you can aim for it deliberately.

That is usually the disqualifying fact. Not always: for a coarse quota where the burst does not matter — “1,000 API calls a day, billing purposes” — fixed window is exactly right and the simplest thing that works.

Algorithm two: sliding window log

Keep the timestamp of every request. Drop the ones older than the window. Count what is left.

```
hits = self._hits[key]
while hits and hits[0] ≤ now - self.period:    # expire from the front
    hits.popleft()
if len(hits) < self.limit:
    hits.append(now)
    return Decision(True, ...)
```


This is **exactly correct** — no boundary effect, no approximation — and it is the version you already wrote on [day 077](#) as a deque of timestamps.

The price is memory:

```
limit 100/minute, per key: up to 100 timestamps × 8 B = 800 B
1,000,000 active keys                                = 800 MB
```

Eight hundred megabytes to be exactly right, against about a hundred for fixed window. For a limit of 100 that is a real but survivable cost; for a limit of 10,000 requests an hour it is not.

And one rule that is easy to get wrong and which you met on day 077: **a rejected request must not be recorded**. Append only when you allow, or a client hammering the endpoint keeps refreshing their own ban and never escapes.

Algorithm three: sliding window counter

The compromise almost every CDN actually ships. Keep two counters — this window and the previous one — and estimate the sliding count by weighting the previous window by how much of it is still in view.

```
elapsed = (now % self.period) / self.period    # how far into the current window
estimate = self.previous_count * (1 - elapsed) + self.current_count
if estimate < self.limit:
    ...
```

```
limit 100/min, 30 seconds into the current minute
previous minute: 80 requests, current minute so far: 40
estimate = 80 × 0.5 + 40 = 80 → allowed, and 20 remain
```

Two numbers per key, and the boundary burst is gone. It assumes requests were spread evenly across the previous window, which is not true — so it can be slightly wrong in both directions. Cloudflare published measurements putting the error under one percent for real traffic, which is why it is the default in a lot of production.

Algorithm four: token bucket — the usual answer

A bucket holds up to `capacity` tokens. Tokens are added at `rate` per second. A request takes one token if there is one.

```
elapsed = now - self.last_refill
self.tokens = min(self.capacity, self.tokens + elapsed * self.rate) # lazy refill
self.last_refill = now
if self.tokens ≥ 1:
    self.tokens -= 1
    return Decision(True, ...)
```

Sundaram's half a litre a week, capped at two litres.

Three properties, and together they are why this wins:

- **Two numbers per key** — the token count and the last refill time. Same memory as fixed window.
- **No boundary at all.** Time is continuous; there is no calendar to game.
- **Bursts are allowed, deliberately and boundedly.** A client that has been quiet accumulates up to `capacity` tokens and may spend them at once. That is usually what you *want* — a user who has not called for an hour should be able to load a page that makes ten requests — and the burst is capped by the bucket, not open-ended like the fixed window's.

Capacity and rate are separate knobs, which is the real advantage: “100 per minute sustained, but up to 20 at once” is `rate = 100/60` and `capacity = 20`, and no other algorithm here can express those two independently.

The **lazy refill** is the implementation trick: no timer, no background job. You compute how many tokens *would* have arrived since the last call, at the moment of the call. Nothing has to be running, which is the same lazy-expiry idea as the cinema seat lock — and here it applies, because nothing has to *happen* when tokens accrue.

Algorithm five: leaky bucket

Requests join a queue that drains at a fixed rate; the queue has a maximum length and overflow is rejected. Output is perfectly smooth — no bursts at all, ever.

Use it when the thing downstream cannot absorb a burst: a payment provider that allows exactly 10 requests a second, a legacy system, a serial device. The cost is **latency** — a request may wait in the queue — and that makes it wrong for a user-facing API, where you would rather refuse quickly than hold a connection.

Token bucket limits the rate you accept. Leaky bucket shapes the rate you emit. That one sentence tells them apart.

The comparison, which is the answer to “which algorithm”

	MEMORY PER KEY	BOUNDARY BURST	BURSTS ALLOWED	EXACT	USE IT WHEN
Fixed window	2 numbers	yes, 2 × the limit	accidental	no	Coarse quotas where a burst is harmless
Sliding log	up to limit timestamps	no	no	yes	Small limits, exactness required
Sliding counter	2 numbers	no	no	~1% error	High-volume API gateways
Token bucket	2 numbers	no	yes, bounded	n/a	The default
Leaky bucket	queue	no	no	n/a	Protecting something downstream

Default to token bucket and be able to say why you would not: a hard quota with no bursts allowed (sliding log), or a billing counter where the boundary does not matter (fixed window).

The distributed version, and the race everybody ships

One process is easy. Ten application servers sharing one limit is not, because the state has to be shared — Redis — and the update has to be atomic.

The obvious Redis fixed-window implementation:

```
INCR ratelimit:user42:10:01
EXPIRE ratelimit:user42:10:01 60
```

Two commands, and there is a gap between them. If the process dies, or the connection drops, or Redis fails over between the `INCR` and the `EXPIRE`, the key has no expiry and lives for ever. The user is now permanently at their limit, and the only symptom is one user complaining that they are rate-limited and nobody else is.

Three correct fixes:

- **A Lua script**, which Redis executes atomically — read, compute, write, expire, in one round trip. This is the standard answer and it is also what makes token bucket possible in Redis at all, since the refill is a read-modify-write.
- `SET key 1 NX EX 60` followed by `INCR` for subsequent hits — the first write sets the expiry atomically.
- **Redis 7’s** `EXPIRE ... NX`, or a pipeline in a `MULTI/EXEC` transaction.

The boundary burst, which is the reason fixed window is usually wrong:

the counter is correct in both windows, and the client sent 2x the limit in one second – at a boundary they can predict exactly

bursts are allowed, and BOUNDED by the capacity – which the fixed window's are not

exactness costs 700 MB, and it buys you correctness at the boundary that the sliding counter already approximates to within about 1%

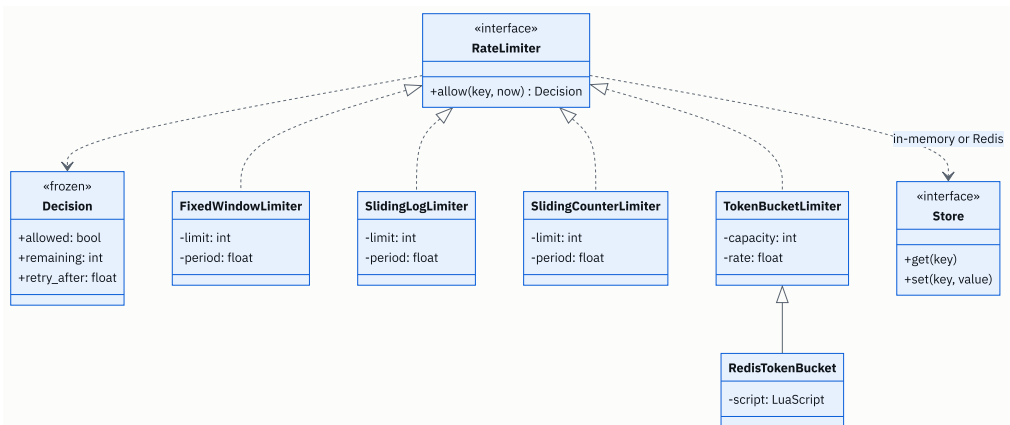


FIGURE 89.1

What to notice: **four implementations of one interface, and every one of them is real**. This is a case where the strategy interface is unarguable — the gate from [day 076](#) is “can you name a second implementation somebody wants”, and here there are four with different trade-offs and different customers.

5 How it actually works

Move 1 • Clarify (minutes 0–5)

“What are we limiting per — user, IP, API key, or user-and-endpoint?” — The key is a caller decision, but I need to know whether it is one limit or several. “Is a burst acceptable, or must the rate be strictly smooth?” — This chooses token bucket versus leaky bucket, and it is the question that actually decides the algorithm. “One process or many?” — Because a shared limit needs shared state and an atomic update. “What should a limited caller be told?” — Propose [429](#) with `Retry-After`, because a client that knows when to come back is worth more than one that guesses.

“I will assume a single limit per key for now, that some burst is acceptable, and that we run several application servers so the state is shared. I am not designing the policy — who gets what limit — only the mechanism.”

Move 2 · The interface, first

```
@dataclass(frozen=True)
class Decision:
    allowed: bool
    remaining: int
    retry_after: float          # seconds; 0.0 when allowed

class RateLimiter(Protocol):
    def allow(self, key: str, now: float) → Decision: ...
```

Write this before any algorithm, and say the two design points: **Decision** rather than a **boolean**, so the caller can send useful headers, and **now injected**, so the tests do not sleep.

Move 3 · Fixed window, and its failure demonstrated

```
class FixedWindowLimiter:
    def __init__(self, limit: int, period: float = 60.0) → None:
        self._limit, self._period = limit, period
        self._counts: dict[tuple[str, int], int] = {}

    def allow(self, key: str, now: float) → Decision:
        window = int(now // self._period)
        slot = (key, window)
        count = self._counts.get(slot, 0)
        if count ≥ self._limit:
            return Decision(False, 0, (window + 1) * self._period - now)
        self._counts[slot] = count + 1
        return Decision(True, self._limit - count - 1, 0.0)
```

Ten lines, and then **show the burst rather than describe it**: allow 100 at `t = 59.9` and 100 more at `t = 60.1`, and print that 200 were allowed inside a quarter of a second. Demonstrating the flaw is worth more than asserting it.

Move 4 · Sliding log, exact and expensive

```
class SlidingLogLimiter:
    def allow(self, key: str, now: float) → Decision:
        hits = self._hits[key]
        cutoff = now - self._period
        while hits and hits[0] ≤ cutoff:
            hits.popleft()          # expire from the FRONT
        if len(hits) ≥ self._limit:
            return Decision(False, 0, hits[0] + self._period - now)
        hits.append(now)           # only on ALLOW
        return Decision(True, self._limit - len(hits), 0.0)
```

Two details worth pointing at. `retry_after` is exact and free here — the oldest hit plus the period is precisely when a slot opens, which no other algorithm can tell you as precisely. And `hits.append` comes after the rejection check, so a rejected request is not recorded; the alternative bans a hammering client for ever.

Move 5 • Token bucket, the one to write

```
class TokenBucketLimiter:
    """capacity = how big a burst is allowed; rate = the sustained limit.

    LAZY refill: nothing runs in the background. At each call, compute how many
    tokens would have accrued since the last one. No timer, no sweeper.
    """

    def __init__(self, capacity: int, rate_per_second: float) → None:
        self._capacity, self._rate = capacity, rate_per_second
        self._state: dict[str, tuple[float, float]] = {} # key → (tokens, last)

    def allow(self, key: str, now: float) → Decision:
        tokens, last = self._state.get(key, (float(self._capacity), now))
        tokens = min(self._capacity, tokens + (now - last) * self._rate)

        if tokens < 1.0:
            self._state[key] = (tokens, now)
            return Decision(False, 0, (1.0 - tokens) / self._rate)

        self._state[key] = (tokens - 1.0, now)
        return Decision(True, int(tokens - 1.0), 0.0)
```

Fifteen lines, two numbers of state, no background job, no boundary. Say the three properties as you write: **capacity and rate are independent knobs**, the refill is lazy, and the burst is bounded.

Note the new-key default is a **full** bucket. That is a deliberate choice — a first-time caller gets their full burst — and the alternative, starting empty, punishes new users. Say which you chose.

Move 6 • Making it distributed, and the race

```
TOKEN_BUCKET_LUA = """
local tokens_key, ts_key = KEYS[1], KEYS[2]
local rate, capacity, now = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])

local tokens = tonumber(redis.call('GET', tokens_key)) or capacity
local last = tonumber(redis.call('GET', ts_key)) or now
tokens = math.min(capacity, tokens + (now - last) * rate)

local allowed = tokens ≥ 1
if allowed then tokens = tokens - 1 end

local ttl = math.ceil(capacity / rate) * 2
redis.call('SET', tokens_key, tokens, 'EX', ttl)
redis.call('SET', ts_key, now, 'EX', ttl)
return { allowed and 1 or 0, tokens }
"""
```

Three things to say about this script, and they are the whole distributed answer.

It is atomic. Redis runs a script to completion with nothing interleaved, so the read-modify-write of the token count cannot race between ten application servers. Doing the same thing with `GET`, compute, `SET` from Python would lose updates under concurrency, and the symptom would be that the limit is silently too generous.

It sets the expiry in the same command as the value. The classic bug is `INCR` followed by `EXPIRE`: if anything fails between the two, the key never expires and that caller is limited **for ever**, with no symptom except one user complaining.

The TTL is derived, not guessed — twice the time it takes to refill a full bucket, after which the state is meaningless anyway. That is what keeps a million idle keys from accumulating.

Real systems

- `nginx`'s `limit_req` is a leaky bucket with an optional `burst` parameter, and `limit_req_zone` sizes the shared memory — the memory arithmetic in §6 is exactly what you are doing when you set it.
 - **Envoy**, **Kong** and **AWS API Gateway** all offer token bucket; AWS's API Gateway exposes it directly as a **rate** and a **burst**, which is the two-knob property made into configuration.
 - **Stripe** publishes its approach and returns `429` with `Retry-After`; **GitHub** returns `X-RateLimit-Remaining` and `X-RateLimit-Reset` on every response, which is why their clients can back off intelligently.
 - **Cloudflare** uses the sliding window *counter* and published the measurement that its error is under one percent on real traffic — which is the evidence behind choosing an approximation.
 - `redis-cell` is a Redis module implementing the “generic cell rate algorithm”, a token bucket variant, as a single atomic command — worth naming as the version where somebody else has already written the Lua.
-

6 The numbers

Memory, which is the decision

```
limit: 100 requests per minute
active keys: 1,000,000
```

fixed window	key + 2 ints	~100 B	→	100 MB
sliding counter	key + 2 ints	~100 B	→	100 MB
token bucket	key + 2 floats	~120 B	→	120 MB
sliding log	key + up to 100 × 8 B timestamps	~800 B	→	800 MB

Exactness costs about 700 MB at a million keys, and buys correctness at a boundary that the sliding counter already approximates to within about one percent. At a limit of 100 that is arguable. At a limit of 10,000 an hour:

```
sliding log at limit 10,000: 10,000 × 8 B = 80 KB per key
1,000,000 keys                      = 80 GB
```

Eighty gigabytes. The sliding log does not scale with the limit, and that is the sentence that decides against it whenever the limit is large.

The boundary burst, exactly

```
fixed window, limit 100/minute
worst case: 100 at t = 59.999 and 100 at t = 60.001
→ 200 requests in 2 milliseconds, against a stated limit of 100 per minute

the effective worst-case rate is 2x the configured limit,
and the boundary is at a predictable instant
```

Twice the limit, aimable. That is the number that disqualifies fixed window for anything security-adjacent — a login limiter that can be doubled at the top of the minute is a login limiter that will be doubled at the top of the minute.

Throughput and the cost of checking

```
50,000 requests/second across the fleet
→ 50,000 rate-limit checks/second

in-memory (single process): ~0.5 µs per check → 0.025 core-seconds/second
Redis Lua script:           ~0.2 ms round trip → 10 core-seconds/second of WAITING
```

The Redis round trip is not CPU, it is latency — but it is **0.2 ms added to every single request**, which on a 20 ms endpoint is one percent, and on a 2 ms endpoint is ten percent. That is the argument for a **two-tier limiter**: a local in-process bucket sized at, say, the fleet limit divided by the number of servers, backed by the shared Redis one. Most requests never leave the process.

Redis sizing

```
1,000,000 keys × 120 B (token bucket) = 120 MB of data
Redis overhead per key, roughly 2x    → ~250 MB
ops: 50,000/second, 1 script call each
a single Redis node handles ~100,000 simple ops/second
→ comfortably one node, and one is a single point of failure
```

Which is the honest note: one node is enough for throughput and is a **single point of failure for every request in the system**. The usual answer is that a rate limiter should **fail open** — if Redis is unreachable, allow the request rather than refuse it, because refusing everything to protect against overuse is a worse outage than the overuse. Say that; it is a real operational decision and it is the opposite of what most people's instinct is.

The bucket knobs, worked

```
"100 per minute sustained, bursts of up to 20"
  rate      = 100 / 60 = 1.67 tokens per second
  capacity = 20

a client quiet for 12 s: tokens = min(20, 12 × 1.67) = 20 → 20 at once
sustained:              1.67 per second = 100 per minute
empty bucket:           retry_after = 1 / 1.67 = 0.6 s
```

No other algorithm here can express those two numbers independently, and that is the practical reason token bucket wins.

7 The trade-offs

What this design gives up

Token bucket allows bursts, and sometimes that is exactly wrong. If the thing you are protecting cannot absorb twenty requests at once — a payment provider with a hard 10/second ceiling, a legacy system, a database with a small connection pool — then a bounded burst still knocks it over. That is what leaky bucket is for, and the cost is queueing latency. **Token bucket limits what you accept; leaky bucket shapes what you emit.**

Approximate algorithms are approximately right, and the error is not symmetric in importance. The sliding counter can allow slightly more than the limit or slightly fewer. Allowing 101 when the limit is 100 is harmless for an API quota and unacceptable for “three login attempts before lockout”. For small, security-relevant limits, use the exact log — the memory cost is trivial because the limit is small.

Shared state adds latency to every request, including the allowed ones. A Redis round trip on the hot path is 0.2 ms whether the answer is yes or no. The two-tier mitigation — a local bucket in front of the shared one — means the local limits are only approximately fair between servers, which is usually fine and occasionally is not.

A rate limiter is a single point of failure in front of everything. Which is why it should **fail open**: if the store is unreachable, allow. Failing closed converts a Redis blip into a total outage, and that trade is almost always wrong — the exception being a limiter protecting something that genuinely cannot survive being overrun, where failing closed is the lesser harm. Decide deliberately and write it in the code as a comment, because the default behaviour of an exception is to fail closed by accident.

The key choice is not the algorithm's problem, and it is where the real bugs are. Limiting by IP punishes everybody behind one office NAT and does nothing against a botnet. Limiting by user account does nothing to the signup endpoint, where there is no user yet. Real systems use several limiters with different keys at once, and the interesting design work is choosing them.

No per-endpoint or per-plan variation here. Real APIs have tiers, and that turns a limiter into a lookup of “which limit applies to this caller and this route” followed by the mechanism. Worth naming as a separate concern — the *policy* and the *mechanism* — rather than tangling them.

“I would change this design if...”

- **...the downstream cannot absorb bursts at all.** Leaky bucket, and accept the latency.
- **...the limit is small and security-relevant** — login attempts, password resets. Sliding log, exact, and the memory is nothing because the limit is five.
- **...the check is on a very low-latency endpoint.** Two-tier: a local bucket in front of the shared one, so most requests never make a network call.
- **...limits vary by plan and endpoint.** Separate the policy lookup from the mechanism, and cache the policy.

The honest concession

Four algorithms and an interface is a lot of machinery for one boolean. If the requirement were “stop one broken script hammering us” with no tiers, no bursts and one server, a dictionary of counters and a fixed window would be twenty lines and completely adequate — the boundary burst does not matter when you are defending against an accident rather than an adversary. The reason to build the strategy interface here is unusual

and worth stating plainly: **all four implementations are genuinely wanted by somebody**, which is exactly the gate an interface has to pass, and the algorithm choice is a per-limit decision rather than a system-wide one.

8 In the interview

How it gets asked

- The direct version: *“Implement a rate limiter. Which algorithm and why?”*
- The flaw probe: *“What is wrong with counting requests per minute and resetting?”*
- The memory probe: *“How much memory does that need for a million users?”*
- The distributed probe, which is the important one: *“Now ten servers share the limit.”*
- The operational probe: *“Redis is down. What happens?”*

The timed script

Minutes 0–5 · Clarify. Limit per what key? Are bursts acceptable, or must the rate be smooth? One process or many? What does a limited caller get told? The burst question decides the algorithm, so ask it first.

Minutes 5–8 · The interface. `Decision` rather than a boolean, with `remaining` and `retry_after`, and `now` injected for testability. Both are one-sentence justifications and both are noticed.

Minutes 8–16 · Fixed window, and demonstrate the burst. Write it, then show 200 requests in two milliseconds against a limit of 100 a minute. Say when it is still the right answer.

Minutes 16–24 · Sliding log and its memory, then the sliding counter as the compromise, with the one-percent error figure.

Minutes 24–32 · Token bucket, written out. Lazy refill, two independent knobs, bounded burst, two numbers of state.

Minutes 32–40 · Distributed. The `INCR` / `EXPIRE` race, the Lua script, the derived TTL, the two-tier latency mitigation, and fail-open.

The follow-ups

“What is wrong with a counter that resets every minute?” “The boundary. A caller can send the full limit in the last moments of one window and the full limit again in the first moments of the next — a hundred at 10:00:59 and a hundred at 10:01:00 — which is twice the configured limit inside about a second. And the boundary is at a predictable instant, so it can be aimed at deliberately. That is disqualifying for anything security-

adjacent, like a login limiter. It is still the right answer for a coarse quota where a burst is harmless — a thousand API calls a day for billing — because it is two numbers per key and trivially distributable.”

“Which algorithm would you use, and why?” “Token bucket, by default. Two numbers of state per key, so the same memory as a fixed window. No boundary to game, because time is continuous. And crucially the **rate and the burst are separate knobs**: ‘a hundred a minute sustained, up to twenty at once’ is a rate of 1.67 tokens a second and a capacity of twenty, and no other algorithm here can express those two independently. The refill is lazy — at each call you compute how many tokens would have accrued since the last one — so nothing runs in the background. I would move off it in two cases: a small security-relevant limit like login attempts, where I want the exact sliding log and the memory is trivial because the limit is five; or a downstream that genuinely cannot take a burst, where I want a leaky bucket and accept the queueing latency.”

“How much memory for a million users?” “Depends entirely on the algorithm, and this is usually the deciding number. Token bucket, fixed window and sliding counter are all two numbers per key — call it a hundred to a hundred and twenty bytes with the key, so a hundred-odd megabytes. A sliding log keeps a timestamp per request in the window: at a limit of a hundred a minute that is eight hundred bytes a key and eight hundred megabytes. And it scales with the *limit* — at ten thousand an hour it is eighty kilobytes per key and eighty gigabytes, which rules it out entirely. So exactness costs about seven hundred megabytes at a small limit and becomes impossible at a large one.”

“Now ten servers share the limit.” “The state moves to Redis, and the update has to be atomic — which is where most implementations have a bug. The obvious version is `INCR` on a key named for the window, then `EXPIRE`. Those are two commands, and if anything fails between them the key never expires and that caller is rate-limited for ever, with no symptom except one user complaining. The fix is a Lua script, which Redis runs atomically: read the tokens and the timestamp, refill, decide, write both back with a TTL, in one round trip. That is also what makes token bucket possible in Redis at all, since the refill is a read-modify-write that would otherwise lose updates under concurrency.”

“What does that cost on the hot path?” “About 0.2 milliseconds per request for the Redis round trip, on every request including the allowed ones. On a twenty-millisecond endpoint that is one percent and I would not care; on a two-millisecond endpoint it is ten percent and I would. The mitigation is two tiers: a local in-process bucket sized at roughly the fleet limit divided by the number of servers, backed by the shared one, so most requests never make a network call. The cost is that the local limits are only approximately fair between servers, which is usually acceptable.”

“Redis is down. What happens?” “It should **fail open** — allow the request. That is the opposite of most people’s instinct, and the reasoning is that a rate limiter sits in front of everything, so failing closed converts a Redis blip into a total outage. Being briefly over-permissive is almost always the lesser harm than serving nothing. The exception is a limiter protecting something that genuinely cannot survive being overrun, where failing closed is correct. Either way it has to be a *decision written in the code*, because the default behaviour of an unhandled exception is to fail closed by accident.”

“What do you return to a limited caller?” “A 429, and — the part that matters — `Retry-After` with the number of seconds until a slot opens, plus `X-RateLimit-Remaining` on every response. A client that knows when to come back backs off correctly; a client that just gets `false` retries immediately and makes the problem worse. That is why the method returns a `Decision` object rather than a boolean, and it is nearly free: the sliding log knows exactly when a slot opens, and the token bucket computes it as the tokens still needed divided by the rate.”

A model answer

Asked: *implement a rate limiter. Which algorithm, and why?*

“Before the algorithm, one question that decides it: are bursts acceptable, or does the rate have to be strictly smooth? Because that is the difference between a token bucket and a leaky bucket, and everything else follows.

I would start with the interface, because two decisions there matter. The method returns a `Decision` — allowed, remaining, and retry-after — not a boolean, because the caller needs to send a `Retry-After` header, and a client that knows when to come back is worth far more than one that retries blindly. And the current time is a parameter, not something the class reads from the clock, because otherwise every test has to sleep and nobody runs tests that sleep.

Now the algorithms, and I want to start with the obvious one specifically to show why it is wrong. Fixed window: count requests in each minute, reset the counter each minute. Two numbers per key, trivially distributable — and a caller can send the full limit at 10:00:59 and the full limit again at 10:01:00, which is twice the limit in about a second, at an instant they can predict exactly. For a billing quota that is harmless. For a login limiter it is disqualifying.

The exact fix is a sliding log: keep the timestamp of every request, drop the ones outside the window, count what is left. It is precisely correct and it tells you exactly when the next slot opens. The cost is memory — one timestamp per request per key, so at a hundred a minute and a million users that is eight hundred megabytes, against a hundred for the counter versions. Worse, it scales with the *limit*: at ten thousand an hour it is eighty gigabytes, which rules it out.

So the answer I would actually give is a **token bucket**. A bucket holds up to `capacity` tokens, tokens accrue at a fixed rate, a request spends one. Three properties make it the default. Two numbers of state, so the same memory as the fixed window. No boundary, because time is continuous and there is no calendar to game. And the rate and the burst are *independent knobs* — ‘a hundred a minute sustained, up to twenty at once’ is a rate of 1.67 a second and a capacity of twenty, and no other algorithm here can express those separately.

The implementation trick is the lazy refill: nothing runs in the background. At each call I compute how many tokens would have accrued since the last call and cap at the capacity. Fifteen lines, no timer, no sweeper.

For several servers sharing a limit, the state goes to Redis and the update must be atomic — and this is where most implementations have a bug. `INCR` followed by `EXPIRE` is two commands, and if anything fails between them the key never expires and that user is limited for ever, with no symptom except one person complaining. So: a Lua script, which Redis runs atomically, doing the read, refill, decide and write with a derived TTL in one round trip.

Two operational notes. That round trip is about 0.2 milliseconds on every request, so on a very fast endpoint I would put a local bucket in front of the shared one. And the limiter should **fail open** — if Redis is unreachable, allow — because it sits in front of everything and failing closed turns a Redis blip into a total outage. That has to be written down deliberately, because an unhandled exception fails closed by accident.”

9 Recall card

- Return a **Decision** (allowed, remaining, retry_after), not a boolean, and inject **now**. A client that knows when to come back backs off correctly; a limiter that reads its own clock cannot be tested without sleeping.
- **Fixed window's flaw is the boundary and it is aimable: the full limit at 10:00:59 plus the full limit at 10:01:00 is $2 \times$ the limit in ~ 1 second.** Fine for a coarse billing quota, disqualifying for anything security-adjacent.
- **Memory is the deciding number.** Fixed window / sliding counter / token bucket are **2 numbers per key** ($\sim 100\text{--}120\text{ B} \rightarrow \sim 100\text{ MB}$ at 1M keys). **Sliding log is exact and keeps a timestamp per request** — 800 B/key at a limit of 100 $\rightarrow$ **800 MB**, and it scales with the *limit*: **80 GB** at 10,000/hour. Cloudflare's sliding **counter** approximates to within $\sim 1\%$ for two numbers.
- **Default to the token bucket.** Two numbers, no boundary, **lazy refill** (nothing runs in the background), and — the real reason — **rate and burst are independent knobs**: “100/min with bursts of 20” = rate 1.67/s, capacity 20. *Token bucket limits what you accept; leaky bucket shapes what you emit.*
- **Distributed: INCR then EXPIRE is the bug everybody ships** — a failure between them leaves a key with no TTL and that caller limited **for ever**. Use a **Lua script** (atomic read-modify-write, TTL derived from capacity $\div$ rate). Add a **local bucket in front** if 0.2 ms per request matters. And **fail open** when the store is down — a limiter sits in front of everything, so failing closed turns a blip into an outage.

PRACTICE

Practice — Writing a recursive function that terminates

DSA topic: Writing a recursive function that terminates **System design topic:** Design a rate limiter, at the object level

Code these, in this order

One rule for the whole set: **write the measure as a comment before the base case.** One line naming what shrinks, by how much, and what its floor is. If you cannot write it, you do not yet know that the function ends.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Greatest Common Divisor of Strings	LeetCode 1071 (Easy)	Euclid's shape, where the obvious argument is not the measure.
2	Number of Islands	LeetCode 200 (Medium)	A grid with no natural measure — the visited marking is the termination argument.
3	Unique Paths	LeetCode 62 (Medium)	A measure that is a combination of two arguments, and the call count it predicts.
4	Sqrt(x)	LeetCode 69 (Easy), then the float version	An integer measure that terminates, then a float one that needs an epsilon.

On problem 1, say the measure before the code

It is not the first argument. Say which one shrinks and why it must, and only then write the three lines.

On problem 2, break it deliberately

Write the flood fill without marking cells visited. Run it on a two-by-two grid of land. Quote the error. Then move the marking to *after* the recursive calls instead of before, and run it again — say why that fails too.

On problem 3, count the calls

Instrument the naive recursion. Run it for a 3×3 , 5×5 and 10×10 grid and write the three counts down. Say what the measure predicts, and then memoise and count again.

On problem 4, do both versions

The integer version terminates because the interval is an integer range. The float version does not, because a halving interval never reaches zero. Write both, and say exactly what you had to add.

The three-conditions drill

State all three from memory, then apply them to each function:

1. `total(numbers, start)`
2. `power(base, n)` with `n - 1`
3. `power(base, n)` with `n - 2`
4. `gcd(a, b)`
5. `paths(row, col)`
6. `bisect_root(low, high)` with no epsilon
7. A depth-first search on a graph with a cycle, with no visited set

For each, say which condition fails, or say that all three hold.

The name-the-measure drill

For each, name the measure in one phrase and say by how much it drops:

1. Summing a list by index.
2. Reversing a string by slicing.
3. Binary search on an integer range.
4. Merge sort.
5. Euclid's algorithm.
6. Counting grid paths.
7. Flood fill on a grid.
8. Ackermann.

Two of the eight have a measure that is not a single argument. Name them and say what the measure is.

The failure drill

Write each of the three failures, run it, and quote the error:

1. The measure never moves.
2. The measure steps over the bound.
3. The measure decreases and never reaches the floor.

For the second one, “fix” it by widening the guard to `≤ 0` and say precisely what you have now got instead of a crash.

The graph drill

1. Write a traversal of a cyclic graph with no visited set. Quote the error.
2. Add the set and say what measure you have just created.
3. Move `seen.add` to after the loop instead of before. Run it and explain the result.
4. Change `seen=None` to `seen=set()` as a default. Call the function twice on different graphs and describe what the second call returns.
5. Say in one sentence why a tree does not need the set, and what property it has that a graph lacks.

The float drill

1. Write a bisection with `if low == high` as its base case. Run it.
2. Add an absolute epsilon and run it again. Count the calls.
3. Compute the number of halvings needed for an epsilon of 1e-9 and of 1e-15 on the interval [0, 2].
4. Try an absolute epsilon on an interval around 1e12 and describe what happens.
5. Add a depth guard as a second, independent bound and say why two reasons to stop is cheap.

The mutual-recursion drill

1. Write `is_even` and `is_odd`.
2. Say what the measure is and around what it decreases.
3. Break one leg so it does not decrement. Run it and quote the error.
4. Say why each function looks correct in isolation.
5. State the general rule about cycles in the call graph.

The measure-predicts-complexity drill

For each, use the measure to state the complexity, then verify by counting calls:

1. Drops by 1, one call per level.
2. Halves, one call per level.
3. Drops by 1, two calls per level.
4. `paths(row, col)`, where the measure is `row + col` and there are two calls per level.

Then run `paths(10, 10)` with a counter and check your prediction.

The honesty drill

1. Write the Collatz step function.
 2. Say whether it terminates and what is actually known.
 3. Add an explicit limit and raise. Say why that is the honest thing to do.
 4. State, in one sentence, why “it worked on the inputs I tried” is not a termination argument.
-

The interface drill

1. Write the `Decision` dataclass and say why it is not a boolean.
2. Say why `now` is a parameter.
3. Say why the key is an opaque string, and what that separates.
4. Name the two HTTP headers this makes possible and what a client does with each.

The boundary drill

1. Implement the fixed window limiter.
2. Allow 100 requests at $t = 59.9$ and 100 more at $t = 60.1$. Print how many were allowed and over what span.
3. State the effective worst-case rate as a multiple of the configured limit.
4. Say why the boundary being *predictable* makes it worse than being merely wrong.
5. Name a case where fixed window is still the right choice.

The memory drill

Compute each, showing the multiplication, for 1,000,000 active keys:

1. Fixed window.
2. Sliding counter.
3. Token bucket.
4. Sliding log at a limit of 100 per minute.
5. Sliding log at a limit of 10,000 per hour.

Then say which number rules the sliding log out, and what property of it causes that.

The token-bucket drill

1. Implement it with lazy refill.
2. Express “100 per minute sustained, bursts of up to 20” as a rate and a capacity.
3. Compute the tokens available after 12 seconds of silence.
4. Compute `retry_after` on an empty bucket.

5. Say what a new key's bucket should start at, and defend the choice.
6. Say which single property no other algorithm here can express.

The distributed drill

1. Write the naive Redis fixed window as two commands.
2. Describe exactly what happens if the process dies between them, and what the symptom looks like.
3. Say why a `GET`, compute, `SET` token bucket in application code is wrong under concurrency.
4. Write the Lua script version and say what makes it safe.
5. Say how you would choose the TTL, and why it is derived rather than guessed.

The operations drill

1. Compute the added latency per request for a Redis round trip, and its percentage on a 20 ms and a 2 ms endpoint.
2. Describe the two-tier mitigation and what it gives up.
3. Say what should happen when the store is unreachable, and defend it.
4. Say what the default behaviour of an unhandled exception is, and why that matters here.
5. Name one case where the opposite choice is correct.

The choose-the-algorithm drill

For each, name the algorithm and the deciding reason:

1. 1,000 API calls per day, for billing.
2. Five login attempts per hour per account.
3. 100 requests per minute per API key on a public API, ten servers.
4. Calls to a payment provider that permits exactly 10 per second.
5. A page load that legitimately makes 15 requests at once.
6. Password reset emails, one per day per account.

Two of the six want the exact algorithm for the same reason. Name it.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why does your recursion terminate?* The measure named first, all three conditions, the argument that actually shrinks (and in Euclid, the one that does not), the combination case, the graph case where the visited set *creates* the measure, and the float case where you supply the floor.
 2. *Implement a rate limiter. Which algorithm, and why?* The interface and its two decisions, fixed window with its boundary demonstrated, the sliding log's memory arithmetic, token bucket as the default with the two independent knobs and the lazy refill, the `INCR / EXPIRE` race, and fail-open.
 3. *What is wrong with counting requests per minute and resetting?* The boundary burst with the exact numbers, why predictability makes it worse, and the one case where it is still right.
-

Before you move on

- ☐ I write the measure as a comment before writing the base case.
- ☐ I can state all three termination conditions from memory.
- ☐ I can name the measure for eight different functions, including two that are combinations.
- ☐ I know why `b` is Euclid's measure and `a` is not.
- ☐ I wrote all three failure modes and quoted the errors.
- ☐ I know what widening a guard to `≤ 0` gives me instead of a crash.
- ☐ I broke a graph traversal by removing the visited set, and by marking too late.
- ☐ I used a mutable default as an accumulator and saw the second call misbehave.
- ☐ I can say why a tree needs no visited set and a graph does.
- ☐ I added an epsilon and a depth guard to a float recursion.
- ☐ I can use the measure to predict complexity, and verified it by counting calls.
- ☐ I can say why "it worked on my inputs" is not a termination argument.
- ☐ I demonstrated the fixed-window boundary burst with real numbers.
- ☐ I can give the memory arithmetic for all four algorithms at a million keys.
- ☐ I can express a rate and a burst as two independent token-bucket knobs.
- ☐ I can describe the `INCR / EXPIRE` race and its symptom.
- ☐ I can defend fail-open, and name the case where the opposite is right.
- ☐ I answered all three questions above out loud.

Recursion on arrays and strings

DATA STRUCTURES AND ALGORITHMS

Recursion on arrays and strings

You can convert any simple loop into recursion, and back again.

SYSTEM DESIGN

Design an in-memory cache with eviction

You can design a cache with a pluggable eviction policy and a TTL.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Reverse a string recursively. Now do it without extra space.*
- *Design an in-memory cache. Now make the eviction policy swappable.*

1 What this is, and why they ask it

Arrays and strings are where recursion is most often *asked for* and least often *needed*. A loop over a list is clearer, faster, and does not die at a thousand elements. So this day is not about convincing you to write recursion here — it is about being able to, correctly, when someone asks.

Two techniques do almost all of the work.

Pass an index, never a slice. `f(items[1:])` copies the list on every call, which turns an $O(n)$ algorithm into $O(n^2)$ in time and allocation, silently. `f(items, start + 1)` does not. This is the single most common performance bug in recursive array code and it never raises an error — the function is correct and mysteriously slow.

Two indices, moving toward each other. Palindromes, reversing in place, comparing from both ends — all of them are `low` and `high` walking inward, and the base case is `low ≥ high`. That is the recursive form of the two-pointer technique from [day 028](#), and it is what “without extra space” means when an interviewer says it.

They ask it because “write this recursively” is a five-minute filter, because the slicing trap catches most people, and because the honest answer to “which would you ship?” — the loop — is a small test of judgement that some candidates fail by over-committing to the clever version.

2 The story

Nagaraj has run the stationery shop by the bus stand for twenty-six years, and once a month he reads his order out to the supplier over the phone.

It is a long list. Three hundred and forty lines on a good month, and it lives in a spreadsheet on his phone that his daughter set up in 2019.

His nephew Manju sat with him last year to help and did it in a way that Nagaraj still talks about.

Every time the call dropped — and it drops, that corner has always been bad — Manju would go back to the top of the list and start reading down again to find his place. Every single time. He said he did not want to lose track. Six drops in one call, and each time he

read from the first line again, out loud, to find where they had stopped. Twenty minutes to read a list that takes six.

Nagaraj does not do that. He puts his thumb on the screen where he has got to and leaves it there. The call drops, he says the last item again to be safe, and carries on from under his thumb. It costs him nothing. The list has not moved; only his place in it has.

He said the same thing to Manju about four times before it stuck. You are not making a new list every time. You are just remembering where you are.

There is a second thing they do together at the end of the year, when the stock has to be checked against the shelves. Two of them, one list. Nagaraj starts at the top and works down; Manju starts at the bottom and works up. They call out to each other as they go, and when they meet in the middle, somewhere around line one hundred and seventy, the job is done and neither of them has walked the shop twice.

Manju is quite proud of that one and points out that it takes half the time. Nagaraj says it takes the same time, there are just two of you.

3 The idea in plain English

Nagaraj's thumb is the **index**. Manju re-reading from the top is **slicing**. And the two of them meeting in the middle is the **two-index** recursion.

Rule one: pass an index, never a slice

```
def total(numbers: list[int]) → int:
    if not numbers:
        return 0
    return numbers[0] + total(numbers[1:])    # SLICE: copies the whole list
```

Correct, and quietly quadratic. There are n calls, and each one builds a new list of nearly n elements, so the total work is $n + (n-1) + \dots + 1 \approx n^2/2$ element copies. Summing a thousand numbers does half a million copies and allocates a thousand lists.

```
def total(numbers: list[int], start: int = 0) → int:
    if start == len(numbers):
        return 0
    return numbers[start] + total(numbers, start + 1)    # INDEX: nothing is copied
```

n calls, constant work each. **$O(n)$ time, no allocation at all.**

Same rule for strings, and it bites harder there because strings are immutable, so `text[1:]` copies every time and so does every `+`.

The recursion should look at a *view* of the data, never at a *copy* of it. An index is a view.

Rule two: two indices, moving inward

For anything symmetric — palindromes, reversing, comparing ends — one index is not enough.

```
def is_palindrome(text: str, low: int = 0, high: int | None = None) → bool:
    if high is None:
        high = len(text) - 1
    if low ≥ high:
        return True
    if text[low] ≠ text[high]:
        return False
    return is_palindrome(text, low + 1, high - 1)
```

The base case is `low ≥ high`, and it must be `≥` rather than `=`: on an even-length string the two indices **cross without ever being equal**. "abba" goes `(0,3)` then `(1,2)` then `(2,1)` — never equal. That single character is the most common bug in this shape.

The measure — from [day 089](#) — is `high - low`, which drops by two each call and is bounded below. Naming it takes five seconds and it is what makes the base case obviously right.

Rule three: an accumulator, when the loop had one

A loop with a running total converts to a recursion with an extra parameter:

```
running = 0
for value in numbers:
    running += value
```

loop: running = 0; for x: running += x

```
def total_acc(numbers, start=0, running=0):
    if start == len(numbers):
        return running
    return total_acc(numbers, start + 1, running + numbers[start])
```

recursion: running is a parameter

Whatever the loop carried between iterations becomes a parameter. That is the whole conversion recipe, and it works in both directions: to turn a recursion into a loop, look at what is being combined on the way back up and make it a running variable.

This version is also **tail-recursive** — nothing happens after the call returns — which a language with tail-call elimination would turn into a loop for free. Python does not, deliberately, as on [day 088](#), so it dies at the same depth. Know the term; write the loop.

The private-helper pattern

Public functions should not have `start` and `high` in their signature — callers should not be able to pass a wrong one, and the defaults are noise.

```
def is_palindrome(text: str) → bool:
    def check(low: int, high: int) → bool:
        if low ≥ high:
            return True
        return text[low] == text[high] and check(low + 1, high - 1)
    return check(0, len(text) - 1)
```

The inner function closes over `text`, so it is not passed on every call, and the outer signature is clean. **This is how recursion on arrays is written in real code**, and it is worth doing in an interview because it shows you are thinking about the caller.

Rule four: divide and conquer, when the answer combines

The two shapes so far shrink the problem by one. The other shape splits it in half:

```
def maximum(numbers: list[int], low: int, high: int) → int:
    if low == high:
        return numbers[low] # one element
    middle = (low + high) // 2
    return max(maximum(numbers, low, middle),
               maximum(numbers, middle + 1, high))
```

The measure is `high - low`, which halves. There are still `n` leaves and `2n - 1` calls in total, so it is $O(n)$ — **the same as the loop, with more calls and log-depth stack**. For `max` that is a strictly worse way to get the same answer, and it is worth saying so.

Divide and conquer earns its keep when **combining two halves is cheaper than redoing the work**: merge sort ([day 084](#)), quickselect, maximum subarray. It is not a general speed-up.

Strings: the extra trap

Strings are immutable, so every `+` builds a new string:

```
def reverse(text: str) → str:
    if len(text) ≤ 1:
        return text
    return reverse(text[1:]) + text[0] # a slice AND a concatenation, per call
```

`n` calls, each building a string of length up to `n`: $O(n^2)$, twice over. At 10,000 characters that is a hundred million character copies to do something `text[::-1]` does instantly.

The fix is to collect into a list and join once, or to work on a `list(text)` in place with two indices. This is the same lesson as building strings on [day 020](#), and recursion makes it easier to commit by accident because the concatenation is hidden in a return statement.

And the honest part

For a flat array or string, **the loop is better** — clearer, $O(1)$ space, and it does not die at a thousand elements. Recursion here is a *demonstration*, and if an interviewer asks for it, give it and then say what you would ship.

Recursion earns its place when the structure is recursive — trees, nested lists, directories — or when the problem splits and recombines. Those are the next ten days.

4 The picture

Slicing against indexing, drawn as what exists in memory.

SLICING — a new list per call

call 1:	[4, 7, 2, 9, 1]	the original
call 2:	[7, 2, 9, 1]	a NEW list, 4 elements copied
call 3:	[2, 9, 1]	a NEW list, 3 copied
call 4:	[9, 1]	2 copied
call 5:	[1]	1 copied

total copied: $4+3+2+1 = 10$ → $n^2/2$ for n elements
lists allocated: 4

INDEXING — one list, a moving marker

```
[4, 7, 2, 9, 1]
^start=0
  ^start=1
    ^start=2
      ^start=3
        ^start=4
```

total copied: 0
lists allocated: 0

Nagaraj's thumb against Manju reading from the top.

The two-index shape, and why the base case is $\geq$:

"abcba" (odd length)

```
a b c b a
^   ^
^   ^   low=0 high=4
^   ^   low=1 high=3
^   ^   low=2 high=2
      low == high → stop
```

"abba" (even length)

```
a b b a
^   ^
^   ^   low=0 high=3
^   ^   low=1 high=2
X       low=2 high=1
      low > high → stop
```

ODD lengths stop with $\text{low} = \text{high}$.

EVEN lengths never have $\text{low} = \text{high}$; they CROSS.

So the base case is $\text{low} \geq \text{high}$, and ``==`` alone loops for ever on even input.

And the two shapes of array recursion:

SHRINK BY ONE

```
f(0, n)
|
f(1, n)
|
f(2, n)
|
...
depth n, n calls

dies at ~1000
```

SPLIT IN HALF

```
          f(0, n)
         /  \
      f(0, n/2) f(n/2, n)
       /  \   /  \
    ...   ... ...   ...

depth log n, 2n-1 calls
O(n) work, O(log n) stack

fine at a million
```

The second shape is safe at any size and the first is not, which is why merge sort recurses happily on a million elements and `total` does not.

5 The code, built step by step

Step 1 — write the loop first, then convert

```
running = 0
for value in numbers:
    running += value
return running
```

Look at it and name three things: **what changes between iterations** (the position), **what is carried across** (the running total), and **when it stops** (the end of the list). Those become the index, the accumulator, and the base case.

Step 2 — the signature, with a private helper

```
def total(numbers: List[int]) → int:
    def go(start: int) → int:
        ...
    return go(0)
```

The public function takes what a caller would sensibly pass. The helper takes the book-keeping. `numbers` is closed over rather than passed, so it is not on the stack `n` times.

Step 3 — the base case as the measure hitting its floor

```
if start == len(numbers):          # measure: len(numbers) - start, floor 0
    return 0
```

Written as the measure reaching zero, exactly as on [day 089](#).

Step 4 — the recursive call, with the index moving

```
return numbers[start] + go(start + 1)
```

Point at `start + 1`. That is the measure decreasing, and it is where the bug lives when there is one.

Step 5 — the two-index version, with the `>=`

```
def check(low: int, high: int) → bool:
    if low >= high:                # >= not ==, or even lengths never stop
        return True
    return text[low] == text[high] and check(low + 1, high - 1)
```

Say “greater than or equal, because even lengths cross rather than meet” as you type the `>=`. It is one character and it is the whole bug.

Step 6 — in-place reversal, which is what “no extra space” means

```
def swap(low: int, high: int) → None:
    if low >= high:
        return
    chars[low], chars[high] = chars[high], chars[low]
    swap(low + 1, high - 1)
```

$O(1)$ extra space beyond the stack, which is the honest caveat: **recursion is never truly $O(1)$ space**, because the frames are space. If the interviewer means strictly constant, the answer is the loop.

The complete solution

```
def total(numbers: list[int]) → int:
    """Sum by INDEX, not by slice.

    The slicing version copies the list on every call: n calls x O(n) copy each
    = O(n^2) time and allocation, silently. This one allocates nothing.

    O(n) time, O(n) stack. The loop is better; this is the demonstration.
    """
    def go(start: int) → int:
        if start == len(numbers):          # measure: len(numbers) - start
            return 0
        return numbers[start] + go(start + 1)
    return go(0)

def total_slicing(numbers: list[int]) → int:
    """The version to write once, measure, and never write again. O(n^2)."""
    if not numbers:
        return 0
    return numbers[0] + total_slicing(numbers[1:])

def total_accumulator(numbers: list[int]) → int:
    """Whatever the loop carried between iterations becomes a parameter.

    Also tail-recursive – nothing happens after the call – which a language
    with tail-call elimination would turn into a loop. Python does not, so
    this dies at the same depth as any other recursion.
    """
    def go(start: int, running: int) → int:
        if start == len(numbers):
            return running
        return go(start + 1, running + numbers[start])
    return go(0, 0)

def is_palindrome(text: str) → bool:
    """Two indices moving inward. O(n) time, O(n) stack, no copying.

    The base case is low ≥ high, NOT ==: on an even length the indices cross
    without ever being equal, so '=' alone never terminates.
    """
    def check(low: int, high: int) → bool:
        if low ≥ high:
            return True
        return text[low] == text[high] and check(low + 1, high - 1)
    return check(0, len(text) - 1)

def reverse_in_place(chars: list[str]) → list[str]:
    """LeetCode 344: reverse a character list in place, recursively.

    O(1) extra space apart from the stack – and that caveat matters: recursion
    is never truly constant space, because the frames ARE space.
    """
    def swap(low: int, high: int) → None:
        if low ≥ high:
            return
        chars[low], chars[high] = chars[high], chars[low]
        swap(low + 1, high - 1)
    swap(0, len(chars) - 1)
    return chars

def reverse_string_slow(text: str) → str:
    """The trap, twice over: a slice AND a concatenation on every call.
    n calls each building a string of up to n characters → O(n^2)."""
    if len(text) ≤ 1:
        return text
    return reverse_string_slow(text[1:]) + text[0]
```

```

def reverse_string_fast(text: str) → str:
    """Collect into a list, join once. O(n) total character copying."""
    parts: list[str] = []

    def go(index: int) → None:
        if index < 0:
            return
        parts.append(text[index])
        go(index - 1)

    go(len(text) - 1)
    return "".join(parts)

def maximum(numbers: list[int]) → int:
    """Divide and conquer. The measure (high - low) HALVES, so the stack depth
    is log n and this survives a million elements where shrink-by-one does not.

    But it is 2n-1 calls to do what one loop does in n steps: for `max`,
    divide and conquer is a demonstration, not an improvement. It earns its
    keep only when combining halves is cheaper than redoing the work.
    """
    if not numbers:
        raise ValueError("maximum() of an empty sequence")

    def go(low: int, high: int) → int:
        if low == high:
            return numbers[low]
        middle = (low + high) // 2
        return max(go(low, middle), go(middle + 1, high))

    return go(0, len(numbers) - 1)

def binary_search(numbers: list[int], target: int) → int:
    """Recursion where the depth is log n, so the stack cost is irrelevant:
    about 20 frames at a million elements."""
    def go(low: int, high: int) → int:
        if low > high:
            return -1
        middle = (low + high) // 2
        if numbers[middle] == target:
            return middle
        if numbers[middle] < target:
            return go(middle + 1, high)
        return go(low, middle - 1)
    return go(0, len(numbers) - 1)

def remove_character(text: str, unwanted: str) → str:
    """Build with a list and one join, never with `+` inside the recursion."""
    kept: list[str] = []

    def go(index: int) → None:
        if index == len(text):
            return
        if text[index] != unwanted:
            kept.append(text[index])
        go(index + 1)

    go(0)
    return "".join(kept)

def flatten(items: list) → list:
    """The case where recursion is genuinely the right tool: the STRUCTURE is
    recursive, so there is no index that expresses the problem."""
    out: list = []

    def go(node) → None:
        for item in node:

```



```

        if isinstance(item, list):
            go(item)          # depth = nesting depth, not length
        else:
            out.append(item)

    go(items)
    return out

if __name__ == "__main__":
    import time

    print(total([4, 7, 2]), total([], total_accumulator([4, 7, 2]))    # 13 0 13
    print(is_palindrome("malayalam"), is_palindrome("abba"),
          is_palindrome("abc"), is_palindrome(""), is_palindrome("a"))
    # True True False True True

    print("".join(reverse_in_place(list("recursion"))))                # noisrucer
    print(reverse_string_fast("recursion"))                             # noisrucer
    print(maximum([3, 9, 2, 9, 1]), binary_search([1, 3, 5, 7, 9], 7)) # 9 3
    print(remove_character("mississippi", "s"))                        # miiippi
    print(flatten([1, [2, [3, [4, 5]], 6], 7]))                          # [1,2,3,4,5,6,7]

    # the slicing trap, measured
    sample = list(range(900))
    start = time.perf_counter()
    total_slicing(sample)
    slicing = time.perf_counter() - start
    start = time.perf_counter()
    total(sample)
    indexing = time.perf_counter() - start
    print(f"slicing {slicing:.5f}s indexing {indexing:.5f}s ratio {slicing / indexing:.1f}x")

    # and the shape that survives a big input, against the one that does not
    big = list(range(100_000))
    print(maximum(big))                                                  # 99999
    try:
        total(big)
    except RecursionError as error:
        print(f"RecursionError: {error}")
    print(sum(big))                                                      # 4999950000

```

6 What it costs

Slicing against indexing

```

slicing:    n calls, each copying up to n elements
            total element copies = n + (n-1) + ... + 1 = n(n+1)/2
            allocations: n lists
            → O(n^2) time AND O(n^2) total allocation

indexing:   n calls, constant work each
            allocations: 0
            → O(n) time, O(n) stack only

```

```

n = 900:    slicing copies ~405,000 elements to add up 900 numbers

```

Measured at $n = 900$ — about the largest that fits under the recursion limit — the slicing version took **0.0037 s against 0.00013 s, roughly $29\times$ slower**, and the gap widens with n because it is a different complexity class rather than a constant factor.

Strings are worse

```
reverse_string_slow("...", n characters:
    n calls
    each builds a new string of up to n characters
    →  $O(n^2)$  character copies, AND  $O(n)$  intermediate strings

n = 10,000: ~50,000,000 character copies
            text[::-1] does it in one pass
```

Two $O(n^2)$ sources in one function — the slice and the concatenation — which is why string recursion written naively is so much slower than array recursion written naively.

The two shapes of stack cost

```
shrink by one:  depth n      → dies at ~1000 in Python
split in half:  depth log n  → 17 frames at 100,000; 20 at 1,000,000
```

```
maximum([...100,000 items...])  works: depth 17
total([...100,000 items...])    RecursionError
```

Same input, same language, opposite outcomes, and the only difference is how the measure shrinks. That is the practical reason merge sort recurses comfortably on a million elements and a sum-by-index does not.

Recursion against the loop, honestly

	loop	recursion (index)
time	$O(n)$	$O(n)$
space	$O(1)$	$O(n)$ stack
max input	unbounded	~1000
speed	baseline	~1.5-2x slower (call overhead)
clarity on a flat array	better	worse

There is no column where recursion wins on a flat array. That is the honest summary, and saying it is a better answer than defending the recursive version.

Where recursion does win

```
nested structure (flatten):  depth = NESTING depth, not length
a list of 1,000,000 items nested 3 deep:  3 frames
the iterative version needs an explicit stack and is longer

divide and conquer (merge sort, quickselect):
depth log n, and combining halves is cheaper than redoing the work
```

The rule: **recursion is for recursive structure and for divide-and-conquer, not for iteration.**

7 The traps

Trap 1 — slicing in the recursive call

```
return numbers[0] + total(numbers[1:])
```

No error, correct answer, $O(n^2)$. It is the single most common recursion bug on arrays and it is invisible until the input grows. **Pass an index.** Same for `text[1:]`, which is worse because strings copy on every operation.

Trap 2 — `==` instead of `>=` in the two-index base case

```
if low == high:
    return True
```

Works on odd lengths, recurses for ever on even ones, because `low` and `high` **cross without meeting**: `"abba"` goes `(0,3) → (1,2) → (2,1)` and never equal.

```
RecursionError: maximum recursion depth exceeded
```

on exactly half of all inputs. This is condition two from [day 089](#) — the measure decreases and steps over the bound.

Trap 3 — building a string with `+` inside the recursion

```
return reverse(text[1:]) + text[0]
```

Two quadratic behaviours at once. Collect into a list and `"".join` at the end, or work on a `list(text)` in place.

Trap 4 — a mutable default as the accumulator

```
def collect(text, index=0, out=[]):    # created ONCE at definition time
```

The second top-level call sees the first call's results. Use `None` and create it inside, or close over a local list in a private helper — which is the pattern that avoids the problem by construction.

Trap 5 — a public signature full of bookkeeping

```
def is_palindrome(text: str, low: int = 0, high: int = -1) → bool:
```

A caller can pass a wrong `low`, the default for `high` has to be a sentinel because it depends on `text`, and the signature lies about what the function is for. **Use a private helper**, and the outer function takes one argument.

Trap 6 — divide and conquer where a loop would do

```
    return max(go(low, middle), go(middle + 1, high))
```

`2n - 1` calls to find a maximum that a loop finds in `n` steps with no stack. It is not wrong, and it is not an improvement. Divide and conquer pays only when **combining the halves is cheaper than redoing the work** — merging two sorted halves, or discarding one half entirely.

Trap 7 — mutating the caller's data without saying so

```
def reverse_in_place(chars: list[str]) → list[str]:  
    ...                                # returns the SAME list, reversed
```

The caller's list is now reversed. That is exactly what “in place” means and it is correct here — but a function that *inspects* and mutates is a real bug, and it is the same rule as the palindrome check that leaves a linked list half-reversed on [day 082](#). Say which you are doing.

Trap 8 — claiming $O(1)$ space for a recursive in-place algorithm

```
    swap(low + 1, high - 1)                #  $O(n)$  stack frames
```

The array is untouched and the stack is not. **Recursion is never truly $O(1)$ space**. If the requirement is strictly constant, the answer is a loop, and saying that is better than being corrected.

8 In the interview

How it gets asked

- The base: “Reverse a string recursively.” Then: “Now without extra space.” — which means two indices on a character list, and the honest note about the stack.
- The classic: “Check if a string is a palindrome, recursively.”
- The conversion: “Here is a loop. Write it recursively.” Or the reverse, which is what production work actually needs.
- The performance probe: “Your version is slower than the loop even though both are $O(n)$. Why?”
- The judgement probe: “Which would you actually ship?”

What to say out loud, in the first ninety seconds

1. **Name the shape.** “This shrinks by one from the front, so one index.” Or: “this is symmetric, so two indices moving inward.”
2. **Say the index rule before writing it.** “I will pass an index rather than a slice — slicing copies the list on every call and turns this into $O(n^2)$ time and allocation.”
3. **Name the measure and the base case together.** “The measure is `len(text) - start`, and the base case fires when it reaches zero.” Or, for two indices: “the measure is `high - low`, dropping by two, and the base case is `low ≥ high` — greater-or-equal, because even lengths cross rather than meet.”
4. **Use a private helper.** “The public function takes just the string; the recursion is an inner function so the bookkeeping is not in the signature.”
5. **State both costs.** “ $O(n)$ time and $O(n)$ stack — which means it dies at about a thousand characters in Python.”
6. **Then give the judgement.** “For a flat string I would ship the loop. Recursion here is a demonstration; it earns its place on recursive structures and divide-and-conquer.”

The follow-ups

“Your recursive version is slower than the loop even though both are $O(n)$. Why?”

“Two reasons, and one of them is usually a bug. There is genuine per-call overhead — building a frame, binding arguments, returning — which is a constant factor of about one and a half to two in Python. But if the gap is much bigger than that, it is slicing: `f(items[1:])` copies the list on every call, so n calls each copying up to n elements is $O(n^2)$ in time *and* in allocation. Passing an index fixes it and allocates nothing. In strings it is worse, because a slice copies and every `+` builds a new string, so a naive recursive reverse is quadratic twice over.”

“Reverse a string without extra space.” “Strings are immutable in Python, so strictly I need a character list — `list(text)` — and then two indices moving inward, swapping and recursing on `low + 1, high - 1`. The array is $O(1)$ extra space. But I would add the honest caveat: **recursion is never truly constant space**, because the frames are space, so this is $O(n)$ stack. If you mean strictly constant, the answer is the loop with the same two indices, and I would write that.”

“Why is the base case `low ≥ high` and not `low = high`?” “Because on an even-length input the indices cross without ever being equal. `\\"abba\\"` goes zero and three, then one and two, then two and one — never equal, so `=` alone recurses for ever and you get a `RecursionError` on exactly half of all inputs. It is the classic ‘measure decreases and steps over the bound’ failure: `high - low` drops by two, so it can jump from one to minus one without passing through zero.”

“Convert this loop to recursion.” “Three things become three things. What changes between iterations becomes the index parameter. What is carried across iterations — the running total, the best-so-far, the accumulated list — becomes an accumulator parameter. And the loop’s stopping condition becomes the base case. Going the other way is the same recipe backwards: look at what is being combined on the way back up and make it a running variable. And if the recursive version happens to be tail-recursive, that conversion is completely mechanical — though it is worth noting Python will not do it for you.”

“When would you actually use recursion on an array?” “Rarely, and I would say so. For a flat array the loop wins on every axis: same time, $O(1)$ space instead of $O(n)$, no thousand-element limit, and clearer. Recursion earns its place in two cases. When the *structure* is recursive — a nested list, a directory tree, a JSON document — because then the depth is the nesting depth rather than the length, and the iterative version means managing a stack by hand. And divide and conquer, where combining two halves is cheaper than redoing the work: merge sort, quickselect, binary search. Splitting an array in half just to find a maximum is $2n - 1$ calls to do what a loop does in n steps.”

“Does the stack depth depend on which shape you chose?” “Entirely, and it is the difference between working and crashing. Shrink-by-one gives depth n , so it dies at about a thousand. Split-in-half gives depth $\log n$ — seventeen frames at a hundred thousand elements, twenty at a million. Same language, same input, opposite outcomes. That is why merge sort recurses happily on a million elements and a sum-by-index does not, and it comes straight out of how the measure shrinks.”

A model answer

Asked: *reverse a string recursively, then do it without extra space.*

“Two things to decide before writing: the shape of the recursion, and how I pass the data.

On the shape: reversing is symmetric, so this is the two-index form — one index at each end, moving inward, swapping as they go. The measure is `high - low`, it drops by two on every call, and the base case is when they meet or cross.

On passing the data: an index, not a slice. This matters more than it looks. The obvious recursive reverse is `reverse(text[1:]) + text[0]`, and that is quadratic twice over — the slice copies the string on every call, and every `+` builds a new string. At ten thousand characters that is about fifty million character copies to do something the slice notation does in one pass. Passing indices copies nothing.

The base case is `low ≥ high`, greater *or equal*, and that one character is where this goes wrong. On an even-length string the two indices cross without ever being equal — `\\"abba\\"` is zero-and-three, then one-and-two, then two-and-one — so `=` alone never terminates, and you get a `RecursionError` on exactly half of all inputs.

Now, ‘without extra space’. Python strings are immutable, so I cannot swap characters in a string; I need a character list. Then the swap is a tuple assignment and the recursion carries the two indices, and the array itself uses no extra space.

But I want to be honest about the caveat, because it is the thing an interviewer is often checking: **recursion is never truly $O(1)$ space**. The frames are space. This is $O(n)$ stack, and in Python it dies at about a thousand characters. If ‘without extra space’ means strictly constant, the answer is a loop with the same two indices, and I would write that one.

I would also structure it with a private helper: the public function takes just the string, and the inner function carries `low` and `high`. That keeps the bookkeeping out of the signature so a caller cannot pass a wrong index, and the inner function closes over the data rather than passing it on every call.

And the judgement, since it usually comes next: for a flat string I would ship the loop. Same complexity, constant space, no depth limit, clearer. Recursion earns its place when the structure is recursive — a nested list where the depth is the nesting rather than the length — or in divide and conquer, where combining two halves is genuinely cheaper than redoing the work.”

9 Recall card

- **Pass an INDEX, never a slice.** `f(items[1:])` copies on every call — n calls $\times O(n)$ copy = $O(n^2)$ **time and allocation**, silently, with no error. `f(items, start + 1)` allocates nothing. In strings it is worse: the slice copies **and** every `+` builds a new string, so a naive recursive reverse is quadratic twice over ($\sim 5 \times 10^7$ character copies at $n = 10,000$).
- **Symmetric problems use TWO indices moving inward, and the base case is** `low  $\geq$  high`. Not `==`: on even lengths they **cross without meeting** — `"abba"` goes $(0,3) \rightarrow (1,2) \rightarrow (2,1)$ — so `==` alone recurses for ever on half of all inputs. The measure is `high - low`, dropping by two.
- **Converting a loop is a three-part recipe:** what *changes* between iterations $\rightarrow$ the index $\cdot$ what is *carried across* $\rightarrow$ an accumulator parameter $\cdot$ the *stopping condition* $\rightarrow$ the base case. Put the bookkeeping in a **private helper** so the public signature takes only what a caller would pass.
- **The shape decides whether it survives.** Shrink-by-one $\rightarrow$ depth n , dies at ~ 1000 . Split-in-half $\rightarrow$ depth $\log n$, 17 frames at 100,000. *Same input, opposite outcomes*. And divide-and-conquer on a flat array is **$2n - 1$ calls to do what a loop does in n** — it only pays when combining halves is cheaper than redoing the work.
- **On a flat array the loop wins on every axis** — same time, **$O(1)$ space vs $O(n)$ stack**, no depth limit, clearer — and **recursion is never truly $O(1)$ space** even when the array is untouched. Recursion earns its place on **recursive structure** (nesting depth, not length) and on **divide-and-conquer**. Say which you would ship.

Design an in-memory cache with eviction

1 What this is, and why they ask it

A cache is a small fast store in front of a big slow one. It holds a bounded number of entries, and when it is full and something new arrives, it has to throw something out.

You built one on [day 076](#) — a hash map plus a doubly linked list, $O(1)$ get and put, least-recently-used eviction. Today the question is what happens when the requirement grows, and it grows in three directions that people routinely merge into one.

Eviction is about capacity. The cache is full; something must go. Which one is a *policy* — least recently used, least frequently used, first in, or at random — and the four have genuinely different behaviour on real workloads.

Expiry is about correctness. This entry is now too old to trust, whether or not the cache is full. An entry can be expired and still resident, and evicted while still perfectly fresh. **They are two different mechanisms and merging them is the standard design mistake.**

And the third thing nobody mentions until it happens: when a popular key expires, every request that wanted it misses at the same instant and they all go to the database together. That is a **cache stampede**, and a cache that does not handle it can be worse than no cache at all — because at least without a cache the load is steady.

They ask it because everyone has used a cache and few have designed one, because the policy interface is a case where Strategy is unarguably justified, and because the stampede is a real production failure that has taken down real systems.

2 The story

Mohan's medical shop is deeper than it is wide, and everything about how it works comes from that.

Behind the counter there is one shelf he can reach without moving his feet. It holds about two hundred boxes. Behind him, in the racks going back into the dark, there are perhaps four thousand more, and fetching one means putting the ladder up, and that is a minute and a half with somebody waiting.

So the two hundred boxes on the front shelf are the whole business. Get them right and he serves someone in fifteen seconds; get them wrong and he is up the ladder all day.

He has never written down how he chooses. Watching him, it is two rules.

The first is that whatever he has just been asked for goes back on the front shelf, near the middle, and something that has not been asked for in a long time gets pushed along and eventually goes to the back. He does this without thinking, while talking to the next customer.

The second rule is different in kind and he is strict about it. Some boxes have dates on them. When the date has gone, the box goes — and it does not matter at all how popular it is. The most-asked-for thing in the shop, if it is out of date, comes off the shelf. Popular and usable are two different questions, and the shelf being full has nothing to do with either.

He checks dates in a way that surprised his son. He does not go through the whole shelf. When he picks a box up for a customer, he looks at the date then — most of the checking happens for free, while doing something else. And once a week, on the quiet Thursday afternoon, he pulls out a handful at random and checks those, because the ones nobody asks for are exactly the ones that quietly go out of date at the back of the shelf and never get looked at.

The bad day is the first cold morning of the season.

On that morning, thirty or forty people come in wanting the same cough syrup, and it is not on the front shelf because nobody has asked for it since February. So the first customer sends him up the ladder. And while he is up the ladder, six more people come in and ask for the same thing, and his son, who does not know the box is already coming down, starts to go for the ladder too.

What they do now is that whoever asks first is told a minute, and everybody who asks after that is told to wait for the same box. One trip up the ladder, not seven.

3 The idea in plain English

Mohan's front shelf is the cache. Pushing the untouched boxes to the back is **eviction**. The dates are **expiry** — a completely separate mechanism. And the first cold morning is a **cache stampede**.

The three mechanisms, kept apart

	QUESTION IT ANSWERS	TRIGGERED BY	APPLIES TO
Eviction	The cache is full — what goes?	a <code>put</code> when at capacity	a <i>fresh</i> entry, chosen by policy
Expiry	Is this entry still trustworthy?	a TTL passing	<i>any</i> entry, regardless of capacity
Invalidation	The source changed — drop it now	an external event	one specific key

An entry can be **expired but resident** (nobody has looked at it since it expired) and **evicted while fresh** (something newer arrived and it was the least recently used). Mohan’s out-of-date bestseller, and his perfectly good box pushed to the back.

Merging them produces the classic bug: an “LRU cache with TTL” where expired entries are only removed when they happen to be the least recently used — so the least popular expired entries sit there for ever, and the cache is full of things it will never serve.

The entry, and what it has to carry

```
@dataclass
class Entry:
    value: object
    expires_at: float | None      # None = never expires
    size_bytes: int              # for byte-based capacity
    # whatever the policy needs lives in the policy, NOT here
```

The last line is the design decision. `Entry` does not hold `last_used` or `hit_count`, because those belong to whichever policy is installed — an LFU cache needs a counter that an LRU cache does not, and putting both on the entry means the entry knows about policies.

The eviction policy, which is a real interface

```
class EvictionPolicy(Protocol):
    def record_access(self, key: str) → None: ...    # a get, or a put of an existing key
    def record_insert(self, key: str) → None: ...
    def record_remove(self, key: str) → None: ...
    def evict_candidate(self) → str | None: ...      # who goes next
```

Four implementations, all real:

- **LRU** — evict the least recently used. Good when access has *recency* locality, which most workloads do. The hash map plus doubly linked list from [day 076](#).

- **LFU** — evict the least frequently used. Better when there is a stable set of hot keys and a long tail of one-off requests, because LRU lets a burst of scans push the hot set out. Needs a third structure and a `min_frequency` counter to stay $O(1)$, and it **never forgets**: an entry that was hot last month holds a huge count and will not leave. Real implementations decay the counts or use a window.
- **FIFO** — evict the oldest inserted. Trivial, ignores usage entirely, and surprisingly acceptable when entries all have similar value.
- **Random** — evict a random entry. Almost no bookkeeping, no locks on the hot path, and its hit rate is usually **within a few percent of LRU**. Redis's default `allkeys-lru` is in fact an *approximation* built on random sampling for exactly this reason.

Four implementations that somebody genuinely wants is the gate an interface has to pass — the same test as [day 076](#) — and this one passes it more clearly than almost any other example in the course.

Expiry: lazy plus sampled, which is what Redis does

Two mechanisms, and you need both.

Lazy expiry, on read. When `get` finds an entry, check the expiry; if it has passed, delete it and report a miss. Costs nothing, needs nothing running, and is *correct* — no expired value is ever served. Mohan checking the date as he picks the box up.

Active expiry, by sampling. Lazy alone leaks: an entry nobody ever asks for again is never read, so its expiry is never noticed, and it occupies capacity for ever. So a background loop samples a handful of keys — Redis takes twenty at a time, ten times a second — deletes the expired ones, and if more than a quarter were expired, samples again immediately. Mohan's Thursday afternoon handful.

Lazy alone is correct but leaks. Sampling alone is wasteful. Both together is the answer, and being able to say why each is insufficient on its own is the point.

Note that this is the opposite of the food order on [day 087](#), where a timeout had to *do* something — refund and notify — so a sweeper was mandatory. Here nothing needs to happen when an entry expires; it only must not be served. **When expiry has no side effect, lazy is enough for correctness**, and the sweep is only about memory.

The stampede, which is the failure that hurts

A hot key expires. Every request for it now misses. Every miss goes to the database. If the key gets five thousand requests a second and the database query takes two hundred milliseconds, then in the two hundred milliseconds it takes the first request to load it, **a thousand more requests have arrived and all of them have also missed**.

$5,000 \text{ requests/second} \times 0.2 \text{ s load time} = 1,000 \text{ concurrent identical queries}$

A thousand identical queries against one row. The database slows down, so the load takes longer, so more requests pile up. This is how a cache expiry takes down a database, and it is not rare — it is the first cold morning, every time.

Two fixes, and the first is the one to name:

Single-flight. The first request to miss takes a per-key lock and loads. Everyone else who misses that key **waits for the same load** rather than starting their own. One trip up the ladder.

```
with self._loading_lock(key):           # per key, not global
    cached = self._store.get(key)
    if cached is not None:               # somebody loaded it while we waited
        return cached.value
    value = loader()
    self.put(key, value)
    return value
```

The double check inside the lock is essential: by the time you get the lock, the first request may have finished, and re-loading would defeat the whole thing.

Probabilistic early expiry. Instead of everything expiring at exactly `t`, each read has a small chance of treating the entry as expired slightly early, rising as the true expiry approaches. Hot keys are read often, so one of those reads refreshes the value *before* it expires — and the misses are spread out rather than simultaneous. Cheaper than locking and it never fully blocks.

A third, blunter one worth naming: **jitter the TTLs**. If a thousand keys are all loaded at start-up with a ten-minute TTL, they all expire in the same second. `ttl + random(0, ttl/10)` costs one line and removes an entire class of synchronised stampede.

Sizing: entries or bytes

“Capacity 10,000 entries” is easy and often wrong, because entries differ:

```
10,000 entries × 200 B    = 2 MB
10,000 entries × 2 MB     = 20 GB
```

Same configuration, four orders of magnitude apart. If values vary in size, **bound by bytes**, which means measuring each value on insert — `sys.getsizeof` is a rough answer and a serialised length is a better one — and evicting until you are under the limit rather than evicting exactly one thing.

The cost is that measurement is not free and is not exact for nested Python objects. Say which you chose and why: entries when values are uniform, bytes when they are not.

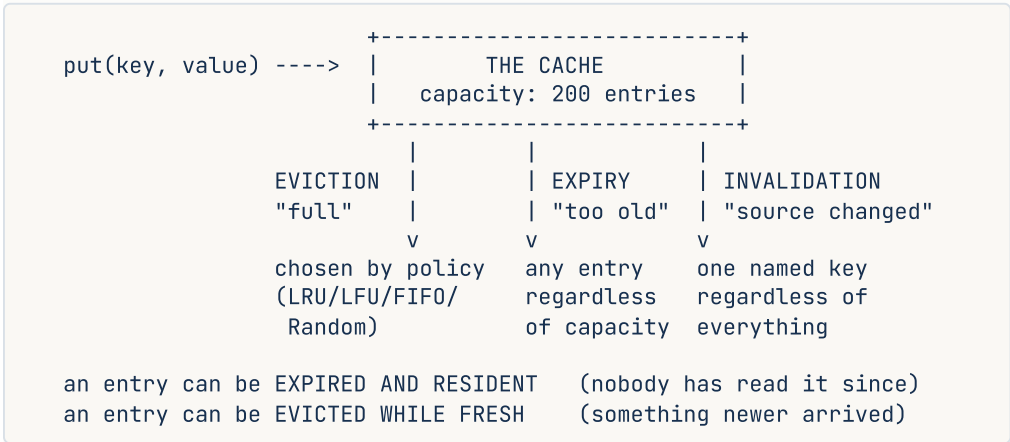
The one number that matters

```
@property
def hit_rate(self) -> float:
    total = self._hits + self._misses
    return self._hits / total if total else 0.0
```

A cache with no hit-rate metric cannot be tuned, and every question about capacity or policy is unanswerable without it. Expose hits, misses, evictions and expirations separately, because they mean different things: rising evictions means too small, rising expirations means the TTL is too short, and a falling hit rate with neither means the workload changed.

4 The picture

The three mechanisms, and how an entry leaves:



The stampede, drawn on a timeline:

hot key, 5,000 requests/second, database load takes 200 ms

```
t=0.000  TTL passes. Entry expires.  
t=0.001  request 1 misses → starts loading  
t=0.002  request 2 misses → starts loading      ← nothing stops it  
t=0.003  request 3 misses → starts loading  
...  
t=0.200  request 1,000 misses → starts loading  
t=0.200  request 1 finishes, fills the cache  
-----  
1,000 identical queries for one row, all in flight at once
```

WITH SINGLE-FLIGHT

```
t=0.001  request 1 misses → takes the key's lock → loads  
t=0.002  request 2 misses → waits on the lock  
...      requests 3..1000 wait on the same lock  
t=0.200  request 1 fills the cache and releases  
t=0.200  requests 2..1000 wake, re-check, FIND IT, return  
-----  
1 query
```

And the class structure, where the policy interface is the design:

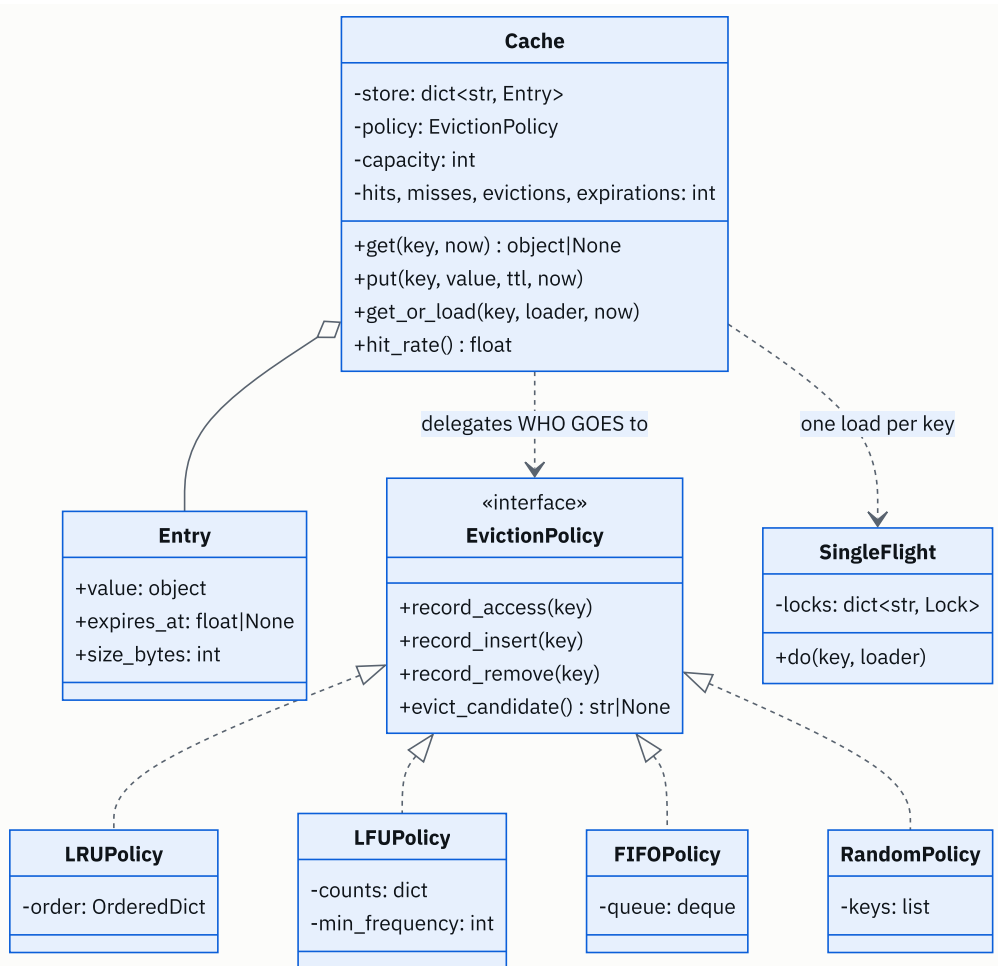


FIGURE 90.1

What to notice: **Entry** holds no policy bookkeeping. No `last_used`, no `hit_count` — those live in whichever policy is installed. An entry that knew about recency *and* frequency would be an entry that knows about policies, and swapping the policy would then mean changing the entry.

5 How it actually works

Move 1 · Clarify (minutes 0–5)

“In-process, or shared across servers?” — In-process. A shared cache is Redis and a different conversation. **“Do entries expire, or only get evicted when full?”** — Both, and I would separate them explicitly. **“Are the values similar in size?”** — Because that decides whether capacity is counted in entries or bytes. **“Is it read by several threads?”** — Because that decides the locking, and a single lock in front of a cache can be slower than no cache.

“I will assume values are immutable once cached, that a stale value is unacceptable but a missing one is fine, and that the loader function is supplied by the caller. I am not designing distribution or persistence.”

Move 2 · The nouns (minutes 5–10)

- **Entry** — the value, its expiry, its size. No policy state.
- **EvictionPolicy** (*interface*) — four implementations.
- **Cache** — the store, the capacity, the stats. Holds no policy logic.
- **SingleFlight** — one in-flight load per key.
- **Stats** — hits, misses, evictions, expirations, exposed separately.

Five, one interface with four implementations. Notice that there is no **TTLManager** class: expiry is a field on the entry plus a check on read, and inventing a manager for it is how the two mechanisms get merged.

Move 3 · `get`, where expiry is lazy

```
def get(self, key: str, now: float) → object | None:
    entry = self._store.get(key)
    if entry is None:
        self._misses += 1
        return None

    if entry.expires_at is not None and entry.expires_at ≤ now:
        self._remove(key)                # expired: delete and report a MISS
        self._expirations += 1
        self._misses += 1
        return None

    self._policy.record_access(key)       # a read is a use – LRU depends on this
    self._hits += 1
    return entry.value
```

Two things to say while writing it. **An expired hit is a miss**, not a hit — counting it as a hit makes the hit-rate metric lie in exactly the situation you need it to be honest. And `record_access` is the line from [day 076](#): **a read is a write**, and leaving it out gives a

cache that evicts by insertion order while returning perfectly correct values.

Move 4 • `put`, where eviction happens

```
def put(self, key: str, value: object, now: float, ttl: float | None = None) → None:
    if key in self._store:
        self._store[key] = Entry(value, now + ttl if ttl else None, sizeof(value))
        self._policy.record_access(key)      # an overwrite is a use
        return

    while len(self._store) ≥ self._capacity:    # WHILE, not IF
        victim = self._policy.evict_candidate()
        if victim is None:
            break
        self._remove(victim)
        self._evictions += 1

    self._store[key] = Entry(value, now + ttl if ttl else None, sizeof(value))
    self._policy.record_insert(key)
```

`while`, `not if`. With byte-based capacity, one large value may require evicting several entries, and even with entry counts a `while` is correct and an `if` is only accidentally correct. The `victim is None` guard stops an infinite loop when the policy has nothing left to give.

Move 5 • The policies

```
class LRUPolicy:
    """Recency. An OrderedDict is a dict plus a doubly linked list, so
    move_to_end and popitem(last=False) are the two O(1) operations."""

    def __init__(self) → None:
        self._order: OrderedDict[str, None] = OrderedDict()

    def record_access(self, key: str) → None:
        self._order.move_to_end(key)      # most recent at the end

    def record_insert(self, key: str) → None:
        self._order[key] = None

    def record_remove(self, key: str) → None:
        self._order.pop(key, None)

    def evict_candidate(self) → str | None:
        return next(iter(self._order), None)  # least recent is first
```

```

class LFUPolicy:
    """Frequency, kept O(1) with a min_frequency counter.

    The honest weakness: plain LFU NEVER FORGETS. An entry that was hot last
    month holds a huge count and refuses to leave. Real implementations decay
    counts periodically or count within a window.
    """

    def evict_candidate(self) → str | None:
        while self._min_frequency in self._buckets and not self._buckets[self._min_frequency]:
            self._min_frequency += 1
        bucket = self._buckets.get(self._min_frequency)
        return next(iter(bucket), None) if bucket else None

```

```

class RandomPolicy:
    """Almost no bookkeeping, and its hit rate is usually within a few percent
    of LRU. Redis's allkeys-lru is itself an APPROXIMATION built on random
    sampling, for exactly this reason."""

    def evict_candidate(self) → str | None:
        return random.choice(self._keys) if self._keys else None

```

Writing `RandomPolicy` and saying that it is nearly as good as LRU is worth doing. It is counter-intuitive, it is true, and it demonstrates that you know why LRU's bookkeeping costs something.

Move 6 · Single-flight, which is the failure fix

```

def get_or_load(self, key: str, loader, now: float, ttl: float | None = None):
    cached = self.get(key, now)
    if cached is not None:
        return cached

    with self._lock_for(key):
        # PER KEY, never a global lock
        cached = self.get(key, time.time()) # somebody may have loaded it
        if cached is not None:
            return cached
        value = loader()
        self.put(key, value, time.time(), ttl)
        return value

```

Three details, all necessary.

The lock is per key. A global load lock serialises every miss in the system, which is slower than having no cache at all.

The re-check inside the lock is what makes it work. By the time you acquire it, the first request has usually finished, and skipping the re-check means every waiter loads anyway.

And now is re-read after waiting, because you may have been blocked for two hundred milliseconds.

Move 7 · Active expiry, sampled

```
def sweep(self, now: float, sample: int = 20) → int:
    """Redis's approach: sample a handful, delete the expired, and if more
    than a quarter were expired, go again. Lazy expiry alone is correct but
    leaks — an entry nobody reads is never noticed."""
    removed = 0
    for _ in range(4):
        keys = random.sample(list(self._store), min(sample, len(self._store)))
        expired = [k for k in keys
                    if (e := self._store[k]).expires_at and e.expires_at ≤ now]
        for key in expired:
            self._remove(key)
            self._expirations += 1
        removed += len(expired)
        if len(expired) ≤ len(keys) // 4:      # not many left; stop
            break
    return removed
```

The “sample again if more than a quarter were expired” rule is the clever part and it is Redis’s: it makes the sweep adaptive, doing almost nothing when few keys are expiring and working hard when many are.

Real systems

- **Redis** does exactly this pairing: lazy expiry on access, plus an active cycle sampling twenty keys and repeating while more than 25 percent are expired. And `maxmemory-policy allkeys-lru` is **approximate** LRU by sampling, not exact — because exact LRU bookkeeping at ten million keys costs more than the accuracy is worth.
- **Caffeine** (Java) uses **W-TinyLFU**, a windowed frequency sketch, and reports hit rates consistently better than LRU on real traces — evidence that “LRU is the best policy” is folklore rather than fact.
- **Memcached** uses a segmented LRU with a lazy expiry and no active sweep at all, which is why an entry can occupy memory long after it expired.
- `functools.lru_cache` is the standard-library version with no TTL and no thread-shared eviction policy — worth naming as the thing you would use before writing any of this.
- **Single-flight** is a named pattern with implementations in most ecosystems — Go’s `golang.org/x/sync/singleflight`, Caffeine’s loading caches — precisely because everybody hits the stampede eventually.

6 The numbers

Why a cache is worth having, and when it stops being

```
database read    100 µs
cache read       1 µs

hit rate 99%:    0.99 × 1 + 0.01 × 100 = 1.99 µs → 50x
hit rate 95%:    0.95 × 1 + 0.05 × 100 = 5.95 µs → 17x
hit rate 90%:    0.90 × 1 + 0.10 × 100 = 10.90 µs → 9x
hit rate 50%:    0.50 × 1 + 0.50 × 100 = 50.50 µs → 2x
```

The value falls off a cliff below about 95 percent. That is why the hit rate is the number to measure, and why “add a cache” without measuring it is not a plan.

Sizing

```
100,000 entries × 2 KB value                = 200 MB
+ Entry object overhead ~120 B                = 12 MB
+ dict entry ~100 B                          = 10 MB
+ LRU OrderedDict node ~100 B                = 10 MB
-----
total                                         ≈ 232 MB
```

About 16 percent overhead on 2 KB values — and on 200-byte values the same book-keeping is 160 percent overhead, more than the data. That is the argument for byte-based capacity and for not caching tiny values individually.

```
the same 100,000 entries at 2 MB each = 200 GB
```

Same “capacity: 100,000”, four orders of magnitude apart. **Count bytes when values vary.**

The stampede, quantified

```
a hot key: 5,000 requests/second
database load for that key: 200 ms

without single-flight:
  concurrent identical queries = 5,000 × 0.2 = 1,000
  each holding a connection from a pool of, say, 100
  → the pool is exhausted 10x over; unrelated queries also fail

with single-flight:
  concurrent queries = 1
  the other 999 wait ~200 ms and are then served from the cache
```

A thousand identical queries against one row, versus one. And note the second-order failure: it is not the hot key that breaks, it is everything *else*, because the connection pool is shared.

Synchronised expiry

```
1,000 keys loaded at start-up, all with a 600 s TTL
→ all 1,000 expire within the same second, 10 minutes later
→ 1,000 stampedes at once

with jitter: ttl + random(0, 60)
→ the same 1,000 expiries spread over 60 seconds = ~17 per second
```

One line of jitter converts a cliff into a slope. This is the cheapest fix in the whole lesson.

Expiry sweep cost

```
100,000 entries, 20 sampled 10 times a second = 200 checks/second
→ 0.2% of the keypace per second
an entry that nobody reads is found, on average, within ~500 s
```

That lag is the honest cost of sampling: expired entries occupy memory for a few minutes on average. Scanning everything would be exact and would cost 100,000 checks per cycle — which is why nobody does it.

Policy comparison, honestly

```
typical web workload, cache holding 10% of the working set:

FIFO      ~ 8-10 percentage points worse than LRU
Random    ~ 1-3 points worse than LRU
LRU        baseline
LFU        better on stable hot sets, worse on shifting ones
W-TinyLFU ~ 3-8 points better than LRU on published traces
```

Random is within a few points of LRU for almost no bookkeeping, which is why Redis approximates rather than implements exact LRU. Quote this rather than asserting that LRU is best.

7 The trade-offs

What this design gives up

A pluggable policy costs an indirection on every access. `record_access` is called on every hit, and with a policy behind an interface that is a method call and a data-structure update on the hottest path in the system. For a cache doing a million gets a second, that is measurable. The mitigation is that the policies are small and the interface is narrow — and the honest note is that if you only ever want LRU, `OrderedDict` inline is faster and shorter.

LFU never forgets, and plain LFU is a trap. An entry that was hit ten thousand times last month outranks one hit fifty times this morning, for ever. Any real LFU needs decay or a window, and a candidate who proposes LFU without mentioning that has not used one.

Byte-based sizing is approximate. `sys.getsizeof` does not follow references, so a list of a thousand strings reports as a few kilobytes when the strings are megabytes. Serialised length is more honest and costs a serialisation. Either way the limit is a guide, not a guarantee, and you should size the process with headroom.

Single-flight adds a lock on the miss path and can cascade. If the loader is slow or hangs, every waiter for that key is blocked, and without a timeout on the wait, one slow query becomes a thousand stuck threads. A bounded wait, after which a waiter either loads anyway or returns a stale value, is the safer shape.

A stale-but-available answer is often better than a correct-but-slow one, and this design does not offer it. `stale-while-revalidate` — serve the expired value immediately and refresh in the background — removes the stampede entirely and trades a few seconds of staleness. For most read paths that is the right trade, and it should be a per-key policy rather than a global one.

Thread safety is not free. One lock around the whole cache serialises every access and can make the cache slower than the database it fronts. Sharding into sixteen caches by `hash(key) % 16` cuts contention by roughly the shard count and is what real implementations do — but then the capacity is per shard, and an uneven key distribution wastes some of it.

In-process caches do not share. Ten servers means ten copies, ten times the memory, and ten independent stampedes on the same key. A shared Redis fixes that and adds a network hop of about 0.2 ms to every hit — which, against a 1 µs local hit, is two hundred times slower. **Local and shared caches are different tools,** and the usual production answer is both, with the local one in front.

“I would change this design if...”

- **...values vary wildly in size.** Byte-based capacity, and refuse to cache anything above a threshold at all.
- **...staleness is acceptable.** `stale-while-revalidate`, which deletes the stampede problem.
- **...the workload is a stable hot set with a scanning tail.** LRU with decay, or W-TinyLFU, because LRU lets the scan evict the hot set.
- **...several processes need the same cache.** Redis, with a local cache in front of it, and accept the invalidation problem that two tiers create.
- **...I only ever need LRU with no TTL.** `functools.lru_cache`, and delete all of this.

The honest concession

The eviction policy is the part that looks like the design and it is the part that matters least. The difference between LRU and random is a few percentage points of hit rate. The difference between having single-flight and not having it is a thousand concurrent queries against one row, and the difference between jittered and unjittered TTLs is whether a thousand keys expire in the same second. **The interesting engineering here is in the failure modes, not in the policy** — and a candidate who spends thirty minutes on LRU internals and never mentions the stampede has answered the wrong question.

8 In the interview

How it gets asked

- The standard: “Design an in-memory cache.” Then: “Now make the eviction policy swappable.”
- The separation probe: “What is the difference between eviction and expiry?”
- The failure probe, which is the good one: “A very popular key expires. What happens?”
- The sizing probe: “How do you decide the capacity?”
- The judgement probe: “Which eviction policy would you use, and does it matter?”

The timed script

Minutes 0–5 · Clarify. In-process or shared? Do entries expire as well as get evicted? Are values similar in size? Multi-threaded? Each answer changes a decision.

Minutes 5–12 · Separate the three mechanisms explicitly. Eviction is capacity, expiry is correctness, invalidation is an external event. Say that an entry can be expired-and-resident or evicted-while-fresh — that sentence is the framing the whole design hangs on.

Minutes 12–22 · The classes. `Entry` with no policy state, the `EvictionPolicy` interface with four real implementations, and the `Cache` holding stats. Point out that four wanted implementations is what justifies the interface.

Minutes 22–30 · Expiry done properly — lazy on read *and* sampled sweep, with why each alone is insufficient.

Minutes 30–38 · The stampede. The arithmetic — $5,000 \text{ rps} \times 200 \text{ ms} = 1,000$ concurrent queries — then single-flight with the re-check inside the lock, then TTL jitter as the one-line fix.

Minutes 38–40 · The hit rate, and that nothing here can be tuned without it.

The follow-ups

“What is the difference between eviction and expiry?” “They answer different questions and merging them is the standard mistake. Eviction is about **capacity**: the cache is full, so something must go, and *which* one is a policy — least recently used, least frequently used, or random. Expiry is about **correctness**: this entry is too old to trust, whether or not the cache is full. So an entry can be expired and still resident, because nobody has read it since, and an entry can be evicted while perfectly fresh, because something newer arrived. The classic bug from merging them is an LRU-with-TTL where expired entries are only removed when they become least-recently-used — so the least popular expired entries stay for ever and the cache fills with things it will never serve.”

“How do entries expire?” “Two mechanisms, and you need both. Lazily on read — check the expiry when you find the entry, and if it has passed, delete it and report a miss. That is correct on its own: no expired value is ever served, and it costs nothing. But it *leaks*, because an entry nobody ever reads again is never checked and occupies capacity for ever. So there is also an active sweep that samples a handful of keys — Redis takes twenty, ten times a second — deletes the expired ones, and repeats immediately if more than a quarter were expired, which makes it adaptive. Lazy alone is correct but leaks; sampling alone is wasteful; both together is the answer.”

“A very popular key expires. What happens?” “A stampede, and the arithmetic is the point. If that key gets five thousand requests a second and the database load takes two hundred milliseconds, then a thousand more requests arrive and miss before the first one finishes — so a thousand identical queries against one row, all in flight. And the failure is not the hot key, it is everything *else*, because they all hold connections from a shared pool. The fix is single-flight: the first request to miss takes a **per-key** lock and loads, and everyone else waits for the same load, then re-checks inside the lock and finds it. That re-check is essential — without it, every waiter loads anyway. I would also jitter the TTLs, because a thousand keys loaded at start-up with the same TTL all expire in the same second, and one line of random jitter turns a cliff into a slope.”

“Which eviction policy would you use?” “LRU by default, because most workloads have recency locality — but I would be careful not to overclaim it. Random eviction is typically within one to three percentage points of LRU for almost no bookkeeping, which is why Redis’s `allkeys-lru` is actually an *approximation* built on random sampling: exact LRU at ten million keys costs more than the accuracy is worth. LFU is better when there is a stable hot set and a long scanning tail, because a scan will push LRU’s hot set out — but plain LFU **never forgets**, so an entry hot last month refuses to leave, and any real LFU needs decay or a window. The honest answer is that the policy is worth a few points of hit rate and the failure handling is worth an outage.”

“How do you decide the capacity?” “By measuring the hit rate, which is the only number that makes any of this tunable. With a hundred microsecond database and a one microsecond cache, ninety-nine percent hits is a fiftyfold improvement, ninety-five is seventeenfold, and fifty percent is barely twofold — so the value falls off a cliff below about ninety-five. I would plot hit rate against capacity and stop where the curve flattens. And I would count **bytes** rather than entries if values vary: a hundred thousand entries is two hundred megabytes at two kilobytes each and two hundred gigabytes at two megabytes each, from the same configuration line.”

“Is it thread-safe?” “Not as written, and making it so is a real trade. One lock around the whole cache serialises every access, and a contended lock in front of a one-microsecond operation can make the cache slower than the database it fronts. The usual answer is sharding — sixteen independent caches keyed by hash of the key — which cuts contention by roughly the shard count, at the cost of the capacity being per shard and an uneven key distribution wasting some of it. The single-flight locks are separate and are per key by design, so they do not serialise anything except duplicate loads of the same key.”

“Would you cache stale data?” “Often, yes, and it is the most underrated option here. `stale-while-revalidate` serves the expired value immediately and refreshes in the background, which removes the stampede entirely — nobody ever waits for a load. The trade is a few seconds of staleness, and it should be a per-key decision rather than a global one: a product price can be five seconds old, an account balance cannot. It is worth raising unprompted, because ‘a stale answer now’ beats ‘a correct answer in two hundred milliseconds’ far more often than people expect.”

A model answer

Asked: *design an in-memory cache, and make the eviction policy swappable.*

“Let me start by separating three things that get merged, because that framing carries the whole design.

Eviction is about capacity: the cache is full, something must go, and which one is a policy. **Expiry** is about correctness: this entry is too old to trust, whether or not the cache is full. And **invalidation** is an external event that drops one named key. They are three different mechanisms — an entry can be expired and still resident because nobody has read it, and evicted while perfectly fresh because something newer arrived.

So: a `Cache` holding a dict of `Entry` objects, where an entry has the value, an optional expiry timestamp and its size — and deliberately **no** policy bookkeeping. No last-used, no hit count. Those live in whichever policy is installed, because an LFU cache needs a counter an LRU cache does not, and an entry that knew about both would know about policies.

The `EvictionPolicy` interface has four methods — record access, record insert, record remove, and choose a victim — and four implementations that somebody genuinely wants: LRU, LFU, FIFO and random. Four real implementations is exactly the gate an interface has to pass, and this one passes it more clearly than most.

Two things about the policies that I would say rather than let you ask. Random eviction is typically within a few percentage points of LRU for almost no bookkeeping, which is why Redis’s LRU is actually an approximation built on random sampling. And plain LFU never forgets: something hot last month outranks something hot this morning for ever, so a real LFU needs decay or a window.

Expiry needs two mechanisms. Lazily on read — check the timestamp when you find the entry, and an expired hit counts as a **miss**, not a hit, or the metric lies exactly when you need it. That is correct on its own but it leaks, because an entry nobody reads again is never checked. So there is also an active sweep that samples twenty keys ten times a second and repeats if more than a quarter were expired. Lazy alone leaks; sampling alone is wasteful; both is the answer, and it is what Redis does.

Now the part I think actually matters, which is the failure mode. When a hot key expires, every request for it misses at the same instant. At five thousand requests a second and a two-hundred millisecond load, a thousand more requests arrive and miss before the first one finishes — so a thousand identical queries for one row, all holding connections from a shared pool, which takes down the queries that had nothing to do with it. The fix is single-flight: the first miss takes a per-key lock and loads; everyone else waits, then **re-checks inside the lock** and finds it. That re-

check is the part people leave out. And I would jitter the TTLs, because a thousand keys loaded at start-up with the same TTL expire in the same second — one line of randomness turns a cliff into a slope.

Finally, the cache must expose hits, misses, evictions and expirations separately, because they mean different things: rising evictions means it is too small, rising expirations means the TTL is too short. And nothing here is tunable without the hit rate — at a hundred microsecond database and a one microsecond cache, ninety-nine percent hits is fiftyfold and ninety percent is ninefold, so the value falls off a cliff below about ninety-five.

If I had to say what the interesting engineering is: it is not the eviction policy. That is worth a few points of hit rate. The stampede is worth an outage.”

9 Recall card

- **Three mechanisms, kept apart — merging them is the standard mistake.** Eviction = capacity (policy chooses a victim) · **Expiry** = correctness (TTL, regardless of capacity) · **Invalidation** = an external event on one key. *An entry can be expired and resident, and evicted while fresh.*
- **Entry carries value, expiry and size — and NO policy bookkeeping.** Recency and frequency live in the installed policy, or swapping the policy means changing the entry. The `EvictionPolicy` interface has **four genuinely wanted implementations** — LRU, LFU, FIFO, Random — which is exactly the gate an interface must pass.
- **Expiry needs BOTH lazy and sampled.** Lazy on read is correct and costs nothing but **leaks** — an entry nobody reads is never noticed. So sample ~ 20 keys $\sim 10 \times$ /second and repeat while $> 25\%$ are expired (Redis’s adaptive rule). An **expired hit is a MISS**, or the hit-rate metric lies.
- **The stampede is the failure that matters, and the policy is not.** A hot key at **5,000 req/s with a 200 ms load = 1,000 concurrent identical queries**, exhausting a shared pool and breaking unrelated work. Fix with **single-flight**: a **per-key** lock, and **re-check inside the lock**. Then **jitter the TTLs** — 1,000 keys with identical TTLs all expire in the same second, and one line of randomness spreads them.
- **The hit rate is the only number that makes any of this tunable:** at $100 \mu\text{s}$ vs $1 \mu\text{s}$, $99\% \rightarrow 50 \times$, $95\% \rightarrow 17 \times$, $90\% \rightarrow 9 \times$, $50\% \rightarrow 2 \times$ — the value falls off a cliff below $\sim 95\%$. **Count bytes, not entries, when values vary** (100,000 entries is 200 MB at 2 KB each and 200 GB at 2 MB each). And **Random is within a few points of LRU** — which is why Redis only *approximates* LRU by sampling.

DSA topic: Recursion on arrays and strings **System design topic:** Design an in-memory cache with eviction

Code these, in this order

One rule for the whole set: **an index, never a slice**. If your recursive call contains `[1:]` or `[:-1]`, stop and rewrite it with a position. The function will still be correct and it will be a different complexity class.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Reverse String	LeetCode 344 (Easy)	Two indices in place, and the <code>>=</code> base case that even lengths need.
2	Valid Palindrome	LeetCode 125 (Easy)	The same shape with characters to skip — where the indices move by different amounts.
3	Fibonacci Number	LeetCode 509 (Easy)	Loop to recursion to memoised recursion, and the call counts for all three.
4	Flatten Nested List Iterator	LeetCode 341 (Medium)	The case where recursion genuinely wins: depth is the nesting, not the length.

On problem 1, write the base case last

Write the swap and the recursive call first, then ask “when do I stop?” and *derive* `low >= high` rather than remembering it. Then test `"abba"` and `"abcba"` — one of them is the reason it is `>=`.

On problem 2, notice the indices move unevenly

Skipping non-alphanumeric characters means `low` may advance several times before a comparison. Say what that does to the measure, and confirm it still strictly decreases.

On problem 3, count the calls three ways

Instrument the naive recursion, the memoised one, and the loop. Run all three at $n = 30$ and write down the three call counts and the three times.

On problem 4, say why this one is different

Compare the depth of the recursion against the length of the input, and say why the iterative version needs an explicit stack while the array versions needed nothing.

The slice-versus-index drill

1. Write `total` with a slice and with an index.
2. Time both at $n = 900$ and write down the ratio.
3. Compute the number of element copies each performs, and state both complexities.
4. Say why the slicing version raises no error and gives the right answer.
5. Do the same for a string version and say why strings are worse.

The two-index drill

1. Write the palindrome check with `low = high` as its base case.
2. Run it on `"abcba"`. Then run it on `"abba"`. Quote what happens.
3. Trace the index pairs for both inputs and say exactly where they diverge.
4. State the measure and say by how much it drops.
5. Say which of day 089's three termination conditions the `=` version fails.

The conversion drill

For each loop, name the three parts and write the recursion:

1. Sum a list.
2. Find the maximum.
3. Count occurrences of a value.
4. Build a reversed copy.
5. Check whether a list is sorted.

Then convert two of the recursions back to loops, and say what you did with the accumulator.

The helper-pattern drill

1. Write `is_palindrome(text, low=0, high=-1)` with the bookkeeping in the signature.
2. List three things wrong with that signature.
3. Rewrite it with a private inner helper.
4. Say what the inner function closes over and what that saves on every call.

The break-it drill

Trigger each and record the exact output or error text:

1. Slice in the recursive call and time it against the index version.
2. Use `=` instead of `>=` in a two-index base case, on an even-length input.
3. Build a string with `+` inside a recursion at $n = 5,000$ and time it.
4. Use a mutable default as an accumulator and call the function twice.

5. Run a shrink-by-one recursion on 100,000 elements. Quote the error.
6. Run a split-in-half recursion on the same input. Say what the depth was.

The two-shapes drill

1. Write a shrink-by-one recursion and a split-in-half recursion over the same array.
2. State the depth of each in terms of n .
3. Run both on 100,000 elements and record which survives.
4. Count the total calls each makes.
5. Say when divide and conquer is an improvement and when it is a demonstration.

The string drill

1. Write a recursive reverse using slicing and concatenation.
2. Count the character copies for $n = 10,000$.
3. Rewrite it collecting into a list and joining once.
4. Rewrite it again working on a `list(text)` in place with two indices.
5. State the complexity of all three, and say which you would ship.

The judgement drill

For each, say whether you would use recursion or a loop, and why in one sentence:

1. Sum a flat list.
2. Reverse a string.
3. Flatten an arbitrarily nested list.
4. Merge sort an array.
5. Walk a directory tree.
6. Find the maximum of an array.
7. Binary search a sorted array.

Two of the seven are the cases where recursion genuinely wins. Name what they have in common.

The three-mechanisms drill

1. Define eviction, expiry and invalidation in one sentence each.
2. Give an example of an entry that is expired and still resident.
3. Give an example of an entry evicted while perfectly fresh.
4. Describe the bug that comes from merging eviction and expiry.
5. Say which of the three is about correctness and which is about capacity.

The policy drill

1. Write the `EvictionPolicy` interface with its four methods.
2. Implement LRU using an `OrderedDict` and name the two $O(1)$ operations.
3. Implement Random and state its hit rate relative to LRU.
4. State plain LFU's structural weakness and the two standard fixes.
5. Apply the interface gate: name all four implementations and say why the interface is justified here more clearly than in most examples.
6. Say what `Entry` must *not* contain, and why.

The expiry drill

1. Implement lazy expiry on read. Say why an expired hit must count as a miss.
2. Say precisely what lazy expiry leaks and why.
3. Implement the sampled sweep with Redis's adaptive rule.
4. Say why sampling alone is wasteful.
5. Compute how long an unread expired entry survives, given 20 keys sampled 10 times a second out of 100,000.
6. Contrast this with the food-delivery timeout from day 087 and say why one needs a sweeper for correctness and the other does not.

The stampede drill

1. Compute the concurrent identical queries for a key at 5,000 req/s with a 200 ms load.
2. Say what actually breaks, and why it is not the hot key.
3. Write single-flight with a per-key lock.
4. Say why the lock must be per key and not global.
5. Say why the re-check inside the lock is essential, and what happens without it.
6. Compute the effect of TTL jitter on 1,000 keys loaded together with a 600 s TTL.
7. Describe `stale-while-revalidate` and say what it trades.

The sizing drill

Compute each, showing the multiplication:

1. Memory for 100,000 entries of 2 KB, including entry, dict and LRU overhead.
2. The overhead as a percentage of the data.
3. The same overhead percentage for 200-byte values.
4. Memory for 100,000 entries of 2 MB.
5. Average read time at hit rates of 99, 95, 90 and 50 percent, given 100 μ s and 1 μ s.

Then say which number tells you to count bytes instead of entries, and which tells you where to stop adding capacity.

The threading drill

1. Say what one global lock does to a cache doing a million gets a second.
2. Describe sharding and what it costs.
3. Say why the single-flight locks are a separate matter.
4. Say what happens to a waiter if the loader hangs, and what you would add.

The would-you-ship-it drill

For each, say what you would actually use and why:

1. Memoising a pure function in one process, no TTL.
2. A per-request cache within one request's lifetime.
3. A hot config value read a million times a second.
4. A shared cache across ten servers.
5. A cache in front of a database where staleness is unacceptable.

One of the five needs no cache at all. Name it and say what it needs instead.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Reverse a string recursively. Now do it without extra space.* The shape named first, the index-not-slice rule with its complexity consequence, two indices with the `>=` base case and *why*, the private helper, both costs including the honest note that recursion is never $O(1)$ space, and what you would ship.
 2. *Design an in-memory cache. Now make the eviction policy swappable.* The three mechanisms separated with the expired-and-resident sentence, `Entry` with no policy state, the four-implementation interface, lazy plus sampled expiry with why each alone fails, the stampede with its arithmetic and single-flight with the re-check, TTL jitter, and the hit rate as the only tunable number.
 3. *A very popular key expires. What happens?* The $5,000 \times 0.2$ arithmetic, why the damage lands on unrelated queries, single-flight with a per-key lock and the inner re-check, and jitter as the one-line prevention.
-

Before you move on

- [] I never slice in a recursive call, and I measured the ratio myself.
- [] I can derive `low ≥ high` rather than remember it, and say why `=` fails.
- [] I can convert a loop to a recursion by naming its three parts.
- [] I use a private helper so the public signature has no bookkeeping.
- [] I built a recursive string with `+` and measured how bad it is.
- [] I know which recursion shape survives 100,000 elements and which does not.
- [] I can say when divide and conquer is an improvement rather than a demonstration.
- [] I can name the two cases where recursion genuinely beats a loop.
- [] I can define eviction, expiry and invalidation and give an example of each mismatch.
- [] I can say what `Entry` must not contain and why.
- [] I implemented LRU and Random and can quote their relative hit rates.
- [] I can state plain LFU's weakness and both fixes.
- [] I can explain why lazy expiry leaks and why sampling alone is wasteful.
- [] I can compute the stampede numbers and write single-flight correctly.
- [] I can say why the re-check inside the lock is essential.
- [] I can give the hit-rate table and say where the value falls off a cliff.
- [] I answered all three questions above out loud.

Subsets: the include-or-exclude tree

DATA STRUCTURES AND ALGORITHMS

Subsets: the include-or-exclude tree

You can generate every subset and draw the decision tree that produced them.

SYSTEM DESIGN

Design a logging framework

You can design log levels, appenders and formatters as replaceable parts.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Generate all subsets of this array.*
- *Design a logging library. How do I add a new destination?*

1 What this is, and why they ask it

Given `[1, 2, 3]`, produce every subset: the empty one, `[1]`, `[2]`, `[3]`, `[1,2]`, `[1,3]`, `[2,3]`, and `[1,2,3]`. Eight of them, and eight is not a coincidence — it is 2^3 .

The reason is the shape of the problem, and the shape is the lesson. **For each element there are exactly two choices: take it, or leave it.** Three elements, three independent yes-or-no decisions, 2^3 combinations. Draw those decisions as a tree and every leaf is one subset, produced exactly once.

That tree is the first **backtracking** problem, and backtracking is the next six days. The pattern — *choose, recurse, un-choose* — is the same for subsets, permutations, combinations, N-Queens, Sudoku and word search. Subsets is where you meet it because it is the smallest possible instance: the choice is binary and there is nothing to check.

They ask it because the recursion is six lines and there are three traps in them, one of which — appending the same list object over and over — produces a result that is confidently, uniformly wrong in a way that is very hard to see. And because “now do it with duplicates” is a one-line change that almost nobody gets right first time.

2 The story

The lunch at Sridhar’s cousin’s wedding was served the way those lunches always are: everyone sitting in a long row, and the men coming down the line one at a time, each carrying one vessel.

Six of them, in a fixed order. Rice. Then the sambar man. Then palya. Then the fried thing. Then curd. Then payasa at the end.

Each one stops in front of you and you either nod or you put your hand over the plate. That is the whole interaction. Six of them, six yes-or-no decisions, and whatever ends up on your plate is the result.

Sridhar’s nephew Karthik, who is fourteen and had been made to sit at the end of the row, worked something out during the meal and would not let it go afterwards.

He said: everybody in this row is getting a different plate, and there are exactly sixty-four possible plates, and that is all there are.

His argument was that the first man splits the whole row into two kinds of people — those who took rice and those who did not. Then the second one splits each of those two groups into two again. Six of them, doubling each time: two, four, eight, sixteen, thirty-two, sixty-four.

His uncle did not believe it, so Karthik listed them. He went along an imaginary row saying yes to every one of them, then yes to five and no to the last, then yes to four and no to the fifth, and so on — walking the decisions in order, changing the last one first. It took him about ten minutes and he got sixty-four, with no plate said twice and none missing.

The only part he got wrong at first was the recording. He was calling each plate out to his cousin, who was keeping a list on her phone, and he kept saying “and now add the payasa” and “now take off the payasa” — describing changes to the same plate rather than the finished plate itself. Halfway through she pointed out that her list was sixty-four lines and every line said the same thing, because she had only ever been told about one plate that kept changing.

After that he read out the whole plate, from the beginning, each time.

3 The idea in plain English

Each man with a vessel is one element. Nodding or covering your plate is **include or exclude**. And Karthik’s cousin’s list is the copy trap, which is the trap of this whole day.

Why 2^n

There are n elements and each one is an independent yes-or-no decision, so the number of combinations is $2 \times 2 \times \dots \times 2 = 2^n$. Six vessels, sixty-four plates.

That is not a bound to be improved on. **The output is 2^n things**, so no algorithm can be faster than 2^n — it has to at least write them down. Saying that early is what stops an interviewer asking “can you do better?”

$n = 10$	$\rightarrow$	1,024 subsets	
$n = 20$	$\rightarrow$	1,048,576	
$n = 30$	$\rightarrow$	1,073,741,824	– about a billion; not going to happen

Subsets problems have small n , always. If a problem says $n \leq 20$, that constraint is telling you it expects an exponential solution.

The decision tree

At index `i`, two branches: take `items[i]`, or do not. Each branch recurses on `i + 1`. When `i` reaches the end, one complete decision path has been made, and that path is a subset.

```
def subsets(items):
    result = []
    current = []

    def choose(index):
        if index == len(items):
            result.append(current[:])    # a COPY of the finished plate
            return
        current.append(items[index])    # 1. take it
        choose(index + 1)
        current.pop()                  # 2. un-take it
        choose(index + 1)              # 3. and go on without it

    choose(0)
    return result
```

Six lines of body, and they are the pattern the next six days are built on:

choose · recurse · un-choose

The `current.pop()` is the un-choose, and it is what makes one list serve every branch. Without it, the left branch's choice leaks into the right branch and you get nonsense.

The copy, which is the trap

```
result.append(current)    # WRONG — appends a REFERENCE
```

`current` is one list object that is mutated throughout the entire recursion. Appending it stores a *pointer* to it, so `result` ends up holding 2^n references to the same list — and that list is empty when the recursion finishes, because every `append` was matched by a `pop`.

```
subsets([1, 2, 3]) with `result.append(current)`
→ [[], [], [], [], [], [], [], []]
```

Eight subsets, all empty, all the same object. Karthik's cousin's list with sixty-four identical lines.

`current[:]` — or `list(current)`, or `current.copy()` — takes a snapshot. Say “copy” out loud as you type it; this is the single most common backtracking bug and it is completely silent.

The other two formulations, both worth knowing

Iterative doubling. Karthik's actual argument: each new element doubles the answer.

```
result = [[]]
for item in items:
    result += [subset + [item] for subset in result]
```

Two lines. Start with the empty subset; for each element, take everything you have and add a copy with that element appended. `[[]] → [[], [1]] → [[], [1], [2], [1,2]] → ...`

No recursion, no stack, no copy trap — the list comprehension builds new lists. This is often the version to write in an interview, and then say the recursive one is the shape that generalises.

Bitmasks. There are 2^n subsets and 2^n numbers with `n` bits, so number them:

```
for mask in range(1 << len(items)):
    subset = [items[i] for i in range(len(items)) if mask & (1 << i)]
```

Bit `i` of the mask says whether element `i` is in. `mask = 5` is binary `101`, which is `[items[0], items[2]]`. Elegant, $O(n \cdot 2^n)$, and it only works when `n ≤ 63` — which for a subsets problem it always is.

Three ways, same answer. Interviewers ask for the recursive one because it is the one that generalises to permutations, combinations and constrained search.

Subsets with duplicates

`[1, 2, 2]` has $2^3 = 8$ decision paths but only **six distinct subsets**, because `[1, 2]` can be made two ways. The fix is two lines, and the second is the one people get wrong:

```
items.sort()                                # 1. equal elements become adjacent
...
for i in range(index, len(items)):
    if i > index and items[i] == items[i - 1]:
        continue                            # 2. skip a duplicate at the SAME level
...
```

`i > index`, not `i > 0`. The condition means “this is not the first choice being considered at this position in the tree”. Using `i > 0` would also skip the *second* `2` when it legitimately follows the first inside a subset, and you would lose `[2, 2]` entirely.

That single comparison is the whole duplicate-handling idea, and it reappears in Combination Sum II, Permutations II and Subsets II. Get it right once.

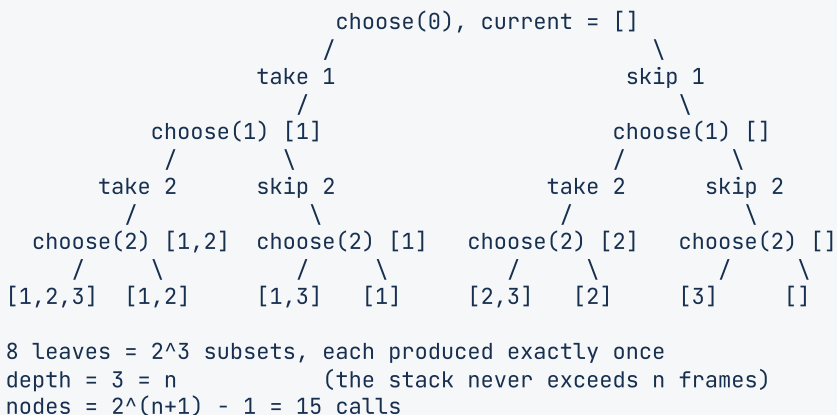
The relationship to what comes next

- **Subsets** — two choices per element: in or out. Today.
- **Combinations** (choose `k` from `n`) — the same tree, pruned when the subset reaches size `k`.
- **Permutations** — `n` choices at the first level, `n-1` at the second: a different tree, same choose-recurse-un-choose.
- **N-Queens, Sudoku, word search** — the same tree with a *validity check* before recursing, which is what makes it “backtracking” rather than “enumeration”.

Subsets is the version with no constraint at all, which is why it is first. Everything after adds a condition on when you may take a branch.

4 The picture

The decision tree for `[1, 2, 3]`. Left branch takes the element, right branch skips it.



What to notice: **the depth is `n` and the leaf count is 2^n** . Time is the whole tree; space is the deepest path — the distinction from [day 088](#), in its most dramatic form. Fifteen calls, three frames.

The `current` list over time, which is why the copy matters:

Step 3 — the base case, and the copy

```
if index == len(items):  
    result.append(current[:])    # COPY. Say the word as you type it.  
    return
```

The base case is the measure — `len(items) - index` — reaching zero, exactly as on [day 089](#).

Step 4 — the other arrangement, which some find clearer

The version above has the leaf at the bottom. There is an equivalent formulation where **every node is a subset**, not just the leaves:

```
def build(start):  
    result.append(current[:])    # every node counts, including the root  
    for i in range(start, len(items)):  
        current.append(items[i])  
        build(i + 1)  
        current.pop()
```

Same 2^n answers, and this shape is the one that generalises to combinations — cap the loop and you get “choose k” — and to Combination Sum, where you check a running total before recursing. If you can only remember one, remember this one, because it is the template for the rest of the phase.

Step 5 — duplicates

```
for i in range(start, len(items)):  
    if i > start and items[i] == items[i - 1]:  
        continue    # '> start', not '> 0'
```

With the array sorted first. Say the two halves out loud: **sort so equal elements are adjacent**, and **skip a repeat only when it is another choice at the same level**.

The complete solution

```
def subsets(items: list[int]) → list[list[int]]:
    """Every subset, via the include-or-exclude tree.

     $2^n$  subsets, so the output alone is exponential – no algorithm can beat it.
    Depth  $n$ , leaves  $2^n$ : TIME is the whole tree, SPACE is the deepest path.

    The pattern for the whole phase: choose, recurse, UN-choose.
    """
    result: list[list[int]] = []
    current: list[int] = []

    def choose(index: int) → None:
        if index == len(items):
            result.append(current[:]) # COPY – `current` is one mutating list
            return
        current.append(items[index]) # choose
        choose(index + 1) # recurse
        current.pop() # un-choose
        choose(index + 1) # and the branch without it

    choose(0)
    return result

def subsets_template(items: list[int]) → list[list[int]]:
    """The same answers, arranged so EVERY node is a subset rather than only
    the leaves. This is the template the rest of the phase uses: cap the loop
    and it becomes combinations; add a check and it becomes constrained search.
    """
    result: list[list[int]] = []
    current: list[int] = []

    def build(start: int) → None:
        result.append(current[:])
        for i in range(start, len(items)):
            current.append(items[i])
            build(i + 1)
            current.pop()

    build(0)
    return result

def subsets_iterative(items: list[int]) → list[list[int]]:
    """Karthik's argument, directly: each element doubles the answer.
    Two lines, no recursion, no stack, and no copy trap."""
    result: list[list[int]] = [[]]
    for item in items:
        result += [subset + [item] for subset in result]
    return result

def subsets_bitmask(items: list[int]) → list[list[int]]:
    """ $2^n$  subsets and  $2^n$  numbers with  $n$  bits, so number them.
    Bit  $i$  of the mask says whether element  $i$  is in. Works up to  $n = 63$ ."""
    n = len(items)
    return [
        [items[i] for i in range(n) if mask & (1 << i)]
        for mask in range(1 << n)
    ]

def subsets_with_duplicates(items: list[int]) → list[list[int]]:
```

```

"""Subsets II. Two lines of difference from the template.

1. SORT, so equal elements are adjacent.
2. Skip a repeat only when it is another choice at the SAME level:
   `i > start`, NOT `i > 0`. With `i > 0` you would also skip the second
   2 when it legitimately follows the first, losing [2, 2] entirely.
"""
items = sorted(items)
result: list[list[int]] = []
current: list[int] = []

def build(start: int) → None:
    result.append(current[:])
    for i in range(start, len(items)):
        if i > start and items[i] == items[i - 1]:
            continue
        current.append(items[i])
        build(i + 1)
        current.pop()

build(0)
return result

def combinations(items: list[int], k: int) → list[list[int]]:
    """The same tree with one cap: stop when the subset reaches size k.

    The extra `if` is PRUNING – if there are not enough elements left to reach
    k, the branch cannot possibly succeed, so do not walk it. That is the idea
    the rest of the phase is built on.
    """
    result: list[list[int]] = []
    current: list[int] = []

    def build(start: int) → None:
        if len(current) == k:
            result.append(current[:])
            return
        for i in range(start, len(items)):
            if len(current) + (len(items) - i) < k:
                break # not enough left: prune
            current.append(items[i])
            build(i + 1)
            current.pop()

    build(0)
    return result

def subsets_broken(items: list[int]) → list[list[int]]:
    """The trap, written out so you can run it: appends a REFERENCE, not a copy.
    Every entry is the same object, and it is empty by the end."""
    result: list[list[int]] = []
    current: list[int] = []

    def choose(index: int) → None:
        if index == len(items):
            result.append(current) # no [:]
            return
        current.append(items[index])
        choose(index + 1)
        current.pop()
        choose(index + 1)

    choose(0)
    return result

```

```

if __name__ == "__main__":
    print(subsets([1, 2, 3]))
    # [[1, 2, 3], [1, 2], [1, 3], [1], [2, 3], [2], [3], []]

    print(sorted(map(sorted, subsets_template([1, 2, 3]))) ==
              sorted(map(sorted, subsets([1, 2, 3])))) # True
    print(sorted(map(sorted, subsets_iterative([1, 2, 3]))) ==
          sorted(map(sorted, subsets([1, 2, 3])))) # True
    print(sorted(map(sorted, subsets_bitmask([1, 2, 3]))) ==
          sorted(map(sorted, subsets([1, 2, 3])))) # True

    print(len(subsets([1, 2, 3, 4, 5])), 2 ** 5) # 32 32
    print(subsets([], subsets([7]))) # [[]] [[7], []]

    print(subsets_with_duplicates([1, 2, 2]))
    # [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]
    print(len(subsets_with_duplicates([1, 2, 2])), "distinct, vs", 2 ** 3, "paths")
    # 6 distinct, vs 8 paths

    print(combinations([1, 2, 3, 4], 2))
    # [[1, 2], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]]

    print(subsets_broken([1, 2, 3]))
    # [], [], [], [], [], [], [], [] ← eight references to one empty list

    # the size, so you feel it
    for n in (10, 20, 22):
        print(f"n = {n}: {2 ** n:,} subsets")

```

6 What it costs

Time

```

subsets produced:      2^n
average subset length: n / 2
elements written:      2^n × n/2 = n · 2^(n-1)
→ 0(n · 2^n) total work

```

The 2^n is the number of answers and the n is the cost of copying each one. **You cannot beat 2^n , because the output is that big** — and saying so removes the “can you do better?” question.

n = 10:	1,024 subsets,	~5,000 elements written
n = 20:	1,048,576 subsets,	~10,000,000 elements
n = 25:	33,554,432 subsets,	~420,000,000 elements – seconds to minutes
n = 30:	1,073,741,824	– not going to finish

Twenty is comfortable, twenty-five is painful, thirty is impossible. A problem that asks for subsets will bound n at twenty-something, and that bound is the hint.

Calls versus depth

```
tree nodes (calls): 2^(n+1) - 1    n = 20 → ~2,000,000 calls
depth:              n              n = 20 → 20 frames
```

The most dramatic instance of the [day 088](#) distinction: **two million calls, twenty frames**. A subsets recursion will never hit the recursion limit — the input would have to be a thousand elements, and 2^{1000} is not a number anyone is waiting for.

Space

```
output:    2^n lists, average length n/2 → O(n · 2^n)  – unavoidable
current:   one list of at most n         → O(n)
stack:     n frames                      → O(n)
-----
extra space beyond the output: O(n)
```

The extra space is $O(n)$, not $O(2^n)$ — one working list and `n` frames. That is worth saying, because it is the honest answer to “how much memory does this use?” and it separates the output from the algorithm.

```
n = 20, integers: 1,048,576 lists × ~120 B overhead ≈ 126 MB of Python list objects
```

Which is a real number, and the reason subsets problems generate rather than store when they can.

The three formulations, compared

```
recursive:  O(n · 2^n) time, O(n) extra  generalises to everything else
iterative:  O(n · 2^n) time, O(n · 2^n)  two lines, no stack, builds all lists as it goes
bitmask:    O(n · 2^n) time, O(n) extra  n ≤ 63; no recursion
```

Same complexity, and the difference is what they teach. **Write the recursive one if the interviewer wants backtracking; write the iterative one if they want the answer.**

Duplicates

```
[1, 2, 2]:    2^3 = 8 decision paths, 6 distinct subsets
[2, 2, 2]:    8 paths, 4 distinct          ([], [2], [2,2], [2,2,2])
all distinct: 2^n paths, 2^n subsets
```

The skip prunes whole branches rather than filtering at the end, which matters: filtering a million-element result with a set costs the memory of the million *and* requires the subsets to be hashable. **Prune, do not filter.**

7 The traps

Trap 1 — appending the list instead of a copy

```
result.append(current)          # a REFERENCE to one mutating list
```

```
subsets([1, 2, 3]) → [[], [], [], [], [], [], [], []]
```

Eight results, all the same object, all empty — because every `append` was undone by a `pop` before the recursion finished. **The single most common backtracking bug**, it produces a uniformly wrong answer with no error, and it looks fine until you print it.

`current[:]`, `list(current)` or `current.copy()`. Say “copy” as you type it.

Trap 2 — forgetting to un-choose

```
current.append(items[index])
choose(index + 1)
                                # missing: current.pop()
choose(index + 1)
```

The “skip this element” branch now runs with the element still in `current`, so both branches include it and half the subsets are wrong. Not a crash — a wrong answer that looks plausible, because the count is still 2^n .

Choose, recurse, un-choose. Write all three as one unit.

Trap 3 — `i > 0` instead of `i > start` in the duplicate skip

```
if i > 0 and items[i] == items[i - 1]:
    continue
```

On `[1, 2, 2]` this loses `[2, 2]` and `[1, 2, 2]`, because it skips the second `2` even when it is being appended *after* the first inside the same subset. The condition must mean “**another choice at this level**”, which is `i > start`.

Test it on `[2, 2]`: the right answer has three subsets — `[]`, `[2]`, `[2, 2]` — and the wrong one has two.

Trap 4 — forgetting to sort before de-duplicating

```
# items = [2, 1, 2] - not sorted
if i > start and items[i] == items[i - 1]:
```

The skip compares *adjacent* elements, so equal values must be adjacent. Unsorted, the two 2s are never neighbours and nothing is skipped, so duplicates come out anyway. `sort()` first, always.

Trap 5 — filtering duplicates at the end

```
return [list(s) for s in {tuple(sorted(sub)) for sub in subsets(items)}]
```

Correct, and it builds all 2^n subsets before removing any. On `[2] * 20` that is a million subsets generated to return twenty-one. **Prune in the tree, do not filter the result.**

Trap 6 — `[]` versus `[[] ]` as the starting point

```
result = []                                # iterative version
for item in items:
    result += [subset + [item] for subset in result]
return result                             # → [] for any input
```

Starting from an empty list means the comprehension never has anything to extend, so the answer is always empty. The seed is `[]`: **one subset, the empty one**. And the empty subset is a real answer that must appear in the output — `subsets([])` is `[]`, not `[]`.

Trap 7 — expecting a particular order

The recursive version emits `[[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`; the iterative one emits `[[], [1], [2], [1,2], [3], ...]`. Both are complete and correct. If a test compares against an exact list, sort both sides — and if an interviewer’s example is in a different order, say “the order differs and the set is the same” rather than assuming you are wrong.

Trap 8 — trying to be clever about the complexity

There are 2^n subsets. Any correct solution must produce all of them, so **no algorithm is faster than $O(2^n)$** . Attempting to “optimise” is a misunderstanding of the problem, and saying the bound out loud early is the way to avoid twenty minutes of it.

8 In the interview

How it gets asked

- The base: “Generate all subsets of this array.” LeetCode 78.
- Immediately after: “Now the array can contain duplicates and the subsets must be unique.” LeetCode 90.

- The variation: “All combinations of k elements from n .” LeetCode 77. Same tree, one cap.
- The escalation, which is the rest of the phase: “All permutations.” “All combinations that sum to a target.”
- The complexity probe: “What is the complexity, and can you do better?”

What to say out loud, in the first ninety seconds

1. **Name the shape and the size together.** “Every element is an independent take-it-or-leave-it decision, so there are 2^n subsets. The output alone is exponential, so no solution can be faster than that — I would expect `n` to be small.”
2. **Describe the tree before writing code.** “I will walk a decision tree: at each index, one branch takes the element and one skips it. Each leaf is a complete set of decisions, which is one subset.”
3. **Say the pattern by name.** “Choose, recurse, un-choose. The un-choose is what lets a single working list serve every branch.”
4. **Flag the copy before you write it.** “When I record a subset I have to append a *copy*, because the working list keeps mutating — appending the list itself gives 2^n references to one list that ends up empty.”
5. **Give both complexities separately.** “ $O(n \cdot 2^n)$ time — 2^n subsets, each costing $n/2$ to copy — and $O(n)$ extra space beyond the output: one working list and n stack frames.”
6. **Offer the iterative one.** “There is also a two-line iterative version: start with the empty subset and double it with each element. Same complexity, no recursion. I will write the recursive one because it is the shape that generalises.”

The follow-ups

“**What is the complexity, and can you do better?**” “ $O(n \cdot 2^n)$ — there are 2^n subsets and each takes on average $n/2$ element copies to write out. And no, you cannot do better, because **the output itself is 2^n things** — any correct algorithm has to produce all of them. Extra space beyond the output is only $O(n)$: one working list of at most n elements, and n stack frames. Worth separating those, because the tree has about two million *calls* at $n = 20$ and only twenty frames — time is the whole tree, space is the deepest path.”

“**Now the input has duplicates and the subsets must be unique.**” “Two changes. Sort the array first, so equal elements are adjacent. Then, inside the loop over candidates, skip an element if it equals the previous one **and** it is not the first candidate at this level — `i > start`, not `i > 0`. That condition means ‘another choice at the same position in the tree’. With `i > 0` you would also skip the second 2 when it legitimately follows the

first inside a subset, and you would lose `[2, 2]` entirely. The important part is that this prunes branches rather than filtering afterwards — filtering would build all 2^n subsets and then throw most away.”

“Why do you need the copy?” “Because `current` is a single list object that is mutated all the way through the recursion. Appending it stores a reference, so the result holds 2^n pointers to the same list — and by the time the recursion finishes, every append has been matched by a pop, so that list is empty. You get 2^n empty lists and no error at all. `current[:]` takes a snapshot of what it holds at that moment.”

“Now do combinations of size k.” “The same tree with a cap: record when the working list reaches size k and return, rather than at the leaves. And I would add a prune — if the number of elements remaining is not enough to reach k, break out of the loop, because that branch cannot possibly succeed. That pruning is really the point of the whole phase: subsets is the version with no constraint, and everything after it adds a condition that lets you cut off branches early.”

“Which version would you actually write?” “For subsets alone, the two-line iterative one — start with the empty subset and double it for each element. Same complexity, no recursion, no copy trap. But I would write the recursive version in an interview, because it is the template: cap the depth and it is combinations, change the loop and it is permutations, add a validity check before recursing and it is N-Queens. Nobody asks about subsets because they want subsets.”

“What if n is 40?” “Then the question is not really ‘generate all subsets’, because 2^{40} is a trillion and the output would be tens of terabytes. If somebody asks for subsets at $n = 40$, they want something else — a count, which is 2^n and needs no enumeration; or a *best* subset under some criterion, which is dynamic programming or meet-in-the-middle, where you split into two halves of twenty, enumerate each in a million, and combine. Meet-in-the-middle turns 2^n into $2^{(n/2)}$ and is the standard trick when n is around forty.”

A model answer

Asked: *generate all subsets of this array.*

“Let me start with the size, because it settles the whole approach. Every element is an independent decision — take it or leave it — so with n elements there are 2^n combinations. That is not a bound I can improve on: **the output itself is 2^n things**, so any correct algorithm has to write them all out. If you tell me n could be thirty, I would say that is a billion subsets and we should talk about what you actually need.

The structure is a decision tree. At index zero there are two branches: one where I take the first element and one where I do not. Each branch does the same at index one, and so on. When I reach the end of the array, one complete set of decisions has been made, and that is one subset. Eight leaves for three elements.

The code is the pattern I will use for the rest of this topic: **choose, recurse, un-choose**. Append the element to a working list, recurse on the next index, then pop it off and recurse again without it. The pop is not tidying up — it is what lets a single working list serve every branch of the tree. Leave it out and the ‘skip’ branch still contains the element, so half the answers are wrong, with no error.

The one trap I would flag before writing it: when I record a subset, I append a **copy**, not the working list itself. The working list is one object being mutated throughout, so appending it stores a pointer — and by the end, every append has been matched by a pop, so all 2^n entries point at the same empty list. It is completely silent and it is the classic bug here.

Complexity: $O(n \cdot 2^n)$ time, because there are 2^n subsets and each costs about $n/2$ element copies to write. Extra space beyond the output is only $O(n)$ — one working list and n stack frames. Those two are worth separating: at $n = 20$ the tree has about two million calls and the stack never exceeds twenty frames.

There is also a two-line iterative version — start with a list containing the empty subset, and for each element add a copy of everything you have with that element appended. Same complexity, no recursion, no copy trap, and it is what I would write if I only needed subsets. I have written the recursive one because it is the shape that generalises: cap the size and it is combinations, add a validity check before recursing and it is N-Queens.

If the array can contain duplicates, it is two more lines: sort it first so equal elements are adjacent, then skip a candidate that equals the previous one when it is not the first candidate at that level — `i > start`, not `i > 0`. That distinction is the whole thing: `i > start` means ‘another choice at the same position in the tree’, whereas `i > 0` would also block the second 2 from following the first inside a subset and lose `[2, 2]`.”

9 Recall card

- **Every element is an independent take-it-or-leave-it decision, so there are exactly 2^n subsets** — and you cannot beat 2^n because the output is 2^n things. Say the size first and the “can you do better?” question never happens. $n = 20$ is a million and comfortable; $n = 30$ is a billion and impossible, so a subsets problem always bounds n at twenty-something.
- **The pattern for the whole phase: choose · recurse · un-choose.** Write the three lines as one unit. The `pop` is **not cleanup** — it is what lets one working list serve every branch, and omitting it makes the “skip” branch still contain the element, with no error.
- **Append a COPY** — `current[:]` — **never the list itself.** `current` is one object mutated throughout, so appending it gives 2^n references to one list that is empty at the end: `[[], [], [], ...]`, and no error. Say “copy” as you type it. This is the defining backtracking bug.
- **Duplicates take two lines: SORT first, then skip when `items[i] == items[i-1]` AND `i > start`** — `i > start` means “another choice at this *level*”, while `i > 0` would also block the second 2 from following the first and lose `[2, 2]`. **Prune in the tree; never filter the result** — filtering builds all 2^n before discarding.
- **Time is the whole tree, space is the deepest path: ~2 million calls at $n = 20$, and only 20 frames.** $O(n \cdot 2^n)$ time; $O(n)$ extra space beyond the output. Three formulations, same complexity: the **recursive** one generalises (cap it → combinations, add a check → N-Queens), the **iterative doubling** one is two lines and has no copy trap (seed it `[[] ]`, not `[ ]`), and the **bitmask** one works to $n \leq 63$.

Design a logging framework

1 What this is, and why they ask it

A logging framework takes a message from anywhere in a program and gets it to zero or more destinations — a file, the console, a network collector — in a chosen format, if it is important enough to bother with.

The question in the prompt is the easy half: adding a destination should be **one class and one line of configuration**, and that is a `Handler` interface with an obvious set of implementations.

The half that decides whether the design is any good is the word “**if**”. A logging call that produces nothing must cost almost nothing, because there are hundreds of thousands of them a second and most of them are switched off. That single requirement drives the most important API decision in the whole library: **the level check must happen before the message is built**, which is why `logger.debug("user %s failed", user_id)` is right and `logger.debug(f"user {user_id} failed")` is wrong — the f-string is evaluated whether or not anybody wants it.

They ask it because everybody uses logging and almost nobody has looked at how it works, because “where it goes” and “what it looks like” being two separate interfaces is a clean example of orthogonal design, and because the follow-ups — asynchronous writing, what happens on a crash, and how you find one request among a billion lines — are real operational problems with real answers.

2 The story

Kumar has run security at the tyre factory since 2011, forty-one people across three shifts.

The rule when he joined was that everything went to the manager. Every gate entry, every vehicle, every visitor, every torch that needed a new battery. The manager’s phone rang about sixty times a night, and by the third month he had simply stopped answering it, which meant the four calls that mattered did not get through either.

What they have now is three levels, and Kumar can recite them.

Routine things go into the shift record and nobody is disturbed. A vehicle in at 2:40, out at 3:15. Nobody reads it, most nights. It is there for the mornings when somebody asks.

Things that are unusual but not dangerous go to the shift manager on the intercom. A lorry arriving without paperwork. A man at the gate saying he is from the electricity office.

And then the siren, which Kumar has sounded four times in fourteen years.

The part he has had to teach every new supervisor is a different thing, and he had to tell Ravi three times.

Ravi joined last year. He is careful and he writes beautifully, and he was spending fifteen minutes at the end of every round composing a full account of what had happened — times, names, vehicle numbers, the state of the lights on the east side — and then reading all of it into the shift record, where nobody would ever look at it.

Kumar's objection was not that the record is useless. It was that Ravi was doing the writing *first* and finding out afterwards that nobody wanted it. **Ask what level a thing is before you spend fifteen minutes describing it.** Routine gets one line. If somebody is going to act on it, then take the time and write it properly.

The other change was who keeps the record. The gate has its own now, and the loading bay has its own, and both also feed up into the factory record. So when there was a problem at the loading bay in March, Kumar could say “turn everything up at the loading bay” and read every detail from there, without wading through four weeks of routine gate entries to find it.

3 The idea in plain English

Kumar's three levels are **log levels**. Ravi writing before checking is the **lazy formatting** problem, and it is the most expensive mistake in logging. And the gate having its own record that also feeds upward is the **logger hierarchy**.

The pieces, and why there are four of them

- **Logger** — what your code calls. Named, hierarchical: `app.payments.stripe`.
- **LogRecord** — one event: level, message, arguments, timestamp, logger name, and the context around it.
- **Handler** (*interface*) — **where** it goes. Console, file, rotating file, network, database.
- **Formatter** (*interface*) — **what it looks like**. Plain text, JSON, one line, multi-line.
- **Filter** (*optional interface*) — a last say on whether this particular record passes.

Handler and **Formatter** are separate on purpose, and that separation is the answer to “how do I add a destination”. They vary independently: you may want JSON to a file and plain text to the console, or the same format to three destinations. Combining them

gives you `JsonFileHandler`, `TextFileHandler`, `JsonNetworkHandler` and so on — **N × M classes instead of N + M.**

Adding a destination is then exactly one class implementing one method, plus one line of wiring. That is the whole answer to the prompt, and it takes ten seconds to say.

Levels, and the one that matters

DEBUG	what I need when I am debugging this	– off in production
INFO	normal, notable events	– on
WARNING	something is odd but handled	– on
ERROR	an operation failed	– on, and someone should see it
CRITICAL	the process cannot continue	– on, and it wakes somebody up

Kumar's shift record, intercom and siren, with two extra gradations.

A level is just an integer, so “is this enabled?” is one comparison. That cheapness is the point, and it leads directly to the important part.

The expensive mistake: building a message nobody wants

```
logger.debug(f"processing order {order.id} for {user.name}: {order.to_dict()}")
```

The f-string is evaluated **before** `debug` is called. Python has no choice — arguments are evaluated first. So `order.to_dict()` runs, a string is built, and *then* the logger looks at the level and throws it all away.

```
logger.debug("processing order %s for %s: %s", order.id, user.name, order.to_dict)
```

Now the formatting is deferred: the logger checks the level first, and only if the record will actually be emitted does it do the `%` substitution. The arguments are still evaluated — `order.to_dict` is passed as a *function*, not called — which is the trick for genuinely expensive values.

Ravi writing his fifteen-minute account before asking whether anyone wanted it.

The numbers in §6 make this concrete, and they are startling: with an expensive argument, suppressed debug logging can burn several cores doing nothing at all.

And when the argument is genuinely costly to compute even lazily:

```
if logger.isEnabledFor(logging.DEBUG):
    logger.debug("state: %s", expensive_dump())
```

Ugly, and correct, and worth doing in the three places per system where it matters.

The hierarchy, which is what makes logging usable in production

Loggers are named with dots and form a tree:

```

root
└─ app
   └─ app.payments
      └─ app.payments.stripe
      └─ app.orders

```

A record logged to `app.payments.stripe` is offered to that logger's handlers, then **propagates upward** to `app.payments`, then `app`, then the root — each of which may have its own handlers and its own level.

That is [day 074](#)'s chain of responsibility, with one difference worth naming: **in a chain, the first able handler stops the walk; here every level gets it**. So it is a chain in structure and a broadcast in behaviour, unless somebody sets `propagate = False` to cut it — which is the `stopPropagation` of the logging world.

Why it matters: you can raise the volume on **one subsystem** without drowning in everything else. `app.payments.stripe` at DEBUG, everything else at INFO, one line of configuration, no code change, no redeploy if the configuration is reloadable. Kumar turning everything up at the loading bay.

Structured logging, because grep does not scale

```

logger.info("payment failed for order %s", order_id)           # a sentence
logger.info("payment_failed", order_id=order_id, amount=amount,  # a record
            provider="razorpay", attempt=2)

```

The first is readable by a person and useless to a machine: finding “all failed payments over ₹10,000 for Razorpay last Tuesday” means regular expressions over terabytes. The second is a dictionary, so the same question is a query.

Log events, not sentences. The message becomes a stable event name and everything variable becomes a field — which also means the message text can be changed without breaking every dashboard built on it.

Context, and the field that matters most

One request produces twenty log lines from ten different modules, and in a system doing ten thousand requests a second those two hundred thousand lines a second are interleaved. Without a shared identifier, reconstructing one request is impossible.

So every record carries a **correlation id** — generated at the edge, stored in a context variable, and attached automatically by the framework rather than passed through every function signature.

```

request_id = ContextVar("request_id")    # per-task, not per-thread; asyncio-safe

```


This is the single most valuable field in a log line, and it costs one middleware and one `ContextVar`. Without it, logs are a stream of unrelated sentences.

Asynchronous writing, and what it trades

A synchronous handler writes on the calling thread. If the destination is a file that is fine; if it is a network collector, a slow collector now makes your request handler slow, and a hung one hangs it.

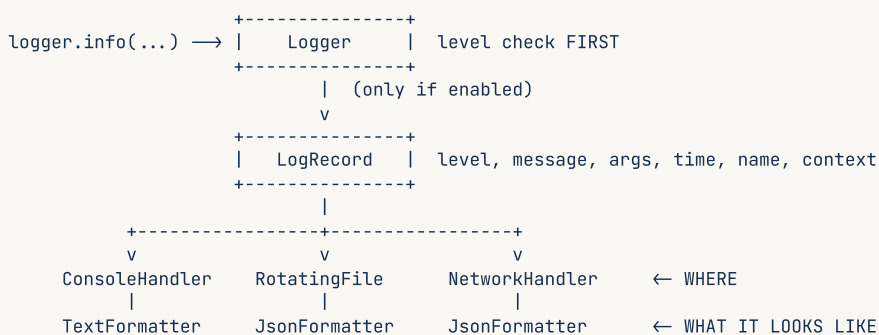
So real systems put a **queue** between the logger and the handler: the caller appends and returns, and a background thread drains it. That decouples the application from the destination, and it introduces two genuine costs:

- **Loss on crash.** Anything still in the queue when the process dies is gone — which is exactly the moment you most want the logs. Mitigate by flushing on shutdown and by writing CRITICAL synchronously.
- **A bounded queue must drop.** If the consumer cannot keep up, you either block the application (which is what you were avoiding) or discard records. Dropping is usually right, and **it must be counted and reported**, or you are silently missing data.

Blocking is a correctness disaster and dropping is a data problem. Choose dropping, count the drops, and alert on them.

4 The picture

The two orthogonal interfaces, which is the answer to the prompt:



two interfaces, $N + M$ classes.

combined, it would be $N \times M$: `JsonFileHandler`, `TextFileHandler`, `JsonNetworkHandler`, ...

The hierarchy and propagation:

```

app.payments.stripe  level DEBUG   handlers: [file]
    | propagate
    v
app.payments         level INFO    handlers: []
    | propagate
    v
app                  level INFO    handlers: [console]
    | propagate
    v
root                 level WARNING handlers: [network]

a DEBUG record from app.payments.stripe:
- passes that logger's level (DEBUG) → written to the file
- propagates up; app's console handler has its own level check
- reaches root; the network handler decides for itself

turn one subsystem up without touching anything else – one line of config.
`propagate = False` on any node cuts the chain there.

```

And the expensive mistake, drawn as what actually executes:

```

logger.debug(f"order {order.id}: {order.to_dict()}")

1. order.to_dict()      ← RUNS. Always. Even at INFO level.
2. build the string     ← RUNS
3. call debug()
4. check the level      ← "not enabled"
5. discard everything from steps 1 and 2

logger.debug("order %s: %s", order.id, order.to_dict())

1. call debug() with a function reference ← nothing is computed
2. check the level → not enabled → RETURN
-----
steps 1 and 2 of the first version never happen

```

5 How it actually works

Move 1 · Clarify (minutes 0–5)

“Library for other people, or the logging inside one service?” — A library, so the API and the extension points matter more than the destinations. **“Structured or free text?”** — Structured, and I would push for it, because grep does not scale. **“How much volume?”** — This decides synchronous versus asynchronous and whether sampling is needed. **“Must a log line ever be guaranteed to survive a crash?”** — Because that decides whether anything is written synchronously.

“I will assume one process with several threads, that destinations may be slow, and that we want to be able to change levels without redeploying. I am not designing the collector, the storage or the query layer.”

Move 2 · The nouns (minutes 5–10)

- **Level** — an `IntEnum`, so comparison is one integer comparison.
- **LogRecord** — frozen: level, event, fields, timestamp, logger name, context.
- **Logger** — named, hierarchical, holds a level and a list of handlers.
- **Handler** (*interface*) — where. Console, file, rotating file, network, null.
- **Formatter** (*interface*) — what it looks like. Text, JSON.
- **Filter** (*interface*) — an optional final say, for sampling and rate limiting.
- **LogManager** — the registry of loggers, so `get_logger("a.b")` returns the same object each time.

Seven, three interfaces — and all three pass the gate, because “console and file”, “text and JSON”, and “sample and rate-limit” are each two real implementations somebody wants.

Move 3 · The hot path, where the level check comes first

```
class Logger:
    def log(self, level: Level, event: str, *args, **fields) → None:
        if level < self._effective_level():          # THE FIRST THING. One int comparison.
            return
        record = LogRecord(
            level=level, event=event, args=args, fields=fields,
            timestamp=time.time(), logger=self._name,
            context=dict(_context.get({})),          # correlation id, user id, ...
        )
        self._dispatch(record)
```

The level check is the first statement in the method, and everything expensive — building the record, reading the context, formatting — is after it. A suppressed call is one comparison and a return.

```
def debug(self, event: str, *args, **fields) → None:
    self.log(Level.DEBUG, event, *args, **fields)
```

And the API is `debug(event, *args)` rather than `debug(message)` **precisely so that callers can defer formatting**. The signature is the mechanism: if the only parameter were a finished string, there would be no way for a caller to avoid building it.

Move 4 · The effective level and propagation

```
def _effective_level(self) → Level:
    """Walk up the hierarchy until a level is explicitly set."""
    logger = self
    while logger is not None:
        if logger._level is not None:
            return logger._level
        logger = logger._parent
    return Level.WARNING # the root default
```

Inheriting the level rather than copying it is what makes “set `app` to DEBUG” affect every logger underneath without touching them.

```
def _dispatch(self, record: LogRecord) → None:
    logger = self
    while logger is not None:
        for handler in logger._handlers:
            if handler.level ≤ record.level and handler.passes_filters(record):
                handler.emit(record) # EVERY level gets it, not just the first
        if not logger._propagate:
            break # stopPropagation
        logger = logger._parent
```

Point at the difference from a plain chain of responsibility: **every ancestor’s handlers run, not just the first that can**. It is a chain in structure and a broadcast in behaviour, and `propagate = False` is the escape.

Each handler also has **its own level**, which is what lets a file capture DEBUG while the console shows only WARNING from the same logger.

Move 5 • The two interfaces

```
class Formatter(Protocol):
    def format(self, record: LogRecord) → str: ...

class JsonFormatter:
    def format(self, record: LogRecord) → str:
        payload = {
            "ts": datetime.fromtimestamp(record.timestamp, UTC).isoformat(),
            "level": record.level.name,
            "logger": record.logger,
            "event": record.event,
            **record.context,                # correlation id and friends
            **record.fields,
        }
        return json.dumps(payload, default=str)    # default=str: never raise on a bad value
```

`default=str` is a small, real decision: **a logger must never raise**. A value that will not serialise should degrade to its `repr`, not take down the request it was describing.

```
class Handler:
    def __init__(self, formatter: Formatter, level: Level = Level.DEBUG) → None:
        self.formatter, self.level = formatter, level
        self._filters: list[Filter] = []

    def emit(self, record: LogRecord) → None:
        raise NotImplementedError

class FileHandler(Handler):
    def emit(self, record: LogRecord) → None:
        try:
            self._stream.write(self.formatter.format(record) + "\n")
        except Exception:
            _handle_internal_error()          # never propagate to the caller
```

Adding a destination is this class with one method. That is the answer to the prompt, and the `try` is the second half of “a logger must never raise”: a full disk must not turn into an exception in unrelated business code.

Move 6 · Asynchronous emission

```
class QueueHandler(Handler):
    """Hand off to a background thread so a slow destination cannot slow the caller.

    Bounded, and it DROPS rather than blocks – blocking would reintroduce
    exactly the coupling this exists to remove. Drops are counted, because a
    silent drop is worse than a slow log.
    """

    def __init__(self, target: Handler, capacity: int = 10_000) → None:
        self._queue: queue.Queue[LogRecord] = queue.Queue(maxsize=capacity)
        self._dropped = 0
        threading.Thread(target=self._drain, args=(target,), daemon=True).start()

    def emit(self, record: LogRecord) → None:
        try:
            self._queue.put_nowait(record)
        except queue.Full:
            self._dropped += 1  # counted and reported, never silent
```

And the two mitigations, stated together: **flush on shutdown**, and **write CRITICAL synchronously**, because the records you most want are the ones written just before the process died.

Move 7 · Sampling, for the line that fires a million times

```
class SampleFilter:
    """Keep 1 in N of a noisy event. Applied per event name, not globally –
    you want all the errors and one percent of the cache hits."""

    def __init__(self, rate: float) → None:
        self._rate = rate

    def passes(self, record: LogRecord) → bool:
        if record.level ≥ Level.ERROR:
            return True  # never sample away an error
        return random.random() < self._rate
```

The `level ≥ ERROR` line is the important one. Sampling that discards errors is a way of losing exactly the information you were collecting logs for.

Real systems

- Python's `logging` is this design: hierarchical dotted loggers, propagation, handlers and formatters as separate objects, per-handler levels, and `logging.QueueHandler` for the async case. The `%`-style deferred formatting exists for the reason in §3, which is why linters warn about f-strings in log calls.
- **Log4j** / **Logback** / **SLF4J** in Java use the same shapes with different names — *appenders* instead of handlers, *layouts* instead of formatters — and Logback's `AsyncAppender` is the queue handler, including the drop-on-full behaviour.

- **structlog** and **Zap** (Go) are the structured-first versions: the log call takes an event name and key-value pairs, and formatting is a pipeline of processors.
- **OpenTelemetry** standardises the context propagation — the trace and span ids that make the correlation id work across services rather than just within one.
- **Log4Shell (CVE-2021-44228)** is the cautionary tale: Log4j interpolated `${...}` expressions inside *log messages*, so logging a user-supplied string could execute code. The lesson is one line — **never interpret user data as a template** — and it belongs in any answer about formatting.

6 The numbers

The cost of a log call that produces nothing

```
10,000 requests/second × 20 log statements each = 200,000 log calls/second
of which ~80% are DEBUG and switched off       = 160,000 suppressed calls/second
```

```
suppressed call, level check only      ~0.05 µs → 160,000 × 0.05 µs = 8 ms/s
suppressed call, f-string already built ~0.50 µs →                80 ms/s
suppressed call, f-string with a
  json.dumps of a real object ~50 µs      →                8,000 ms/s
```

Eight seconds of CPU per second — eight cores — spent building strings that are thrown away. That is the whole argument for deferred formatting, and it is a number rather than a principle. The first row is what you get by checking the level first; the third is what you get from an f-string with an expensive argument.

Volume and disk

```
200,000 log calls/second, ~200 B per line
at DEBUG (everything): 200,000 × 200 B = 40 MB/s = 3.4 TB/day
at INFO (20% of lines): 40,000 × 200 B = 8 MB/s = 690 GB/day
at WARNING (1%):       2,000 × 200 B = 0.4 MB/s = 35 GB/day
```

Three and a half terabytes a day at DEBUG. That is why DEBUG is off in production, why sampling exists, and why “just log everything and search later” is not a plan. And at a typical ₹2 per GB per month for hot log storage, the DEBUG option is roughly ₹2 lakh a month in storage alone.

Why writes must be buffered, and asynchronous

```
an fsync to disk:           ~5 ms
a buffered write to a file: ~1 µs
a network write to a collector: ~0.5 ms, and it can hang

at 40,000 lines/second:
fsync per line:      40,000 × 5 ms   = 200 seconds of I/O per second – impossible
buffered:           40,000 × 1 µs   = 40 ms/s – fine
network, synchronous: 40,000 × 0.5 ms = 20 seconds per second – impossible
```

Synchronous network logging is arithmetically impossible at this volume, which is why the queue is not an optimisation. And the queue's size is a time budget:

```
queue of 10,000 records at 40,000 records/second = 250 ms of buffer
a collector stalling for 1 second → 30,000 records dropped
```

Two hundred and fifty milliseconds of tolerance. That number is what you tune, and it is why drops must be counted: a one-second collector hiccup silently loses thirty thousand records unless somebody is watching the counter.

Finding one request

```
200,000 lines/second × 86,400 s = 17,280,000,000 lines/day
one request produces ~20 of them
```

Seventeen billion lines a day, and the twenty you want are scattered through it. Grep is not a plan; the correlation id is. With it, “show me this request” is an indexed lookup. Without it, it is impossible — and this is why the correlation id is the highest-value field in the record.

Structured versus text, for the same query

```
"failed Razorpay payments over ₹10,000 last Tuesday"

free text:  regex over ~700 GB of that day's INFO lines
structured: a query on indexed fields – event, provider, amount
```

The storage cost of JSON is roughly 30–50 percent more bytes per line than plain text. **That is the trade: about 40 percent more disk to make the data queryable at all.**

Sampling, priced

```
one very noisy event: cache_hit, 100,000/second
unsampled:           100,000 × 200 B = 20 MB/s = 1.7 TB/day
sampled at 1%:       1,000 × 200 B = 0.2 MB/s = 17 GB/day
```


A hundredfold reduction on one event name, with the statistics preserved as long as you record the sampling rate in the line — so a consumer can multiply back up. And errors are never sampled.

7 The trade-offs

What this design gives up

Asynchronous logging loses records exactly when you need them. A crash discards whatever is in the queue, and that is the last few hundred milliseconds before the failure — the most valuable logs in the system. Flushing on shutdown handles a graceful exit and does nothing for a segmentation fault or an OOM kill. Writing CRITICAL synchronously is the usual compromise: the volume is tiny and the value is enormous.

A bounded queue drops, and dropping is invisible unless you make it visible. A counter and an alert are not optional. The alternative — an unbounded queue — converts a slow collector into an out-of-memory kill, which is strictly worse.

Deferred formatting makes the API worse. `logger.info("order %s", id)` is uglier than an f-string and easier to get wrong — a mismatched placeholder count is a runtime error inside the logger, in the error path, which is the worst possible place. Structured logging with keyword fields is nicer, and the reason the %-style survives is that it is what defers the work. A linter rule catching f-strings in log calls is worth more than any amount of documentation.

A logger must never raise, which means it swallows its own errors. A full disk, an unserialisable value, a broken network handler — none of them may propagate into business code. So logging failures are themselves invisible unless there is an internal error counter and a fallback destination. “The logs stopped” is a genuinely common incident with no log line explaining it.

The hierarchy is global mutable state. Loggers are registered in a process-wide manager, so tests interfere with each other and a library that configures the root logger breaks its host application. The rule — **libraries add handlers to nothing and configure nothing; applications configure the root** — exists because that mistake is so common.

Structured logging costs discipline. The moment somebody writes `logger.info(f"paid {amount}")` instead of an event name and fields, that line is invisible to every dashboard. Consistency has to be enforced by review or by a linter, because the framework cannot tell.

“I would change this design if...”

- **...records must never be lost.** Then write synchronously to a local file and let a separate agent ship it — the file is the durable buffer, and the process never waits on a network.
- **...volume exceeds what a queue can absorb.** Then sample at the source, per event name, and record the rate in the line so consumers can scale it back up.
- **...several services must be correlated.** Then the correlation id becomes a propagated trace context — OpenTelemetry — and logging becomes one signal among traces and metrics rather than the only one.
- **...the destination is genuinely fast and local.** Then skip the queue: a buffered file write is a microsecond, and a queue adds a thread, a hand-off and a loss mode for no benefit.

The honest concession

The part the prompt asks about — adding a destination — is the easy part, and it is one interface with one method. The parts that decide whether the library is any good are the level check coming first, the correlation id, and the queue’s drop policy. Those are all *performance and operations* decisions rather than object-oriented ones, and a design answer that produces a beautiful class diagram and never mentions that a suppressed f-string can burn eight cores has answered a different question from the one that matters in production.

8 In the interview

How it gets asked

- The prompt: *“Design a logging library. How would I add a new destination?”*
- The performance probe, which is the real question: *“What does a `debug` call cost when debug logging is off?”*
- The operational probe: *“Logging to a network collector is slowing down our requests. What do you do?”*
- The scale probe: *“How do you find all the log lines for one request among a billion?”*
- The API probe: *“Why does Python’s logging use `%s` and arguments instead of f-strings?”*

The timed script

Minutes 0–5 · Clarify. A library or one service’s logging? Structured or text? What volume? Must any line be guaranteed to survive a crash?

Minutes 5–12 · The four pieces, with `Handler` and `Formatter` kept separate and the $N \times M$ argument. Answer the prompt’s question directly and early: adding a destination is one class, one method, one line of wiring.

Minutes 12–20 · The hot path, and this is where to spend the time. The level check first, deferred formatting, and the arithmetic — eight cores burned on suppressed f-strings with expensive arguments.

Minutes 20–28 · The hierarchy and propagation, and what it buys: turning one subsystem up without touching anything else.

Minutes 28–34 · Structured logging and the correlation id, with the seventeen-billion-lines number.

Minutes 34–40 · Asynchronous emission, the drop policy, the loss-on-crash trade, and CRITICAL written synchronously.

The follow-ups

“How do I add a new destination?” “One class implementing `Handler`, with a single `emit` method, plus one line of configuration attaching it to a logger. Nothing else changes — and it is one class rather than several because `Handler` and `Formatter` are deliberately separate interfaces: *where* it goes and *what it looks like* vary independently. If they were combined you would need a class per combination — `JsonFileHandler`, `TextFileHandler`, `JsonNetworkHandler` — so $N \times M$ classes instead of $N + M$. The handler also carries its own level, which is what lets a file capture DEBUG while the console shows only warnings from the same logger.”

“What does a `debug` call cost when debug is off?” “It should be one integer comparison and a return — about fifty nanoseconds — and the API is designed around making that possible. Which is why `logger.debug("user %s failed", user_id)` is right and `logger.debug(f"user {user_id} failed")` is wrong: arguments are evaluated before the call, so the f-string is built whether or not anybody wants it. With an expensive argument the difference is dramatic. At ten thousand requests a second with twenty log statements each, and eighty percent of them suppressed, an f-string that serialises an object at fifty microseconds costs **eight seconds of CPU per second** — eight cores producing strings that are immediately discarded. The deferred version costs about eight milliseconds a second.”

“Logging to a network collector is slowing down our requests.” “Put a bounded queue between the logger and the handler: the caller appends and returns, and a background thread drains it. Synchronous network logging is arithmetically impossible anyway — at forty thousand lines a second and half a millisecond per write, that is twenty seconds of I/O per second. The queue introduces two costs I would name explicitly. It must **drop** when full rather than block, because blocking reintroduces exactly the coup-

ling it exists to remove — and the drops must be counted and alerted on, or you silently lose thirty thousand records every time the collector hiccups for a second. And anything still queued when the process dies is lost, which is precisely the moment you want it. So: flush on shutdown, and write CRITICAL synchronously, because the volume is tiny and the value is enormous.”

“How do you find all the lines for one request?” “A correlation id on every record, generated at the edge, stored in a `ContextVar`, and attached automatically by the framework rather than passed through every function signature. The arithmetic makes the case: two hundred thousand lines a second is seventeen billion a day, and one request produces about twenty of them, interleaved with everything else. With the id it is an indexed lookup; without it, it is not a hard problem, it is an impossible one. I would use a `ContextVar` rather than a thread-local so it works with async code, and across services the id becomes a propagated trace context.”

“Why structured logging?” “Because a sentence is readable by a person and useless to a machine. ‘Failed Razorpay payments over ten thousand rupees last Tuesday’ is a regular expression over hundreds of gigabytes if the lines are prose, and an indexed query if they are records with an event name and typed fields. It costs about thirty to fifty percent more bytes per line for the JSON, which is the trade: forty percent more disk to make the data queryable at all. It also decouples the log text from the dashboards — you can reword a message without breaking every alert built on it, because the alert matches an event name.”

“What happens if the logging itself fails?” “It must not propagate. A full disk or an unserialisable value cannot be allowed to throw an exception inside unrelated business code, so every handler wraps its emit in a `try` and the JSON formatter falls back to `repr` rather than raising. The consequence, and it is a real one, is that logging failures are themselves invisible — so there has to be an internal error counter and ideally a fall-back destination. ‘The logs stopped’ is a common incident with, by construction, no log line explaining it.”

“Any security concerns?” “Two. First, never interpolate user data as a *template* — that is Log4Shell: Log4j evaluated `${...}` expressions inside log messages, so logging a user-supplied string could execute code. User data is a value, never a format string. Second, logs are where secrets leak: a request body dumped at DEBUG contains passwords and card numbers, and it is now in a system with much weaker access control than the database. I would add a redaction filter for known sensitive field names and treat the log store as production data, not as a convenience.”

A model answer

Asked: *design a logging library. How would I add a new destination?*

“Let me answer the direct question first and then spend the time on the part I think decides whether the library is any good.

Adding a destination is one class implementing a `Handler` interface with a single `emit` method, plus one line of configuration attaching it to a logger. That is deliberately small, and it is small because `Handler` and `Formatter` are two separate interfaces — *where* a record goes and *what it looks like* vary independently, so you can send JSON to a file and plain text to the console. If they were one interface you would need a class per combination, $N \times M$ instead of $N + M$.

Now the part that matters more. There are four pieces: a `Logger` that code calls, a `LogRecord` describing one event, handlers, and formatters. And the single most important design decision is that **the level check is the first statement in the logging call**, before anything is built.

The reason is that most logging calls produce nothing. At ten thousand requests a second with twenty log statements each, that is two hundred thousand calls a second and perhaps eighty percent of them are debug and switched off. A suppressed call should cost one integer comparison — about fifty nanoseconds.

This is why the API takes a message *template* and arguments rather than a finished string. Arguments are evaluated before the call, so an f-string is built whether or not anybody wants it. If that f-string serialises an object — say fifty microseconds — then a hundred and sixty thousand suppressed calls a second is **eight seconds of CPU per second**. Eight cores, producing strings that are immediately discarded. That number is the whole argument for the awkward `%s` API, and it is why linters warn about f-strings in log calls.

Loggers are hierarchical and dotted — `app.payments.stripe` — and a record propagates upward through its ancestors, each of which has its own handlers and its own level. That is what makes logging usable in production: you can turn `app.payments` up to `DEBUG` and leave everything else at `INFO`, in one line of configuration, with no code change. It is structurally a chain of responsibility with one difference — every ancestor gets the record, not just the first — and `propagate = False` cuts it.

Two more things I would insist on. **Structured records rather than sentences**, because at seventeen billion lines a day ‘failed Razorpay payments over ten thousand rupees’ has to be a query, not a regular expression. And a **correlation id** on every record, from a context variable, attached by the framework — one request produces twenty lines scattered through two hundred thousand a second, and without a shared id, reconstructing it is impossible rather than merely hard.

Finally, emission goes through a bounded queue to a background thread, because synchronous network logging at forty thousand lines a second would need twenty seconds of I/O per second. The queue drops rather than blocks when full — blocking would reintroduce the coupling it exists to remove — and the drops are counted and alerted on. The honest cost is that a crash loses whatever is queued, which is exactly the logs you want, so CRITICAL is written synchronously and everything is flushed on shutdown.”

9 Recall card

- `Handler` (WHERE) and `Formatter` (WHAT IT LOOKS LIKE) are separate interfaces, and that is the answer to the prompt: adding a destination is **one class, one method, one line of wiring**. Combined, you would need a class per combination — $N \times M$ instead of $N + M$. Each handler carries its **own level**, so a file can capture DEBUG while the console shows only warnings.
- The level check must be the **FIRST** statement, and the API exists to make **deferred formatting possible**. `logger.debug("user %s", uid)` defers; `logger.debug(f"user {uid}")` **always builds the string**. At 200,000 calls/s with 80% suppressed and an expensive argument, that is **8 seconds of CPU per second — eight cores — discarded**, against ~ 8 ms/s done properly.
- Dotted hierarchical loggers with **upward propagation** let you **turn one subsystem up in one line of config**. Structurally a chain of responsibility, but **every ancestor gets the record**, not just the first — `propagate = False` cuts it.
- Log events, not sentences, and carry a **correlation id**. 200,000 lines/s is **17 billion a day**, and one request's 20 lines are scattered through it — with the id it is an indexed lookup, without it it is impossible. Use a `ContextVar` (async-safe), attached by the framework, never threaded through signatures. JSON costs $\sim 40\%$ more bytes and makes the data queryable at all.
- **Emit through a BOUNDED queue on a background thread — synchronous network logging is arithmetically impossible** ($40,000 \text{ lines/s} \times 0.5 \text{ ms} = 20 \text{ s}$ of I/O per second). It must **drop, not block**, and **count the drops** (a 1 s collector stall loses $\sim 30,000$ records). A crash loses the queue — the logs you most want — so **flush on shutdown and write CRITICAL synchronously**. And a logger must **never raise**: wrap every emit, fall back to `repr`, and **never interpret user data as a template** (Log4Shell).

DSA topic: Subsets: the include-or-exclude tree **System design topic:** Design a logging framework

Code these, in this order

One rule for the whole set: say “copy” out loud as you type `current[:]`. The single most common backtracking bug appends the working list itself, and the result is uniformly wrong with no error at all.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Subsets	LeetCode 78 (Medium)	The tree, choose-recurse-un-choose, and the copy.
2	Subsets II	LeetCode 90 (Medium)	Sorting first, and <code>i > start</code> rather than <code>i > 0</code> .
3	Combinations	LeetCode 77 (Medium)	The same tree with a cap, and the first real pruning.
4	Letter Combinations of a Phone Number	LeetCode 17 (Medium)	The same shape where the branching factor is 3 or 4 rather than 2.

On problem 1, run the broken version first

Write it with `result.append(current)` and print the output. Eight empty lists. Then fix it and say, in one sentence, why every entry was the same object.

On problem 2, test `[2, 2]` before anything else

The right answer has three subsets. Write the version with `i > 0` and confirm it gives two. Then fix the condition and say what `i > start` means in terms of the tree.

On problem 3, add the prune and measure it

Write it without the “not enough elements left” check, then with it. Count the calls both ways for choosing 8 from 20 and write both numbers down.

On problem 4, notice what changed

The branching factor is no longer two. Say what stayed exactly the same, and what the complexity became in terms of the digits.

The size drill

1. Say why there are exactly 2^n subsets, in one sentence about decisions.
2. Compute the count for $n = 10, 20, 25$ and 30 .
3. Say why no algorithm can be faster, and what that sentence prevents in an interview.
4. Given a problem that says `n ≤ 20`, say what that constraint is telling you.

The pattern drill

1. Write the three lines of choose-recurse-un-choose from memory.
2. Say what the `pop` is for, and what happens if you leave it out — including whether it raises.
3. Draw the decision tree for `[1, 2, 3]` and label every leaf.
4. Count the leaves, the nodes and the depth.
5. Say which of those three is the time and which is the space.

The copy drill

1. Write the version that appends the list instead of a copy.
2. Print the result for `[1, 2, 3]` and describe exactly what you see.
3. Explain why every entry is empty at the end.
4. Name three ways to make the copy.
5. Say why this bug produces no error.

The two-arrangements drill

1. Write the version where only the leaves are subsets.
2. Write the version where every node is a subset.
3. Confirm they produce the same set of answers.
4. Say which one generalises, and give two problems it generalises to.

The other-formulations drill

1. Write the iterative doubling version in two lines.
2. Say what happens if you seed it with `[]` instead of `[[[]]]`.
3. Write the bitmask version.
4. Say what `mask = 5` means for a three-element array.
5. State the limit of the bitmask approach and say why it never matters here.
6. Say which version you would write in an interview, and why it is not the shortest one.

The duplicates drill

1. Sort, then write the skip with `i > start`.

2. Run it on `[1, 2, 2]` and count the distinct subsets against the number of decision paths.
3. Change the condition to `i > 0` and run `[2, 2]`. Say which subset disappeared and why.
4. Remove the sort and run `[2, 1, 2]`. Say why nothing gets skipped.
5. Write the filter-at-the-end version and say what it costs on `[2] * 20`.

The break-it drill

Trigger each and record the exact output:

1. `result.append(current)` without the copy.
2. Omit the `pop`.
3. `i > 0` instead of `i > start`, on `[2, 2]`.
4. Forget to sort before de-duplicating.
5. Seed the iterative version with `[]`.
6. Compare your recursive output against the iterative one without sorting either. Explain the mismatch.

The generalisation drill

For each, say what changes from the subsets template and what stays the same:

1. Combinations of size k.
2. Permutations.
3. Combination sum, where numbers may be reused.
4. N-Queens.
5. Word search on a grid.

Name the one thing that all five share, and the one thing that makes the last two “back-tracking” rather than “enumeration”.

The two-interfaces drill

1. Name the two interfaces and what each one decides.
2. Say how many classes you need for 4 destinations and 3 formats, kept separate and combined.
3. Write the `Handler` interface and one implementation.
4. Say what else a handler carries besides the formatter, and what that enables.
5. Answer the prompt — “how do I add a destination?” — in one sentence.

The hot-path drill

1. Write the `log` method with the level check in the right place.
2. Say why the check must be the first statement.
3. Write the same log call two ways — f-string and deferred — and say what executes in each.
4. Compute the CPU cost per second for 200,000 calls at 80 percent suppressed, for a cheap f-string and an expensive one.
5. Say what the API's shape has to be for deferral to be possible at all.
6. Write the guard for a genuinely expensive argument.

The hierarchy drill

1. Draw four loggers in a tree.
2. Say what “effective level” means and how it is computed.
3. Describe what happens to a record logged at the deepest node.
4. Say how this differs from a plain chain of responsibility.
5. Say what one line of configuration you would write to debug one subsystem.
6. Say what `propagate = False` does and when you would use it.

The structured drill

1. Write the same event as a sentence and as a record.
2. Write the query “failed Razorpay payments over ₹10,000 last Tuesday” against each.
3. State the storage cost difference as a percentage.
4. Say what structured logging decouples, beyond queryability.
5. Say what the highest-value field in any log line is and why.

The async drill

1. Compute the I/O time per second for synchronous network writes at 40,000 lines a second.
2. Write the queue handler with a bounded queue.
3. Say why it must drop rather than block.
4. Say what must accompany every drop.
5. Compute the buffer in milliseconds for a 10,000-record queue at 40,000 records a second.
6. Say what is lost on a crash and name the two mitigations.

The volume drill

Compute each, showing the multiplication:

1. Log calls per second at 10,000 req/s and 20 statements each.
2. Bytes and terabytes per day at DEBUG, INFO and WARNING.
3. Lines per day, and how many belong to one request.
4. Storage saved by sampling one noisy event at 1 percent.
5. The rupee cost per month of the DEBUG option at ₹2 per GB.

The failure drill

For each, say what happens and what you would add:

1. The disk fills.
2. A value cannot be serialised to JSON.
3. The network collector hangs for one second.
4. The process is killed by the OOM killer.
5. A library configures the root logger.
6. A user-supplied string containing `${ ... }` is logged.
7. A request body is logged at DEBUG in production.

Two of the seven are security problems rather than reliability problems. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Generate all subsets of this array.* The 2^n size stated first and why nothing can beat it, the decision tree described before any code, choose-recurse-un-choose with what the pop is for, the copy flagged before you write it, both complexities kept separate, and the iterative alternative offered with a reason for choosing the recursive one.
 2. *Design a logging library. How do I add a new destination?* One class and one line, because `Handler` and `Formatter` are separate — then the part that matters: the level check first with the eight-cores arithmetic, the hierarchy and what it buys, structured records with a correlation id and the seventeen-billion number, and the bounded queue with its drop policy and crash loss.
 3. *What does a suppressed `debug` call cost?* One integer comparison if the API allows deferral, and eight seconds of CPU per second if it does not — with the reason arguments are evaluated before the call.
-

Before you move on

- [] I can say why there are 2^n subsets and why nothing can be faster.
- [] I can write choose-recurse-un-choose without thinking, and say what the pop is for.
- [] I ran the no-copy version and can explain the eight empty lists.
- [] I can draw the decision tree and say which count is time and which is space.
- [] I wrote all three formulations and know which one generalises.
- [] I can state what happens if the iterative version is seeded with `[]`.
- [] I can write the duplicate skip and explain `i > start` versus `i > 0`.
- [] I tested `[2, 2]` and know which subset the wrong condition loses.
- [] I can say why pruning beats filtering, with the `[2] * 20` example.
- [] I can name what changes for combinations, permutations and N-Queens.
- [] I can answer “how do I add a destination” in one sentence.
- [] I can say why `Handler` and `Formatter` are separate, with the $N \times M$ argument.
- [] I can compute the cost of suppressed f-strings at 200,000 calls a second.
- [] I can explain why the API takes a template and arguments.
- [] I can describe the hierarchy, effective level and propagation.
- [] I can state the correlation id’s value with the lines-per-day number.
- [] I can defend dropping over blocking, and say what must accompany a drop.
- [] I answered all three questions above out loud.

Permutations

DATA STRUCTURES AND ALGORITHMS

Permutations

You can generate every ordering, with and without duplicates.

SYSTEM DESIGN

Design a notification service

You can model channels, templates, user preferences and retries.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Generate all permutations. Now handle duplicate values.*
- *Design a notification service that supports email, SMS and push.*

1 What this is, and why they ask it

Given $[1, 2, 3]$, produce every ordering: $[1, 2, 3]$, $[1, 3, 2]$, $[2, 1, 3]$, $[2, 3, 1]$, $[3, 1, 2]$, $[3, 2, 1]$. Six of them, and six is not a coincidence — it is $3 \times 2 \times 1$.

A **permutation** is an arrangement of all the elements in some order. Nothing is left out and nothing is repeated; only the order changes. That is the difference from yesterday: a subset asks *which elements*, a permutation asks *in what order*.

The shape is the lesson. **At the first position you have n choices. Once you commit to one, the second position has $n - 1$ choices left, then $n - 2$, and so on.** Multiply them and you get $n \times (n-1) \times \dots \times 1$, which is written $n!$ and said “ n factorial”. Three elements, six orderings. Four elements, twenty-four.

They ask it because it is [yesterday's tree](#) with one thing changed — the branching factor is no longer two, and it shrinks as you descend — and because that change breaks the `start` index that made subsets work. You now have to track *which elements are still available*, and there are two completely different ways to do that. Then they say “now the input has duplicates”, and the fix that worked for subsets is wrong here, for a reason worth understanding.

2 The story

The photo happened at the end, the way it always does, when everyone had eaten and the hall was half empty.

Ravi's four children were being lined up in front of the stage. Anu, Bharat, Chitra and Deepak. The photographer, a thin man in a blue shirt who had been there since seven that morning, said “stand in a line”, and they did, and then the trouble started.

Anu wanted to be on the end. Bharat said he was always put on the end. Chitra said that last time she had been standing directly behind Deepak and you could only see her shoulder. Deepak said nothing and walked to the middle.

The photographer had clearly seen this before. He lowered the camera and said: fine. We will do all of them.

Ravi laughed and asked how many that was.

The man worked it out standing there, holding the camera against his chest. He said, look. The first spot in the line can be any one of the four. Once that is settled, the second spot can be any of the three who are left. Then two. Then one. Four times three times two times one. Twenty-four.

Then he actually did it. He put Anu on the far left and made the other three run through their six orders. Then Bharat on the far left, and the same six behind him. Then Chitra. Then Deepak.

He was careful about one thing, and he said so out loud while he worked. Each time he pulled a child out of the line and moved them to the front, he put them back in exactly the spot they had come from before he pulled the next one out. The line always looked the same as it had a moment ago.

He said if you do not put them back, you lose track of who is still waiting. You take the same photo twice and you miss another one completely, and you do not notice, because twenty-four photos of four children all look the same afterwards.

It took eleven minutes. Nobody complained, because everybody had been on the end of the line once.

3 The idea in plain English

The four children are the elements. A finished line is one permutation. And putting each child back where they came from is the un-choose step — the same one from [day 091](#), now doing visible work.

Why $n!$

At the first position, any of the n elements may go. Having placed one, $n - 1$ remain for the second position, then $n - 2$, and so on until one element is left with nowhere else to go.

$n = 3$	$\rightarrow 3 \times 2 \times 1$	$= 6$
$n = 4$	$\rightarrow 4 \times 3 \times 2 \times 1$	$= 24$
$n = 5$	$\rightarrow 5 \times 4 \times 3 \times 2 \times 1$	$= 120$
$n = 10$	$\rightarrow 3,628,800$	
$n = 12$	$\rightarrow 479,001,600$	— about half a billion; minutes
$n = 13$	$\rightarrow 6,227,020,800$	— not going to finish

$n!$ grows faster than 2^n . At $n = 20$, subsets is a million and permutations is 2.4×10^{18} . A permutations problem always has a tiny n . LeetCode 46 bounds it at 6. When you see $n \leq 8$, the constraint is telling you the answer is factorial.

And as with subsets, **you cannot beat $n!$** , because the output is $n!$ things. Say that first.

Why `start` no longer works

Yesterday's template walked `for i in range(start, len(items))`, and `start` meant "do not look backwards". That was correct for subsets because `[1, 2]` and `[2, 1]` are the *same* subset, so looking backwards would only produce repeats.

For permutations, `[1, 2]` and `[2, 1]` are **different answers**. You must look backwards. So `start` is gone, and something has to replace it — something that says *which elements are already in the line* rather than *how far along the list you are*.

There are two standard replacements, and interviewers ask about both.

Replacement one: a `used` list

Keep one boolean per element. `used[i]` is `True` when element `i` is already standing in the line.

```
for i in range(len(items)):
    if used[i]:
        continue                # already in the line; skip
    used[i] = True               # choose
    current.append(items[i])
    build()                     # recurse
    current.pop()               # un-choose
    used[i] = False             # un-choose
```

Two things are chosen and two things must be un-chosen. That symmetry is the whole discipline of backtracking. Miss the `used[i] = False` and the element is marked as used forever, so every branch after the first is missing elements and you get far fewer than $n!$ answers.

This is the version to write in an interview. It is obvious, it handles duplicates cleanly, and the `used` list costs n booleans.

Replacement two: swapping in place

The photographer's actual method. Do not build a separate line — rearrange the one you already have.


```
def build(first):
    if first == len(items):
        result.append(items[:])
        return
    for i in range(first, len(items)):
        items[first], items[i] = items[i], items[first]      # choose
        build(first + 1)
        items[first], items[i] = items[i], items[first]      # un-choose
```

At depth `first`, everything before `first` is already fixed and everything from `first` onward is still available. Swapping element `i` into position `first` says “this one goes here next”. The second swap puts it back — the child returning to exactly the spot they came from.

No extra list, no `used` array. It is what people mean by “the in-place version”. It also has a defect that matters, and it is the third trap below: **it cannot handle duplicates**.

Duplicates: `[1, 1, 2]`

Three elements, so $3! = 6$ decision paths — but only **three distinct orderings**: `[1,1,2]`, `[1,2,1]`, `[2,1,1]`. Each one is produced twice, because the two `1`s can swap places without changing anything you can see.

The fix looks like yesterday’s and is not.

```
items.sort()                                # 1. equal elements become adjacent
...
    if used[i]:
        continue
    if i > 0 and items[i] == items[i - 1] and not used[i - 1]:
        continue                            # 2. the previous equal one is NOT placed
```

Read the second condition slowly, because it is the hardest single line in the phase.

`items[i] == items[i - 1]` means “this element is interchangeable with the one before it”. `not used[i - 1]` means “the one before it is **not** currently in the line”.

Put together: *I am about to place a duplicate whose identical twin is still sitting unused*. If I do that, I am starting a branch identical to the branch I already started when I placed the twin here. So skip it.

The rule that comes out of it: among identical elements, always place them left to right. The second `1` may only be used if the first `1` has already been used. That forces exactly one representative of each group of identical arrangements.

Yesterday’s condition was `i > start`. Today’s is `not used[i - 1]`. They are doing the same job — “do not re-explore a branch already explored at this level” — in two different tree shapes.

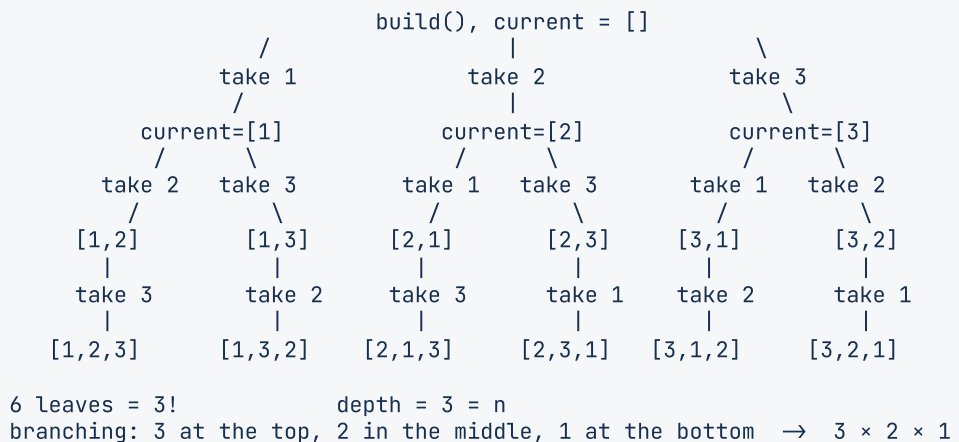
The library call

```
from itertools import permutations
list(permutations([1, 2, 3]))
# [(1,2,3), (1,3,2), (2,1,3), (2,3,1), (3,1,2), (3,2,1)]
```

Tuples, not lists, and it does **not** de-duplicate: `permutations([1,1,2])` gives six tuples with repeats. Know it exists, mention it, then write the recursive one, because the interviewer is asking about the tree and not about the standard library.

4 The picture

The tree for `[1, 2, 3]`. Notice the branching factor shrinking as you go down — three, then two, then one. That shrinking is the difference from yesterday.



What to notice: **subsets branched two ways at every level; permutations branch `n` ways at the top and one way at the bottom**. Same depth, different width, and the width is what makes `n!` bigger than `2n`.

The swap version, traced. The bar marks the boundary: fixed on the left, still available on the right.

```

items = [1, 2, 3]

first=0   [ | 1 2 3 ]      three choices: swap position 0 with 0, 1, or 2

    swap(0,0) [ 1 | 2 3 ] → first=1
        swap(1,1) [ 1 2 | 3 ] → [1,2,3] LEAF
        swap(1,2) [ 1 3 | 2 ] → [1,3,2] LEAF
        swap(1,2) [ 1 2 3 ] undo, back to how it was
    swap(0,0) [ 1 2 3 ] undo

    swap(0,1) [ 2 | 1 3 ] → first=1
        ... [2,1,3], [2,3,1]
    swap(0,1) [ 1 2 3 ] undo – the line is exactly as it started

    swap(0,2) [ 3 | 2 1 ] → [3,2,1], [3,1,2]
    swap(0,2) [ 1 2 3 ] undo

```

The array is back to [1, 2, 3] at the end. Every child returned to their spot.

And the duplicate rule, which is the fiddly part:

```

items = [1a, 1b, 2] (the letters are only for us; the code cannot tell them apart)

at the top level, position 0:
    place 1a → used = [T, F, F] allowed
    place 1b → items[1]=items[0] and used[0] is F SKIP
               (this branch would be identical to the 1a branch)
    place 2 → different value allowed

one level down, having placed 1a:
    place 1b → items[1]=items[0] and used[0] is T ALLOWED
               (the twin is already in the line, so this is
                "the second 1 following the first", not a repeat branch)

rule in one sentence: among equal elements, use them LEFT TO RIGHT.

```

5 The code, built step by step

Step 1 – say the size before writing anything

“There are $n!$ permutations, so the output alone is factorial and no algorithm can be faster. At $n = 10$ that is 3.6 million, which is fine. At $n = 13$ it is 6 billion, which is not. So I am expecting a small n .”

Thirty seconds, and it removes “can you do better?” from the conversation.

Step 2 – say why yesterday’s `start` is gone

```

for i in range(start, len(items)):      # subsets: never look backwards
for i in range(len(items)):             # permutations: look everywhere

```

“For subsets, `[1,2]` and `[2,1]` are the same answer, so I only look forward. For permutations they are different answers, so I have to look at every element every time — which means I need to know which ones are already placed.”

That sentence is the whole design decision, and saying it out loud is worth more than the code.

Step 3 — the `used` list, and the two un-chooses

```
used[i] = True
current.append(items[i])
build()
current.pop()
used[i] = False
```

Five lines: choose twice, recurse, un-choose twice. **Write them as one block, always.** The two un-chooses mirror the two chooses in reverse order. It is a habit rather than a rule, and the habit is what stops you forgetting one under pressure.

Step 4 — the base case, and the copy

```
if len(current) == len(items):
    result.append(current[:])    # COPY. Say the word as you type it.
    return
```

Same copy trap as [day 091](#), same silent failure. `current` is one list mutated all the way through; appending it stores a reference, and every reference points at the same list, which is empty by the end.

The base case here is `len(current) == len(items)` rather than an index reaching the end, because there is no index walking to the end — you are not traversing the input in order.

Step 5 — the swap version, if they ask for less space

```
for i in range(first, len(items)):
    items[first], items[i] = items[i], items[first]
    build(first + 1)
    items[first], items[i] = items[i], items[first]
```

The loop starts at `first`, and the first iteration swaps a position with itself — that is the “leave this one where it is” branch, and it is not a wasted step.

The second swap is mandatory. Without it the input is scrambled for the next iteration and you produce a mixture of wrong orderings, silently.

Step 6 — duplicates, the hardest line in the phase

```
if i > 0 and items[i] == items[i - 1] and not used[i - 1]:  
    continue
```

With `items.sort()` first. Say the meaning out loud rather than memorising the symbols: **“skip a duplicate whose identical twin has not been placed yet.”** Equivalently: among equal elements, use them left to right.

The complete solution

```
import math
from itertools import permutations as itertools_permutations

def permutations(items: list[int]) → list[list[int]]:
    """Every ordering, via a used-list. The version to write in an interview.

    n! permutations, so the output alone is factorial – nothing can beat it.
    Branching is n at the top, n-1 next, down to 1: n × (n-1) × ... × 1.

    TWO things are chosen and TWO must be un-chosen. That symmetry is the
    discipline of the whole phase.
    """
    result: list[list[int]] = []
    current: list[int] = []
    used = [False] * len(items)

    def build() → None:
        if len(current) == len(items):
            result.append(current[:])      # COPY – current is one mutating list
            return
        for i in range(len(items)):
            if used[i]:
                continue
            used[i] = True                 # choose
            current.append(items[i])       # choose
            build()                        # recurse
            current.pop()                  # un-choose
            used[i] = False                 # un-choose

    build()
    return result

def permutations_swap(items: list[int]) → list[list[int]]:
    """The in-place version: no used-list, no working list, O(n) stack only.

    Everything before `first` is fixed; everything from `first` on is still
    available. Swapping i into position `first` says "this one goes next".
    The second swap puts it back, exactly where it came from.

    Cannot handle duplicates – see permutations_unique for why.
    """
    result: list[list[int]] = []

    def build(first: int) → None:
        if first == len(items):
            result.append(items[:])        # COPY of the array as it stands
            return
        for i in range(first, len(items)):
            items[first], items[i] = items[i], items[first]  # choose
            build(first + 1)                    # recurse
            items[first], items[i] = items[i], items[first]  # un-choose

    build(0)
    return result

def permutations_unique(items: list[int]) → list[list[int]]:
    """Permutations II. Two changes from the plain version.

    1. SORT, so equal elements are adjacent.
    2. Skip a duplicate whose identical twin has NOT been placed yet:
       `not used[i - 1]`. Among equal elements, use them LEFT TO RIGHT.

    This is NOT yesterday's `i > start` condition. Different tree shape,
    same job: do not re-explore a branch already explored at this level.
```

```

"""
items = sorted(items)
result: list[list[int]] = []
current: list[int] = []
used = [False] * len(items)

def build() → None:
    if len(current) == len(items):
        result.append(current[:])
        return
    for i in range(len(items)):
        if used[i]:
            continue
        if i > 0 and items[i] == items[i - 1] and not used[i - 1]:
            continue # twin still unused: repeat branch
        used[i] = True
        current.append(items[i])
        build()
        current.pop()
        used[i] = False

build()
return result

def permutations_of_length(items: list[int], k: int) → list[list[int]]:
    """Orderings of exactly k elements out of n - "partial permutations".

    Only the base case changes: stop at k rather than at n. The count is
     $n \times (n-1) \times \dots \times (n-k+1)$ , which is  $n! / (n-k)!$ .
    """
    result: list[list[int]] = []
    current: list[int] = []
    used = [False] * len(items)

    def build() → None:
        if len(current) == k:
            result.append(current[:])
            return
        for i in range(len(items)):
            if used[i]:
                continue
            used[i] = True
            current.append(items[i])
            build()
            current.pop()
            used[i] = False

    build()
    return result

def permutations_broken_no_unmark(items: list[int]) → list[list[int]]:
    """The trap, written out so you can run it: `used[i]` is never reset.
    Elements are consumed permanently, so every branch after the first
    dead-ends without ever reaching the base case."""
    result: list[list[int]] = []
    current: list[int] = []
    used = [False] * len(items)

    def build() → None:
        if len(current) == len(items):
            result.append(current[:])
            return
        for i in range(len(items)):
            if used[i]:
                continue
            used[i] = True
            current.append(items[i])
            build()

```

```

        current.pop()
        # missing: used[i] = False

    build()
    return result

if __name__ == "__main__":
    print(permutations([1, 2, 3]))
    # [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

    print(len(permutations([1, 2, 3, 4])), "=", 4 * 3 * 2 * 1)    # 24 = 24
    print(permutations([]), permutations([7]))                    # [[]] [[7]]

    print(sorted(permutations_swap([1, 2, 3])) == sorted(permutations([1, 2, 3]))) # True

    print(permutations_unique([1, 1, 2]))
    # [[1, 1, 2], [1, 2, 1], [2, 1, 1]]
    print(len(permutations_unique([1, 1, 2])), "distinct, vs", 3 * 2 * 1, "paths")
    # 3 distinct, vs 6 paths

    print(permutations_of_length([1, 2, 3], 2))
    # [[1, 2], [1, 3], [2, 1], [2, 3], [3, 1], [3, 2]]

    print(permutations_broken_no_unmark([1, 2, 3]))
    # [[1, 2, 3]] ← one answer instead of six

    print([list(t) for t in itertools.permutations([1, 1, 2])])
    # [[1, 1, 2], [1, 2, 1], [1, 1, 2], [1, 2, 1], [2, 1, 1], [2, 1, 1]]
    # ^ six tuples, three distinct: itertools does NOT de-duplicate

    for n in (8, 10, 12, 13):
        print(f"n = {n}: {math.factorial(n):,} permutations")

```

6 What it costs

Time

```

permutations produced:    n!
length of each:          n
copying them out:        n! × n
→ O(n × n!) total work

```

The $n!$ is the number of answers; the n is the cost of copying each one into the result. **You cannot beat $n!$ because the output is $n!$ things.**

There is a second n hiding in the `used` version — the loop scans all n elements at every node, even though most are already used. That does not change the big-O, because the copy already costs n , but it is worth being able to say why the two versions are the same complexity.

n = 8:	40,320 permutations,	~320,000 elements written	
n = 10:	3,628,800,	~36,000,000 elements	– a few seconds
n = 11:	39,916,800,	~440,000,000	– a minute or more
n = 12:	479,001,600		– minutes
n = 13:	6,227,020,800		– not going to finish

Ten is comfortable, eleven is painful, thirteen is impossible. Compare with subsets, where twenty was comfortable:

n	2^n (subsets)	n! (permutations)
5	32	120
10	1,024	3,628,800
15	32,768	1,307,674,368,000
20	1,048,576	2,432,902,008,176,640,000

At $n = 20$ subsets is a million and permutations is 2.4 quintillion. **$n!$ overtakes 2^n at $n = 4$ and never looks back.** That gap is why a permutations problem always has a smaller bound than a subsets problem, and noticing the bound is how you know which one is wanted.

Calls versus depth

tree nodes:	$1 + n + n(n-1) + \dots \approx e \times n!$	$n = 10 \rightarrow \sim 9.9$ million calls
depth:	n	$n = 10 \rightarrow 10$ frames

Ten million calls, ten stack frames. Time is the whole tree; space is the deepest path — the [day 088](#) distinction again, and the reason permutations never hits the recursion limit however slow it gets.

Space

output:	$n!$ lists of length n	$\rightarrow O(n \times n!)$ — unavoidable
current:	one list of at most n	$\rightarrow O(n)$
used:	n booleans	$\rightarrow O(n)$
stack:	n frames	$\rightarrow O(n)$

extra space beyond the output: $O(n)$

The extra space is $O(n)$, not $O(n!)$. Say the two separately. The swap version drops **current** and **used** and is $O(n)$ for the stack alone — genuinely less memory, and the honest reason to prefer it if someone asks for the in-place solution.

$n = 10$, integers: $3,628,800$ lists $\times \sim 136$ B ≈ 490 MB just for the result

Which is why real code that needs permutations *yields* them one at a time rather than building the whole list. If the interviewer says “for each permutation, check something”, say “I would make this a generator so we never hold **$n!$** lists in memory at once”.

Duplicates

```
[1, 1, 2]:      3! = 6 paths,  3 distinct
[1, 1, 1]:      6 paths,      1 distinct
[1, 1, 2, 2]:  4! = 24 paths,  6 distinct      (24 / (2! × 2!))
```

The general count is `n!` divided by the factorial of each group size. The skip prunes the repeated branches **before** walking them, which is the point: filtering a list of `n!` results afterwards costs the full `n!` in time and memory, and also requires the results to be hashable — `set(list_of_lists)` does not even run.

7 The traps

Trap 1 — appending the list instead of a copy

```
result.append(current)          # a REFERENCE to one mutating list
```

```
permutations([1, 2, 3]) → [[], [], [], [], [], []]
```

Six results, all the same object, all empty, no error. Identical to yesterday's trap and just as silent. `current[:]`.

In the swap version the same trap wears a different coat: `result.append(items)` appends the input list itself, and since the recursion restores it, every entry reads `[1, 2, 3]`.

Trap 2 — forgetting `used[i] = False`

```
used[i] = True
current.append(items[i])
build()
current.pop()

# missing: used[i] = False
```

```
permutations([1, 2, 3]) → [[1, 2, 3]]
```

One answer instead of six. Every element is consumed permanently, so after the first full line is built, nothing is available to anyone and every other branch dead-ends without reaching the base case.

Two chooses need two un-chooses. The `pop` on its own is not enough here, and this is the specific thing that changes from subsets.

Trap 3 — the swap version with duplicates

```
permutations_swap([1, 1, 2])  
# [[1, 1, 2], [1, 2, 1], [1, 1, 2], [1, 2, 1], [2, 1, 1], [2, 1, 1]]
```

Six answers, three distinct. And the fix that works for the `used` version **cannot be applied here**, because swapping destroys the sorted order — after one swap, equal elements are no longer adjacent, so “compare with the previous element” is meaningless.

The usual patch is a per-level `set` of values already tried:

```
seen = set()  
for i in range(first, len(items)):  
    if items[i] in seen:  
        continue  
    seen.add(items[i])
```

That works, and it costs a set per node. **If the input has duplicates, use the `used` version.** Say that out loud rather than trying to rescue the swap version under time pressure.

Trap 4 — using yesterday’s duplicate condition

```
if i > 0 and items[i] == items[i - 1]:  
    continue                                # subsets' rule, wrong here
```

Missing `and not used[i - 1]`. This skips the second `1` *always*, including when it should legitimately follow the first one inside a line. The branch that would place both `1`s can never complete, so the base case is never reached down that path.

Test it on `[1, 1]`. The right answer is `[[1, 1]]` — one permutation. The wrong condition gives `[]`.

Trap 5 — forgetting to sort first

```
# items = [2, 1, 1] - not sorted  
if i > 0 and items[i] == items[i - 1] and not used[i - 1]:
```

The rule compares **adjacent** elements, so equal values must be adjacent. Unsorted, the check finds nothing to skip and duplicates come out anyway. `sorted(items)` first, every time.

Trap 6 — trying to de-duplicate with a set

```
>>> set(permutations([1, 1, 2]))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

Lists are not hashable, so you cannot put them in a set. The workaround is `{tuple(p) for p in ...}`, which does run — and which builds all $n!$ results first. On `[1] * 11` that is forty million lines generated to return one. **Prune in the tree; never filter the result.**

Trap 7 — mutating the caller's list

The swap version rearranges the list it was given. It puts everything back by the end, so the caller usually never notices — but if the recursion raises part-way through, or you return early from inside it, the caller's list is left scrambled. If that matters, copy the input once at the top: `items = items[:]`.

Trap 8 — saying “2ⁿ” when they asked for permutations

Under pressure, people say “exponential” for both. $n!$ is not exponential; it is worse. Saying “2ⁿ” for permutations tells the interviewer you have pattern-matched rather than counted. **Count it out loud: n choices, then $n - 1$, then $n - 2$.**

8 In the interview

How it gets asked

- The base: “Generate all permutations of this array.” LeetCode 46.
- Immediately after: “Now the array can contain duplicates and the permutations must be unique.” LeetCode 47.
- The memory probe: “Can you do it without the extra `used` array?” — they want the swap version.
- The scale probe: “What if n is 12? What if n is 20?”
- The disguise: “Find all valid arrangements such that...” — Beautiful Arrangement, LeetCode 526, which is permutations with a check before recursing.

What to say out loud, in the first ninety seconds

1. **Count the size, do not name it.** “At the first position there are n choices, then $n - 1$, then $n - 2$ — so $n!$ permutations. The output alone is factorial, so nothing can beat it. $n = 10$ is 3.6 million and fine; $n = 13$ is 6 billion and not.”

2. **Say what changes from subsets.** “For subsets I walked forward with a `start` index because `[1, 2]` and `[2, 1]` are the same answer. Here they are different answers, so I have to consider every element at every position — which means tracking which ones are already placed.”
3. **Pick the representation and justify it.** “I will keep a boolean per element. There is also an in-place swap version that needs no extra array, and I will mention it, but the boolean version handles duplicates cleanly and that is usually the follow-up.”
4. **Say the symmetry.** “Choose, choose, recurse, un-choose, un-choose. Two things are marked and both have to be unmarked. Forgetting `used[i] = False` gives one permutation instead of `n!`, with no error.”
5. **Flag the copy before writing it.** “I append a copy when the line is complete, because `current` is one list being mutated throughout.”
6. **Give both complexities separately.** “ $O(n \times n!)$ time — `n!` answers, each costing `n` to copy — and $O(n)$ extra space beyond the output: one working list, `n` booleans, `n` stack frames.”

The follow-ups

“**What is the complexity?**” “ $O(n \times n!)$. There are `n!` permutations and writing each one out costs `n`. And no, you cannot do better, because the output itself is `n!` things — any correct solution has to produce all of them. Extra space beyond the output is $O(n)$: the working list, the `used` array and `n` stack frames. Worth separating, because at $n = 10$ the tree has roughly ten million calls and the stack never exceeds ten frames.”

“**Now handle duplicates.**” “Two changes. Sort the input so equal elements are adjacent. Then, before placing element `i`, skip it if it equals the previous element **and the previous element is not currently used**. The meaning is: among identical elements, always place them left to right. If I let the second `1` go first while the first `1` is still sitting unused, I am starting a branch identical to one I have already walked. Note this is *not* the `i > start` rule from subsets — different tree shape, same job. And it prunes rather than filters: filtering afterwards would build all `n!` results, and you cannot even put lists in a set, so you would have to convert to tuples first.”

“**Can you avoid the extra `used` array?**” “Yes — swap in place. At depth `first`, everything before `first` is fixed and everything from `first` on is available. Loop `i` from `first`, swap `i` into position `first`, recurse on `first + 1`, then swap back. The swap back is essential; without it the array is scrambled for the next iteration. That gets extra space down to just the $O(n)$ stack. The catch is duplicates: swapping destroys sorted order, so the adjacency rule stops working, and you need a per-level set of values already tried instead. If duplicates are in scope I would use the `used` version and say why.”

“What if n is 20?” “Then generating all permutations is not the question, because $20!$ is 2.4 quintillion — you would not finish this century. If someone asks about permutations at $n = 20$ they want something else: a *count*, which is arithmetic and needs no enumeration; or the *k-th* permutation in order, which you build one position at a time using factorials and never enumerate; or a *best* permutation under some criterion, which is usually bitmask dynamic programming — $2^n \times n$ states rather than $n!$, and at $n = 20$ that is about twenty million, which is fine.”

“What about permutations of a string with repeated characters?” “Same problem. Sort the characters, use the `used`-with-`not used[i-1]` rule, and join at the end. The count is $n!$ divided by the factorial of each character’s frequency — `'aab'` has $3!/2! = 3$.”

“Would you actually use your own implementation?” “In production, no — `itertools.permutations` is C and it is a generator, so it does not hold $n!$ results in memory. Two things to know about it: it yields tuples, not lists, and it does **not** deduplicate, so `permutations([1,1,2])` gives six tuples with repeats. In an interview I write the recursive one, because the question is about the tree.”

A model answer

Asked: *generate all permutations of this array.*

“Let me count the size first, because it decides everything. The first position can hold any of the n elements. Once that is fixed, the second position can hold any of the $n - 1$ that are left, then $n - 2$, and so on. So there are $n!$ permutations. That is not a bound I can improve on — **the output is $n!$ things**, so any correct solution has to write them all out. Practically that means $n = 10$ is 3.6 million and fine, and $n = 13$ is 6 billion and hopeless, so I am assuming a small n .

The important difference from subsets is this. For subsets I walked the array forward with a `start` index, because `[1,2]` and `[2,1]` are the *same* subset and looking backwards would only repeat myself. Here they are **different** answers, so I have to consider every element at every position. `start` is gone, and I need something else to tell me which elements are already in the arrangement.

I will keep one boolean per element. At each level I loop over all n elements, skip the ones already marked used, and for the rest: mark it used, append it, recurse, pop it, unmark it. **Two things are chosen and both have to be un-chosen** — that symmetry is the discipline of the whole pattern. If I forget the unmark, elements are consumed permanently and I get exactly one permutation instead of $n!$, with no error at all.

When the working list reaches full length, I record a **copy**. `current` is a single list being mutated throughout, so appending it directly would store $n!$ references to one list that is empty at the end.

Complexity is $O(n \times n!)$ time — $n!$ answers, n to copy each — and $O(n)$ extra space beyond the output: the working list, the booleans, and n stack frames. Worth separating those, because at $n = 10$ the tree has about ten million calls and the stack never goes past ten deep.

If you want it without the extra array, there is an in-place version: at depth `first`, swap each candidate into position `first`, recurse, and swap it back. It is the same tree, and the swap-back is the un-choose. I would still write the boolean version first, because if you then tell me the input has duplicates, the boolean version takes two clean lines — sort, and skip a duplicate whose identical twin is still unused — whereas the swap version breaks, because swapping destroys the sorted order that rule depends on.”

9 Recall card

- n choices, then $n - 1$, then $n - 2$ — so $n!$ permutations, and you cannot beat $n!$ because the output is $n!$ things. Count it out loud; never say “exponential”, because $n!$ is worse than 2^n and overtakes it at $n = 4$. $n = 10$ is 3.6 million and fine; $n = 13$ is 6 billion and impossible, so the bound in the question tells you which of the two is wanted.
- **start does not work here.** For subsets $[1,2]$ and $[2,1]$ are the same answer; for permutations they are different, so you look at every element every time and must track **which are already placed** — a `used` boolean per element.
- **Two chooses, two un-chooses:** `used[i] = True`, **append, recurse, pop**, `used[i] = False`. Forgetting the unmark gives **one permutation instead of $n!$** , silently. And `append current[:]` — a **copy** — or you get $n!$ references to one empty list.
- **Duplicates: sort, then skip when `items[i] == items[i-1]` AND `not used[i-1]`** — meaning “*my identical twin has not been placed yet, so this branch is a repeat*”. In one sentence: **among equal elements, place them left to right**. This is **not** subsets’ `i > start`. Prune, never filter — `set(list_of_lists)` raises `TypeError: unhashable type: 'list'`.
- **The swap version needs no `used` array:** fix positions left to right, swap `i` into `first`, recurse, **swap back**. $O(n)$ space total. But it **cannot use the adjacency rule for duplicates**, because swapping destroys sorted order — use a per-level `set`, or use the `used` version. And `itertools.permutations` exists, yields **tuples**, and does **not** de-duplicate.

1 What this is, and why they ask it

A **notification service** is the one piece of code in a company that every other team calls. Orders calls it when a parcel ships. Payments calls it when a card fails. Growth calls it when it wants to win somebody back. It takes “tell this user this thing” and turns it into an email, a text message, or a banner on their phone.

The three sentences that matter: it decides **which channels** a message goes out on, it renders **the same event into different wordings** for each channel, and it deals with the fact that **sending can fail** and the message still has to arrive.

They ask it because it is the low-level design prompt where almost every candidate gets the easy part right and the hard part wrong. The easy part is an interface with three implementations — email, SMS, push. Everybody writes that. The hard part is the four things underneath it: **user preferences** (when may you contact this person, and about what), **templates** (one event, several wordings), **retries** (the network failed; try again, but not forever and not immediately), and **de-duplication** (the caller retried, and the user must not get the same text twice). A candidate who only produces the interface has designed a small part of the problem and thinks they have finished.

It is also the prompt where the interviewer’s follow-up is guaranteed: “*now add WhatsApp*”. The whole design is judged on how many files that touches.

2 The story

The water tanker announcement was Suresh’s job that year, because he was the society secretary, and there were two hundred and four flats.

He got the message from the plumber at half past eight in the evening: no water tomorrow, from six in the morning until about two in the afternoon. Suresh sat down at the dining table with his phone and started.

He did not do the same thing for everybody, and that is the part that had taken him three years to work out.

Most of the flats were in the building group on his phone. One message there covered maybe a hundred and forty of them. Another forty or so were not in the group — new tenants, mostly — and they got the message sent to them directly, one at a time. And

then there were eleven people he had to actually ring, because they were old and did not look at their phone, and if he did not call them they would come down at seven in the morning with a bucket and shout at him.

He had the wording saved. He had typed it out once, months ago, and each time he only changed the two things that change: the date, and the hours. The rest of it — the apology, the line about the tanker coming, the line asking everybody to fill their buckets tonight — stayed exactly as it was. He was not going to write all that again from the beginning every time.

There were rules he had learned the hard way. Mrs D’Souza in 302 had told him plainly never to ring her after nine at night. The Nairs had asked to be taken off the group altogether and told only about water and electricity, nothing else. Flat 108 had two brothers in it, and only one of them wanted to hear from him.

Three of the calls did not connect. He did not give up on them, and he did not sit there pressing the button over and over either. He tried the first one again after ten minutes, and when it still rang out he tried once more before going to bed, and then he stopped.

At the end of it he went down his line of two hundred and four flats and could say, for every single one, whether they had been told, and how.

3 The idea in plain English

Suresh has built a notification service. Every part of it is in the story.

- The group message, the direct message and the phone call are **channels** — the different ways a message can physically reach a person. In software: **email**, **SMS** (a text message to a phone number), and **push** (a message that makes a phone buzz, delivered by Apple or Google to the app).
- The saved wording with the date and the hours swapped in is a **template** — a fixed piece of text with holes in it, filled in per message.
- “Never ring after nine”, “only water and electricity”, “only one of the two brothers” are **user preferences** — per-person rules about when you may contact them, on which channel, and about what.
- Trying again after ten minutes, then once more, then stopping, is a **retry policy** with a **backoff** (each wait longer than the last) and a **maximum attempt count**.
- Going down the line at the end and being able to say who was told is **delivery status tracking**.

The five decisions the service makes, in order

For every notification request, in this order:

1. **What happened?** An `event` — `ORDER_SHIPPED`, `PAYMENT_FAILED`, `PASSWORD_RESET`. Not a message. The caller says what happened, never what to send. That single rule is the most important line in the design.
2. **May we contact this user about this?** Look up preferences. If the user has muted this category, stop here. This is a **hard stop**, not a warning — in many countries it is the law.
3. **On which channels?** The event has defaults (`PASSWORD_RESET` → email; `ORDER_SHIPPED` → push and email), the user's preferences narrow them, and quiet hours may delay some of them.
4. **What exactly do we say?** Render the template for that event *and that channel*. The push version is forty characters; the email version has a subject line, a greeting and a footer. Same event, different text.
5. **Send it, and remember what happened.** Hand the rendered message to the channel, record the attempt, and if it failed with a retryable error, schedule another attempt.

The one rule that decides whether the design is good

Callers send events, never messages.

If the orders team calls `notify(user, "Your parcel has shipped!")`, then every wording change is a deploy of the orders service, adding a language means touching every caller, and the SMS version and the email version are two different strings in two different repositories. If they call `notify(user, ORDER_SHIPPED, {"tracking_id": "AB123"})`, the notification service owns every word, and nobody else has to know that WhatsApp exists.

Why “sending” is not one thing

A **request** to notify is not a **delivery**. Between them sit: the preference check, a queue, one or more provider calls, and possibly several attempts. So a notification has a **status** that moves through states, and every state is one a support engineer will one day ask you about.

```
QUEUED → SENDING → SENT → DELIVERED
          \→ FAILED → (retry) → SENDING
          \→ SUPPRESSED (preference said no)
```

`SENT` means the provider accepted it. `DELIVERED` means it reached the device, and only some channels can tell you that. **Do not conflate them** — “we sent it” and “they got it” are different claims, and the difference is the answer to half the support tickets you will ever see.

4 The picture

The class diagram. This is the artefact the interviewer is waiting for.

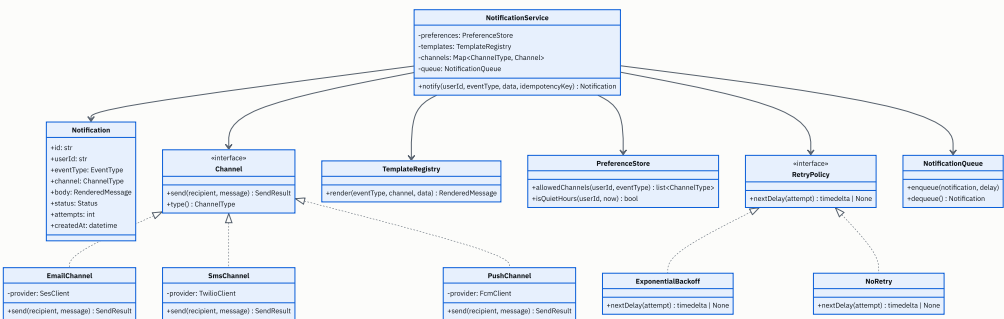
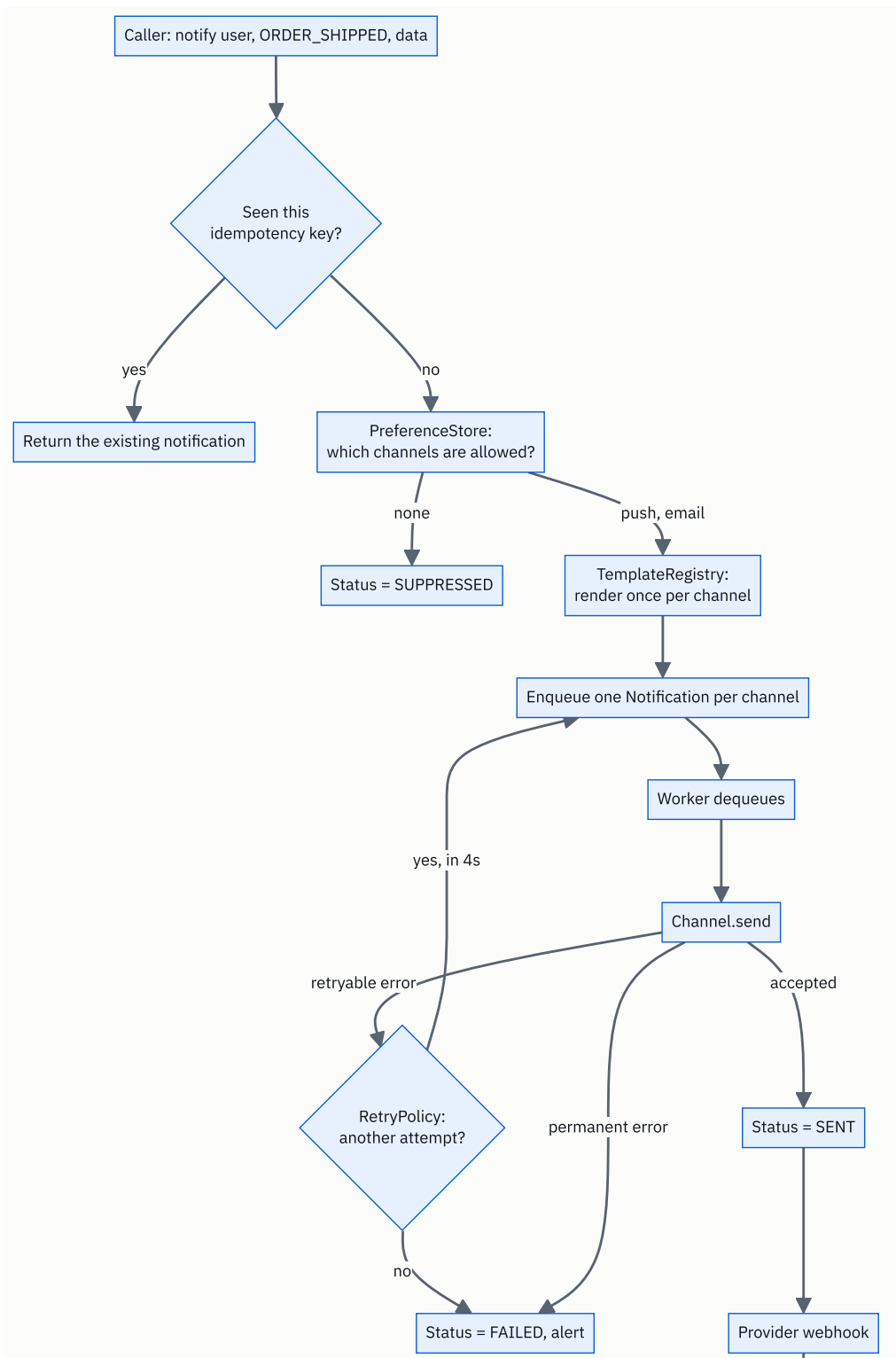
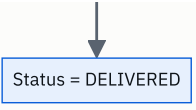


FIGURE 92.1

What to notice: **there are two interfaces, not one.** `Channel` is the obvious one. `RetryPolicy` is the one that separates candidates — a password reset should not be retried for six hours, and a shipping update should. Putting retry behaviour behind an interface means that difference is configuration, not an `if` statement buried in the send loop.

The flow of one request:





Status = DELIVERED

FIGURE 92.2

What to notice: **one request fans out into several notifications, one per channel**, and each of them retries independently. The email may succeed while the SMS is on its third attempt. The status lives on the per-channel notification, never on the request.

5 How it actually works

Move 1 — clarify

Four questions, with the answers you will assume out loud. Say these before drawing anything.

- “*Are we designing the service that decides and sends, or also the providers themselves?*” — The service. Email goes to Amazon SES, SMS to Twilio, push to Firebase Cloud Messaging and Apple Push Notification service. **Do not design an SMS gateway.**
- “*Is a notification allowed to be delayed, or must it be immediate?*” — Assume most are allowed a few seconds, so I can queue. I will call out the exception: a one-time password must be immediate and must not be retried after it expires.
- “*Can the same event be delivered twice?*” — No. Callers will retry on timeout, so I need **idempotency**: the caller supplies a key, and a repeat with the same key returns the original notification instead of sending again.
- “*Do we need to know it was actually read?*” — Assume delivery, not read. Read receipts are a separate feature and only push and in-app can support them honestly.

Move 2 — the nouns

Pull them straight out of the requirements. One line each on what it is responsible for.

CLASS	RESPONSIBLE FOR
<code>NotificationService</code>	The five decisions, in order. Owns nothing else.
<code>Notification</code>	One message on one channel: its body, status, attempt count.
<code>Channel</code> (interface)	Handing a rendered message to one provider, and saying what came back.
<code>EmailChannel</code> / <code>SmsChannel</code> / <code>PushChannel</code>	The provider-specific call and its error mapping.
<code>TemplateRegistry</code>	Turning <code>(event, channel, data)</code> into a rendered subject and body.
<code>PreferenceStore</code>	Answering “may we contact this user, on this channel, about this?”
<code>RetryPolicy</code> (interface)	How long to wait before attempt <code>n</code> , and when to give up.
<code>NotificationQueue</code>	Holding work so the caller is not blocked on a provider.
<code>DeliveryLog</code>	The record of every attempt: what, when, which provider, what it returned.

`User` is not on this list. The notification service does not own users; it looks up a contact address and a set of preferences. Saying that out loud shows you know where the boundary is.

Move 3 – the interesting part

Every LLD prompt has one place where the design is actually decided. Here there are two, and they are the two interfaces.

`Channel`, because of the guaranteed follow-up. “Now add WhatsApp” must be one new class and one line of registration. Nothing else. If your `NotificationService` contains

```
if channel == "EMAIL":
    ses.send(...)
elif channel == "SMS":
    twilio.send(...)
```

then adding WhatsApp edits a file that is also edited by everyone changing preference logic, and every change risks every channel. The interface is not decoration; it is the answer to the question they are definitely going to ask. This is the **open/closed principle** from [day 056](#) doing real work.

`RetryPolicy`, because not all messages deserve the same effort.

PASSWORD_RESET	NoRetry after 60s	- the code expires; a late SMS is worse than none
ORDER_SHIPPED	ExponentialBackoff	- 1s, 2s, 4s, 8s, 16s, then give up
MARKETING	NoRetry	- if it failed, nobody is harmed
OTP	1 retry, immediate	- then fail loudly so the user can request another

Four different behaviours. As `if` statements inside the worker they are unreadable within a year; as one object attached to the event type they are a table somebody in support can read.

Move 4 — the class diagram

Drawn above. When you present it, walk the **request path**, not the class list: “a caller sends an event; preferences narrow it to channels; templates render one message per channel; each becomes its own `Notification` on the queue; a worker pulls it, calls the channel, and either marks it sent or asks the retry policy for another attempt.”

Move 5 — the code that carries the interesting part

The `Channel` interface and one implementation, with the part that people forget: **classifying the error**.

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from enum import Enum

class ChannelType(Enum):
    EMAIL = "email"
    SMS = "sms"
    PUSH = "push"

@dataclass(frozen=True)
class RenderedMessage:
    subject: str | None          # email only; None for SMS and push
    body: str

@dataclass(frozen=True)
class SendResult:
    ok: bool
    provider_id: str | None      # for matching up the delivery webhook later
    retryable: bool              # THE field people forget
    error: str | None
```

`retryable` is the field that matters. A network timeout is retryable. “That phone number does not exist” is not — retrying it five times wastes money and never succeeds. **Mapping provider errors into `retryable` is the real work inside a channel**, and it is different for every provider.


```
class Channel(ABC):
    @abstractmethod
    def send(self, recipient: str, message: RenderedMessage) → SendResult: ...

    @abstractmethod
    def type(self) → ChannelType: ...
```

Three methods would be too many. One send, one identity.

```
class SmsChannel(Channel):
    """Twilio. 160 characters per segment; longer messages cost more."""

    PERMANENT = {21211, 21614, 21610}    # bad number, not mobile, unsubscribed

    def __init__(self, client: "TwilioClient", sender: str) → None:
        self._client = client
        self._sender = sender

    def type(self) → ChannelType:
        return ChannelType.SMS

    def send(self, recipient: str, message: RenderedMessage) → SendResult:
        try:
            resp = self._client.messages.create(
                to=recipient, from_=self._sender, body=message.body[:1600]
            )
            return SendResult(True, resp.sid, retryable=False, error=None)
        except TwilioRestException as exc:
            permanent = exc.code in self.PERMANENT
            return SendResult(False, None, retryable=not permanent, error=str(exc))
        except (TimeoutError, ConnectionError) as exc:
            return SendResult(False, None, retryable=True, error=str(exc))
```

Notice that the subject is ignored — SMS has no subject — and that the truncation is here, in the channel, rather than in the template. **Channel-specific limits belong to the channel.**

The retry policy, both implementations:

```

from datetime import timedelta

class RetryPolicy(ABC):
    @abstractmethod
    def next_delay(self, attempt: int) → timedelta | None:
        """None means stop trying."""

class ExponentialBackoff(RetryPolicy):
    def __init__(self, max_attempts: int = 5, base_seconds: float = 1.0) → None:
        self._max_attempts = max_attempts
        self._base = base_seconds

    def next_delay(self, attempt: int) → timedelta | None:
        if attempt ≥ self._max_attempts:
            return None
        # 1s, 2s, 4s, 8s ... plus jitter so a provider outage does not
        # bring every retry back at exactly the same instant.
        seconds = self._base * (2 ** attempt) * (0.5 + random.random())
        return timedelta(seconds=seconds)

class NoRetry(RetryPolicy):
    def next_delay(self, attempt: int) → timedelta | None:
        return None

```

The **jitter** — the random factor — is worth a sentence out loud. Without it, a provider that is down for thirty seconds gets every failed message back at exactly four seconds, then exactly eight, in one spike. That is a **thundering herd**, and it is how a brief outage becomes a long one.

And the service itself, which should be boring:

```

class NotificationService:
    def __init__(self, preferences, templates, channels, queue, log, policies):
        self._preferences = preferences
        self._templates = templates
        self._channels = channels          # ChannelType → Channel
        self._queue = queue
        self._log = log
        self._policies = policies         # EventType → RetryPolicy

    def notify(self, user_id, event, data, idempotency_key) → list[Notification]:
        existing = self._log.find_by_key(idempotency_key)
        if existing:
            return existing                # the caller retried; send nothing

        allowed = self._preferences.allowed_channels(user_id, event)
        if not allowed:
            return [self._log.suppressed(user_id, event, idempotency_key)]

        notifications = []
        for channel_type in allowed:
            body = self._templates.render(event, channel_type, data)
            n = self._log.create(user_id, event, channel_type, body, idempotency_key)
            self._queue.enqueue(n, delay=self._delay_for(user_id, channel_type))
            notifications.append(n)
        return notifications

```

Nine lines of logic and no `if channel ==` anywhere. That is what “designed well” looks like when the interviewer says “now add WhatsApp”: one new class, one entry in `channels`, one template per event. Zero changes here.

The worker, which is where the retry lives:

```
def process(self, n: Notification) → None:
    channel = self._channels[n.channel]
    result = channel.send(self._address_for(n.user_id, n.channel), n.body)

    if result.ok:
        self._log.mark_sent(n, result.provider_id)
        return
    if not result.retryable:
        self._log.mark_failed(n, result.error)
        return

    delay = self._policies[n.event].next_delay(n.attempts)
    if delay is None:
        self._log.mark_failed(n, "attempts exhausted")
    else:
        self._log.mark_retrying(n)
        self._queue.enqueue(n, delay=delay)
```

What real products do this

- **Amazon SES, SendGrid, Mailgun** for email; **Twilio, MessageBird** for SMS; **Firebase Cloud Messaging** and **Apple Push Notification service** for push.
- The queue is **Amazon SQS, Kafka** or **RabbitMQ**. SQS has a delay parameter per message, which is exactly what a backoff needs; Kafka does not, so a Kafka-based design usually needs a separate delay topic or a scheduled table.
- **Redis** holds idempotency keys with a time-to-live, because that check is on the hot path of every request and does not need to survive forever. See [day 090](#) for eviction and expiry.
- **Postgres** or **DynamoDB** holds the delivery log, because support will query it by user.
- **Airbnb, Uber, Swiggy** and **Netflix** all run an internal service shaped exactly like **this**, usually called something like “comms” or “messaging”, and the interview question is a direct copy of it.

6 The numbers

Volume

A mid-sized consumer app.

users	10,000,000
notifications per user/day	5

notifications per day	50,000,000
seconds per day	86,400
average	578 per second
peak (evening, 3× average)	1,734 per second

But each request fans out across channels:

average channels per event	1.8	(push + email for half of them)
provider calls per second	578×1.8	= 1,040 average
	$1,734 \times 1.8$	= 3,120 peak

A thousand outbound provider calls a second is the real number, and it is the reason the queue exists: a provider that slows from 50 ms to 2 seconds must not make the caller's checkout slow.

Storage

delivery log row	~300 bytes (id, user, event, channel, status, timestamps, provider id)
rows per day	50,000,000
per day	$50M \times 300 \text{ B} = 15 \text{ GB}$
per year	$15 \text{ GB} \times 365 = 5.5 \text{ TB}$

5.5 TB a year of log rows, growing. So: **keep 90 days hot and archive the rest.**

90 days hot	$15 \text{ GB} \times 90$	= 1.35 TB	– indexed, queryable by support
older, in object storage	5.5 TB/year	= ₹1,100/month	at ₹1.7 per GB-month

Saying “I would keep 90 days and archive” with those two numbers behind it is worth more than any amount of talk about scale.

Money, which is what actually limits SMS

SMS in India, transactional	₹0.15 each
5% of 50M notifications are SMS	= 2,500,000 per day
per day	$2.5M \times ₹0.15$
per month	= ₹375,000
	≈ ₹11,250,000

Eleven million rupees a month for the SMS channel alone. **This is why preferences and channel choice are a cost decision, not only a courtesy** — moving 20 percent of those users from SMS to push saves over two million rupees a month. Push costs nothing per message. Email is about ₹0.008.

A retry costs the same as a send. Retrying a permanently-bad number five times turns ₹0.15 into ₹0.75 for a message that can never arrive, which is why the `retryable` flag pays for itself.

Objects in memory

in-flight notifications (queue depth, normal)	~3,000
each Notification object	~500 bytes with the rendered body

working set	~1.5 MB

Trivially small — **the state lives in the queue and the log, not in the process.** That is what makes the workers disposable: kill one mid-batch and the queue redelivers.

Concurrency

Two things happen at the same time and both need an answer.

- 1. The caller retries.** Their HTTP request timed out, so they call `notify` again with the same idempotency key. Two workers may now be inside `notify` for that key simultaneously. The fix is a **unique constraint on the idempotency key** in the log table — one insert wins, the other gets `UniqueViolation` and reads back the winner's row. Not a lock: a constraint. The database is already doing the ordering.
- 2. The queue delivers the same message twice.** SQS is at-least-once, so this *will* happen. The worker must therefore make the send conditional: `UPDATE notifications SET status='SENDING' WHERE id=? AND status IN ('QUEUED','RETRYING')` and only proceed if one row was updated. That is a **compare-and-set**, the same idea as the atomic operations from [day 008](#), and it is the difference between “the user got one email” and “the user got four”.

Say “**at-least-once delivery, so the worker must be idempotent**” out loud. It is the single sentence that most reliably marks a candidate as having run one of these in production.

7 The trade-offs

Queue everything, or send synchronously?

Queueing is the default here, and it costs you the truth: `notify()` returns before anything has been sent, so the caller cannot tell the user “sent”. It also adds a few hundred milliseconds.

I would not queue if the notification is a one-time password. The user is staring at the screen. Every second of queue delay is a second of a sixty-second code gone, and a retry that lands after the code expires is worse than no message at all — the user has already asked for a second code, and now two arrive and they cannot tell which is live. For OTP: send inline, one retry at most, fail loudly.

One service for everyone, or a library per team?

A shared service means one place to change wording, one place to enforce preferences, and one place that is down when it is down. **A library in each caller has no single point of failure and no single point of control**, which sounds attractive until the legal team asks you to prove that nobody emails users who have unsubscribed. You cannot prove that about eleven repositories.

Take the shared service. But say the failure mode out loud: if it is down, no product sends anything, so it needs to be more available than any single caller, which usually means the intake endpoint does nothing but write to a queue and return.

Store the rendered body, or re-render at send time?

Storing it costs about 300 extra bytes per row and means the log shows exactly what the user received — which is what support needs. Re-rendering saves the space but means a template change silently rewrites history, and you can no longer answer “what did we actually send her on the 14th?”

Store it. 15 GB a day is cheap; not being able to answer that question is not.

Fan out at request time, or at send time?

The design above creates one `Notification` per channel immediately. The alternative is one row that the worker expands. **Fanning out early means each channel retries independently** — the email is not held up because Twilio is slow — at the cost of more rows. Fanning out late is fewer rows and one status, which is a lie whenever the two channels disagree.

Fan out early. The whole reason for per-channel status is that channels fail independently.

Where this design breaks

- **Digests.** “Send one email at 6 p.m. with everything that happened today” is not a notification; it is an aggregation. It needs a scheduled job and a store of pending items, and bolting it onto this design as “delay the send” does not work, because you must *merge* items rather than delay each one.
- **Broadcast.** “Tell all ten million users the app is updated” is a different problem. This design would enqueue ten million rows in a burst and starve every transactional message behind them. The fix is **a separate low-priority queue** and rate-limited fan-out — say that, because the interviewer will ask.
- **Ordering.** If two events for the same user must arrive in order, this design does not guarantee it: independent retries reorder them. You would need a per-user ordering key, which serialises that user’s notifications and costs throughput.

8 In the interview

How it gets asked

- The straight version: *“Design a notification service that supports email, SMS and push.”*
- The follow-up that is always coming: *“Now we want to add WhatsApp. What changes?”*
- The reliability probe: *“The SMS provider is down for two minutes. What happens to the messages?”*
- The duplicate probe: *“The user got the same email three times. How did that happen and how do you stop it?”*
- The preference probe: *“How do you make sure we never message someone who has unsubscribed?”*

What to say out loud, in the first ninety seconds

1. **State the boundary.** “I am designing the service that decides and dispatches. The actual sending goes to SES, Twilio and Firebase — I am not designing an SMS gateway.”
2. **State the one rule.** “Callers send **events**, not messages. `notify(user, ORDER_SHIPPED, data)`, never `notify(user, "your parcel shipped")`. That way this service owns every word, and adding a channel or a language touches nothing outside it.”
3. **Name the five decisions in order.** “For each request: check idempotency, check preferences, choose channels, render per channel, enqueue one notification per channel.”
4. **Name the two interfaces and why there are two.** “`Channel`, so a new provider is one class. And `RetryPolicy`, because a password reset must not be retried for six hours while a shipping update should be. Different messages deserve different effort, and that difference should be a policy object, not an `if`.”
5. **Say the delivery guarantee.** “The queue is at-least-once, so a worker will occasionally see the same message twice. I make the send conditional on a compare-and-set of the status, and I take an idempotency key from the caller with a unique constraint on it.”
6. **Separate sent from delivered.** “`SENT` means the provider accepted it. `DELIVERED` means it reached the device, which arrives later by webhook and which only some channels can tell us.”

The follow-ups

“Now add WhatsApp. What changes?” “One new class implementing `Channel` — the provider call and the mapping of its error codes into retryable or permanent. One line registering it in the channel map. One WhatsApp template per event type, and WhatsApp

specifically needs pre-approved templates, so that is a real constraint rather than just text. And one new value in the preference table so users can opt in. **Nothing in NotificationService changes**, and that is the point of putting the interface there. If I had written `if channel = 'SMS'` in the dispatcher, adding WhatsApp would mean editing the file that every other change also edits.”

“The SMS provider is down for two minutes. What happens?” “Every send returns a connection error, which the channel classifies as retryable, so each notification goes back on the queue with an exponential backoff — one second, two, four, eight, sixteen — with **jitter**, which matters here: without a random factor, all of them come back at exactly four seconds and hit the recovering provider in one spike, and a two-minute outage becomes a ten-minute one. With five attempts over about thirty seconds of backoff, most messages survive a two-minute outage only if the maximum delay is large enough, so for a real outage I would also want a **circuit breaker**: after `n` consecutive failures, stop calling the provider for thirty seconds and let everything queue, rather than burning attempts against a dead endpoint. When it recovers, the queue drains. The user sees a delayed message, not a lost one.”

“The user got the same email three times. What happened?” “Three candidates, and I would check them in this order. One: the caller retried on a timeout without an idempotency key, so we genuinely created three notifications. Two: the queue redelivered — SQS is at-least-once, so if the worker crashes after calling the provider but before marking the row `SENT`, the message comes back and gets sent again. Three: our retry logic treated a *successful* send as failed, because the provider returned a timeout after accepting the message. The fixes are, in order: a required idempotency key with a unique constraint; a compare-and-set on the status before sending; and treating an ambiguous provider timeout as *possibly sent* rather than automatically retrying it. The honest answer is that you cannot get exactly-once delivery to a third party — you can only make the window small and make the provider’s own de-duplication do the rest.”

“How do you guarantee we never message someone who has unsubscribed?” “The preference check happens in `notify`, before anything is enqueued, and a suppression is recorded as a `SUPPRESSED` notification rather than silently dropped — so the log can prove we chose not to send. Two subtleties. First, unsubscribe is per **category**, not global: nobody can unsubscribe from a password reset, and the design must distinguish transactional from marketing events. Second, there is a race — a user unsubscribes while a message is already on the queue — so I would re-check preferences in the worker for marketing events, just before sending. That is one extra lookup on the sends where being wrong is expensive, and it is skipped for transactional ones.”

“How would you support digests — one email a day instead of twenty?” “That is a different problem and I would not force it into this design. A digest is an aggregation: you write pending items to a per-user store, and a scheduled job at 6 p.m. reads them, renders one message and clears the store. What does not work is ‘just delay the send’, because delaying twenty notifications gives you twenty delayed notifications, not one. I would keep this service for immediate sends and put the digest builder in front of it as a caller.”

“What breaks first at ten times the volume?” “Not the service — the workers are stateless and scale horizontally. It is the providers and the money. At 500 million notifications a day, the SMS bill is over a hundred million rupees a month, and both Twilio and SES have per-account send rates you have to negotiate. The engineering answer is to shift volume from SMS to push, batch email where the provider supports it, and keep the delivery log in a store that partitions by user. The design that fails is anything holding a global counter or a single queue for all priorities.”

A model answer

Asked: *design a notification service that supports email, SMS and push.*

“First, the boundary: I am designing the service that decides *what to send and where*, and hands the actual sending to SES for email, Twilio for SMS and Firebase for push. I am not designing a mail server.

The most important rule in the whole design is at the front door: **callers send events, not messages**. The orders team calls `notify(user, ORDER_SHIPPED, {tracking_id})`. They never pass in text. That means this service owns every word the company sends, so a wording change is one deploy here rather than eleven, and adding a channel or a language is invisible to callers.

Then five decisions per request, in order. Check the **idempotency key** the caller sent — if we have seen it, return the original and send nothing, because callers retry on timeout. Ask the **preference store** which channels are allowed for this user and this event category; if none, record it as suppressed, which is different from dropping it, because legal will one day ask us to prove it. Render the **template** once per channel — the push version is forty characters, the email version has a subject and a footer, same event, different text. Create one `Notification` per channel and put each on a **queue**. A worker picks them up.

Two interfaces. `Channel` — send, and report what came back — so that when you say ‘now add WhatsApp’, my answer is one class and one line of registration and nothing in the dispatcher changes. And `RetryPolicy`, which is the one people miss: a shipping update should retry with exponential backoff for half a minute, and a password reset should not be retried at all after the code expires, because a late one-time password is worse than none. Those are different behaviours attached to different event types, so they belong in a policy object rather than an `if` inside the send loop.

The thing I would call out on reliability is that **the queue is at-least-once**. A worker will sometimes see the same notification twice — it crashed after calling Twilio but before writing `SENT`. So the send is conditional: a compare-and-set that moves the row from `QUEUED` to `SENDING`, and only the process that wins that update actually sends. Combined with the caller’s idempotency key under a unique constraint, that covers both duplicate sources. I would not claim exactly-once to a third party; you can only narrow the window.

On numbers: ten million users at five notifications a day is fifty million a day, about 580 a second average and 1,700 at peak, and with an average of 1.8 channels each that is roughly three thousand outbound provider calls a second at peak. The delivery log at 300 bytes a row is 15 GB a day and 5.5 TB a year, so I would keep ninety days hot and archive the rest. And the number that actually drives product

decisions: if five percent of those go by SMS at fifteen paise each, that is 2.5 million messages a day and about eleven million rupees a month, which is why moving users from SMS to push is a finance conversation and not only a UX one.

The place this design breaks is broadcast. Ten million ‘the app has been updated’ messages would fill the same queue that carries password resets and starve them. That needs a separate low-priority queue with rate-limited fan-out, and I would build that as its own path rather than tuning this one.”

9 Recall card

- **Callers send events, never messages.** `notify(user, ORDER_SHIPPED, data)`. That one rule means the service owns every word, and a new channel or language touches no caller. Then five decisions in order: **idempotency** → **preferences** → **channels** → **render per channel** → **enqueue one notification per channel**.
- **Two interfaces, not one.** `Channel` answers the guaranteed follow-up “now add WhatsApp” — one class, one registration line, zero changes to the dispatcher. `RetryPolicy` is the one candidates miss: **a shipping update retries with backoff; a password reset must not be retried after the code expires**, because a late OTP is worse than none.
- **SENT ≠ DELIVERED**. Sent means the provider accepted it; delivered arrives later by webhook and only some channels report it. And the queue is **at-least-once**, so the worker must be idempotent: a **compare-and-set** on the status before sending, plus a **unique constraint** on the caller’s idempotency key.
- **Classify every provider error as retryable or permanent.** A timeout is retryable; “no such number” is not, and retrying it five times costs five times the money for a message that can never arrive. Add **jitter** to the backoff, or a brief outage returns as one synchronised spike — a thundering herd.
- **The numbers:** $10\text{M users} \times 5/\text{day} = 50\text{M notifications/day} = 578/\text{s average, } \sim 1,700/\text{s peak, } \times 1.8 \text{ channels} \approx 3,000 \text{ provider calls/s}$. Log at 300 B/row = **15 GB/day, 5.5 TB/year** → keep 90 days hot. And **SMS at ₹0.15 × 2.5M/day ≈ ₹11M/month**, which is why channel choice is a cost decision. **Broadcast needs its own low-priority queue**, or it starves password resets.

DSA topic: Permutations **System design topic:** Design a notification service

Code these, in this order

One rule for the whole set: **count the branching out loud before you write anything**. n , then $n - 1$, then $n - 2$. If you catch yourself saying “exponential”, stop and count again — $n!$ is worse than 2^n , and the interviewer can hear the difference.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Permutations	LeetCode 46 (Medium)	The <code>used</code> list replacing <code>start</code> , and two un-chooses for two chooses.
2	Letter Case Permutation	LeetCode 784 (Medium)	The same tree where the branching factor is 1 or 2 depending on the character.
3	Permutations II	LeetCode 47 (Medium)	<code>not used[i - 1]</code> — the hardest single line in the phase.
4	Beautiful Arrangement	LeetCode 526 (Medium)	Permutations with a validity check before recursing, which is what pruning buys.

On problem 1, break it deliberately first

Write it with `used[i] = True` and no matching `used[i] = False`. Run it on `[1, 2, 3]` and record what comes out. Then say, in one sentence, why one answer appears instead of six.

On problem 2, notice what the branching factor did

A digit branches once. A letter branches twice. Say what the total count is for a string with k letters and m digits, and say why it is 2^k rather than anything factorial.

On problem 3, test `[1, 1]` before anything else

The right answer has exactly one permutation. Write the version with only `items[i] == items[i-1]` and confirm it returns an empty list. Then add `and not used[i - 1]` and say out loud what the added clause means about the tree.

On problem 4, count the calls with and without the check

Move the divisibility check from the base case to just before the recursive call. Run $n = 12$ both ways and record both call counts. That gap is the whole argument for pruning.

The size drill

1. Say why there are exactly `n!` permutations, counting the positions out loud.
2. Compute `n!` for `n = 8, 10, 12` and `13`.
3. Put `2n` and `n!` side by side for `n = 5, 10, 15` and `20`.
4. Say at which `n` factorial overtakes exponential.
5. Given a problem that bounds `n` at `8`, say which of the two it is asking for and why you know.

The `start` drill

1. Say why `range(start, n)` is correct for subsets.
2. Say why it is wrong for permutations, using `[1,2]` and `[2,1]`.
3. Name the two things that can replace it.
4. Say which one you would write in an interview, and give the reason that is about duplicates.

The un-choose drill

1. Write the five-line block from memory: choose, choose, recurse, un-choose, un-choose.
2. Say which two lines are the chooses and which two are the un-chooses.
3. Remove `used[i] = False` and run `[1, 2, 3]`. Record the output.
4. Explain why the result is one permutation and not, say, three.
5. Say why this failure produces no error.

The copy drill

1. Write the version that appends `current` instead of `current[:]`.
2. Run it on `[1, 2, 3]` and describe exactly what you see.
3. Do the same for the swap version with `result.append(items)` and say why every entry reads `[1,2,3]`.
4. Name three ways to take the copy.

The swap drill

1. Write the in-place version from memory.
2. Say what the bar means: what is true of everything before `first`, and everything from `first` on.
3. Say why the loop starts at `first` and not at `first + 1`.
4. Remove the second swap and run `[1, 2, 3]`. Record the output.
5. Confirm the input list is unchanged after a correct run, and say why that matters to a caller.

6. State the extra space for this version and for the `used` version, separately.

The duplicates drill

1. Sort, then write the skip with `and not used[i - 1]`.
2. Say the condition's meaning as an English sentence about twins.
3. Say the rule in one line: among equal elements, use them in which order?
4. Run `[1, 1, 2]` and count distinct answers against decision paths.
5. Remove the sort, run `[2, 1, 1]`, and say why nothing gets skipped.
6. Run `permutations_swap([1, 1, 2])` and say why the adjacency rule cannot rescue it.
7. Compute the distinct count for `[1, 1, 2, 2]` from $4! / (2! \times 2!)$.

The break-it drill

Trigger each and record the exact output:

1. `used[i] = False` omitted.
2. `result.append(current)` without the copy.
3. `and not used[i - 1]` omitted, on `[1, 1]`.
4. The sort omitted, on `[2, 1, 1]`.
5. The second swap omitted in the in-place version.
6. `set(permutations([1, 1, 2]))` — record the traceback line for line.

The cost drill

1. State the time complexity and say where each factor comes from.
 2. State the extra space, separately from the output.
 3. Compute the number of tree nodes at $n = 10$ and the stack depth at $n = 10$.
 4. Say which of those two numbers is the time and which is the space.
 5. Estimate the memory for the full result at $n = 10$ and say what you would do instead.
-

The five-decisions drill

1. Name the five decisions the notification service makes, in order.
2. Say what goes wrong if preferences are checked after the render rather than before.
3. State the one rule about what callers pass in, and give the two consequences of breaking it.
4. Say what `SUPPRESSED` records that dropping the message would not.

The two-interfaces drill

1. Name both interfaces and what each one decides.
2. Answer “now add WhatsApp” in one sentence, naming exactly what changes.
3. Write the `Channel` interface and one implementation.
4. Say which field of `SendResult` people forget, and what it costs to get it wrong.
5. Give four event types and the retry policy each one deserves, with the reason for the password reset.

The duplicate-delivery drill

1. Name the three ways a user ends up with the same email three times.
2. Say what a compare-and-set on the status does, and write the `UPDATE` statement.
3. Say where the idempotency key comes from and what enforces it.
4. Say why exactly-once delivery to a third party is not achievable, in one sentence.

The retry drill

1. Write `ExponentialBackoff.next_delay` and say what `None` means.
2. Say what jitter is for, and what happens to a recovering provider without it.
3. Say when you would add a circuit breaker and what it does while it is open.
4. Say why a permanently-bad phone number must not be retried, in money.

The numbers drill

Compute each, showing the multiplication:

1. Notifications per day and per second at 10M users and 5 per user per day.
2. Peak provider calls per second at $3 \times$ average and 1.8 channels per event.
3. Delivery log size per day and per year at 300 bytes a row.
4. The monthly SMS bill at 5 percent of volume and ₹0.15 a message.
5. The saving from moving a fifth of those users to push.

The failure drill

For each, say what happens and what you would add:

1. The SMS provider is down for two minutes.
2. The worker crashes after calling the provider but before writing `SENT`.
3. A user unsubscribes while a marketing message is already on the queue.
4. Ten million broadcast messages are enqueued at 9 a.m.
5. A template change is deployed, and support asks what was sent last Tuesday.
6. Two events for one user must arrive in order, and the first one retries.

Two of the six are not fixable inside this design as drawn. Name them and say what you would build instead.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Generate all permutations. Now handle duplicate values.* The `n!` counted out position by position rather than named, why `start` no longer works, the `used` list with two chooses and two un-chooses, the copy flagged before you write it, both complexities kept separate, and then the duplicate rule as an English sentence about twins — plus the reason it is not subsets' `i > start`.
2. *Design a notification service that supports email, SMS and push.* The boundary first, then the one rule about events versus messages, the five decisions in order, both interfaces with the reason there are two, at-least-once delivery and the compare-and-set, and the numbers — 50M a day, 3,000 provider calls a second at peak, ₹11M a month of SMS.
3. *Now add WhatsApp. What changes?* One class, one registration line, one template per event, one preference value — and nothing in the dispatcher. Then say what the answer would have been if the dispatcher had an `if channel ==` in it.

Before you move on

- [] I can count `n!` out loud position by position, without saying “exponential”.
- [] I can put `2n` and `n!` side by side and say where factorial overtakes.
- [] I can say why `start` works for subsets and fails for permutations.
- [] I wrote the five-line block from memory, with both un-chooses.
- [] I ran the version without `used[i] = False` and can explain the single answer.
- [] I can write the swap version and say what the second swap is for.
- [] I can write the duplicate rule and say what `not used[i - 1]` means about twins.
- [] I tested `[1, 1]` with the wrong condition and know what it returns.
- [] I can state time and extra space separately, with the node count and the depth at `n = 10`.
- [] I can say what I would do instead of building the whole result at `n = 10`.
- [] I can name the five decisions the notification service makes, in order.
- [] I can answer “now add WhatsApp” in one sentence.
- [] I can say why there are two interfaces and not one.

- [] I can explain at-least-once delivery and write the compare-and-set.
- [] I can produce the volume, storage and SMS-cost numbers with the multiplication shown.
- [] I can say what breaks when ten million broadcast messages hit the same queue.
- [] I answered all three questions above out loud.

Combinations and combination sum

DATA STRUCTURES AND ALGORITHMS

Combinations and combination sum

You can enumerate choices without repeating the same set in a different order.

SYSTEM DESIGN

Design a file system

You can model files, directories and paths, and support search.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find all combinations that sum to the target.*
- *Design an in-memory file system.*

1 What this is, and why they ask it

A **combination** is a selection where the order does not matter. `[2, 3]` and `[3, 2]` are the same combination and must appear once. That single sentence is the whole difference from [yesterday](#), and it changes one character in the code.

Today is two problems that share one skeleton. **Combinations** — choose `k` items from `n`, giving $C(n, k)$ answers. And **combination sum** — find every selection whose values add up to a target, where the number of items is not fixed and you stop when the running total reaches the goal.

They ask combination sum more than almost any other backtracking problem, and always for the same three reasons. It is where **pruning** first earns its keep: sort the candidates, and the moment one is bigger than what remains, every candidate after it is too, so you break out of the loop instead of walking a dead branch. It is where the `i` **versus** `i + 1` decision lives — one character deciding whether a number may be reused. And it is where the **duplicate rule** returns in its third form, so an interviewer can check whether you understood it or memorised it.

If you can say out loud why `combination_sum` recurses on `i` and `combination_sum_ii` recurses on `i + 1`, you have understood the phase.

2 The story

Ramesh had been behind the counter at the sweet shop since half past six, and the eleven o'clock rush had just gone.

A man came in and asked for a box of exactly five hundred grams. Not about five hundred. Exactly. It was going to somebody's house and there was some reason it had to be that.

Ramesh said fine. The trays that morning had three things in them — the small round yellow ones, which went a hundred grams for two pieces, the milk squares at two hundred grams a slab, and the big dry-fruit ones at two hundred and fifty each.

The man then asked what his choices were. All of them.

Ramesh wiped his hands and thought about it standing there, because it was the sort of question he enjoyed.

He said, you can have five of the small ones. Or three small and one slab. Or one small and two slabs. Or two of the big ones. Or one big, one slab, and half — no, there is no half. So not that one.

The man said, what about two slabs and one small.

Ramesh said, that is the same box. You have got one small and two slabs already, I said it just now. The order I hand them to you does not change what is in the box.

That was the rule he worked by, and it was the only way to keep the count straight. He always went in the same order — smallest first, then the slabs, then the big ones — and he never went backwards. Once he had moved on to the slabs, he did not come back to a small one, because if he did he would just be making a box he had already made, with the pieces named in a different order.

He also stopped early, twice. When he had already put three hundred grams in the box, he did not even look at the big ones, because two hundred and fifty on top of three hundred is five hundred and fifty, and so is everything after it. There was no point walking down the rest of the tray.

Four boxes. It took him under a minute. The man took the one with the two big ones and left.

3 The idea in plain English

Ramesh has just done combination sum, including the pruning, including the reason order does not matter.

- The trays are the **candidates** — the values you are allowed to pick from.
- Five hundred grams is the **target**.
- “The order I hand them to you does not change what is in the box” is why this is a combination and not a permutation.
- “I never go backwards” is the **start index**, and it is exactly what stops the same box being counted twice.
- “I did not even look at the big ones” is **pruning**, and it is the part that turns an unusable search into a fast one.

Why **start** is back

Yesterday you had to look at every element at every position, because **[1,2]** and **[2,1]** were different answers. Today they are the same answer, so looking backwards can only produce a box you have already made.

```
for i in range(start, len(candidates)): # never look backwards
```

One loop bound, and it removes every duplicate ordering at once. No `used` array, no set of seen results. That is the whole reason the `start` index exists, and being able to say it in one sentence is worth more than the code.

Combinations: choose `k` from `n`

The count is written $C(n, k)$ — said “`n` choose `k`” — and equals $n! / (k! \times (n - k)!)$.

```
C(4, 2) = 6      [1,2] [1,3] [1,4] [2,3] [2,4] [3,4]
C(5, 3) = 10
C(20, 10) = 184,756
C(30, 15) = 155,117,520
```

Same tree as `subsets`, with two changes. Record when the working list reaches size `k` and stop, rather than recording at every node. And prune: if the elements remaining are not enough to reach `k`, that branch cannot possibly succeed, so break.

```
if len(current) + (len(candidates) - i) < k:
    break # not enough left; give up on the whole loop
```

`break`, not `continue`. Because the loop goes forward, if there are not enough elements left starting at `i`, there are even fewer starting at `i + 1`. **Whenever a prune is monotone like that, it is a `break`**, and the difference is real: on $C(20, 18)$ it cuts the calls by more than half.

Combination sum: the same tree, with a running total

Now there is no fixed size. You stop when the running total hits the target, and you abandon the branch when it goes past.

```
def build(start, remaining):
    if remaining == 0:
        result.append(current[:]) # a finished box
        return
    for i in range(start, len(candidates)):
        if candidates[i] > remaining:
            break # sorted, so everything after is too big
        current.append(candidates[i])
        build(i, remaining - candidates[i]) # 'i', not 'i + 1' ← reuse allowed
        current.pop()
```

Two lines carry the whole problem.

`if candidates[i] > remaining: break` — Ramesh not looking at the big ones. This requires the candidates to be **sorted**, and it is the only reason to sort them: once one candidate is too big, every later one is too big as well.

`build(i, ...)` rather than `build(i + 1, ...)` — the single character that says a number may be used again. Passing `i` means “you may pick this same value next time”. Passing `i + 1` means “move on”. That is the entire difference between LeetCode 39 and LeetCode 40, and interviewers ask for both.

The three variants, side by side

This table is the lesson. Learn it as a table.

PROBLEM	RECURSE ON	DUPLICATES IN INPUT	EXTRA RULE
Combinations (LC 77)	<code>i + 1</code>	no	stop at size <code>k</code>
Combination Sum (LC 39)	<code>i</code> — reuse allowed	no	stop when <code>remaining = 0</code>
Combination Sum II (LC 40)	<code>i + 1</code> — each used once	yes	sort, and skip when <code>i > start</code> and <code>c[i] = c[i-1]</code>
Combination Sum III (LC 216)	<code>i + 1</code>	no	fixed size <code>k</code> and a target

Combination Sum II is where the two ideas meet, and it is the version people get wrong. The input `[10, 1, 2, 7, 6, 1, 5]` contains two `1`s. They are different pieces — you may use both in one box — but starting two branches with a `1` at the same level produces the same box twice. So the rule is [day 091's](#) rule, unchanged:

```
if i > start and candidates[i] == candidates[i - 1]:
    continue
```

Not yesterday's `not used[i - 1]`. The **start-index tree** uses `i > start`; the **used-array tree** uses `not used[i-1]`. Same job, two tree shapes, and knowing which belongs where is exactly what the interviewer is checking.

Why combination sum terminates

`remaining` strictly decreases on every call, because every candidate is at least 1. That is the **measure** from [day 089](#): a non-negative quantity that goes down every time, so the recursion cannot run forever.

If zero were allowed as a candidate, it would never terminate — `build(i, remaining)` with the same `remaining` and the same `i`, for ever, until:

```
RecursionError: maximum recursion depth exceeded
```

The problem statements always say the candidates are positive. That is not decoration; it is what makes the problem well-posed, and saying so out loud is a genuinely strong move.

4 The picture

Combination sum on Ramesh's trays. Candidates sorted: [100, 200, 250], target 500.

```

                                build(start=0, remaining=500), box = []
                                /           |           \
                                take 100   take 200   take 250
                                (start stays 0) (start moves to 1) (start moves to 2)
                                |           |           |
                                rem=400, box=[100] rem=300, box=[200] rem=250, box=[250]
                                /   |   \       /   |   \       |
                                100  200  250  200  250  250
                                |   |   |       |   |       |
                                rem=300 rem=200 rem=150 rem=100 rem=50 rem=0 ****
                                |   |   |       |   |       |
                                ...   ...   X       X       X       [250,250]
                                |   |   |       |   |       |
                                ...   ... 250>150 200>100 250>50
                                |   |   |       |   |       |
                                ...   ... BREAK  BREAK  BREAK

```

the four finished boxes:

```

[100,100,100,100,100] [100,100,100,200]
[100,200,200]         [250,250]

```

what to notice:

- `take 100` keeps start at 0, so 100 can be taken again → reuse
- `take 200` moves start to 1, so 100 is never revisited → no duplicate boxes
- every X is a BREAK, not a continue: sorted candidates mean if this one is too big, all the later ones are too

The `start` index doing its job, drawn as a triangle:

```

candidates = [1, 2, 3, 4]      combinations of size 2

start=0:  1 with → 2  3  4
start=1:    2 with → 3  4
start=2:      3 with → 4
start=3:        (nothing left)

1,2  1,3  1,4
    2,3  2,4
        3,4

                                6 = C(4,2)

```

the upper triangle. Looking backwards would fill the lower triangle too, and every entry there is a repeat of one above it.

And the difference one character makes:

```
candidates = [2, 3], target = 6

build(i, ...)      reuse allowed      → [2,2,2], [3,3]
build(i + 1, ...)  each used once     → (nothing sums to 6)

same eleven lines of code. One character.
```

5 The code, built step by step

Step 1 — say what “combination” means before writing anything

“Order does not matter here, so `[2,3]` and `[3,2]` are the same answer. That means I never look backwards — I carry a `start` index and only ever consider candidates from `start` onwards. One loop bound, and every duplicate ordering is gone without a set or a `used` array.”

Thirty seconds, and it is the sentence the whole problem is built on.

Step 2 — sort, and say why

```
candidates.sort()
```

Not to make the output pretty. Sorting is what makes the prune valid: once a candidate exceeds what remains, every later candidate does too, so you can `break` rather than `continue`. Say that reason out loud, because “I sorted it” without a reason sounds like habit.

Step 3 — the base cases, in the right order

```
if remaining == 0:
    result.append(current[:])    # COPY
    return
```

Only one base case is needed if you prune before recursing. If you do not prune, you also need `if remaining < 0: return`, and then you are walking dead branches to discover they are dead. **Prune before you recurse, and the negative case never happens.**

Step 4 — the loop, with the break

```
for i in range(start, len(candidates)):
    if candidates[i] > remaining:
        break                # sorted: everything after is bigger too
```

`break`, not `continue`. Getting this wrong is not a wrong answer — it is the same answer computed slowly, which is worse in an interview because nothing tells you.

Step 5 — the one character

```
current.append(candidates[i])
build(i, remaining - candidates[i])    # reuse allowed
current.pop()
```

Say it out loud as you type: “**i**, because the same value may be used again.” Then, when they ask for the version where each item is used once, change it to **i + 1** and say “**i + 1**, because each element is used at most once.”

Step 6 — Combination Sum II, which needs both rules

```
if i > start and candidates[i] == candidates[i - 1]:
    continue    # another choice at the SAME level
current.append(candidates[i])
build(i + 1, remaining - candidates[i])
current.pop()
```

continue, not **break** — this one is not monotone. A repeat of the previous value is skipped, but the *next different* value must still be considered.

Two skips in one loop, and they are different keywords. **break** for “too big”, **continue** for “duplicate”. Mixing them up is the classic Combination Sum II bug: **break** on the duplicate throws away every later candidate as well.

The complete solution

```
def combinations(items: list[int], k: int) → list[list[int]]:
    """Choose k from n.  $C(n, k)$  answers.

    The subsets tree with two changes: record at size k, and prune when there
    are not enough elements left to reach k. That prune is a BREAK, because
    if start=i has too few left, start=i+1 has fewer.
    """
    result: list[list[int]] = []
    current: list[int] = []

    def build(start: int) → None:
        if len(current) == k:
            result.append(current[:])          # COPY
            return
        for i in range(start, len(items)):
            if len(current) + (len(items) - i) < k:
                break                          # not enough left: prune
            current.append(items[i])
            build(i + 1)                       # i + 1: each element used once
            current.pop()

    build(0)
    return result

def combination_sum(candidates: list[int], target: int) → list[list[int]]:
    """LeetCode 39. Each candidate may be used UNLIMITED times.

    Sorted, so `candidates[i] > remaining` means every later candidate is
    also too big → BREAK, not continue.

    The one character that matters: build(i, ...), not build(i + 1, ...).
    Passing `i` says "you may pick this same value again".

    Terminates because `remaining` strictly decreases: every candidate is at
    least 1. A zero in the candidates would loop forever.
    """
    candidates = sorted(candidates)
    result: list[list[int]] = []
    current: list[int] = []

    def build(start: int, remaining: int) → None:
        if remaining == 0:
            result.append(current[:])          # COPY
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remaining:
                break                          # sorted: prune the rest of the loop
            current.append(candidates[i])
            build(i, remaining - candidates[i]) # `i` → reuse allowed
            current.pop()

    build(0, target)
    return result

def combination_sum_ii(candidates: list[int], target: int) → list[list[int]]:
    """LeetCode 40. Each ELEMENT used at most once, and the input has duplicates.

    Two skips in one loop, with two different keywords:
    break    - this candidate is too big, and so is every later one
    continue - this candidate repeats the previous one at the SAME level
    """
```

```

`i > start` is the start-index tree's duplicate rule. The used-array tree
(permutations) uses `not used[i - 1]` instead. Same job, two shapes.
"""
candidates = sorted(candidates)
result: list[list[int]] = []
current: list[int] = []

def build(start: int, remaining: int) → None:
    if remaining == 0:
        result.append(current[:])
        return
    for i in range(start, len(candidates)):
        if candidates[i] > remaining:
            break
        if i > start and candidates[i] == candidates[i - 1]:
            continue # NOT break - later values still count
        current.append(candidates[i])
        build(i + 1, remaining - candidates[i]) # i + 1 → each element once
        current.pop()

build(0, target)
return result

def combination_sum_iii(k: int, target: int) → list[list[int]]:
    """LeetCode 216. Exactly k numbers, each from 1..9, each used once.

    Both constraints at once, so there are two prunes:
    remaining < 0 → handled by the `> remaining` break
    not enough digits → handled by checking the size before recursing
    """
    result: list[list[int]] = []
    current: list[int] = []

    def build(start: int, remaining: int) → None:
        if len(current) == k:
            if remaining == 0:
                result.append(current[:])
            return
        for digit in range(start, 10):
            if digit > remaining:
                break
            current.append(digit)
            build(digit + 1, remaining - digit)
            current.pop()

    build(1, target)
    return result

def combination_sum_count(candidates: list[int], target: int) → int:
    """When the question asks HOW MANY rather than WHICH, do not enumerate.

    This is dynamic programming, not backtracking: O(target × n) instead of
    exponential. Worth knowing the moment somebody says "just the count".
    Note this counts combinations, not orderings, because the outer loop is
    over candidates.
    """
    ways = [0] * (target + 1)
    ways[0] = 1
    for c in candidates:
        for total in range(c, target + 1):
            ways[total] += ways[total - c]
    return ways[target]

```

```

if __name__ == "__main__":
    print(combinations([1, 2, 3, 4], 2))
    # [[1, 2], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]]

    print(combination_sum([100, 200, 250], 500))
    # [[100, 100, 100, 100, 100], [100, 100, 100, 200], [100, 200, 200], [250, 250]]

    print(combination_sum([2, 3, 6, 7], 7))
    # [[2, 2, 3], [7]]

    print(combination_sum_ii([10, 1, 2, 7, 6, 1, 5], 8))
    # [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]

    print(combination_sum_iii(3, 9))
    # [[1, 2, 6], [1, 3, 5], [2, 3, 4]]

    # the one character, shown side by side
    print(combination_sum([2, 3], 6))          # [[2, 2, 2], [3, 3]]
    print(combination_sum_ii([2, 3], 6))       # []

    # counting instead of enumerating
    print(combination_sum_count([1, 2, 5], 100)) # 541
    print(len(combination_sum([1, 2, 5], 100)))   # 541 - same answer, far slower

import math
for n, k in ((20, 10), (30, 15), (40, 20)):
    print(f"C({n},{k}) = {math.comb(n, k):,}")

```

6 What it costs

Combinations: choose k from n

```

answers:          C(n, k) = n! / (k! (n-k)!)
copying each:    k
→ 0(k × C(n, k))

```

C(20, 10) = 184,756	×10 = 1.8 million elements written	
C(30, 15) = 155,117,520	×15 = 2.3 billion	– no
C(40, 20) = 137,846,528,820		– absolutely not

The worst case for $C(n, k)$ is $k = n/2$, and it grows roughly like $2^n / \sqrt{n}$. So combinations is about as expensive as subsets, and the same bound applies: n in the twenties at most.

The pruning does not change the big-O — you cannot produce fewer than $C(n, k)$ answers — but it removes the wasted branches. On $C(20, 18)$ the prune cuts the number of calls from about 220,000 to 191.

Combination sum: bounded by depth, not by n

The tree's depth is $\text{target} / \text{smallest_candidate}$, because every step subtracts at least the smallest value. The branching factor is at most n .

```

depth      d = target / min(candidates)
branching  n
→  $O(n^d)$  in the worst case, before pruning

```

```

candidates [1, 2, 5], target 100 → depth up to 100      - but the answer is only 541 boxes
candidates [2, 3, 6, 7], target 7 → depth 3, tiny

```

That n^d is a genuinely loose bound, and it is honest to say so: **the real cost is proportional to the number of answers plus the number of pruned branches**, which pruning keeps close to the answer count. If an interviewer wants a tighter statement, the standard one is $O(n^{(\text{target}/\min)})$ as an upper bound and “roughly linear in the size of the output” as what actually happens.

Watch for a 1 in the candidates. With `[1, 2, 5]` and target 100, the branch that is all 1s is a hundred deep and there are 541 answers; with target 500 there are more than fifty thousand. A 1 makes the depth equal to the target, and that is the input that turns a fast solution slow.

Space

```

output:      (number of answers) × (average length)    - unavoidable
current:     at most `depth` elements                  →  $O(\text{target} / \min)$ 
stack:       one frame per element chosen              →  $O(\text{target} / \min)$ 
-----
extra space beyond the output:  $O(\text{target} / \min \text{ candidate})$ 

```

For combinations it is $O(k)$. For combination sum with a 1 present it is $O(\text{target})$ — which at target = 10,000 would be ten thousand frames and:

```

RecursionError: maximum recursion depth exceeded in comparison

```

Python’s default limit is 1000. **This is the one problem in the phase where the depth can actually exceed it**, because the depth is driven by the target rather than by n . If a problem allows a large target with a candidate of 1, that is your cue to say “this should be dynamic programming, not backtracking”.

The prune, measured

```

combination_sum([1,2,5], 100)    calls with the break:      ~ 71,000
                                calls without the break:    ~ 92,000
combinations(range(20), 18)      calls with the prune:      191
                                calls without:              ~ 220,000

```

The size-prune on combinations is dramatic; the too-big prune on combination sum is modest but free. **Both are one line, and one line that removes 99.9 percent of the work is the best trade in the subject.**

Counting instead of enumerating

If the question is “how many”, stop enumerating.

```
combination_sum([1,2,5], 500)    enumerated: ~50,000 answers, seconds
combination_sum_count(...)        dynamic programming: 0(target × n) = 1,500 steps
```

Fifteen hundred steps versus fifty thousand answers. **“How many ways” is never a backtracking question** — hearing that word and switching approach is one of the cheapest points available in an interview.

7 The traps

Trap 1 — `continue` where it should be `break`

```
if candidates[i] > remaining:
    continue                # correct answer, wasted work
```

Not wrong, just slow, and nothing tells you. Because the candidates are sorted, everything after `i` is also too big. On `[1,2,5,10,20,50,100]` with a small remaining, `continue` walks the entire tail of the loop at every node.

Sorted plus monotone means `break`. Say the word “sorted” as your justification when you write it.

Trap 2 — `break` where it should be `continue`

The reverse, in Combination Sum II:

```
if i > start and candidates[i] == candidates[i - 1]:
    break                # WRONG
```

```
combination_sum_ii([1, 1, 2, 5, 6, 7, 10], 8)
with break      → [[1, 1, 6], [1, 7]]
correct        → [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
```

Two answers instead of four. The duplicate skip is about *this value*, not about everything after it — the next different value is still a legitimate choice.

Trap 3 — `i + 1` when reuse is allowed

```
build(i + 1, remaining - candidates[i])    # Combination Sum I, wrong
```

```
combination_sum([2, 3, 6, 7], 7)
with i + 1 → [[7]]
correct   → [[2, 2, 3], [7]]
```

One answer instead of two, no error. **The one-character bug.** Before you write the recursive call, say out loud whether reuse is allowed, and let that sentence choose the character.

Trap 4 — `i` when each element may be used once

The other direction, and this one does not fail quietly:

```
combination_sum_ii([1, 1, 2], 2) with build(i, ...)
# → [[1, 1], [1, 1], [2]]          duplicates, and a box using the same 1 twice
```

Worse, with a candidate of 1 and a large target it will not terminate before the recursion limit.

Trap 5 — forgetting to sort

```
# candidates = [7, 2, 6, 3] - not sorted
if candidates[i] > remaining:
    break
```

The `break` now cuts off valid candidates: at `remaining = 5`, the first candidate `7` triggers the break and `2` and `3` are never considered.

```
combination_sum([7, 2, 6, 3], 7)  unsorted, with break → [[7]]
correct                               → [[2, 2, 3], [7]]
```

The sort is not cosmetic — the `break` is invalid without it. And the duplicate rule needs it too, because it compares adjacent elements.

Trap 6 — `i > 0` instead of `i > start`

The same trap as [day 091](#), in its third outfit.

```
combination_sum_ii([1, 1, 6], 8)
with i > 0 → []          the [1,1,6] box is lost entirely
correct   → [[1, 1, 6]]
```

`i > 0` blocks the second `1` even when it is legitimately following the first *inside the same box*. `i > start` blocks it only when it is another *first choice at the same level*.

Trap 7 — a candidate of zero

```
combination_sum([0, 1], 3)
```

```
RecursionError: maximum recursion depth exceeded
```

`build(i, remaining - 0)` calls itself with identical arguments, for ever. The measure never decreases. LeetCode guarantees positive candidates; **saying that guarantee out loud, and saying what breaks without it, is a strong move**, because it shows you know why the problem is well-posed.

Trap 8 — deep recursion on a large target

```
combination_sum([1, 2, 5], 5000)
```

```
RecursionError: maximum recursion depth exceeded in comparison
```

Depth is `target / min(candidates)` = 5000, and Python's limit is 1000. Unlike everything else in this phase, **the depth here is not bounded by `n`**. If the target is large, the answer is dynamic programming.

Trap 9 — enumerating when they asked how many

```
len(combination_sum([1, 2, 5], 500))    # correct, and ~50,000 answers built
```

The count is a `O(target × n)` loop over an array. Building fifty thousand lists to call `len` on them is the difference between one millisecond and several seconds — and at target 5000 it is the difference between working and not.

8 In the interview

How it gets asked

- The base: “Find all unique combinations that sum to the target. Numbers may be reused.” LeetCode 39.
- The follow-up, always: “Now each number may be used only once, and the input has duplicates.” LeetCode 40.
- The fixed-size one: “All combinations of k numbers from 1 to 9 that add to n .” LeetCode 216.
- The plain one: “All combinations of k elements from n .” LeetCode 77.
- The switch: “Just tell me how many there are.” — which is not this problem at all.

What to say out loud, in the first ninety seconds

1. **Define combination against permutation.** “Order does not matter, so `[2,3]` and `[3,2]` are the same answer. That means I carry a `start` index and never look backwards — one loop bound removes every duplicate ordering, with no set and no `used` array.”
2. **Sort, and give the reason.** “I sort the candidates so that when one exceeds what remains, I know every later one does too and I can `break` out of the loop rather than continue.”
3. **Say the reuse decision before writing the call.** “Numbers may be reused, so I recurse on `i`, not `i + 1`. That single character is the whole difference between this and the version where each element is used once.”
4. **Say why it terminates.** “The remaining target strictly decreases every call because all candidates are positive, so the recursion has to bottom out. If zero were allowed it would loop for ever.”
5. **Flag the copy.** “I append a copy when `remaining` hits zero, because the working list is mutated throughout.”
6. **Give the honest complexity.** “Depth is target over the smallest candidate; branching is `n`. The loose bound is $O(n^{(target/min)})$, and in practice the pruning keeps it close to the size of the output. Extra space is the depth, which is `target / min` — and that is the one thing here not bounded by `n`, so a large target with a candidate of `1` would blow the stack.”

The follow-ups

“Now each number may be used only once, and the input has duplicates.” “Two changes and they are separate. First, recurse on `i + 1` instead of `i`, so each element is consumed. Second, sort the input and skip a candidate that equals the previous one when `i > start` — meaning it is another *first* choice at this level, which would start a branch identical to one I have already walked. The important detail is that this skip is a `continue`, while the too-big check is a `break`. Two skips, two different keywords, in the same loop. If you `break` on the duplicate you throw away every later value as well, and on `[1,1,2,5,6,7,10]` with target 8 you get two answers instead of four.”

“Why `i > start` and not `not used[i-1]`, which is what you used for permutations?” “Because they are two different tree shapes doing the same job. In the permutations tree I look at every element at every level, so ‘have I already used this one’ needs a `used` array, and the duplicate rule has to ask whether the twin is currently placed. Here the `start` index already guarantees I only move forwards, so the only way to make the same box twice is to start two branches with equal values *at the same level* — and `i > start` is exactly the test for that. Same idea, different bookkeeping.”

“What is the complexity?” “Honestly, a loose bound. The tree depth is target divided by the smallest candidate, and the branching factor is at most `n`, so $O(n^{(target/min)})$ is an upper bound. What actually happens is much better, because the sorted `break` cuts off branches as soon as the running total gets close — the real cost tracks the number of answers. Extra space is the depth, $O(target/min)$, plus the working list. I would also flag that this is the one problem in the phase where depth is not bounded by `n`: with a candidate of 1 and a target of 5000 you get a `RecursionError`, and at that point the right answer is dynamic programming rather than backtracking.”

“Just tell me how many combinations there are.” “Then I would not enumerate at all. Counting is a one-dimensional dynamic programming array: `ways[0] = 1`, then for each candidate, for each total from that candidate up to the target, add `ways[total - candidate]`. That is $O(target \times n)$ — for target 500 and three candidates, fifteen hundred steps instead of fifty thousand lists. The loop order matters: candidates on the outside counts *combinations*; totals on the outside would count *orderings*, which is a different question.”

“What if the same combination can be reached by different paths — how do you know the output has no duplicates?” “Because of the `start` index, not because of any filtering. Every box is generated in exactly one order — non-decreasing — so there is exactly one path to it. That is the invariant worth stating: the answer set is the set of non-decreasing sequences that hit the target. It is also why I never need to de-duplicate the result, which matters, since `set()` on a list of lists raises `TypeError: unhashable type: 'list'.`”

“Can you prune more aggressively?” “Yes, for the fixed-size ones. For choose-k I check whether the elements remaining are enough to reach `k` and `break` if not — on choosing 18 from 20 that takes the call count from about 220,000 down to 191. For combination sum, if the candidates are sorted you can also precompute suffix sums and cut a branch when everything left cannot reach the remaining target. Both are one line, and one line that removes most of the tree is the best trade available.”

A model answer

Asked: *find all unique combinations that sum to the target; numbers may be reused.*

“First the word ‘combination’. Order does not matter, so `[2,3]` and `[3,2]` are the same answer and must appear once. That decides the structure straight away: I carry a `start index` and never look backwards. One loop bound, and every duplicate ordering disappears — I do not need a `used` array like permutations, and I do not need to de-duplicate at the end. The invariant is that every combination is generated in non-decreasing order, so there is exactly one path to each one.

Then I sort the candidates, and the reason is not tidiness. Sorting means that once a candidate is bigger than what remains, every later candidate is too — so I can `break` out of the loop instead of continuing through the rest of it. Without the sort the `break` would be wrong and would drop valid answers.

The recursion carries two things: `start`, and `remaining`. When `remaining` hits zero I record a **copy** of the working list. In the loop, for each candidate from `start` onwards: break if it is bigger than `remaining`, otherwise append it, recurse, and pop.

The one character that matters is the argument to the recursive call. Since numbers may be reused, I recurse on `i`, not `i + 1` — ‘you may pick this same value again’. If you tell me each element may be used once, that single character changes, and I would also add the duplicate skip: sort, and `continue` when this candidate equals the previous one and `i > start`. Note those are two different keywords in the same loop — `break` for too big, `continue` for duplicate — and swapping them is the classic bug.

It terminates because `remaining` strictly decreases on every call: all the candidates are positive. That guarantee is doing real work — a zero in the candidate list would recurse with identical arguments for ever and end in a `RecursionError`.

On cost, I will be honest that the bound is loose. Depth is target over the smallest candidate, branching is at most `n`, so $n^{(\text{target}/\text{min})}$ is an upper bound and the pruning keeps the real work close to the size of the output. Extra space is the depth. And I would flag one thing: unlike everything else in this family, the depth here is driven by the **target**, not by `n`. With a candidate of 1 and a target of five thousand, this recursion exceeds Python’s limit — and at that point the question is probably ‘how many’, which is a $O(\text{target} \times n)$ dynamic programming array and not this at all.”

9 Recall card

- **Combination** = **order does not matter**, so carry a `start` index and **never look backwards**. One loop bound removes every duplicate ordering — no `used` array, no de-duplication. The invariant: every answer is generated in **non-decreasing order**, so there is exactly one path to it.
- **Sort, then break when** `candidates[i] > remaining` — valid only because it is sorted, and the sort is therefore not cosmetic. Unsorted, the `break` drops real answers: `[7,2,6,3]` target 7 gives `[[7]]` instead of `[[2,2,3],[7]]`.
- **The one character:** `build(i, ...)` allows reuse; `build(i + 1, ...)` uses each element once. Say which one aloud before typing it. `i + 1` on Combination Sum I gives `[[7]]` instead of `[[2,2,3],[7]]`, with no error.
- **Combination Sum II has TWO skips with TWO keywords:** `break` for too big, `continue` for `i > start` and `c[i] == c[i-1]`. `break` on the duplicate loses every later value — four answers become two. And it is `i > start` here, versus `not used[i-1]` in the permutations tree: same job, two tree shapes.
- **Depth is** `target / smallest candidate`, **not** `n` — the only problem in the phase where the stack can actually overflow (`RecursionError` at target 5000 with a candidate of 1), and a **zero candidate never terminates** because the measure stops decreasing. “How many” is **never backtracking**: it is `ways[t] += ways[t-c]`, $O(\text{target} \times n)$ — 1,500 steps instead of 50,000 lists.

1 What this is, and why they ask it

A **file system** is the thing that turns a name like `/home/anu/notes.txt` into the actual bytes. It stores two kinds of thing — **files**, which hold content, and **directories**, which hold other things — and it lets you look them up by a path.

The three sentences that matter: a directory contains files *and other directories*, so the structure is recursive; a **path** is the sequence of names you walk from the root to get to something; and every operation is a walk down that sequence followed by one small action.

They ask it because it is the shortest route to finding out whether a candidate can model a recursive structure without special-casing it. The weak design has a `Directory` class and a `File` class with nothing in common, and then every operation contains `if isinstance(x, Directory)`. The strong design has **one type for “thing that lives in a directory”**, with files and directories both being that thing — and then `ls`, `du`, `find` and `delete` are each about six lines, because the recursion falls out of the structure instead of being written by hand.

It is also a real interview question in its own right — LeetCode 588 is exactly this — and it is the foundation of the harder ones: design Dropbox, design S3, design a search over a hierarchy.

2 The story

The loft above Lakshmi’s kitchen had not been opened properly since her father-in-law died, and she had told herself for two years that she would do it before Deepavali.

She went up on a Sunday morning with a torch. What she found was boxes. Big cardboard cartons, and inside those, smaller boxes, and inside some of those, tins.

There was no way to describe where anything was except by naming the whole chain. The good steel plates were not “in the loft”. They were in the big carton on the left, inside the blue trunk, inside the tin with the flower on the lid. If you left out any part of that, nobody could find them.

She noticed something that afternoon that she had not expected. Two of the tins had exactly the same word written on them — *candles* — and both of them had candles in them. That was not a mistake. One was inside the box of lamps and one was inside the

box of festival decorations, and because they were in different boxes, nobody had ever been confused. It was only a problem if two tins with the same name were sitting inside the *same* box, and that had never happened, because whoever put the second one in would have noticed.

Her son came up in the afternoon and asked how much was up there. She said, I do not know, and there is only one way to find out. You cannot look at a big carton and know what it weighs. You have to open it, and for every box inside it you have to open that one too, and keep going until you reach something that is not a box — a plate, a lamp, a bundle of cloth — and that is the only kind of thing that actually weighs anything on its own. A box weighs whatever is inside it.

He said that sounded like it would take all day. She said it took all day.

By evening they had a full account. Every box, what was in it, and the weight of everything. And the useful thing, she said, was that from now on you could ask about any single box on its own and the answer worked exactly the same way, whether it was the biggest carton or a tin inside a tin.

3 The idea in plain English

Lakshmi's loft is a file system, and every term in the subject is in that story.

- A **file** is a thing that is not a box: a plate, a lamp. It holds content and nothing else.
- A **directory** (also called a folder) is a box: it holds other things, which may themselves be boxes.
- The **root** is the loft itself, written `/`. Everything is inside it.
- A **path** is the chain of names from the root inward: `/loft/carton/trunk/tin`. Written with `/` between the names, and starting with `/` when it starts at the root — an **absolute path**.
- Two tins named *thread* in different boxes is fine; two in the same box is not. That is the only uniqueness rule a file system has: **names are unique within a directory, not globally**.
- “A box weighs whatever is inside it” is the recursive definition of directory size, and it is the reason `du` — the command that reports disk usage — is a six-line function.

The one design decision

A file and a directory are both “an entry in a directory”. Give them a common parent type with a name and a size, and then a directory's children can be a list of that type without caring which kind each one is.

```

class Entry:          # the common type
    name: str
    def size(self) → int: ...

class File(Entry):    # holds content
    content: str

class Directory(Entry): # holds other Entries — including other Directories
    children: dict[str, Entry]

```

This arrangement — **a container and a leaf sharing one interface, so a client can treat a single item and a group of items the same way** — is called the **composite** structure. You have met the idea without the name: a directory is a composite, a file is a leaf, and code that walks them never asks which is which.

The payoff is immediate:

```

class File(Entry):
    def size(self): return len(self.content)

class Directory(Entry):
    def size(self): return sum(child.size() for child in self.children.values())

```

Four lines, and `du` works to any depth. Compare with the design where `Directory` and `File` have no common type: every caller writes `if isinstance(...)`, and adding a third kind of entry — a symbolic link, say — means editing every one of those callers.

Every operation is “resolve, then act”

This is the sentence to say in the interview, because it makes the whole design sound like one idea instead of six methods.

```

resolve("/a/b/c") → walk from root: "a", then "b", then "c"
                    return the Entry, or raise

```

Then:

OPERATION	RESOLVE	ACT
<code>ls(path)</code>	resolve the path	if it is a file, return <code>[its name]</code> ; if a directory, return sorted child names
<code>mkdir(path)</code>	walk the path, creating each missing directory	nothing else
<code>add_content(path, text)</code>	resolve the parent, create the file if absent	append to its content
<code>read(path)</code>	resolve the path	return the content
<code>du(path)</code>	resolve the path	<code>entry.size()</code> — recursion does the rest

`ls` returning the file's own name when the path is a file is the detail everybody forgets, and it is the first thing the LeetCode test cases check. It is not arbitrary — it is what the real `ls` command does.

Why the children are a dictionary

A directory could hold its children in a list. It should not.

list:	look up one child	→	scan every child	$O(\text{number of children})$
dict:	look up one child	→	one hash lookup	$O(1)$

Resolving `/a/b/c/d/e` does five lookups. With a list and a directory holding 100,000 files — which is common, and is what a photo folder looks like — every one of those five is 100,000 comparisons.

The cost is that `ls` must sort, because a dictionary is not ordered by name. Sorting `k` children is $O(k \log k)$, and `ls` is a rare operation while lookup is on every single call. **Optimise the common path; pay on the rare one.** That trade is the answer to “why a dict and not a list?”, and the interviewer is asking it to see whether you can name which operation is hot.

4 The picture

The class diagram.

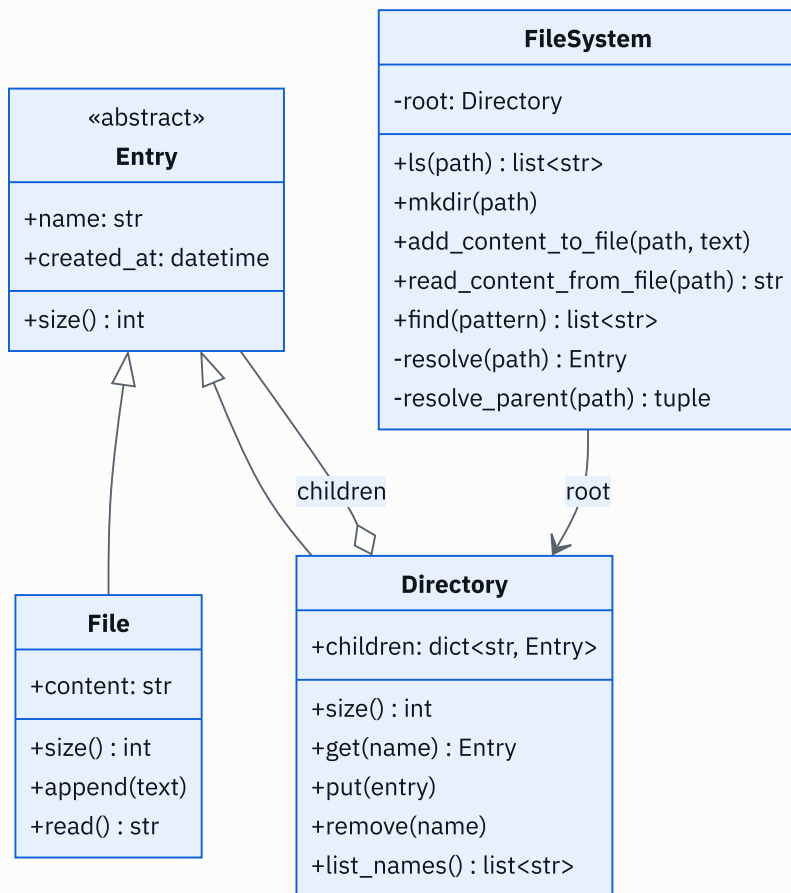


FIGURE 93.1

What to notice: **Directory** holds **Entry**, and **Directory** is itself an **Entry**. That one arrow pointing back at the parent type is the whole design. It is what makes boxes able to contain boxes, and it is what lets `size()` recurse without a single type check.

The tree that comes out of it:



What to notice: **only the leaves have a size of their own.** Every directory's size is the sum of its children's, computed the same way at every level — which is why the code has one method and no depth argument.

And path resolution, step by step:

```

resolve("/home/anu/notes.txt")

parts = ["home", "anu", "notes.txt"]      ← "/" .split gives an empty first element; drop it

current = root
current = current.children["home"]         exists, is a Directory → keep going
current = current.children["anu"]          exists, is a Directory → keep going
current = current.children["notes.txt"]    exists, is a File      → last part, so return it

three lookups, one per name. Depth is the cost, not the total number of files.

```

5 How it actually works

Move 1 — clarify

Four questions, with the answers you will assume. Say them before you draw.

- “*In-memory, or backed by a disk?*” — In-memory. That removes blocks, inodes and the journal, and keeps the question about modelling. I will say what changes for a real disk at the end.
- “*Absolute paths only, or relative ones with . and .. too?*” — Absolute only to start. Relative paths need a working directory and a normalisation step, and I will add that if you want it.
- “*Does mkdir create intermediate directories?*” — Yes, like `mkdir -p`. `mkdir("/a/b/c")` creates all three.

- “Single-threaded, or do I need concurrent access?” — Assume single-threaded for the model, then I will say where the locks go, because that is where this design actually gets hard.

One more, which candidates rarely ask and which changes the answer a lot: “Do I need to support search by name across the whole tree?” If yes, a plain tree walk is $O(\text{number of files})$ per search and you will want a separate structure. Flag it now rather than being ambushed by it.

Move 2 — the nouns

CLASS	RESPONSIBLE FOR
Entry	The common type: it has a name and it has a size. Nothing else.
File	Holding content, and reporting its own length.
Directory	Holding named children, and reporting the sum of theirs.
FileSystem	Turning a path string into an Entry, and the public operations.
Path	Splitting, validating and joining path strings. Small, but keep it separate.

FileSystem owns the root and nothing else. Every method on it is “resolve, then act”, and if a method is longer than eight lines, some of it belongs on **Directory**.

Move 3 — the interesting part

Two places, and both come up.

The composite, because it decides how much code every operation takes. With one shared Entry type, size is four lines, find is six, and deleting a subtree is one dictionary removal — Python frees the whole subtree because nothing references it any more. Without it, every operation has a type check and adding a symbolic link means editing all of them.

Resolution, because that is where every error comes from. Every failure a user ever sees is a resolve failure, and they are not the same failure:

```
/a/b/c  where /a/b does not exist      → FileNotFoundError: no such directory: /a/b
/a/b/c  where /a/b is a FILE           → NotADirectoryError: /a/b is a file
/a/b/c  where c does not exist, on read → FileNotFoundError: no such file: /a/b/c
/a/b/c  where c does not exist, on mkdir → not an error at all – create it
```

Four different outcomes from the same walk, and which one you get depends on the operation. That is why resolve takes a flag, or why there are two of them: one that creates missing directories and one that raises.

Move 4 – the class diagram

Drawn above. Present it by walking one path — “`add_content_to_file('/a/b/note.txt', 'hi')` splits into three parts, resolves `/a/b` as a directory, looks for `note.txt` in its children, creates a `File` if it is not there, and appends” — rather than by reading out the class list.

Move 5 – the code

The entries. This is the part worth writing out properly.

```
from __future__ import annotations

from abc import ABC, abstractmethod
from datetime import datetime, timezone

class Entry(ABC):
    """A file or a directory. The common type is the whole design."""

    def __init__(self, name: str) → None:
        self.name = name
        self.created_at = datetime.now(timezone.utc)

    @abstractmethod
    def size(self) → int:
        """Bytes. A file knows its own; a directory sums its children's."""
```

```
class File(Entry):
    def __init__(self, name: str) → None:
        super().__init__(name)
        self._chunks: list[str] = []          # not one big string – see the trade-offs
        self._length = 0

    def size(self) → int:
        return self._length

    def append(self, text: str) → None:
        self._chunks.append(text)
        self._length += len(text)

    def read(self) → str:
        if len(self._chunks) > 1:             # collapse lazily, on first read
            self._chunks = ["".join(self._chunks)]
        return self._chunks[0] if self._chunks else ""
```

The chunk list is the one non-obvious choice, and it is worth defending out loud: appending to a Python string builds a whole new string each time, so ten thousand appends of 100 bytes copies about five gigabytes in total. A list of chunks makes each append `0(len(text))` and pays the join once, on read.

```

class Directory(Entry):
    def __init__(self, name: str) → None:
        super().__init__(name)
        self.children: dict[str, Entry] = {}    # dict, not list: 0(1) lookup

    def size(self) → int:
        return sum(child.size() for child in self.children.values())

    def list_names(self) → list[str]:
        return sorted(self.children)           # ls is sorted; lookup is not

    def get(self, name: str) → Entry | None:
        return self.children.get(name)

    def put(self, entry: Entry) → None:
        self.children[entry.name] = entry

```

`size()` is the four lines the composite bought you. No `isinstance`, no depth parameter, no explicit recursion beyond the natural one.

The file system itself. Resolution first, because everything else is one line on top of it.

```

class FileSystem:
    def __init__(self) → None:
        self.root = Directory("")

    @staticmethod
    def _split(path: str) → list[str]:
        if not path.startswith("/"):
            raise ValueError(f"path must be absolute: {path!r}")
        return [part for part in path.split("/") if part]    # "/" → []

    def _resolve(self, path: str) → Entry:
        """Walk to the entry, raising if any step is missing or is a file."""
        current: Entry = self.root
        for part in self._split(path):
            if not isinstance(current, Directory):
                raise NotADirectoryError(f"{part}: parent is a file")
            nxt = current.get(part)
            if nxt is None:
                raise FileNotFoundError(f"no such file or directory: {path}")
            current = nxt
        return current

    def _resolve_dir_creating(self, parts: list[str]) → Directory:
        """Walk, creating missing directories. This is `mkdir -p`."""
        current = self.root
        for part in parts:
            nxt = current.get(part)
            if nxt is None:
                nxt = Directory(part)
                current.put(nxt)
            elif not isinstance(nxt, Directory):
                raise NotADirectoryError(f"{part} exists and is a file")
            current = nxt
        return current

```

Two resolvers, because the two behaviours are genuinely different and a boolean flag would make both harder to read.

```
def ls(self, path: str) → list[str]:
    entry = self._resolve(path)
    if isinstance(entry, File):
        return [entry.name]          # the detail everyone forgets
    return entry.list_names()

def mkdir(self, path: str) → None:
    self._resolve_dir_creating(self._split(path))

def add_content_to_file(self, path: str, content: str) → None:
    parts = self._split(path)
    parent = self._resolve_dir_creating(parts[:-1])
    existing = parent.get(parts[-1])
    if existing is None:
        existing = File(parts[-1])
        parent.put(existing)
    elif not isinstance(existing, File):
        raise IsADirectoryError(f"{path} is a directory")
    existing.append(content)

def read_content_from_file(self, path: str) → str:
    entry = self._resolve(path)
    if not isinstance(entry, File):
        raise IsADirectoryError(f"{path} is a directory")
    return entry.read()

def du(self, path: str) → int:
    return self._resolve(path).size() # one line, any depth
```

And search, which is the follow-up:

```
def find(self, path: str, pattern: str) → list[str]:
    """Every path under 'path' whose name matches. 0(entries in the subtree)."""
    import fnmatch

    found: list[str] = []

    def walk(entry: Entry, so_far: str) → None:
        if fnmatch.fnmatch(entry.name, pattern):
            found.append(so_far)
        if isinstance(entry, Directory):
            for name in sorted(entry.children):
                walk(entry.children[name], f"{so_far}/{name}".replace("//", "/"))

    walk(self._resolve(path), path)
    return found
```

find is a full walk of the subtree, and you should say so. There is no shortcut in a plain tree. If search has to be fast, you need a second structure — a map from name to the set of paths holding it, maintained on every create and delete. That is a real trade: it makes writes slower and doubles the memory to make reads instant.

What real systems do this way

- **ext4** on Linux stores directories as **htrees** — hashed B-trees — precisely because a linear directory scan collapses at a hundred thousand files. **NTFS** uses B-trees for the same reason. Your `dict` is the in-memory version of that decision.
- Real file systems separate the **inode** (the metadata and the pointers to the data blocks) from the **directory entry** (the name). That separation is what makes **hard links** possible: two names in two directories pointing at one inode. In this design, two `Directory` entries could reference the same `File` object and it would work — the model already supports it, which is a nice thing to notice out loud.
- **Amazon S3 is the interesting contrast: it has no directories at all.** The key `photos/2024/a.jpg` is one flat string, and the “folders” you see in the console are the console faking it by splitting on `/`. That is why renaming an S3 “folder” with a million objects costs a million copy-and-delete operations, while renaming a directory here is one dictionary key change. **This is the single best example of how the data structure decides what is cheap.**
- **HDFS** keeps the whole namespace — every path, in a structure like the one above — in the memory of a single NameNode. That is why HDFS is famously bad at small files: the limit is not disk, it is the NameNode’s RAM, at roughly 150 bytes of metadata per file.

6 The numbers

Memory

Entry base (name, timestamp, object overhead)	~120 bytes	
File adds the chunk list and length	~ 80 bytes	→ ~200 B per file
Directory adds an empty dict	~120 bytes	→ ~240 B + 100 B per child

For a million files spread over fifty thousand directories:

files	1,000,000 × 200 B	=	200 MB
directories	50,000 × 240 B	=	12 MB
child slots	1,050,000 × 100 B	=	105 MB

metadata alone		≈	317 MB

Three hundred megabytes of metadata for a million empty files, before a single byte of content. That number is exactly why HDFS’s NameNode is the bottleneck of a Hadoop cluster, and quoting it is far better than saying “it uses a lot of memory”.

Lookup cost

```
path "/a/b/c/d/e"          5 parts → 5 dict lookups → ~5 × 50 ns = 250 ns
same path, children in a list, 100,000 files per directory:
                           5 × 100,000 comparisons = 500,000 ops = 5 ms
```

Two hundred and fifty nanoseconds versus five milliseconds — a factor of twenty thousand — and it is one word in the design, `dict` instead of `list`. That is the number to have ready when they ask why a dictionary.

`ls` and `du`

```
ls on a directory of k children    sort k names          O(k log k)
    k = 100,000                   ~1.7 million comparisons ≈ 20 ms
du on a subtree of m entries       visit every one        O(m)
    m = 1,000,000                 ≈ 300 ms
```

`du` on a big subtree is slow and there is no way around it in this design. If it is called often, cache the size on each directory and invalidate up the chain of parents on every write — which means every `Entry` needs a `parent` reference, and every append walks up to the root. **That trade turns a 300 ms read into an $O(\text{depth})$ write cost of about ten pointer hops.** Worth it if reads outnumber writes, which for `du` they usually do not.

Search

```
find("/", "*.jpg") over 1,000,000 entries    one full walk          = 400 ms
with a name → paths map maintained on write  one dict lookup        = 1 μs
cost of the map: ~1,000,000 extra entries    = 150 MB, plus a write on every create/delete
```

A factor of four hundred thousand on reads, paid for with 150 MB and slower writes. State it as that trade, not as “we could add an index”.

Concurrency

Two things happen at once and both have a right answer.

- 1. Two threads call `mkdir("/a/b")` at the same time.** Both see `b` missing, both create a `Directory`, and one overwrites the other — silently discarding whatever the loser had already put inside it. The fix is not a global lock; it is **creating under the parent’s lock**, or using `dict.setdefault`, which is atomic in CPython and returns the winner’s object.
- 2. One thread reads a file while another appends.** With the chunk list, a reader that collapses the chunks while a writer appends can lose data. **A per-file lock around append and read** is enough, and per-file rather than global is the point — a million files are a million independent locks and there is no contention between them.

The rule to say out loud: **lock the smallest thing that makes the operation correct**. A single global lock on the file system makes every operation correct and makes the whole thing single-threaded, which at a thousand operations a second is fine and at a hundred thousand is not.

7 The trade-offs

`dict` of children, or a sorted list?

`dict` gives $O(1)$ lookup and unordered `ls`, so `ls` pays $O(k \log k)$ to sort. A sorted list gives $O(\log k)$ lookup by binary search, free ordering, and $O(k)$ insertion because everything after the insertion point shifts.

Take the dict. Lookup happens on every path component of every operation; `ls` is rare. **I would not use a dict if the dominant operation were listing directories in order** — a file browser back-end, say — where a sorted structure or a tree map would pay for itself.

One string per file, or a list of chunks?

One string is simpler and reads are free. But `content += text` builds a new string each time:

```
10,000 appends of 100 bytes to one string:
  total copying = 100 + 200 + ... + 1,000,000 ≈ 5 GB copied
with a chunk list:
  total copying = 1 MB, plus one join on the first read
```

Take the chunks. The exception is a file that is written once and read a thousand times, where the join cost is noise and the simplicity wins.

Store the size, or compute it?

Computing means `du` is $O(\text{subtree})$ and writes are free. Storing means `du` is $O(1)$ and every write walks up to the root updating parents, which also forces a `parent` reference on every entry and makes moving a subtree an update of two chains.

Compute it, unless `du` is a listed requirement. Say the alternative and its cost; that is the answer, not choosing.

One tree, or a tree plus a search structure?

Covered in the numbers. The honest framing: **a tree is optimised for “where is this exact path”, and search is the query it is worst at**. Everything that makes search fast is a second structure that duplicates information and has to be kept in step — which is the same trade as an index on a database table, from [day 025](#).

Where this design breaks

- **Relative paths, `.` and `..`.** These need a per-user working directory and a normalisation pass before resolution. `..` is genuinely awkward here because entries do not know their parent — you would add a `parent` pointer, and then you must keep it correct on every move.
- **Permissions.** A real file system checks read/write/execute for a user at **every level** of the walk, not only at the target. That turns `_resolve` from five lines into fifteen and is the reason path resolution is a security-sensitive operation.
- **Persistence.** The moment this has to survive a restart, everything changes: you need blocks, a free-space map, and a **journal** so that a crash halfway through a rename does not leave a directory pointing at nothing. That is the real subject of file systems and it is not what this question is asking.
- **Very wide directories.** A million files in one directory makes `ls` cost 20 million comparisons and the child dict alone 100 MB. Real systems answer this with B-trees on disk; in memory you would page the listing rather than sort it all.
- **Moving a subtree.** Cheap here — one key removed, one key added — and this is worth saying, because it is exactly what S3 cannot do, and the contrast makes the point that the structure decides the cost.

8 In the interview

How it gets asked

- The direct version: “Design an in-memory file system with `ls`, `mkdir`, `addContentToFile` and `readContentFromFile`.” LeetCode 588.
- The modelling probe: “How do you represent a file and a directory?” — they want to hear one shared type.
- The recursion probe: “Now give me the total size of a directory.”
- The search probe: “Find all files matching `*.log` under this path.”
- The scale probe: “A directory has a million files. What breaks?”
- The contrast: “How is this different from S3?”

What to say out loud, in the first ninety seconds

1. **Say the model in one sentence.** “Two kinds of entry — file and directory — sharing one abstract type with a name and a size. A directory holds a map from name to entry, and since a directory is an entry, directories nest naturally.”

2. **Say why the shared type matters.** “That is what makes `du` four lines and `find` six, with no type checks. Without it every operation would have an `isinstance` in it and adding symbolic links later would mean editing all of them.”
3. **Name the one repeated operation.** “Every method is ‘resolve the path, then do one small thing’. Resolution is a walk down the name list, one dictionary lookup per component.”
4. **Say `dict`, and why.** “Children in a dictionary, not a list — lookup is on every component of every call. A hundred thousand files in a directory turns a 250-nanosecond resolve into a five-millisecond one if it is a list.”
5. **State the uniqueness rule.** “Names are unique within a directory, not globally. Two files called `notes.txt` in different directories is normal.”
6. **Flag the error cases.** “Resolution has four outcomes, not two: missing directory, a file where a directory was expected, missing final component — and for `mkdir` that last one is not an error at all, it is the work.”

The follow-ups

“**Now give me the total size of a directory.**” “One method on `Entry`. `File.size()` returns its own content length; `Directory.size()` returns the sum of its children’s sizes. Because both are `Entry`, the sum does not care what the children are, and it recurses to any depth with no explicit depth handling. That is the payoff of the shared type — four lines instead of a walker with type checks. The cost is `O(entries in the subtree)`, so on a million entries it is a few hundred milliseconds. If `du` were a hot path I would cache the size on each directory and invalidate upward on writes, which needs a parent pointer on every entry and turns a slow read into about ten pointer hops per write.”

“**Find all files matching a pattern.**” “In this design it is a full walk of the subtree — there is no shortcut, and I would say that plainly rather than pretend. If search has to be fast, the answer is a second structure: a map from name, or from extension, to the set of full paths, updated on every create, delete and move. That takes a 400-millisecond walk down to a microsecond lookup, and it costs about 150 MB per million files plus a write on every mutation. It is the same trade as adding an index to a database table — faster reads, slower writes, duplicated state that can drift.”

“**A directory has a million files. What breaks?**” “Three things, in order. `ls` has to sort a million names — around twenty million comparisons, so a few hundred milliseconds, and it returns a million-element list to a caller who almost certainly wanted the first page. The child dictionary alone is around 100 MB. And if I had used a list instead of a dict, every single lookup in that directory would be a million comparisons. The fixes are: paginate `ls` with a cursor instead of returning everything, keep the dict for lookup,

and if ordered listing is genuinely needed, hold a sorted structure alongside. Real file systems solve this with B-trees on disk — ext4's htree exists exactly because linear directory scans died at this scale.”

“How is this different from S3?” “S3 has no directories. The key `photos/2024/a.jpg` is one flat string in a giant sorted map, and the folders you see in the console are the console splitting on slashes. The consequence is the interesting part: renaming a directory here is one dictionary key change, no matter how much is inside it, because the children hang off the entry. Renaming an S3 ‘folder’ with a million objects is a million copy operations followed by a million deletes, because there is nothing to rename — the prefix is part of every key. Same user-visible concept, completely different cost, decided entirely by the data structure.”

“How would you make this thread-safe?” “Not with one global lock, though I would say that is the correct first version and it is fine up to maybe ten thousand operations a second. The two real races are: two threads creating the same directory, where both see it missing and one silently overwrites the other along with everything already inside it — fixed by creating under the parent’s lock, or `dict.setdefault`, which is atomic; and a reader collapsing a file’s chunks while a writer appends — fixed by a per-file lock. The rule is to lock the smallest thing that makes the operation correct, because a million files then means a million independent locks with no contention between them.”

“What changes if this has to survive a restart?” “Almost everything, and it stops being this question. You need blocks and a free-space map instead of Python objects, and you need a **journal**: write the intent to a log, then perform the change, so a crash mid-way leaves something recoverable rather than a directory pointing at nothing. That is what ext4’s journal and NTFS’s log are for. I would keep the same logical model — entries, names, a tree — and change only the storage underneath it, which is a good sign the model was right.”

A model answer

Asked: *design an in-memory file system.*

“The model first, because everything follows from it. There are two kinds of thing: a **file**, which holds content, and a **directory**, which holds other things. The key decision is that they share one abstract type — call it `Entry` — with a name and a `size()`. A directory’s children are a map from name to `Entry`, and since a directory is an `Entry`, directories nest without any special handling.

That shared type is not tidiness; it is what makes the operations short. `File.size()` returns its content length, `Directory.size()` returns the sum of its children’s — four lines, and `du` works to any depth with no type checks anywhere. Without the shared type, every operation would carry an `isinstance`, and adding symbolic links later would mean editing all of them.

Every public method is then the same two steps: **resolve the path, then do one small thing**. Resolution splits the string on slashes and walks down, one lookup per component. `ls` resolves and either returns the sorted child names or — if the path is a file — the file’s own name, which is the detail everybody forgets and the first thing the tests check. `mkdir` walks the same way but creates what is missing. `addContentToFile` resolves the parent, creates the file if absent, and appends.

Two implementation choices I would defend. **Children in a dictionary, not a list**, because lookup happens on every component of every call: with a hundred thousand files in a directory, a list makes resolving a five-part path five hundred thousand comparisons — about five milliseconds instead of two hundred and fifty nanoseconds. The price is that `ls` has to sort, which is fine because listing is rare. And **file content as a list of chunks rather than one string**, because `content += text` copies the whole thing every time — ten thousand appends of a hundred bytes copies about five gigabytes. The chunks are joined once, lazily, on the first read.

The uniqueness rule is worth stating: names are unique **within a directory**, not globally. Two files called `notes.txt` in different directories is normal and must work.

On scale, a million files is roughly three hundred megabytes of metadata before any content — which is exactly why HDFS keeps its whole namespace in one NameNode’s RAM and why that machine is the limit of the cluster. And search is the weak point: `find` is a full walk of the subtree, and if it needs to be fast, that is a second structure mapping names to paths, kept in step on every write — faster reads, slower writes, duplicated state.

The contrast I would offer is S3, because it makes the point about structure. S3 has no directories at all; the whole path is one flat key. So renaming a directory here is one key change regardless of size, and renaming an S3 prefix with a million objects

is a million copies and a million deletes. Identical concept to the user, completely different cost, decided by the data structure alone.”

9 Recall card

- One abstract `Entry` with a name and `size()`; `File` and `Directory` both extend it, and a `Directory` holds a `dict[str, Entry]` — so a directory can hold directories. That shared type is the design: `du` is four lines and `find` is six, with no `isinstance` anywhere.
- Every operation is “resolve the path, then do one small thing.” Resolution is one dict lookup per component. It has **four outcomes, not two**: missing directory, a file where a directory was expected, missing final component — and for `mkdir` that last one is the work, not an error. And `ls` on a file returns the file’s own name — the detail everybody forgets.
- `dict` of children, not a list. Lookup is on every component of every call: 100,000 files in a directory makes a five-part resolve **5 ms instead of 250 ns**. The price is that `ls` sorts, which is the rare operation. **Names are unique within a directory, not globally.**
- **File content is a list of chunks, joined lazily** — `content += text` copies the whole string each time, about **5 GB of copying for 10,000 appends of 100 bytes**. Lock **per file**, not globally; the two real races are two `mkdir`s of the same directory (one silently discards the other’s contents) and a read collapsing chunks during an append.
- **A million files $\approx$ 300 MB of metadata before any content** — the reason HDFS’s NameNode is the limit of a Hadoop cluster. `find` is a **full subtree walk**; making it fast means a second name→paths map, $\sim$ 150 MB per million and a write on every mutation. **S3 has no directories** — the whole path is one flat key — so renaming a folder is $O(1)$ here and a million copy-and-deletes there.

DSA topic: Combinations and combination sum **System design topic:** Design a file system

Code these, in this order

One rule for the whole set: **before you type the recursive call, say out loud whether the same value may be used again.** `i` if yes, `i + 1` if no. That one character is the difference between two LeetCode problems, and it fails silently.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Combinations	LeetCode 77 (Medium)	The <code>start</code> index, and the size prune that is a <code>break</code> .
2	Combination Sum	LeetCode 39 (Medium)	<code>build(i, ...)</code> for reuse, and the sorted <code>break</code> on too-big.
3	Combination Sum III	LeetCode 216 (Medium)	Two constraints at once: a fixed size <i>and</i> a target.
4	Combination Sum II	LeetCode 40 (Medium)	Two skips, two keywords — <code>break</code> for too big, <code>continue</code> for duplicate.

On problem 1, measure the prune

Write it without the “not enough elements left” check, then with it. Count the calls both ways for choosing 18 from 20 and record both numbers. Then say why the check is a `break` and not a `continue`.

On problem 2, change one character and run it again

Run it correctly, then change `build(i, ...)` to `build(i + 1, ...)` and run `[2,3,6,7]` with target 7. Record both outputs. Then remove the `sort()` and run `[7,2,6,3]` with target 7, and say which answer disappeared and why the `break` caused it.

On problem 3, name both prunes

There is a target prune and a size prune. Write both, and say which one is a `break` and which one a `return`.

On problem 4, swap the two keywords deliberately

Run `[1,1,2,5,6,7,10]` with target 8 correctly — four answers. Then change the duplicate `continue` to a `break` and record what you get. Then change the too-big `break` to a `continue` and say what changed in the output versus what changed in the running time.

The definition drill

1. Say the difference between a combination and a permutation in one sentence.
2. Say what `start` guarantees about the order every answer is generated in.
3. Say why that guarantee means you never have to de-duplicate the result.
4. Say what `set()` on a list of lists does, and quote the error.

The sort drill

1. Give the reason for sorting that is *not* about tidy output.
2. Run `combination_sum([7,2,6,3], 7)` unsorted, with the break. Record the output.
3. Say which answer was lost and at which step.
4. Name the second thing in this lesson that also requires the sort.

The one-character drill

1. Say which problem uses `build(i, ...)` and which uses `build(i + 1, ...)`, and why.
2. Run `combination_sum([2,3], 6)` and `combination_sum_ii([2,3], 6)`. Explain both outputs.
3. Use `i` in a version where elements may only be used once, on `[1,1,2]` with target 2, and describe what is wrong with the answer.
4. Say what happens to that version if a candidate is `1` and the target is large.

The two-keywords drill

1. Write the too-big check and say why it is a `break`.
2. Write the duplicate check and say why it is a `continue`.
3. Say what “monotone” means here, and use it to decide which keyword any new prune should get.
4. Swap them both and record the two different kinds of damage.

The duplicate-rule drill

1. Write the Combination Sum II skip from memory.
2. Say what `i > start` means about position in the tree.
3. Change it to `i > 0`, run `[1,1,6]` with target 8, and say which box disappears.
4. State the permutations version of this rule, and say in one sentence why the two differ.

5. Name the three problems so far where this rule has appeared in a different outfit.

The termination drill

1. Name the measure that decreases in combination sum, and say why it must.
2. Put a `0` in the candidates and run it. Quote the error.
3. Say why every LeetCode statement guarantees positive candidates.
4. Run `combination_sum([1,2,5], 5000)` and quote the error.
5. Say what makes this problem's depth different from every other one in the phase.

The cost drill

1. State the number of answers for choose-k, and the worst `k` for a given `n`.
2. Compute `C(20,10)`, `C(30,15)` and `C(40,20)`.
3. State the depth and branching factor for combination sum, and give the loose bound.
4. Say honestly what the real cost tracks, and why the bound is loose.
5. State the extra space and say what it depends on.

The counting drill

1. Write the counting version with a one-dimensional array.
2. Say what `ways[0] = 1` means.
3. Say which loop must be on the outside to count combinations rather than orderings.
4. Compare the step count against enumerating, for `[1,2,5]` and target 500.
5. Say what word in a question should make you switch to this immediately.

The model drill

1. Name the three classes and say which one is abstract.
2. Say why `Directory` holding `Entry` — rather than holding `File` and `Directory` separately — is the whole design.
3. Write `size()` for both concrete classes.
4. Say what code you avoid writing by having the shared type, and name a change it makes cheap.

The resolve drill

1. Write `_split` and say what it does with `"/"`.
2. Write `_resolve` and name the four possible outcomes of a walk.
3. Say which outcome is not an error for `mkdir`.
4. Say what `ls` returns when the path names a file, and why.

5. Say how many lookups resolving `/a/b/c/d/e` costs.

The dict-versus-list drill

1. State the lookup cost of each.
2. Compute the resolve time for a five-part path with 100,000 children per directory, both ways.
3. Say what the dict costs you, and on which operation.
4. Say which operation is hot and which is rare, and use that to justify the choice.
5. Name the real file system that made the same decision and what it uses on disk.

The content drill

1. Say why file content is a list of chunks rather than one string.
2. Compute the total bytes copied by 10,000 appends of 100 bytes into one string.
3. Say when you would prefer the single string instead.
4. Say where the join happens and why it is lazy.

The numbers drill

Compute each, showing the multiplication:

1. Metadata memory for a million files across fifty thousand directories.
2. `ls` cost on a directory of 100,000 children.
3. `du` cost on a subtree of a million entries.
4. `find` cost with and without a name→paths map, and the memory that map costs.
5. Say which real system's bottleneck is exactly number 1, and why.

The failure drill

For each, say what happens and what you would add:

1. Two threads call `mkdir("/a/b")` at the same instant.
2. A thread reads a file while another appends to it.
3. `ls` is called on a directory with a million files.
4. Someone asks to rename a directory containing a million files — here, and in S3.
5. `addContentToFile` is called with a path whose parent is a file.
6. The process restarts.

Two of the six are not solvable inside this design as drawn. Name them and say what you would build.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find all combinations that sum to the target.* The definition of combination first, the `start` index and what it guarantees, the sort with the reason that is about the `break`, the one character that allows reuse, why it terminates, and an honest complexity with the depth-is-target caveat.
 2. *Design an in-memory file system.* The shared `Entry` type and what it buys, “resolve then act” as the shape of every method, the dict with the 5 ms versus 250 ns number, `ls` on a file returning its own name, and the S3 contrast about renaming.
 3. *Now each number may be used only once, and the input has duplicates.* `i + 1`, sort, and `i > start` — then the part that separates people: two skips in one loop with two different keywords, and what each wrong keyword costs.
-

Before you move on

- ☐ I can define combination against permutation in one sentence.
- ☐ I can say what the `start` index guarantees about generation order.
- ☐ I can give the reason for sorting that is about the `break`.
- ☐ I ran the unsorted version and know which answer it loses.
- ☐ I can say which of `i` and `i + 1` allows reuse, without hesitating.
- ☐ I can write both skips in Combination Sum II with the right keyword each.
- ☐ I swapped the keywords and recorded both kinds of damage.
- ☐ I can state the duplicate rule in all three outfits and say which tree each belongs to.
- ☐ I can name the measure that terminates combination sum, and what a `0` candidate does.
- ☐ I can say why the depth here is not bounded by `n`, and quote the error it causes.
- ☐ I can write the counting version and say which loop goes on the outside.
- ☐ I can name the three file system classes and write `size()` for both concrete ones.
- ☐ I can say what “resolve then act” means and name the four outcomes of a walk.
- ☐ I can justify the dict with the 5 ms versus 250 ns numbers.
- ☐ I can say what `ls` returns for a file path.
- ☐ I can state the metadata memory for a million files and name the system it limits.
- ☐ I can explain the S3 renaming contrast in two sentences.
- ☐ I answered all three questions above out loud.

Backtracking: the undo step

DATA STRUCTURES AND ALGORITHMS

Backtracking: the undo step

You can add, recurse, and remove, and say why the removal is the whole pattern.

SYSTEM DESIGN

Design snake and ladder

You can model the board, the dice and the players with clean separation.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Why did you pop that element after the recursive call?*
- *Design snake and ladder for n players.*

1 What this is, and why they ask it

Backtracking is depth-first search over a tree of choices, where you make a choice, explore everything that follows from it, and then **put the state back exactly as it was** before making the next choice.

You have written the pattern three times already — [subsets](#), [permutations](#), [combination sum](#) — and each time the un-choose was one line at the bottom of the loop. Today is about that line on its own, because it is the line the interviewer asks about, and because every remaining problem in this phase has *more state to undo* than a single list.

The three sentences that matter. **Anything you change before the recursive call must be changed back after it.** That includes the partial answer, the availability marks, and every derived counter you are keeping. And the alternative — passing a fresh copy down instead of mutating and restoring — is correct too, costs `O(depth)` extra allocation per node, and is worth knowing so that you can say why you did not choose it.

They ask “*why did you pop that element?*” because the answer separates two kinds of candidate. One says “to clean up”. The other says “because `current` is a single object shared by every branch of the tree, and the pop is what makes it correct for the next branch — without it the sibling branch inherits a choice it never made.” Only the second person has understood what they wrote.

2 The story

Meena decided on a Sunday morning that the hall was wrong and had been wrong for four years.

There were three big things in it — the sofa, the low wooden table, and the corner shelf with the television on it — and four places they could sensibly go: the wall with the window, the wall by the kitchen door, the long blank wall, and the corner near the passage.

Her plan was to try every way round, properly, and then pick.

She started with the sofa on the window wall. Then, with the sofa there, she tried the table by the kitchen door, and with those two settled she put the shelf on the long wall and stood at the doorway and looked. Too dark. She moved the shelf to the corner

instead and looked again. Better, but the television faced the window and you would see nothing in the afternoon.

Then she wanted to try the table somewhere else. And this is the part she had learned from doing it badly the year before.

Before she moved the table, she carried the shelf back to where it had been standing when she started. Not roughly. Exactly. Because if she left the shelf in the corner and then moved the table, she would be looking at an arrangement that was half of one idea and half of another, and she would have no way of telling which of the two changes had made it worse.

She did the same with the sofa. When she had finished every arrangement with the sofa on the window wall, she put the table and the shelf back where they had been at the start, and only then moved the sofa to the kitchen wall.

Her husband came in at about eleven, saw the sofa in a new place and the room otherwise untouched, and asked whether she had actually done anything.

She said she had done nine of them so far, and that the room looking normal in between was the only reason she could tell one from another.

By two o'clock she had been through all of them. The room, at that moment, looked exactly as it had at nine in the morning. Then she put the sofa on the long wall, the table in front of it, the shelf in the corner, and stopped.

3 The idea in plain English

Meena has just run a backtracking search, and the thing she learned the hard way is the whole lesson.

- Each big piece of furniture is a **level** of the recursion. The sofa is decided first, then the table, then the shelf.
- Putting a piece somewhere is the **choose** step.
- Trying everything that follows from that placement is the **recurse** step.
- **Carrying it back to exactly where it was is the un-choose step**, and her reason is precisely the technical reason: without it, the next branch starts from a state that no branch actually chose.

The template, and it never changes

```
def explore(state):
    if is_complete(state):
        record(copy_of(state))
        return
    for choice in options(state):
        if not is_valid(choice, state):
            continue
        apply(choice, state)
        explore(state)
        undo(choice, state)
```

Seven lines. **Every problem for the rest of this phase is this template with different `options`, `is_valid` and `apply`.** N-Queens, Sudoku, word search, palindrome partitioning — all of them.

What counts as “state”

This is the part people get wrong once they move past subsets, because in subsets there was only one thing to undo.

State is anything the recursion can see that you changed. It comes in three kinds:

1. **The partial answer.** The working list — `current`. Undone with `pop()`.
2. **Availability marks.** `used[i] = True`, `visited[row][col] = True`, `board[r][c] = digit`. Undone by setting them back.
3. **Derived bookkeeping.** A running sum, a set of occupied columns, a count of remaining blanks. Undone by subtracting, discarding, incrementing.

The third kind is where the bugs live, because it does not look like state. If you write `running_total += value` before the call, you owe a `running_total -= value` after it — unless you passed it as an argument instead, which is the trick below.

The rule, in one sentence: for every line before the recursive call that changes something the recursion can see, there is a line after it that changes it back. Count them. They should match.

Two styles, and why the mutating one wins

Style A — mutate and undo. One shared object, restored after every branch.

```
current.append(x)
explore(...)
current.pop()
```

Style B — pass a fresh copy. Nothing is shared, so nothing needs undoing.

```
explore(current + [x])
```

```
# a NEW list; the caller's is untouched
```

Style B is shorter and cannot be got wrong. It also allocates a new list of length up to `k` at **every node of the tree**, not every leaf. For subsets at $n = 20$ that is two million allocations averaging ten elements each — twenty million element copies that style A does not do.

Write style A in an interview, and say that style B exists. The sentence to have ready: *“I mutate one working list and undo, because copying at every node would multiply the work by the depth. The copy-based version is easier to get right and I would use it if the state were small or if I needed the partial answers to be immutable.”*

Note the exception that makes style B free: **immutable values need no undo**. If you pass an integer, a string, or a tuple, the callee physically cannot change yours.

```
explore(index + 1, remaining - value)
explore(prefix + character)
```

```
# nothing to undo: ints are values
```

```
# nothing to undo: strings are immutable
```

That is why `combination_sum` never undoes `remaining` — it was passed as an argument, not mutated in place. **Passing by argument is the undo**. Recognising which pieces of state you can move into the argument list is the single best way to reduce the number of things you can forget.

What makes it “backtracking” rather than “enumeration”

One line: `if not is_valid(...): continue`.

Subsets and permutations walk the whole tree — every leaf is an answer, nothing is rejected. That is **enumeration**. Backtracking proper adds a check *before* recursing, so branches that cannot lead to a valid answer are never walked at all. That check is called **pruning**, and it is where all the speed comes from.

```
N-Queens, n = 8:
```

```
all placements, no check: 8^8 = 16,777,216 leaves
```

```
one queen per row:      8! =      40,320
```

```
with the column/diagonal check while descending: 2,057 nodes visited
```

Sixteen million down to two thousand, from one `if`. The check has to happen *before* the recursive call — checking at the leaf gives you the 40,320, checking on the way down gives you the 2,057.

Where the undo goes, exactly

Immediately after the recursive call, inside the loop, before the next iteration.


```
for choice in options:
    apply(choice)
    explore()
    undo(choice)                                # ← here. Not after the loop, not at the top.
```

Putting it after the loop undoes only the last choice. Putting a “reset everything” at the top of the function is worse: it works, hides the pairing, and quietly costs `0(state)` per node.

4 The picture

The state of `current` through a small search, with the undo marked. This is the diagram to be able to draw from memory.

subsets of [1, 2, 3], template form

depth	action	current	result
0	record	[]	[[]]
0	choose 1	[1]	
1	record	[1]	[[], [1]]
1	choose 2	[1,2]	
2	record	[1,2]	[[], [1], [1,2]]
2	choose 3	[1,2,3]	
3	record	[1,2,3]	[[], [1], [1,2], [1,2,3]]
2	UNDO 3	[1,2]	← back to depth 2's state
1	UNDO 2	[1]	← back to depth 1's state
1	choose 3	[1,3]	
...			

Notice: after every UNDO, `current` is EXACTLY what it was when that level started. That is the invariant, and it is the only one.

The invariant, stated properly:

+-----+

| When explore() returns, every piece of shared state is |

| byte-for-byte what it was when explore() was called. |

+-----+

If that holds at every level, the tree is correct.
If it fails at ONE node, every sibling after it is wrong.

What the failure looks like, drawn:

permutations of [1,2,3] with `used[i] = False` forgotten

build()	used = [F,F,F]
take 1	used = [T,F,F]
take 2	used = [T,T,F]
take 3	used = [T,T,T] → record [1,2,3]
return	used = [T,T,T] ← NOT restored
try to take 3 here	used[2] is True → skipped
nothing available	→ branch dies with no answer

One missing line, and 5 of the 6 answers vanish. No error is raised.

And the two styles, side by side:

style A: mutate + undo

current.append(x)
explore()
current.pop()

one list, reused
O(1) per node
O(depth) total memory
must not forget the undo

style B: pass a copy

explore(current + [x])

one new list per NODE
O(len(current)) per node
O(depth^2) live at once
cannot be got wrong

5 The code, built step by step

Step 1 — write the three lines as one unit, always

```
apply(choice)
explore(...)
undo(choice)
```

Type all three before you type anything inside them. It is a physical habit, and it is the reason experienced people do not forget the undo: they never wrote the two-line version in the first place.

Step 2 — count your chooses and your undoes

Before running anything, look at the block and count. Two lines that change shared state before the call means two lines after it.

```

used[i] = True           # change 1
current.append(items[i]) # change 2
build()
current.pop()           # undo 2
used[i] = False         # undo 1

```

Undo in reverse order. It does not matter for independent state, and it matters enormously for anything stacked — and doing it in reverse always is one less thing to think about.

Step 3 — move what you can into the arguments

```

build(i + 1, remaining - candidates[i])    # nothing to undo

```

versus

```

self.remaining -= candidates[i]           # needs an undo
build(i + 1)
self.remaining += candidates[i]

```

Both correct. The first cannot be got wrong. **Every piece of state you can pass as a value is a piece of state you cannot forget to restore**, and it costs nothing when the value is an integer or a string.

Step 4 — the validity check goes *before* the apply

```

for choice in options:
    if not is_valid(choice):
        continue                # prune: never walked at all
    apply(choice)
    explore()
    undo(choice)

```

Not inside `explore`, and not at the leaf. Checking on the way down is what takes N-Queens from forty thousand leaves to two thousand nodes.

Step 5 — the early-return trap

Search problems return `True` the moment they find an answer, and that `return` skips the undo:

```

apply(choice)
if explore():
    return True                # ← the undo below never runs
undo(choice)

```

This is correct if you are unwinding all the way out and never touching the state again, which is the usual case for “does a solution exist”. It is a bug if the caller inspects the state afterwards, or if the same board is reused for a second query. When in doubt, undo before returning:

```
    apply(choice)
    found = explore()
    undo(choice)
    if found:
        return True
```

Two extra lines, and it removes an entire class of “works the first time, fails the second time” bugs.

The complete solution

A generic template plus the three problems you already know, all written against it, so you can see that they are the same code.

```

from typing import Any, Callable, Iterable

def backtrack(
    is_complete: Callable[[Any], bool],
    record: Callable[[Any], None],
    options: Callable[[Any], Iterable[Any]],
    is_valid: Callable[[Any, Any], bool],
    apply: Callable[[Any, Any], None],
    undo: Callable[[Any, Any], None],
    state: Any,
) → None:
    """The seven-line pattern, written once.

    You would not literally use this in an interview – you would inline it –
    but writing it out once makes the shape impossible to forget.
    """
    if is_complete(state):
        record(state)
        return
    for choice in options(state):
        if not is_valid(choice, state):
            continue # PRUNE – the branch is never walked
        apply(choice, state) # CHOOSE
        backtrack(is_complete, record, options, is_valid, apply, undo, state)
        undo(choice, state) # UN-CHOOSE

def subsets(items: list[int]) → list[list[int]]:
    """One kind of state: the partial answer. One undo."""
    result: list[list[int]] = []
    current: list[int] = []

    def explore(start: int) → None:
        result.append(current[:]) # every node is an answer
        for i in range(start, len(items)):
            current.append(items[i]) # choose
            explore(i + 1) # recurse
            current.pop() # un-choose

    explore(0)
    return result

def permutations(items: list[int]) → list[list[int]]:
    """Two kinds of state: the partial answer AND the availability marks.
    Two chooses, two undoes, undone in reverse order."""
    result: list[list[int]] = []
    current: list[int] = []
    used = [False] * len(items)

    def explore() → None:
        if len(current) == len(items):
            result.append(current[:])
            return
        for i in range(len(items)):
            if used[i]:
                continue # prune: already placed
            used[i] = True # choose 1
            current.append(items[i]) # choose 2
            explore()
            current.pop() # undo 2
            used[i] = False # undo 1

```

```

explore()
return result

def path_exists(grid: list[list[int]], target: tuple[int, int]) → bool:
    """Three kinds of state, and the early-return trap made safe.

    The `visited` grid is marked on the way in and cleared on the way out, so
    a cell blocked on one path is free on another. Forget the clear and you
    are computing "is there a path that never revisits a cell ACROSS ALL
    ATTEMPTS", which is not a question anybody asked.
    """
    rows, cols = len(grid), len(grid[0])
    visited = [[False] * cols for _ in range(rows)]

    def explore(r: int, c: int) → bool:
        if (r, c) == target:
            return True
        visited[r][c] = True
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if not (0 ≤ nr < rows and 0 ≤ nc < cols):
                continue
            if visited[nr][nc] or grid[nr][nc] == 1:
                continue
            if explore(nr, nc):
                visited[r][c] = False
                return True
        visited[r][c] = False
        return False

    return explore(0, 0)

def subsets_by_copy(items: list[int]) → list[list[int]]:
    """Style B: pass a fresh list down, so there is nothing to undo.

    Correct, shorter, and impossible to get wrong. It also allocates a new
    list at EVERY NODE – 2^n of them for subsets – where style A allocates
    only at the leaves. Know it; usually do not write it.
    """
    result: list[list[int]] = []

    def explore(start: int, current: list[int]) → None:
        result.append(current)
        for i in range(start, len(items)):
            explore(i + 1, current + [items[i]])

    explore(0, [])
    return result

def permutations_broken(items: list[int]) → list[list[int]]:
    """The failure, written out so you can run it: one missing undo."""
    result: list[list[int]] = []
    current: list[int] = []
    used = [False] * len(items)

    def explore() → None:
        if len(current) == len(items):
            result.append(current[:])
            return
        for i in range(len(items)):
            if used[i]:

```

```

        continue
        used[i] = True
        current.append(items[i])
        explore()
        current.pop()
        # missing: used[i] = False

    explore()
    return result

def count_nodes_with_and_without_pruning(n: int) → tuple[int, int]:
    """N-Queens, counted both ways, so the number is yours and not a claim.

    Without pruning: place a queen in every row, check only at the bottom.
    With pruning:    reject a column or diagonal clash before descending.
    """
    without = 0
    with_prune = 0

    def blind(row: int, cols: list[int]) → None:
        nonlocal without
        without += 1
        if row == n:
            return
        for c in range(n):
            cols.append(c)
            blind(row + 1, cols)
            cols.pop()

    def pruned(row: int, cols: set[int], diag: set[int], anti: set[int]) → None:
        nonlocal with_prune
        with_prune += 1
        if row == n:
            return
        for c in range(n):
            if c in cols or (row - c) in diag or (row + c) in anti:
                continue # THE line that does all the work
            cols.add(c); diag.add(row - c); anti.add(row + c)
            pruned(row + 1, cols, diag, anti)
            cols.remove(c); diag.remove(row - c); anti.remove(row + c)

    blind(0, [])
    pruned(0, set(), set(), set())
    return without, with_prune

if __name__ == "__main__":
    print(len(subsets([1, 2, 3])), len(subsets_by_copy([1, 2, 3]))) # 8 8

    print(permutations([1, 2, 3]))
    # [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

    print(permutations_broken([1, 2, 3]))
    # [[1, 2, 3]] ← one missing line, five answers gone, no error

    maze = [
        [0, 0, 1, 0],
        [1, 0, 1, 0],
        [0, 0, 0, 0],
        [0, 1, 1, 0],
    ]

    print(path_exists(maze, (3, 3))) # True

```

```
print(count_nodes_with_and_without_pruning(6))
# (55987, 543) ← the whole argument for checking on the way down
print(count_nodes_with_and_without_pruning(8))
# (19173961, 2057)
```

6 What it costs

The undo itself

```
current.pop()          0(1)
used[i] = False        0(1)
cols.remove(c)         0(1)  - set removal
board[r][c] = "."      0(1)
```

Every undo in this phase is $O(1)$. That is not a coincidence — it is a design requirement. If your undo is $O(k)$, you have chosen the wrong representation, and the classic example is undoing by recomputing:

```
current = current[:-1]    # 0(k): builds a new list every time
current.pop()             # 0(1): the right way
```

So the undo adds $O(1)$ per node to a tree that already costs $O(1)$ per node to walk. The un-choose is free.

Mutate-and-undo versus copy-down

```
style A: one list, 0(1) per node,    0(depth) memory live
style B: a new list per node,        0(depth) per node,    0(depth^2) live on one path
```

Concretely, subsets at $n = 20$:

```
nodes in the tree          2^20      = 1,048,576
average length of 'current' 10
style A copies              only at record time    ~10 million elements
style B copies              at every node          ~10 million elements + 1M allocations
```

For subsets they end up close, because every node records. For permutations they do not:

```
permutations, n = 9
nodes              ~ 986,000
leaves             362,880
style A element copies  362,880 × 9  = 3.3 million
style B element copies  986,000 × ~5  = 4.9 million, plus 986,000 allocations
```

Roughly a $1.5\times$ to $3\times$ difference, and it grows with depth. Not the difference between passing and failing — but “I mutate and undo because copying at every node multiplies the work by the depth” is a better sentence than “I just do it this way”.

Pruning, which is the real number

This is the one to memorise, because it is the argument for the whole technique.

N-Queens	nodes without pruning	nodes with pruning	ratio
n = 6	55,987	543	103×
n = 8	19,173,961	2,057	9,321×
n = 10	6,257,890,000 (est.)	35,538	176,000×

One `if` before the recursive call, and `n = 8` goes from nineteen million nodes to two thousand. The undo is what makes the pruning possible — you can only afford to check “is this column free?” cheaply if the column set is kept correct as you descend, and it is only kept correct because you remove from it on the way out.

Space

shared state (current, used, sets)	$O(n)$	– one copy, reused
stack	$O(\text{depth})$	

extra space:	$O(n + \text{depth})$	

Backtracking’s memory is the *deepest path*, never the tree. That is what makes it usable at all: the tree can have nineteen million nodes while the memory never exceeds a few dozen frames.

7 The traps

Trap 1 — forgetting one of two undoes

```
used[i] = True
current.append(items[i])
explore()
current.pop()

# missing: used[i] = False

permutations([1, 2, 3]) → [[1, 2, 3]]
```

One answer instead of six, no error. **Count the chooses, count the undoes, make them match** — before you run it.

Trap 2 — undoing after the loop instead of inside it

```
for i in range(len(items)):
    current.append(items[i])
    explore()
current.pop()                                # WRONG: outside the loop
```

Every iteration appends and only the last one is removed, so `current` grows without bound.

```
subsets([1,2,3]) → [[], [1], [1, 2], [1, 2, 3], [1, 2, 3, 3], ...]
```

Nonsense answers, growing lengths, and eventually:

```
RecursionError: maximum recursion depth exceeded
```

Trap 3 — returning early past the undo

```
visited[r][c] = True
if explore(nr, nc):
    return True                    # visited[r][c] stays True for ever
visited[r][c] = False
```

Correct for a single query, wrong the moment the same grid is reused — the second call finds cells already marked and returns `False` for a path that exists. It is the classic “passes the first test, fails the test suite” bug. Undo, then return.

Trap 4 — undoing a copy, which does nothing

```
explore(current + [x])            # style B: the callee got its OWN list
current.pop()                     # removes something the callee never saw
```

Mixing the two styles. The `pop` now removes an element from `current` that was never appended to it:

```
IndexError: pop from empty list
```

Pick one style per function. If you pass a copy, there is nothing to undo; if you mutate, undo exactly what you mutated.

Trap 5 — shallow copies of nested state

```
snapshot = board[:]              # a NEW outer list of the SAME inner lists
explore()
board = snapshot                  # restores nothing at all
```

`board[:]` copies the outer list only. Every row is still the same object, so a change to `board[2][3]` is visible in the “snapshot”. You need `copy.deepcopy`, which is expensive, or — better — do not snapshot at all: **mutate one cell and set that one cell back**.

```
board[r][c] = digit
explore()
board[r][c] = "." # 0(1), correct, no copying
```

Trap 6 — mutating a collection while looping over it

```
for choice in available:
    available.remove(choice) # modifying the thing being iterated
    explore()
    available.add(choice)
```

```
RuntimeError: Set changed size during iteration
```

Loop over a snapshot — `for choice in list(available):` — or, better, loop over the indices and use a boolean array, which is what `used` is for.

Trap 7 — checking validity at the leaf instead of on the way down

```
if row == n:
    if is_valid_board(placement): # checking only at the bottom
        record()
    return
```

Correct answer, catastrophically slow. On N-Queens at $n = 8$ it is 19 million nodes instead of 2,057. **The check must happen before you descend**, because the whole point is to not walk the branch.

Trap 8 — thinking the undo is “cleanup”

It is not cleanup, and calling it that in an interview costs you. Nothing is leaked and nothing is tidied. The pop is **what makes the sibling branch correct**: `current` is one object shared by every branch of the tree, and the un-choose is what restores it to the state that branch is entitled to see.

8 In the interview

How it gets asked

- Directly, after you write any backtracking solution: *“Why did you pop that element after the recursive call?”*

- The variation: “*What happens if you leave that line out?*” — they want the specific wrong output, not “it breaks”.
- The design probe: “*Could you write this without mutating anything?*”
- The efficiency probe: “*Where does the pruning happen, and how much does it save?*”
- The definitional one: “*What is the difference between backtracking and brute force?*”

What to say out loud, in the first ninety seconds

1. **State the invariant.** “When a call returns, every piece of shared state is exactly what it was when the call started. That is the only invariant, and if it holds at every node the tree is correct.”
2. **Say what the pop is, and is not.** “It is not cleanup. `current` is a single object shared by every branch. The pop is what makes the *next* branch see the state it is entitled to see.”
3. **Enumerate your state.** “I am changing three things before I recurse: the partial answer, the used flags, and the column set. So there are three lines after the call, undoing them in reverse.”
4. **Name what you moved into the arguments.** “`remaining` is a parameter, not a mutation, so there is nothing to undo for it. Anything I can pass by value is one thing I cannot forget to restore.”
5. **Point at the pruning line.** “The `continue` before the apply is what makes this backtracking rather than enumeration. Checking on the way down instead of at the leaf is the whole speed-up.”
6. **Give the alternative and dismiss it with a reason.** “I could pass a fresh copy down and never undo anything. That is impossible to get wrong, and it allocates at every node rather than every leaf, so it multiplies the work by the depth.”

The follow-ups

“**What happens if you leave the pop out?**” “For subsets, the ‘skip this element’ branch still contains the element, so about half the answers are wrong — and the count is still 2^n , so nothing looks obviously broken. For permutations, if it is the `used[i] = False` I forget, elements are consumed permanently and I get exactly one answer instead of $n!$. Neither raises an error. That is the thing worth saying: **the missing undo is always silent**, which is why I write the choose, the recurse and the undo as one block rather than three decisions.”

“**Could you write it without mutating anything?**” “Yes. Pass `current + [x]` down instead of appending, and the callee gets its own list, so there is nothing to restore. It is shorter and it cannot be got wrong. The cost is that it allocates a new list at every *node*, not every leaf — for permutations at $n = 9$ that is nearly a million allocations against

360,000 in the mutating version. I would use it if the state were tiny, or if I wanted the partial answers to be immutable so I could share them. And I would never mix the two styles in one function: if you pass a copy and then also pop, you get an `IndexError` from popping something the callee never saw.”

“What is the difference between backtracking and brute force?” “One line: the validity check before the recursive call. Brute force generates every candidate and tests it at the end; backtracking rejects a partial candidate as soon as it cannot possibly lead to a solution, so entire subtrees are never walked. On N-Queens at $n = 8$, testing at the leaf visits about nineteen million nodes and pruning on the way down visits two thousand and fifty-seven. Same answer, a factor of nine thousand. And the pruning is only affordable because the undo keeps the running checks — the column set, the diagonal sets — correct as you descend.”

“How do you decide what needs undoing?” “I list everything the recursion can see that I changed. It is three kinds: the partial answer, the availability marks, and derived bookkeeping like a running total or a set of occupied diagonals. The third kind is where bugs live, because it does not feel like state. Then I count: lines that change something before the call must equal lines that change it back after, and I undo in reverse order. And wherever I can, I move state into the parameter list instead, because a value passed by argument physically cannot need undoing.”

“Your search returns True early. Is the undo still correct?” “It is skipped, and whether that matters depends on the caller. If I am unwinding straight out and nobody looks at the state again, it is fine and it saves a little work. If the same grid or board is reused for a second query, it is a real bug — cells stay marked and the second call reports no path when one exists. The safe habit is to capture the result, undo, and then return, which is two extra lines and removes an entire category of works-once bugs.”

“Where would this pattern not be the right tool?” “When the same subproblem is reached by many different paths. Backtracking has no memory — it will re-solve an identical state every time it arrives there. The moment I notice that the state can be described by a few numbers and is reached repeatedly, that is dynamic programming, and the tree collapses from exponential to polynomial. Backtracking is for problems where I genuinely need to enumerate or where the pruning is strong enough that the tree stays small.”

A model answer

Asked: *why did you pop that element after the recursive call?*

“Because `current` is one list object shared by every branch of the tree, and the pop is what makes the **next** branch correct. It is not cleanup — nothing is leaked if I skip it.

The invariant I am maintaining is this: **when a recursive call returns, every piece of shared state is exactly what it was when that call started.** If that holds at every node, then each branch sees only the choices on its own path from the root, which is precisely what ‘explore this subtree’ means. If it fails at one node, every sibling after that node is exploring a state that no branch actually chose — and the answers are wrong with no error at all.

Concretely: I append the element, recurse to explore everything that follows from taking it, and pop so that the sibling branch — the one that does *not* take it — starts from the same list I started from. Leave the pop out and the skip branch still contains the element, so half the subsets are wrong, and the count is still 2^n so nothing looks broken.

The rule I actually apply while writing is to count. Every line before the recursive call that changes something the recursion can see needs a matching line after it, undone in reverse order. In the permutations version there are two — the `used` flag and the append — so there are two undoes. In N-Queens there are four: the row’s column, the two diagonal sets, and the placement. And I move whatever I can into the parameter list instead, because a value passed as an argument cannot need undoing — that is why `remaining` in combination sum has no undo line.

The alternative is to pass a fresh copy down: `explore(current + [x])`. Then nothing is shared and nothing needs restoring. It is shorter and impossible to get wrong, and it allocates at every node instead of every leaf, so it multiplies the work by roughly the depth. I would use it for very small state.

One more thing the undo buys, which is really the point of the whole technique. Because the state is kept correct as I descend, I can run cheap validity checks *on the way down* and skip whole subtrees. On N-Queens at $n = 8$ that is the difference between about nineteen million nodes and two thousand and fifty-seven. The undo is what makes that check affordable.”

9 Recall card

- **The invariant, and there is only one: when a call returns, every piece of shared state is exactly what it was when the call started.** The pop is **not cleanup** — `current` is one object shared by every branch, and the un-choose is what makes the

sibling branch correct. A missing undo is **always silent**: subsets gets half its answers wrong, permutations returns **one answer instead of $n!$** .

- **State is three kinds: the partial answer, the availability marks, and derived bookkeeping** (a running sum, an occupied-column set). The third is where the bugs live. **Count the lines before the call and the lines after; they must match, and undo in reverse order.**
- **Anything passed as an argument needs no undo** — ints, strings and tuples are values, so `build(i+1, remaining - c)` has nothing to restore. Move state into the parameter list wherever you can; it is state you cannot forget.
- The `if not is_valid: continue` **BEFORE the apply is what makes it backtracking rather than enumeration.** Check on the way down, never at the leaf: **N-Queens at $n = 8$ is 19,173,961 nodes without it and 2,057 with it.** Every undo must be $O(1)$ — `board[r][c] = "."`, not a fresh copy — or the pruning stops being affordable.
- **Two styles: mutate-and-undo ($O(1)$ per node, must not forget) or pass-a-copy (`explore(current + [x])`, cannot be got wrong, allocates at every node).** Never mix them — popping after passing a copy gives `IndexError: pop from empty list`. And **undo before an early return**, or a reused grid reports no path when one exists.

Design snake and ladder

1 What this is, and why they ask it

Snake and ladder is a board of numbered squares, some dice, and a rule that certain squares move you somewhere else the moment you land on them. Players take turns; first to the last square wins.

The three sentences that matter. **A snake and a ladder are the same object** — a jump from one square to another, one going down and one going up — and a candidate who writes two classes for them has already lost the point of the exercise. **The dice must be an interface**, not a call to `random`, or the game cannot be tested. And **the rules that vary between households** — do you need an exact roll to finish, does a six give you another turn — are configuration, not `if` statements.

They ask it because it is small enough to finish in thirty minutes and rich enough to separate people. Everybody can produce `Board`, `Player`, `Dice`, `Game`. The interesting questions are: how many classes do snakes and ladders need between them (one), where does randomness live (behind an interface), what happens when a jump lands you on another jump (a decision you must make out loud), and what do you change when the same code has to run a hundred thousand online games at once.

It is the friendliest of the “design a game” prompts, and the same skeleton answers *design tic-tac-toe*, *design ludo* and *design chess* — which is usually the next question.

2 The story

The board came out after dinner because it was raining and the power had gone, and the two children had been fighting since four o'clock.

It was the old cloth one, folded so many times that the creases had gone white. A hundred squares. Curling green snakes across some of them, and thin brown ladders across others.

Arjun, who was nine, went first. He got a six, moved to the sixth square, and found the bottom of a ladder there, which took him up to twenty-six in one go. He was extremely pleased.

His sister Ira, who was six and did not fully trust any of this, landed on a snake at forty-seven three turns later and slid all the way back to nineteen, and there was a period of about ten minutes when the game was not being played.

Their grandmother had been watching from the chair by the window without saying much.

When it started again she said something that neither of them expected. She said, you know, that snake and that ladder are the same thing.

Ira said they were absolutely not the same thing.

Her grandmother said: look at what actually happens. You land on a square. The square says go to a different square. That is all either of them does. One of them sends you forwards and one sends you backwards, and that is the only difference between them. If you painted the snake as a ladder nothing about the game would change except how you felt about it.

Then she asked them a question they could not answer. She said, what happens if the top of that ladder puts you on the head of a snake?

They looked. It did not, on that board. But they could not say what the rule *would* be, and they argued about it for a while — Arjun said you would go down the snake as well, Ira said that was unfair and you should stay put — and eventually their mother said that in this house you stay put, and wrote nothing down, and that was that.

The other two arguments that night were the usual ones. Whether you needed to land on the last square exactly, or whether going past it counted. And whether a six gave you another turn.

Both were settled the same way: by their mother saying how it was going to be, before anybody rolled.

3 The idea in plain English

The grandmother has given you the design. Everything else is bookkeeping.

- The board is a set of numbered **cells**, 1 to 100.
- A snake and a ladder are both a **jump**: a pair `(from, to)`. If `to > from` it is a ladder, if `to < from` it is a snake. **The type is not stored; it is derived from the numbers.**
- A **player** has a name and a current position, and that is genuinely all.
- The **dice** produces a number between 1 and 6.
- A **turn** is: roll, move, apply any jump, check for a win, decide who goes next.
- “Whether you need to land exactly” and “whether a six gives another turn” are **rules**, decided before anybody rolls — configuration, not code.

- “What if a ladder puts you on a snake’s head?” is a rule you must **state**, because the code has to do something and every household does it differently.

Why one class and not two

```
class Snake:
    head: int
    tail: int

class Ladder:
    bottom: int
    top: int
```

Two classes with identical shape, identical behaviour, and different words. Every operation then needs both:

```
for snake in board.snakes:
    if snake.head == position: ...
for ladder in board.ladders:
    if ladder.bottom == position: ...
```

Against:

```
position = board.jumps.get(position, position)    # one line, both cases
```

One dictionary from square to destination, and the whole feature is a `dict.get` with a default. That is the answer to “how do you model snakes and ladders”, and saying it in that form takes ten seconds.

If you genuinely need to *display* them differently, that is a presentation concern and one property answers it:

```
@property
def is_ladder(self) → bool:
    return self.end > self.start
```

Why the dice is an interface

This is the single highest-value idea in the whole prompt, and it is not about the dice.

```
def roll(self) → int:
    return random.randint(1, 6)    # the game is now untestable
```

If the game calls `random` directly, you cannot write a test that says “a player at 97 who rolls a 5 does not move”. You can only run it a thousand times and hope. Put an interface in front of it:

```

class Dice(ABC):
    @abstractmethod
    def roll(self) → int: ...

class RandomDice(Dice): ...           # production
class ScriptedDice(Dice): ...         # tests: returns 6, 6, 2, 1, ...

```

Now every rule in the game can be tested in three lines. **The general principle is that randomness, time and input/output are the three things you must be able to replace**, and a design that hard-codes any of them is a design nobody can write a test for. Say that sentence — it applies to every LLD prompt you will ever get.

It also matters in production for a reason candidates rarely mention: for an online game, the dice **must** be rolled on the server. If the phone rolls it, the phone can cheat. The interface makes that a one-line substitution.

The rules that vary

RULE	OPTIONS	WHERE IT LIVES
Finishing	exact roll needed / overshoot wins / bounce back	<code>GameRules.finish</code>
Rolling a six	another turn / nothing special / three sixes forfeits	<code>GameRules.extra_turn_on</code>
Starting	any roll / must roll a six to enter	<code>GameRules.entry</code>
Chained jumps	apply repeatedly / apply once only	<code>GameRules.chain_jumps</code>

All four are data. A `GameRules` object holds them, the turn logic reads them, and adding the household's fifth rule does not touch the `Game` class. The alternative — a boolean parameter per rule threaded through three methods — is what the interviewer is watching for.

The chained-jump question

The grandmother's question is the one that separates a thorough candidate. If square 6 has a ladder to 26 and square 26 has a snake to 4, landing on 6 could mean:

```

apply once:      6 → 26, stop.           (simplest; most published rules)
apply repeatedly: 6 → 26 → 4, stop.      (needs cycle protection)

```

If you apply repeatedly, you must prove it terminates. Two jumps pointing at each other — 30 to 60 and 60 to 30 — is an infinite loop:

```

while position in jumps:
    position = jumps[position]

```

runs for ever. The fixes are a validated board (no jump may land on another jump's start) or a hard cap on the number of hops. **Say which one you chose.** Board validation is the better answer, because it catches the problem when the board is loaded rather than when a child is waiting for their turn.

4 The picture

The class diagram.

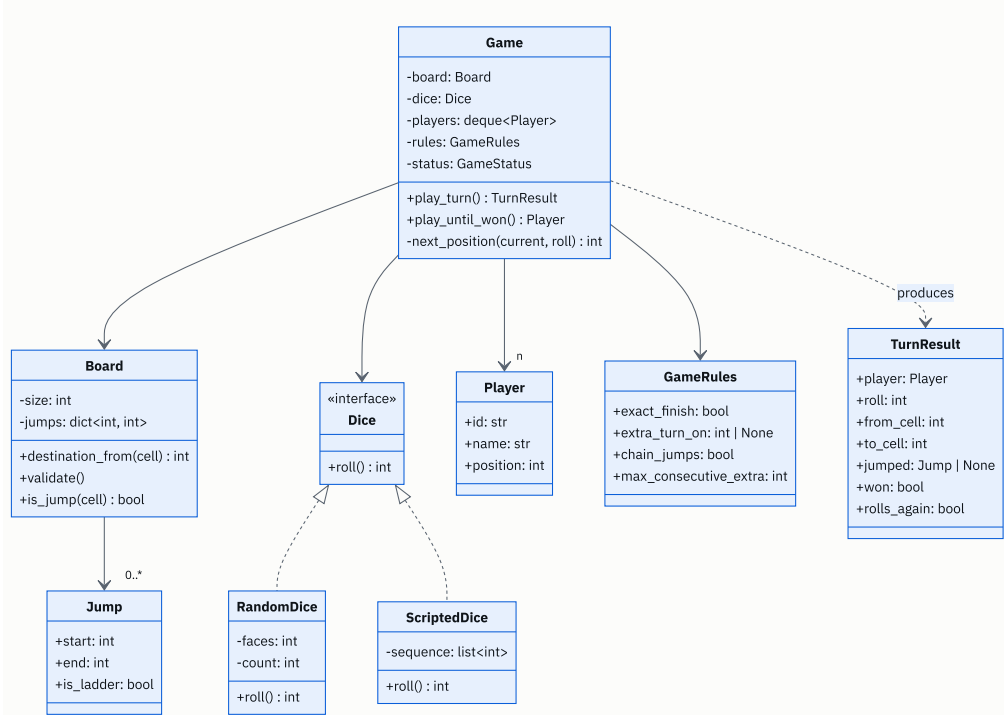
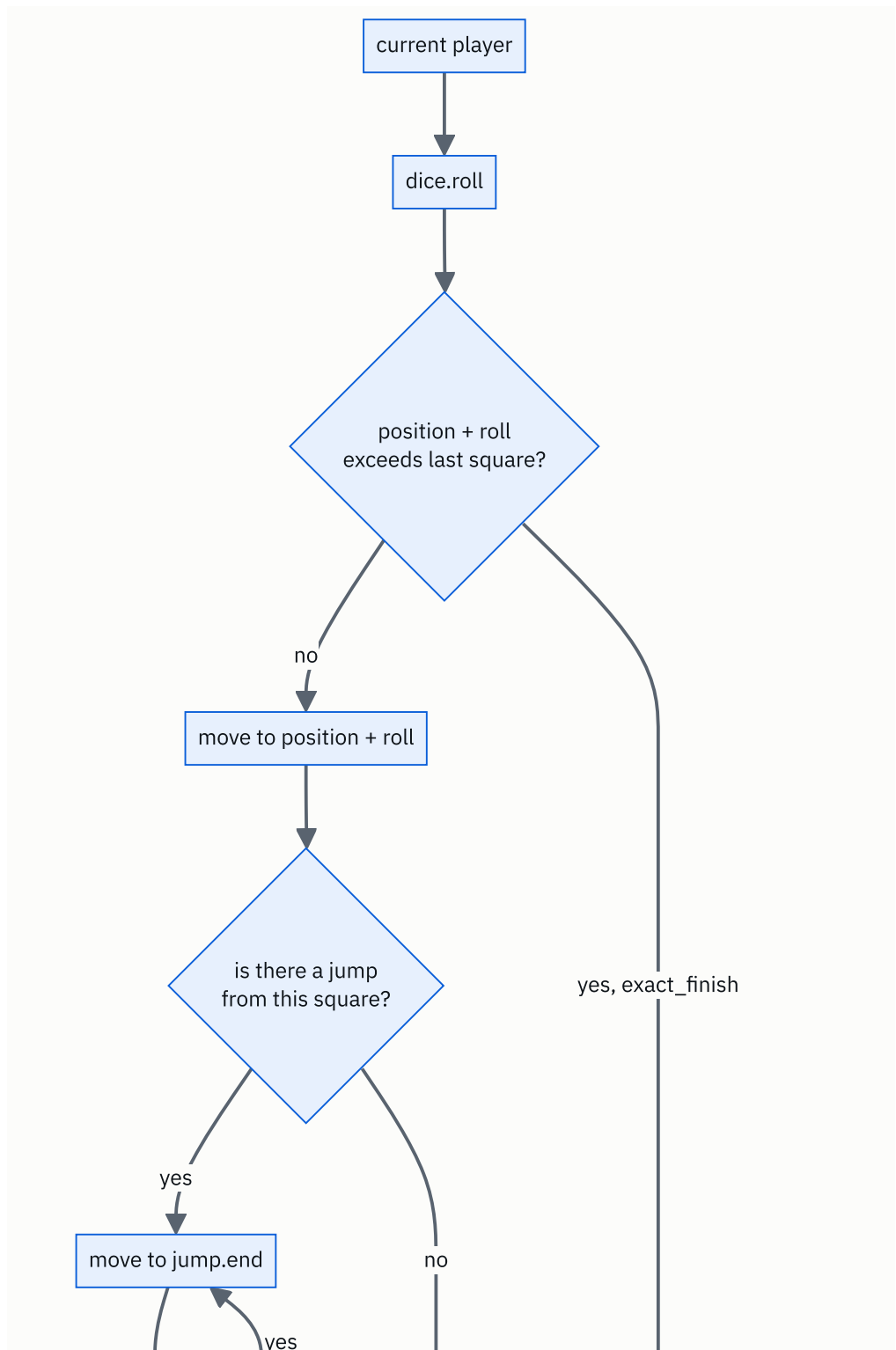


FIGURE 94.1

What to notice: **there is no Snake class and no Ladder class.** There is one Jump , and is_ladder is derived. And Dice is an interface with two implementations, one of which exists only so the rules can be tested.

TurnResult is the other thing worth noticing. The game does not print anything — it **returns a description of what happened**, and whoever is displaying the game decides what to do with it. That is what makes the same Game class work behind a terminal, a phone screen and an automated simulation.

One turn, end to end:



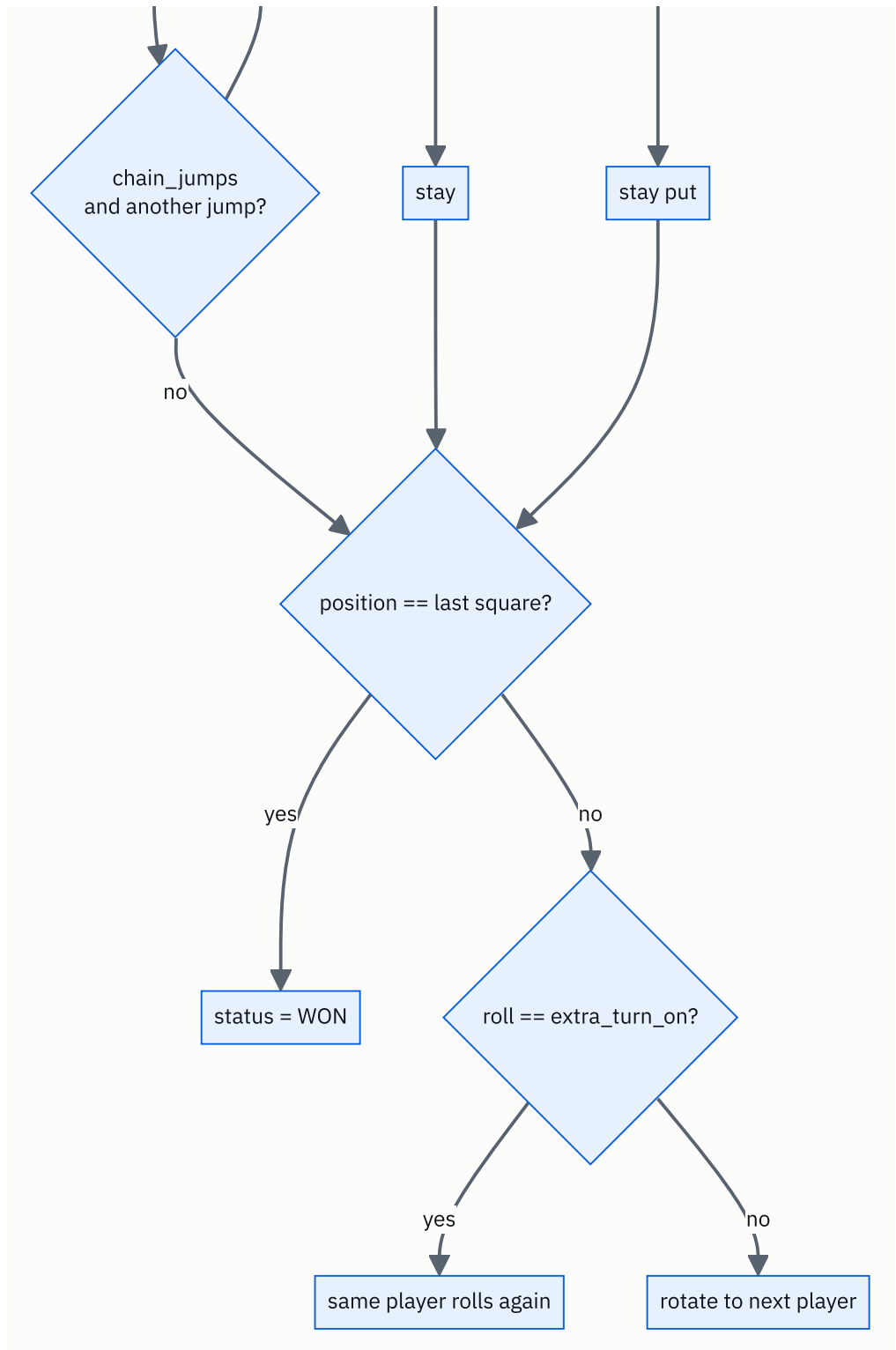


FIGURE 94.2

What to notice: **the win check comes after the jump, not after the move.** A ladder can win the game. Checking before the jump is a real bug, and it is the kind an interviewer will construct a test for.

The board, drawn as what it actually is:

```
jumps = { 6:26, 11:56, 20:88, 36:44, 49:11, 62:19, 74:53, 88:24, 95:56, 98:8 }
         ^ladder ^ladder ^ladder ^ladder  ^snake  ^snake  ^snake  ^snake  ^snake ^snake

destination_from(6)  → 26      ladder: end > start
destination_from(49) → 11      snake:  end < start
destination_from(7)  → 7       no entry: the default is the square itself

note 20 → 88 and 88 → 24.
With chain_jumps ON, landing on 20 takes you to 24 – LOWER than 88 you just reached.
With it OFF, you stop at 88.
The board above is INVALID under "no jump may land on another jump's start".
```

5 How it actually works

Move 1 — clarify

Four questions, with the answers you will assume.

- “*Is this one game on one device, or an online service running many games?*” — Start with the game object; I will say what changes for the online version, because that is where the concurrency questions live.
- “*Do you need an exact roll to finish, or does overshooting win?*” — Assume exact roll, and make it configurable, because households differ and the interviewer will change it.
- “*Does a six give another turn?*” — Assume yes, with a cap of three in a row, and make both configurable.
- “*If a jump lands you on the start of another jump, does it chain?*” — Assume no, apply once. If you want chaining, I will need to validate the board so it cannot loop.

The question people forget: “*how many dice, and how many faces?*” Ludo uses one six-sided die, some variants use two. Making `RandomDice(faces=6, count=1)` costs nothing and answers the follow-up before it arrives.

Move 2 — the nouns

CLASS	RESPONSIBLE FOR
<code>Jump</code>	A pair of squares, and whether it goes up. Immutable.
<code>Board</code>	The size, the jump map, validation, and “where does this square send me”.
<code>Dice</code> (interface)	Producing a roll. Replaceable, so the rules can be tested.
<code>Player</code>	A name and a position. Nothing else.
<code>GameRules</code>	The four decisions that vary between households. Data, not code.
<code>Game</code>	One turn: roll, move, jump, check, rotate. Owns the turn order.
<code>TurnResult</code>	A description of what happened, returned rather than printed.

`Player` **having no methods is correct and worth defending**. A player does not “move themselves” — where a player ends up depends on the board and the rules, which the player does not and should not know about. Putting `player.move(roll)` on the player forces `Player` to hold a reference to the board, and then the two classes cannot be understood separately.

Move 3 — the interesting part

Where the turn logic lives, and what it returns. Every rule variation meets in one method, and the temptation is to make that method print. Do not. Return a `TurnResult` and let the caller render it.

```
terminal game:    print(f"{r.player.name} rolled {r.roll} and moved to {r.to_cell}")
online game:      broadcast(r.as_json()) to both players' connections
simulation:       count turns; render nothing
```

Three completely different front ends over one unchanged `Game` class. That is the payoff, and it is the same separation as the logging framework’s handler on [day 091](#): decide what happened in one place, decide how to show it in another.

The second interesting part is `Dice`, covered above, and the third is board validation:

```
def validate(self) → None:
    for start, end in self.jumps.items():
        if not (1 ≤ start ≤ self.size and 1 ≤ end ≤ self.size):
            raise ValueError(f"jump {start}→{end} is off the board")
        if start == end:
            raise ValueError(f"jump {start}→{start} goes nowhere")
        if start == self.size:
            raise ValueError("the winning square cannot have a jump")
        if start == 1:
            raise ValueError("the starting square cannot have a jump")
    if self.chain_jumps:
        overlapping = set(self.jumps) & set(self.jumps.values())
        if overlapping:
            raise ValueError(f"chained jumps would loop at {sorted(overlapping)}")
```


Validating at load time rather than at play time is the difference between a clear error message when someone edits a configuration file and a game that hangs on turn forty.

Move 4 – the class diagram

Drawn above. Present it by walking one turn out loud, not by listing classes.

Move 5 – the code

```
from __future__ import annotations

import random
from abc import ABC, abstractmethod
from collections import deque
from dataclasses import dataclass
from enum import Enum, auto


@dataclass(frozen=True)
class Jump:
    """A snake AND a ladder. The difference is derived, not stored."""

    start: int
    end: int

    @property
    def is_ladder(self) → bool:
        return self.end > self.start
```

Six lines, and the whole “how do you model snakes and ladders” question is answered.

```

class Dice(ABC):
    @abstractmethod
    def roll(self) → int: ...

class RandomDice(Dice):
    def __init__(self, faces: int = 6, count: int = 1) → None:
        self._faces = faces
        self._count = count

    def roll(self) → int:
        return sum(random.randint(1, self._faces) for _ in range(self._count))

class ScriptedDice(Dice):
    """For tests. Give it the rolls you want, in order."""

    def __init__(self, sequence: list[int]) → None:
        self._sequence = list(sequence)
        self._i = 0

    def roll(self) → int:
        value = self._sequence[self._i % len(self._sequence)]
        self._i += 1
        return value

```

`ScriptedDice` is nine lines and it is the reason every rule in this game can be tested deterministically. Write it in the interview; it takes twenty seconds and it makes a point.

```

class Board:
    def __init__(self, size: int = 100, jumps: dict[int, int] | None = None) → None:
        self.size = size
        self.jumps = dict(jumps or {})

    def destination_from(self, cell: int) → int:
        return self.jumps.get(cell, cell) # the entire feature, in one line

    def jump_at(self, cell: int) → Jump | None:
        end = self.jumps.get(cell)
        return Jump(cell, end) if end is not None else None

```

```

@dataclass
class GameRules:
    exact_finish: bool = True
    extra_turn_on: int | None = 6
    max_consecutive_extra: int = 3
    chain_jumps: bool = False

@dataclass
class Player:
    id: str
    name: str
    position: int = 0 # 0 = not yet on the board

class GameState(Enum):
    IN_PROGRESS = auto()
    WON = auto()

@dataclass(frozen=True)
class TurnResult:
    player: Player
    roll: int
    from_cell: int
    to_cell: int
    jumped: Jump | None
    won: bool
    rolls_again: bool

```

And the turn itself, which should read like the rules read out loud:

```

class Game:
    def __init__(self, board: Board, dice: Dice, players: list[Player],
                 rules: GameRules | None = None) → None:
        if len(players) < 2:
            raise ValueError("need at least two players")
        self.board = board
        self.dice = dice
        self.rules = rules or GameRules()
        self.players = deque(players)          # rotate() is the turn order
        self.status = GameStatus.IN_PROGRESS
        self.winner: Player | None = None
        self._consecutive_extra = 0

    def play_turn(self) → TurnResult:
        if self.status is GameStatus.WON:
            raise RuntimeError("the game is over")

        player = self.players[0]
        roll = self.dice.roll()
        start = player.position
        target = start + roll

        if target > self.board.size and self.rules.exact_finish:
            target = start                      # overshoot: stay put

        jumped = self.board.jump_at(target)
        target = self._apply_jumps(target)
        player.position = target

        won = target == self.board.size        # AFTER the jump, not before
        if won:
            self.status = GameStatus.WON
            self.winner = player

        again = self._rolls_again(roll, won)
        if not again:
            self._consecutive_extra = 0
            self.players.rotate(-1)           # next player

        return TurnResult(player, roll, start, target, jumped, won, again)

    def _apply_jumps(self, cell: int) → int:
        cell = self.board.destination_from(cell)
        if not self.rules.chain_jumps:
            return cell
        seen = {cell}                         # cycle guard, even with validation
        while (nxt := self.board.destination_from(cell)) ≠ cell:
            if nxt in seen:
                break
            seen.add(nxt)
            cell = nxt
        return cell

    def _rolls_again(self, roll: int, won: bool) → bool:
        if won or self.rules.extra_turn_on is None or roll ≠ self.rules.extra_turn_on:
            return False
        self._consecutive_extra += 1
        if self._consecutive_extra ≥ self.rules.max_consecutive_extra:
            self._consecutive_extra = 0
            return False                       # three sixes: turn forfeited
        return True

    def play_until_won(self, max_turns: int = 10_000) → Player:
        for _ in range(max_turns):
            result = self.play_turn()

```

```

        if result.won:
            return result.player
        raise RuntimeError("game did not finish")

```

Notice `play_until_won` has a turn cap. **Any loop whose exit depends on randomness needs one**, and being the candidate who writes it unprompted is a small, cheap win.

Testing a specific rule, which is what the `Dice` interface bought:

```

def test_exact_finish_keeps_you_put():
    board = Board(size=100, jumps={})
    game = Game(board, ScriptedDice([5]), [Player("a", "Anu"), Player("b", "Bharat")])
    game.players[0].position = 97
    result = game.play_turn()
    assert result.to_cell == 97 and not result.won      # 97 + 5 = 102, so no move

def test_a_ladder_can_win_the_game():
    board = Board(size=100, jumps={95: 100})
    game = Game(board, ScriptedDice([3]), [Player("a", "Anu"), Player("b", "Bharat")])
    game.players[0].position = 92
    result = game.play_turn()
    assert result.won and result.jumped.is_ladder      # the win check is AFTER the jump

```

Three lines each, and deterministic. That is what “the dice is an interface” is worth.

What real systems look like

- Online board games — **Ludo King**, **Board Kings** — run the dice on the server for exactly the anti-cheat reason above, and send the client a signed roll. The client animates a result it did not choose.
- Turn state lives in **Redis** while a game is live (a hash per game, a few hundred bytes) and is written to **Postgres** or **DynamoDB** when the game ends, because finished games are read rarely and live games are read constantly.
- Turns are pushed over **WebSockets**, not polled, because a poll every second for a game where a turn takes eight seconds is seven wasted requests.
- The board configuration is data — a JSON document per board layout — which is what lets a live game ship a new board without a release.

6 The numbers

One game in memory

Player object	~120 bytes
4 players	480 bytes
Board: 100 cells, ~20 jumps	dict of 20 int→int ≈ 1,200 bytes
Game object + deque + rules	~ 600 bytes

one live game	≈ 2.3 KB

Boards are shared, not copied. A hundred thousand concurrent games on the same layout share one `Board` instance, so the per-game cost is really the players and the position:

```
100,000 concurrent games × ~1.1 KB (players + game state) = 110 MB
plus ONE shared board                                     = 1.2 KB
```

A hundred and ten megabytes for a hundred thousand live games. **State this, because the naive answer — one board per game — would be 120 MB of duplicated identical dictionaries.**

How long a game takes

Measured by simulation on the standard board, one player, no extra turns:

```
10,000 simulated games:
  mean turns to finish    ≈ 39
  median                  ≈ 32
  90th percentile         ≈ 72
  longest seen            ≈ 300+
```

The distribution has a long tail because a snake near the end sends you a long way back. **That tail is the reason `play_until_won` needs a cap**, and it is also the reason an online version needs a turn timer: a real game where a player walks away must not hold a slot for ever.

For a four-player game, the game ends when the *first* player finishes, so it is shorter in turns per player and longer in wall-clock time:

```
4 players × ~30 rounds × ~8 seconds per turn ≈ 16 minutes per game
```

At service scale

concurrent games	100,000	
turns per game per minute	~7	(4 players, 8s per turn)

turns per minute	700,000	
turns per second	~11,700	

Around twelve thousand turns a second, and **each turn is a dictionary lookup and an integer addition** — microseconds of CPU. The load is not computation; it is **connections**. A hundred thousand games at four players is four hundred thousand open WebSockets, and that is the number that decides the architecture.

```
400,000 WebSockets ÷ ~50,000 per server = 8 servers, plus headroom
memory: 110 MB of game state, trivially sharded by game id
```

Concurrency, which is where the real bugs are

Two things happen at once and both need a stated answer.

1. **Two players in the same game act simultaneously.** One is on a slow connection and their turn times out at the same moment they finally send a roll. Without protection you get two rolls applied for one turn. The fix is a **version number on the game state** and a compare-and-set: the turn is accepted only if it names the version the server currently holds, exactly like the optimistic locking on [day 034](#). Not a lock — a version check, which costs nothing and cannot deadlock.
2. **The same player sends the same roll twice** because their phone retried. A **turn number** in the request makes it idempotent: turn 14 can only be applied once.

A single game is inherently serial — one turn at a time — so the correct model is **one writer per game, not a lock per field**. Say that: it means a hundred thousand games are a hundred thousand independent serial streams with no contention between them, which is why this scales so easily.

7 The trade-offs

`dict` of jumps, or a 101-element array?

An array indexed by square, holding the destination or the square itself, gives you `0(1)` with no hashing and is arguably clearer. It costs 101 entries whether there are two jumps or forty.

Take the `dict` for a sparse board and a general `size`; take the array if the board is fixed at 100 and you want to remove hashing from the hot path. Either is defensible — **what is not defensible is two lists to scan**.

Should `Player` know how to move?

`player.move(roll, board, rules)` reads nicely and puts the board and the rules inside `Player`, so you can no longer test or reason about a player alone. **Keep `Player` a data holder**. The counter-argument is worth acknowledging: in a richer game — chess, ludo with four tokens each — a player genuinely owns pieces and has real behaviour, and then the anaemic version starts to hurt.

Return a `TurnResult`, or print?

Printing is two fewer classes and thirty seconds faster to write. It also welds the game to a terminal. **Return the result**. The moment the interviewer says “now make it an online game”, printing is a rewrite and returning is a new caller.

Server-side dice, or client-side?

Client-side is free and instant. It is also trivially cheatable, and for anything with stakes it is not an option. **Server-side, always, for a real product** — the `Dice` interface makes this a one-line change, which is exactly the argument for having had the interface.

Where this design breaks

- **Multiple tokens per player** — ludo, where each player has four pieces and must choose which to move — breaks `Player.position`, because position becomes a list and a turn now requires a *decision*. That decision is a `MoveStrategy`, and it is the point where a human player and a computer player become two implementations of one interface.
- **A board where cells have behaviour beyond jumping** — “miss a turn”, “roll again”, “swap with the player ahead” — outgrows `dict[int, int]`. That is the moment `Cell` becomes a real object with an `on_land(game, player)` method, and jumps become one implementation of it. **Do not build that up front**; say that you would build it the moment a second kind of cell appears.
- **Reconnection.** A player closing the app mid-game needs the full state on reconnect, which means the state must be serialisable and must not live only in the memory of one process. That is the change that forces Redis rather than a Python object, and it is the single biggest difference between the interview answer and a shipped game.
- **Fairness.** Going first is a real advantage — roughly a few percentage points in win rate. If that matters, the fix is product design (rotate who starts across a match series), not code.

8 In the interview

How it gets asked

- The prompt: *“Design snake and ladder for n players.”*
- The modelling probe, always: *“How do you represent snakes and ladders?”*
- The testing probe: *“How would you test that a player at 97 who rolls a 5 does not move?”*
- The rules probe: *“Now a six gives another turn.”* / *“Now you must land exactly on 100.”*
- The nasty one: *“What if a ladder lands you on a snake?”*
- The scale switch: *“Now make it an online game with a hundred thousand concurrent matches.”*

What to say out loud, in the first ninety seconds

1. **Collapse snakes and ladders immediately.** “They are the same object — a jump from one square to another. One dictionary from square to destination, and `is_ladder` is derived from whether the destination is higher. There is no `Snake` class.”
2. **Put the dice behind an interface, and say why.** “`Dice` is an interface with a random implementation and a scripted one. That is not ceremony — without it I cannot write a deterministic test for any rule in the game, and for an online version the roll has to happen on the server anyway.”
3. **Name the rules that vary and make them data.** “Exact finish, extra turn on six, chained jumps, and whether you need a six to start. Those are a `GameRules` object, because every household plays differently and the interviewer is going to change one.”
4. **Say the turn as one sentence.** “Roll, move, apply any jump, check for a win, decide who goes next — and the win check comes *after* the jump, because a ladder can win the game.”
5. **Return, do not print.** “`play_turn` returns a `TurnResult` describing what happened. The same `Game` class then works behind a terminal, a phone and a simulation.”
6. **Keep `Player` a data holder, and defend it.** “A player is a name and a position. Where they end up depends on the board and the rules, which a player should not know about.”

The follow-ups

“How do you represent snakes and ladders?” “As one thing. Both are a jump from a square to another square; a ladder’s destination is higher and a snake’s is lower, and that is the only difference. So the board holds `dict[int, int]` from start square to destination, and looking one up is `jumps.get(cell, cell)` — the default being the cell itself, so ‘no jump here’ needs no branch. If I need to display them differently I derive `is_ladder` from the two numbers. Two classes for these would double every piece of logic that touches them and buy nothing.”

“How would you test that a player at 97 who rolls a 5 does not move?” “That is why `Dice` is an interface. I construct the game with a `ScriptedDice([5])`, set the player’s position to 97, call `play_turn`, and assert the resulting position is still 97 and nobody won. Three lines, deterministic, runs in a millisecond. With `random.randint` inside the game, that test is impossible — you can only run it many times and hope. The general rule is that **randomness, time and input/output are the three things a design must let you replace**, and any of them hard-coded makes the thing untestable.”

“What if a ladder lands you on a snake’s head?” “That is a rule you have to state, because the code must do something. Two sensible options: apply the jump once and stop, which is what most published rules say and what I would default to; or apply repeatedly. If you want repeated application, I need to prove it terminates — two jumps pointing at each other, 30 to 60 and 60 to 30, is an infinite loop. So I would validate the board at load time: no jump may end on the start of another jump. That catches the problem when the configuration is written rather than when a child is waiting for their turn. I would also keep a seen-set in the loop as a belt-and-braces guard.”

“Now a six gives another turn.” “One field in `GameRules` and one branch in the turn method: if the roll equals the extra-turn value and the player has not won, do not rotate the deque. I would add the cap that most people play with — three sixes in a row forfeits the turn — because without it a player with a very lucky streak, or a broken dice implementation, holds the turn for ever. Both of those are configuration, so the `Game` class does not change when you tell me your house rule is different.”

“Now make it an online game with a hundred thousand concurrent matches.” “The game logic does not change at all, which is the point of returning a `TurnResult` rather than printing. What changes is around it. First, the dice moves to the server — the client must not choose its own roll, and the `Dice` interface makes that a one-line substitution. Second, state moves out of process into Redis, one hash per game at about a kilobyte, so a player can reconnect and so any server can serve any game. Third, turns arrive over WebSockets rather than polling. The numbers: four players per game means four hundred thousand open connections, which at fifty thousand per server is eight servers plus headroom, and the state is only about a hundred and ten megabytes because the board is shared rather than copied per game. The compute is nothing — a turn is a dictionary lookup and an addition. The concurrency risk is two actions landing for the same game at once, which I would handle with a version number on the game state and a compare-and-set, plus a turn number to make retries idempotent. A single game is inherently serial, so it is one writer per game and there is no contention between different games.”

“What would you change to support ludo?” “Two real changes. A player has four tokens instead of one position, so `Player.position` becomes a collection and a turn now requires a **choice** of which token to move — and that choice is a strategy object, which is also how a computer player and a human player become two implementations of one interface. And cells acquire behaviour beyond jumping: safe squares, capture, the home column. That is the point at which `dict[int, int]` becomes `dict[int, Cell]` with an `on_Land` method, and jumps become one kind of cell behaviour among several. I would not build that for snake and ladder — but knowing where the seam is means the change is additive rather than a rewrite.”

A model answer

Asked: *design snake and ladder for n players.*

“Let me start with the modelling decision that carries the whole design. **A snake and a ladder are the same object.** Both say: you landed here, now go there. One goes up and one goes down, and that is the only difference. So the board holds a single map from square to destination, and looking up is `jumps.get(cell, cell)` — defaulting to the square itself, so there is no branch for ‘no jump here’. If I need to draw them differently, `is_ladder` is derived from whether the destination is higher. There is no `Snake` class and no `Ladder` class.

The classes are: `Jump`, which is two integers; `Board`, which owns the size and the jump map and validates itself; `Player`, which is a name and a position and deliberately has no behaviour; `Dice`, which is an **interface**; `GameRules`, which is data; and `Game`, which runs a turn.

`Dice` being an interface is the highest-value decision here and it is not really about dice. If the game calls `random.randint` directly, I cannot write a test that says ‘a player at 97 who rolls a 5 stays put’ — I can only run it a thousand times and hope. With a `ScriptedDice` that returns the rolls I give it, every rule in the game is testable in three deterministic lines. The general principle is that randomness, time and input/output are the three things a design has to let you replace. It also happens to be what lets the roll move to the server for an online game, where the client must not be allowed to choose its own number.

The rules that vary between households — exact roll to finish, an extra turn on six, whether jumps chain, whether you need a six to start — are fields on a `GameRules` object rather than branches scattered through the code, because you are going to change one of them in a minute and I would like that to cost nothing.

A turn is: roll; move, unless exact-finish says the overshoot keeps you put; apply any jump from the square you landed on; check whether you have reached the last square; then decide whether the same player rolls again or the turn passes. Two details there. **The win check comes after the jump**, because a ladder can win the game — checking before it is a real bug. And `play_turn` **returns** a `TurnResult` describing what happened rather than printing anything, so the same `Game` works behind a terminal, a phone and a simulation that runs ten thousand games to measure the turn distribution.

The question you might ask next is what happens when a ladder puts you on a snake’s head. That is a rule, not a fact, so I state it: apply once by default. If you want chaining, I have to validate the board at load time so that no jump ends on another jump’s start — otherwise two jumps pointing at each other loop for ever. I would rather fail when the configuration is loaded than when a player is waiting.

On numbers: one live game is about two kilobytes, and the board is shared rather than copied, so a hundred thousand concurrent games is roughly a hundred and ten megabytes of state. A game takes about thirty-nine turns on average with a long tail — a snake near the end can send you a long way back — which is why any ‘play until someone wins’ loop needs a cap. And at that scale the load is not computation, since a turn is a dictionary lookup and an addition; it is four hundred thousand open connections.”

9 Recall card

- **A snake and a ladder are the same object.** One `dict[int, int]` from square to destination; `jumps.get(cell, cell)` defaults to the square itself so there is no branch for “no jump”. `is_ladder` is **derived** from `end > start`, never stored. Two classes here doubles every piece of logic and buys nothing.
- **Dice is an interface with a `ScriptedDice` for tests** — otherwise no rule in the game is testable, only surveyable. **Randomness, time and I/O are the three things a design must let you replace.** It is also what moves the roll to the server for an online game, where a client must never choose its own number.
- **The rules that vary are data, not branches:** exact finish, extra turn on six (capped at three), chained jumps, entry rule — all fields on `GameRules`. And `play_turn` returns a `TurnResult`, **never prints**, so one `Game` serves a terminal, a phone and a simulation.
- **The win check comes AFTER the jump** — a ladder can win the game. **Chained jumps must be validated at load time** (no jump may end on another jump’s start) or 30↔60 loops for ever. Any “play until someone wins” loop needs a **turn cap**, because the distribution has a long tail.
- **The numbers:** one game $\approx$ 2.3 KB, **board shared not copied** $\rightarrow$ 100,000 concurrent games $\approx$ **110 MB**; mean $\sim$ **39 turns** to finish, 90th percentile $\sim$ 72; $\sim$ 12,000 turns/second is nothing in CPU — the real constraint is **400,000 WebSockets $\approx$ 8 servers**. A game is **inherently serial: one writer per game**, so use a **version compare-and-set** plus a turn number for idempotency, never a lock.

DSA topic: Backtracking: the undo step **System design topic:** Design snake and ladder

Code these, in this order

One rule for the whole set: **before you run anything, count the lines that change shared state before the recursive call and the lines that change it back after.** They must match. Do the counting with your finger on the screen; the missing undo is always silent.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Letter Combinations of a Phone Number	LeetCode 17 (Medium)	One piece of state, one undo — the template with nothing else in the way.
2	Palindrome Partitioning	LeetCode 131 (Medium)	The validity check placed <i>before</i> the recursive call, not at the leaf.
3	Generate Parentheses	LeetCode 22 (Medium)	Two counters as arguments, so there is nothing to undo at all.
4	Restore IP Addresses	LeetCode 93 (Medium)	Several prunes, and a partial answer that must be undone by exact length.

On problem 1, write both styles

Write it with `current.append(...)` plus a pop, then again with `explore(prefix + letter)`. Confirm the same answers. Then say which one allocates more, and where.

On problem 2, move the check and measure

Write it checking `is_palindrome` at the leaf, then checking it before recursing. Count the calls both ways for a 16-character string of all the same letter. Record both numbers.

On problem 3, notice there is no undo

Both counters are parameters. Say why that means nothing needs restoring, and name the one thing in the function that still does.

On problem 4, break the undo deliberately

Remove the undo and run `"25525511135"`. Record the output length. Then put the undo after the loop instead of inside it and record what happens.

The invariant drill

1. State the invariant in one sentence.
2. Say what is true of every sibling branch if the invariant holds at every node.
3. Say what happens to the siblings if it fails at exactly one node.
4. Say why the failure raises no error.

The three-kinds-of-state drill

1. Name the three kinds of state that need undoing.
2. For permutations, name which of your lines belongs to which kind.
3. For N-Queens, list all four things that must be undone.
4. Say which kind is the one that does not look like state, and give an example.
5. Say what the rule is for whether something needs an undo at all.

The arguments drill

1. Write a version of combination sum that mutates a running total, with its undo.
2. Write the version that passes `remaining` as a parameter.
3. Say why the second one has nothing to undo.
4. Name three types that are safe to pass this way, and say what they have in common.
5. Say what this technique buys you beyond correctness.

The placement drill

1. Write the three-line block and say where exactly the undo goes.
2. Move it outside the loop, run subsets on `[1, 2, 3]`, record the output.
3. Say why the answers get longer and longer.
4. Quote the error that eventually appears.
5. Say why a “reset everything at the top of the function” version is worse than correct.

The early-return drill

1. Write a grid search that returns `True` on success.
2. Say whether the undo runs on that path.
3. Say when that is harmless and when it is a bug.
4. Rewrite it so the undo always runs, and say how many extra lines that cost.
5. Describe the symptom of the bug in terms of a test suite.

The two-styles drill

1. Write subsets both ways.
2. Say how many allocations each makes, in terms of nodes and leaves.

3. Compute both for permutations at $n = 9$.
4. Say what happens if you mix the styles, and quote the error.
5. Give the one-sentence justification you would say out loud for choosing the mutating version.

The pruning drill

1. Say the single line that turns enumeration into backtracking.
2. Say where it must go, and where it must not.
3. Run the N-Queens counter for $n = 6$ and $n = 8$ and record all four numbers.
4. Say what the undo has to do with whether pruning is affordable.
5. Say why every undo in this phase is $O(1)$, and what it would mean if yours were not.

The break-it drill

Trigger each and record the exact output or error:

1. One of two undoes omitted, in permutations.
 2. The undo placed after the loop.
 3. `current.pop()` after passing `current + [x]` down.
 4. `board[:]` used as a snapshot of a list of lists.
 5. `available.remove(x)` inside `for x in available`.
 6. The validity check moved to the leaf, timed rather than printed.
-

The one-object drill

1. Say why a snake and a ladder are the same object, in one sentence.
2. Write the board lookup in one line, and say what the default argument does.
3. Say how `is_ladder` is obtained, and why it is not stored.
4. Say what having two classes would cost, concretely.

The dice drill

1. Write the `Dice` interface and both implementations.
2. Write the test for “a player at 97 rolls a 5”.
3. Say why that test is impossible without the interface.
4. Name the three things a design must always let you replace.
5. Give the production reason, separate from the testing reason, for server-side rolls.

The rules drill

1. Name the four rules that vary between households.

2. Say where each one lives.
3. Add “you must roll a six to start” and say which files change.
4. Say what cap you would put on consecutive extra turns, and why.

The chained-jump drill

1. State the two possible answers to “a ladder lands you on a snake”.
2. Write the loop for the chaining version.
3. Construct a two-jump board that loops for ever.
4. Write the validation that rejects it, and say when it runs.
5. Say why load-time validation beats a runtime guard, and why you would still keep the guard.

The turn drill

1. Say the five steps of a turn in order.
2. Say where the win check goes and give the input that proves it.
3. Say what `play_turn` returns and why it does not print.
4. Name three callers that would use the same `Game` unchanged.
5. Say why `play_until_won` needs a turn cap.

The numbers drill

Compute each, showing the multiplication:

1. Memory for one live game, and for 100,000 of them with a shared board.
2. What the figure would be if each game copied the board.
3. Turns per second at 100,000 games, 4 players and 8 seconds a turn.
4. Servers needed for the WebSocket connections at 50,000 per server.
5. Wall-clock length of a four-player game.

The failure drill

For each, say what happens and what you would add:

1. Two actions arrive for the same game in the same millisecond.
2. A player’s phone retries the same roll.
3. A player closes the app mid-game and comes back an hour later.
4. A configuration file is edited to add a jump from square 100.
5. Someone asks for ludo instead.
6. A player rolls sixes twenty times in a row.

Two of the six are not code problems. Say which, and what the actual fix is.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why did you pop that element after the recursive call?* The invariant stated first, “not cleanup” said explicitly, the sibling-branch explanation, the counting rule for chooses against undoes, what you moved into the arguments, and the specific silent wrong answer that appears without it.
2. *Design snake and ladder for n players.* Snakes and ladders collapsed into one map in the first fifteen seconds, `Dice` as an interface with the testing reason and the anti-cheat reason, the varying rules as data, the turn in one sentence with the win check after the jump, and `TurnResult` returned rather than printed.
3. *What is the difference between backtracking and brute force?* One line — the validity check before the recursive call — plus the N-Queens numbers, and the sentence about why the undo is what makes that check affordable.

Before you move on

- ☐ I can state the invariant in one sentence, without saying “cleanup”.
- ☐ I can name the three kinds of state and give an example of the one that hides.
- ☐ I can count chooses against undoes before running anything.
- ☐ I know why anything passed as an argument needs no undo.
- ☐ I ran the version with the undo outside the loop and can explain the growing answers.
- ☐ I can say when an early return past the undo is safe and when it is a bug.
- ☐ I wrote both styles and can say where each one allocates.
- ☐ I know what happens if I mix the styles, and can quote the error.
- ☐ I can name the one line that makes it backtracking, and where it must go.
- ☐ I have the N-Queens pruning numbers for $n = 6$ and $n = 8$.
- ☐ I can say why every undo must be $O(1)$.
- ☐ I can collapse snakes and ladders into one object in one sentence.
- ☐ I can write the `Dice` interface and the three-line test it enables.
- ☐ I can name the four varying rules and say where they live.
- ☐ I can construct a board that loops, and write the validation that rejects it.
- ☐ I can say where the win check goes and why.

- [] I can give the 110 MB figure for 100,000 games and say what makes it that small.
- [] I answered all three questions above out loud.

N-Queens and constraint grids

DATA STRUCTURES AND ALGORITHMS

N-Queens and constraint grids

You can prune a search tree with a validity check and count the work you avoided.

SYSTEM DESIGN

Design an online auction

You can model bids, the closing rule, and concurrent bidding.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Place n queens on an n by n board.*
- *Design an auction site. Two bids arrive in the same millisecond.*

1 What this is, and why they ask it

A **queen** in chess attacks along its whole row, its whole column, and both diagonals. N-Queens asks you to put n of them on an $n \times n$ board so that no two attack each other.

Three sentences. The board has n^2 squares and you are choosing n of them, which looks like a gigantic search — $C(64, 8)$ is about four billion for the standard board. But **no two queens can share a row, so there is exactly one queen per row**, and that single observation collapses the problem from four billion to $8! = 40,320$. Then a validity check while you descend collapses it again, to two thousand nodes.

That collapse, from four billion to two thousand, is why they ask it. N-Queens is the standard demonstration that **the way you frame the choices matters more than the code**, and that **checking early beats checking late**. It is also where you meet the trick that makes the check $O(1)$: every diagonal has a constant $row - col$, and every anti-diagonal has a constant $row + col$, so three sets replace scanning the board.

Expect it in the second half of a coding round, usually as “*place n queens*” followed by “*now just count them*” followed by “*how would you make it faster*”.

2 The story

The hall had been the dining hall until Thursday, when the tables were carried out and forty-nine single desks were carried in, seven across and seven deep, for the scholarship test on Sunday morning.

Mr Iyer had been given the seating and he had one instruction from the principal, delivered in the corridor without stopping: the seven children from the coaching centre must not be able to see one another's work. Not in the same line across. Not in the same line front to back. And not along a slant, because the desks had been set out with wide gaps and if you sat at the corner of the hall you could see clean through to the opposite corner.

He worked it out on the Saturday evening with the lights on and the fans off.

There were seven of them and seven rows, so straight away one per row. That was not a choice; it was just true. Two of them in one row would be side by side, which was the first thing forbidden.

So he went row by row. First child in the front row, first desk. Second child in the second row — not the first desk, that was the same line front to back, and not the second desk either, that was on the slant. Third desk, then. Third child in the third row, and now two desks were blocked for being in line, and two more for the slants.

By the fifth row he was stuck. Every desk in it was blocked by somebody already sitting.

So he went back one row, moved that child one desk along, and carried on from there. When that ran out too, he went back another row. He did this eleven or twelve times over the course of an hour, and each time he moved somebody he made sure to free up the desks that child had been blocking, because twice early on he forgot and ended up with a hall where nothing fitted anywhere and he could not see why.

He was not walking the whole hall each time either. He kept three things in his head: which desks across were taken, which slants going one way were taken, and which slants going the other way. Then a desk was fine if it was in none of the three.

He finished at about half past eight and wrote the arrangement in his phone. Forty of the forty-nine desks were empty, and the seven children were placed so that no two of them shared a line in any direction at all.

3 The idea in plain English

Mr Iyer has just solved 7-Queens, including the pruning and including the reason the undo matters.

- Each child is a **queen**. Each desk is a square.
- “Seven children and seven rows, so one per row” is the **framing decision** that makes the problem small.
- “In line across”, “in line front to back” and “on the slant” are the three ways two queens attack.
- Keeping three things in his head, rather than checking the whole hall, is the **O(1) validity check**.
- Going back a row when he got stuck is **backtracking**, and freeing up the blocked desks when he moved somebody is the **un-choose** that he twice forgot.

The framing decision, which is the whole problem

choosing n squares from n^2	$C(64, 8)$	= 4,426,165,368	for n = 8
one queen per row	8^8	= 16,777,216	
one per row AND one per column	$8!$	= 40,320	
with the diagonal check while descending		= 2,057 nodes visited	

Four billion, then sixteen million, then forty thousand, then two thousand. **Each step is one observation, not one line of code.** Saying those four numbers out loud in an interview is worth more than the implementation, because it shows you reduce before you code.

The representation follows from the framing. If there is one queen per row, you do not need a grid at all:

```
placement = [3, 1, 6, 2, 5, 7, 4, 0]           # placement[row] = column
```

One list of n integers, not an $n \times n$ grid. The row is the position in the list, so a row clash is impossible by construction — it cannot be expressed.

The diagonal trick

Two queens are on the same diagonal if the difference between their rows equals the difference between their columns. That is awkward to check pairwise. The trick is to give every diagonal a name.

```
a "\" diagonal (top-left to bottom-right): row - col is CONSTANT along it
a "/" diagonal (top-right to bottom-left): row + col is CONSTANT along it
```

Check it on a 4×4 board:

row - col					row + col				
	c0	c1	c2	c3		c0	c1	c2	c3
r0	[0	-1	-2	-3]	r0	[0	1	2	3]
r1	[1	0	-1	-2]	r1	[1	2	3	4]
r2	[2	1	0	-1]	r2	[2	3	4	5]
r3	[3	2	1	0]	r3	[3	4	5	6]

every "\" diagonal is one value on the left; every "/" diagonal is one value on the right

So the check becomes three membership tests:

```
if col in used_cols or (row - col) in used_diag or (row + col) in used_anti:
    continue                               # attacked; do not descend
```

Three set lookups, $O(1)$ each, instead of scanning the board. Scanning would be $O(n)$ per candidate and $O(n^2)$ per row, and on $n = 12$ the difference is minutes against milliseconds.

The ranges are worth knowing, because the array version needs them:

row - col	ranges from $-(n-1)$ to $(n-1)$	→ $2n - 1$ values, needing an offset of $n-1$
row + col	ranges from 0 to $2n - 2$	→ $2n - 1$ values, no offset needed

If you use lists instead of sets — which is faster in Python — you must add `n - 1` to `row - col` or you index from the wrong end of the list and get silent nonsense. That is the trap below.

The shape of the recursion

```
def place(row):
    if row == n:
        record()                # all n rows filled: a solution
        return
    for col in range(n):
        if attacked(row, col):
            continue            # PRUNE
        mark(row, col)          # choose
        place(row + 1)          # recurse
        unmark(row, col)        # un-choose
```

Row is the depth. You never search for “where to put the next queen” across the whole board — you ask “which column in *this* row”, which is exactly one loop of `n`.

And notice there is no explicit “is this a dead end” test. **A dead end is a row where every column is attacked**, and it handles itself: the loop finishes without recursing, the function returns, and the caller undoes its own choice and tries the next column. Mr Iyer going back a row.

Counting versus listing

LeetCode 51 wants the boards; LeetCode 52 wants only how many there are.

n:	1	2	3	4	5	6	7	8	9	10	11	12
solutions:	1	0	0	2	10	4	40	92	352	724	2680	14200

n = 2 and n = 3 have zero solutions, which is the edge case an interviewer will hand you. n = 8 has 92, and n = 12 has 14,200 — and if you only need the count, you skip building 14,200 board strings, which is most of the running time.

There is no formula. The sequence is computed, not derived, and saying so is better than guessing.

The same shape, one level up: constraint grids

Once you have N-Queens, Sudoku is the same program with different bookkeeping:

	N-QUEENS	SUDOKU
Depth	one row	one empty cell
Choices	<code>n</code> columns	digits 1–9
Constraint sets	column, <code>r-c</code> , <code>r+c</code>	row, column, 3×3 box
Prune	3 membership tests	3 membership tests
Undo	remove from 3 sets	remove from 3 sets, clear the cell

Three sets in both cases, and the box index for Sudoku is `(r // 3) * 3 + (c // 3)`, which is the same kind of arithmetic trick as `row + col`. That is why these two problems are always taught together — they are one technique with two boards.

4 The picture

The search on a 4×4 board, with the pruning visible.

```
row 0: try col 0      [Q . . .]      cols={0} diag={0} anti={0}
      row 1: col 0 blocked (column)
            col 1 blocked (anti: 1+1=2? no — diag: 1-1=0 IS in diag) → blocked
            col 2 ok      [. . Q .]      cols={0,2} diag={0,-1} anti={0,3}
      row 2: col 0 col 1 anti(2+1=3) col 2 col 3 diag(2-3=-1)
            ALL FOUR BLOCKED → dead end, return
            col 3 ok      [. . . Q]      cols={0,3} diag={0,-2} anti={0,4}
      row 2: col 1 ok      [. Q . .]      cols={0,3,1} ...
            row 3: every column blocked → dead end
            ... backtrack out of col 3 as well
      col 0 in row 0 yields nothing. UNDO and try col 1.

row 0: try col 1      [. Q . .]
      row 1: col 3      [. . . Q]
            row 2: col 0 [Q . . .]
                  row 3: col 2 [. . Q .] ← SOLUTION
```

The solution, drawn:

```
      c0  c1  c2  c3
r0 [ .  Q  .  . ]      placement = [1, 3, 0, 2]
r1 [ .  .  .  Q ]
r2 [ Q  .  .  . ]      no two share a row      (one per row by construction)
r3 [ .  .  Q  . ]      no two share a column    (cols = {1,3,0,2}, all distinct)
                        no two share r-c        ({-1,-2,2,1}, all distinct)
                        no two share r+c        ({1,4,2,5}, all distinct)
```

What to notice: **the three sets are exactly the three conditions**, and “all distinct” is what a set enforces for free.

The two diagonal families, so the arithmetic is not a claim:

"\" diagonals: constant row - col

```
      c0 c1 c2 c3
r0 [  0 -1 -2 -3 ]
r1 [  1  0 -1 -2 ]
r2 [  2  1  0 -1 ]
r3 [  3  2  1  0 ]
      \  \  \  \
      the 0s form one diagonal
      (0,0) (1,1) (2,2) (3,3)
```

row - col ranges -3..3 → 7 values, offset by n-1 = 3 to index a list
row + col ranges 0..6 → 7 values, no offset

"/" diagonals: constant row + col

```
      c0 c1 c2 c3
r0 [  0  1  2  3 ]
r1 [  1  2  3  4 ]
r2 [  2  3  4  5 ]
r3 [  3  4  5  6 ]
      /  /  /  /
      the 3s form one anti-diagonal
      (0,3) (1,2) (2,1) (3,0)
```

And the work avoided, which is the point of the day:

n = 8

approach	nodes visited	ratio
choose 8 squares from 64, check at the end	4,426,165,368	—
one per row, check at the end	19,173,961	231×
one per row + column, check at the end	109,601	40,384×
check all three sets while descending	2,057	2,151,756×

the last row is the same program as the second, with one `if` moved from the bottom of the tree to the top of the loop.

5 The code, built step by step

Step 1 — reduce out loud before writing anything

“No two queens can share a row, and there are `n` queens and `n` rows, so there is exactly one per row. That means I am not choosing squares — I am choosing one column per row, which is `n!` orderings rather than `C(n2, n)` squares. For `n = 8` that is forty thousand instead of four billion.”

That sentence is most of the interview. Say it before you touch the keyboard.

Step 2 — choose the representation the framing implies

```
placement: list[int] = []
```

```
# placement[row] = column
```

One list, `n` integers. A row conflict is now unrepresentable, which is the best kind of invariant: enforced by the data structure rather than by a check.

Step 3 — the three sets

```
cols: set[int] = set()           # columns in use
diag: set[int] = set()          # row - col
anti: set[int] = set()          # row + col
```

Say why each one identifies a diagonal: “every square on a `\` diagonal has the same `row - col`, and every square on a `/` diagonal has the same `row + col`.” One sentence, and it justifies the whole check.

Step 4 — the check, placed before the descent

```
for col in range(n):
    if col in cols or (row - col) in diag or (row + col) in anti:
        continue                # PRUNE — never descend into this
```

This `if` is the entire lesson. Move it to the leaf and `n = 8` goes from two thousand nodes to nineteen million.

Step 5 — choose, recurse, un-choose — three of each

```
cols.add(col); diag.add(row - col); anti.add(row + col)
placement.append(col)
place(row + 1)
placement.pop()
cols.remove(col); diag.remove(row - col); anti.remove(row + col)
```

Four things changed, four things restored. Count them on the screen before you run it. This is the day-094 rule doing real work: with one piece of state you can get away with sloppiness, with four you cannot.

Step 6 — render only at the end

```
if row == n:
    solutions.append(["." * c + "Q" + "." * (n - c - 1) for c in placement])
    return
```

Build the board strings **only at a solution**, never while descending. For `n = 12` there are 14,200 solutions and about 850,000 nodes; rendering at every node would do sixty times the string work for nothing.

The complete solution

```
def solve_n_queens(n: int) → List[List[str]]:
    """LeetCode 51. Every arrangement of n queens on an n x n board.

    The framing decision does the work: one queen per row, so a placement is
    a list of n columns rather than a set of n squares.  $C(n^2, n)$  becomes  $n!$ .

    Then three  $O(1)$  checks replace scanning the board:
        same column      → col
        same "\" diagonal → row - col is constant along it
        same "/" diagonal → row + col is constant along it

    Four things are changed before the recursive call and four restored after.
    """
    solutions: List[List[str]] = []
    placement: List[int] = []
    cols: Set[int] = set()
    diag: Set[int] = set()          # row - col
    anti: Set[int] = set()         # row + col

    def place(row: int) → None:
        if row == n:
            solutions.append(
                ["." * c + "Q" + "." * (n - c - 1) for c in placement]
            )
            # render ONLY at a solution
            return
        for col in range(n):
            if col in cols or (row - col) in diag or (row + col) in anti:
                continue
            # PRUNE before descending
            cols.add(col); diag.add(row - col); anti.add(row + col)
            placement.append(col)
            # choose (4 changes)
            place(row + 1)
            # recurse
            placement.pop()
            cols.remove(col); diag.remove(row - col); anti.remove(row + col)
            # un-choose (4 restores)

    place(0)
    return solutions

def total_n_queens(n: int) → int:
    """LeetCode 52. Only the count, so never build a board.

    Same tree, and roughly twice as fast for large n because 14,200 board
    renderings at n = 12 is most of the running time of the other function.
    """
    cols: Set[int] = set()
    diag: Set[int] = set()
    anti: Set[int] = set()
    count = 0

    def place(row: int) → None:
        nonlocal count
        if row == n:
            count += 1
            return
        for col in range(n):
            if col in cols or (row - col) in diag or (row + col) in anti:
                continue
            cols.add(col); diag.add(row - col); anti.add(row + col)
            place(row + 1)
            cols.remove(col); diag.remove(row - col); anti.remove(row + col)
```

```

place(0)
return count

def total_n_queens_bitmask(n: int) → int:
    """The version to mention, not necessarily to write.

    Each set becomes an integer whose bits mark blocked columns. The available
    columns are computed in one expression, and the lowest set bit is picked
    with `available & -available`. Roughly 5-10x faster than the set version
    because there is no hashing and no allocation at all.

    `cols` is absolute. `diag` and `anti` SHIFT by one each row, because a
    diagonal moves one column across as you move one row down.
    """
    full = (1 << n) - 1

    def place(cols: int, diag: int, anti: int) → int:
        if cols == full:
            return 1
        available = full & ~(cols | diag | anti)
        count = 0
        while available:
            bit = available & -available          # lowest set bit
            available -= bit
            count += place(cols | bit, (diag | bit) << 1, (anti | bit) >> 1)
        return count

    return place(0, 0, 0)

def solve_sudoku(board: list[list[str]]) → bool:
    """The same technique on a different board. Three sets again:
    row, column, and 3x3 box – where the box index is (r//3)*3 + (c//3).

    Modifies `board` in place and returns True when solved.
    """
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    blanks: list[tuple[int, int]] = []

    for r in range(9):
        for c in range(9):
            value = board[r][c]
            if value == ".":
                blanks.append((r, c))
            else:
                rows[r].add(value); cols[c].add(value)
                boxes[(r // 3) * 3 + (c // 3)].add(value)

    def fill(k: int) → bool:
        if k == len(blanks):
            return True
        r, c = blanks[k]
        b = (r // 3) * 3 + (c // 3)
        for digit in "123456789":
            if digit in rows[r] or digit in cols[c] or digit in boxes[b]:
                continue          # PRUNE
            rows[r].add(digit); cols[c].add(digit); boxes[b].add(digit)
            board[r][c] = digit    # choose (4 changes)
            if fill(k + 1):
                return True        # found: unwind, no undo needed
            rows[r].remove(digit); cols[c].remove(digit); boxes[b].remove(digit)
            board[r][c] = "."      # un-choose (4 restores)

```

```

        return False

    return fill(0)

def count_nodes(n: int) → tuple[int, int]:
    """Nodes visited with the check on the way down, and with it at the leaf.
    Run this rather than trusting the numbers in the lesson."""
    early = 0
    late = 0

    def with_prune(row, cols, diag, anti):
        nonlocal early
        early += 1
        if row == n:
            return
        for col in range(n):
            if col in cols or (row - col) in diag or (row + col) in anti:
                continue
            with_prune(row + 1, cols | {col}, diag | {row - col}, anti | {row + col})

    def at_the_leaf(row, placement):
        nonlocal late
        late += 1
        if row == n:
            return
        for col in range(n):
            at_the_leaf(row + 1, placement + [col])

    with_prune(0, set(), set(), set())
    at_the_leaf(0, [])
    return early, late

if __name__ == "__main__":
    for board in solve_n_queens(4):
        for line in board:
            print(line)
        print()
    # . Q . .
    # . . . Q
    # Q . . .
    # . . Q .      (and its mirror image)

    print([total_n_queens(n) for n in range(1, 11)])
    # [1, 0, 0, 2, 10, 4, 40, 92, 352, 724]
    #   ^^^^ n = 2 and n = 3 have NO solutions

    print(total_n_queens(8), total_n_queens_bitmask(8))      # 92 92

    print(count_nodes(6))      # (543, 55987)
    print(count_nodes(8))      # (2057, 19173961)

    puzzle = [list(row) for row in [
        "53..7....", "6..195...", ".98....6.",
        "8...6...3", "4..8.3..1", "7...2...6",
        ".6....28.", "...419..5", "....8..79",
    ]]
    print(solve_sudoku(puzzle, "".join(puzzle[0]))
    # True 534678912

```

6 What it costs

Time

The honest statement, and interviewers accept it because it is the truth:

upper bound: $O(n!)$ – at most one queen per row and per column, so at most $n!$ placements
with pruning: far less, and there is no closed form

There is no formula for the number of solutions or the number of nodes. Both are computed. Quote the measurements instead:

n	nodes visited	solutions	nodes per solution
6	543	4	136
8	2,057	92	22
10	35,538	724	49
12	856,189	14,200	60
14	27,358,552	365,596	75

$n = 8$ in milliseconds, $n = 12$ in about a second, $n = 14$ in half a minute, $n = 16$ is minutes. That progression is the answer to “how large can n be”, and having it ready is far better than “it is exponential”.

Per node the work is a loop of n columns and three set lookups each, so $O(n)$ per node with a small constant.

The pruning, measured

n = 8		
check at the leaf, one per row:	19,173,961 nodes	
check while descending:	2,057 nodes	
ratio	9,321×	
n = 10		
check at the leaf:	~6,257,890,000 nodes (estimated; it does not finish)	
check while descending:	35,538 nodes	

One **if**, moved from the bottom of the tree to the top of the loop. This is the single most convincing number in the whole phase and it is worth memorising.

Sets versus lists versus bitmasks

All three are $O(1)$ per check. The constants differ a great deal in Python:

approach	relative time at $n = 12$	why
three sets	1.0×	hashing on every add/remove
three bool lists	0.6×	indexing, no hashing
three integers (bits)	0.15×	no allocation, no hashing at all

Write the set version; mention the bitmask one. The set version is readable and correct in thirty seconds; the bitmask version is the answer to “can you make it faster” and takes two minutes to explain.

The bitmask shift is the part to be able to justify: `cols` is absolute so it never moves, but a diagonal moves one column sideways for each row you move down — so `diag` shifts left and `anti` shifts right on every descent.

Space

```
placement:      n integers                0(n)
three sets:     at most n entries each    0(n)
stack:          n frames                  0(n)
-----
extra space:                    0(n)
output:         (number of solutions) × n strings of length n
```

Extra space is $O(n)$, not $O(n^2)$. You never allocate the board — that is what the one-queen-per-row representation bought. The output for $n = 12$ is $14,200 \times 12$ strings of 12 characters ≈ 2 MB, which is why `total_n_queens` is meaningfully faster than `solve_n_queens`.

The symmetry halving

Every solution has a mirror image, so you may fix the first queen to the left half of the first row and double the count (with a correction for odd `n`, where the middle column is its own mirror).

```
n = 12    full search: 856,189 nodes
          half search: ~428,000 nodes, then double the count
```

A factor of two, not a change of complexity. Worth mentioning as a refinement, never worth presenting as the main idea.

7 The traps

Trap 1 — checking at the leaf

```
if row == n:
    if is_valid(placement):      # checking only once everything is placed
        solutions.append(...)
    return
```


Correct answer, 19 million nodes at $n = 8$ instead of 2,057, and it will not finish at $n = 10$. Nothing tells you it is wrong; it just never returns. **The check must run before the descent.**

Trap 2 — negative list indices from `row - col`

```
diag = [False] * (2 * n - 1)
if diag[row - col]:                # row - col can be NEGATIVE
```

Python happily accepts a negative index — it counts from the end — so `row - col = -3` reads `diag[-3]`, which is a real entry belonging to a different diagonal. **No error, silently wrong answers.**

```
solve_n_queens(8) with this bug → fewer than 92 solutions, and some invalid
```

The fix is an offset: `diag[row - col + n - 1]`. This is the reason the set version is worth writing first — sets have no such trap.

Trap 3 — forgetting one of the four undoes

```
cols.add(col); diag.add(row - col); anti.add(row + col)
placement.append(col)
place(row + 1)
placement.pop()
cols.remove(col); diag.remove(row - col)
# missing: anti.remove(row + col)
```

The anti-diagonal stays blocked for ever, so most solutions are never found.

```
total_n_queens(8) → 0
```

Zero, with no error. **Four changes, four restores.** Count them before running.

Trap 4 — mixing up which diagonal is which

`row + col` is the `/` diagonal and `row - col` is the `\` diagonal. Swapping them does not break anything — both sets still identify a family of diagonals correctly, and the answer stays right. It only matters if you also switch to lists, where the offset differs between the two. Worth knowing so you do not waste time debugging a non-bug.

Trap 5 — building the board at every node

```
board = [ "." * n for _ in range(n) ]    # inside place(), at every call
```

$O(n^2)$ allocation per node, on a tree with hundreds of thousands of nodes. At $n = 12$ this is the difference between one second and a minute. **Carry the column list; render once, at a solution.**

Trap 6 — assuming there is always a solution

```
solve_n_queens(2) → []  
solve_n_queens(3) → []
```

$n = 2$ and $n = 3$ have **no** solutions, and code that assumes a non-empty result will fail there. It is the first edge case an interviewer reaches for, and answering “two and three have none, everything from four up does” immediately is a small, cheap signal.

Trap 7 — `for col in range(n)` inside a per-cell search

For Sudoku, the equivalent mistake is scanning for the next blank cell from the start of the board on every call:

```
for r in range(9):  
    for c in range(9):  
        if board[r][c] == ".":      # 81 cells scanned per call, every call
```

Pre-compute the list of blanks once and index into it by depth. On a puzzle with 55 blanks that is a $50\times$ reduction in the constant factor for free.

Trap 8 — quoting $O(n!)$ as if it were tight

$n!$ is an upper bound and it is loose. The pruning removes almost all of it. If asked, say: **“ $O(n!)$ as a bound, but that is not what happens — at $n = 8$ it is 2,057 nodes against 40,320 placements, and there is no closed form for the real figure. I would quote measurements.”** That is a more accurate and more impressive answer than a symbol.

8 In the interview

How it gets asked

- The base: “Place n queens on an n by n board so that none attack each other. Return all arrangements.” LeetCode 51.
- The lighter version: “Just return how many there are.” LeetCode 52.
- The optimisation: “Can you make the conflict check faster?” — they want the three sets, then the bitmask.
- The sibling: “Now solve a Sudoku board.” LeetCode 37.
- The probe that catches people: “What is $n = 3$?”

What to say out loud, in the first ninety seconds

1. **Reduce before coding.** “No two queens share a row and there are n of each, so there is exactly one queen per row. That turns ‘choose n squares from n^2 ’ — four billion for $n = 8$ — into ‘choose one column per row’, which is at most $8! = 40,320$.”
2. **State the representation.** “So a placement is a list of n column numbers, not a grid. A row conflict becomes unrepresentable rather than checked.”
3. **Give the diagonal trick with its justification.** “Every square on a $\backslash$ diagonal has the same $\text{row} - \text{col}$, and every square on a $/$ diagonal has the same $\text{row} + \text{col}$. So I keep three sets — columns, $\text{row} - \text{col}$, $\text{row} + \text{col}$ — and the conflict check is three $O(1)$ lookups instead of scanning the board.”
4. **Point at the pruning line explicitly.** “The check happens *before* I descend. That is the whole algorithm: at $n = 8$, checking on the way down visits about two thousand nodes and checking at the leaf visits nineteen million.”
5. **Count the undoes.** “Four things change before the recursive call — three sets and the placement — so four things are restored after it. With one piece of state you can be sloppy; with four you cannot.”
6. **Be honest about the complexity.** “ $O(n!)$ is an upper bound and it is loose, because the pruning removes almost all of it. There is no closed form. $n = 8$ is milliseconds, $n = 12$ is about a second, $n = 14$ is half a minute.”

The follow-ups

“**What is the time complexity?**” “The upper bound is $O(n!)$ — one queen per row and no two in a column means at most $n!$ placements, and each costs $O(n)$ to check. But that bound is loose and I would not pretend otherwise: the pruning cuts almost all of it and there is no closed form for what remains. The measured node counts are 2,057 at $n = 8$ and about 856,000 at $n = 12$, against 40,320 and 479 million placements. Space is $O(n)$ — the column list, three sets of at most n entries, and n stack frames. I never allocate a board, which is what the one-queen-per-row representation bought.”

“**Can you make the conflict check faster?**” “Yes, in two steps. The sets can become three boolean lists, which drops the hashing — but then $\text{row} - \text{col}$ is negative and Python will happily index from the end of the list and give me silently wrong answers, so it needs an offset of $n - 1$. Beyond that, the three sets become three integers used as bitmasks: available columns are $\text{full} \& \sim(\text{cols} | \text{diag} | \text{anti})$ in one expression, and I take the lowest set bit with $\text{available} \& -\text{available}$. The subtlety is that cols is absolute but the diagonals *shift* — a diagonal moves one column sideways for each row down — so diag shifts left and anti shifts right on each descent. That is roughly five to ten times faster in Python because there is no hashing and no allocation. I would write the set version first because it is readable, and offer this when asked.”

“What about $n = 2$ and $n = 3$?” “No solutions at all — the answer is an empty list, and any code that assumes a non-empty result breaks there. $n = 1$ has one, $n = 4$ has two, $n = 8$ has 92. There is no formula for the count; the sequence is computed.”

“Now solve a Sudoku board.” “Same technique, different bookkeeping. Depth becomes the index of the next blank cell rather than the row — and I would pre-compute the list of blanks once rather than scanning eighty-one cells on every call, which is a fifty-fold constant-factor saving on a typical puzzle. The choices are the digits one to nine, and the three constraint sets are the row, the column, and the 3×3 box, where the box index is $(r // 3) * 3 + (c // 3)$ — the same kind of arithmetic trick as $row + col$ for diagonals. The prune is three membership tests before descending, and I restore all three plus the cell on the way out. The one difference is that Sudoku returns as soon as it finds a solution rather than exploring everything, so it returns `True` up the stack.”

“Would ordering the choices help?” “Yes, and it is the standard next optimisation. For Sudoku, instead of taking blanks in order, always fill the cell with the fewest legal digits remaining — that is the ‘most constrained variable’ heuristic, and it turns hard puzzles from seconds into milliseconds because it fails fast rather than deep. For N-Queens the equivalent is picking the row with the fewest available columns. Both cost a scan per node to compute, so they pay off exactly when the tree would otherwise be deep, which is precisely when you need them.”

“How much does the symmetry save?” “A factor of two, no more. Every solution has a mirror image, so I can fix the first queen to the left half of row zero and double the count — with a correction for odd `n`, where the middle column is its own mirror. It is a constant-factor refinement, not a change in complexity, and I would mention it after the real answer rather than instead of it.”

A model answer

Asked: *place n queens on an n by n board.*

“Let me reduce the problem before I write anything, because the reduction is most of the work here.

A queen attacks along its row, so **no two queens can be in the same row**. There are n queens and n rows, so there is exactly one queen per row — that is not a strategy, it is forced. That single observation changes what I am searching over. I am not choosing n squares out of n^2 , which for $n = 8$ is $C(64, 8)$, about four and a half billion. I am choosing **one column for each row**, which is at most $8! = 40,320$.

So the representation is a list of n integers, where position i holds the column of the queen in row i . A row conflict is now impossible to even express, which is better than checking for it.

That leaves columns and diagonals. Columns are easy — a set of used columns. Diagonals use a trick worth stating carefully: **every square on a \ diagonal has the same $\text{row} - \text{col}$, and every square on a / diagonal has the same $\text{row} + \text{col}$** . So I keep two more sets, keyed on those values, and the whole conflict test is three constant-time membership checks rather than a scan of the board.

The recursion is one function taking the row. If the row equals n , every queen is placed and I record a solution. Otherwise I loop over the columns, and — this is the important part — **I test the three sets before I descend, not after**. If the square is attacked I skip it entirely and that whole subtree is never walked. That one placement of one `if` is the difference between two thousand nodes and nineteen million at $n = 8$.

Then choose, recurse, un-choose. **Four things change before the call** — the three sets and the placement list — **so four things are restored after it**. I count them on the screen before running, because with four pieces of state a missing restore is silent: forget the anti-diagonal removal and the function returns zero solutions with no error.

I render the board strings only when I reach a solution, never on the way down — at $n = 12$ there are 14,200 solutions and about 850,000 nodes, so rendering per node would be sixty times the string work for nothing.

On complexity I would be honest: $O(n!)$ is an upper bound, it is loose, and there is no closed form for what the pruning leaves. I would give measurements — 2,057 nodes at $n = 8$, about 856,000 at $n = 12$ — and say that $n = 8$ is milliseconds, $n = 12$ is around a second and $n = 14$ is half a minute. Space is $O(n)$: the column list, three sets, and n frames.

Two things I would add if you want it faster. The sets can become three integers used as bitmasks, where the available columns are one bitwise expression and the diagonals shift by one each row — that is five to ten times quicker in Python. And symmetry gives a factor of two, since every solution has a mirror. And one edge case worth stating up front: **$n = 2$ and $n = 3$ have no solutions at all.**“

9 Recall card

- **The reduction is the answer: no two queens share a row, so there is exactly one per row.** That turns “choose n squares from n^2 ” — 4.4 billion at $n = 8$ — into “one column per row”, at most $8! = 40,320$. The representation follows: **a list of n column numbers, never a grid**, so a row conflict is unrepresentable rather than checked.
- **Every $\backslash$ diagonal has a constant $\text{row} - \text{col}$; every $/$ diagonal has a constant $\text{row} + \text{col}$.** So three sets — `cols`, `row - col`, `row + col` — make the conflict test **three $O(1)$ lookups**. With lists instead of sets, `row - col` is negative and Python indexes from the end: **silently wrong answers**, so add an offset of `n - 1`.
- **The `if` goes BEFORE the descent.** Checking on the way down visits **2,057 nodes at $n = 8$** ; checking at the leaf visits **19,173,961**. Same program, one line moved. This is the most convincing number in the phase.
- **Four changes, four restores** — three sets and the placement. Forget the anti-diagonal removal and `total_n_queens(8)` returns **0**, with no error. And **render the board only at a solution**: at $n = 12$ that is 14,200 renders instead of 856,000.
- **$O(n!)$ is a loose upper bound and there is no closed form** — quote measurements: $n = 8$ ms, $n = 12$ a second, $n = 14$ half a minute. **$n = 2$ and $n = 3$ have no solutions.** Space is $O(n)$. Faster: **bitmasks**, where `cols` is absolute but `diag` shifts left and `anti` shifts right each row (5–10 $\times$); **symmetry** gives $2\times$. **Sudoku is the same program**: depth = next blank, choices = 1–9, sets = row, column, box at `(r//3)*3 + (c//3)`.

1 What this is, and why they ask it

An **auction** is a listing with a price that only ever goes up, a deadline, and a rule for who wins. People place **bids**; the highest one when the deadline passes takes the item.

Three sentences. The state is tiny — an item, a current price, a leading bidder, an end time — and the difficulty is entirely in **what happens when two people act at the same instant**. The second difficulty is that a real auction does not store “the price you offered”: it stores **the maximum you are willing to pay**, and shows a lower number. And the third is that **nothing happens at the deadline unless somebody makes it happen**, which is a design decision candidates almost always skip.

They ask it because it is the smallest realistic problem where **you cannot get away with eventual consistency**. A like count that is stale for two seconds is fine. A price that is stale for two seconds means you accepted a bid of ₹500 on an item that was already at ₹600, told the bidder they were winning, and are now going to have to explain that to them. Money forces a correct answer, and the correct answer is a single conditional write — not a lock, not a read-then-write, and definitely not a cache.

“Two bids arrive in the same millisecond” is the question. Everything else in the prompt is setup for it.

2 The story

The fish came in at about half past four and the selling started at five, on the wet concrete under the one working light.

Sekhar had been going for eleven years and knew how it worked. The man with the stick stood by the crate, called out a number, and men in the ring called out higher ones until they stopped.

Three things happened that morning that were worth watching.

The first was a crate of prawns where two men called out eight hundred at the same instant. Not nearly the same. The same. There was a second of complete silence and then both of them looked at the man with the stick, because there was no way for either of them to decide it and only he could. He pointed at the man on his left and said, you, and

then immediately said eight hundred and fifty to the ring so that nobody could argue about it. Sekhar noticed that he did not hesitate. If he had hesitated, both of them would have started shouting.

The second was Ibrahim, who had to be at the ice house by half past five and could not stay. Before he left he went up to the man with the stick and said something quietly, and Sekhar knew what it was because everybody did it. He was leaving a number. Go up to twelve hundred for me on the second crate, but only as far as you have to. And that is what happened — the crate went to Ibrahim at nine hundred and fifty, because that is where the next man stopped, and Ibrahim never paid his twelve hundred and never knew what it would have gone to.

The third was the end of a crate of mackerel. The man with the stick said seven hundred, going, going — and on the word going, somebody at the back said seven fifty. And instead of finishing, he started again from seven fifty, going, going. He did that four times on that crate. Sekhar asked him about it once, years ago, and the man said the obvious thing: if I stop the moment somebody speaks, then the clever ones learn to speak at the last possible second and nobody else gets a turn. So I always start again.

By six the concrete was being hosed down.

3 The idea in plain English

Sekhar has watched the three hard parts of an auction system, in order.

- Two men calling eight hundred at once, and one person deciding, is **concurrency control**. The important detail is *where* the decision is made: not by the bidders, and not by two separate people — by **one authority, in one place, without hesitating**.
- Ibrahim leaving a number is **proxy bidding**, also called automatic or maximum bidding. You tell the system the most you will pay; it bids on your behalf only as high as it needs to.
- Starting the count again when somebody speaks at the last moment is **anti-sniping**: extending the deadline whenever a bid arrives near the end.

The state, which is almost nothing

```
Auction:  item, seller, start_price, reserve_price, current_price,
          leading_bidder, leading_max, ends_at, status, version
```

Ten fields. The whole difficulty is in changing three of them atomically.

An auction moves through states, and each transition has exactly one cause:


```
DRAFT → SCHEDULED → OPEN → ENDED → SOLD
                                   \→ UNSOLD (reserve not met, or no bids)
                                   \→ CANCELLED (seller, before any bid)
```

ENDED and **SOLD** are different states on purpose. The deadline passing ends the bidding; deciding who won and whether the reserve was met is a separate step that may involve a payment attempt. Merging them is how you end up with an auction that is “sold” to someone whose card then declines.

Proxy bidding: what is stored is not what is shown

This is the part that surprises people, and it is how eBay has always worked.

When you “bid ₹1,200”, the system does **not** set the price to ₹1,200. It records that your maximum is ₹1,200, and then sets the visible price to **just enough to beat the previous leader** — the second highest maximum plus one increment.

```
increment = ₹50

Ibrahim bids max 1200    → price 800 (the opening), Ibrahim leading, his max 1200 hidden
Sekhar  bids max  900    → Sekhar's 900 loses to Ibrahim's 1200
                        → price becomes 950 (Sekhar's 900 + one increment, capped at 1200)
                        → Ibrahim still leading, and never paid 1200
Sekhar  bids max 1300    → beats 1200
                        → price becomes 1250 (Ibrahim's 1200 + increment)
                        → Sekhar leading
```

The displayed price is derived from the loser’s maximum, not the winner’s. Two consequences worth saying out loud:

1. **A bid can lose without changing the price at all** — if your maximum is below the leader’s, the price moves to your maximum plus an increment, still under the leader’s ceiling. You “bid and immediately lost”.
2. **Maximums must never be visible**, not in the response, not in the timeline, not in an admin endpoint that the front end happens to call. Leaking a maximum is the single worst bug this system can have, because it lets a seller’s friend bid the winner up to exactly their ceiling.

The closing problem

At `ends_at`, nothing happens. Code does not run because a timestamp passed.

Two designs, and you must pick one out loud:

	SCHEDULED CLOSE	LAZY CLOSE
How	a job (or a delayed queue message) fires at <code>ends_at</code> and finalises	the auction is treated as closed by any read after <code>ends_at</code> ; a sweeper finalises later
Bid at <code>ends_at</code> + 1ms	rejected, because the state is already <code>ENDED</code>	rejected, because the check is <code>now < ends_at</code>
If the job is late	the auction stays <code>OPEN</code> and accepts a late bid	correct anyway — reads compute the truth
Cost	one timer per auction	one comparison per read

Lazy close is the correct default, with a sweeper for the side effects. The rule to state: *the truth about whether bidding is open is `now < ends_at` , evaluated at bid time, not a status field that something has to remember to update.* Then the scheduled job only does the things that genuinely need doing once — charging, notifying, releasing the item — and if it runs three seconds late, nothing has been accepted that should not have been.

Anti-sniping

A **snipe** is a bid placed in the last second, leaving nobody time to respond. It is not cheating, and it is rational, and it makes auctions worse: honest bidders stop bothering.

The fix is the man with the stick starting again:

```
if bid_time > auction.ends_at - EXTENSION_WINDOW:      # e.g. the last 2 minutes
    auction.ends_at = bid_time + EXTENSION_WINDOW
```

Two decisions to state. Is the extension capped — can an auction be extended for ever? And is the window the same for a ₹500 item and a ₹5,00,000 one? Both are policy, so both belong in a rules object rather than in the bidding code, exactly as with the game rules on [day 094](#).

4 The picture

The class diagram.

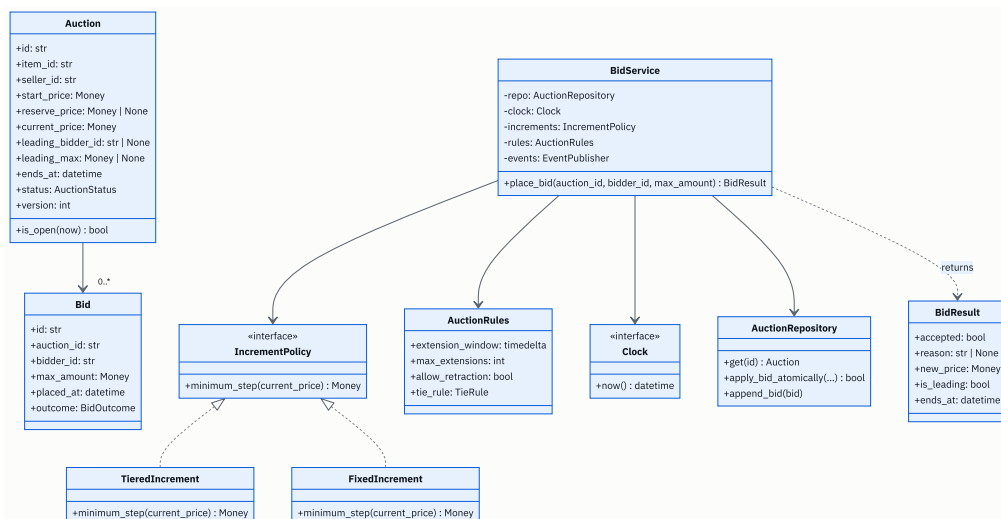
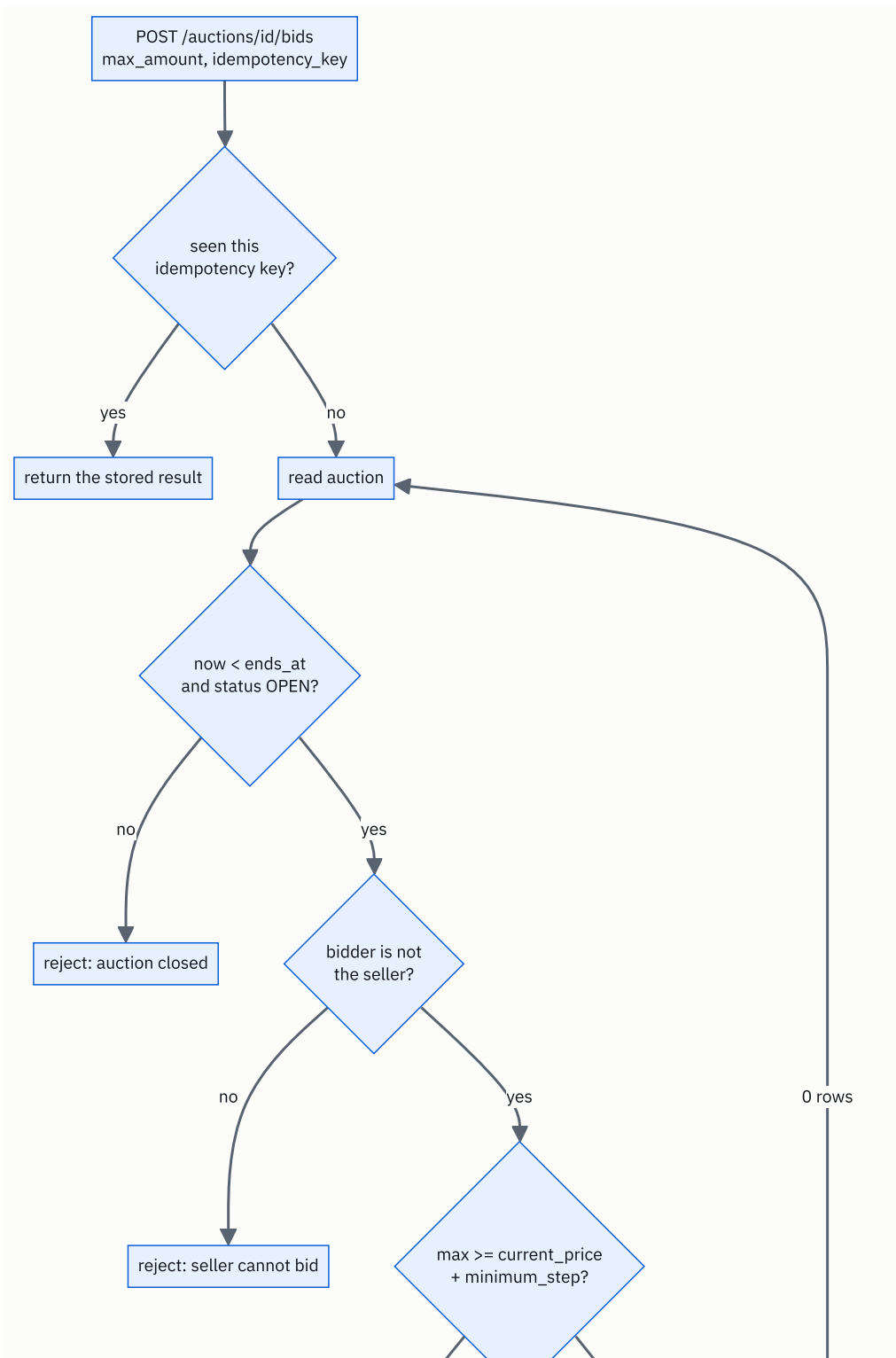


FIGURE 95.1

What to notice: `Auction` has no `place_bid` method. Placing a bid needs the clock, the increment policy, the rules and an atomic write — none of which an `Auction` object should own. The entity holds state and answers questions about itself; the service performs the transaction. Putting `place_bid` on `Auction` is the most common structural mistake in this prompt.

Also notice `Clock` is an interface, for the same reason `Dice` was on [day 094](#): you cannot test anti-sniping, closing, or “a bid one millisecond late” against the real system clock.

One bid, end to end:



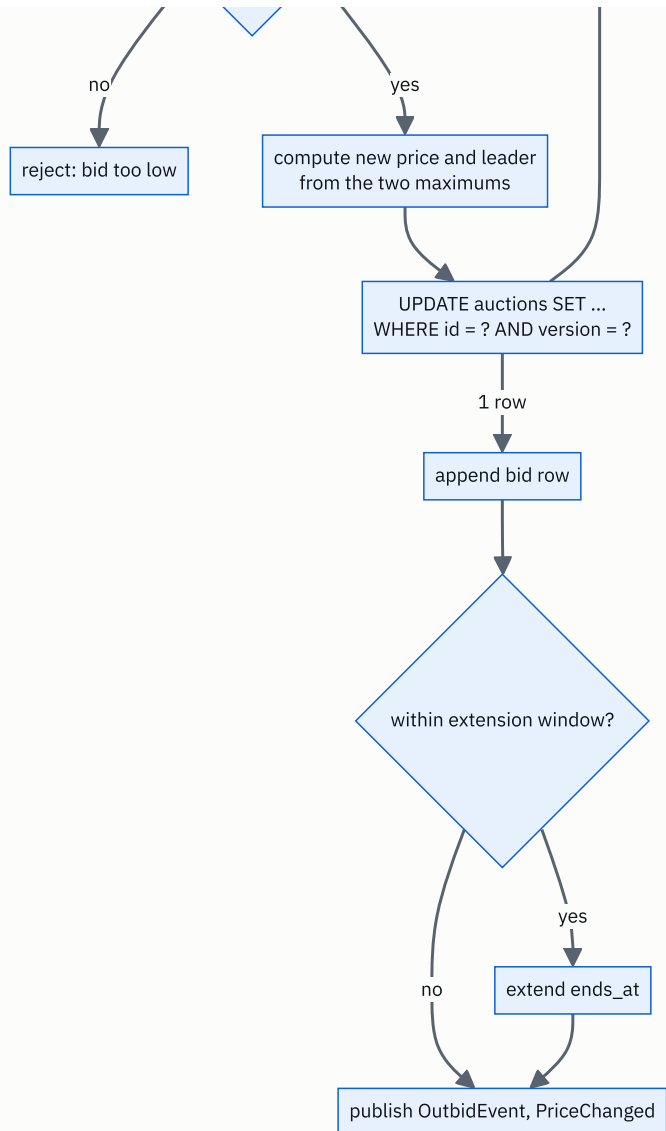


FIGURE 95.2

What to notice: **the retry arrow goes back to the read.** The conditional update either wins or it does not; if it does not, someone else changed the auction, so you re-read and recompute. That loop is the whole answer to “two bids in the same millisecond”.

The two-bids-at-once race, drawn:

Auction at ₹800, version 7. Two requests arrive together.

A: read → price 800, version 7	B: read → price 800, version 7
A: compute new price 850	B: compute new price 850
A: UPDATE ... WHERE version = 7 → 1 row, version becomes 8	B: UPDATE ... WHERE version = 7 → 0 rows
	B: re-read → price 850, version 8
	B: recompute → 900, or reject as too low
	B: UPDATE ... WHERE version = 8 → 1 row

Exactly one of them wins each round. Nobody waits on a lock.

The loser is told the truth: "the price moved, you are now at 900" or "your max is too low".

Compare with the version that is wrong, and which most first drafts contain:

A: read 800	B: read 800	
A: write 850	B: write 850	← B overwrites A
result: price 850, ONE of the two bids silently vanished, and its bidder was told they were leading.		

That is a lost update, and it is money. Read-modify-write without a condition is the bug this whole prompt exists to catch.

5 How it actually works

Move 1 — clarify

- “Which kind of auction?” — English ascending, single item, fixed end time. Not Dutch, not sealed-bid, not multi-unit. Say this; it removes half the ambiguity in one sentence.
- “Proxy bidding, or literal bids?” — Proxy. Bidders submit a maximum and the system bids on their behalf. This is what eBay does and it changes the data model.
- “Is there a reserve price?” — Yes, hidden. The auction can end with a highest bid that does not meet it, and then nobody wins.
- “What happens to a bid one millisecond after the end time?” — Rejected. And I will make the end time the authority rather than a status flag, so a late background job cannot accidentally accept it.
- “Do we handle payment?” — Out of scope. I will end at **SOLD** with a winner and publish an event.

Move 2 — the nouns

CLASS	RESPONSIBLE FOR
Auction	The state, and answering <code>is_open(now)</code> . No bidding logic.
Bid	One submitted maximum, its bidder, its time, and what happened to it. Immutable.
BidService	The transaction: validate, compute, write atomically, publish.
IncrementPolicy (interface)	The minimum step at a given price. Tiered on real sites.
AuctionRules	Extension window, extension cap, tie rule, retraction policy. Data.
Clock (interface)	<code>now()</code> . Replaceable, or none of the timing is testable.
AuctionRepository	Reads, and the one conditional write .

`Money` is a class, not a float. Say this in one sentence and move on: `0.1 + 0.2 ≠ 0.3` , so amounts are integer minor units — paise, not rupees — and the type carries the currency.

Move 3 — the interesting part

There are three, and an interviewer will push on all three. Handle them in this order.

One: the concurrent bid. The answer is a single conditional `UPDATE` with a retry loop.

```
UPDATE auctions
  SET current_price      = :new_price,
      leading_bidder_id = :new_leader,
      leading_max       = :new_max,
      ends_at          = :maybe_extended,
      version           = version + 1
WHERE id = :auction_id
  AND version = :read_version
  AND status = 'OPEN';
```

If it updates one row, you won. If it updates zero rows, someone else moved the auction: re-read, recompute, try again. **This is optimistic concurrency, and it is right here for a specific reason** — contention is low for almost every auction and extreme for a few in the last thirty seconds, and optimistic control costs nothing in the common case.

The alternative — `SELECT ... FOR UPDATE` , a pessimistic row lock — is also correct and is what you should mention as the fallback. It serialises bidders on a hot row and holds a lock for the duration of your transaction, so a slow bit of logic inside the transac-

tion becomes everyone's problem. **Never a distributed lock in Redis:** the database row is already the single point of truth, and adding a second one creates the possibility of them disagreeing.

Two: proxy resolution. Given the current leader's hidden maximum and the new bidder's maximum, compute the new price and leader. It is four cases and it is worth writing out, because getting it right is the difference between a working auction and a fake one.

Three: the close. Covered above — the end time is the authority, and the job only does side effects.

Move 4 — the class diagram

Drawn above. Present it by walking a bid, and lead with `BidService.place_bid`, because that is the only interesting method in the system.

Move 5 – the code

```
from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass
from datetime import datetime, timedelta
from enum import Enum, auto


@dataclass(frozen=True, order=True)
class Money:
    """Integer minor units. Never a float: 0.1 + 0.2 ≠ 0.3, and this is money."""

    paise: int
    currency: str = "INR"

    def __add__(self, other: "Money") → "Money":
        assert self.currency == other.currency
        return Money(self.paise + other.paise, self.currency)


class AuctionStatus(Enum):
    SCHEDULED = auto()
    OPEN = auto()
    ENDED = auto()
    SOLD = auto()
    UNSOLD = auto()
    CANCELLED = auto()


@dataclass
class Auction:
    id: str
    seller_id: str
    start_price: Money
    reserve_price: Money | None
    current_price: Money
    leading_bidder_id: str | None
    leading_max: Money | None          # NEVER leaves the service layer
    ends_at: datetime
    status: AuctionStatus
    version: int

    def is_open(self, now: datetime) → bool:
        """The end TIME is the authority, not the status field.

        A background job that runs late cannot make this return True.
        """
        return self.status is AuctionStatus.OPEN and now < self.ends_at
```

`is_open` taking `now` as an argument rather than calling the clock itself is deliberate: the entity stays free of dependencies and the caller decides which clock.

```

class IncrementPolicy(ABC):
    @abstractmethod
    def minimum_step(self, current: Money) → Money: ...

class TieredIncrement(IncrementPolicy):
    """What real sites use: the step grows with the price, so a ₹5 lakh item
    does not go up in ₹50 steps and take four thousand bids to move."""

    TIERS = [
        (Money(100_00), Money(1_00)),          # under ₹100      → ₹1
        (Money(1_000_00), Money(25_00)),       # under ₹1,000    → ₹25
        (Money(10_000_00), Money(100_00)),      # under ₹10,000   → ₹100
    ]
    TOP = Money(1_000_00)                       # above that      → ₹1,000

    def minimum_step(self, current: Money) → Money:
        for ceiling, step in self.TIERS:
            if current < ceiling:
                return step
        return self.TOP

```

The proxy resolution, which is the heart of it:

```

@dataclass(frozen=True)
class Resolution:
    price: Money
    leader_id: str
    leader_max: Money

def resolve(
    auction: Auction,
    bidder_id: str,
    new_max: Money,
    step: Money,
) → Resolution:
    """Given the leader's hidden maximum and a new maximum, work out the
    visible price and who is leading.

    The rule: the price rises to just enough to beat the LOSER, never to the
    winner's maximum. That is why Ibrahim's 1200 stayed hidden.
    """
    old_max = auction.leading_max
    old_leader = auction.leading_bidder_id

    if old_leader is None:                # first bid on the auction
        return Resolution(auction.start_price, bidder_id, new_max)

    if bidder_id == old_leader:           # raising your own maximum
        return Resolution(auction.current_price, bidder_id, max(old_max, new_max))

    if new_max > old_max:                 # the challenger wins
        price = min(old_max + step, new_max)  # beat the loser, capped at your max
        return Resolution(price, bidder_id, new_max)

    if new_max == old_max:               # tie: the EARLIER bid holds
        return Resolution(new_max, old_leader, old_max)

    price = min(new_max + step, old_max)  # challenger loses, price still moves
    return Resolution(price, old_leader, old_max)

```


Note what `place_bid` returns when the bidder is outbid instantly: `accepted=True`, `is_leading=False`. That is a real outcome and the UI has to show it — “your bid was accepted but you are not the highest bidder” — and a design that has no way to express it will produce a screen that lies.

What real systems do

- **eBay** has used proxy bidding since 1995 and it is the reason the price you see is rarely a round number: it is the loser’s maximum plus one increment. eBay’s increments are tiered exactly as above.
- Bids are written to a **strongly consistent** store — a relational database, or DynamoDB with conditional writes and strongly consistent reads. **This is not a place for eventual consistency.**
- The auction row is the single writer point. Hot auctions in their final minute are handled by putting bids for one auction onto **one partition of a queue keyed by auction id**, so they are serialised by arrival rather than by contention — the same one-writer-per-entity idea as [day 094](#).
- Price updates are pushed to watchers over **WebSockets**; the read path for “current price” can be cached for a second or two, **but the write path must never read from that cache.**
- Closing is a **delayed queue message** (SQS delay, or a scheduled sweeper over `ends_at < now AND status = 'OPEN'`), and the sweeper is the safety net for messages that were lost.

6 The numbers

Volume

active auctions	10,000,000
average bids per auction	12
average auction length	7 days

bids per auction-week	12
bids per day	$10M \times 12 / 7 \approx 17,100,000$
bids per second	$17.1M / 86,400 \approx 198$

Two hundred bids a second is nothing. That is the point — and it is why candidates who spend the interview on horizontal scaling have missed the question.

The real number is the distribution:

share of bids in the final 60 seconds of an auction	~35%
auctions ending in the same busy minute (evening)	~2,000
bids in that minute	$17.1\text{M/day} \times 35\% \div (\text{auctions/day})$... concentrated:
a popular auction can take	1,000-5,000 bids in the last 30 seconds
= up to	~170 bids per second ON ONE ROW

One hundred and seventy writes a second to a single row, while ten million other auctions are idle. That is the actual engineering problem, and it is a concurrency problem, not a throughput one.

Storage

bid row: id, auction_id, bidder_id, max_amount, placed_at, outcome	~ 200 bytes
bids per day	17,100,000
per day	$17.1\text{M} \times 200\text{ B}$
per year	= 3.4 GB
auction rows	$10\text{M} \times 400\text{ B}$
	= 1.25 TB
	= 4 GB

Bids are **append-only and never updated**, which makes them ideal for partitioning by auction id and archiving by month. Keep the last ninety days queryable and move the rest to object storage: about 310 GB hot, the rest at roughly ₹1.7 per GB-month.

Reads against writes

page views per auction over its life	~400
bids per auction	12
read:write ratio	~33:1

Thirty-three reads per write, and every viewer of a live auction wants the current price. **Cache the displayed price with a one-to-two second TTL, and push updates over WebSockets** — but the bid path reads the row directly. Say that distinction explicitly: *the display may be one second stale; the decision must not be stale at all.*

Retries

optimistic write success rate, idle auction	~100%
same, at 170 bids/second on one row:	
window between read and conditional write	~2 ms
probability of a collision	$170/\text{s} \times 2\text{ ms}$
expected attempts	$1 / (1 - 0.34)$
P(more than 5 attempts)	0.34^5
	≈ 34%
	≈ 1.5
	≈ 0.5%

About one and a half attempts on average even on the hottest auction, and half a percent of bidders would exhaust five retries — which is why the queue-per-auction option exists for the top few hundred listings. Being able to produce that calculation is what makes “I would use optimistic concurrency” into an argument rather than a preference.

Closing

auctions ending per day	10M / 7	≈ 1,430,000
per second, average		≈ 16
evening peak (3×)		≈ 50

Fifty finalisations a second — trivial work, but it must not be late in a way that lets bids through. Hence: the end time is the authority and the sweeper only does side effects.

Concurrency, stated plainly

1. **Two bids on the same auction at once.** Conditional update on `version`; loser re-reads and recomputes. Never a read-then-write.
2. **The same bid submitted twice** because the phone retried. An **idempotency key** per bid, with a unique constraint, so the second submission returns the first result instead of bidding again.
3. **A bid racing the deadline.** The check is `now < ends_at` inside the same transaction as the write, so there is no window between checking and accepting.

7 The trade-offs

Optimistic concurrency, or a row lock?

Optimistic costs nothing when there is no contention, which is 99.99 percent of auctions, and degrades gracefully — 1.5 attempts on the hottest row. It can starve a very unlucky bidder, which is why there is a retry cap and an honest error.

A **pessimistic** `SELECT ... FOR UPDATE` never retries and is easier to reason about, and it holds a lock for however long your transaction takes — so one slow call inside the transaction becomes every bidder's wait. **I would not use optimistic concurrency if the write path had to do several dependent updates**; at that point the lock's simplicity wins.

Neither is a distributed lock. The row is already the single source of truth; a Redis lock adds a second one that can disagree with it, and now you have two problems.

Store the maximum, or the literal bid?

Proxy bidding is more code and it is what users expect. Literal bidding — the price is what you typed — is simpler and produces a much worse auction: it rewards whoever is refreshing the page at 3 a.m. **Take proxy bidding**, and accept the hard constraint it brings: the maximum must never leak, anywhere.

Anti-snipe extension, or a hard close?

Extension makes the auction fairer and makes the end time unpredictable, which breaks “this ends at 8 p.m.” in listings, calendars and notifications. A hard close is predictable and rewards sniping.

Take the extension with a cap — say, at most ten extensions — and show the rule in the listing. **I would not extend** for a fixed-price “buy it now” flow or a scheduled charity event where the end time is the product.

Push the price, or let clients poll?

At thirty-three reads per write, polling every second wastes most of its requests. Web-Sockets cost a persistent connection per viewer — and a popular auction has thousands of viewers in its final minute. **Push for live auctions in their last hour, poll otherwise**, which is a real hybrid that real sites use.

Where this design breaks

- **Multi-unit auctions** — ten identical items, top ten bidders win — break `leading_bidder_id` and the whole proxy resolution, because “the price” becomes a clearing price and there are `k` winners.
- **Sealed-bid or Dutch auctions** are different rules, not extensions: no visible price, or a price that falls. The `IncrementPolicy` interface does not help; you would want a `AuctionType` strategy above the service.
- **Payment failure after SOLD**. The winner’s card declines. Do you offer it to the second-highest bidder at their maximum? That is a business rule with legal weight, and it is why `ENDED` and `SOLD` are separate states.
- **Shill bidding** — the seller bidding through a friend to push the price up — is not solved by any of this. It is a fraud-detection problem: graph analysis on bidder-seller pairs who repeatedly meet, plus the rule that a seller cannot bid on their own listing. Mention it; it shows you know the system has adversaries.

8 In the interview

How it gets asked

- The prompt: “Design an online auction site.”
- The one they actually care about: “Two bids arrive in the same millisecond. What happens?”
- The data-model probe: “What exactly do you store when I bid ₹1,200?”
- The timing probe: “A bid arrives one millisecond after the end time.” / “Your closing job is thirty seconds late.”

- The fairness probe: “*Someone bids in the last second every time. Is that a problem?*”
- The scale probe: “*One auction is taking a thousand bids in the final thirty seconds.*”

What to say out loud, in the first ninety seconds

1. **Name the kind of auction.** “English ascending, single item, fixed end time, proxy bidding, hidden reserve. Payment is out of scope; I will end at a winner and publish an event.”
2. **Say what is stored, because it is surprising.** “A bid is a **maximum**, not a price. The visible price is the *loser’s* maximum plus one increment, so a winner usually pays far less than they offered. That means maximums must never appear in any response.”
3. **Go straight to the race.** “The interesting part is two bids at once. I do a conditional update: set the new price and leader **where the version is still the one I read**. One row updated means I won; zero rows means somebody moved it, so I re-read and recompute. Never read-then-write, or one bid silently overwrites the other and its bidder is told they are leading.”
4. **Make the deadline the authority.** “Whether bidding is open is `now < ends_at`, evaluated inside the same transaction as the write — not a status field that a job has to remember to flip. Then a job running late cannot accept a bid it should not have.”
5. **Mention anti-sniping as a rule, not a feature.** “A bid inside the last two minutes extends the end by two minutes, capped. It is policy, so it lives in a rules object.”
6. **Give the numbers that reframe the problem.** “Ten million active auctions at twelve bids each is about two hundred bids a second globally — nothing. The problem is that one popular auction can take a hundred and seventy writes a second **to a single row** in its last thirty seconds. This is a concurrency problem, not a throughput problem.”

The follow-ups

“**Two bids arrive in the same millisecond. What happens?**” “Exactly one is applied first and the other is told the truth. Concretely: both requests read the auction at price ₹800, version 7. Both compute a new price. Both issue an `UPDATE ... WHERE id = ? AND version = 7`. The database serialises them, so one updates one row and moves the version to 8, and the other updates **zero** rows. The loser does not fail — it re-reads at version 8 and recomputes, and either it now wins at a higher price or its maximum is too low and it is rejected with a clear reason. Nobody waits on a lock. The version I would never write is read, decide, write — that is a lost update, and it means one bidder’s money quietly disappeared while they were told they were winning.”

“**What do you store when I bid ₹1,200?**” “Your **maximum**, ₹1,200 — not the price. The price becomes just enough to beat the current leader: their maximum plus one increment, capped at yours. So if the leader’s hidden maximum was ₹900 and the increment is ₹50, the price becomes ₹950 and you are leading with ₹1,200 still hidden. Two con-

sequences. A bid can be accepted and immediately losing, if your maximum is under the leader's — and the price still moves, because you have pushed them further up their own ceiling. And the maximums must never leak: not in the API response, not in an event, not in an admin view the front end calls. Leaking a maximum lets someone bid you up to exactly your ceiling, which is the worst bug this system can have.”

“**A bid arrives one millisecond after the end time.**” “Rejected, and the reason it is reliably rejected is that **the end time is the authority, not a status column**. The open check is `now < ends_at` evaluated in the same transaction as the write, so there is no gap between checking and accepting. If I had relied on a background job to flip the status to `ENDED`, then a job that ran three seconds late would accept three seconds of bids that should not exist — and that is a real incident, not a theoretical one. The job still exists, but it only does side effects: decide the winner, check the reserve, notify, release the item.”

“**Someone bids in the last second every time.**” “That is sniping, and it is rational rather than cheating — but it makes auctions worse, because honest bidders learn there is no point participating and the seller gets less. The fix is what a live auctioneer does: any bid inside the final window extends the end by that window, so there is always time to respond. I would cap the number of extensions so an auction cannot run for ever, and I would put both the window and the cap in a rules object rather than in the bidding code, because they are policy and they differ by category. The cost is a real one: the end time stops being predictable, which breaks ‘ends at 8 p.m.’ in listings and reminders.”

“**One auction is taking a thousand bids in its final thirty seconds.**” “That is about 170 writes a second to one row. With optimistic concurrency the arithmetic is: if the gap between read and conditional write is around two milliseconds, the collision probability is roughly thirty-four percent, so the expected number of attempts is about 1.5 and fewer than one in two hundred bidders would exhaust five retries. That is acceptable. If I wanted it bounded rather than probabilistic, I would route bids for a single auction through **one partition of a queue keyed by auction id**, so they are serialised by arrival order with no contention at all — one writer per auction, which also gives a clean, fair ordering. I would only do that for the top few hundred hot listings, because it adds latency to every bid it touches.”

“**Could you cache the current price?**” “For **display**, yes — a one-to-two second TTL, plus a WebSocket push on every change, and at thirty-three reads per write that saves most of the read load. For the **bid path**, absolutely not. The whole design rests on comparing against the true current state inside the transaction that writes. A cached price would let me accept a ₹500 bid on an item already at ₹600 and tell that bidder they were winning. That is the sentence I would want on the record: **the display may be stale; the decision may not be.**”

A model answer

Asked: *design an auction site. Two bids arrive in the same millisecond.*

“Let me set the scope in one line: English ascending auction, single item, fixed end time, proxy bidding, hidden reserve. Payment is out of scope — I end at a winner and publish an event.

The state is small: item, seller, current price, leading bidder, the leader's hidden maximum, end time, status, version. All the difficulty is in changing three of those fields at once.

First, what a bid actually is, because it surprises people. When you bid ₹1,200 I do **not** set the price to ₹1,200. I record ₹1,200 as your **maximum** and set the visible price to just enough to beat the current leader — their maximum plus one increment, capped at yours. So the displayed price is derived from the *loser's* maximum, not the winner's. Two consequences: a bid can be accepted and instantly losing, and it still moves the price because it pushed the leader further up their own ceiling. And maximums must never leave the service layer — leaking one lets somebody walk a bidder up to exactly their ceiling.

Now the question. Two bids in the same millisecond. Both requests read the auction at, say, ₹800 with version 7. Both compute a new price. Then both issue a conditional update: set the price, leader and maximum **where the id matches and the version is still 7**. The database serialises those two statements, so one of them updates a row and bumps the version to 8, and the other updates **zero** rows. The loser is not an error — it re-reads at version 8, recomputes against the new price, and either wins at a higher number or is rejected with ‘the price moved, your maximum is too low’. Nobody holds a lock and nobody waits.

The version I would never write is read, decide, then write unconditionally. That is a lost update: the second write silently overwrites the first, one bid disappears, and the bidder who placed it was told they were leading. With money, that is not a bug you can log and move on from.

On the deadline: **the end time is the authority, not a status column**. The check is `now < ends_at` inside the same transaction as the write, so a bid one millisecond late is rejected even if a background job has not got round to marking the auction ended. The job still runs, but only for side effects — winner, reserve check, notifications. If it is thirty seconds late, nothing incorrect has been accepted in the meantime.

I would add anti-sniping: a bid in the final two minutes extends the end by two minutes, capped at ten extensions. That is what a live auctioneer does when someone shouts on ‘going, going’ — he starts the count again, because otherwise everyone learns to shout last and nobody else gets a turn. It is policy, so it lives in a rules object with the increment tiers, not in the bidding code.

Finally the numbers, because they reframe the problem. Ten million active auctions at twelve bids each over a week is about two hundred bids a second globally, which is nothing. But roughly a third of all bids land in the final minute, and a single popular auction can take a thousand bids in thirty seconds — about a hundred and seventy writes a second **to one row**. So this is a contention problem, not a throughput problem. Optimistic concurrency handles it: with a two-millisecond window the expected number of attempts is around 1.5. If I wanted it bounded rather than probabilistic, I would route that auction's bids through a single queue partition keyed by auction id — one writer per auction — and I would do that only for the hottest few hundred listings, because it adds latency to every bid."

9 Recall card

- **A bid is a MAXIMUM, not a price.** The visible price is the **loser's** maximum plus one increment, capped at the winner's — so ₹1,200 usually wins at ₹950. Consequences: a bid can be **accepted and instantly losing**, a losing bid **still raises the price**, and **maximums must never leak** anywhere. Ties go to the **earlier** bid.
- **Two bids at once: a conditional write, never read-then-write.** `UPDATE ... WHERE id = ? AND version = ?` — one row means you won, **zero rows means re-read and recompute**. Read-modify-write is a **lost update**: one bid vanishes and its bidder is told they are leading. Not a Redis lock — the row is already the single source of truth.
- **The end TIME is the authority, not a status column.** `now < ends_at` is checked inside the same transaction as the write, so a late closing job cannot accept bids it should not. The job does only side effects, and `ENDED` and `SOLD` are **separate states** because payment can fail.
- **Anti-snipe: a bid in the final window extends the end by that window, capped.** Policy, so it lives in a rules object with the tiered increments. The price the design pays is that the end time stops being predictable.
- **The numbers reframe it: 10M auctions × 12 bids ÷ 7 days ≈ 200 bids/second globally — nothing.** The problem is **~170 writes/second to ONE row** in a hot auction's last 30 seconds. At a 2 ms window that is a **~34% collision rate**, **~1.5 attempts expected**, 0.5% exhausting five retries. For the hottest listings, **one queue partition per auction id = one writer, zero contention**. Cache the price for **display** (1–2 s TTL, ~33:1 read:write) — **never for the decision**.

DSA topic: N-Queens and constraint grids **System design topic:** Design an online auction

Code these, in this order

One rule for the whole set: **reduce out loud before writing**. Say how many candidates the obvious framing has, then say how many yours has. On N-Queens that is four billion against forty thousand, and the reduction is worth more than the code.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	N-Queens II	LeetCode 52 (Hard)	The three constraint sets, and counting without building boards.
2	N-Queens	LeetCode 51 (Hard)	The same tree, plus rendering only at a solution.
3	Sudoku Solver	LeetCode 37 (Hard)	Row, column and box sets, and the $(r//3)*3 + c//3$ index.
4	Valid Sudoku	LeetCode 36 (Medium)	The same three sets with no search at all — do it last, as the check.

On problem 1, count the nodes both ways

Write it once with the conflict check before the recursive call and once with it at the leaf. Record the node counts for $n = 6$ and $n = 8$. Then say the ratio out loud.

On problem 2, time the rendering

Run $n = 12$ with the board strings built at every call, and again with them built only at a solution. Record both times and say what fraction of the work the rendering was.

On problem 3, pre-compute the blanks

Write it scanning for the next blank from $(0,0)$ every call, then with a pre-computed list of blanks indexed by depth. Record the call counts and the times for a hard puzzle.

On problem 4, notice what is missing

There is no recursion here at all. Say in one sentence what problem 4 shares with problems 1 to 3, and what it does not have.

The reduction drill

1. State the four framings and their sizes for $n = 8$, in order.
2. Say which observation takes you from each one to the next.
3. Say what representation the “one per row” framing implies.
4. Say what conflict becomes unrepresentable, and why that is better than checking for it.

The diagonal drill

1. Say what is constant along a `\` diagonal and along a `/` diagonal.
2. Draw both value grids for a 4×4 board.
3. Give the range of each expression for general `n`.
4. Say what offset a list-based version needs, and which of the two needs it.
5. Run the version without the offset and describe what you get — and why there is no error.

The pruning drill

1. Say the one line that separates this from brute force.
2. Say exactly where it goes.
3. Record the node counts for $n = 6$ and $n = 8$, both ways.
4. Say why the number of *solutions* is unchanged.
5. Say what the undo has to do with whether the check is affordable.

The undo drill

1. Count the changes before the recursive call, and the restores after.
2. Delete the anti-diagonal restore and run `total_n_queens(8)`. Record the answer.
3. Say why the answer is what it is, and why nothing was raised.
4. Say what makes four pieces of state harder than one.

The edge-case drill

1. Give the solution counts for $n = 1$ through $n = 10$ from memory or by running it.
2. Say which two values of `n` have none.
3. Say what breaks in code that assumes a non-empty result.
4. Say whether there is a formula for the count.

The bitmask drill

1. Write the bitmask version of the counter.
2. Say what `available & -available` does.

3. Say why `cols` does not shift but `diag` and `anti` do, and in which direction each.
4. Time it against the set version at $n = 12$ and record the ratio.
5. Say when you would write it in an interview and when you would only mention it.

The Sudoku drill

1. Name the three constraint sets.
2. Write the box index expression and check it for `(4, 7)`.
3. Say what depth means here, versus in N-Queens.
4. Say why the search returns `True` up the stack instead of exploring everything.
5. Describe the most-constrained-cell heuristic and say when it pays for itself.

The cost drill

1. State the upper bound and say honestly why it is loose.
 2. Give the measured node counts and running times for $n = 8, 12$ and 14 .
 3. State the extra space and say what the representation saved you.
 4. Compute the output size for $n = 12$ and say why the counting version is faster.
 5. Say what the symmetry trick saves, and what it does not change.
-

The proxy drill

1. Say what is stored when someone bids ₹1,200.
2. Work the Ibrahim example through: three bids, and the price after each.
3. Name the four cases in `resolve` and say which one people forget.
4. Say what a losing bid does to the price, and why.
5. Say where a leaked maximum could appear, and what it would let someone do.

The race drill

1. Write the conditional `UPDATE` from memory.
2. Trace two simultaneous bids through it, both branches.
3. Write the version that is wrong, and name the failure.
4. Say what the loser does after a zero-row update.
5. Say why you would not use a distributed lock here.

The deadline drill

1. Say what the authority for “is bidding open” is, and what it is not.
2. Say what goes wrong if a status column is the authority and the job runs late.
3. Say why `ENDED` and `SOLD` are separate states.

4. Compare scheduled close against lazy close on the “job is late” row.

The anti-snipe drill

1. Write the extension rule in two lines.
2. Say what a live auctioneer does and why.
3. Name the two policy decisions it needs.
4. Say what the design gives up by extending, in terms of the product.

The numbers drill

Compute each, showing the multiplication:

1. Bids per day and per second at 10M active auctions, 12 bids each, over 7 days.
2. Writes per second on a single row for a hot auction with 1,000 bids in 30 seconds.
3. The collision probability and expected attempts at a 2 ms window.
4. Bid storage per day and per year at 200 bytes a row.
5. The read:write ratio, and what it says about caching.

The failure drill

For each, say what happens and what you would add:

1. Two bids read the same version and both write unconditionally.
2. A bidder’s phone submits the same bid twice.
3. The closing job runs thirty seconds late.
4. The current price is served from a two-second cache on the bid path.
5. A seller’s friend bids the item up and then withdraws.
6. The winner’s payment declines after `SOLD`.

Two of the six are not concurrency problems. Name them and say what kind of problem they are.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Place n queens on an n by n board.* The reduction first with all four numbers, the list-of-columns representation, the two diagonal expressions with their justification, the check placed before the descent with the 2,057 against 19,173,961, four changes and four restores, and an honest complexity that admits the bound is loose.

2. *Design an auction site. Two bids arrive in the same millisecond.* What is actually stored, the conditional update with the version and both branches traced, the lost update named as the thing you are avoiding, the end time as the authority, anti-sniping as policy, and the numbers that make it a contention problem rather than a throughput one.
 3. *Now solve a Sudoku board.* The same three-set technique, the box index expression, the pre-computed blanks, and the one thing that differs — returning as soon as a solution is found.
-

Before you move on

- [] I can give the four framings of N-Queens and their sizes for $n = 8$.
- [] I can say why there is exactly one queen per row without being told.
- [] I can justify both diagonal expressions in one sentence each.
- [] I know which one needs an offset in a list version, and what happens without it.
- [] I recorded the node counts for $n = 6$ and $n = 8$ both ways.
- [] I can count four changes and four restores before running.
- [] I deleted one restore and can explain the zero.
- [] I know that $n = 2$ and $n = 3$ have no solutions.
- [] I can explain the bitmask shifts, and which mask does not shift.
- [] I can write the Sudoku box index and check it on a cell.
- [] I can state the complexity honestly, including that the bound is loose.
- [] I can say what is stored when someone bids ₹1,200, and what is displayed.
- [] I can name the four proxy cases and the one people forget.
- [] I can write the conditional `UPDATE` and trace two simultaneous bids.
- [] I can name the lost update and say why it is worse than an error.
- [] I can say what the authority for “bidding is open” must be, and why.
- [] I can produce the 200 bids/second and 170 writes/second-on-one-row numbers.
- [] I answered all three questions above out loud.

Sudoku, word search, and grid backtracking

DATA STRUCTURES AND ALGORITHMS

Sudoku, word search, and grid backtracking

You can search a two-dimensional space with backtracking and mark visited cells safely.

SYSTEM DESIGN

Low-level design revision and full mock

You can take an unseen LLD prompt to a defended class diagram in forty minutes.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Solve the sudoku board. Now find the word in the grid.*
- *Design a chess game. Forty minutes. Begin.*

Sudoku, word search, and grid backtracking

1 What this is, and why they ask it

A **grid** is a two-dimensional array — rows and columns of cells. Grid backtracking is the same choose-recurse-un-choose pattern where the choices are *directions to move* or *values to write*, and the state to undo lives in the grid itself.

Three sentences. In a grid problem the position is two numbers instead of one, so every recursive call starts with a **bounds check**. The “used” marker from [permutations](#) becomes a **visited grid**, and the crucial question is whether you unmark it on the way out. And a **path** problem unmarks, while a **region** problem does not — that single distinction decides half of all grid problems, and getting it backwards produces wrong answers with no error.

They ask it because it is the first time the state you are undoing is the input itself. Word Search asks you to mark cells you have walked through and then release them, and the neat trick — overwrite the cell with a character that cannot match, then put the original back — is exactly the un-choose step wearing a different coat. Sudoku is the same shape with values instead of directions. Together they are the two grid problems that appear most often, and after today the rest of the family — rat in a maze, all paths, unique paths with obstacles — is the same eight lines.

2 The story

Anand was sent to the Tuesday market to find the woman who sold the good curd, and he had never bought anything in his life.

His mother did not know the woman’s name and could not tell him a stall number, because there were no numbers. What she told him instead was a sequence. Go in, and find the man with the drumsticks. From him, the very next stall is the one with the curry leaves, piled up. Right beside that one there is a woman selling flowers. And next to the flower lady is the curd.

Four stalls in a row, she said, each one right next to the last. It does not matter which way — they might go across or they might go up the lane — but each one has to be immediately beside the one before.

The market was laid out in lanes, six or seven across and about the same deep, and there were three ways in.

He went in at the first entrance. There was a man with drumsticks almost immediately, so he stood in front of him and looked at the four stalls beside that one — left, right, in front, behind. None of them had curry leaves. He backed out.

He found another drumstick seller two lanes over. Beside him, curry leaves. He was pleased with himself. But beside the curry leaves there were no flowers anywhere, only a man with a weighing scale and somebody selling combs. So he went back to the drumstick man and started again from there, and there was nothing else next to him either, so he backed all the way out.

The third time it worked. Drumsticks, then curry leaves beside them, then flowers beside those, then the curd woman.

There was one rule he had worked out for himself by the second attempt, without being told. Within one attempt, he was not allowed to count the same stall twice. He had gone round in a small circle once — drumsticks, curry leaves, and then back to the drumstick man again, and briefly thought he was getting somewhere. But a stall he had rejected on one attempt was perfectly fine on the next one. It was only the current attempt that had to stay clean.

He got home at half past eleven with the curd and a very long account of how he had found it.

3 The idea in plain English

Anand has run Word Search. Every part of the technique is in that morning.

- The market lanes are the **grid**. Each stall is a **cell**.
- “Each one immediately beside the one before” is the **four-direction move**: up, down, left, right.
- Trying every entrance is the outer loop: **start a search from every cell**.
- “Not allowed to count the same stall twice **within one attempt**” is the **visited marker**.
- “A stall rejected on one attempt is fine on the next” is the **un-mark**, and it is the whole difference between this and a region problem.

Position is two numbers, so bounds come first

```
for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):      # up, down, left, right
    nr, nc = row + dr, col + dc
    if not (0 ≤ nr < rows and 0 ≤ nc < cols):
        continue                                       # off the grid
```

Write the bounds check as the first thing inside the loop, every time. In Python, forgetting it does not raise — `grid[-1][2]` is a perfectly valid read of the *last* row, so your search silently wraps around the board and finds words that are not there.

The four offsets as a tuple of tuples is the standard idiom. For eight-direction problems the list has eight entries and nothing else changes; write it as data, never as four copies of the same block.

The visited marker, and the trick

The obvious version keeps a second grid:

```
visited = [[False] * cols for _ in range(rows)]
```

Correct, and it costs `rows × cols` booleans. The trick, which interviewers like to see, is to mark the input itself:

```
original = grid[row][col]
grid[row][col] = "#"          # a character no word can contain
...recurse...
grid[row][col] = original     # put it back
```

Zero extra space, and the restore is the un-choose. Say the trade-off out loud when you use it: it mutates the caller's grid, so if the function raises part-way through, the grid is left with `#` in it. For an interview that is acceptable and worth mentioning; for a library function it is not.

The distinction that decides everything: path or region

This is the sentence to remember from today.

	PATH PROBLEMS	REGION PROBLEMS
Question	<i>is there a route that...</i>	<i>how many connected groups / how big is this one</i>
Examples	Word Search, rat in a maze, all paths	Number of Islands, flood fill, largest region
Marker	set on the way in, cleared on the way out	set on the way in, never cleared
Why	a cell used by one route must be free for another	a cell belongs to exactly one region, for ever
Cost	exponential — cells are revisited across routes	linear — each cell is visited once

Getting this backwards is the classic grid bug. Leave the mark in place on Word Search and you will report that a word is absent when it is present, because the first failed attempt has permanently consumed cells. Clear the mark on Number of Islands and you will count the same island many times and recurse for ever between two adjacent cells:

```
RecursionError: maximum recursion depth exceeded
```

Say which kind you are solving before you write anything. *“This is a path problem, so the visited marker is undone on the way out”* takes four seconds and prevents the whole class of mistake.

Word Search, in full

```
def search(row, col, k):
    if k == len(word):
        return True                    # matched every character
    if not in_bounds(row, col) or grid[row][col] != word[k]:
        return False                  # off the grid, or the wrong letter
    original = grid[row][col]
    grid[row][col] = "#"              # choose
    found = any(search(row + dr, col + dc, k + 1) for dr, dc in DIRECTIONS)
    grid[row][col] = original         # un-choose
    return found
```

Eight lines. Notice that the bounds check and the character check are combined into one guard at the *top* of the function rather than being done before each recursive call. Both styles work; the guard at the top is shorter and is the one to write under pressure, because there is exactly one place to get it right.

Then the outer loop:

```
return any(search(r, c, 0) for r in range(rows) for c in range(cols))
```

The prunes that actually matter

Three of them, and the third is the one that impresses.

One — a character-count check before searching at all.

```
if not Counter(word) ≤ Counter(ch for row in grid for ch in row):  
    return False                                # the grid does not contain enough letters
```

One pass over the grid, and it rejects hopeless inputs like a 200-character word on a 6 × 6 board instantly instead of after millions of calls.

Two — start from the rarer end. If the last character of the word appears far less often in the grid than the first, reverse the word. The number of starting positions is the branching factor of the whole search:

```
first = sum(row.count(word[0]) for row in grid)  
last = sum(row.count(word[-1]) for row in grid)  
if last < first:  
    word = word[::-1]                        # same answer, far fewer starting points
```

On a grid of mostly **A**s where the word starts with **A** and ends with **Z**, this is the difference between thirty-six starting searches and one.

Three — the most-constrained cell, for Sudoku. Instead of filling blanks in order, always fill the blank with the fewest legal digits. It costs a scan per node and it fails fast rather than deep, which turns hard puzzles from seconds into milliseconds.

Sudoku, and why it is the same program

From [yesterday](#), condensed:

depth	the index of the next blank cell (PRE-COMPUTED, not re-scanned)
choices	the digits 1..9
sets	rows[r], cols[c], boxes[(r//3)*3 + (c//3)]
prune	three membership tests before descending
undo	remove from three sets, and set the cell back to "."

The one structural difference from Word Search: **Sudoku returns as soon as it finds a solution**, so it propagates **True** up the stack and stops. Word Search does the same. Number of Islands does not — it has to visit everything.

4 The picture

Anand's third attempt, drawn. The word is **DCFC** — drumsticks, curry leaves, flowers, curd.

```

      c0    c1    c2    c3
r0 [  A    B    C    E  ]
r1 [  S    F    C    S  ]
r2 [  A    D    E    E  ]

word = "ABCCED"

start (0,0)='A' = word[0]      mark: [ # ]
  right (0,1)='B' = word[1]    mark: [ # ][ # ]
    right (0,2)='C' = word[2]  mark: [ # ][ # ][ # ]
      down (1,2)='C' = word[3] mark
        down (2,2)='E' = word[4] mark
          left (2,1)='D' = word[5] → k reaches len(word) → TRUE

grid on the way back up, with every mark restored:

r0 [  A    B    C    E  ]      exactly as it started
r1 [  S    F    C    S  ]
r2 [  A    D    E    E  ]

```

A failed attempt, and why the un-mark is what makes the next one possible:

```

looking for "ABCB"

start (0,0)='A'   mark → [ # ]
(0,1)='B'         mark → [ # ][ # ]
(0,2)='C'         mark → [ # ][ # ][ # ]
  neighbours of (0,2): (0,1) is '#' ← correctly blocked, we came from there
                        (0,3)='E' ≠ 'B'
                        (1,2)='C' ≠ 'B'
    no match → return False
  UNMARK (0,2) → [ # ][ # ][ C ]
  UNMARK (0,1) → [ # ][ B ][ C ]
  UNMARK (0,0) → [ A ][ B ][ C ]

the grid is clean, so the search starting at (0,3) sees the real board.
WITHOUT the unmarks it would see [ # ][ # ][ # ] and fail wrongly.

```

The path-versus-region difference, side by side:

WORD SEARCH (path)	NUMBER OF ISLANDS (region)
mark(r,c)	mark(r,c)
for each direction:	for each direction:
recurse	recurse
UNMARK(r,c) ← present	(no unmark) ← absent
a cell may appear on many routes	a cell belongs to ONE island
0(rows.cols.4^L)	0(rows.cols)
swap them and:	
word search → reports "not found" for words that ARE there	
islands → counts the same island repeatedly, then	
	RecursionError: maximum recursion depth exceeded

And the branching, so the complexity is not a claim:

```
first cell:    4 neighbours
after that:    3, because one of the four is where you came from
               (it is marked, so it is rejected immediately)

length-L word: 4 × 3^(L-1) paths from one starting cell
all starts:    rows × cols × 4 × 3^(L-1)

6×6 grid, L = 8:  36 × 4 × 3^7 = 36 × 4 × 2,187 = 314,928 paths, worst case
6×6 grid, L = 12: 36 × 4 × 3^11 = 36 × 4 × 177,147 = 25.5 million
```

5 The code, built step by step

Step 1 — say which kind of problem it is

“This is a path problem — I am asking whether a route exists — so the visited marker has to be cleared on the way out. If it were a region problem like counting islands, I would leave it set, because a cell belongs to exactly one region.”

Four seconds, and it is the sentence that separates people who have seen grids from people who have understood them.

Step 2 — the directions, as data

```
DIRECTIONS = ((-1, 0), (1, 0), (0, -1), (0, 1))    # up, down, left, right
```

Never four copies of the recursive call. Eight-direction problems change this line and nothing else.

Step 3 — one guard at the top

```
if k == len(word):
    return True
if not (0 ≤ row < rows and 0 ≤ col < cols) or grid[row][col] ≠ word[k]:
    return False
```

Base case first, then the combined bounds-and-match guard. **In Python the bounds check is not optional and its absence does not raise** — a negative index reads from the other end of the row, so the search silently wraps around the grid.

Step 4 — mark, explore, restore

```
original = grid[row][col]
grid[row][col] = "#"
found = any(search(row + dr, col + dc, k + 1) for dr, dc in DIRECTIONS)
grid[row][col] = original
return found
```

any **short-circuits**, so the moment one direction succeeds the rest are not tried — and the restore still runs, because it is after the assignment rather than after a `return`. Writing it as `if search(...): return True` inside a loop is the version that skips the restore, which is [day 094's](#) early-return trap.

Step 5 — the outer loop, and the cheap prunes

```
if len(word) > rows * cols:
    return False
if not Counter(word) ≤ Counter(ch for r in grid for ch in r):
    return False
if sum(r.count(word[-1]) for r in grid) < sum(r.count(word[0]) for r in grid):
    word = word[::-1]
```

Three lines, all `0(rows * cols)`, and on adversarial inputs they are the difference between passing and timing out.

The complete solution

```
from collections import Counter

DIRECTIONS = ((-1, 0), (1, 0), (0, -1), (0, 1))    # up, down, left, right

def word_search(board: list[list[str]], word: str) → bool:
    """LeetCode 79. Is `word` spelled by a path of adjacent cells?

    A PATH problem, so the visited marker is CLEARED on the way out – a cell
    used by one route must be free for another. Marking the board itself and
    restoring it costs no extra space; the price is that it mutates the input.

    Time  $O(\text{rows} \times \text{cols} \times 4 \times 3^{(L-1)})$  – after the first step only 3 of the 4
       neighbours are new, because one is where you came from.
    Space  $O(L)$  for the stack.
    """
    if not board or not board[0]:
        return False
    rows, cols = len(board), len(board[0])

    if len(word) > rows * cols:
        return False                                # cannot fit
    if not Counter(word) ≤ Counter(ch for row in board for ch in row):
        return False                                # not enough letters exist
    if sum(row.count(word[-1]) for row in board) < \
       sum(row.count(word[0]) for row in board):
        word = word[::-1]                            # start from the rarer end

    def search(row: int, col: int, k: int) → bool:
        if k == len(word):
            return True                                # every character matched
        if not (0 ≤ row < rows and 0 ≤ col < cols):
            return False                                # BOUNDS – never optional
        if board[row][col] ≠ word[k]:
            return False

        original = board[row][col]
        board[row][col] = "#"                            # choose: no word contains '#'
        found = any(
            search(row + dr, col + dc, k + 1) for dr, dc in DIRECTIONS
        )
        board[row][col] = original                        # `any` short-circuits
        return found

    return any(search(r, c, 0) for r in range(rows) for c in range(cols))

def all_paths(grid: list[list[int]], start: tuple[int, int],
              end: tuple[int, int]) → list[list[tuple[int, int]]]:
    """Every simple path from start to end, avoiding cells marked 1.

    The other kind of path problem: not "does one exist" but "list them all",
    so there is no early return and the copy trap from day 091 is back.
    """
    rows, cols = len(grid), len(grid[0])
    paths: list[list[tuple[int, int]]] = []
    current: list[tuple[int, int]] = []
    visited = [[False] * cols for _ in range(rows)]

    def walk(r: int, c: int) → None:
        current.append((r, c))                            # choose 1
        visited[r][c] = True                                # choose 2
        if (r, c) == end:
            paths.append(current[:])                        # COPY
            current.pop()
        for dr, dc in DIRECTIONS:
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < rows and 0 ≤ nc < cols and not visited[nr][nc] and grid[nr][nc] != 1:
                walk(nr, nc)
```

```

        else:
            for dr, dc in DIRECTIONS:
                nr, nc = r + dr, c + dc
                if not (0 ≤ nr < rows and 0 ≤ nc < cols):
                    continue
                if visited[nr][nc] or grid[nr][nc] == 1:
                    continue
                walk(nr, nc)
            visited[r][c] = False                # un-choose 2
            current.pop()                        # un-choose 1

        walk(*start)
        return paths

def number_of_islands(grid: list[list[str]]) → int:
    """LeetCode 200 – the REGION version, for contrast.

    The marker is set and NEVER cleared. A cell belongs to exactly one island
    for ever. Clearing it would count islands repeatedly and would recurse
    for ever between two adjacent cells.

    Time O(rows × cols) – each cell is visited once. Not exponential.
    """
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    islands = 0

    def sink(r: int, c: int) → None:
        if not (0 ≤ r < rows and 0 ≤ c < cols) or grid[r][c] ≠ "1":
            return
        grid[r][c] = "0"                # mark, and NEVER restore
        for dr, dc in DIRECTIONS:
            sink(r + dr, c + dc)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                islands += 1
                sink(r, c)
    return islands

def solve_sudoku(board: list[list[str]]) → bool:
    """LeetCode 37, with the two optimisations that matter.

    1. Pre-compute the blank cells once instead of scanning 81 cells per call.
    2. Fill the MOST CONSTRAINED blank first – the one with the fewest legal
       digits – so the search fails fast rather than deep.
    """
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    blanks: list[tuple[int, int]] = []

    for r in range(9):
        for c in range(9):
            value = board[r][c]
            if value == ".":
                blanks.append((r, c))
            else:
                rows[r].add(value)
                cols[c].add(value)
                boxes[(r // 3) * 3 + (c // 3)].add(value)

    def candidates(r: int, c: int) → set[str]:

```

```

        b = (r // 3) * 3 + (c // 3)
        return set("123456789") - rows[r] - cols[c] - boxes[b]

def fill(remaining: list[tuple[int, int]]) → bool:
    if not remaining:
        return True
    # most-constrained cell first: fail fast, not deep
    cell = min(remaining, key=lambda rc: len(candidates(*rc)))
    rest = [x for x in remaining if x ≠ cell]
    r, c = cell
    b = (r // 3) * 3 + (c // 3)

    for digit in candidates(r, c):
        rows[r].add(digit); cols[c].add(digit); boxes[b].add(digit)
        board[r][c] = digit # choose (4 changes)
        if fill(rest):
            return True
        rows[r].remove(digit); cols[c].remove(digit); boxes[b].remove(digit)
        board[r][c] = "." # un-choose (4 restores)
    return False

return fill(blanks)

def word_search_broken(board: list[list[str]], word: str) → bool:
    """The classic bug: the mark is never restored. Run it and see."""
    rows, cols = len(board), len(board[0])

    def search(row: int, col: int, k: int) → bool:
        if k == len(word):
            return True
        if not (0 ≤ row < rows and 0 ≤ col < cols) or board[row][col] ≠ word[k]:
            return False
        board[row][col] = "#"
        return any(search(row + dr, col + dc, k + 1) for dr, dc in DIRECTIONS)
        # missing: board[row][col] = original

    return any(search(r, c, 0) for r in range(rows) for c in range(cols))

if __name__ == "__main__":
    board = [list(r) for r in ("ABCE", "SFCS", "ADEE")]
    print(word_search([r[:] for r in board], "ABCCED")) # True
    print(word_search([r[:] for r in board], "SEE")) # True
    print(word_search([r[:] for r in board], "ABCB")) # False – cannot reuse (0,1)

    print(word_search_broken([r[:] for r in board], "SEE"))
    # False ← WRONG. An earlier failed attempt consumed the cells.

    maze = [[0, 0, 1], [1, 0, 0], [0, 0, 0]]
    for path in all_paths(maze, (0, 0), (2, 2)):
        print(path)
    # [(0, 0), (0, 1), (1, 1), (1, 2), (2, 2)]
    # [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2)]
    # [(0, 0), (0, 1), (1, 1), (2, 1), (2, 0)] ... (only those reaching the end)

    islands = [list(r) for r in ("11000", "11000", "00100", "00011")]
    print(number_of_islands(islands)) # 3

    puzzle = [list(r) for r in (
        "53..7....", "6..195...", ".98....6.",
        "8...6...3", "4..8.3..1", "7...2...6",
        ".6....28.", "...419..5", "....8..79",
    )]
    print(solve_sudoku(puzzle), "".join(puzzle[0])) # True 534678912

```

6 What it costs

Word Search

```
starting cells:      rows × cols
first move:         4 directions
every move after that: 3, because one neighbour is where you came from
                      (it is marked, so it is rejected immediately)
→ 0(rows × cols × 4 × 3(L-1)) for a word of length L
```

Concretely:

```
6 × 6 grid
L = 4    36 × 4 × 33 =      3,888 paths
L = 8    36 × 4 × 37 =    314,928
L = 12   36 × 4 × 311 =  25,509,168
L = 16   36 × 4 × 315 = 2,066,242,608 – not going to finish
```

the practical limit is a word of about 12 characters on a small grid, and that is exactly what the LeetCode constraints allow.

The **3** rather than **4** is worth saying out loud — it shows you thought about the marker rather than quoting a formula. And the bound is a worst case that assumes every letter matches everywhere; on real inputs the character check kills most paths at depth 1 or 2.

The prunes, measured

grid of 36 'A's, word = "AAAAAAAAAAB" (12 chars, ending in a letter not present)

```
no prunes:      ~25 million calls, several seconds
character count: rejected in 36 comparisons          ~0 ms
```

grid mostly 'A' with one 'Z', word = "AAAZ"

```
search forwards (36 'A' starts):    36 starting searches
search reversed (1 'Z' start):       1 starting search      ~36× fewer
```

Both prunes are **0(rows × cols)** and both are one line. This is the same trade as [yesterday's](#) early check: a linear pass that removes an exponential subtree.

Path against region

```
Word Search      0(rows × cols × 3L)    cells revisited across routes
Number of Islands 0(rows × cols)          each cell visited exactly once
```

Exponential against linear, and the only difference in the code is one line. That is the sharpest way to state why the path-or-region question matters.

Space

```
marking the board in place: 0 extra
a separate visited grid:   rows × cols booleans
recursion stack:           O(L) for word search, O(rows × cols) for islands
```

For Number of Islands the stack is the danger: a 1000×1000 grid that is entirely land recurses a million deep.

```
RecursionError: maximum recursion depth exceeded
```

Say this before they ask: for large region problems, use an explicit stack or a queue instead of recursion. For Word Search it never happens, because the depth is the word length.

Sudoku, with and without the heuristic

typical newspaper puzzle	in-order blanks: ~15,000 calls	~10 ms
	most-constrained: ~200 calls	~2 ms
a deliberately hard puzzle	in-order blanks: ~2,500,000 calls	~3 s
	most-constrained: ~15,000 calls	~30 ms

A hundred-fold on hard inputs, and nothing on easy ones, which is exactly the profile of a good heuristic: it costs a scan per node and it pays only when the search would otherwise be deep.

7 The traps

Trap 1 — no bounds check, and no error either

```
if board[row][col] ≠ word[k]:      # row could be -1
```

Python treats `board[-1]` as the last row. There is no `IndexError` — the search wraps around the grid and reports words that are not there.

```
board = ["AB", "CD"], word = "ADA"
without bounds: True    (it walks off the top and comes back on the bottom)
with bounds:   False
```

The absence of an error is what makes this dangerous. Check bounds first, always.

Trap 2 — never restoring the mark

```
board[row][col] = "#"  
return any(search(...) for ...)  
# missing the restore
```

```
word_search_broken(board, "SEE") → False
```

The word is present. An earlier failed attempt consumed the cells and never gave them back. **This is the path-problem bug**, and it reports absence rather than crashing, so a small test may well pass.

Trap 3 — restoring in a region problem

The mirror image. In Number of Islands, clearing the mark means two adjacent land cells send each other back and forth for ever:

```
RecursionError: maximum recursion depth exceeded
```

and, if you cap the depth, the same island is counted many times.

Trap 4 — the early return that skips the restore

```
for dr, dc in DIRECTIONS:  
    if search(row + dr, col + dc, k + 1):  
        return True          # board[row][col] is still '#'  
board[row][col] = original
```

The successful path leaves `#` scattered through the grid. Harmless if you return immediately and never look at the board again — a real bug if the caller searches for a second word, which is exactly what a test suite does.

Use `any(...)` and assign to a variable, so the restore is unconditional.

Trap 5 — a shallow copy of the grid

```
copy = board[:]          # new outer list, SAME row objects  
copy[r][c] = "#"         # also modifies board[r][c]
```

`board[:]` copies the outer list only. Every row is shared, so nothing is protected. If you genuinely need a copy it is `[row[:] for row in board]` — and you almost never need one, because marking and restoring is `O(1)` while copying is `O(rows × cols)` per node.

Trap 6 — using a `set` of coordinates when a grid would do

```
visited.add((row, col))    # tuple allocation + hashing per cell
```


Correct but slow: every step allocates a tuple and hashes it. A boolean grid, or marking the board itself, is several times faster and no harder to write. Fine to mention; do not make it your first version if the grid is large.

Trap 7 — scanning for the next Sudoku blank on every call

```
for r in range(9):
    for c in range(9):
        if board[r][c] == ".":      # 81 cells, every single call
```

Eighty-one comparisons per node on a tree with thousands of nodes. Pre-compute the blanks once. It is a pure constant-factor win and it costs three lines.

Trap 8 — '#' when the alphabet might contain it

Marking with a sentinel character assumes the character cannot appear in the word or the grid. For letters that is safe; for an arbitrary alphabet it is not. **Say the assumption out loud** — “I am marking with a character that cannot appear in the input; if that is not guaranteed I would use a separate visited grid” — and it becomes a considered choice rather than a lucky one.

8 In the interview

How it gets asked

- The grid classic: “Given a board and a word, does the word exist in the grid?” LeetCode 79.
- The escalation: “Now find all the words in this list.” — Word Search II, LeetCode 212, which needs a trie and is [day 123](#).
- The contrast: “Count the islands in this grid.” LeetCode 200.
- The constraint grid: “Solve this Sudoku.” LeetCode 37.
- The probe: “Why did you set the cell back to its original value?”
- The scale probe: “The grid is 1000 by 1000.”

What to say out loud, in the first ninety seconds

1. **Classify the problem before anything else.** “This is a path problem — I want to know whether a route exists — so the visited marker is cleared on the way out. If it were a region problem like counting islands, I would leave it set, and that one line is the difference between exponential and linear.”
2. **Say the search shape.** “Depth-first from every cell, matching one character per level. Four directions, as a tuple of offsets, so an eight-direction variant is one line.”

3. **Bounds first, and say why.** “The bounds check is the first thing in the function, because in Python a negative index does not raise — it reads from the other end of the row, so the search would silently wrap around the grid.”
4. **Name the marking trick and its price.** “I mark by overwriting the cell with a character no word can contain, and restore it after. That is zero extra space; the cost is that it mutates the caller’s grid, so if this were library code I would use a separate visited grid instead.”
5. **Give the complexity with the 3.** “Rows times columns starting points, four directions on the first step and three thereafter — because one neighbour is always the cell you came from and it is marked — so $O(\text{rows} \times \text{cols} \times 4 \times 3^{(L-1)})$. Space is $O(L)$ for the stack.”
6. **Offer the prunes.** “Two cheap ones before searching: reject if the grid does not contain enough of each letter, and reverse the word if its last character is rarer than its first, since the number of starting cells is the branching factor of the entire search.”

The follow-ups

“**Why did you set the cell back to its original value?**” “Because this is a path problem. A cell that was part of one attempted route has to be available to a different route — the only thing that must stay clean is the *current* path, so that it does not use the same cell twice. If I leave the mark in place, a failed attempt permanently consumes cells and the search reports that a word is absent when it is present. That is the failure mode worth naming, because it produces a wrong answer rather than a crash. The contrast is a region problem like counting islands: there, a cell belongs to exactly one island for ever, so I set the mark and never clear it — and if I *did* clear it, two adjacent land cells would bounce back and forth until the recursion limit.”

“**What is the complexity?**” “ $O(\text{rows} \times \text{cols} \times 4 \times 3^{(L-1)})$ for a word of length L . Rows times columns starting cells; four directions from the first cell; and only three from every cell after that, because one of the four neighbours is the cell you just came from and it is currently marked. Space is $O(L)$ for the stack — constant extra beyond that, since I mark the board in place. Concretely, on a 6×6 grid a 12-character word is about twenty-five million paths in the worst case, which is roughly the practical limit and is exactly where the problem constraints sit. In practice it is far less, because a wrong character kills a branch at depth one.”

“**Can you speed it up?**” “Three things, all cheap. First, a character-count check: if the grid does not contain at least as many of each letter as the word needs, return immediately — one pass over the grid, and it kills adversarial inputs like a long word ending in a letter that is not there. Second, reverse the word if its last character occurs less often in the grid than its first, because the count of starting cells multiplies the entire search — on a grid of `A s` with one `Z`, searching for `AAAZ` backwards is thirty-six times fewer

searches. Third, if I had to find *many* words rather than one, I would stop searching per word entirely and put all the words in a trie, then walk the grid once and descend the trie alongside — that is Word Search II and it turns $\text{words} \times \text{grid} \times 3^L$ into one traversal.”

“Now count the islands instead.” “Same traversal, one line different, and the difference is fundamental. I mark a land cell as visited and never restore it, because a cell belongs to exactly one island. Then I loop over every cell, and every time I find unvisited land I increment the count and sink the whole connected region. That makes it $O(\text{rows} \times \text{cols})$ — each cell is visited once — instead of exponential. The thing I would flag for a large grid is the recursion depth: a thousand-by-thousand grid that is all land recurses a million deep and raises `RecursionError`. For region problems at scale I would use an explicit stack or a queue.”

“The grid is 1000 by 1000.” “For word search that is fine — the depth is the word length, not the grid size, so the stack is small and the starting-cell loop is a million iterations of a check that usually fails on the first character. The prunes matter much more at that size. For a region problem it is the opposite: the traversal is a million cells, which is fast, but the recursion depth can also be a million, so it has to be iterative. That asymmetry is worth stating — in grid problems, the thing that breaks at scale is almost always the stack, not the work.”

“Why not use a set of visited coordinates?” “It is correct and it is slower: every step allocates a tuple and hashes it, where a boolean grid is one array write. I would use a set if the grid were sparse or unbounded — an infinite plane, or coordinates that are not small integers — because then there is no grid to index into. On a dense fixed grid, mark the grid.”

A model answer

Asked: *find the word in the grid.*

“Before I write anything, let me classify it, because that decides one line that matters more than the rest of the code. **This is a path problem** — the question is whether a route exists — so the visited marker gets cleared on the way out. A cell used by one attempted route must be free for a different route; the only thing that has to stay clean is the current path. Contrast that with counting islands, where a cell belongs to one region for ever and the mark is never cleared. Same traversal, one line different, and it is the difference between exponential and linear.

The search is depth-first from every cell. Each level matches one more character of the word. The four directions are a tuple of offsets rather than four copies of the call, so an eight-direction variant is a one-line change.

Inside the function: base case first — if I have matched every character, return true. Then the guard, and **the bounds check has to come before the character comparison**, because in Python a negative index does not raise. `board[-1]` is the last row, so without the check the search silently wraps around the grid and finds words that are not there. That is the kind of bug that passes small tests.

To mark a cell I overwrite it with a character no word can contain and restore the original afterwards. That is zero extra space, and the restore is the un-choose. I would say the price out loud: it mutates the caller’s grid, so for library code I would keep a separate visited array instead. I also write the four recursive calls with `any`, assigning the result to a variable before restoring — if I wrote `if search(...): return True` inside a loop, the successful path would return past the restore and leave marks scattered through the board, which breaks the *next* search rather than this one.

Complexity: rows times columns starting cells, four directions from the first cell and **three** from every cell after, because one of the four neighbours is where I came from and it is marked. So $O(\text{rows} \times \text{cols} \times 4 \times 3^{(L-1)})$, and space $O(L)$ for the stack. On a 6×6 grid, a 12-character word is around twenty-five million paths in the worst case.

Two cheap prunes I would add before searching at all. A character-count check — if the grid does not hold enough of each letter, return false after one pass. And reversing the word when its last character is rarer in the grid than its first, because the number of starting cells multiplies the whole search. Both are linear in the grid and both can remove an exponential subtree.

If you then asked me to find a whole list of words, I would stop doing this per word: put all the words in a trie, walk the grid once, and descend the trie alongside the path, so shared prefixes are searched once instead of once per word.”

9 Recall card

- **Classify first: path or region.** A **path** problem (word search, maze, all paths) **clears the marker on the way out**, because a cell used by one route must be free for another — `0(rows*cols*3^L)`. A **region** problem (islands, flood fill) **never clears it**, because a cell belongs to one region for ever — `0(rows*cols)`. **One line, exponential versus linear.** Swap them and word search reports absent words; islands raises `RecursionError`.
- **Bounds check FIRST, and it does not raise if you skip it** — `board[-1]` is the last row, so the search silently wraps around the grid. Directions live in one tuple `((-1,0),(1,0),(0,-1),(0,1))`, never four copies of the call.
- **Mark by overwriting the cell with a sentinel, restore after** — zero extra space, and the restore is the un-choose. Use `found = any(...)` then restore, never `if search(...): return True` inside a loop — that **early return skips the restore** and leaves `#` in the grid for the next search.
- **Complexity is `0(rows * cols * 4 * 3^(L-1))`** — the `3`, not `4`, because one neighbour is always where you came from and it is marked. 6×6 grid: $L = 8$ is 315,000 paths, $L = 12$ is 25 million, $L = 16$ will not finish. Space `0(L)`.
- **Two one-line prunes that remove exponential subtrees:** reject if `Counter(word) > Counter(grid)`, and **reverse the word when its last letter is rarer than its first** ($36\times$ fewer starting cells on a grid of `A`s with one `Z`). **Sudoku is the same program** — three sets, pre-computed blanks, and the **most-constrained cell first**, which is $\sim 100\times$ on hard puzzles and free on easy ones.

1 What this is, and why they ask it

The **low-level design round** gives you a prompt in one sentence — design a parking lot, an elevator, a chess game — and forty minutes to produce classes, their responsibilities, and the code for the two or three that matter.

Three sentences. It is not a knowledge test; there is nothing to recall. It is a **performance**, and the thing being assessed is whether you can turn vague requirements into objects with clear responsibilities while thinking out loud. And because it is a performance, **the same five moves work for every prompt**, which is what today is for.

You have now done eighteen of these — [parking lot](#), [elevator](#), [ATM](#), [vending machine](#), [library](#), [chess](#), [card game](#), [Splitwise](#), [BookMyShow](#), [food delivery](#), [ride-hailing](#), [rate limiter](#), [cache](#), [logging](#), [notifications](#), [file system](#), [snake and ladder](#) and [auction](#). Today is not new material. It is the extraction: what was the same in all eighteen, and how do you produce it under a clock with somebody watching.

2 The story

The shop was one room with a glass front on the main road, and Basheer had been in it since 1989.

People came in wanting all sorts of things. A shirt. A suit for a son's wedding. A blouse to go with a sari somebody's mother had left them. Curtains, twice. Once, a cover for a motorbike seat.

He did the same thing every time, and it took about four minutes.

First he asked what it was for. Not what they wanted made — what it was *for*. Wedding, office, every day. Because a shirt for the office and a shirt for a wedding are not the same shirt, and if he did not ask, he would make the wrong one perfectly.

Then he asked two or three questions that narrowed it. Full sleeve or half. Loose or fitted. Is this going to be washed every week or twice a year.

Then he took the measurements, and this was the part that never changed. Chest, waist, shoulder, sleeve, length, neck. The same six, in the same order, for everybody, every time. He called them out and his nephew wrote them into the phone. Even for the

curtains he went through the list in his head first, which made no sense to anybody watching, and then took the two he actually needed.

Then he said what he was going to do, out loud, before touching anything. This much cloth, this cut, these buttons, ready on Thursday. And people would correct him at that point — no, not those buttons — which was exactly why he said it.

Only after all that did he pick up the chalk.

A young man who had come to learn from him spent a week being annoyed by this. He kept saying, you know what a shirt is. Why do you ask the same six things every single time.

Basheer said: because the day I stop asking is the day somebody walks out with a shirt they cannot wear to the thing they bought it for. The questions are not for me. They are so that both of us know what we agreed before I cut anything, and cloth does not go back together.

3 The idea in plain English

Basheer has an interview script, and it is the same one that works for every LLD prompt.

- “What is it for” is **clarifying requirements**. The prompt is one sentence and it is deliberately incomplete.
- The two or three narrowing questions are **scoping** — deciding out loud what you are *not* building.
- The six measurements in the same order are **the five moves**, below. The same procedure regardless of the prompt.
- Saying what he is going to do before touching anything is **presenting the design before writing code**, so the interviewer can correct you while it is still free.
- “Cloth does not go back together” is why: **in forty minutes there is no time to restart**.

The five moves

Every one of the eighteen case studies used these, in this order.

#	MOVE	TIME	WHAT IT PRODUCES
1	Clarify	0–5 min	3–4 questions, each with the answer you will assume
2	The nouns	5–10 min	A list of classes, one line each on responsibility
3	The class diagram	10–20 min	Fields, key methods, relationships
4	The interesting part	20–30 min	The one decision the prompt is really about, behind an interface
5	The code	30–40 min	Two or three classes, written properly

Move 4 is what you are being marked on. Moves 1 to 3 are table stakes; almost everybody gets a `Board`, a `Player` and a `Game`. What separates candidates is spotting that every prompt has exactly one place where the design is genuinely decided, putting an interface there, and showing two implementations.

The interesting part, prompt by prompt

This table is the revision. It is the answer to “what was the same in all eighteen”.

PROMPT	THE ONE DECISION	THE INTERFACE THAT GOES THERE
Parking lot	which spot does this vehicle get	<code>SpotAllocationStrategy</code>
Elevator	which lift answers this call	<code>SchedulingStrategy</code>
ATM	how do you make up ₹2,300 from the cassettes	<code>DispenseStrategy</code>
Vending machine	what does the machine do next	<code>State</code> — a state machine
Library	when is a book due, and what is the fine	<code>LoanPolicy</code>
Chess	is this move legal for this piece	<code>Piece.moves_from(board, square)</code>
Card game	what are the rules of <i>this</i> game	<code>GameRules</code>
Splitwise	how is this expense divided	<code>SplitStrategy</code>
BookMyShow	how do you hold a seat while someone pays	seat locking , with a TTL
Food delivery	which courier gets this order	<code>AssignmentStrategy</code>
Ride-hailing	what does this ride cost	<code>PricingStrategy</code>
Rate limiter	which counting rule	<code>RateLimitAlgorithm</code>
Cache	what gets thrown out	<code>EvictionPolicy</code>
Logging	where does this line go, and in what shape	<code>Handler</code> + <code>Formatter</code>
Notifications	which channel, and how hard do we retry	<code>Channel</code> + <code>RetryPolicy</code>
File system	how do files and directories share a type	one <code>Entry</code> , the composite
Snake and ladder	where does randomness come from	<code>Dice</code>
Auction	what happens when two bids collide	a conditional write

Fifteen of the eighteen are a strategy — one method, several implementations, chosen at runtime. That is not a coincidence and it is the single most useful pattern in this round. When you cannot see the interesting part, ask yourself: *which sentence in the prompt would a product manager change next year?* Put an interface there.

The four tells

Four phrases in a prompt, each of which names its own pattern.

IF THE PROMPT SAYS...	YOU WANT...
"...depending on", "...based on the type of"	Strategy : one interface, several implementations
"...and then it becomes", "while it is being..."	State : an explicit state machine, and only legal transitions
"notify", "when X happens, also..."	Observer : publish an event, do not call the other subsystem
"contains other", "inside another"	Composite : one shared type for the leaf and the container

And two structural rules that came up in nearly every case study:

- **Randomness, time and input/output must be replaceable.** `Dice`, `Clock`, `Channel`. If any of them is hard-coded, nothing in the design can be tested. This came up on [snake and ladder](#) and again on [auction](#), and it will come up in your round.
- **Return a result object; do not print.** `TurnResult`, `BidResult`, `MoveResult`. It is what lets one engine serve a terminal, a phone and a simulation.

The five mistakes that lose the round

1. **Coding at minute three.** You will design the wrong thing beautifully. Clarify first.
2. **Silence.** They are marking your reasoning, and they cannot mark what they cannot hear. Narrate even the boring parts.
3. **Anaemic classes with a God service.** `Order`, `User`, `Item` holding only fields, and one `OrderService` with every rule in it. Behaviour belongs with the data it uses.
4. **Enums where polymorphism belongs.** `if piece.type == KNIGHT` repeated in four methods. Every one of those is a `switch` you will have to edit when a variant appears.
5. **Getting stuck on completeness.** Nobody finishes. Say what you are leaving out and why, and spend the time on the interesting part.

4 The picture

The forty minutes, drawn.

0	5	10	20	30	40
CLARIFY	NOUNS	CLASS DIAGRAM	INTERESTING PART	CODE	
3-4 Qs	one line	fields, methods,	interface +	2-3 classes,	
with your	each	relationships	2 implementations	properly	
assumed	class		← THE MARK IS HERE		
answers					

checkpoints, said out loud:

min 5 "so I am building X, not Y – tell me if that is wrong"
 min 10 "here are my classes; anything you would add before I draw?"
 min 20 "here is the diagram; the interesting part is Z"
 min 30 "I will code Z and the class it lives on"
 min 38 "what I have left out: A, B, C. Here is how each would fit."

The decision procedure for finding the interesting part:

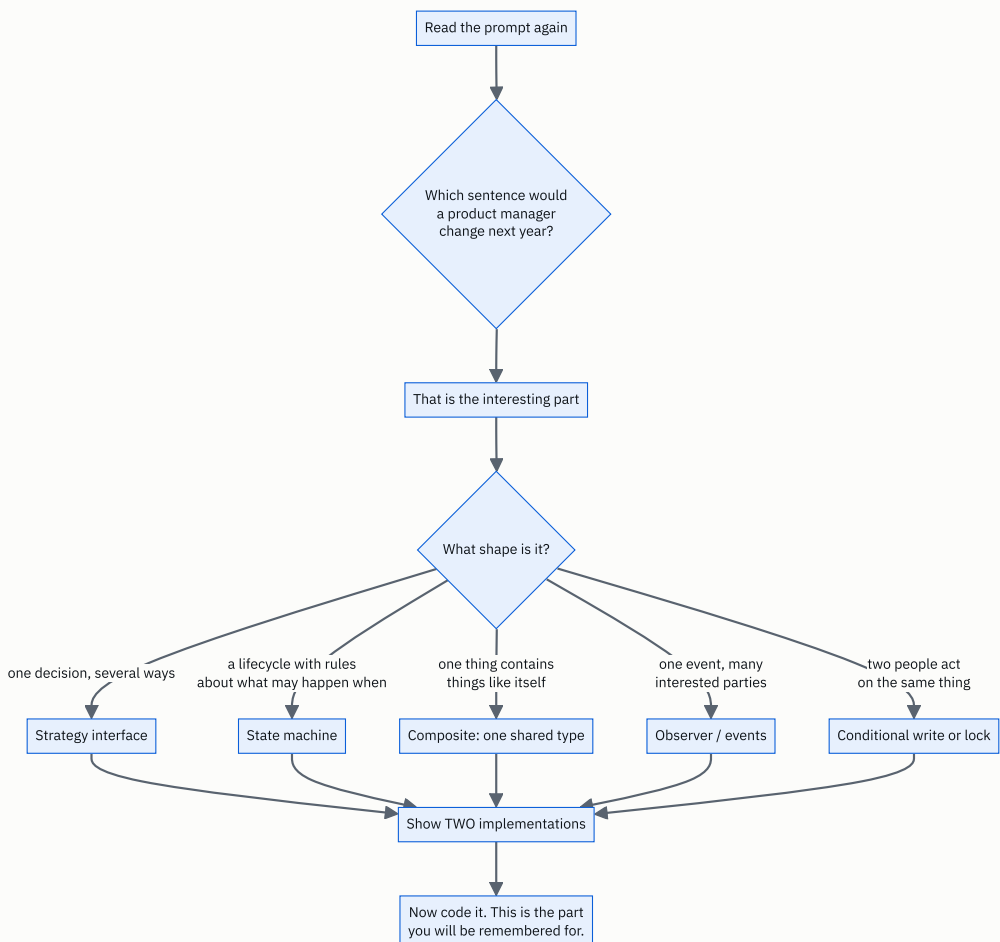
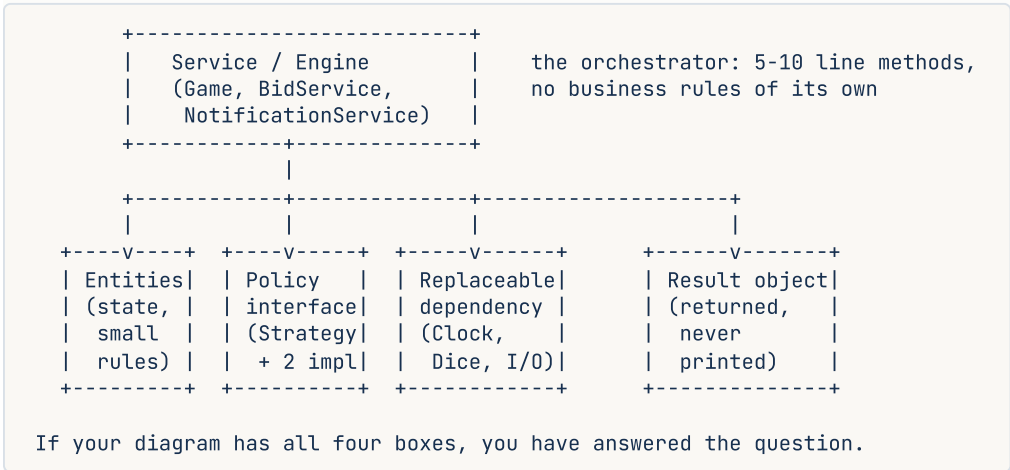


FIGURE 96.1

And the shape almost every LLD answer takes:



5 How it actually works

The mock: *Design a chess game. Forty minutes. Begin.*

You met chess on [day 083](#). This is the timed performance, written out as it should sound.

“Let me check the scope. Four questions.

One — am I designing the rules engine, or a playable product? I will assume the engine: a board, pieces, legal move generation, check and checkmate detection, and a game that tracks whose turn it is. Not networking, not a UI, not an opponent that chooses moves — although I will leave a seam for that.

Two — full rules, including castling, en passant and promotion? I will design so that all three fit, and implement promotion, because it is the one that changes the board in an unusual way. If we run out of time, castling and en passant are the two I would leave as named gaps.

Three — do I need move history? Yes, and not for undo — for *correctness*. Castling depends on whether the king or rook has ever moved, en passant depends on the immediately previous move, and the threefold-repetition draw depends on the whole history. So history is part of the state, not a feature.

Four — two humans, or does one side need to be a program? I will put the move *choice* behind an interface so a human and an engine are two implementations, but I will not write a chess engine.

Anything there you want changed before I start?”

Question three is the one that impresses, because it shows you know the domain has hidden state. Most candidates discover castling at minute thirty-five.

“The classes, one line each.

- `Square` — a coordinate. Immutable, and it knows how to add an offset and stay on the board.
- `Piece` — colour, type, and whether it has moved. **Abstract**, with a subclass per kind.
- `Board` — the 8×8 arrangement, plus ‘which piece is here’, ‘is this square attacked’.
- `Move` — from, to, the piece, anything captured, promotion, and the flags for castling and en passant. Immutable, and rich enough to be undone.
- `MoveValidator` — is this move legal *in this position*, which is not the same as whether the piece moves that way.
- `Game` — turn order, history, status, and the one public method: `make_move`.
- `GameStatus` — in progress, check, checkmate, stalemate, draw.
- `MoveChooser` (interface) — a human or a program deciding what to play.

I am deliberately not making `Player` do anything. It is a colour and a name.”

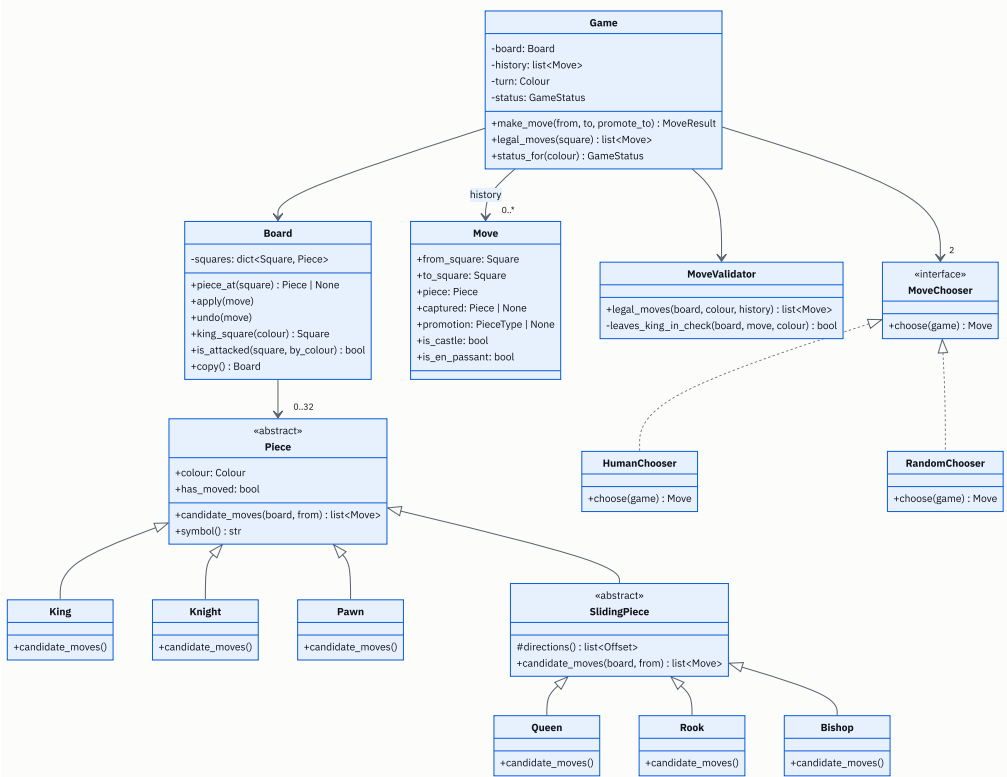


FIGURE 96.2

“Two things to point at.

`Piece` is abstract with a subclass per kind, and `candidate_moves` is the polymorphic method. The alternative — a `PieceType` enum and a `switch` inside the validator — means every rule change edits the same method, and a chess variant with a new piece edits it again. This is the enum-versus- polymorphism decision, and chess is the clearest example of it there is.

`SlidingPiece` sits between them because rook, bishop and queen differ only in their direction list. Sliding — walk outward until you hit the edge, an enemy piece which you may capture, or a friendly piece which you may not — is one method written once. Queen’s directions are the rook’s plus the bishop’s.”

“Chess has one place where the design is really decided, and it is **not** how each piece moves. That part is easy. The interesting part is that **legality has two layers**.

A knight moves in an L. That is the piece’s rule and it is local. But a knight that is pinned against its own king **may not move at all**, even though the L is available — and no amount of knowledge about knights tells you that. It depends on the position of a completely different piece.

So I split it in two:

- `Piece.candidate_moves(board, from)` — *how this piece moves*. Local, per subclass, easy to test.
- `MoveValidator.legal_moves(...)` — *what is actually allowed in this position*: take the candidates and remove any that leave your own king attacked.

And the way you test the second one is the trick worth knowing: **make the move, ask whether your king is attacked, then unmake it**. That is exactly the choose-recurse-un-choose from the DSA side of this week — apply, check, undo — and it is why `Move` has to carry the captured piece: without it, undo cannot restore what was taken.

```
for move in candidate_moves:
    board.apply(move)
    safe = not board.is_attacked(board.king_square(me), by=opponent)
    board.undo(move)           # ← this is why Move stores `captured`
    if safe:
        legal.append(move)
```

That single loop gives me pins, discovered check, and ‘you must get out of check’ for free — three rules I never have to write, because they all fall out of ‘your king must not be attacked after your move’. **Checkmate then needs no special code at all: it is `in_check` and not `legal_moves`. Stalemate is not `in_check` and not `legal_moves`**. Two lines, two of the hardest concepts in the game.”

Minutes 30–40 • The code

```
from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass
from enum import Enum, auto

class Colour(Enum):
    WHITE = auto()
    BLACK = auto()

    @property
    def other(self) → "Colour":
        return Colour.BLACK if self is Colour.WHITE else Colour.WHITE

@dataclass(frozen=True)
class Square:
    file: int          # 0..7, a..h
    rank: int          # 0..7, 1..8

    def offset(self, df: int, dr: int) → "Square | None":
        f, r = self.file + df, self.rank + dr
        return Square(f, r) if 0 ≤ f < 8 and 0 ≤ r < 8 else None
```

`offset` returning `None` off the board is the same “bounds check first” habit as the grid problems — one place to get it right instead of eight.

```
@dataclass
class Move:
    from_square: Square
    to_square: Square
    piece: "Piece"
    captured: "Piece | None" = None      # REQUIRED for undo
    promotion: type["Piece"] | None = None
    is_castle: bool = False
    is_en_passant: bool = False
```

```

class Piece(ABC):
    def __init__(self, colour: Colour) → None:
        self.colour = colour
        self.has_moved = False          # castling and the pawn double-step need this

    @abstractmethod
    def candidate_moves(self, board: "Board", origin: Square) → list[Move]:
        """How this piece moves. NOT whether the move is legal in this position."""

class SlidingPiece(Piece):
    """Rook, bishop and queen differ ONLY in their direction list."""

    @abstractmethod
    def directions(self) → list[tuple[int, int]]: ...

    def candidate_moves(self, board: "Board", origin: Square) → list[Move]:
        moves: list[Move] = []
        for df, dr in self.directions():
            square = origin.offset(df, dr)
            while square is not None:
                occupant = board.piece_at(square)
                if occupant is None:
                    moves.append(Move(origin, square, self))
                    square = square.offset(df, dr)          # keep sliding
                    continue
                if occupant.colour is not self.colour:
                    moves.append(Move(origin, square, self, captured=occupant))
                    break                                   # blocked either way
            return moves

class Rook(SlidingPiece):
    def directions(self):
        return [(1, 0), (-1, 0), (0, 1), (0, -1)]

class Bishop(SlidingPiece):
    def directions(self):
        return [(1, 1), (1, -1), (-1, 1), (-1, -1)]

class Queen(SlidingPiece):
    def directions(self):
        return Rook(self.colour).directions() + Bishop(self.colour).directions()

```

Three pieces in nine lines, because the shared behaviour was factored out rather than repeated. Saying “rook, bishop and queen are one algorithm with three direction lists” out loud is worth a lot.

```

class Knight(Piece):
    JUMPS = [(1, 2), (2, 1), (2, -1), (1, -2),
              (-1, -2), (-2, -1), (-2, 1), (-1, 2)]

    def candidate_moves(self, board: "Board", origin: Square) → list[Move]:
        moves = []
        for df, dr in self.JUMPS:
            square = origin.offset(df, dr)
            if square is None:
                continue
            occupant = board.piece_at(square)
            if occupant is None or occupant.colour is not self.colour:
                moves.append(Move(origin, square, self, captured=occupant))
        return moves

```

And the validator, which is the part the round is really about:

```

class MoveValidator:
    def legal_moves(self, board: "Board", colour: Colour) → list[Move]:
        """Candidates minus anything that leaves your own king attacked.

        Apply, check, undo – the same pattern as backtracking, and the reason
        `Move` must carry the captured piece.

        This one loop gives pins, discovered check, and "you must escape
        check" for free. None of the three is written anywhere.
        """
        legal: list[Move] = []
        for origin, piece in board.pieces_of(colour):
            for move in piece.candidate_moves(board, origin):
                board.apply(move)                                # choose
                king = board.king_square(colour)
                safe = not board.is_attacked(king, by=colour.other)
                board.undo(move)                                   # un-choose
                if safe:
                    legal.append(move)
        return legal

    def status(self, board: "Board", colour: Colour) → "GameStatus":
        moves = self.legal_moves(board, colour)
        in_check = board.is_attacked(board.king_square(colour), by=colour.other)
        if moves:
            return GameStatus.CHECK if in_check else GameStatus.IN_PROGRESS
        return GameStatus.CHECKMATE if in_check else GameStatus.STALEMATE

```

status is six lines and it defines checkmate and stalemate. That is the payoff of splitting legality into two layers, and it is what you want the interviewer to remember.

```

class Game:
    def __init__(self, board: Board, validator: MoveValidator) → None:
        self.board = board
        self.validator = validator
        self.history: list[Move] = []
        self.turn = Colour.WHITE
        self.status = GameStatus.IN_PROGRESS

    def make_move(self, origin: Square, target: Square,
                  promote_to: type[Piece] | None = None) → MoveResult:
        if self.status in (GameStatus.CHECKMATE, GameStatus.STALEMATE):
            return MoveResult.rejected("the game is over")

        legal = self.validator.legal_moves(self.board, self.turn)
        chosen = next(
            (m for m in legal
             if m.from_square == origin and m.to_square == target
              and m.promotion == promote_to),
            None,
        )
        if chosen is None:
            return MoveResult.rejected("illegal move")

        self.board.apply(chosen)
        chosen.piece.has_moved = True
        self.history.append(chosen)
        self.turn = self.turn.other
        self.status = self.validator.status(self.board, self.turn)
        return MoveResult.accepted(chosen, self.status)

```

“What I have not built, and where each would go.

Castling — a `King` candidate move guarded by four conditions: neither piece has moved, the squares between are empty, and the king is not in check on its start, path or destination square. The `has_moved` flag and `is_attacked` are already there, so it is one method on `King` plus a special case in `Board.apply` to move the rook as well.

En passant — needs the previous move, which is why history is on `Game`. `Pawn.candidate_moves` takes the last move and adds the diagonal capture when it was an adjacent double-step.

Draws — fifty-move rule and threefold repetition. Both are history queries, not board queries, and repetition needs a position hash, so I would add a Zobrist-style key on `Board`.

Performance — `legal_moves` currently applies and undoes every candidate, which is about thirty- five moves in a typical position. For a playable game that is fine. For an engine searching millions of positions, you would switch to bitboards and incremental attack maps, and that is a different program.”

6 The numbers

LLD numbers are lighter than high-level design numbers, and skipping them entirely is still a mistake. Three kinds always apply.

Objects in memory

```
chess position: 32 pieces × ~80 B + a 64-entry dict × ~100 B ≈ 9 KB
move history:   ~80 moves × 100 B                               ≈ 8 KB
-----
one live game                                     ≈ 17 KB
100,000 concurrent games = 1.7 GB
```

1.7 GB is the number that decides whether games live in process memory or in Redis. Compare with [snake and ladder](#) at 1.1 KB a game, where a hundred thousand games was 110 MB and the answer was different. Same question, different answer, because the state is fifteen times bigger.

Work per operation

```
legal_moves(): ~35 candidates in a typical position
  each one:    apply + is_attacked + undo
  is_attacked: ~8 sliding rays + 8 knight squares + pawns/king  = 40 checks
→ ~1,400 checks per legal_moves() call
→ tens of microseconds in Python
```

Fine for a human game at one move every ten seconds. **Hopeless for a search engine**, which is exactly the sentence to say: it identifies the limit of your own design before the interviewer does.

Concurrency

Every LLD prompt has a two-people-at-once question, and it is worth having the map:

two bids on one auction	→ conditional write on a version	(day 095)
two people booking one seat	→ a lock with a TTL, then payment	(day 086)
two mkdir of the same directory	→ create under the parent's lock	(day 093)
two players moving at once	→ a game is SERIAL: one writer	(day 094, today)
two threads in one cache	→ per-shard locks, not one global	(day 090)

The chess answer is the serial one. A game accepts one move at a time by definition, so the model is one writer per game, and a hundred thousand games are a hundred thousand independent streams with no contention between them. A version number on the game plus a move number for idempotency covers a client that retries.

7 The trade-offs

Polymorphic pieces, or an enum with a switch?

The enum version is fewer files and, for a fixed rule set, genuinely defensible — some real engines use it because a switch on an integer is faster than a virtual call.

Take polymorphism in an interview. The prompt is a design question, and the enum version puts the same `switch` in move generation, in attack detection, and in scoring — three places to edit for one new piece. **I would use the enum if this were a performance-critical engine**, and I would say so, because knowing *why* the ugly version exists is worth more than reflexively rejecting it.

Apply-check-undo, or copy the board?

Copying is simpler and cannot leave the board corrupted if something throws. It also allocates a whole position per candidate move — thirty-five copies per `legal_moves` call.

Take **apply-undo**, which is why `Move` carries the captured piece. Say the risk: an exception between apply and undo leaves the board wrong, so in production the pair belongs in a context manager.

Validate inside `Piece`, or in a separate validator?

Putting “does this leave my king in check” inside the piece means every piece needs to know about kings, checks and the whole board — and the same code is repeated six times.

Keep the two layers separate. *How a piece moves* is local and belongs to the piece. *Whether a move is legal here* is global and belongs to one validator. This split is the single best idea in the chess design and it generalises: **local rules on the entity, global rules in a service.**

Where this design breaks

- **Chess variants.** Chess960 randomises the back rank and changes castling completely; a variant with a new piece is one new subclass, which is the win, but a variant that changes the *board* — three players, a hexagonal board — breaks `Square` and every direction list.
- **An engine.** Object-per-piece and dictionary lookups are the wrong representation for searching millions of positions. Bitboards make the whole thing arithmetic on 64-bit integers. Different program, same rules.
- **Time controls.** A clock per player, with increments, is a separate concern that most candidates forget entirely. It is a `Clock` and two accumulated durations, and it changes `GameStatus` to include “lost on time”.

8 In the interview

How it gets asked

- The prompt is one sentence: “*Design a chess game.*” / “*Design a parking lot.*” / “*Design an elevator.*”
- The scope probe, at minute five: “*Would you support X?*” — they are testing whether you scope or panic.
- The extension, at minute thirty-five: “*Now add a new piece / a new pricing rule / a second currency.*” This is the real question and everything before it was setup.
- The self-criticism probe: “*What is wrong with this design?*”

What to say out loud, in the first ninety seconds

Whatever the prompt:

1. **Restate the scope in one sentence, including what you are excluding.** “I am designing the rules engine — board, pieces, legal moves, check and mate. Not net-working, not a UI, not an opponent.”
2. **Ask the three or four questions, with your assumed answers attached.** Never ask an open question and wait; ask it and answer it yourself so the round keeps moving.
3. **Announce the plan.** “I will list the classes, draw the relationships, then spend most of the time on the one part where the design is actually decided, and code that.”
4. **Name the interesting part early.** “For chess, the interesting part is that legality has two layers — how a piece moves, and whether that move leaves your own king attacked.”
5. **Say what you are deliberately deferring.** “Castling and en passant fit in places I will point at; I will implement promotion because it changes the board in an unusual way.”

The follow-ups

“**How do you handle check and checkmate?**” “I do not write them as rules, which is the point. Legality is two layers: each piece produces its candidate moves, and then the validator filters out anything that leaves my own king attacked — by applying the move, asking whether the king square is attacked, and undoing it. That one filter gives me pins, discovered check and ‘you must get out of check’ for free. Then **checkmate is in_check and no legal moves**, and **stalemate is not in_check and no legal moves** — two lines. If I had written checkmate as its own rule I would have three overlapping rule sets that can disagree.”

“**Now add a new piece — one that moves like a knight and a bishop.**” “One new subclass. It implements `candidate_moves` by returning the knight jumps plus the bishop slides, and I register it in the piece factory. **Nothing else changes** — not the validator, not the board, not check detection, because all of those work in terms of `Piece`. That is the whole argument for polymorphic pieces over a type enum with a switch: with the enum, adding a piece means editing move generation, attack detection and any scoring, and every edit risks the existing pieces.”

“**Why is `MoveValidator` separate from `Board` and from `Piece`?**” “Because there are two genuinely different kinds of rule and they have different scopes. *How a bishop moves* is local — it needs the bishop and the board, and it is the same in every position, so it belongs on the bishop and is trivially testable on its own. *Whether this move is legal right now* is global — it depends on the position of a piece that may be at the other end of the board — and it is identical for all six piece types, so writing it once in a validator is the difference between one implementation and six. The general form of that rule is: **local rules go on the entity, global rules go in a service.**”

“What is wrong with this design?” “Three things I would name before you do. First, `legal_moves` applies and undoes every candidate move, which is about fourteen hundred board checks per call — fine for a human game, hopeless for an engine searching millions of positions, where you would move to bitboards. Second, apply-undo without a context manager means an exception between the two leaves the board corrupted; the pair should be wrapped. Third, `Board.is_attacked` recomputes attacks from scratch every time, where a real implementation keeps incremental attack maps. All three are the same trade: I optimised for clarity and for a design conversation, and I would change all three if this had to search.”

“You have five minutes left and you have not done castling.” “Then I will describe it rather than write it, which is the right call at five minutes. Castling is a king move with four guards: neither the king nor that rook has ever moved — which is why `Piece` carries `has_moved`; the squares between them are empty; and the king is not attacked on its current square, the square it passes through, or its destination — which uses `is_attacked`, already written. The only special case in `Board.apply` is that it moves two pieces instead of one, so `Move` needs the `is_castle` flag it already has. Everything it needs exists; it is one method on `King` and one branch in `apply`.”

A model answer

Asked, at minute five: *so what are you building?*

“I am building the chess rules engine: a board, pieces that know how they move, legal move generation, check, checkmate and stalemate, and a game that enforces turn order and keeps history. I am not building networking, a user interface, or an opponent that chooses moves — although I will put move *choice* behind an interface so a human and a program are two implementations of the same thing.

History is part of the state rather than a feature, and I want to say why now rather than discover it later: castling depends on whether the king and rook have ever moved, en passant depends on the immediately previous move, and threefold repetition depends on the entire game. Any design that treats history as an add-on has to be rebuilt when those come up.

The class list is `Square`, `Piece` — abstract, with a subclass per kind — `Board`, `Move`, `MoveValidator`, and `Game`. I will add a `SlidingPiece` layer between `Piece` and rook, bishop and queen, because those three differ only in a list of directions; sliding until you hit the edge, an enemy or a friend is one method written once.

The part I want to spend the most time on is this: **legality has two layers, and conflating them is what makes chess designs collapse**. How a knight moves is local — an L shape — and belongs on the knight. But a pinned knight may not move at all, and nothing about knights can tell you that; it depends on a piece somewhere else entirely. So `Piece.candidate_moves` answers ‘how does this piece move’, and `MoveValidator` answers ‘what is allowed in this position’ by taking each candidate, applying it, asking whether my own king is now attacked, and undoing it. That is apply-check-undo, the same pattern as backtracking, and it is why `Move` has to store the captured piece — otherwise undo cannot put it back.

The payoff is that three of the hardest rules are never written. Pins, discovered check, and ‘you must get out of check’ all fall out of that one filter. And then **checkmate is ‘in check and no legal moves’, stalemate is ‘not in check and no legal moves’** — six lines for the two concepts that beginners find hardest.

I will code `SlidingPiece`, the validator, and `Game.make_move`, and at the end I will tell you exactly where castling, en passant and the draw rules attach, so you can see the design has room for them.”

9 Recall card

- Five moves, always, in this order: clarify (0–5) · nouns (5–10) · class diagram (10–20) · the interesting part (20–30) · code (30–40). Moves 1–3 are table stakes. You are marked on move 4. Find it by asking: *which sentence in the prompt would a product manager change next year?* Put an interface there and show **two** implementations.
- Fifteen of eighteen case studies had a strategy at their centre — `EvictionPolicy`, `PricingStrategy`, `SplitStrategy`, `Channel`, `Dice`, `SpotAllocationStrategy`. The tells: “depending on” → **strategy**; “and then it becomes” → **state machine**; “notify” → **observer**; “contains other” → **composite**.
- Randomness, time and I/O must be replaceable (`Dice`, `Clock`, `Channel`) or nothing is testable. Return a result object, never print — one engine then serves a terminal, a phone and a simulation. Local rules on the entity, global rules in a service.
- Chess: legality has two layers. `Piece.candidate_moves` is local and per subclass; `MoveValidator` applies each candidate, asks whether your own king is attacked, and undoes it — which is why `Move` must carry the captured piece. That one filter gives pins, discovered check and escaping check for free, and then `checkmate = in_check and not legal_moves`, `stalemate = not in_check and not legal_moves`.
- The five ways to lose: coding at minute three; silence; anaemic classes plus a God service; an enum with a switch where polymorphism belongs; and chasing completeness instead of naming what you left out. Nobody finishes — at minute 38, say what is missing and where each piece would attach.

Practice — Sudoku, word search, and grid backtracking

DSA topic: Sudoku, word search, and grid backtracking **System design topic:** Low-level design revision and full mock

Code these, in this order

One rule for the whole set: **say “path” or “region” out loud before writing a single line.** Path clears the marker on the way out; region never does. That one word decides whether your solution is exponential or linear, and getting it backwards produces a wrong answer rather than a crash.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Number of Islands	LeetCode 200 (Medium)	The region version: mark and never restore. Do this first, as the contrast.
2	Word Search	LeetCode 79 (Medium)	The path version: mark, explore, restore — and bounds before the match.
3	Flood Fill	LeetCode 733 (Easy)	Region again, with the trap that the new colour may equal the old one.
4	Sudoku Solver	LeetCode 37 (Hard)	Three sets, pre-computed blanks, and the most-constrained cell.

On problem 1, break the marker deliberately

Write it so the cell is restored after the four recursive calls. Run it on a 2×2 grid of land and record what happens. Then say why the same island gets counted more than once.

On problem 2, remove the bounds check

Run `["AB", "CD"]` looking for `"ADA"` with and without the bounds check. Record both answers, and say why the version without it raises nothing at all.

On problem 3, run it where the new colour equals the old

Start at a cell whose colour already equals the target colour. Say what happens and what one line fixes it. This is a region problem whose marker is the colour itself.

On problem 4, time the heuristic

Solve an easy puzzle and a deliberately hard one, in-order and most-constrained. Record four numbers. Say when the heuristic earns its keep and when it costs more than it saves.

The classify drill

1. Say the one-sentence test for path versus region.
2. Put these into the two buckets: word search, islands, rat in a maze, flood fill, all paths, largest connected region.
3. Say what happens to a path problem if you do not clear the marker.
4. Say what happens to a region problem if you do clear it, including the error.
5. State the complexity of each kind and say which line causes the difference.

The bounds drill

1. Write the direction tuple from memory.
2. Say why the bounds check must be first, and what Python does without it.
3. Give the input that finds a word that is not there.
4. Say what changes for an eight-direction problem.

The marking drill

1. Write the mark-and-restore in four lines.
2. Say what the sentinel character assumes about the input.
3. Say what you would use instead if that assumption is not safe.
4. Write it with `any(...)` and say why that is safer than a loop with an early return.
5. Say what the in-place marking costs the caller.

The prune drill

1. Write the character-count check.
2. Write the reverse-the-word check and say what quantity it reduces.
3. Construct a grid and word where reversing gives a $36\times$ improvement.
4. Say what both prunes cost, and why that cost is always worth paying.
5. Say what you would do differently if you had to find a hundred words instead of one.

The cost drill

1. State the complexity of word search and explain where the 3 comes from.
2. Compute the worst-case path count on a 6×6 grid for $L = 4, 8, 12$ and 16 .
3. State the space, and say what dominates it in each of the two kinds of problem.
4. Say which one can raise `RecursionError`, on what input, and what you would do instead.

The Sudoku drill

1. Name the three sets and write the box index expression.
2. Say why the blanks are pre-computed and what it saves.
3. Describe the most-constrained-cell rule in one sentence.
4. Say why it costs a scan per node and why that is still a win.
5. Count the changes and the restores in the inner loop.

The break-it drill

Trigger each and record the exact output or error:

1. Word search with the restore removed.
 2. Islands with a restore added.
 3. Word search without bounds, on a two-row grid.
 4. `board[:]` used as a copy of a grid.
 5. Flood fill where the new colour equals the old.
 6. Sudoku scanning for blanks from `(0,0)` every call, timed against the pre-computed version.
-

The five-moves drill

1. Name the five moves and the minute each one ends.
2. Say what you say out loud at minutes 5, 10, 20, 30 and 38.
3. Say which move you are actually being marked on.
4. Say the question that finds the interesting part.

The pattern-recall drill

For each prompt, name the interesting part and the interface that goes there, without looking:

1. Parking lot. 2. Elevator. 3. ATM. 4. Vending machine. 5. Splitwise. 6. BookMyShow.
2. Ride-hailing. 8. Rate limiter. 9. Cache. 10. Logging. 11. Notifications. 12. File system.
3. Snake and ladder. 14. Auction.

Then say how many of the fourteen are a strategy, and what that tells you to reach for first.

The tells drill

1. Give the four phrases and the pattern each one names.
2. Give an example prompt for each.

3. Name the two structural rules that came up in nearly every case study.
4. Say what each of the five losing mistakes looks like in a transcript.

The chess drill

1. State the four clarifying questions, with your assumed answers.
2. Say which of the four is the one that impresses, and why.
3. Name the classes and say which one is abstract.
4. Say what `SlidingPiece` exists for and which three pieces use it.
5. State the two layers of legality and give the example that proves they are different.
6. Write the apply-check-undo loop and say why `Move` must carry the captured piece.
7. Define checkmate and stalemate in one line each.
8. Say where castling attaches and name its four guards.

The self-criticism drill

1. Name three things wrong with the chess design before anyone asks.
2. For each, say what you optimised for instead.
3. Say what you would change if this had to search millions of positions.
4. Say what you would say at minute 38 if castling were still unwritten.

The numbers drill

Compute each, showing the multiplication:

1. Memory for one chess game, and for 100,000 concurrent games.
2. The same for snake and ladder, and say why the two answers lead to different architectures.
3. Board checks per `legal_moves()` call.
4. Say what that figure means for a human game and for an engine.

The concurrency-map drill

For each, say the mechanism from memory:

1. Two bids on one auction.
2. Two people booking the same seat.
3. Two `mkdir` calls for the same directory.
4. Two players acting in one game.
5. Two threads writing to one cache.

Say which of the five is the “inherently serial” case and what that buys you at scale.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Solve the sudoku board. Now find the word in the grid.* Path or region named first, the three sets and the box index for Sudoku, bounds before match for the grid, mark-and-restore with the reason, the complexity with the `3` explained, and the two one-line prunes.
 2. *Design a chess game. Forty minutes. Begin.* Scope and exclusions in one sentence, the history question asked at minute two, the class list with `SlidingPiece` justified, the two layers of legality with the pinned-knight example, apply-check-undo with the reason `Move` stores the capture, and checkmate and stalemate in two lines.
 3. *What is the same in every low-level design prompt?* The five moves with their minutes, the question that finds the interesting part, the fact that most prompts resolve to a strategy, the four tells, and the two structural rules about replaceable dependencies and returned results.
-

Before you move on

- ☐ I say “path” or “region” before writing any grid code.
- ☐ I can state what happens if I get that backwards, in both directions.
- ☐ I write the bounds check first, and know why its absence raises nothing.
- ☐ I can write mark-and-restore with `any(...)` and say why not a loop with an early return.
- ☐ I can name the assumption behind the sentinel character.
- ☐ I can write both one-line prunes and say what each one reduces.
- ☐ I can explain where the `3` in the complexity comes from.
- ☐ I know which kind of grid problem raises `RecursionError`, and on what input.
- ☐ I can write the Sudoku box index and describe the most-constrained rule.
- ☐ I can name the five moves and their minutes.
- ☐ I can name the interesting part for at least ten of the fourteen prompts.
- ☐ I can give the four tells and the pattern each names.
- ☐ I can state the two layers of chess legality with the pinned-knight example.
- ☐ I can define checkmate and stalemate in one line each.
- ☐ I can say why `Move` must carry the captured piece.
- ☐ I can name three faults in my own chess design.

- [] I answered all three questions above out loud.

Recursion and backtracking revision and mock round

DATA STRUCTURES AND ALGORITHMS

Recursion and backtracking revision and mock round

You can solve two unseen backtracking problems cold.

SYSTEM DESIGN

What scale actually means, in numbers

You stop saying millions of users and start saying requests per second.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two problems, no hints, talk as you go.*
- *This app has ten million users. What is the read QPS?*

Recursion and backtracking revision and mock round

1 What this is, and why they ask it

Eleven days ago you had not written a recursive function. Since then: [the leap of faith](#), [the call stack](#), [termination](#), [arrays and strings](#), [subsets](#), [permutations](#), [combinations](#), [the undo step](#), [N-Queens](#) and [grid search](#).

Three sentences. Every one of those problems is **one of four templates**, and the whole skill is recognising which one in the first thirty seconds. The recognition is not about the words in the problem; it is about **two questions** — does order matter, and may things be reused — whose four answers are the four templates. And once you have picked, the code is eight lines you have already written five times.

They run a revision round like this because in a real interview nobody tells you it is a backtracking problem. The prompt says *“find all the ways to...”* or *“is there a sequence such that...”*, and your first minute decides the next thirty. Today is that first minute, practised until it is automatic.

2 The story

The bicycle repair stand was two planks and an umbrella at the corner where the school road meets the main road, and Salim had been standing there for nineteen years.

What he did that nobody else on that road did was listen first.

A boy would arrive pushing a bicycle and start explaining. Salim would take it from him, hold the handlebars, and lift the back wheel about an inch off the ground with his foot on the stand. Then he would turn the pedal with his hand, once, slowly, and listen.

That was usually enough. He would say chain, or bearing, or brake block, and be right.

His nephew Faisal came to work with him one summer and could not do it. He would take a bicycle and open things — the chain, the brake, the wheel nuts — one after another until he found the problem. Some days he found it in two minutes and some days he took forty and had a bicycle in fifteen pieces on the ground.

Salim watched this for about a week and then told him what he was actually doing, which turned out to be very small.

He said, before you touch anything, ask two things. Does it make a noise when it is moving, or only when you press something. And does the noise happen every turn of the wheel, or only sometimes.

That is it. Two questions, four answers. Noise all the time and every turn — the chain or a bearing. Noise all the time but not every turn — something bent, and the wheel is the first place to look. Only when you press — a brake. Only sometimes and only under load — the gears.

Faisal said that seemed too simple to be the whole thing.

Salim said it was not the whole thing, it was the first thirty seconds. After the first thirty seconds you still have to do the work, and the work is the same work whichever it is — open it, look at it, fix it, put it back. But if you get the first thirty seconds wrong you spend the morning taking apart the wrong half of the bicycle, and the boy is late for school.

By August Faisal was doing it too. Not as fast, but he had stopped opening things at random.

3 The idea in plain English

Salim’s two questions are the whole revision. Every problem in this phase is one of four templates, and two questions choose between them.

The two questions

Q1: Does ORDER matter?
yes → [1,2] and [2,1] are different answers → the used-array tree
no → [1,2] and [2,1] are the same answer → the start-index tree

Q2: May an element be used MORE THAN ONCE?
yes → recurse on i
no → recurse on i + 1 (or mark it used)

Four combinations, four templates:

ORDER MATTERS?	REUSE ALLOWED?	TEMPLATE	EXAMPLE
No	No	start index, recurse on i + 1	subsets, combinations, Combination Sum II
No	Yes	start index, recurse on i	Combination Sum
Yes	No	used array, loop over all n	permutations
Yes	Yes	no bookkeeping at all — every choice always available	letter combinations of a phone number, generate parentheses

That table is the phase. Everything else — pruning, duplicate handling, grids — sits on top of one of those four.

The one template, written once

```
def explore(state):
    if is_complete(state):
        record(copy_of(state))      # COPY, always
        return
    for choice in options(state):
        if not is_valid(choice, state):
            continue                # PRUNE, before descending
        apply(choice, state)        # choose
        explore(state)              # recurse
        undo(choice, state)         # un-choose
```

The four templates differ only in `options`. Everything else is identical.

The five rules that survive from the whole phase

1. **Append a copy.** `current[:]`. `current` is one object mutated throughout; appending it stores a reference, and every reference points at the same list, empty by the end.
2. **Count the chooses and the undoes.** Two chooses need two undoes; four need four. Undo in reverse. A missing undo is **always silent**.
3. **Prune before descending, never at the leaf.** N-Queens at $n = 8$: 2,057 nodes versus 19,173,961.
4. **Anything passed as an argument needs no undo.** Move state into the parameter list wherever you can; it is state you cannot forget to restore.
5. **Say the output size first.** 2^n , $n!$, $C(n, k)$ — and that no algorithm can beat it, because the output is that big. It removes “can you do better?” from the conversation.

The duplicate rule, in its three outfits

The same idea appeared three times and interviewers use the difference to check whether you understood it or memorised it.

```
start-index tree (subsets, combination sum II):
    sort, then: if i > start and items[i] == items[i-1]: continue

used-array tree (permutations):
    sort, then: if i > 0 and items[i] == items[i-1] and not used[i-1]: continue

swap-based tree (in-place permutations):
    a per-level set of values already tried
```

All three mean the same sentence: *do not start two branches with the same value at the same level*. The bookkeeping differs because the tree shape differs. If you can say that sentence and then derive the right condition, you never have to remember which is which.

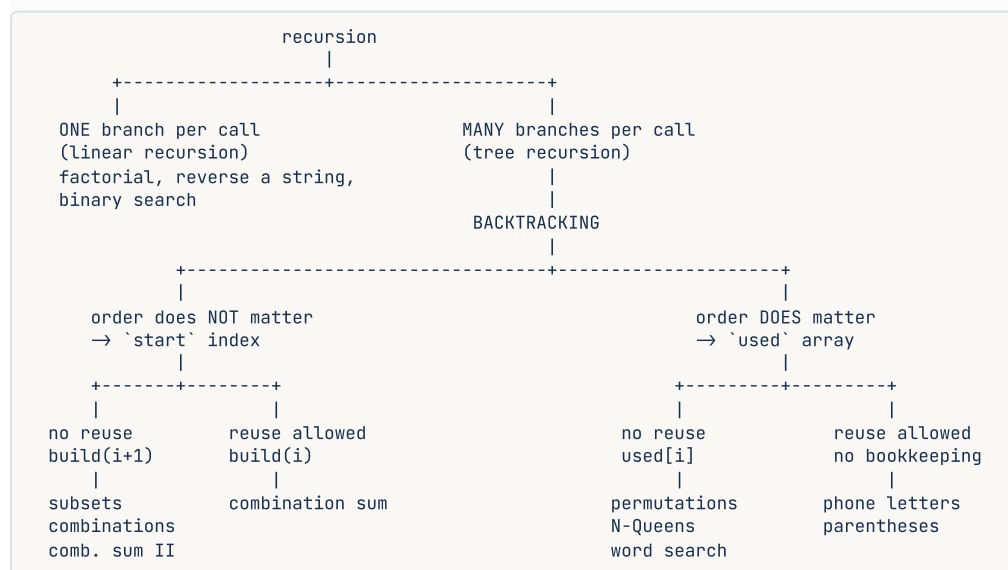
How to recognise a backtracking problem at all

The prompt almost never says “backtrack”. It says one of these:

- “**Find all...**”, “**return every...**”, “**list the ways...**” — enumeration. The output is exponential and they know it.
- “**Is there a ... such that**” — a search with an early return.
- “**Count the number of ways...**” — careful. If you only need the count and the same sub-state is reachable by many paths, that is **dynamic programming**, not this.
- A **tiny constraint**: `n ≤ 8` means factorial, `n ≤ 20` means 2^n , `target ≤ 500` with small candidates means combination sum. **The bound is the hint**, and reading it is free.

4 The picture

The family tree of the phase.



The four templates, side by side, with only the differences shown:

SUBSETS / COMBINATIONS

```
for i in range(start, n):  
    choose(items[i])  
    build(i + 1)      ← i+1  
    unchoose()
```

PERMUTATIONS

```
for i in range(n):  
    if used[i]: continue  
    used[i] = True  
    choose(items[i])  
    build()  
    unchoose()  
    used[i] = False
```

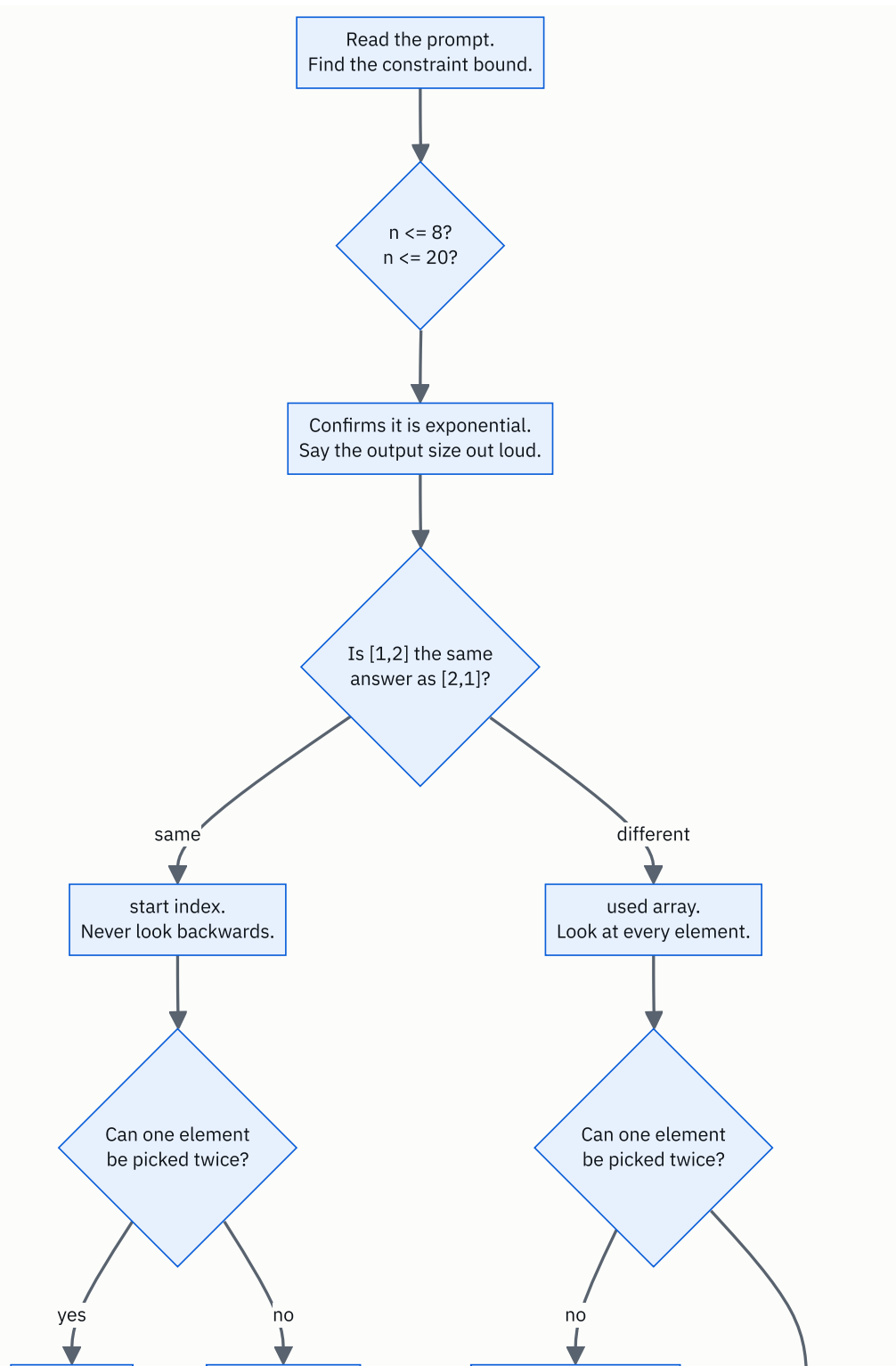
COMBINATION SUM

```
for i in range(start, n):  
    choose(items[i])  
    build(i)      ← i  
    unchoose()
```

PHONE LETTERS / PARENTHESES

```
for ch in options_at_this_level:  
    choose(ch)  
    build(depth + 1)  
    unchoose()  
(nothing is consumed, so nothing  
is tracked)
```

The decision, as a thirty-second procedure:



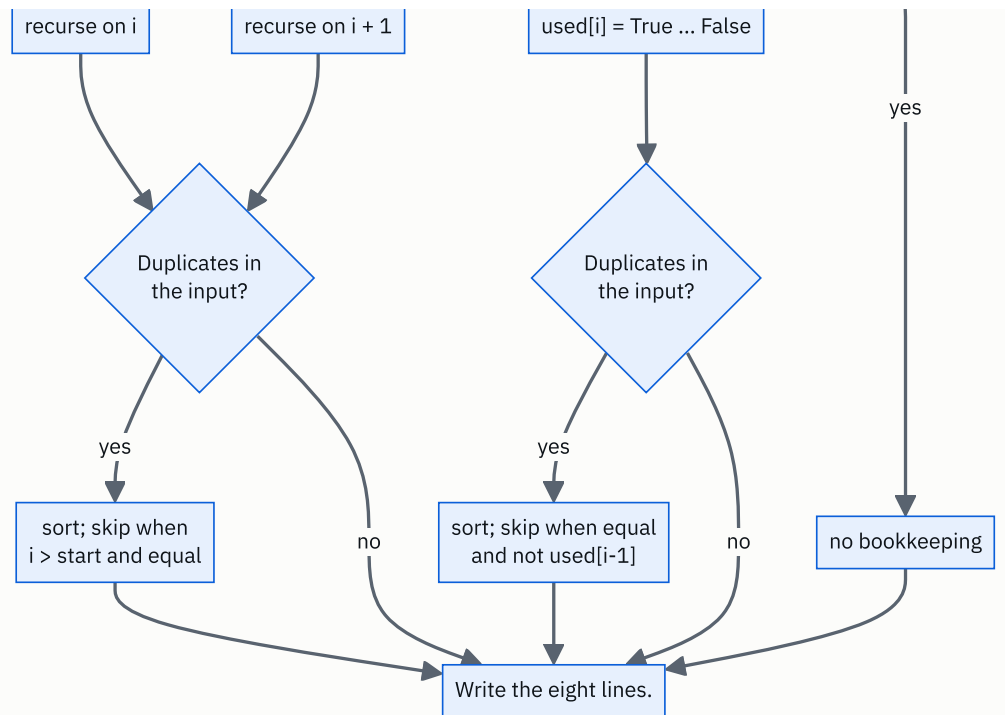


FIGURE 97.1

5 The code, built step by step

Step 1 — the four templates, from memory

Write these four before the mock. If any of them takes more than a minute, that is the one to practise.

Step 2 — mock problem one: Palindrome Partitioning

“Given a string, return every way of cutting it into pieces where every piece is a palindrome. For “aab”, the answer is `[["a", "a", "b"], ["aa", "b"]]`.”

Think out loud, in this order:

“Order does not matter in the sense that matters here — I am cutting left to right, so each choice is ‘where does the next cut go’, and I never go backwards. That is the `start` -index shape. Nothing is reused, because each character is consumed exactly once, so I move `start` forward past whatever I took.

The choices at each level are the possible end points of the next piece: `start+1` up to `n`. The validity check is ‘is `s[start:end]` a palindrome’, and it goes **before** the recursive call, so I never descend into a cut that is already invalid.

Size: at worst every character is its own palindrome, so a string of `n` characters has $2^{(n-1)}$ cutting points — for “aaaa” that is 8 partitions, all valid. So this is exponential and `n` will be small; the constraint says 16, which confirms it.”

```
def partition(s: str) → list[list[str]]:
    """Every partition of s into palindromic pieces.

    start-index tree, no reuse: each character is consumed once.
    The palindrome check is the PRUNE and it goes before the recursion.
    """
    result: list[list[str]] = []
    current: list[str] = []

    def build(start: int) → None:
        if start == len(s):
            result.append(current[:])          # COPY
            return
        for end in range(start + 1, len(s) + 1):
            piece = s[start:end]
            if piece != piece[::-1]:
                continue                       # PRUNE before descending
            current.append(piece)              # choose
            build(end)                         # recurse: past what we took
            current.pop()                      # un-choose

    build(0)
    return result
```

The follow-up is guaranteed: “the palindrome check is $O(n)$ inside a loop — can you avoid it?” Yes: pre-compute a table of which substrings are palindromes, in $O(n^2)$, then each check is one lookup.

```

def partition_with_table(s: str) → list[list[str]]:
    """The same, with an  $O(n^2)$  palindrome table so each check is  $O(1)$ .

    is_pal[i][j] means s[i:j+1] is a palindrome. Built from short pieces
    outward, so is_pal[i+1][j-1] is always already known.
    """
    n = len(s)
    is_pal = [[False] * n for _ in range(n)]
    for length in range(1, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            is_pal[i][j] = s[i] == s[j] and (length < 3 or is_pal[i + 1][j - 1])

    result: list[list[str]] = []
    current: list[str] = []

    def build(start: int) → None:
        if start == n:
            result.append(current[:])
            return
        for end in range(start, n):
            if not is_pal[start][end]:
                continue
            current.append(s[start:end + 1])
            build(end + 1)
            current.pop()

    build(0)
    return result

```

Step 3 — mock problem two: Restore IP Addresses

“Given a string of digits, return every valid IP address you can make by inserting three dots. Each of the four parts must be between 0 and 255, and must not have a leading zero unless it is exactly “0”.”

“This is the same `start`-index shape — I am cutting left to right — with two extra constraints: there must be exactly four pieces, and each piece has its own validity rule. So the state is `start` and the number of pieces so far, and there are two prunes rather than one.

The prunes are worth stating separately, because one of them is easy to miss. Obviously a piece must be a number from 0 to 255 with no leading zero. Less obviously, **if the characters remaining cannot possibly fill the remaining parts, stop** — with `k` parts left, I need between `k` and `4k` characters. That second check is what stops a 12-character string wasting the whole tree.

The size is tiny — at most three positions for each of three dots, so at most $C(11, 3) = 165$ candidates. That is worth saying, because it means the answer is ‘this is not really exponential at all’.”

```

def restore_ip_addresses(digits: str) → list[str]:
    """LeetCode 93. Insert three dots to make four valid parts.

    start-index tree with a fixed piece count. Two prunes:
    1. each part is 0..255 with no leading zero
    2. the characters left must be able to fill the parts left
    """
    n = len(digits)
    if not 4 ≤ n ≤ 12:
        return [] # cannot possibly work

    result: list[str] = []
    parts: list[str] = []

    def build(start: int) → None:
        if len(parts) == 4:
            if start == n: # used every character
                result.append(".".join(parts))
            return

        remaining_parts = 4 - len(parts)
        remaining_chars = n - start
        if not remaining_parts ≤ remaining_chars ≤ remaining_parts * 3:
            return # PRUNE: cannot fit

        for length in (1, 2, 3):
            if start + length > n:
                break
            piece = digits[start:start + length]
            if piece[0] == "0" and length > 1:
                break # leading zero: and longer is worse
            if int(piece) > 255:
                break # sorted by length, so stop
            parts.append(piece) # choose
            build(start + length) # recurse
            parts.pop() # un-choose

    build(0)
    return result

```

Two **break**s, and both are justified by monotonicity: a longer piece starting with 0 is also invalid, and a longer piece is a larger number. That is the same “sorted plus monotone means **break**” rule from [day 093](#).

The complete revision file

All four templates in one place, to be written from memory.

```

# ----- TEMPLATE 1: order does NOT matter, no reuse -----
def template_subsets(items: list[int]) → list[list[int]]:
    """Subsets, combinations, Combination Sum II, palindrome partitioning."""
    result, current = [], []

    def build(start: int) → None:
        result.append(current[:])          # or: if complete → record
        for i in range(start, len(items)):
            current.append(items[i])
            build(i + 1)                   # i + 1: consumed
            current.pop()

    build(0)
    return result

# ----- TEMPLATE 2: order does NOT matter, reuse allowed -----
def template_combination_sum(candidates: list[int], target: int) → list[list[int]]:
    """Combination Sum. The ONLY difference from template 1 is `i` vs `i + 1`."""
    candidates = sorted(candidates)
    result, current = [], []

    def build(start: int, remaining: int) → None:
        if remaining == 0:
            result.append(current[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remaining:
                break                    # sorted → break, not continue
            current.append(candidates[i])
            build(i, remaining - candidates[i]) # i: may reuse
            current.pop()

    build(0, target)
    return result

# ----- TEMPLATE 3: order DOES matter, no reuse -----
def template_permutations(items: list[int]) → list[list[int]]:
    """Permutations, N-Queens, word search. TWO chooses, TWO un-chooses."""
    result, current = [], []
    used = [False] * len(items)

    def build() → None:
        if len(current) == len(items):
            result.append(current[:])
            return
        for i in range(len(items)):
            if used[i]:
                continue
            used[i] = True
            current.append(items[i])
            build()
            current.pop()
            used[i] = False              # the one people forget

    build()
    return result

# ----- TEMPLATE 4: order matters, everything always available -----
def template_free_choice(digits: str) → list[str]:
    """Letter combinations of a phone number. Nothing is consumed, so there
    is no `start`, no `used`, and nothing to restore beyond the prefix."""
    keypad = {"2": "abc", "3": "def", "4": "ghi", "5": "jkl",
              "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"}

    if not digits:
        return []
    result, current = [], []

    def build(depth: int) → None:

```

```

        if depth == len(digits):
            result.append("".join(current))
            return
        for letter in keypad[digits[depth]]:
            current.append(letter)
            build(depth + 1)
            current.pop()

    build(0)
    return result

# ----- the duplicate rule, in both outfits -----
def dedupe_start_index(items: list[int]) → list[list[int]]:
    """`i > start`: another FIRST choice at this level."""
    items = sorted(items)
    result, current = [], []

    def build(start: int) → None:
        result.append(current[:])
        for i in range(start, len(items)):
            if i > start and items[i] == items[i - 1]:
                continue
            current.append(items[i])
            build(i + 1)
            current.pop()

    build(0)
    return result

def dedupe_used_array(items: list[int]) → list[list[int]]:
    """`not used[i-1]`: my identical twin has not been placed yet."""
    items = sorted(items)
    result, current = [], []
    used = [False] * len(items)

    def build() → None:
        if len(current) == len(items):
            result.append(current[:])
            return
        for i in range(len(items)):
            if used[i]:
                continue
            if i > 0 and items[i] == items[i - 1] and not used[i - 1]:
                continue
            used[i] = True
            current.append(items[i])
            build()
            current.pop()
            used[i] = False

    build()
    return result

if __name__ == "__main__":
    print(len(template_subsets([1, 2, 3]))) # 8
    print(template_combination_sum([2, 3, 6, 7], 7)) # [[2,2,3],[7]]
    print(len(template_permutations([1, 2, 3]))) # 6
    print(template_free_choice("23"))
    # ['ad', 'ae', 'af', 'bd', 'be', 'bf', 'cd', 'ce', 'cf']

    print(partition("aab")) # [['a', 'a', 'b'], ['aa', 'b']]
    print(restore_ip_addresses("25525511135"))
    # ['255.255.11.135', '255.255.111.35']

    print(dedupe_start_index([1, 2, 2]))
    # [[], [1], [1,2], [1,2,2], [2], [2,2]]
    print(dedupe_used_array([1, 1, 2]))
    # [[1,1,2],[1,2,1],[2,1,1]]

```

6 What it costs

The whole phase in one table. Learn it as a table; interviewers ask for these directly.

problem	answers	time	extra space
subsets	2^n	$O(n \cdot 2^n)$	$O(n)$
subsets with dupes	$\leq 2^n$	$O(n \cdot 2^n)$	$O(n)$
permutations	$n!$	$O(n \cdot n!)$	$O(n)$
combinations $C(n,k)$	$C(n,k)$	$O(k \cdot C(n,k))$	$O(k)$
combination sum	varies	$O(n^{\text{target}/\text{min}})$	$O(\text{target}/\text{min})$
palindrome partition	$\leq 2^{(n-1)}$	$O(n \cdot 2^n)$	$O(n)$
N-Queens	see below	$O(n!)$ loose	$O(n)$
word search	n/a (boolean)	$O(\text{rows} \cdot \text{cols} \cdot 3^L)$	$O(L)$
phone letters	4^n worst	$O(n \cdot 4^n)$	$O(n)$

Extra space is $O(\text{depth})$ in every single row. That is the sentence to have ready: time is the whole tree, space is the deepest path.

The growth, so the constraint bounds make sense

n	2^n	$n!$	$C(n, n/2)$
8	256	40,320	70
10	1,024	3,628,800	252
15	32,768	1,307,674,368,000	6,435
20	1,048,576	2.4×10^{18}	184,756
25	33,554,432	1.6×10^{25}	5,200,300

The reading rule: $n \leq 8$ means they expect $n!$. $n \leq 20$ means they expect 2^n . $n \leq 25$ means 2^n and they expect it to be tight. Anything larger is not this phase.

The prunes, measured

Two numbers worth carrying into the room:

N-Queens $n = 8$	check while descending	2,057 nodes	
	check at the leaf	19,173,961 nodes	9,321×
combinations(20, 18)	with the size prune	191 calls	
	without	~220,000 calls	1,150×

Where the stack actually matters

subsets $n = 20$	~2,000,000 calls,	20 frames	
permutations $n = 10$	~9,900,000 calls,	10 frames	
N-Queens $n = 12$	~856,000 calls,	12 frames	
combination sum, target 5000, min candidate 1:	5,000 frames	→ RecursionError	

Only one problem in the entire phase can overflow the stack, and it is combination sum, because its depth is driven by the target rather than by `n`. Everything else has depth `n` and `n` is small by construction.

7 The traps

The eight that account for almost every wrong answer in this phase.

Trap 1 — appending the list instead of a copy

```
subsets([1,2,3]) with result.append(current) → [[], [], [], [], [], [], [], []]
```

Right count, all empty, all the same object, no error. `current[:]`.

Trap 2 — an unmatched undo

```
permutations([1,2,3]) without used[i] = False → [[1, 2, 3]]
```

One answer instead of six. **Count the chooses, count the undoes.**

Trap 3 — `i` where `i + 1` belongs, or the reverse

```
combination_sum([2,3,6,7], 7) with i + 1 → [[7]]           instead of [[2,2,3],[7]]  
combination_sum_ii([1,1,2], 2) with i   → duplicates, and a 1 used twice
```

One character, no error either way. Say “reuse is allowed” or “each element once” out loud before typing the call.

Trap 4 — the wrong duplicate condition for the tree

```
permutations_unique([1,1]) with `i > start`-style condition → []  
subsets_with_dupes([2,2]) with `i > 0`                      → loses [2,2]
```

`i > start` for the start-index tree; `not used[i-1]` for the used-array tree.

Trap 5 — `break` and `continue` swapped

```
combination_sum_ii: break on the duplicate → 2 answers instead of 4  
combination_sum:    continue on too-big   → correct, and much slower
```

Sorted plus monotone means `break`. Anything else means `continue`. One swap gives wrong answers, the other gives silence and a timeout.

Trap 6 — checking validity at the leaf

Correct answer, catastrophic cost. N-Queens $n = 8$ is nineteen million nodes instead of two thousand, and $n = 10$ does not finish. **The check goes before the descent.**

Trap 7 — forgetting the sort

Every duplicate rule and every “too big, so break” prune compares **adjacent** elements. Without the sort, both are silently wrong: duplicates come out anyway, and the `break` cuts off valid candidates.

Trap 8 — enumerating when they asked how many

```
len(combination_sum([1,2,5], 500))    ~50,000 lists built,    seconds  
dp count                             1,500 steps,              instant
```

“How many ways” is never backtracking when the sub-state repeats. Hearing the word “count” and switching approach is the cheapest point in the phase.

Two real errors to be able to quote:

```
RecursionError: maximum recursion depth exceeded  
TypeError: unhashable type: 'list'
```

The first comes from combination sum with a large target, or a region-style grid search on a big grid. The second comes from trying to de-duplicate a list of lists with a `set`.

8 In the interview

How it gets asked

The prompt never says “backtracking”. It says:

- “Return all possible ...” / “Find every way to ...” — enumeration, and the size is exponential.
- “Is there a ... such that ...” — search with an early return.
- “Partition this string so that ...” — the `start`-index tree with a validity check.
- “Place these items so that no two ...” — constraint search with three sets.
- “How many ways are there to ...” — probably **not** this phase. Check whether the sub-state repeats.

What to say out loud, in the first ninety seconds

The same six things, whatever the problem.

1. **Read the constraint out loud.** “ $n \leq 14$, so an exponential answer is expected. That is the first thing I want to confirm.”
2. **State the output size.** “There are at most $2^n / n! / C(n, k)$ answers, so nothing can be faster than that — the output alone is that big.”
3. **Answer the two questions.** “Does order matter here? ... so it is the `start`-index tree. Can an element be reused? ... so I recurse on `i + 1`.”
4. **Name the pattern.** “Choose, recurse, un-choose. I will append a copy at the base case, and I am changing two things before the call, so there are two lines after it.”
5. **Point at the prune.** “The validity check goes before the recursive call so the branch is never walked — that is what makes this backtracking rather than brute force.”
6. **Give both complexities separately.** “Time is the whole tree; extra space is the deepest path, which is $O(n)$.”

The follow-ups

“Can you do better than exponential?” “Not for this question as asked, because the **output** is exponential — I have to write down every answer, and there are 2^n of them. What I can do is not walk branches that cannot produce an answer, which is what the pruning does, and that is often the difference between milliseconds and not finishing. If you change the question to ‘how many’ or ‘the best one’, that is a completely different answer: counting is usually a dynamic programming array, and ‘the best one’ is either DP or greedy. So the honest answer is: no for enumeration, and yes if the question is really something else.”

“Your solution is too slow. What would you do?” “Three things in order, cheapest first. **One: prune earlier.** Move any validity check from the leaf to before the descent, and add cheap impossible-to-succeed checks — ‘not enough elements left’, ‘the remaining characters cannot fill the remaining parts’, ‘the grid does not contain enough of this letter’. Those are one line each and they cut whole subtrees. **Two: order the choices** so that the most constrained decision is made first — the most-constrained-cell rule for Sudoku, or reversing the word in word search so the search starts from the rarer letter. That makes the search fail fast rather than deep. **Three: reconsider the approach.** If the same state is reachable by many different paths, backtracking is re-solving it every time and the answer is memoisation or dynamic programming.”

“How do you know it terminates?” “There is a quantity that strictly decreases on every call and cannot go below zero. For subsets and permutations it is the number of elements left to consider. For combination sum it is the remaining target, which decreases because every candidate is positive — and that guarantee matters: a candidate of zero would recurse with identical arguments for ever. For grid search it is the number of unmarked cells. If I cannot name that quantity, I do not yet have a correct recursion.”

“Which of the four templates would you write for a problem you have never seen?” “I answer two questions before writing anything. Does order matter — is `[1,2]` the same answer as `[2,1]`? If they are the same, I carry a `start` index and never look backwards, which removes every duplicate ordering with one loop bound. If they are different, I need to track which elements are already used, which is a boolean per element. Then: can an element be used more than once? If yes I recurse on `i`, if no I recurse on `i + 1` or mark it used. Those two questions have four answers and they are exactly the four templates. Everything after that — pruning, duplicate handling — is the same in all four.”

“What is the space complexity, really?” “Extra space is `O(depth)` in every problem in this family — one working list, any marker arrays, and the stack. The output is separate and is usually the dominant term, and I would state the two separately because they say different things. The one exception worth flagging is combination sum, where the depth is `target / smallest candidate` rather than `n` — so a target of five thousand with a candidate of one is five thousand frames and a `RecursionError`. That is the only place in this phase where the stack is a real risk.”

A model answer

Asked, cold: *given a string, return every way of cutting it into palindromic pieces.*

“Let me check the constraint first — the string is at most sixteen characters. That tells me an exponential answer is expected, and it also tells me roughly which one: with n characters there are $n - 1$ places I could cut, so at most $2^{(n-1)}$ partitions. For “aaaa” every piece is a palindrome and all eight partitions are valid, so that bound is reachable. **Nothing can beat it, because the output is that big.**

Now the two questions I ask on every problem in this family. **Does order matter?** Here I am cutting left to right, so each decision is ‘where does the next piece end’, and I never go backwards — that is the `start`-index shape. **Can anything be reused?** No; each character is consumed exactly once, so after taking a piece I move `start` past it.

So: one function taking `start`. If `start` has reached the end of the string, every character has been used, so the current list of pieces is a complete answer and I record a **copy** of it — a copy, because the working list is one object that keeps being mutated. Otherwise I loop over the possible end points, take the substring, and — this is the part that matters — **check that it is a palindrome before I recurse**, not at the bottom. If it is not a palindrome, that entire subtree is pointless and I skip it. Then choose, recurse, un-choose: append the piece, recurse from the new position, pop.

One choose and one un-choose, so one line before the call and one after. I count them before running, because a missing undo here would be completely silent.

Complexity: $O(n \cdot 2^n)$ in the worst case — up to $2^{(n-1)}$ partitions, each costing $O(n)$ to copy — and extra space $O(n)$ for the working list and the stack.

If you tell me it is too slow, the first thing I would change is that palindrome check: it is $O(n)$ inside the loop, so I would pre-compute a table of which substrings are palindromes in $O(n^2)$ once, and then each check is a single lookup. That does not change the exponential output, but it removes a factor of n from every node in the tree.”

9 Recall card

- **Two questions choose the template, and that is the whole first minute.** *Does order matter?* Same answer for `[1,2]` and `[2,1]` → **start index, never look backwards**; different → **used array, look at everything**. *Reuse allowed?* Yes → **recurse on `i`**; no → **`i + 1` or `used[i]`**. Four combinations, four templates, and everything else sits on top of one of them.

- **The five rules that survive the phase:** append a **copy**; count the **chooses** and the **undoes** and undo in reverse; **prune before descending, never at the leaf** (N-Queens $n = 8$: 2,057 vs 19,173,961); anything passed as an argument needs **no undo**; and **say the output size first** — 2^n , $n!$, $C(n, k)$ — which removes “can you do better?”.
- **The duplicate rule, three outfits, one sentence:** *do not start two branches with the same value at the same level.* `i > start` in the start-index tree, `not used[i-1]` in the used-array tree, a per-level `set` in the swap tree. **Always sort first**, or both the dedupe and the `break` are silently wrong.
- `break` **when sorted and monotone**; `continue` **otherwise**. Too-big → `break`; duplicate → `continue`. Swapping the first loses speed silently; swapping the second loses **half the answers**.
- **Read the bound:** `n ≤ 8` means $n!$, `n ≤ 20` means 2^n , `n ≤ 25` means 2^n and **it is tight**. Extra space is $O(\text{depth})$ in every problem here — time is the whole tree, space is the deepest path — and the **only** stack risk in the phase is **combination sum**, whose depth is `target / min candidate`. “**How many ways**” is **not backtracking** when the sub-state repeats: that is dynamic programming.

1 What this is, and why they ask it

QPS stands for *queries per second* — the number of requests arriving at your system each second. It is the only number that decides how many machines you need, and “ten million users” does not tell you what it is.

Three sentences. A user count is a **stock**; a request rate is a **flow**, and you get from one to the other by asking how often each user does something. Every capacity decision in system design — how many servers, how big a database, whether you need a cache — is downstream of a rate, never of a total. And the arithmetic to get there is four multiplications you can do in your head.

They ask it because this is the exact point where the interview changes from vague to concrete. A candidate who says “we’ll need to scale horizontally because there are a lot of users” has said nothing. A candidate who says “ten million daily users at twenty requests each is two hundred million a day, which is about 2,300 a second average and maybe 7,000 at peak — so roughly ten application servers and one database with read replicas” has just designed the system’s shape in twenty seconds, and every later decision now has a number behind it.

This is the first day of the scaling phase, and every day after it uses these numbers.

2 The story

Ganesh had catered about four hundred weddings and he had one rule he would not bend: he did not accept a booking until somebody told him a number.

The father of the bride sat across from him in November and said it would be a big gathering. Ganesh asked how many. The man said, you know, the whole family, plus my colleagues, plus her mother’s side. Ganesh asked again how many, and the man said maybe four hundred, maybe five.

Ganesh wrote down five hundred and said that was the figure they were both agreeing to now, in November, because in February it would be too late to agree to anything.

Then he asked the question that the man had not been expecting, which was about time.

He said, five hundred people is not the problem. Five hundred people eating over two hours is one problem and five hundred people eating in twenty-five minutes is a completely different one, and they need different kitchens. When is the muhurtham?

Half past eleven.

Then they will all come to eat at once, Ganesh said, between twelve and half past. Not spread out. In about thirty minutes I have to serve five hundred people, which is roughly seventeen a minute, which means I need six counters, not three, and I need the rice to be coming out of the kitchen continuously from half past eleven rather than being cooked at twelve.

Then he did the rest of it aloud while the man watched. Five hundred people, three sweets each because people take more than they eat, that is fifteen hundred pieces. Two hundred grams of rice per head is a hundred kilos, and he always made a hundred and twenty because running out of rice at a wedding is the one thing nobody forgets. Curd, one small cup each, five hundred cups, and they come in trays of fifty, so ten trays.

The man said he had been to weddings where the food ran out.

Ganesh said yes, and it was never because somebody had cooked too little for five hundred people. It was because they had cooked exactly enough for five hundred people, spread over two hours, and then everybody arrived at the same time.

3 The idea in plain English

Ganesh does back-of-the-envelope estimation, and he does the two things candidates forget: he converts a total into a **rate**, and he multiplies that rate by a **peak factor**.

Stock and flow

- **Stock** is a total: five hundred guests, ten million users, fifty million photos.
- **Flow** is a rate: seventeen guests a minute, 2,300 requests a second, 600 uploads a second.

Servers are sized by flow. Storage is sized by stock. Almost every wrong answer in a system design interview comes from using one where the other belongs.

The words, defined

- **DAU** — daily active users. People who use the product on a given day.
- **MAU** — monthly active users. Usually three to five times DAU for a consumer app; a DAU/MAU ratio above 0.5 is exceptional and means people open it every day.
- **QPS** — queries per second. Requests arriving per second, averaged over the day.

- **Peak QPS** — the rate at the busiest moment. **Two to three times average for a global product**, and much higher for anything with a scheduled event: a ticket sale, a match, a results announcement.
- **Read:write ratio** — how many reads there are per write. **This single number decides whether you need a cache**, and for social products it is between 10:1 and 1000:1.
- **Latency** — how long one request takes. Always quoted as a percentile, never as an average.

The conversion, which is one division

$$\text{seconds in a day} = 24 \times 60 \times 60 = 86,400$$

Round it to 100,000. Every estimate in an interview is approximate anyway, and dividing by 100,000 is moving a decimal point. It makes your answer about 15 percent low, which nobody minds and everybody notices you did deliberately.

1 million requests per day	≈ 10 per second
100 million per day	≈ 1,000 per second
1 billion per day	≈ 10,000 per second

Memorise those three lines. They convert almost any daily figure in one step.

The four multiplications

Given “ten million users”, you produce every number an interviewer wants with four multiplications.

1. DAU × actions per user per day	= requests per day
2. requests per day ÷ 100,000	= average QPS
3. average QPS × peak factor	= peak QPS
4. writes per day × bytes per record	= storage per day

Worked, out loud:

10,000,000 DAU × 20 actions	= 200,000,000 requests/day
200,000,000 ÷ 100,000	= 2,000 QPS average
2,000 × 3	= 6,000 QPS peak
writes are 5% of that: 100 QPS average, 300 peak	
10,000,000 writes/day × 1 KB	= 10 GB/day = 3.65 TB/year

Thirty seconds, five numbers, and the shape of the system is decided. That is the skill.

What one machine can actually do

Numbers are useless without something to compare them to. These are the ones to carry:

a plain application server, simple request	1,000 - 5,000 QPS
the same, doing real work per request	200 - 1,000 QPS
one relational database, simple indexed read	5,000 - 15,000 QPS
one relational database, writes	500 - 5,000 QPS
Redis, single instance	100,000 + ops/s
one machine's network	~1 GB/s (10 Gbps link)
one SSD	~500 MB/s, ~50,000 IOPS

So 6,000 peak QPS is **a handful of application servers and one database**, and saying that is the whole point of having done the arithmetic.

Latency, and why the average is a lie

memory read (L1 cache)	~1 nanosecond
main memory read	~100 nanoseconds
SSD random read	~100 microseconds
network round trip, same datacentre	~0.5 milliseconds
network round trip, same continent	~30 milliseconds
network round trip, across the world	~150 milliseconds
spinning disk seek	~10 milliseconds

Two things fall out of that table, and both come up in every interview.

Memory is roughly a thousand times faster than an SSD, and an SSD is roughly a hundred times faster than a cross-continent hop. That is why caching works and why it is always the first thing you reach for.

And you cannot beat the speed of light. Mumbai to New York is about 12,000 km; light in fibre goes roughly 200,000 km per second, so one way is 60 ms and a round trip is 120 ms before any computer does anything at all. **That is why content delivery networks exist** — no amount of server optimisation can fix distance.

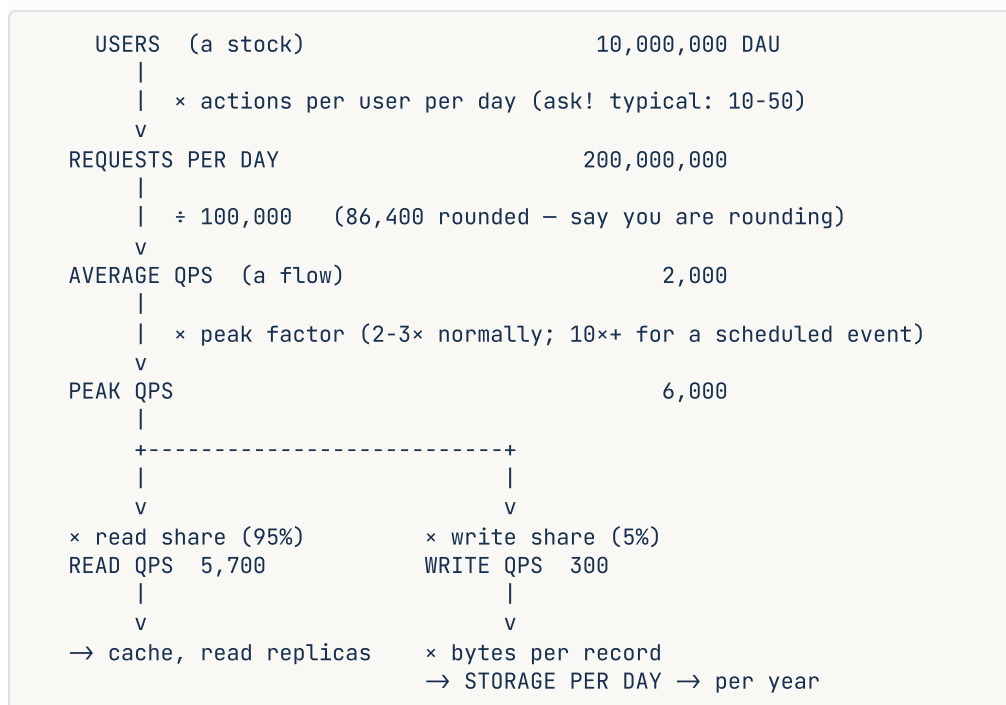
Percentiles, not averages

p50	— half of requests are faster than this. The typical experience.
p95	— 1 request in 20 is slower. What your unlucky users see.
p99	— 1 in 100. What your busiest users see, constantly.

A user who makes 100 requests loading one page will hit their p99 on almost every page load. That is why p99 is the number that gets quoted, and why “our average latency is 50 ms” is a claim that hides a p99 of two seconds.

4 The picture

The conversion ladder. This is the diagram to reproduce from memory.



Where the load actually lands:

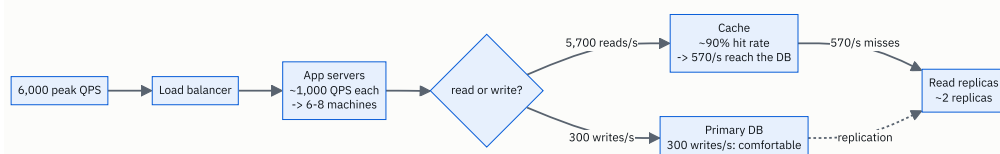
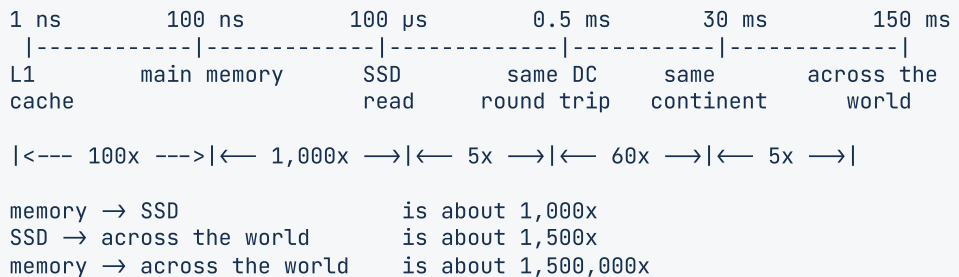


FIGURE 97.2

What to notice: **the cache turns 5,700 reads a second into 570.** That is the entire reason a cache is the first thing you add — and it is a number, not an opinion. Without it you would need six or seven read replicas; with it you need two.

The orders of magnitude, drawn on one line so the gaps are visible:



Every caching and placement decision in the whole course is explained by that line.

5 How it actually works

The six-step estimation, in order

Do these in this order, out loud, every time. It takes about three minutes.

Step 1 — get a number, and say you are assuming it. “You said ten million users. I will assume that is daily active, not registered — and if it is registered, daily active is usually ten to twenty percent of that, so a million, and every number below divides by ten.”

Never proceed on ‘a lot of users’. Pick a number, say it is an assumption, and continue.

Step 2 — actions per user per day. This is the number you have to reason about rather than look up.

a messaging app	30-100 messages sent + far more read
a social feed	10-20 opens, each loading 20 items
an e-commerce app	5-10 page views, 0.05 orders
a ride-hailing app	0.3 rides, but ~120 location updates per active ride
a video service	2-4 sessions, plus continuous streaming

Say where the number came from: “I will assume twenty actions a day, which is a user opening the app three or four times and doing a few things each time.”

Step 3 — convert to QPS. Divide by 100,000. Say “I am rounding 86,400 to a hundred thousand.”

Step 4 — apply a peak factor. Three, normally. Then ask whether the product has a *spike* shape:

always-on global product	peak $\approx$ 2-3 $\times$ average
single-country consumer app	peak $\approx$ 3-5 $\times$ (evenings)
ticket sales, exam results	peak $\approx$ 100-1,000 $\times$ for a few minutes
live sport, an election	peak $\approx$ 50-100 $\times$

The spike case changes the architecture, not just the machine count. A system that has to absorb a thousand times its average for two minutes needs a queue, not more servers.

Step 5 — split reads from writes. Ask for the ratio, or assume it and say so.

social feed / news	100:1 to 1000:1 reads to writes
messaging	$\sim$ 1:1 (roughly one read per message sent, in a 1-1 chat)
e-commerce browse/buy	$\sim$ 200:1
analytics ingestion	write-heavy, sometimes 1:100 the other way

This ratio is the single most useful number in the interview, because it tells you whether to reach for a cache and read replicas or for sharding and a write-optimised store.

Step 6 — storage. Stock, not flow.

records per day $\times$ bytes per record	=	bytes per day
$\times$ 365	=	bytes per year
$\times$ replication factor (usually 3)	=	bytes actually stored

Do not forget the replication factor. Three copies is the default in almost every distributed store, so your 3.65 TB of data occupies about 11 TB of disk. Candidates who mention it unprompted stand out.

A full worked example: a photo-sharing app

Do this out loud in under three minutes.

GIVEN: 100 million daily active users

1. Reads

each user opens the feed 5 times a day, 20 photos per feed
 $100M \times 5 = 500,000,000$ feed loads/day
 $500M \div 100,000 = 5,000$ QPS average
 $\times 3$ peak = 15,000 QPS peak
photo fetches: $\times 20 = 300,000$ image requests/s at peak
^^^ this is a CDN's job, not a server's

2. Writes

each user uploads 0.2 photos a day
 $100M \times 0.2 = 20,000,000$ uploads/day
 $20M \div 100,000 = 200$ QPS average
 $\times 3 = 600$ QPS peak

3. Read:write ratio

$500,000,000 : 20,000,000 = 25:1$
→ read-heavy: cache the feed, replicate the database

4. Storage

photo, compressed = 2 MB
plus 3 thumbnail sizes = ~0.3 MB
 $20M$ uploads $\times 2.3$ MB = 46 TB/day
 $\times 365 = 16,790$ TB/year ≈ 16.8 PB/year
 $\times 3$ replicas = ~50 PB/year
^^^ object storage, never a database

metadata row (id, user, caption, timestamps, geo) ≈ 500 bytes
 $20M \times 500$ B = 10 GB/day
 $\times 365 = 3.6$ TB/year ← this fits in one database

5. Bandwidth

outbound at peak: $300,000$ image requests/s $\times 200$ KB (thumbnail)
= 60 GB/s
 $\div 1$ GB/s per machine's link = 60 machines' worth of network
^^^ which is exactly what a CDN is

The two conclusions fall straight out of the arithmetic, and they are the answer to the question: photos go to object storage behind a CDN, and metadata goes in a database that is not large by modern standards. Nobody needed an opinion.

Where these numbers come from in the real world

- **Twitter** disclosed around 6,000 tweets per second average and 143,000 at peak (a 2013 spike during a television broadcast) — a $24\times$ peak factor for an event, which is why the event case is worth calling out separately.
- **WhatsApp** ran roughly 2 million connections per server on tuned FreeBSD boxes, which is the number people quote when arguing that connection counts and request counts are different problems.
- **Amazon** reported that every 100 ms of added latency cost about 1 percent of sales, and **Google** found that half a second of extra delay reduced traffic by 20 percent. That is why latency budgets are treated as product requirements rather than engineering preferences.

- The **latency table** above descends from Jeff Dean’s “numbers every programmer should know”. The original figures are from 2009; SSDs and networks have improved, and the *ratios* — which are what matter — have not.

6 The numbers

The reference table to memorise

seconds in a day	86,400	→ round to 100,000
seconds in a month	2,600,000	→ round to 2.5 million
seconds in a year	31,500,000	→ round to 30 million
1 million/day	≈	10 QPS
100 million/day	≈	1,000 QPS
1 billion/day	≈	10,000 QPS
1 KB × 1 million	=	1 GB
1 KB × 1 billion	=	1 TB
1 MB × 1 million	=	1 TB
1 MB × 1 billion	=	1 PB

That last block is the one that saves you. “A billion records of a kilobyte each” is one terabyte, and you should be able to say it without pausing.

Typical record sizes

a tweet / short post	~300 bytes	(text + ids + timestamps)
a database row with 10 columns	~500 bytes	- 1 KB
a chat message	~200 bytes	
a log line	~200 - 500 bytes	
a JSON API response	~2 - 20 KB	
a web page, total	~2 MB	
a compressed photo	~1 - 5 MB	
a minute of 1080p video	~50 MB	

What things cost, roughly

cloud object storage	~\$1.7 per GB-month
cloud block storage (SSD)	~\$8 per GB-month
outbound bandwidth	~\$7 per GB
a mid-size application server	~\$8,000 - 15,000 per month
a managed database instance	~\$25,000 - 200,000 per month

The one that surprises people: **bandwidth is usually more expensive than storage.** Serving 60 GB/s of images is a bandwidth bill, not a disk bill, and that is a large part of why CDNs exist commercially as well as technically.

The conversions worth having automatic

peak factor	×3 by default; say when you think it is more
replication factor	×3 by default
DAU from registered users	~10-20%
MAU to DAU	÷3 to ÷5
cache hit rate, well-chosen key	90-95%
compression on text	~3-5×

An availability table, since it always comes up

99%	"two nines"	3.65 days of downtime per year
99.9%	"three nines"	8.8 hours per year
99.99%	"four nines"	53 minutes per year
99.999%	"five nines"	5.3 minutes per year

Four nines means you may be down for less than an hour in a whole year, which is why it costs so much more than three. Being able to convert nines to minutes on the spot is a small, reliable signal.

7 The trade-offs

Estimating is not the same as being right

The purpose of these numbers is **to decide the shape of the design**, not to be accurate. Six thousand QPS and nine thousand QPS lead to the same architecture; six thousand and six million do not.

Round aggressively and say that you are. "I am rounding 86,400 to a hundred thousand, so this is about fifteen percent low, and that does not change the answer." That sentence buys you both speed and credibility.

Averages hide everything that matters

Ganesh's whole point. Five hundred people over two hours and five hundred people in thirty minutes need different kitchens, and the average — 250 an hour — describes neither.

The same is true of latency. **Report p99, and design for peak, not for average.** A system sized for average QPS falls over every evening.

When the peak factor is the design

A $3\times$ peak is a machine-count question. A $500\times$ peak is an architecture question, and they are not the same conversation:

exam results published at 10 a.m.
normal traffic
at 10:00:00

500 QPS
250,000 QPS for about 90 seconds

You do not buy 250 times the servers for ninety seconds. **You put a queue in front, you serve a pre-computed static file from a CDN, and you accept that some requests wait.** Recognising which kind of peak you have is more valuable than the arithmetic itself.

Where the estimate misleads

- **Fan-out multiplies writes invisibly.** A celebrity with 50 million followers posting once is one write to you and 50 million to the feed system. The user-facing rate says nothing about it, and it is the single biggest trap in estimating a social product.
- **Connections are not requests.** A chat app with a million idle users has a million open connections and almost no QPS. Memory per connection, not CPU per request, is the constraint — which is why WhatsApp's number is about connections.
- **Storage grows and never shrinks.** QPS is roughly flat if usage is flat; storage accumulates for ever. Always quote storage per year, and always ask about retention.
- **The 80/20 of data is real.** Ninety percent of reads hit a few percent of the data. That is what makes a 90 percent cache hit rate achievable on a small cache, and assuming a uniform distribution makes caching look much worse than it is.

8 In the interview

How it gets asked

- Directly: *"This app has ten million users. What is the read QPS?"*
- As the opening move of any design: *"Before we design it, give me some rough numbers."*
- As a probe: *"How many servers do you need?"* / *"How much storage in a year?"*
- As a trap: *"Would a single database handle this?"* — they want to know if you can compare your number against a machine's capacity.
- As a follow-up: *"What is your p99 latency budget for this endpoint?"*

What to say out loud, in the first ninety seconds

1. **Turn the stock into a flow, and name the assumption.** "Ten million — I will assume that is daily active. Each user does about twenty actions a day, so two hundred million requests a day."
2. **Convert, and say you are rounding.** "There are 86,400 seconds in a day and I will round that to a hundred thousand, so about two thousand QPS on average."

3. **Peak it.** “Peak is two to three times average for a product used all day, so call it six thousand QPS. If there is a scheduled event in this product, that number is completely different and I would want to know.”
4. **Split reads and writes.** “What is the read-to-write ratio? If it is a feed, I would assume something like fifty to one — so 5,900 reads a second and 120 writes.”
5. **Compare against a machine.** “One application server handles roughly a thousand QPS of real work, so six to eight machines. One database handles maybe ten thousand simple reads a second, so with a cache in front, one primary and a couple of replicas.”
6. **Then storage, separately.** “Ten million writes a day at a kilobyte each is 10 GB a day, 3.6 TB a year, and with three replicas about 11 TB on disk.”

The follow-ups

“**Where did your peak factor come from?**” “Two to three times average is the normal shape for a product used throughout the day — traffic follows waking hours, and if the users are in one country it is more pronounced, maybe three to five times in the evening. But the factor depends entirely on whether the product has a **scheduled** event. Ticket sales, exam results, a live match — those are hundreds of times average for a few minutes, and that is not a bigger-servers problem, it is a different architecture: a queue in front, pre-computed responses served from a CDN, and admitting that some requests will wait. So I would ask you which shape this is, because it changes the design rather than the machine count.”

“**Would a single database handle this?**” “Let us compare. One well-provisioned relational database handles roughly five to fifteen thousand simple indexed reads a second, and something like five hundred to five thousand writes a second depending on how much index maintenance each write causes. My estimate was 5,900 reads and 120 writes at peak. The writes are comfortable. The reads are borderline on their own — but with a cache at even a ninety percent hit rate, only 590 reach the database, which is very comfortable. So yes, one primary plus a cache, and I would add read replicas as headroom rather than necessity. If the write rate were five thousand a second I would be having a completely different conversation, about sharding.”

“**How much storage in a year?**” “Depends on which data. Metadata and media are different problems and I would separate them. Twenty million uploads a day, with a metadata row of about five hundred bytes, is 10 GB a day and 3.6 TB a year — that fits comfortably in one database. The media is 2 MB a photo plus thumbnails, so 46 TB a day and about 17 petabytes a year, and with three-way replication around 50 petabytes. That obviously does not go in a database; it goes in object storage like S3, with the database holding only the key. And I would ask about retention, because storage is the one number that only ever goes up.”

“What is a good p99 target?” “For a user-facing read, I would aim for under 200 milliseconds at p99 for the server portion, because the perceptual threshold where an interaction stops feeling instant is around 100 to 200 milliseconds and the network is already spending some of that. I would quote p99 rather than the average deliberately: a page that makes a hundred requests will hit its p99 on nearly every load, so the p99 is the *typical* page experience, not the rare one. And I would ask whether this endpoint is on the critical path, because a background write can happily take a second and nobody notices.”

“Why do you keep dividing by a hundred thousand?” “Because it is 86,400 rounded, and every input to this calculation is an assumption anyway. Rounding up makes my QPS about fifteen percent low, which never changes the architecture — the difference between 2,000 and 2,300 QPS is nothing, and the difference between 2,000 and 200,000 is everything. I say I am rounding so it is clearly a choice. The three lines I actually carry are: a million a day is ten a second, a hundred million a day is a thousand a second, a billion a day is ten thousand a second.”

“Your users are in India and your servers are in Virginia. What does that cost you?” “About 200 milliseconds of round trip before anything computes, and there is no way to engineer it away — it is distance. Light in fibre travels around 200,000 kilometres a second, so 13,000 kilometres is 65 milliseconds each way, and real routes are not straight lines. If the page makes several sequential requests, that multiplies. The fixes are all about placement rather than speed: put static content and media on a CDN with edge locations in India, put a read replica or a full regional deployment closer to the users, and reduce the number of round trips the client needs. This is the one performance problem that no amount of faster code solves.”

A model answer

Asked: *this app has ten million users. What is the read QPS?*

“Let me convert that into a rate, because a user count on its own does not size anything.

First, an assumption I want to state: I am taking ten million as **daily active** users. If that is registered users instead, daily active is usually ten to twenty percent of it, so a million, and every number I am about to give divides by ten.

Next, actions per user per day. For a feed-shaped product I would assume each person opens it three or four times and does a handful of things each time — call it twenty requests a day. So ten million times twenty is **two hundred million requests a day**.

There are 86,400 seconds in a day and I am going to round that to a hundred thousand, which makes the arithmetic a decimal shift and my answer about fifteen percent low — that does not change any decision. Two hundred million divided by a hundred thousand is **two thousand QPS on average**.

Average is not what you size for. For a product used throughout the day, peak is two to three times average, so I will design for **six thousand QPS**. I would want to check one thing though: does this product have a scheduled event — a sale, a results announcement, a live match? Because that is not a three-times peak, it is a hundred-times peak for a couple of minutes, and that is a queue-and-CDN problem rather than a more-servers problem.

Then reads against writes. For a feed I would assume something like fifty to one, so about **5,900 reads a second and 120 writes a second at peak**. That ratio is the most useful number here, because it is what tells me this is a caching problem and not a sharding problem.

Now I can compare against what machines actually do. One application server handles roughly a thousand requests a second of real work, so **six to eight app servers** with headroom. One relational database does five to fifteen thousand simple indexed reads a second, so 5,900 is borderline alone — but a cache at a ninety percent hit rate takes it to **590 reaching the database**, which is very comfortable. One primary, a couple of read replicas, and a cache in front.

And storage separately, because that is a stock rather than a flow: if writes are ten million a day at about a kilobyte each, that is 10 GB a day, **3.65 TB a year**, and roughly 11 TB once you count three replicas. If there is media involved, that number changes by three orders of magnitude and belongs in object storage behind a CDN rather than in the database.”

9 Recall card

- A user count is a stock; QPS is a flow. Servers are sized by flow, storage by stock. Four multiplications get you there: $\text{DAU} \times \text{actions/day} = \text{requests/day}$; $\div 100,000 = \text{average QPS}$; $\times 3 = \text{peak QPS}$; $\text{writes/day} \times \text{bytes} = \text{storage/day}$. Round 86,400 to **100,000** and say you are rounding.
- The three conversions to carry: $1 \text{ million/day} \approx 10 \text{ QPS}$; $100 \text{ million/day} \approx 1,000 \text{ QPS}$; $1 \text{ billion/day} \approx 10,000 \text{ QPS}$. And for storage: $1 \text{ KB} \times 1 \text{ billion} = 1 \text{ TB}$; $1 \text{ MB} \times 1 \text{ billion} = 1 \text{ PB}$.
- Compare against what one machine does, or the numbers mean nothing: app server $\sim 1,000 \text{ QPS}$, relational DB $\sim 10,000 \text{ simple reads/s}$ and $\sim 1,000 \text{ writes/s}$, Redis $100,000+ \text{ ops/s}$, one network link $\sim 1 \text{ GB/s}$. Then **6,000 peak QPS = 6–8 app servers + one DB with a cache**.
- Peak factor is $2\text{--}3\times$ normally — but $100\times+$ for a scheduled event, and that changes the architecture, not the machine count (queue in front, pre-computed responses on a CDN). Always multiply storage by a replication factor of 3. And the read:write ratio is the most useful single number: high means cache and replicas, low means sharding and write-optimised stores.
- Memory $\approx 1,000\times$ faster than SSD; SSD $\approx 1,500\times$ faster than a cross-world round trip. Mumbai $\leftrightarrow$ New York is $\sim 200 \text{ ms}$ of pure distance that no code can remove — which is what CDNs are for. Quote **p99, never the average**: a page making 100 requests hits its p99 on nearly every load. And **99.99% availability is 53 minutes of downtime a year**.

PRACTICE

Practice — Recursion and backtracking revision and mock round

DSA topic: Recursion and backtracking revision and mock round **System design topic:** What scale actually means, in numbers

Code these, in this order

This is a mock round, so treat it like one. **Set a timer for twenty-five minutes per problem, talk out loud the whole time, and do not look at anything.** If you cannot name which of the four templates it is within thirty seconds, that is the thing to practise, not the code.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Palindrome Partitioning	LeetCode 131 (Medium)	Recognising the <code>start</code> -index tree from a string problem, and pruning before descending.
2	Restore IP Addresses	LeetCode 93 (Medium)	Two prunes, both <code>break</code> , and a fixed piece count.
3	Combination Sum II	LeetCode 40 (Medium)	Both skips with the right keyword each, cold.
4	Word Search	LeetCode 79 (Medium)	Path versus region, said before any code is written.

Before each problem, say the four things

Out loud, timed, before typing:

1. The constraint bound, and what it implies.
2. The output size, and that nothing can beat it.
3. Order matters or not — so which tree.
4. Reuse allowed or not — so `i` or `i + 1` .

If those four take more than forty seconds, do them again on the next problem until they do not.

On problem 1, add the table afterwards

Solve it with the inline palindrome check, then again with the pre-computed table. Say what factor came off the per-node cost, and what did not change.

On problem 2, justify both `break` s

Neither is obvious. For each, say the monotonicity that makes `break` correct rather than `continue`.

On problem 3, write both skips before the loop body

Write the two guard lines first, with the right keyword each, then fill in the rest. Then swap the keywords and record both kinds of damage.

On problem 4, say the word first

Say “path” or “region” out loud before typing. Then write it, and afterwards say what the answer would have been had you said the other word.

The two-questions drill

1. State both questions from memory.
2. Give the four combinations and the template each one names.
3. For each template, name two problems that use it.
4. Take five problems you have never seen and classify each in under ten seconds.

The template drill

1. Write all four templates from memory, in under six minutes total.
2. Say what is identical across all four.
3. Say what changes between templates 1 and 2, character by character.
4. Say what template 4 does not need, and why.

The five-rules drill

1. Name the five rules that survive the phase.
2. For each, give the specific wrong output it prevents.
3. Say which of the five is silent in every case.
4. Say which one removes a question from the interview entirely.

The duplicate drill

1. Write both duplicate conditions.
2. Say the one English sentence they both mean.
3. Say which tree shape each belongs to, and why the bookkeeping differs.
4. Use the wrong one in each tree and record what you lose.
5. Say what both of them require to have happened first.

The keyword drill

1. State the rule for `break` versus `continue` in one sentence.
2. Give two examples of each from the phase.
3. Swap them in Combination Sum II and record the output.
4. Swap them in Combination Sum and say what changed — and what did not.

The complexity drill

Fill this in from memory, then check it:

PROBLEM	ANSWERS	TIME	EXTRA SPACE
subsets			
permutations			
combinations C(n,k)			
combination sum			
word search			
N-Queens			

Then say the sentence that is true of the whole right-hand column.

The bound-reading drill

For each constraint, say which template the problem probably wants:

1. $1 \leq n \leq 6$
2. $1 \leq n \leq 20$
3. $1 \leq n \leq 16$ on a string
4. $1 \leq \text{target} \leq 500$, candidates as small as 1
5. $1 \leq n \leq 10^5$

The last one is the important one. Say what it rules out.

The break-it drill

Trigger each, record the exact output or error, and say which rule it violates:

1. `result.append(current)` in any template.
2. `used[i] = False` omitted.
3. `i` instead of `i + 1` in Combination Sum II.
4. `i > 0` instead of `i > start`.
5. `continue` instead of `break` on the too-big check — and time it.
6. Combination sum with a target of 5,000 and a candidate of 1.

```
7. set(list_of_lists) .
```

The mock drill

Take two problems you have never seen, from the list below, and do them under a timer with no references. Talk the whole time.

- Generate Parentheses (LeetCode 22)
- Letter Case Permutation (LeetCode 784)
- Combination Sum III (LeetCode 216)
- Beautiful Arrangement (LeetCode 526)
- Additive Number (LeetCode 306)

Afterwards, write down: how long the first minute took, whether you named the template before coding, and whether you said the output size out loud.

The conversion drill

Compute each in your head, then check:

1. 50 million requests a day, in QPS.
2. 3 billion a day, in QPS.
3. 800 QPS, in requests per day.
4. 5 million DAU $\times$ 40 actions, average and peak QPS.
5. 2 KB $\times$ 500 million records, in TB.

The ladder drill

1. Draw the conversion ladder from users to peak QPS from memory.
2. Say what number goes on each arrow, and where you would ask a question instead of assuming.
3. Say which side of the ladder gives you servers and which gives you storage.
4. Say what you must multiply storage by, and why.

The capacity drill

State from memory:

1. QPS for one application server, doing real work.
2. Simple indexed reads per second for one relational database.
3. Writes per second for the same.
4. Operations per second for one Redis instance.
5. Bandwidth of one machine's network link.

Then size a system at 6,000 peak QPS with a 50:1 read ratio and say how many of each you need.

The latency drill

1. Write the six latency figures in order.
2. State the memory-to-SSD ratio and the SSD-to-across-the-world ratio.
3. Compute the pure-distance round trip for Mumbai to New York and show the working.
4. Say what that number means for the design, and what it rules out.
5. Say why p99 is quoted rather than the average, using the hundred-requests-per-page argument.

The photo-app drill

Reproduce the whole worked example for 100 million DAU, out loud, in under three minutes:

1. Feed loads per day, average QPS, peak QPS.
2. Image requests per second at peak, and where they must be served from.
3. Uploads per day and write QPS.
4. The read:write ratio and what it implies.
5. Media storage per day and per year, with replication.
6. Metadata storage per year, and where it goes.
7. Peak outbound bandwidth, and what that number is really telling you.

The peak-shape drill

For each product, give the peak factor and say whether it is a machine-count problem or an architecture problem:

1. A global messaging app.
2. An Indian food-delivery app on a Friday evening.
3. Ticket sales for a concert.
4. A university publishing results at 10 a.m.
5. A live cricket match.

The misleading-estimate drill

For each, say what the naive estimate misses:

1. A celebrity with 50 million followers posts once.
2. A chat app with a million idle users.
3. A system that has run for three years.

4. A cache sized assuming every key is equally likely.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two problems, no hints, talk as you go.* For each: the bound read aloud, the output size stated, the two questions answered so the template is named, the pattern with the copy flagged, the prune placed before the descent, and both complexities given separately.
 2. *This app has ten million users. What is the read QPS?* The daily-active assumption named, actions per user reasoned about rather than guessed, the division with the rounding admitted, the peak factor with the question about scheduled events, the read:write split, and then the comparison against what one machine actually does.
 3. *Which of the four templates would you write for a problem you have never seen?* The two questions, the four answers, and the sentence about what is identical in all four.
-

Before you move on

- ☐ I can state the two questions and the four templates they choose between.
- ☐ I can write all four templates from memory in under six minutes.
- ☐ I can name the five rules and the wrong output each one prevents.
- ☐ I can write both duplicate conditions and say the one sentence they share.
- ☐ I can state the `break` versus `continue` rule and give two examples of each.
- ☐ I filled in the complexity table from memory and got the space column right.
- ☐ I can read a constraint bound and name the expected template.
- ☐ I know the only problem in the phase that can overflow the stack, and why.
- ☐ I did two unseen problems under a timer and named the template in the first thirty seconds.
- ☐ I can convert any daily figure to QPS in my head.
- ☐ I can draw the conversion ladder and say what goes on every arrow.
- ☐ I know what one app server, one database and one Redis instance can do.
- ☐ I can give the six latency figures and the two ratios.
- ☐ I can compute the Mumbai–New York round trip and say what it rules out.
- ☐ I can do the photo-app estimate out loud in under three minutes.
- ☐ I can tell a machine-count peak from an architecture peak.

- [] I answered all three questions above out loud.

The nine sections, and what each is for

Every lesson in this book, on both tracks, carries the same nine sections in the same order. This is what each one is for, and how to read it.

1. What this is, and why they ask it

The topic in plain words, and the reason an interviewer spends time on it. This section exists so you never learn something without knowing what it buys you.

2. The story

A scene with a person in it, told in ordinary language, with no technical words at all. Two hundred to four hundred words about matching socks, or a canteen queue, or laying tables.

This is the most important section for a beginner and the easiest to skip. The story gives you a place to put the idea before you have the vocabulary for it. Later, when you are asked the question under pressure, the story is what you will reach for first.

3. The idea in plain English

The story, translated. Every term is defined the first time it appears, and defined terms are set in **bold** so you can find them again. Appendix D collects them all.

4. The picture

A diagram. Arrays, memory and pointers are drawn as ASCII boxes with indices above and values below, because adjacency matters and a box drawing keeps it. Trees, graphs, architectures and request flows are drawn as proper figures.

Every diagram is followed by a line telling you what to notice in it.

5. The code, built step by step

Fragments of ten lines or fewer, each one followed by prose explaining it. Then the complete solution in one block, ready to run.

The full answer is always here. This book does not withhold solutions.

6. What it costs

The arithmetic, done where you can see it.

On the DSA track that means counting the loop iterations out loud: not “this is quadratic” but “the inner loop runs sixty times for each of sixty items, so about three thousand comparisons”. On the system design track it means doing the multiplication: not “that is a lot of storage” but $50\text{M} \times 20 \times 2\text{KB} = 2\text{TB}$.

7. The traps

The mistakes people actually make, with the real error text printed exactly as it appears on screen. When your own program breaks this way, you will recognise it.

8. In the interview

The point of the whole document.

Real interviewer phrasings. A script for the first ninety seconds, because that is when candidates either take control of the question or lose it. The three follow-ups that usually come next. And a model answer, written out in full, so you have something to measure yours against.

9. Recall card

The two or three sentences that are the entire lesson. Say them aloud, from memory, in full sentences.

If you can do that, you have finished the lesson. If you cannot, you have read it, which is a different thing.

Recall cards

Section 9 of every lesson in this volume, collected. Cover the card, say it aloud, then check. Cards are grouped by the phase they belong to, on both tracks.

Linked lists

Day 078 · dsa — Nodes and links: pointers without pointers

- A node is a value plus a reference to the next node; the head is just a name for the first one, and losing it loses the list. In Python a variable holds a **reference**, not an object — so `node.next` is exactly a pointer, minus the address arithmetic.
- One difference explains everything: an array is contiguous and finds element `i` by arithmetic; a linked list is scattered and must follow `k` links. So access is $O(n)$ vs $O(1)$ — and in exchange, insertion and deletion **move nothing**.
- Say the qualifier or the claim is wrong: insertion and deletion are $O(1)$ *given the node*. Given only a **position**, both structures are $O(n)$ — the array pays for the shift, the list pays for the walk. The genuine win is when a **hash map hands you the node** (the LRU cache).
- The measurements, at 1,000,000 elements: traversal 0.062 s list vs 0.123 s linked — $2.0 \times$ (cache locality, not complexity) · 100,000 front insertions: 1.443 s vs 0.0246 s — $\sim 59 \times$ · memory 8 B per list slot vs 48 B per node with `__slots__` (336 B without) — $6 \times$ at best.
- The three bugs that will cost you the next eight days. Relink in the **wrong order** and a node points at itself — build the new node first, then attach. `AttributeError: 'NoneType' object has no attribute 'next'` — every `.next.next` promises two nodes exist. And **rebinding head inside a function does not change the caller's variable** — return the new head, always. The head as a special case is what [day 080](#) deletes.

Day 079 · dsa — Traversal, insertion, and deletion

- One rule generates all three operations: to change the list at position `k`, stand at `k - 1`. You cannot reach backwards, so you can never unlink a node you are holding. Deletion is `previous.next = previous.next.next`; insertion is `previous.next = Node(value, previous.next)` — **build first, then relink**, or the node points at itself and the rest of the list is gone.

- `k = 0` is a different operation, not a harder one — the head *reference* changes, not a link. In a bare function you must **return the new head**, because rebinding a parameter changes nothing for the caller and raises no error. Tomorrow's **dummy head** deletes this case entirely.
- **Insertion and deletion have different bounds.** Inserting at `size` is legal (it appends), so insertion has `size + 1` valid positions and deletion has `size: index > size` versus `index ≥ size`. Check bounds **before** any dereference.
- **Store `size` and `tail`, and maintain them in every mutation.** `size` makes `len` $O(1)$; `tail` makes `append` $O(1)$ — **without it, n appends is $O(n^2)$** , about 5×10^9 steps at $n = 100,000$. The bug with **no error message** is deleting the last node without moving the tail back: the next `append` attaches to an unreachable node and the value vanishes. A three-assertion `_check` catches it.
- **A tail reference does not make delete-last $O(1)$** — you need the node *before* the tail and a singly linked list cannot walk backwards. That is the argument for the **doubly linked list**. And the honest summary against a Python list: **the linked list wins at the front and loses everywhere else** (100,000 front inserts: **1.443 s vs 0.0246 s**, $\sim 59\times$).

Day 080 · dsa — The dummy head trick

- **One line deletes the head special case:** `dummy = Node(0, head)`. The real first node now has a predecessor, so it is an ordinary node and **one loop handles everything**. The head case is not just an `if` — on `[7, 7, 7, 1]` it is a *run*, plus a second case when everything is deleted. The dummy takes the count of special cases to **zero**, not to fewer.
- **return `dummy.next`, never return `head`**. The original head may have been deleted, and returning it is a **silent wrong answer** — `remove_elements([7,7,1], 7)` gives back `[7,7,1]`. Write the return line at the same moment you write the dummy. And **dummy must never move** — walk with a separate cursor.
- **The second rule has nothing to do with the dummy: advance ONLY when you keep.** After unlinking, `previous.next` is a node you have not inspected, so an unconditional advance leaves consecutive matches behind — `[1, 7, 7, 2]` becomes `[1, 7, 2]`.
- **Two forms, one idea — a node that holds nothing so the first one stops being different.** The **guard** form is `Node(0, head)`, for pruning. The **builder** form is `Node(0)` plus a moving `tail`, for merging, partitioning and copying — and any list you build out of existing nodes must be **terminated explicitly** (`more.next = None`) or you construct a cycle and the print never returns.

- **Cost: one node, $O(1)$ space, no complexity change** — it removes branches, and branches are where the bugs are. Recognition rule: **use a dummy when the operation can change which node is first, or when you are building a new list**. Not for counting, finding the middle, or cycle detection. Test inputs that matter: `[7,7,7,1,7]` · `[7,7,7]` · `[1,7,7,2]` · `[]`.

Day 081 · dsa — Reversing a linked list

- **Nothing moves and nothing is allocated** — you turn each `next` round exactly once. Three variables, four lines, one fixed order: **save the next node · turn the arrow · previous up · current up**. *Look behind you, let go, take hold.*
- **Line 1 is the whole problem.** `current.next` is overwritten on line 2, so failing to save it first loses the entire rest of the list — no exception, `[1,2,3]` just returns `[1]`.
- **Return `previous`, not `head`.** At the end, `current` is `None` and `previous` is the old tail, which is the new head; `head` now refers to the last node, so returning it gives a one-element list. And loop on `while current is not None`, never `current.next` — that stops one node early and detaches the tail.
- **Recursive: trust the call, then `head.next.next = head` and `head.next = None`.** The second line is **mandatory** — without it you build a two-node cycle and printing never returns. It is **$O(n)$ stack**, and `RecursionError: maximum recursion depth exceeded` fires at about **1000** nodes in Python, so ship the iterative one.
- **The two escalations are this loop plus bookkeeping.** *Sublist* (LeetCode 92): use a **dummy**, and **capture the node at position `left` before reversing** — afterwards it is the last node and unfindable; then two reattachments. *Groups of k* (LeetCode 25): **count k nodes first**, before touching anything, so a short final group is never reversed and undone; $O(2n) \rightarrow O(n)$, $O(1)$ space.

Day 082 · dsa — Finding the middle, and the runner technique

- **Two variants of one idea, and naming which you need is the first move.** Different **speeds** (`slow += 1`, `fast += 2`) finds the **middle**; a fixed **gap** (`fast` starts `n` ahead, then both move together) finds a **fixed distance from the end**. Neither needs the length.
- **The middle's loop condition is doing two jobs and the order matters:** `while fast is not None and fast.next is not None` — the first guard protects even lengths, the second odd, and `and` short-circuits so the null check must come first.
- **On an even length there are TWO middles — ask which.** `while fast and fast.next` → the **second** (LeetCode 876). `while fast.next and fast.next.next` → the **first**, which is what **splitting for a merge sort** needs: with the second, `[1,2]`

splits into itself and nothing and you get `RecursionError`. And **the cut is mandatory** — `middle.next = None`, or the halves share a tail.

- **To remove the Nth from the end, start BOTH pointers at a dummy** — it lands `slow` on the **predecessor** and makes removing the head an ordinary case. Guard the opening walk, or a short list gives `AttributeError: 'NoneType' object has no attribute 'next'`.
- **One pass is not asymptotically better** — both are $O(n)$ time and $O(1)$ space and **do about the same pointer moves**. What it buys is **one traversal instead of two** (and read-once data). Applications: **palindrome in $O(1)$ space** (middle $\rightarrow$ reverse $\rightarrow$ compare $\rightarrow$ **reverse back**, because a read-only question must leave the data as it found it) · **reorder** (split, reverse, interleave) · and tomorrow, **cycle detection** — the same two pointers, a completely different conclusion.

Day 083 · dsa — Cycle detection, and why Floyd's algorithm works

- **A list with a cycle is a tail plus a loop — a rho shape**. Two runners, **one step and two steps**. Fast falls off the end $\rightarrow$ no cycle. Fast catches slow **from behind** $\rightarrow$ cycle, because *a faster runner can only catch a slower one from behind on a route that returns to itself*. Compare with `is`, not `=`, and **advance before comparing**.
- **They cannot skip past each other: inside the loop the forward gap shrinks by exactly one per step**, so it must hit zero. That argument needs the **2:1 ratio** — at 3:1 the gap falls by two and could step over zero on an odd cycle.
- **Phase two, and the proof is four lines**. `a` = head $\rightarrow$ start, `b` = start $\rightarrow$ meeting, `c` = cycle length. slow walked `a + b`; fast walked `2(a+b) = a+b+kc`; so `a + b = kc`, so `a = kc - b`. Hence from the meeting point, `a` more steps = a whole number of laps = the entrance. Put one pointer back at the **head**, move both at **single speed**, they meet at the start. **No safety counter** — the arithmetic guarantees it.
- **The meeting point is NOT the entrance** (it only looks right when `a = 0`, which is exactly the test case people try). Run the two degenerate shapes last: **one node pointing at itself**, and **a list that is entirely one cycle**.
- **Costs: phase one $\leq a + c$, phase two exactly `a`, so $\leq 2n$ — $O(n)$ time, $O(1)$ space, no mutation**. Name the **hash-set** baseline first ($O(n)$ space, ~ 32 MB at a million nodes, entrance for free) and then improve it; reject **marking nodes** because it destroys the caller's data. The disguise to recognise: **Find the Duplicate Number** — read the array as `i  $\rightarrow$  nums[i]`, the walk can never leave the array, and the **duplicate is the entrance**.

Day 084 · dsa — Merging and sorting linked lists

- **The merge creates nothing and copies nothing — only `next` pointers move, so it is $O(1)$ extra space.** Use a **dummy builder** so the first node attaches like every other, `≤ not <` so the merge is **stable**, and attach the remainder **in one assignment** (`tail.next = a if a else b`), because whatever is left is already sorted and already linked.
- **Sort = split, sort each half, merge — five lines, all of them days you have already done.** Two fatal details in the split: the **FIRST middle** (`while fast.next and fast.next.next`), or `[1,2]` splits into itself and nothing and you get `RecursionError`; and the **cut** (`slow.next = None`), or both halves are the whole list. Base case needs **both** `head is None` **and** `head.next is None`.
- **Why merge sort, in one sentence each.** Quicksort's partition needs **random access**, which a list does not have. And merging **arrays** needs a scratch array while merging **lists** needs nothing — so **merge sort is the array's worst case for space and the list's best**.
- **$O(n \log n)$ time — every level does `n` work, and there are $\log_2 n$ levels (≈ 20 at a million nodes).** Space is **$O(\log n)$ stack**, not $O(1)$; the **bottom-up** version (merge runs of $1 \rightarrow 2 \rightarrow 4 \dots$) is genuinely $O(1)$ at the cost of ~ 20 fiddlier lines. Copying values into a Python list is $O(n)$ space and throws away node identity.
- **Merge k lists: one at a time is $O(nk)$ ($\sim 5 \times 10^9$ at $k = 10^4$, $n = 10^6$); a heap of the k heads is $O(n \log k)$; pairwise rounds are also $O(n \log k)$ and reuse `merge_two` unchanged — prefer pairwise.** The heap's trap: **include a tie-breaker in the tuple** or `TypeError: '<' not supported between instances of 'Node'`, and **terminate the output with** `tail.next = None`.

Day 085 · dsa — Doubly and circular linked lists

- **The second pointer buys exactly one thing: given a node, remove it in $O(1)$ —** `node.prev.next = node.next; node.next.prev = node.prev`. Two assignments, no walk, no length. In a singly linked list the same operation is **$O(n)$** , because reaching the predecessor means walking from the head. *The second pointer converts having a node into being able to act on it.*
- **That is the LRU cache's whole reason, with a number:** the map hands you the node in $O(1)$, so unlinking must be $O(1)$ too — at capacity 100,000 a singly linked list would make every **read** a 100,000-step walk instead of ~ 6 pointer writes.
- **Write into the new node first, then rewire the neighbours** — or take both neighbours as parameters, which makes the ordering bug impossible. **Use sentinels at BOTH ends**; with two directions there are twice as many `None` checks to forget, so they stop being a convenience.

- **The cost that is not memory: a doubly linked list can be HALF correct** — right forwards, broken backwards — and it passes every forward test before failing somewhere else entirely. A singly linked list cannot be in that state. **Write `_check` : walk both ways, assert one is the reverse of the other.** Price: +8 B per node (~17%), 2 writes per removal instead of 1, 4 per insertion instead of 2.
- **Circular = the last node points at the first: no head, no tail, no wrap-around branch, and with one sentinel no `None` anywhere.** `while node is not None` **never terminates** — compare against the starting node or count. Good for round-robin, playlists, turn order, ring buffers, and **Josephus** (simulate in $O(n \cdot k)$, or use `survivor(n) = (survivor(n-1) + k) mod n` in $O(n)$ time and $O(1)$ space). Real ones: `collections.deque`, `OrderedDict`, the Linux kernel's `list_head`.

Day 086 · dsa — Linked lists revision and mock round

- **Six trigger questions cover eight days.** *Can the first node change?* → **dummy head**. *A position I cannot count to, in one pass?* → **runner** (speeds for the middle, a gap for the end). *Might it not end?* → **Floyd's**. *Turning the arrows?* → **three-pointer reverse**. *Combining or ordering?* → **merge**. *Do I already hold the node?* → **doubly linked**. Diagnose first, then repair.
- **The five sentences.** *Stand at $k-1$.* · *Save the next node before you overwrite the pointer.* · *Return `dummy.next`, never `head`* — and a bare function must return the new head. · *Insertion is $O(1)$ given the node.* · *Advance only when you keep.*
- **Composites are the real questions, and they are two or three techniques each.** Reorder = runner + reverse + interleave. Palindrome in $O(1)$ = runner + reverse + compare + **reverse back**. Copy with random pointers = a node map, or **interleave so the copy of X is `X.next`** and then unweave to **restore the caller's list**. Sort = split (first middle + **cut**) + merge.
- **Almost everything here is $O(1)$ space** — if yours allocates, say what you traded. The exceptions are the recursive reverse ($O(n)$ **stack**, `RecursionError` at ~1000) and merge sort's $O(\log n)$. The measurements: traversal **2× slower** than a list, ~59× faster at front insertion, **48 B vs 8 B** per element, and the LRU read ~10⁴× worse with a singly linked list.
- **The traps that end rounds:** overwriting before saving · `return head` · discarding the returned head · `.next.next` unguarded · the head as an `if` instead of a dummy · advancing after a deletion · the **second** middle when splitting · the missing **cut** · returning the meeting point as the cycle start · `=` instead of `is` · `while node is not None` on a circular list · one direction updated · an unterminated built list · a heap with no tie-breaker · and **mutating the caller's list in a read-only operation**.

Low-level design case studies

Day 078 · system-design — Design a parking lot

- **The interesting part is spot allocation, and everything else is bookkeeping.** Version one is a scan inside `ParkingLot.park`; it breaks on the first policy change (spread across floors · reserve a floor · pass holders). Pull it behind `SpotAllocation` and show **two** implementations — *one implementation is a class, two is a demonstration*. Same for `PricingPolicy`. **Seven classes, two interfaces.**
- **Keep policy out of bookkeeping.** `Floor.free_spots(size)` answers a *factual* question — spots of exactly this size. The **fallback rule** (a car may take a bus bay only if no car bays are free) is *policy* and lives in the allocation implementation. `Size` as an `IntEnum` makes the fallback a comparison.
- **The numbers that shape the design.** 1,000 spots $\approx$ 441 KB $\rightarrow$ *the whole model fits in memory, so do not design around a database*. 5,300 vehicles/day $\approx$ 0.18 events/s, $\sim$ 4/s at peak $\rightarrow$ *one lock is adequate; being clever here would be a mistake*. Change it at $\sim$ 10,000 spots or $\sim$ 50 arrivals/s: free-spot lists instead of scans, and a lock per floor.
- **Raise the race yourself: find-then-claim is two steps $\sim$ 151 μ s apart, $\approx$ 3 double-allocations a day.** Hold one lock across both steps, and keep the guard inside `Spot.assign` as the last line of defence. Across processes: `UPDATE ... WHERE id = ? AND plate IS NULL`, 0 rows updated = you lost, retry. `Ticket` is frozen and `unpark` uses `pop`, so it cannot be settled twice.
- **“Free” and “usable” are different numbers** — a bus is turned away while three motorcycle bays stand empty, so the board reports **per floor and per size**. The guaranteed extension, **EV charging bays**, costs **one class**: **spot features**, **vehicle requirements**, a subset test in the policy — and `ParkingLot` does not change, which is the entire payoff.

Day 079 · system-design — Design an elevator system

- **Separate the two scheduling problems before designing anything: *which car answers a hall call* (system-level, dispatch) and *in what order does one car serve its stops* (per-car, ordering).** Merging them is the standard mistake. A **hall call** carries a floor and a direction and any car may take it; a **car request** carries only a floor and belongs to that car alone.
- **The per-car answer is LOOK, the elevator algorithm: keep going while anything ahead wants this direction, then turn.** Against stops at 2, 9, 3, 8, 4 from floor 1: FCFS 23 floors · nearest-first 8 but it STARVES · LOOK 8 and cannot starve. In seconds, 46 vs 16 of travel — about 35 percent shorter journeys. The strict-to-the-end variant is SCAN; disk schedulers use the same algorithm.

- **Keep TWO stop sets per car, `up_stops` and `down_stops`**, because floor 7 means different things to someone going up and someone going down. Then `next_stop` is `min(above)` while going up, and the turn is `max(below)`. Every arrival goes through a **doors-open** state and every direction change through closed doors — that is the safety property, drawn.
- **The estimate is the part that makes this prompt different — do it before the classes.** 2 s/floor, 8 s/stop, 13 capacity → round trip **88 s** carrying 8 → 3 cars ≈ **982 people/hour against 800 demanded**, average wait ≈ **15 s**. Total system state ≈ **3 KB**: *there is nothing to scale here; the constraint is physics.*
- **The interface goes on `Dispatcher` and NOT on the stop ordering** — two real dispatch policies exist (nearest-suitable, zoned) and no second ordering algorithm does. **Hall calls must be idempotent** (five presses on floor 4 = one call, or five cars converge). **A full car declines hall calls but still serves its own passengers.** Next improvements in cost order: reassignment · lobby parking at up-peak · zoning · **destination dispatch** (25–30% faster) · a fourth car. **Fire mode pre-empts everything and must not be schedulable.**

Day 080 · system-design — Design an ATM

- **Three parts: a state machine, a dispenser, and a failure protocol — and only the third is the question.** State machine with **refusal by default** (fail closed), **every path returns the card**, and `Dispensing` **has no cancel** because a motor cannot be rolled back.
- **Debit first, then dispense, then confirm — and the reason is asymmetry, not convention.** Cash out with no debit is *unrecoverable*; a debit with no cash is *reversible*. At 0.5% of 200/day across 5,000 ATMs that is **5,000 failures a day** either way — about **₹1.5 crore a day unrecoverable** under the wrong ordering, against an automated reversal pipeline under the right one.
- **On a timeout you do not know whether the debit happened — silence is not “no”.** Write `AUTH_UNKNOWN` to the **electronic journal** *before* acting, queue a **reversal**, return the card. Every transaction carries a unique id (**STAN**, in ISO 8583) so retries are idempotent. `authorise` **is safe to retry**; `dispense` **never is**. The journal, not the network call, is the source of truth at reconciliation.
- **Greedy note selection breaks when a cassette runs low:** ₹600 with ₹500×1 and ₹200×3 — greedy takes the ₹500 and strands ₹100; the answer is ₹200×3. **Greedy first, bounded search as fallback** (<1 ms, a few percent of the time). **Decide the whole plan before moving one note** — a half-dispense cannot be undone. The **reject and retract bins** exist so the cash count still balances.

- **The PIN is never seen by the ATM** — encrypting keypad, verified by an HSM at the bank; the ATM tracks only the attempt count. **Nothing here is atomic and nothing can be:** a transaction cannot span a motor. The design *picks which side takes the risk, writes every step before doing it, and settles at a daily count* — capacity ₹32.5 lakh, ~200 withdrawals/day, replenished every ~5.4 days.

Day 081 · system-design — Design a vending machine

- **The rule that carries the design: verify the change BEFORE dispensing the item.** Same principle as the ATM even though the order looks opposite — **do the reversible thing first, the irreversible thing last.** ATM: the debit is reversible, cash is not → debit first. Vending: the money is already held, the item is irreversible → item last.
- **Cannot-make-change is a REFUSAL TO START, not a failure.** Sold out · not enough money · cannot make change are three arrows that leave `Collecting` and come **straight back to it:** the money stays in, the customer chooses again. The “exact change only” lamp is a machine-level approximation of a **per-item** truth — the same coins may buy a ₹25 item and not a ₹22 one.
- **Hold inserted coins in ESCROW, not in the box.** Then cancel returns *exactly what went in* and is correct even after a power cut — and the escrow coins still count as available change, which real designs forget. **Cancel must be legal at any point** before dispensing.
- **Greedy change, bounded search as fallback** (greedy strands you when a denomination empties); **compute the whole plan before anything moves; never retry a jammed motor** — refund fully and **disable the slot**, or fifteen more customers hit the same jam.
- **The float runs out before the stock does: ~₹576/day of change against a ~₹800 float ≈ 1.4 days, while 320 items at 120 sales/day ≈ 2.7 days.** Levers: bigger float · **prices in multiples of five** (removes ₹1 and ₹2 from the problem) · per-slot telemetry (the top 3 slots empty in **half a day**) · and the real fix, **cashless**, which deletes change-making entirely and moves the ordering back to *authorise → dispense → capture*. Total state ≈ 7 KB, **one customer at a time, no concurrency**.

Day 082 · system-design — Design a library management system

- A “book” is two classes, and this is the modelling insight the round is testing. `Book` is the **title** (one per ISBN); `BookItem` is a **physical copy** (barcode, rack, condition, status). The payoff sentence: **reservations queue for the title, loans are of a copy** — and on the diagram those two arrows go to *different classes*.

- The fine rule lives in a `FinePolicy`, and you win the question by rejecting each wrong home with a reason. Not `Book / BookItem` (a fine is not a property of a novel) · not `Member` (a person does not know rates) · not `Loan` either, because computing a fine needs the **tier rate, the cap, the holiday calendar and today's date** — four things that change without any loan changing. The tell is the constructor: the policy needs a member directory and a calendar.
- Draw the line: `days_overdue` belongs on the loan; `rupees_owed` does not. The loan is not anaemic — it answers `is_overdue`, `days_overdue`, `can_renew` from its own fields. It just does not own money. Matching policy on the way out: `LendingPolicy` for limits and periods per tier.
- `HELD` is a real state, distinct from available and loaned, and holds must EXPIRE. Return a copy → if the queue is empty it goes available, otherwise it is **held for the front of the queue for ~3 days** and then offered to the next member. No expiry means one absent member blocks a copy for ever; no `HELD` state means a walk-in gets a promised copy.
- The numbers that justify the design rather than decorate it. 50k titles + 80k copies + 20k members ≈ **42 MB, all in memory**; traffic ≈ **one operation every 12 seconds** — not a scale problem. The library is shut **18% of days**, so naive date subtraction overcharges a short overdue by up to **50%**. **An uncapped fine is uncollectable** — ₹2,190 of fine on a ₹450 book — so cap at replacement cost. The last copy is a **find-then-claim** race: the claim must be a **conditional update**.

Day 083 · system-design — Design tic-tac-toe, and then chess

- The $N \times N$ question is the real question, and the answer is one observation: a move can only change the lines through the square it was played on. Keep a running count per row, per column, and for the two diagonals; encode one player +1 and the other -1; a line is won when `abs(count) == n`. **$O(1)$ per move** against $O(N^2)$ — **8 operations vs 2,000,000 at $N = 1000$** — for $O(N)$ memory, about **0.2%** of the board.
- The two diagonal updates are two separate `if` s, never `if/elif` — the centre of an odd board is on both, and an `elif` drops one, which is invisible except on a diagonal win through the middle. Guard three things before every move: **on the board · not taken · game not already over**.
- `WinCondition` earns an interface because k-in-a-row is a real second implementation — and it does NOT share the counters' algorithm. A run of 5 inside a row of 15 is invisible in a row total, so it scans outward from the played cell in **four** directions (each direction plus its opposite), $O(k)$ per move. *The two rules share a question, not an implementation.*

- **Chess: legality is not a property of a piece.** Three reasons — the **path** (needs the board), **check** (a pinned piece may not move though nothing about it changed), and moves involving **two pieces or the past** (castling, en passant, promotion). So: `Piece.pseudo_legal_moves` for geometry, and `Game` filters by **make** → **test is_attacked** → **unmake**. The method name `pseudo_legal_moves` rather than `can_move` carries the design.
- **Make-and-unmake instead of copying, and the reason is a number:** ~35 moves per position × 512 B is **18 KB of copying per position**, so a million-position search would need **18 GB/s**. And **remember the state people forget** — side to move, **castling rights**, **en passant square**, **half-move clock**, move number: under 4 bytes, and without them castling and the fifty-move draw are unimplementable. **Checkmate and stalemate are one test plus one question:** no legal moves, and are you in check.

Day 084 · system-design — Design a deck of cards and a card game

- **A card is a value object: frozen dataclass over `Rank` and `Suit` enums** — immutability, equality by value and hashing all come free, and a typo fails at construction rather than silently. Deliberately **not orderable**, because “is the king above the ace” has no answer until a game is named.
- **A card has NO value, and this is the decision the prompt is testing.** Ace = 1 or 11 in blackjack, high in poker, variable in rummy; a king is 10 in one game and merely beats a queen in another. So `Rank` is an **identity** and the **game** supplies a `HandEvaluator` — and the proof is that a blackjack evaluator returns a *number* while a poker evaluator returns a *category*, so “value” was never one idea.
- **The naive shuffle is biased and the argument is counting, not statistics.** Swapping each position with a random position in `0..n-1` gives **n^n paths for $n!$ orderings**, and n^n is not divisible by $n!$ — for $n = 3$ that is $27/6 = 4.5$, so three orderings land at **4/27 (14.8%)** and three at **5/27 (18.5%)** against a uniform 16.7%. **Fisher-Yates** picks from `0..i` instead: **exactly $n!$ paths, one per ordering**.
- **If money is involved the seed matters more than the algorithm.** $52! \approx 8.07 \times 10^{67}$ (~226 bits), but a **32-bit seed reaches only 4.3×10^9 shuffles** — 5×10^{59} of them — and Mersenne Twister is reconstructible from **624 outputs**. Use `secrets / SystemRandom`, log the seed, and **inject the RNG** so the deck is both testable and swappable.
- **Blackjack’s only interesting rule: count every ace as 11, then subtract 10 while bust** — correct for `A A A A` ($44 \rightarrow 14$), and a per-card decision is impossible because an ace depends on cards dealt *after* it. Return `soft` too, because the dealer rule is “stand on hard 17, hit on soft 17” — and that rule lives on the **game**, not the evaluator. The **cut card at ~75%** exists because a counter’s edge is ~1.5% at full penetration and ~0.5% at three quarters.

Day 085 · system-design — Design Splitwise

- **Money is integer paise, never floats — and the remainder must be assigned deterministically.** ₹1,000 ÷ 3 = 33,333 paise each with **one paisa left over**; sort the participants and give it to the first (or to the payer), then **assert** `sum(shares) == total`. A dropped paisa per 3-way split across 50M expenses is **₹5 lakh** and, worse, balances stop summing to zero.
- **Double entry: the ledger is the truth, the balance is derived.** `Expense` and `LedgerEntry` are **immutable**, the entries for one expense **sum to zero**, and every group's balances therefore sum to zero. Materialise balances for speed, update them with `net = net + delta` **inside the SQL** (not read-modify-write), and run a **reconciliation job** — if they disagree, the ledger wins. An edit is a **reversal plus a new expense**, never an overwrite.
- **Minimising transactions is two steps.** *Net each person* — one number instead of a debt per pair, which alone collapses mutual debts. Then *repeatedly match the largest creditor with the largest debtor* with two heaps, $O(k \log k)$. **Every payment zeroes at least one person, so at most $k - 1$ payments:** 6 people → 5, against 15 possible pairs; 20 people → 19 against 190.
- **Say it is NOT optimal.** The exact minimum needs zero-sum subsets found and separated, which is the **set-partition problem — NP-hard**. “At most $k - 1$, in one pass, not guaranteed minimal” is the strong answer; claiming optimality is the most common mistake in this prompt.
- **Simplification destroys provenance, and that is a design decision not a bug** — you end up owing someone you never transacted with. Make it a **per-group setting**, produce **suggested settlements** rather than overwriting the ledger, and keep the unsimplified view derivable. Scale is a non-issue: 10M users and 250M ledger entries ≈ **19 GB**, so spend the interview on the arithmetic and the edit path.

Day 086 · system-design — Design BookMyShow

- **A seat is two classes:** `Seat` (physical, in a screen) and `ShowSeat` (that seat, for that show). Availability, price, `locked_by` and `locked_until` live on the second. The map you draw is a `ShowSeat` query and the row you lock is a `ShowSeat` row — locking the physical seat would block it for every show in the building. Same split as `Book` / `BookItem`.
- **Selecting is not booking:** AVAILABLE → LOCKED (with an expiry) → BOOKED, and **both** arrows out of LOCKED are required — payment failure, and the expiry. *A lock with no expiry is a leak.*

- **The race is fixed by making the check and the claim one statement.** `UPDATE ... SET LOCKED WHERE status='AVAILABLE' OR (LOCKED AND locked_until < now)`, then **check the affected row count** — fewer than requested means roll back and tell the user. Check-then-act at a 2 ms gap and ~3,000 attempts/s gives roughly **90 double-sold seats per second**. Multiple seats: **one statement, all-or-nothing**; if you ever take row locks, **order them by seat id** or you deadlock.
- **Expire locks LAZILY, in the WHERE clause** — then correctness needs no scheduler; a sweeper is only cosmetic. And **the confirming update must also be conditional** (`AND status='LOCKED' AND locked_by = user`), or a late payment overwrites somebody else's booking — the worst failure here — and when it affects zero rows, **refund automatically**.
- **Reads and writes need opposite answers.** ~100,000 reads/s on a hot show's 200 rows, on data that may be seconds stale → **cache** (every seat map in the country is ~2 GB); ~3,000 claims/s that must be exact → **never cache**. Sharding does not help — *there is one show* — so the real lever is **admission control (a virtual waiting room)**. The hold duration is the main knob: **10 minutes**, because a card-plus-OTP flow is 1–2 minutes, and 20% abandonment means hundreds of seats are dead inventory at peak.

Day 087 · system-design — Design a food delivery order flow

- **Everybody draws the eight states; the column that matters is next to them — WHO causes each transition, and WHAT HAPPENS IF THEY DO NOT.** Three parties (customer, restaurant, partner) plus **the clock as a third actor**. *A transition performed by somebody else with no timeout is an order that sits for ever, and silence is what customers actually complain about.*
- **The restaurant timeout is a feature, not an edge case:** ~5% of 2M orders/day = **100,000 auto-cancels a day**, each needing a refund, an *immediate* notification, and a metric that feeds restaurant ranking. Implement time as **one sweeper plus a per-state rules table** (`state_deadline`, indexed, `FOR UPDATE SKIP LOCKED` so instances share the work) — one thing to keep correct instead of six that fail quietly. **Not lazy expiry**, because something must actively *happen*.
- `OrderItem` is a **SNAPSHOT of name and price, not a reference to the menu** — and store every money component, not just the total. ~2% of menu items change daily → ~40,000 orders/day touch a changed item, which is where billing disputes come from. Same immutability rule as a loan and a booking.
- **Cancellation is a question you ask the state, and the answer also depends on the actor.** Placed → full refund; preparing → allowed, no refund (ingredients spent); picked up → refused, *the food is on a scooter*. Expect the percentages to move to configuration; the *shape* stays.

- **The one genuinely interesting trade is when to assign the partner.** At **ACCEPTED**: the partner waits ~ 8 min $\rightarrow \sim 266,000$ partner-hours/day. At **READY**: the food waits ~ 6 min and arrives cold. So assign at **ACCEPTED + prep - travel**, which makes it a scheduled job and moves the difficulty into a *prediction* system — name that as separate rather than pretend it is easy. And at **111 orders/s**, this is a correctness problem, not a throughput one.

Day 088 · system-design — Design a ride-hailing booking flow

- **Split it in two before designing anything: the location firehose and the trip records.** $100,000 \text{ drivers} \times \text{a ping every } 4 \text{ s} = 25,000 \text{ writes/second}$, $\sim 600 \times$ the booking rate, with **zero durability requirement** $\rightarrow$ Redis geo set with a TTL. Trips at $\sim 42/\text{s}$ must never be lost $\rightarrow$ the database. *The hard-looking part is a different system.*
- **An offer to a driver is a LOCK with a short expiry** — while it is out, that driver is unavailable to every other request, exactly like a cinema seat. Claim with a **conditional update** (`WHERE status='ONLINE' OR offer_expires < now`), **0 rows = you lost**. Same find-then-claim as the parking spot, the seat and the shipped order.
- **Sequential vs batched is a real trade with real numbers.** 60% acceptance $\rightarrow$ sequential averages **1.7 offers ≈ 14 s**; a batch of 3 accepts in the first round **94%** of the time ≈ 6 s. And **release the losers immediately** rather than letting them time out — at 42 matches/s that is ~ 250 drivers locked instead of $\sim 1,260$, a full percent of supply.
- **Spatial queries need a spatial index: geohash / S2 / H3, because separate lat and lng indexes cannot answer a circle.** Cells have edges, so read the cell and its **8 neighbours**, then filter by true distance. Give every driver key a **staleness TTL**, or a dead phone gets offered rides for ever. And **rank by predicted ETA, not distance** — 500 m across a river beats 2 km down the road.
- **Freeze the fare quote, and the surge multiplier especially.** Surge $1.4 \times \rightarrow 2.0 \times$ over an 18-minute trip is **+43%, about ₹107 on a ₹250 fare** — a trust incident, not a pricing decision. Same immutability rule as the menu price and the library fine. And **“no driver found” is a designed outcome** ($\sim 2\%$ of requests, $\sim 40,000/\text{day}$) with a screen, a metric and a product decision behind it.

Day 089 · system-design — Design a rate limiter, at the object level

- **Return a `Decision` (allowed, remaining, retry_after), not a boolean, and inject now.** A client that knows when to come back backs off correctly; a limiter that reads its own clock cannot be tested without sleeping.

- **Fixed window's flaw is the boundary and it is aimable: the full limit at 10:00:59 plus the full limit at 10:01:00 is $2 \times$ the limit in ~ 1 second.** Fine for a coarse billing quota, disqualifying for anything security-adjacent.
- **Memory is the deciding number.** Fixed window / sliding counter / token bucket are **2 numbers per key** ($\sim 100\text{--}120\text{ B} \rightarrow \sim 100\text{ MB}$ at 1M keys). **Sliding log is exact and keeps a timestamp per request** — 800 B/key at a limit of 100 $\rightarrow$ **800 MB**, and it scales with the *limit*: **80 GB** at 10,000/hour. Cloudflare's sliding **counter** approximates to within $\sim 1\%$ for two numbers.
- **Default to the token bucket.** Two numbers, no boundary, **lazy refill** (nothing runs in the background), and — the real reason — **rate and burst are independent knobs**: “100/min with bursts of 20” = rate 1.67/s, capacity 20. *Token bucket limits what you accept; leaky bucket shapes what you emit.*
- **Distributed: INCR then EXPIRE is the bug everybody ships** — a failure between them leaves a key with no TTL and that caller limited **for ever**. Use a **Lua script** (atomic read-modify-write, TTL derived from $\text{capacity} \div \text{rate}$). Add a **local bucket in front** if 0.2 ms per request matters. And **fail open** when the store is down — a limiter sits in front of everything, so failing closed turns a blip into an outage.

Day 090 · system-design — Design an in-memory cache with eviction

- **Three mechanisms, kept apart — merging them is the standard mistake.** Eviction = capacity (policy chooses a victim) · **Expiry** = correctness (TTL, regardless of capacity) · **Invalidation** = an external event on one key. *An entry can be expired and resident, and evicted while fresh.*
- **Entry carries value, expiry and size — and NO policy bookkeeping.** Recency and frequency live in the installed policy, or swapping the policy means changing the entry. The `EvictionPolicy` interface has **four genuinely wanted implementations** — LRU, LFU, FIFO, Random — which is exactly the gate an interface must pass.
- **Expiry needs BOTH lazy and sampled.** Lazy on read is correct and costs nothing but **leaks** — an entry nobody reads is never noticed. So sample ~ 20 keys $\sim 10 \times$ /second and repeat while $> 25\%$ are expired (Redis's adaptive rule). An **expired hit is a MISS**, or the hit-rate metric lies.
- **The stampede is the failure that matters, and the policy is not.** A hot key at 5,000 req/s with a 200 ms load = 1,000 concurrent identical queries, exhausting a shared pool and breaking unrelated work. Fix with **single-flight**: a **per-key lock**, and **re-check inside the lock**. Then **jitter the TTLs** — 1,000 keys with identical TTLs all expire in the same second, and one line of randomness spreads them.

- **The hit rate is the only number that makes any of this tunable:** at 100 μ s vs 1 μ s, 99% $\rightarrow$ 50 $\times$, 95% $\rightarrow$ 17 $\times$, 90% $\rightarrow$ 9 $\times$, 50% $\rightarrow$ 2 $\times$ — the value falls off a cliff below $\sim$ 95%. **Count bytes, not entries, when values vary** (100,000 entries is 200 MB at 2 KB each and 200 GB at 2 MB each). And **Random is within a few points of LRU** — which is why Redis only *approximates* LRU by sampling.

Day 091 · system-design — Design a logging framework

- **Handler** (WHERE) and **Formatter** (WHAT IT LOOKS LIKE) are separate interfaces, and that is the answer to the prompt: adding a destination is **one class, one method, one line of wiring**. Combined, you would need a class per combination — **N $\times$ M instead of N + M**. Each handler carries its **own level**, so a file can capture DEBUG while the console shows only warnings.
- **The level check must be the FIRST statement, and the API exists to make deferred formatting possible.** `logger.debug("user %s", uid)` defers; `logger.debug(f"user {uid}")` **always builds the string**. At 200,000 calls/s with 80% suppressed and an expensive argument, that is **8 seconds of CPU per second — eight cores — discarded**, against $\sim$ 8 ms/s done properly.
- **Dotted hierarchical loggers with upward propagation let you turn one subsystem up in one line of config.** Structurally a chain of responsibility, but **every ancestor gets the record**, not just the first — `propagate = False` cuts it.
- **Log events, not sentences, and carry a correlation id.** 200,000 lines/s is **17 billion a day**, and one request's 20 lines are scattered through it — with the id it is an indexed lookup, without it it is impossible. Use a `ContextVar` (async-safe), attached by the framework, never threaded through signatures. JSON costs $\sim$ 40% more bytes and makes the data queryable at all.
- **Emit through a BOUNDED queue on a background thread — synchronous network logging is arithmetically impossible** (40,000 lines/s $\times$ 0.5 ms = 20 s of I/O per second). It must **drop, not block**, and **count the drops** (a 1 s collector stall loses $\sim$ 30,000 records). A crash loses the queue — the logs you most want — so **flush on shutdown and write CRITICAL synchronously**. And a **logger must never raise**: wrap every emit, fall back to `repr`, and **never interpret user data as a template** (Log4Shell).

Day 092 · system-design — Design a notification service

- **Callers send events, never messages.** `notify(user, ORDER_SHIPPED, data)`. That one rule means the service owns every word, and a new channel or language touches no caller. Then five decisions in order: **idempotency $\rightarrow$ preferences $\rightarrow$ channels $\rightarrow$ render per channel $\rightarrow$ enqueue one notification per channel**.

- **Two interfaces, not one.** `Channel` answers the guaranteed follow-up “now add WhatsApp” — one class, one registration line, zero changes to the dispatcher. `RetryPolicy` is the one candidates miss: **a shipping update retries with backoff; a password reset must not be retried after the code expires**, because a late OTP is worse than none.
- **SENT $\neq$ DELIVERED**. Sent means the provider accepted it; delivered arrives later by webhook and only some channels report it. And the queue is **at-least-once**, so the worker must be idempotent: a **compare-and-set** on the status before sending, plus a **unique constraint** on the caller’s idempotency key.
- **Classify every provider error as retryable or permanent.** A timeout is retryable; “no such number” is not, and retrying it five times costs five times the money for a message that can never arrive. Add **jitter** to the backoff, or a brief outage returns as one synchronised spike — a thundering herd.
- **The numbers:** $10\text{M users} \times 5/\text{day} = 50\text{M notifications/day} = 578/\text{s}$ average, $\sim 1,700/\text{s}$ peak, $\times 1.8$ channels $\approx 3,000$ provider calls/s. Log at 300 B/row = **15 GB/day, 5.5 TB/year** $\rightarrow$ keep 90 days hot. And **SMS at ₹0.15 $\times$ 2.5M/day $\approx$ ₹11M/month**, which is why channel choice is a cost decision. **Broadcast needs its own low-priority queue**, or it starves password resets.

Day 093 · system-design — Design a file system

- One abstract `Entry` with a name and `size()`; `File` and `Directory` both extend it, and a `Directory` holds a `dict[str, Entry]` — so a directory can hold directories. That shared type is the design: `du` is four lines and `find` is six, with **no `isinstance` anywhere**.
- Every operation is “**resolve the path, then do one small thing.**” Resolution is one dict lookup per component. It has **four outcomes, not two**: missing directory, a file where a directory was expected, missing final component — and for `mkdir` that last one is the work, not an error. And `ls` **on a file returns the file’s own name** — the detail everybody forgets.
- **dict of children, not a list.** Lookup is on every component of every call: 100,000 files in a directory makes a five-part resolve **5 ms instead of 250 ns**. The price is that `ls` sorts, which is the rare operation. **Names are unique within a directory, not globally.**
- **File content is a list of chunks, joined lazily** — `content += text` copies the whole string each time, about **5 GB of copying for 10,000 appends of 100 bytes**. Lock **per file**, not globally; the two real races are two `mkdir`s of the same directory (one silently discards the other’s contents) and a read collapsing chunks during an append.

- **A million files $\approx$ 300 MB of metadata before any content** — the reason HDFS's NameNode is the limit of a Hadoop cluster. `find` is a full subtree walk; making it fast means a second name $\rightarrow$ paths map, $\sim$ 150 MB per million and a write on every mutation. **S3 has no directories** — the whole path is one flat key — so renaming a folder is $O(1)$ here and a million copy-and-deletes there.

Day 094 · system-design — Design snake and ladder

- **A snake and a ladder are the same object.** One `dict[int, int]` from square to destination; `jumps.get(cell, cell)` defaults to the square itself so there is no branch for “no jump”. `is_ladder` is **derived** from `end > start`, never stored. Two classes here doubles every piece of logic and buys nothing.
- **Dice is an interface with a `ScriptedDice` for tests** — otherwise no rule in the game is testable, only surveyable. **Randomness, time and I/O are the three things a design must let you replace.** It is also what moves the roll to the server for an online game, where a client must never choose its own number.
- **The rules that vary are data, not branches:** exact finish, extra turn on six (capped at three), chained jumps, entry rule — all fields on `GameRules`. And `play_turn` returns a `TurnResult`, **never prints**, so one `Game` serves a terminal, a phone and a simulation.
- **The win check comes AFTER the jump** — a ladder can win the game. **Chained jumps must be validated at load time** (no jump may end on another jump's start) or 30 $\leftrightarrow$ 60 loops for ever. Any “play until someone wins” loop needs a **turn cap**, because the distribution has a long tail.
- **The numbers:** one game $\approx$ 2.3 KB, **board shared not copied** $\rightarrow$ 100,000 concurrent games $\approx$ **110 MB**; mean $\sim$ 39 turns to finish, 90th percentile $\sim$ 72; $\sim$ 12,000 turns/second is nothing in CPU — the real constraint is **400,000 WebSockets $\approx$ 8 servers**. A game is **inherently serial: one writer per game**, so use a **version compare-and-set** plus a turn number for idempotency, never a lock.

Day 095 · system-design — Design an online auction

- **A bid is a MAXIMUM, not a price.** The visible price is the **loser's** maximum plus one increment, capped at the winner's — so ₹1,200 usually wins at ₹950. Consequences: a bid can be **accepted and instantly losing**, a losing bid **still raises the price**, and **maximums must never leak** anywhere. Ties go to the **earlier** bid.
- **Two bids at once: a conditional write, never read-then-write.** `UPDATE ... WHERE id = ? AND version = ?` — one row means you won, **zero rows means re-read and recompute**. Read-modify-write is a **lost update**: one bid vanishes and its bidder is told they are leading. Not a Redis lock — the row is already the single source of truth.

- **The end TIME is the authority, not a status column.** `now < ends_at` is checked inside the same transaction as the write, so a late closing job cannot accept bids it should not. The job does only side effects, and `ENDED` and `SOLD` are **separate states** because payment can fail.
- **Anti-snipe: a bid in the final window extends the end by that window, capped.** Policy, so it lives in a rules object with the tiered increments. The price the design pays is that the end time stops being predictable.
- **The numbers reframe it: 10M auctions × 12 bids ÷ 7 days ≈ 200 bids/second globally — nothing.** The problem is ~170 writes/second to ONE row in a hot auction's last 30 seconds. At a 2 ms window that is a ~34% collision rate, ~1.5 attempts expected, 0.5% exhausting five retries. For the hottest listings, **one queue partition per auction id = one writer, zero contention.** Cache the price for **display** (1–2 s TTL, ~33:1 read:write) — **never for the decision.**

Day 096 · system-design — Low-level design revision and full mock

- **Five moves, always, in this order: clarify (0–5) · nouns (5–10) · class diagram (10–20) · the interesting part (20–30) · code (30–40).** Moves 1–3 are table stakes. **You are marked on move 4.** Find it by asking: *which sentence in the prompt would a product manager change next year?* Put an interface there and show **two** implementations.
- **Fifteen of eighteen case studies had a strategy at their centre** — `EvictionPolicy`, `PricingStrategy`, `SplitStrategy`, `Channel`, `Dice`, `SpotAllocationStrategy`. The tells: “*depending on*” → **strategy**; “*and then it becomes*” → **state machine**; “*notify*” → **observer**; “*contains other*” → **composite**.
- **Randomness, time and I/O must be replaceable** (`Dice`, `Clock`, `Channel`) or nothing is testable. **Return a result object, never print** — one engine then serves a terminal, a phone and a simulation. **Local rules on the entity, global rules in a service.**
- **Chess: legality has two layers.** `Piece.candidate_moves` is local and per subclass; `MoveValidator` applies each candidate, asks whether your own king is attacked, and **undoes** it — which is why `Move` must carry the captured piece. That one filter gives **pins, discovered check and escaping check for free**, and then **checkmate = `in_check` and not `legal_moves`, stalemate = not `in_check` and not `legal_moves`.**
- **The five ways to lose:** coding at minute three; silence; anaemic classes plus a God service; an **enum with a switch where polymorphism belongs**; and chasing completeness instead of naming what you left out. **Nobody finishes** — at minute 38, say what is missing and where each piece would attach.

Recursion and backtracking

Day 087 · dsa — Recursion: the leap of faith

- **Three questions, in order, and they *are* the function.** *What is the smallest input I can answer without asking anybody?* (the base case) · *What is a smaller version of the same problem?* · *Given that answer, what one small step gives me mine?* Write the base case **first, at the top.**
- **The leap of faith: assume the recursive call already works, and write only the line that uses it.** Do not trace it — nobody can hold four levels in their head. **Three checks replace tracing:** the base case returns the right answer · every call is on a **strictly smaller** input · the combining step is right.
- **A base case can be present and unreachable, which is the harder bug.** Stepping by two towards `= 0` never terminates on odd inputs. Say both halves: *it exists, and every input can reach it.*
- **Count the recursive calls in the body: one is a line, two is a tree.** `total` makes one $\rightarrow O(n)$. `fib` makes two $\rightarrow \sim 2^n$: `fib(40)` is **~ 331 million calls, ~ 13 s**, and `@cache` turns it into **41 calls** — that is memoisation, and it is the foundation of DP.
- **Recursion always costs $O(\text{depth})$ stack, and Python's limit is ~ 1000 frames** — so a linear-depth recursion dies on a **5,000-element list**, and `sys.setrecursionlimit` converts a clean `RecursionError` into a **segfault**. *Log-depth recursion is free; linear-depth is a bug waiting for a big input.* And **never slice in the recursive call** — `f(items[1:])` copies every time and makes an $O(n)$ algorithm **$O(n^2)$** ; pass an index.

Day 088 · dsa — The call stack, drawn

- **A stack frame holds the arguments, the locals, and the return address** — where to resume in the caller. Frames pile up and come off in **reverse order**, and every frame except the top is **suspended mid-expression**. **All the work happens on the way back up**; nothing is computed on the way down.
- **The recursion TREE and the call STACK are different things, and this answers most questions here.** *Total calls determine the time; maximum depth determines the space.* Measured: `fib(20)` = **21,891 calls, max depth 20** — exponential in time, **linear** in space, because the left branch pops before the right one starts.
- **To trace, draw the stack downward and write what each frame is waiting for**, then read the two phases. The five-minute tool: add a `depth` parameter and indent the prints — the indentation *is* the depth. Trace the **smallest** input that shows the shape.

- Python's limit is 1000 frames, ~996 reachable from a script, so a linear-depth recursion dies on a 5,000-element list. Read the traceback from the **bottom** (most recent call last). **Never raise the limit** — it exists to turn a C-stack overflow into a catchable exception, so raising it gives a **segmentation fault** with no traceback. **Tail recursion does not help in Python**, deliberately, to keep tracebacks complete.
- The conversion to iteration always exists — build your own stack — but it is **only mechanical when the work happens BEFORE the recursive calls**. When it happens after, you must push `(item, visited)` pairs and handle each item twice; that flag is the return address by hand. And watch for **degenerate structures**: a balanced tree of 10^6 nodes has height ~ 20 , the same class fed sorted insertions has height 10^6 — the depth is a property of the input, not the algorithm.

Day 089 · dsa — Writing a recursive function that terminates

- “Why does it terminate?” is answered with the **MEASURE**, not the base case. Three conditions, all three needed: it **strictly decreases** every call · it is **bounded below** · the **base case fires at that bound**. “There is a base case” only covers the third — and an unreachable one is worth nothing.
- The measure need not be an argument — it is any **non-negative integer function of the arguments** that strictly decreases. Euclid's is `b`, not `a` (which can grow: `gcd(3,100) → gcd(100,3)`); a grid path's is `row + col`; Ackermann's is the **pair**, **lexicographically**. And in **mutual recursion** it must decrease **around the cycle**, not within one function.
- Three ways it fails, and only the first is obvious. The measure never moves (`start` instead of `start + 1`) · it decreases but **steps over** the bound (`n - 2` towards `= 0` on odd inputs — works for even, never for odd) · it decreases and **never reaches** the floor (a halving float interval). **Fix the step, not the guard** — widening to `≤ 0` turns a crash into a wrong answer.
- A graph has **NO natural measure**, so the visited set creates one: `|V| - |seen|`. It is the **termination argument**, not an **optimisation** — which is the whole reason graph DFS differs from tree DFS. Mark the node **before** recursing, and pass the set explicitly (**never a mutable default**).
- The measure also gives you the complexity, from the same sentence. Drops by 1, one call per level → $O(n)$. Halves → $O(\log n)$ (Euclid: ~ 90 calls at 10^{18}). Drops by 1 with two calls per level → $O(2^n)$. And **termination ≠ feasibility**: Ackermann provably terminates and `ackermann(4,2)` has 19,729 digits. If you cannot name a measure — **Collatz**, verified past 2^{68} and unproved — add an explicit limit and raise.

Day 090 · dsa — Recursion on arrays and strings

- **Pass an INDEX, never a slice.** `f(items[1:])` copies on every call — n calls $\times O(n)$ copy = $O(n^2)$ time *and* allocation, silently, with no error. `f(items, start + 1)` allocates nothing. In strings it is worse: the slice copies **and** every `+` builds a new string, so a naive recursive reverse is quadratic twice over ($\sim 5 \times 10^7$ character copies at $n = 10,000$).
- **Symmetric problems use TWO indices moving inward, and the base case is `low  $\geq$  high`.** Not `==`: on even lengths they **cross without meeting** — `"abba"` goes $(0,3) \rightarrow (1,2) \rightarrow (2,1)$ — so `==` alone recurses for ever on half of all inputs. The measure is `high - low`, dropping by two.
- **Converting a loop is a three-part recipe:** what *changes* between iterations $\rightarrow$ the index · what is *carried across* $\rightarrow$ an accumulator parameter · the *stopping condition* $\rightarrow$ the base case. Put the bookkeeping in a **private helper** so the public signature takes only what a caller would pass.
- **The shape decides whether it survives.** Shrink-by-one $\rightarrow$ depth n , dies at ~ 1000 . Split-in-half $\rightarrow$ depth $\log n$, 17 frames at 100,000. *Same input, opposite outcomes*. And divide-and-conquer on a flat array is $2n - 1$ calls to do what a loop does in n — it only pays when combining halves is cheaper than redoing the work.
- **On a flat array the loop wins on every axis** — same time, $O(1)$ space vs $O(n)$ stack, no depth limit, clearer — and **recursion is never truly $O(1)$ space** even when the array is untouched. Recursion earns its place on **recursive structure** (nesting depth, not length) and on **divide-and-conquer**. Say which you would ship.

Day 091 · dsa — Subsets: the include-or-exclude tree

- **Every element is an independent take-it-or-leave-it decision, so there are exactly 2^n subsets** — and you cannot beat 2^n because the output is 2^n things. Say the size first and the “can you do better?” question never happens. $n = 20$ is a million and comfortable; $n = 30$ is a billion and impossible, so a subsets problem always bounds n at twenty-something.
- **The pattern for the whole phase: choose · recurse · un-choose.** Write the three lines as one unit. The `pop` is **not cleanup** — it is what lets one working list serve every branch, and omitting it makes the “skip” branch still contain the element, with no error.
- **Append a COPY** — `current[:]` — **never the list itself.** `current` is one object mutated throughout, so appending it gives 2^n references to one list that is empty at the end: `[[], [], [], ...]`, and no error. Say “copy” as you type it. This is the defining backtracking bug.

- **Duplicates take two lines: SORT first, then skip when `items[i] == items[i-1]` AND `i > start` — `i > start` means “another choice at this level”, while `i > 0` would also block the second 2 from following the first and lose `[2, 2]`. Prune in the tree; never filter the result** — filtering builds all 2^n before discarding.
- **Time is the whole tree, space is the deepest path: ~2 million calls at $n = 20$, and only 20 frames. $O(n \cdot 2^n)$ time; $O(n)$ extra space beyond the output.** Three formulations, same complexity: the **recursive** one generalises (cap it $\rightarrow$ combinations, add a check $\rightarrow$ N-Queens), the **iterative doubling** one is two lines and has no copy trap (seed it `[[] ]`, not `[ ]`), and the **bitmask** one works to $n \leq 63$.

Day 092 · dsa — Permutations

- **n choices, then $n - 1$, then $n - 2$ — so $n!$ permutations, and you cannot beat $n!$ because the output is $n!$ things.** Count it out loud; never say “exponential”, because $n!$ is worse than 2^n and overtakes it at $n = 4$. $n = 10$ is 3.6 million and fine; $n = 13$ is 6 billion and impossible, so the bound in the question tells you which of the two is wanted.
- **start does not work here.** For subsets `[1,2]` and `[2,1]` are the same answer; for permutations they are different, so you look at every element every time and must track **which are already placed** — a `used` boolean per element.
- **Two chooses, two un-chooses: `used[i] = True`, `append`, `recurse`, `pop`, `used[i] = False`.** Forgetting the unmark gives **one permutation instead of $n!$** , silently. And `append current[:]` — a **copy** — or you get $n!$ references to one empty list.
- **Duplicates: sort, then skip when `items[i] == items[i-1]` AND `not used[i-1]`** — meaning “*my identical twin has not been placed yet, so this branch is a repeat*”. In one sentence: **among equal elements, place them left to right.** This is **not** subsets’ `i > start`. Prune, never filter — `set(list_of_lists)` raises `TypeError: unhashable type: 'list'`.
- **The swap version needs no `used` array:** fix positions left to right, swap `i` into `first`, `recurse`, **swap back**. $O(n)$ space total. But it **cannot use the adjacency rule for duplicates**, because swapping destroys sorted order — use a per-level `set`, or use the `used` version. And `itertools.permutations` exists, yields **tuples**, and does **not** de-duplicate.

Day 093 · dsa — Combinations and combination sum

- **Combination = order does not matter, so carry a `start` index and never look backwards.** One loop bound removes every duplicate ordering — no `used` array, no de-duplication. The invariant: every answer is generated in **non-decreasing order**, so there is exactly one path to it.

- **Sort, then break when `candidates[i] > remaining`** — valid only because it is sorted, and the sort is therefore not cosmetic. Unsorted, the `break` drops real answers: `[7,2,6,3]` target 7 gives `[[7]]` instead of `[[2,2,3],[7]]`.
- **The one character: `build(i, ...)` allows reuse; `build(i + 1, ...)` uses each element once.** Say which one aloud before typing it. `i + 1` on Combination Sum I gives `[[7]]` instead of `[[2,2,3],[7]]`, with no error.
- **Combination Sum II has TWO skips with TWO keywords:** `break` for too big, `continue` for `i > start` and `c[i] == c[i-1]`. `break` on the duplicate loses every later value — four answers become two. And it is `i > start` here, versus `not used[i-1]` in the permutations tree: same job, two tree shapes.
- **Depth is `target / smallest candidate`, not `n`** — the only problem in the phase where the stack can actually overflow (`RecursionError` at target 5000 with a candidate of 1), and a **zero candidate never terminates** because the measure stops decreasing. “How many” is never backtracking: it is `ways[t] += ways[t-c]`, $O(\text{target} \times n)$ — 1,500 steps instead of 50,000 lists.

Day 094 · dsa — Backtracking: the undo step

- **The invariant, and there is only one: when a call returns, every piece of shared state is exactly what it was when the call started.** The pop is **not cleanup** — `current` is one object shared by every branch, and the un-choose is what makes the *sibling* branch correct. A missing undo is **always silent**: subsets gets half its answers wrong, permutations returns **one answer instead of $n!$** .
- **State is three kinds: the partial answer, the availability marks, and derived bookkeeping** (a running sum, an occupied-column set). The third is where the bugs live. **Count the lines before the call and the lines after; they must match, and undo in reverse order.**
- **Anything passed as an argument needs no undo** — ints, strings and tuples are values, so `build(i+1, remaining - c)` has nothing to restore. Move state into the parameter list wherever you can; it is state you cannot forget.
- **The `if not is_valid: continue` BEFORE the apply is what makes it backtrack rather than enumeration.** Check on the way down, never at the leaf: **N-Queens at $n = 8$ is 19,173,961 nodes without it and 2,057 with it.** Every undo must be $O(1)$ — `board[r][c] = "."`, not a fresh copy — or the pruning stops being affordable.
- **Two styles: mutate-and-undo ($O(1)$ per node, must not forget) or pass-a-copy** (`explore(current + [x])`, cannot be got wrong, allocates at every node). Never mix them — popping after passing a copy gives `IndexError: pop from empty list`. And **undo before an early return**, or a reused grid reports no path when one exists.

Day 095 · dsa — N-Queens and constraint grids

- **The reduction is the answer: no two queens share a row, so there is exactly one per row.** That turns “choose n squares from n^2 ” — 4.4 billion at $n = 8$ — into “one column per row”, at most $8! = 40,320$. The representation follows: **a list of n column numbers, never a grid**, so a row conflict is unrepresentable rather than checked.
- **Every $\backslash$ diagonal has a constant $\text{row} - \text{col}$; every $/$ diagonal has a constant $\text{row} + \text{col}$.** So three sets — cols , $\text{row} - \text{col}$, $\text{row} + \text{col}$ — make the conflict test **three $O(1)$ lookups**. With lists instead of sets, $\text{row} - \text{col}$ is negative and Python indexes from the end: **silently wrong answers**, so add an offset of $n - 1$.
- **The `if` goes BEFORE the descent.** Checking on the way down visits **2,057 nodes at $n = 8$** ; checking at the leaf visits **19,173,961**. Same program, one line moved. This is the most convincing number in the phase.
- **Four changes, four restores** — three sets and the placement. Forget the anti-diagonal removal and `total_n_queens(8)` returns **0**, with no error. And **render the board only at a solution**: at $n = 12$ that is 14,200 renders instead of 856,000.
- **$O(n!)$ is a loose upper bound and there is no closed form** — quote measurements: $n = 8$ ms, $n = 12$ a second, $n = 14$ half a minute. **$n = 2$ and $n = 3$ have no solutions.** Space is $O(n)$. Faster: **bitmasks**, where `cols` is absolute but `diag` shifts left and `anti` shifts right each row ($5-10\times$); **symmetry** gives $2\times$. **Sudoku is the same program**: depth = next blank, choices = 1–9, sets = row, column, box at $(r//3)*3 + (c//3)$.

Day 096 · dsa — Sudoku, word search, and grid backtracking

- **Classify first: path or region.** A path problem (word search, maze, all paths) **clears the marker on the way out**, because a cell used by one route must be free for another — `0(rows*cols*3^L)`. A **region** problem (islands, flood fill) **never clears it**, because a cell belongs to one region for ever — `0(rows*cols)`. **One line, exponential versus linear.** Swap them and word search reports absent words; islands raises `RecursionError`.
- **Bounds check FIRST, and it does not raise if you skip it** — `board[-1]` is the last row, so the search silently wraps around the grid. Directions live in one tuple `((-1,0),(1,0),(0,-1),(0,1))`, never four copies of the call.
- **Mark by overwriting the cell with a sentinel, restore after** — zero extra space, and the restore is the un-choose. Use `found = any(...)` then restore, never `if search(...): return True` inside a loop — that **early return skips the restore** and leaves `#` in the grid for the next search.

- Complexity is $O(\text{rows} \times \text{cols} \times 4 \times 3^{(L-1)})$ — the 3, not 4, because one neighbour is always where you came from and it is marked. 6×6 grid: $L = 8$ is 315,000 paths, $L = 12$ is 25 million, $L = 16$ will not finish. Space $O(L)$.
- Two one-line prunes that remove exponential subtrees: reject if `Counter(word) > Counter(grid)`, and reverse the word when its last letter is rarer than its first ($36 \times$ fewer starting cells on a grid of A's with one Z). Sudoku is the same program — three sets, pre-computed blanks, and the most-constrained cell first, which is $\sim 100 \times$ on hard puzzles and free on easy ones.

Day 097 · dsa — Recursion and backtracking revision and mock round

- Two questions choose the template, and that is the whole first minute. *Does order matter?* Same answer for `[1,2]` and `[2,1]` → start index, never look backwards; different → used array, look at everything. *Reuse allowed?* Yes → recurse on `i`; no → `i + 1` or `used[i]`. Four combinations, four templates, and everything else sits on top of one of them.
- The five rules that survive the phase: append a copy; count the chooses and the undoes and undo in reverse; prune before descending, never at the leaf (N-Queens $n = 8$: 2,057 vs 19,173,961); anything passed as an argument needs no undo; and say the output size first — 2^n , $n!$, $C(n,k)$ — which removes “can you do better?”.
- The duplicate rule, three outfits, one sentence: *do not start two branches with the same value at the same level*. `i > start` in the start-index tree, `not used[i-1]` in the used-array tree, a per-level set in the swap tree. Always sort first, or both the dedupe and the break are silently wrong.
- break when sorted and monotone; continue otherwise. Too-big → break; duplicate → continue. Swapping the first loses speed silently; swapping the second loses half the answers.
- Read the bound: $n \leq 8$ means $n!$, $n \leq 20$ means 2^n , $n \leq 25$ means 2^n and it is tight. Extra space is $O(\text{depth})$ in every problem here — time is the whole tree, space is the deepest path — and the only stack risk in the phase is combination sum, whose depth is `target / min candidate`. “How many ways” is not backtracking when the sub-state repeats: that is dynamic programming.

Scaling fundamentals

Day 097 · system-design — What scale actually means, in numbers

- **A user count is a stock; QPS is a flow. Servers are sized by flow, storage by stock.** Four multiplications get you there: $\text{DAU} \times \text{actions/day} = \text{requests/day}$; $\div 100,000 = \text{average QPS}$; $\times 3 = \text{peak QPS}$; $\text{writes/day} \times \text{bytes} = \text{storage/day}$. Round 86,400 to **100,000** and say you are rounding.
- **The three conversions to carry:** $1 \text{ million/day} \approx 10 \text{ QPS}$ · $100 \text{ million/day} \approx 1,000 \text{ QPS}$ · $1 \text{ billion/day} \approx 10,000 \text{ QPS}$. And for storage: $1 \text{ KB} \times 1 \text{ billion} = 1 \text{ TB}$; $1 \text{ MB} \times 1 \text{ billion} = 1 \text{ PB}$.
- **Compare against what one machine does**, or the numbers mean nothing: app server $\sim 1,000 \text{ QPS}$, relational DB $\sim 10,000 \text{ simple reads/s}$ and $\sim 1,000 \text{ writes/s}$, Redis $100,000+$ ops/s, one network link $\sim 1 \text{ GB/s}$. Then **6,000 peak QPS = 6–8 app servers + one DB with a cache.**
- **Peak factor is $2\text{--}3\times$ normally — but $100\times+$ for a scheduled event, and that changes the architecture, not the machine count** (queue in front, pre-computed responses on a CDN). **Always multiply storage by a replication factor of 3.** And the **read:write ratio is the most useful single number**: high means cache and replicas, low means sharding and write-optimised stores.
- **Memory $\approx 1,000\times$ faster than SSD; SSD $\approx 1,500\times$ faster than a cross-world round trip.** Mumbai $\leftrightarrow$ New York is $\sim 200 \text{ ms}$ of pure distance that no code can remove — which is what CDNs are for. Quote **p99, never the average**: a page making 100 requests hits its p99 on nearly every load. And **99.99% availability is 53 minutes of downtime a year.**

The complete 180-day map

All one hundred and eighty days, both tracks, in order. Days in **bold** are in this volume.

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
078	Nodes and links: pointers without pointers	Design a parking lot
079	Traversal, insertion, and deletion	Design an elevator system
080	The dummy head trick	Design an ATM
081	Reversing a linked list	Design a vending machine
082	Finding the middle, and the runner technique	Design a library management system
083	Cycle detection, and why Floyd's algorithm works	Design tic-tac-toe, and then chess
084	Merging and sorting linked lists	Design a deck of cards and a card game
085	Doubly and circular linked lists	Design Splitwise
086	Linked lists revision and mock round	Design BookMyShow
087	Recursion: the leap of faith	Design a food delivery order flow
088	The call stack, drawn	Design a ride-hailing booking flow
089	Writing a recursive function that terminates	Design a rate limiter, at the object level
090	Recursion on arrays and strings	Design an in-memory cache with eviction
091	Subsets: the include-or-exclude tree	Design a logging framework
092	Permutations	Design a notification service
093	Combinations and combination sum	Design a file system
094	Backtracking: the undo step	Design snake and ladder
095	N-Queens and constraint grids	Design an online auction
096	Sudoku, word search, and grid backtracking	Low-level design revision and full mock
097	Recursion and backtracking revision and mock round	What scale actually means, in numbers

Glossary

Every term this book defines, with the day it first appears. Definitions are the sentence that introduced the term.

4

4 kb [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

A

a replicated log [Day 119](#) — Real systems need a never-ending sequence of decisions, which is why every practical consensus system is built around a replicated log: an ordered, numbered list of entries, identical on every machine.

abstract class [Day 052](#) — Definition. An abstract class is a partly-built class: it can hold fields, constructors and working methods, and it declares some methods that subclasses must fill in.

acid [Day 025](#) — When Postgres says COMMIT succeeded, the data is on disk, and that promise is the D in ACID — Atomicity, Consistency, Isolation, Durability — the four letters you will meet properly on [\[day 033\]\(../day-033-window-with-a-map/README.md\)](#).

adaptee [Day 068](#) — The adaptee — the washing machine, which has a different shape and cannot be changed.

adapter [Day 059](#) — An adapter is an implementation that translates to a specific technology (PostgresOrderRepository).

add a tie-breaker [Day 027](#) — And add a tie-breaker — without one, two customers with the same total may swap places between runs, which makes paginated results duplicate and skip.

algorithm [Day 076](#) — A strategy is an algorithm plugged into a caller; a command is a request with its receiver and arguments already bound, so it can be stored, queued and undone.

always comparable [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

amortised [Day 005](#) — The word for that is amortised: `list.append` is $O(1)$ amortised, meaning any single append might be expensive but a long run of them averages out to constant time.

anaemic model [Day 044](#) — A design where classes hold only data and all the rules live in some `IdolService` is called an anaemic model — objects in name only — and it is the most common way candidates lose this round.

and a direction [Day 079](#) — It carries a floor and a direction — "I am on 4 and I want to go up".

anti-diagonals [Day 016](#) — To collect the down-left diagonals — usually called the anti-diagonals — group cells by the value of $r + c$.

anti-sniping [Day 095](#) — Starting the count again when somebody speaks at the last moment is anti-sniping: extending the deadline whenever a bid arrives near the end.

approximate [Day 102](#) — Also: Redis's LRU is approximate — it samples a handful of keys and evicts the least recently used among them, rather than maintaining a true global ordering, because exact LRU would cost a linked-list update on every access.

array [Day 015](#) — Padma's shelf is an array — the fixed row of numbered boxes you met on [day 009](../day-009-what-an-array-is/README.md).

artefact [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

assignment rule [Day 108](#) — "Walk forwards until you reach a point" is the assignment rule: a key belongs to the first node clockwise.

at-least-once [Day 129](#) — Two: you gain duplicates. Queues deliver at-least-once: if the consumer does not acknowledge in time, the message comes back.

atomic [Day 008](#) — The fix is to make check-and-act atomic — indivisible, so that no other thread can act in the middle of it.

atomicity [Day 025](#) — That is atomicity — the A in ACID — and it is impossible to get right with a file unless you write the whole log-and-replay machinery yourself, at which point you have written a database.

availability [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that *every* non-failing node answers.

aws ecs [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

B

backend [Day 002](#) — The backend is the code that never leaves the kitchen.

bearer token [Day 019](#) — The token is a bearer token: whoever bears it is treated as the user.

best case [Day 002](#) — The best case is 1, the worst case is n.

binary tree [Day 098](#) — A binary tree is a tree where every node has at most two children, called left and right.

blast radius [Day 111](#) — The eleven days revealing water, diesel and medicine is blast radius: what actually stops.

block [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

blocked [Day 007](#) — When the barber stood there holding a head while the boy fetched hair colour, he was blocked — occupying a chair while doing no work.

both [Day 030](#) — Once both are inside the loop, look at the gap between them, measured the way round the loop from fast to slow.

boundary [Day 118](#) — The unifying idea worth stating: *two heaps facing each other let you maintain a boundary in a changing set — and the boundary is the answer.*

bounded context [Day 051](#) — The name for the boundary between those worlds is a bounded context: each context gets its own model of "customer", and they are linked by an id rather than merged.

bounded optimism [Day 116](#) — Spending up to two thousand is bounded optimism — act early where the downside is small.

breadth-first [Day 100](#) — The other family is breadth-first — one whole level at a time — which needs a queue instead of a stack and is [tomorrow](../day-101-bfs-level-order/README.md).

break point [Day 045](#) — The result has one break point — one place where a value is followed by a smaller value.

broken links [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

bucket by time [Day 157](#) — The one thing that breaks this model is a channel that never stops growing. A partition with ten million messages is too large for most stores, so bucket by time: the partition key becomes (channel_id, month).

bucket counts [Day 171](#) — The fix is histograms. Each machine exports bucket counts — "how many requests fell in 0–10 ms, 10–50 ms, 50–100 ms" — and you sum the buckets across machines and compute the percentile from the sum. That is exactly what Prometheus's `histogram_quantile` does, and it is why counters and histograms are the metric types that aggregate and...

bulkhead [Day 111](#) — A bulkhead is a partition that stops a failure spreading — from ships, where a holed compartment does not sink the vessel.

burst size [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

C

cache [Day 003](#) — Saving it in his phone is caching. A cache is a local copy of an answer, kept so that you do not have to ask again.

cache it [Day 107](#) — Fix: you cannot split a single key across shards, so cache it — the photocopy — or replicate it to every shard, or give it its own shard.

cache stampede [Day 090](#) — And the first cold morning is a cache stampede.

candidates [Day 093](#) — The trays are the candidates — the values you are allowed to pick from.

capacity [Day 005](#) — A list keeps a capacity — how many slots it has room for — which is bigger than its length, the number of slots actually used.

cdn [Day 070](#) — CDN — a geographically distributed caching proxy.

certificate [Day 006](#) — When your browser connects to `google.com`, the machine sends back a certificate — a small document containing the name it claims (`google.com`), a public key, an expiry date, and a signature from a Certificate Authority.

chaining [Day 060](#) — The standard fix is chaining: each bucket holds a small list of the (key, value) pairs that landed there, and a lookup checks the two or three entries in that one bucket.

channels [Day 092](#) — The group message, the direct message and the phone call are channels — the different ways a message can physically reach a person.

checks permission [Day 011](#) — The kernel checks the arguments and checks permission — this is Balan saying no.

chunks [Day 160](#) — A file is not stored as a file. It is split into chunks — fixed or variable-size blocks, typically around 4 MB — and each chunk is identified by the hash of its contents. The file itself becomes a small piece of metadata: a name, a version, and an ordered list of chunk hashes.

circular buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position $7 + 1$ is position 0.

class [Day 043](#) — A class is the description; an object is one thing made from that description.

client [Day 068](#) — The client — the wall socket, which will only accept one shape.

collapse key [Day 141](#) — Collapsing is Devaraj refusing to carry the same message four times. Both platforms support a collapse key: a new notification with the same key replaces any undelivered one.

collapsing writes [Day 102](#) — Its real advantage is collapsing writes — a video watched ten thousand times in a minute becomes one database write instead of ten thousand.

collection [Day 016](#) — A collection is the set of things: `/bags`.

column [Day 026](#) — A column is one attribute of it, with a type that constrains what can go there.

compact [Day 011](#) — The price is that you now need a separate way to know which slots are real, and you must eventually compact: one pass that closes all the holes at once.

comparator [Day 058](#) — And the two uncles are the one case that needs a comparator — a rule about a pair, not a fact about an item.

comparison [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

complete [Day 114](#) — Insert appends first, and that is not a detail. Putting the new value at the end of the array is the only place that keeps the tree complete — any other position leaves a gap and destroys the array layout.

component [Day 069](#) — The tumblers are the component — the real object that does the actual job.

composite [Day 029](#) — This only ever applies when the primary key is composite — two or more columns together.

composite key [Day 026](#) — Its primary key is usually the two columns together — a composite key — which also enforces that the same pair cannot appear twice.

composition [Day 071](#) — Strategy uses composition — the varying part is an object passed in.

composition root [Day 053](#) — The composition root is the one place in the program where the real concrete classes are named and wired together — usually `main()` or a startup module.

connection [Day 041](#) — Under all of this, every query needs a connection — and opening one is genuinely expensive: TCP, then TLS ([[day 006](#)](../day-006-python-strings-dicts-sets/README.md)'s handshake), authentication, and in Postgres a whole operating-system process per connection ([[day 008](#)](../day-008-reading-a-problem/README.md)'s processes, ~5–10 MB each).

connection-oriented [Day 004](#) — This is why TCP is called connection-oriented — there is a real, agreed-upon connection with state on both sides.

consequence [Day 109](#) — The lorries are the consequence — the number that changes a decision, which is the only number that actually mattered.

consistency [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that *every* non-failing node answers.

constraints [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

constructor [Day 044](#) — `__init__` is the constructor: the code that runs when a new object is made, whose job is to leave the object in a valid state.

container [Day 013](#) — A container is an ordinary process — the same kind of process you met on [[day 008](#)](../day-008-reading-a-problem/README.md) — that has been given a restricted view of the machine it is running on and a hard limit on what it may consume.

container image [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

containerd [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

context [Day 073](#) — The object — the context — holds one of those classes and forwards every request to it.

coordinator [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

correlation id [Day 091](#) — So every record carries a correlation id — generated at the edge, stored in a context variable, and attached automatically by the framework rather than passed through every function signature.

cost [Day 032](#) — It enumerates the plausible combinations, assigns each a cost — an abstract number, not milliseconds — and picks the smallest.

count map [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

counter [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

cri-o [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

critical path [Day 134](#) — The number of levels is the critical path — the minimum time to finish everything if you had unlimited workers.

cycle [Day 125](#) — Cyclic or acyclic is the third. A cycle is a path that starts and ends at the same vertex without reusing an edge.

D

dangerous [Day 104](#) — Compact, and dangerous: any statement that is not deterministic gives a different result on the follower.

dashed [Day 050](#) — Four: implements an interface — the same triangle, but a dashed line:

dau [Day 097](#) — DAU — daily active users.

deep-copied [Day 067](#) — The cooker and the dabbas were deep-copied — genuinely new things that happen to look the same.

deleted [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

delta of delta [Day 137](#) — So you store the difference of the differences — the delta of delta — which is almost always zero, and zero costs one bit.

dependency [Day 053](#) — A dependency is anything a class needs in order to do its job but does not own — a repository, a mailer, a clock, a payment gateway.

depth [Day 046](#) — First, depth: with Vehicle → MotorVehicle → Car → ElectricCar → LeasedElectricCar, understanding one method means reading five files, and a change at the top can break anything below it, at any depth.

depth-first [Day 100](#) — Going depth-first means you follow one branch all the way to the bottom before backing up and trying the next one, and there are exactly three places you can do the visiting: before the children, between them, or after them.

deque [Day 031](#) — The fix for max is to keep a deque — a double-ended queue, from [day 074](../day-074-deques-and-window-max/README.md) — holding indices, arranged so that their values are decreasing from front to back.

diminishing returns [Day 135](#) — But with diminishing returns: the eighth mention adds much less than the second.

direct addressing [Day 066](#) — The wall of numbered holes is direct addressing: because the keys are the numbers 1 to 120, you do not need to hash anything.

domain name [Day 003](#) — "Anjali Sharma" is the domain name. A domain name is a human-readable label for a machine or a group of machines: google.com, leetcode.com, api.github.com.

draw two diagrams [Day 050](#) — If your design genuinely has fifteen classes, draw two diagrams: the core five or six, and then a second showing one subsystem in detail.

duck typing [Day 047](#) — This is duck typing: if it has a send method that takes a string, it is a notifier.

dummy node [Day 080](#) — Sister Mary is a dummy node — sometimes called a sentinel head.

E

earlier [Day 118](#) — The safe pattern: the leader treats its lease as expiring earlier than the grantor does, so there is a gap in which nobody believes they are leader.

edge cases [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

edge compute [Day 103](#) — There is a real modern exception, and mentioning it is a good signal: edge compute — Cloudflare Workers, Lambda@Edge — runs your code at the edge, so some personalisation and some logic can happen there without a trip to the origin.

effect [Day 113](#) — Say it that way: "exactly-once delivery is impossible; exactly-once effect is achievable, with at-least-once delivery and an idempotency key." Systems that advertise exactly-once are doing precisely this.

elastic scaling [Day 100](#) — The locum starting with no handover is elastic scaling: add a machine and it is immediately useful.

election timeout [Day 118](#) — "By half past five" is the election timeout — the guess that the leader is gone.

empty [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

etl [Day 139](#) — ETL versus ELT is about where the reshaping happens. ETL — extract, transform, load — transforms on a separate machine before loading, which is what you did when warehouse compute was scarce.

event object [Day 072](#) — In practice you push a small, immutable event object — a plain record describing what happened, like `OrderPlaced(order_id, user_id, total, items)`.

eventual consistency [Day 115](#) — At the bottom is eventual consistency: if writes stop, the copies will agree — eventually, with no promise about when or what you see meanwhile.

exactly [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

exactly correct [Day 089](#) — This is exactly correct — no boundary effect, no approximation — and it is the version you already wrote on [\[day 077\]\(../day-077-stacks-queues-revision/README.md\)](#) as a deque of timestamps.

exactly one [Day 171](#) — XOR is exactly one — which is the same as "different", and that is the property the whole of the next XOR day rests on.

exclusive lock [Day 035](#) — When a transaction updates a row, the database gives it an exclusive lock on that row: nobody else may change the row until the transaction commits or rolls back.

expiry [Day 090](#) — The dates are expiry — a completely separate mechanism.

F

facts [Day 082](#) — So: a FinePolicy. The loan holds the facts — who, what, when it went out, when it was due, when it came back.

false negative [Day 124](#) — A false negative is failing to notice a machine that really has died.

false positive [Day 124](#) — A false positive is declaring a healthy machine dead.

fanout [Day 030](#) — The critical property is the fanout: because a page is 8 KB and an entry is small, one node holds hundreds of entries.

fee [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day."

fifo [Day 090](#) — FIFO — evict the oldest inserted.

file [Day 093](#) — A file is a thing that is not a box: a plate, a lamp.

filled diamond [Day 050](#) — A filled diamond means *composition* — the parts die with the whole, so Order ♦ — OrderLine because a line item has no meaning without its order.

finite state machine [Day 073](#) — This is a finite state machine — a fixed set of states, a fixed set of events, and a table saying which event moves you from which state to which.

float [Day 081](#) — Devi's second tin is a float: coins reserved so tomorrow can start.

four-direction move [Day 096](#) — "Each one immediately beside the one before" is the four-direction move: up, down, left, right.

fragile [Day 021](#) — It is also fragile: it silently breaks on capital letters, spaces, digits and anything non-English, and §7 shows exactly how silently.

frontend [Day 002](#) — The frontend is the code that crosses the counter.

G

galloping mode [Day 057](#) — When merging, if one run keeps winning, Timsort switches to galloping mode — it binary-searches ahead to find how many elements it can take in one go instead of comparing one at a time.

gap [Day 047](#) — The gap is the answer she is searching for — not something in the array, a number she is choosing.

generator expression [Day 020](#) — That is a generator expression — it produces each item as join asks for it.

greedy simulation [Day 046](#) — It is a greedy simulation: load as much as fits, then start a new trip.

guest operating system [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

H

hash map [Day 066](#) — Kiran's phone is a hash map: a general system that can look up anything by any key, at the price of storing the keys and computing where they go.

hash ring [Day 108](#) — The eleven-kilometre loop is the hash ring — the space of all hash values, joined end to end.

hateoas [Day 016](#) — And then there is the seventh idea, part of the uniform interface, that everyone quotes and almost nobody implements: HATEOAS — Hypermedia As The Engine Of Application State.

header interface [Day 058](#) — A header interface is named after the implementation and lists everything it can do: Repository, Machine, UserService.

heap [Day 117](#) — The six letters on the table are the heap: one candidate per list.

heapsort [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A^* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

heartbeat [Day 124](#) — The whistle is a heartbeat. A heartbeat is a small, regular message that says "I am alive".

hidden spof [Day 111](#) — The shared approach road is a hidden SPOF — two components that are not independent.

hiding mechanism [Day 052](#) — Abstraction is about hiding mechanism: the caller says "charge this card" and does not learn which payment provider ran it.

highest node [Day 104](#) — Every path has exactly one highest node — the point where it turns around.

historical snapshot [Day 029](#) — | Copy | Instead of | Why | |---|---|---| | posts.comment_count | COUNT(*) on comments | counting on every read is unaffordable at a high read ratio | | orders.customer_name | joining to customers | a historical snapshot: the name at the time of the order | | order_items.unit_price | joining to products | the price *then*, which must...

hollow triangle [Day 050](#) — Three: inheritance — a line with a hollow triangle pointing at the parent:

hook [Day 075](#) — award is a hook: the base class implements it as **doing nothing**, so a subclass may override it and most do not.

horizontal scaling [Day 098](#) — Horizontal scaling means putting more machines beside it and dividing the work.

hypervisor [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

I

image [Day 013](#) — An image is a read-only stack of filesystem layers — a base operating system's files, then your dependencies, then your code.

implicit graph [Day 130](#) — This is an implicit graph — the edges exist by rule rather than by record — and you met the idea on [\[day 125\]\(../day-125-what-a-graph-is/README.md\)](#).

in parallel [Day 111](#) — Components in parallel — any one will do — multiply their **failure** probabilities, which is a completely different shape.

in series [Day 111](#) — Components in series — every one is needed — multiply their availabilities, so the total is always worse than the worst one.

in-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

index [Day 025](#) — A database keeps an index — a separate structure, usually a B-tree, that maps a value to the location of the matching rows.

indexed heap [Day 114](#) — Hence the indexed heap: a hash map from value to index, updated on every swap.

inheritance [Day 071](#) — Template Method uses inheritance — an abstract base with the skeleton and subclasses filling in the holes.

inorder [Day 110](#) — The standing order is inorder: left subtree, then root, then right subtree.

input [Day 008](#) — In a problem: the input — its type, its size, and what is in it.

integrity [Day 006](#) — TLS also guarantees integrity: each piece carries a check value, so if anything in transit is altered by even one bit, the other end notices and drops the connection.

intent plus multiplicity [Day 070](#) — The honest distinction is intent plus multiplicity: you stack decorators and expect several; a proxy is usually one and is about access rather than behaviour.

interface [Day 052](#) — An interface is a pure list of method signatures with no state and no implementation — a contract that says what a type can do.

invariant [Day 028](#) — That is called an invariant — something true before the loop starts, still true after every turn, and therefore true when the loop ends.

is possible [Day 046](#) — $>$ **Find the smallest X such that some task is possible.** $>$ **Find the largest X such that some condition still holds.**

item [Day 016](#) — An item is one of them: `/bags/213`.

iterator [Day 075](#) — Iterator is the pattern where "go through this collection one item at a time" is separated from "this collection".

J

json [Day 015](#) — JSON — JavaScript Object Notation — is the plain-text format most web APIs use for both request bodies and responses, and you met it on [\[day 007\]\(../day-007-space-complexity/README.md\)](#); it looks like `{"id": 42, "name": "Zoya"}`.

jump [Day 094](#) — A snake and a ladder are both a jump: a pair (from, to).

jwt [Day 020](#) — a session id — an opaque reference, meaningless by itself, that the server looks up; - a JWT — a self-contained signed token that carries the facts inside it, so no lookup is needed; - OAuth 2.0 and OpenID Connect — a protocol for getting one of the above from *somebody else*, so a user can log in with Google without your service ever...

K

k closest points [Day 116](#) — k closest points — the same shape with a computed key.

k most frequent [Day 116](#) — k most frequent — count first, then top-k on the counts.

k-th largest specifically [Day 116](#) — k-th largest specifically — quickselect is the natural fit, because you want one element rather than a set, and quickselect gives it directly.

key [Day 023](#) — The key is as much of the design as the algorithm.

key property [Day 108](#) — Adding the temple point disturbing only sixty houses is the key property: adding a node moves only the keys between it and its predecessor.

L

lag [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

largest [Day 055](#) — the 1st smallest is at position 0, so the k-th smallest is at position k - 1; - the 1st largest is at position n - 1, so the k-th largest is at position n - k.

latency [Day 089](#) — The cost is latency — a request may wait in the queue — and that makes it wrong for a user-facing API, where you would rather refuse quickly than hold a connection.

lazy [Day 117](#) — Merging sorted streams — `heapq.merge(*iterables)` does exactly this and is lazy, so it works on inputs larger than memory.

lazy deletion [Day 114](#) — Python has neither. The idiomatic workaround is lazy deletion: push a new entry and mark the old one stale, skipping stale entries as they surface.

lazy loading [Day 041](#) — That is lazy loading: the ingredient bought at the moment it is needed.

lazy refill [Day 089](#) — The lazy refill is the implementation trick: no timer, no background job.

leader [Day 105](#) — Counter two is the leader: writes go there, and reads from it are always current.

leaf [Day 098](#) — He is a leaf — an item with no children.

lease [Day 118](#) — A lease is leadership with an expiry: you are the leader until time T unless you renew.

ledger entries [Day 085](#) — An expense produces a set of ledger entries: the payer is owed, each participant owes their share.

left [Day 040](#) — Remove the strip to the left: `prefix[r2 + 1][c1]`.

lfu [Day 090](#) — LFU — evict the least frequently used.

linear scan [Day 062](#) — Scrolling the whole list of names is a linear scan: to answer one question you touch everything.

linearizability [Day 114](#) — C — Consistency. Specifically linearizability: every read sees the most recent write, and the system behaves as if there were one copy of the data.

list [Day 001](#) — heap is a list: a row of values kept in order, like the socks piled on the bed.

list of lists [Day 016](#) — A matrix is a list of lists: the outer list holds the rows, and each row is itself a list holding that row's values.

load balancer [Day 098](#) — Sending students to the right counter is a load balancer.

load factor [Day 060](#) — The number that controls this is the load factor: the number of items divided by the number of buckets.

load shedding [Day 007](#) — This idea has a name — load shedding — and it is one of the things that separates a system that degrades from one that collapses.

log-structured merge tree [Day 031](#) — A log-structured merge tree — used by Cassandra, RocksDB, LevelDB and ScyllaDB — takes the opposite trade.

logical cohesion [Day 061](#) — Anjali's first kitchen is logical cohesion — grouped by kind, which looks tidy and is wrong, because the unit of use is the job, not the kind.

logical time [Day 113](#) — The replacement is logical time — Lamport clocks and vector clocks — which order events by *causality* rather than by wall time.

loop [Day 001](#) — A loop is a statement that says, "do the following again and again".

loop body [Day 002](#) — A loop body is the indented block underneath the loop header — the part that runs again and again.

low cohesion [Day 061](#) — The kitchen arranged by size is low cohesion — things used together kept apart, so every job takes trips.

lru [Day 090](#) — LRU — evict the least recently used.

M

manifest [Day 158](#) — Adaptive bitrate streaming. The client downloads a manifest — an HLS .m3u8 or DASH .mpd — listing every rendition and every segment.

many times [Day 149](#) — Virtual nodes fix that. Each physical machine is placed on the circle many times — 100 to 256 positions each, by hashing machine-1#0, machine-1#1, and so on.

many-to-many [Day 026](#) — | Shape | Example | Where the key goes | |---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

matrix [Day 016](#) — A matrix is a rectangle of values addressed by two numbers instead of one: which row, then which column.

mau [Day 097](#) — MAU — monthly active users.

max-heap [Day 116](#) — The mirror rule: to find the k smallest, keep a max-heap of size k — in Python, a min-heap of negated values.

method resolution order [Day 052](#) — The Python answer. The method resolution order — D._mro_ — is computed by a rule called C3 linearisation, and super() follows that order rather than jumping straight to a parent.

milliseconds [Day 009](#) — About 10 milliseconds — a hundred times slower again than SSD, and a hundred thousand times slower than RAM.

min-heap [Day 118](#) — Each half is a heap, and they face each other: a max-heap holding the lower half so its largest is at the top, and a min-heap holding the upper half so its smallest is at the top — the two elements at the boundary are exactly the ones you need. And the only real work is keeping the two halves the same size, which is two lines and is...

minimum spanning forest [Day 139](#) — Both algorithms then produce a minimum spanning forest — one tree per component — and you detect it by counting: fewer than $V - 1$ edges taken means the graph was disconnected.

mixin [Day 049](#) — A mixin is a small class carrying one piece of behaviour, meant to be inherited alongside others:

monotone [Day 043](#) — A yes-or-no question with that shape is called monotone: once the answer flips from no to yes, it stays yes.

monotonic deque [Day 074](#) — The good solution keeps a small monotonic deque of "elements that could still be the maximum" and is $O(n)$ overall.

N

name mangling [Day 045](#) — Two underscores triggers name mangling: inside the class the attribute is rewritten as `_Account_key`, so a subclass defining its own `_key` does not collide.

namespace [Day 013](#) — A namespace is a kernel feature that gives a process its own private version of one part of the system.

negative [Day 035](#) — A sum over values that can be negative is not monotonic — the window is the wrong room, and the case goes to prefix sums, which arrive on [\[day 037\]\(../day-037-prefix-sums/README.md\)](#).

negative infinity [Day 049](#) — LeetCode 162 states it: treat `nums[-1]` and `nums[n]` as negative infinity — imaginary values smaller than anything real, just off each end.

negatives [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

nested loop [Day 001](#) — A nested loop is a loop inside another loop.

network partition [Day 114](#) — The exchange fault is a network partition — the nodes are alive but cannot talk.

never equal [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

never forgets [Day 090](#) — Needs a third structure and a `min_frequency` counter to stay $O(1)$, and it never forgets: an entry that was hot last month holds a huge count and will not leave.

never persisted [Day 157](#) — Typing indicators are presence with a shorter fuse. Same mechanism, a two-to-three second TTL, throttled so a keystroke does not become a network message, and never persisted — a typing event that arrives late is worse than one that is dropped.

no children [Day 121](#) — It has no children — nothing continues past it.

node [Day 125](#) — The word node means the same thing and both are used; this course will say vertex when talking about the structure and node when talking about code.

not a map [Day 067](#) — | not a map — heap or sorted array |

O

$o(1)$ [Day 079](#) — tail — a reference to the last node, which makes append $O(1)$ instead of $O(n)$.

object [Day 043](#) — An object is data and the behaviour that belongs to that data, kept together in one place.

observers [Day 072](#) — The objects on the list are the observers.

occupied [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

offset [Day 130](#) — Each viewer's position is an offset. An offset is just a number — the position in the log that a particular reader has got to.

on first touch [Day 041](#) — Each `post.author` is a `*different table*`, not yet loaded — so the ORM, helpfully and silently, fetches it on first touch.

one-shot [Day 075](#) — A list can be walked many times; a generator is one-shot — walk it twice and the second walk is empty, which is a bug people hit constantly.

opens [Day 126](#) — When failures exceed a threshold, it opens: from then on, calls to that dependency fail instantly without being attempted.

orchestrator [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

origin [Day 101](#) — The storeroom is the origin — the source of truth.

origin pull [Day 103](#) — Sending the finished pages down the wire is origin pull: the edge fetches from the origin on demand.

orphan rows [Day 026](#) — Without it you get orphan rows — a post whose author does not exist — and they are impossible to clean up later, because you cannot tell which ones were mistakes.

orphans [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

out-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

overloading [Day 047](#) — Compile-time polymorphism, or overloading: several methods with the same name and different argument types, and the compiler picks.

overriding [Day 047](#) — Runtime polymorphism, or overriding: a subclass replaces a parent's method, and which one runs is decided by the object at run time.

P

page [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

parallel edges [Day 132](#) — And the case where excluding by vertex is not enough. If the graph can have two separate edges between the same pair — parallel edges — then A—B twice genuinely **is** a cycle, and skipping "the parent vertex" wrongly ignores it.

parameters [Day 015](#) — In an API those are the parameters — the values you must supply with a request — and leaving a required one out gets you an error, not a guess.

parent [Day 132](#) — The fix: pass down where you came from, and skip it. Each recursive call carries the vertex that called it — its parent — and the check becomes "seen, and not my parent".

partial failure [Day 113](#) — The one that matters most is partial failure: a call over a network has a third outcome that does not exist locally, *"I do not know"*, and almost every difficult problem in the rest of this phase is a consequence of it.

participants [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

partition key [Day 039](#) — A wide-column store — Cassandra is the name to know — organises data by two keys: a partition key that decides which machine (and which bucket on it) a row belongs to, and clustering columns that keep rows **sorted inside** that bucket.

partitions [Day 130](#) — It is split into partitions — independent logs that together make up the topic.

path [Day 005](#) — A request has four parts: a method saying what you want done, a path saying what you want it done to, headers carrying facts about the request itself, and an optional body carrying the content.

port [Day 003](#) — A port is a second number that says which program on that machine you want, the way a flat number says which door inside the building.

prefix sums [Day 037](#) — His noted meter readings are the prefix sums — and his godown reading, taken before anything was driven, is the extra zero at the front that today's code lives or dies by.

preorder [Day 110](#) — The calling order is preorder: root, then left subtree, then right subtree.

priority queue [Day 139](#) — Finding "the cheapest edge leaving the tree" efficiently means a priority queue — which makes Prim's structurally almost identical to [Dijkstra's](../day-136-dijkstra/README.md).

procedure [Day 062](#) — It is a procedure: a fixed order in which to read a piece of code you have never seen, so that by the end of it you can say what is wrong, what evidence you have, and what you would change first.

process [Day 008](#) — A process is a running program with its own private memory.

program [Day 001](#) — A program is a sequence of statements.

protecting state [Day 052](#) — Definition. Encapsulation is about protecting state: the balance cannot go negative because the only way to change it runs a check.

Q

qps [Day 097](#) — QPS stands for *queries per second* — the number of requests arriving at your system each second.

queue [Day 127](#) — The queue holds the frontier in order. A queue is a line where things come out in the order they went in — first in, first out — which you met on [day 73](../day-073-queues/README.md).

quorum [Day 118](#) — "Three of the four must agree" is a quorum — the majority from [day 117](../day-117-merge-k-sorted/README.md).

R

race condition [Day 008](#) — A race condition is when the result depends on the exact timing of two things running at once.

ram [Day 009](#) — The suitcase on the almirah — main memory, RAM. RAM — random access memory — is where your running program's data lives.

rank [Day 138](#) — Union by rank is the same idea using an upper bound on height rather than the exact size; both work, size is easier to reason about, and size gives you group sizes for free, which problems often want.

rebalancing [Day 107](#) — Moving twenty bundles is rebalancing — moving whole logical shards, not rehashing keys.

rebuilding the tree [Day 109](#) — Repacking the whole cart is rebuilding the tree — $O(n)$.

recursion [Day 053](#) — That is recursion: a function whose body calls the same function on a smaller piece of the problem.

references [Day 009](#) — A Python list stores references — memory addresses of objects that live elsewhere:

refill rate [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

register [Day 009](#) — The chair beside the bed — registers. A register is a tiny store inside the processor itself, holding one value.

relative epsilon [Day 048](#) — The fix is a relative epsilon — stop when the gap is small *compared to the value*:

removal [Day 118](#) — The same two heaps, plus removal, which a heap cannot do.

replica [Day 136](#) — A replica is a copy of a shard on another node, serving reads and taking over if the primary fails.

replica divergence [Day 115](#) — Different people knowing different things is replica divergence — and none of them is wrong.

request-aware routing [Day 099](#) — The joint-family card going to counter one is request-aware routing — some requests are much more expensive than others.

resolver [Day 021](#) — Each field in the schema has a resolver — a function that knows how to produce that field.

resources [Day 017](#) — Before writing a single path, list the resources — the things the feature is about.

returned [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day.*

ring buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position 7 + 1 is position 0.

role interface [Day 058](#) — A role interface is named after what a client needs: UserReader, Printer, PriceQuoter.

root [Day 093](#) — The root is the loft itself, written /.

rotation [Day 109](#) — The fix is a rotation: a local rearrangement of three or four pointers that reduces the height by one without touching the rest of the tree or changing the inorder order.

routine [Day 109](#) — Counting paces and multiplying is the routine — the same sequence every time, said out loud.

row [Day 026](#) — A row is one of those things.

run [Day 044](#) — So the copies of a value form one run: a contiguous stretch, described completely by its two ends.

S

same remainder [Day 038](#) — A stretch is divisible by k when the prefixes at its two ends leave the same remainder — so carry running % k and count matching remainders.

schema [Day 021](#) — A GraphQL API is defined by a schema: every type, every field, and every field's type, written down and strongly typed.

scoping [Day 096](#) — The two or three narrowing questions are scoping — deciding out loud what you are *not* building.

script [Day 077](#) — Today is the script — the fixed order of moves you run on every one of these prompts for the next twenty days.

seam [Day 053](#) — A seam is a place where you can change behaviour without editing the class.

search space [Day 042](#) — The stretch `nums[low..high]` is called the search space — the part of the array you have not ruled out yet.

selectivity [Day 032](#) — From those numbers the planner estimates selectivity: what fraction of rows a condition keeps.

sentinel [Day 037](#) — That extra entry is called a sentinel: a value placed at the boundary so that the boundary needs no special treatment.

separator [Day 019](#) — The string before the `.join` is the separator — `"".join` glues with nothing between, `",".join` puts a comma and a space between each pair.

sequence diagram [Day 050](#) — That is a sequence diagram: participants as columns, time running down, and each message an arrow from one column to the next.

serialisable [Day 033](#) — The strongest form is serialisable: the result is as if the transactions had run one after another, in some order.

service credits [Day 173](#) — An SLA is a contract with a customer: a promise, and a consequence when you break it. The consequence is almost always service credits — money back off the next bill — rather than damages.

session [Day 020](#) — A session is a record the server keeps about a logged-in user.

session guarantees [Day 115](#) — And the interesting rungs are in the middle, particularly causal consistency and the four session guarantees, because they are what users actually notice and they are much cheaper than the top.

set [Day 006](#) — A set stores values and answers "is this in here?" in a single step, no matter how many it holds.

shallow-copied [Day 067](#) — The milk and the newspaper were shallow-copied — the new kitchen got a *reference* to the same arrangement, and changing it from either end changed it for both.

shape [Day 003](#) — It throws away every detail that does not change as the input grows — the exact number of steps, how fast your laptop is, whether one line takes two nanoseconds or twenty — and keeps only the shape of the growth.

shard [Day 106](#) — Each room is a shard — a machine holding a subset of the rows.

shared lock [Day 035](#) — A shared lock is for reading: many transactions can hold one on the same row at once, because readers do not disturb each other.

shared store [Day 100](#) — The shared cabinet at the front desk is a shared store — a database, or Redis.

singleton [Day 064](#) — Singleton is the pattern that guarantees a class has exactly one instance, and gives everybody a global way to reach it.

singly linked list [Day 076](#) — A singly linked list — where each node knows only the *next* one — cannot do that.

slice [Day 019](#) — `s[1:4]` is a slice — a new string made of part of the old one.

smallest [Day 046](#) — `>` *Find the smallest X such that some task is possible.* `>` *Find the largest X such that some condition still holds.*

snapshot [Day 034](#) — The transaction takes a snapshot at its first read and every statement in it sees that snapshot, however long it runs.

snipe [Day 095](#) — A snipe is a bid placed in the last second, leaving nobody time to respond.

sorted region [Day 052](#) — And the sorted region is the part of the list you have already finished; every one of these methods grows a sorted region and shrinks an unsorted one, and they differ in *which end* the region grows from and *how* the next value gets there.

sorting everything [Day 116](#) — The nephew laying out the whole load is sorting everything — correct, and far more work than asked for.

splits [Day 031](#) — If the leaf is full, it splits: half the keys move to a new page, and a key is pushed up to the parent.

spof [Day 111](#) — The bridge is an SPOF — one component, and everything depends on it.

squares [Day 003](#) — | Example | |---|---|---| | $O(1)$ | constant | does not change | reading items[5] | | $O(\log n)$ | logarithmic | goes up by one step | halving until you reach 1 | | $O(n)$ | linear | doubles | one loop over the list | | $O(n \log n)$ | linearithmic | slightly more than doubles | sorting | | $O(n^2)$ | quadratic | goes up four times | every...

stable [Day 051](#) — Python's sort is Timsort: $O(n \log n)$ worst case, stable, and adaptive — it finds runs that are already ordered and merges them, so nearly-sorted input runs close to $O(n)$.

stack [Day 075](#) — The fridge is a stack: last in, first out.

stack frame [Day 068](#) — When a function calls another function, the machine pushes a stack frame — the local variables and the place to return to — onto a stack.

stack overflow [Day 068](#) — And the bad Sunday is a stack overflow: too many unfinished things, and the one at the bottom is forgotten.

state [Day 143](#) — And the state is the thing to get right. The state is what identifies a subproblem — the arguments to the recursive function.

statements [Day 001](#) — A program is a sequence of statements.

status [Day 086](#) — It has a status — available, locked, booked — and a price, because the same seat costs more on a Saturday night.

still one machine [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

stonith [Day 118](#) — STONITH — "shoot the other node in the head": the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting any write.

strategy [Day 071](#) — Strategy turns an algorithm into a value.

strictly shorter [Day 072](#) — left[i] — the index of the nearest bar to the left that is strictly shorter.

structural [Day 058](#) — In Python, typing.Protocol is structural — it matches on the shape of the methods, and the implementing class inherits nothing and imports nothing ([day 052](../day-052-quadratic-sorts/README.md)).

subarray [Day 024](#) — There is a third word you will meet: a subarray is the same thing as a substring, for arrays.

subject [Day 072](#) — Observer is the pattern where one object — the subject — keeps a list of other objects that want to be told when something happens to it, and tells all of them with one call.

subscription list [Day 072](#) — The group itself is the subscription list, and the one message is the notification.

substring [Day 155](#) — First the definitions. A substring is contiguous — "aba" is a substring of "xabay", but "aa" is not.

subtree [Day 122](#) — If you get to the end, the node you are standing on is the root of a subtree — the whole set of words that begin with what the user typed.

surge multiplier especially [Day 088](#) — The surge multiplier especially is frozen.

swap [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

system call [Day 011](#) — The mechanism for asking is a system call, and the boundary it crosses is the most important line in a computer.

T

table [Day 026](#) — A table holds one kind of thing.

tail call [Day 088](#) — A tail call is a recursive call whose result is returned directly, with nothing left to do afterwards:

tail-recursive [Day 090](#) — This version is also tail-recursive — nothing happens after the call returns — which a language with tail-call elimination would turn into a loop for free.

tcp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

template [Day 092](#) — The saved wording with the date and the hours swapped in is a template — a fixed piece of text with holes in it, filled in per message.

test double [Day 053](#) — A test double is a stand-in you pass in during a test.

the return address [Day 088](#) — the arguments it was called with, - the local variables it creates, - the return address — the exact point in the caller to resume at, - and a link to the frame below it.

the running median [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

third table [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

thread [Day 008](#) — A thread is one line of execution inside a process, and all the threads in a process share that memory.

three distinct orderings [Day 092](#) — = 6 decision paths — but only three distinct orderings: [1,1,2], [1,2,1], [2,1,1].

three-way partition [Day 054](#) — The fix is a three-way partition: split into *less than*, *equal to*, and *greater than*, and recurse only into the outer two.

tight coupling [Day 061](#) — The balcony light on the reading lamp's switch is tight coupling — two things wired together that have no reason to be, so you cannot change one without changing the other.

time-to-live [Day 037](#) — Bhaskar's clear-out is time-to-live: attach an expiry to a key and the store deletes it, automatically.

tiny constraint [Day 097](#) — A tiny constraint: $n \leq 8$ means factorial, $n \leq 20$ means 2^n , target ≤ 500 with small candidates means combination sum.

toast [Day 134](#) — And there is a middle option worth knowing. Postgres stores large values out-of-line automatically — TOAST — in a side table, compressed, and only reads them when the column is selected.

token [Day 019](#) — The server verifies them and issues a token — a session id or a signed token.

tombstone [Day 011](#) — There is a real technique that does leave holes — marking a slot as deleted instead of removing it, called a tombstone — and databases and hash tables use it constantly.

top-k problems [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

total order [Day 123](#) — It gives you a total order — sort by the number, break ties by machine name — that never contradicts causality.

transitions itself [Day 071](#) — In State, the object transitions itself: a Draft order moves itself to Submitted, and the states know about each other.

transitive dependency [Day 029](#) — This is a transitive dependency: id → department_id → department_name.

tree [Day 040](#) — A domain of independent records — a tree — tolerates anything.

ttl [Day 003](#) — The family moving out is the staleness problem, and it is the reason for TTL. Every DNS answer comes with a TTL — time to live — which is a number of seconds saying how long the answer may be trusted before it must be looked up again.

tuple [Day 005](#) — A tuple is written with round brackets — (3, 7, 2) — and is a list you cannot change.

two [Day 087](#) — In fib, each call makes two — so the work explodes.

two implementations [Day 079](#) — And, as always, two implementations: NearestSuitable as above, and ZonedDispatcher where each car owns a band of floors.

U

udp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

under-fetching [Day 021](#) — And from [day 015](../day-015-the-write-pointer/README.md)'s arithmetic: a screen needing four different resources makes four round trips, which on a mobile network is roughly 480 ms instead of 120 — under-fetching, usually called the N + 1 problem when it happens in a list.

union filesystem [Day 013](#) — The stack of layers is a union filesystem — several directories presented as one merged tree.

upper bound [Day 044](#) — The upper bound is the first index where `nums[i] > 4`, which is index 4 — one past the run.

uri [Day 016](#) — In REST the thing is called a resource, and its address is a URI — the path in the URL.

url [Day 001](#) — A URL is an address for something on the internet.

V

value object [Day 051](#) — A value object is defined entirely by its values.

version [Day 117](#) — And the second half, which people forget: you must be able to tell which value is newest. The overlapping replica has the new value, but the others may return an old one, so every record carries a version — a counter, a timestamp, or a vector clock — and the reader takes the highest.

vertex [Day 125](#) — The places are the vertices. A vertex is one of the things in your collection — a bus stop, a person, a city, a course, a web page, a cell in a grid.

vertical scaling [Day 098](#) — Vertical scaling means replacing the machine with a bigger one — more cores, more memory, faster disk.

victim [Day 035](#) — It watches who waits for whom, and when it finds a circle, it picks a victim — one transaction in the cycle — and kills it with an error.

virtual address space [Day 011](#) — Memory. Each process gets its own virtual address space — its own set of addresses that the OS maps onto real physical memory.

virtual machine [Day 013](#) — A virtual machine is an emulated computer.

volatile [Day 009](#) — It is also volatile: switch the power off and it is gone.

W

weight [Day 122](#) — To do that in code, every word needs a weight — a number saying how often it is chosen.

which class [Day 076](#) — Factory vs Builder — a factory decides which class; a builder assembles one complicated object step by step.

which is hours [Day 166](#) — The cost is that it must be rebuilt, and that is the trade to state. New queries do not appear until the next rebuild, which is hours — and there is a separate fast path for trending terms, which is the third idea below.

within [Day 131](#) — Consumer group — Kafka's answer, and it is genuinely both: within a group, consumers compete like a queue; between groups, each gets everything like a topic.

worst case [Day 002](#) — The best case is 1, the worst case is n.

write skew [Day 034](#) — It appears in backend interviews at every product company, usually right after ACID, and the follow-up about write skew is where senior candidates are separated from the table-reciters.

write-around with invalidation [Day 102](#) — Wiping a line when the price moves is write-around with invalidation: update the truth, delete the cache entry, and let the next reader repopulate it.

write-through [Day 102](#) — Changing the board at four fifteen is write-through: update the cache when you update the truth.

Y

yes **Day 093** — | Problem | Recurse on | Duplicates in input | Extra rule | |---|---|---| | Combinations (LC 77) | $i + 1$ | no | stop at size k | | Combination Sum (LC 39) | i — reuse allowed | no | stop when remaining $= 0$ | | Combination Sum II (LC 40) | $i + 1$ — each used once | yes | sort, and skip when $i > \text{start}$ and $c[i] == c[i-1]$ | |...

References and further reading

Further reading, arranged by the part of the book it belongs to. Everything listed is a primary source, an official specification, or reference material that is free to read. None of this book is copied from any of them. They are here for the reader who wants the long version, the formal proof, or the original paper.

FOR THE WHOLE BOOK

Python Software Foundation. *The Python Language Reference and Standard Library.* <https://docs.python.org/3/> [PSF Licence, free to read]

The authority for every language behaviour this book relies on.

Python Wiki. *TimeComplexity - cost of the built-in container operations.* <https://wiki.python.org/moin/TimeComplexity> [Free to read]

The table behind almost every complexity claim about list, dict and set.

David Galles, University of San Francisco. *Data Structure Visualizations.* <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html> [Free to use]

Step-through animations for most structures in Parts II to XV.

Steven Halim and others. *VisuAlgo - visualising data structures and algorithms.* <https://visualgo.net/en> [Free for self-study]

Useful when a diagram in this book is not enough and you want to press play.

MIT OpenCourseWare. *6.006 Introduction to Algorithms.* <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> [CC BY-NC-SA]

Full lecture videos and notes. The formal treatment this book deliberately avoids.

LeetCode. *Problem set.* <https://leetcode.com/problemset/> [Free tier]

Where every problem named in the practice sheets is solved.

DATA STRUCTURES AND ALGORITHMS, BY PART

PART X · LINKED LISTS (DAYS 78–86)

Wikipedia. *Linked list.* https://en.wikipedia.org/wiki/Linked_list [CC BY-SA]

The variants - singly, doubly, circular - and their trade-offs.

Wikipedia. *Cycle detection.* https://en.wikipedia.org/wiki/Cycle_detection [CC BY-SA]

Floyd's tortoise and hare, with the proof of why the pointers meet.

PART XI · RECURSION AND BACKTRACKING (DAYS 87–97)

Python Software Foundation. *sys.setrecursionlimit.* <https://docs.python.org/3/library/sys.html#sys.setrecursionlimit> [PSF Licence]

Why deep recursion raises RecursionError, and what the limit really guards.

Wikipedia. *Backtracking.* <https://en.wikipedia.org/wiki/Backtracking> [CC BY-SA]

The general template that Days 90 to 97 specialise.

Python Software Foundation. *functools.lru_cache.* https://docs.python.org/3/library/functools.html#functools.lru_cache [PSF Licence]

One decorator that turns the recursion of Part XI into the memoisation of Part XVI.

SYSTEM DESIGN, BY PHASE

LOW-LEVEL DESIGN CASE STUDIES (DAYS 77–96)

Wikipedia. *Unified Modeling Language - class diagrams.* https://en.wikipedia.org/wiki/Class_diagram [CC BY-SA]
The notation interviewers expect on the whiteboard.

Python Software Foundation. *dataclasses.* <https://docs.python.org/3/library/dataclasses.html> [PSF Licence]
The fastest honest way to show entities in a timed design round.

Python Software Foundation. *enum.* <https://docs.python.org/3/library/enum.html> [PSF Licence]
State machines in the parking-lot and vending-machine problems.

SCALING FUNDAMENTALS (DAYS 97–112)

Google. *Site Reliability Engineering - the SRE Book.* <https://sre.google/sre-book/table-of-contents/> [Free to read online]
Chapters on load balancing, overload and cascading failure.

NGINX. *Load balancing documentation.* <https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/> [Free to read]
Round robin, least connections and hashing, as actually configured.

Redis. *Redis documentation.* <https://redis.io/docs/latest/> [Free to read]
Eviction policies and expiry, which is most of the caching chapter.

Wikipedia. *Cache replacement policies.* https://en.wikipedia.org/wiki/Cache_replacement_policies [CC BY-SA]
LRU, LFU and the others, compared.

Colin Scott. *Latency numbers every programmer should know.* https://colin-scott.github.io/personal_website/research/interactive_latency.html [Free to use]
An interactive version of Jeff Dean's table. Learn the orders of magnitude.

Colophon: how this book was made

This book was typeset from the course sources with free software only. Every typeface is published under the SIL Open Font Licence, every library under a permissive licence, and the page engine is the open-source Chromium print pipeline. Licence labels below describe the linked project, not this book.

TYPE, TOOLS AND LIBRARIES

SIL International, after Matthew Carter. *Charis SIL*. <https://software.sil.org/charis/> [SIL Open Font Licence 1.1]

The text face. An extension of Bitstream Charter, which Carter drew in 1987 to stay readable on coarse output. It is the standard free choice for technical books.

Bold Monday for IBM. *IBM Plex Sans and Plex Sans Condensed*. <https://github.com/IBM/plex> [SIL Open Font Licence 1.1]

Headings, labels, tables, folios and the covers.

JetBrains. *JetBrains Mono*. <https://github.com/JetBrains/JetBrainsMono> [SIL Open Font Licence 1.1]

All code, output and ASCII diagrams.

Google. *Noto Serif Devanagari*. <https://github.com/notofonts/devanagari> [SIL Open Font Licence 1.1]

The Devanagari on the covers and title page.

Knut Sveidqvist and contributors. *Mermaid*. <https://github.com/mermaid-js/mermaid> [MIT Licence]

Renders every flow chart, sequence diagram and tree.

Georg Brandl and contributors. *Pygments*. <https://pygments.org/> [BSD 2-Clause]

Colours the code. The palette used here was written for this book.

Waylan Limberg and contributors. *Python-Markdown*. <https://python-markdown.github.io/> [BSD 3-Clause]

Turns the lesson sources into the pages you are reading.

Martin Thoma and contributors. *pypdf*. <https://github.com/py-pdf/pypdf> [BSD 3-Clause]

Joins the covers, front matter and body, and writes the bookmarks.

The Chromium Authors. *Chromium print-to-PDF, driven by Puppeteer*. <https://pptr.dev/> [Apache 2.0 / BSD 3-Clause]

The typesetting engine. No proprietary software was used to make this book.

HOW A PAGE WAS MADE

The lesson sources are Markdown. Fenced blocks are lifted out before the Markdown is parsed, so diagrams keep every space they have and code reaches the colouring pass untouched. Each monospace block is measured, and its type size chosen so the widest line fits the column rather than falling off the page.

The result is rendered in a browser engine at 6.75 by 9.5 inches and printed to PDF. The body is printed first so that the contents and the index can be built from the page numbers the printing actually produced, then the covers and front matter are printed and the three are joined.

Problem index

Every problem named in a practice sheet, in the order the book reaches it, with its source and the chapter that sets it. Problem statements are not reproduced anywhere in this book. Solve them where they live.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
078	Delete Node in a Linked List	LeetCode 237 (Medium)	Whether you notice the tail is impossible, and whether you say so before being asked.
078	Remove Linked List Elements	LeetCode 203 (Easy)	The head as a special case — the exact ugliness day 080 removes.
078	Design Linked List	LeetCode 707 (Medium)	Every operation in one class, including the index arithmetic that everybody gets wrong once.
078	Convert Binary Number in a Linked List to Integer	LeetCode 1290 (Easy)	The traversal loop, clean, with an accumulator. Warm-up for the whole phase.
079	Middle of the Linked List	LeetCode 876 (Easy)	The plain traversal loop, and whether you can state the two-answer ambiguity for even lengths before coding.
079	Design Linked List	LeetCode 707 (Medium)	Every boundary at once: index 0, index size, index out of range, empty list, single element.
079	Remove Linked List Elements	LeetCode 203 (Easy)	Look-one-ahead, consecutive matches, and the leading run that makes the head case a <code>*loop*</code> , not an <code>if</code> .
079	Insert into a Sorted Circular Linked List	LeetCode 708 (Medium)	The walk that has no <code>None</code> to stop it, and the all-equal-values case that hangs.
080	Remove Linked List Elements	LeetCode 203 (Easy)	The guard form. <code>[7,7,7,1,7]</code> for the leading run, <code>[1,7,7,2]</code> for the advance rule.
080	Merge Two Sorted Lists	LeetCode 21 (Easy)	The builder form, and attaching the remainder wholesale instead of node by node.
080	Remove Nth Node From End of List	LeetCode 19 (Medium)	Where the dummy does real work: the head case <code>*and*</code> landing on the predecessor.
080	Remove Duplicates from Sorted List II	LeetCode 82 (Medium)	A head that is part of a duplicate run, so no <code>if</code> can save you.
081	Reverse Linked List	LeetCode 206 (Easy)	The three-pointer dance, iteratively and recursively, with the stack cost volunteered.
081	Palindrome Linked List	LeetCode 234 (Easy)	Reversal as a <code>*tool*</code> : find the middle, reverse half, compare — and restore it afterwards.
081	Reverse Linked List II	LeetCode 92 (Medium)	Capturing the sublist's future tail before you start, plus both reattachments.
081	Reverse Nodes in k-Group	LeetCode 25 (Hard)	Counting before reversing, and the three-way reattachment per group.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
082	Middle of the Linked List	LeetCode 876 (Easy)	The two-guard loop condition, and whether you raise the even-length ambiguity before being told.
082	Remove Nth Node From End of List	LeetCode 19 (Medium)	The fixed gap, the dummy putting slow on the predecessor, and n larger than the list.
082	Palindrome Linked List	LeetCode 234 (Easy)	Three techniques in one, and restoring the caller's list afterwards.
082	Reorder List	LeetCode 143 (Medium)	Split, reverse, interleave — and the interleave loop's terminating condition.
083	Linked List Cycle	LeetCode 141 (Easy)	The two runners, is versus =, and advancing before comparing.
083	Linked List Cycle II	LeetCode 142 (Medium)	Phase two, and whether you can prove it rather than recite it.
083	Happy Number	LeetCode 202 (Easy)	Recognising a cycle question that has no linked list in it.
083	Find the Duplicate Number	LeetCode 287 (Medium)	The hardest reframing: an array read as a function, with the duplicate as the cycle entrance.
084	Merge Two Sorted Lists	LeetCode 21 (Easy)	The builder dummy, $\leq$ for stability, and attaching the remainder in one assignment.
084	Sort List	LeetCode 148 (Medium)	The first middle, the cut, and whether you can say why merge sort and not quicksort.
084	Merge k Sorted Lists	LeetCode 23 (Hard)	$O(nk)$ versus $O(n \log k)$, the heap tie-breaker, and terminating the output.
084	Insertion Sort List	LeetCode 147 (Medium)	The $O(n^2)$ baseline, so you can say what merge sort bought you and at what n it matters.
085	Design Linked List	LeetCode 707 (Medium)	Do it doubly this time. Every boundary, twice as many pointers.
085	Design Browser History	LeetCode 1472 (Medium)	A cursor in a doubly linked list, and truncating the forward chain on a new visit.
085	Flatten a Multilevel Doubly Linked List	LeetCode 430 (Medium)	Splicing a whole sublist in, with both directions maintained at three joints.
085	LRU Cache	LeetCode 146 (Medium)	Build it again, and this time say the one-sentence justification before writing a line.
086	Odd Even Linked List	LeetCode 328 (Medium)	Two builder chains at once, and terminating both — the simplest composite.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
086	Copy List with Random Pointer	LeetCode 138 (Medium)	The map baseline, then the interleaving trick, and restoring the original.
086	Reorder List	LeetCode 143 (Medium)	Runner + reverse + interleave. Three techniques, no new idea.
086	Sort List	LeetCode 148 (Medium)	First middle, the cut, and the quicksort question you will be asked.
087	Fibonacci Number	LeetCode 509 (Easy)	The classic, and whether you notice it is exponential before being told.
087	Reverse String	LeetCode 344 (Easy)	Recursion with two indices instead of slicing, in $O(1)$ extra space.
087	Pow(x, n)	LeetCode 50 (Medium)	Halving instead of decrementing, negative exponents, and calling the recursion once.
087	Merge Two Sorted Lists	LeetCode 21 (Easy)	You solved this iteratively on day 084. Now do it recursively and compare the two.
088	Binary Tree Inorder Traversal	LeetCode 94 (Easy)	Write it recursively, then iteratively. This is the conversion where the work happens <i>after</i> the calls.
088	Maximum Depth of Binary Tree	LeetCode 104 (Easy)	Depth as a value <i>and</i> depth as a cost — and the degenerate tree that kills the recursive version.
088	Climbing Stairs	LeetCode 70 (Easy)	Naive recursion, count the calls, then memoise. Fibonacci wearing a hat.
088	Flatten Nested List Iterator	LeetCode 341 (Medium)	An explicit stack because the depth is attacker-controlled — the practical reason for the conversion.
089	Greatest Common Divisor of Strings	LeetCode 1071 (Easy)	Euclid's shape, where the obvious argument is not the measure.
089	Number of Islands	LeetCode 200 (Medium)	A grid with no natural measure — the visited marking <i>is</i> the termination argument.
089	Unique Paths	LeetCode 62 (Medium)	A measure that is a combination of two arguments, and the call count it predicts.
089	Sqrt(x)	LeetCode 69 (Easy), then the float version	An integer measure that terminates, then a float one that needs an epsilon.
090	Reverse String	LeetCode 344 (Easy)	Two indices in place, and the $\geq$ base case that even lengths need.
090	Valid Palindrome	LeetCode 125 (Easy)	The same shape with characters to skip — where the indices move by different amounts.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
090	Fibonacci Number	LeetCode 509 (Easy)	Loop to recursion to memoised recursion, and the call counts for all three.
090	Flatten Nested List Iterator	LeetCode 341 (Medium)	The case where recursion genuinely wins: depth is the nesting, not the length.
091	Subsets	LeetCode 78 (Medium)	The tree, choose-recurse-un-choose, and the copy.
091	Subsets II	LeetCode 90 (Medium)	Sorting first, and $i > \text{start}$ rather than $i > 0$.
091	Combinations	LeetCode 77 (Medium)	The same tree with a cap, and the first real pruning.
091	Letter Combinations of a Phone Number	LeetCode 17 (Medium)	The same shape where the branching factor is 3 or 4 rather than 2.
092	Permutations	LeetCode 46 (Medium)	The used list replacing start, and two un-chooses for two chooses.
092	Letter Case Permutation	LeetCode 784 (Medium)	The same tree where the branching factor is 1 or 2 depending on the character.
092	Permutations II	LeetCode 47 (Medium)	not used[i - 1] — the hardest single line in the phase.
092	Beautiful Arrangement	LeetCode 526 (Medium)	Permutations with a validity check before recursing, which is what pruning buys.
093	Combinations	LeetCode 77 (Medium)	The start index, and the size prune that is a break.
093	Combination Sum	LeetCode 39 (Medium)	build(i, ...) for reuse, and the sorted break on too-big.
093	Combination Sum III	LeetCode 216 (Medium)	Two constraints at once: a fixed size *and* a target.
093	Combination Sum II	LeetCode 40 (Medium)	Two skips, two keywords — break for too big, continue for duplicate.
094	Letter Combinations of a Phone Number	LeetCode 17 (Medium)	One piece of state, one undo — the template with nothing else in the way.
094	Palindrome Partitioning	LeetCode 131 (Medium)	The validity check placed *before* the recursive call, not at the leaf.
094	Generate Parentheses	LeetCode 22 (Medium)	Two counters as arguments, so there is nothing to undo at all.
094	Restore IP Addresses	LeetCode 93 (Medium)	Several prunes, and a partial answer that must be undone by exact length.
095	N-Queens II	LeetCode 52 (Hard)	The three constraint sets, and counting without building boards.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
095	N-Queens	LeetCode 51 (Hard)	The same tree, plus rendering only at a solution.
095	Sudoku Solver	LeetCode 37 (Hard)	Row, column and box sets, and the $(r//3)*3 + c//3$ index.
095	Valid Sudoku	LeetCode 36 (Medium)	The same three sets with no search at all — do it last, as the check.
096	Number of Islands	LeetCode 200 (Medium)	The region version: mark and never restore. Do this first, as the contrast.
096	Word Search	LeetCode 79 (Medium)	The path version: mark, explore, restore — and bounds before the match.
096	Flood Fill	LeetCode 733 (Easy)	Region again, with the trap that the new colour may equal the old one.
096	Sudoku Solver	LeetCode 37 (Hard)	Three sets, pre-computed blanks, and the most-constrained cell.
097	Palindrome Partitioning	LeetCode 131 (Medium)	Recognising the start-index tree from a string problem, and pruning before descending.
097	Restore IP Addresses	LeetCode 93 (Medium)	Two prunes, both break, and a fixed piece count.
097	Combination Sum II	LeetCode 40 (Medium)	Both skips with the right keyword each, cold.
097	Word Search	LeetCode 79 (Medium)	Path versus region, said before any code is written.

Index

Page numbers in **bold** are where the term is defined. Terms are those the book defines; the glossary in Appendix D gives each one a sentence.

#

4 kb, 911, 912, 922

A

a replicated log, 911
 abstract class, 911
 acid, 911, 912, 928
 adaptee, 911
 adapter, 911
 add a tie-breaker, 911
 algorithm, 1, 14, 46, 58, 65, 66, 67, 72, 74, ...
 always comparable, 911, 921
 amortised, 13, 57, 911
 anaemic model, 196, 207, 911
 and a direction, 911
 anti-diagonals, 911
 anti-sniping, **767**, 911
 approximate, 505, 911
 array, 2, 3, 5, 6, 7, 9, 11, 13, 16, ...
 artefact, 912, 914
 assignment rule, 912
 at-least-once, 642, 644, 645, 647, 648, 653, 654, 899, 912
 atomic, 26, 30, 121, 126, 333, 339, 368, 369, 370, ...
 atomicity, 911, 912
 availability, 25, 30, 706, 868, 912, 914, 919, 926
 aws ecs, 912, 922

B

backend, 293, 912, 928
 bearer token, 912
 best case, 265, 276, 912, 928
 binary tree, 912, 936
 blast radius, 912
 block, 5, 6, 7, 14, 17, 19, 37, 111, 123, ...
 blocked, 246, 552, 556, 636, 707, 747, 750, 754, 758, ...
 both, 3, 6, 13, 14, 16, 17, 18, 19, 20, ...
 boundary, 83, 505, 510, 521, 912, 916, 920, 924, 926, ...
 bounded context, 912
 bounded optimism, 912
 breadth-first, 912
 break point, 912

broken links, 912, 922
 bucket by time, 912
 bucket counts, 913
 bulkhead, 913
 burst size, 913, 923

C

cache, 3, 6, 14, 16, 17, 18, 19, 20, 41, ...
 cache it, 913
 cache stampede, 913
 candidates, 28, 73, 329, 456, 463, 464, 466, **658**, 659, ...
 capacity, 72, 73, 75, 118, 361, 502, 503, 505, 506, ...
 cdn, 502, 866, 867, 869, 870, 871, 872, 873, 909, ...
 certificate, 913
 chaining, 913
 channels, **610**, 633, 634, 639, 640, 641, 913
 checks permission, 913
 chunks, 681, 685, 686, 689, 690, 691, 695, 899, 913
 circular buffer, 913, 924
 class, 2, 4, 8, 11, 15, 18, 21, 22, 23, ...
 client, 114, 115, 122, 123, 131, 505, 511, 633, 638, ...
 collapse key, 913
 collapsing writes, 913
 collection, 290, 306, 309, 315, 321, 322, 324, 345, 712, ...
 column, 235, 236, 237, 251, 253, 254, 371, 413, 416, ...
 compact, 913, 915
 comparator, 914
 comparison, 85, 99, 179, 224, 248, 261, 353, 504, 555, ...
 complete, 11, 54, 97, 117, 119, 141, 183, 226, 270, ...
 component, 914, 917, 918, 921, 925
 composite, 816, 914, 935
 composite key, 914
 composition, 214, 301, 914, 916
 composition root, 914
 connection, 378, 554, 638, 725, 914, 918, 919, 931
 connection-oriented, 914
 consequence, 914, 922, 924
 consistency, 911, 912, 914, 916, 919, 925
 constraints, 664, 803, 914, 916, 937
 constructor, 100, 911, 914
 container, 912, 914, 915, 930

container image, 912, 914
containerd, 914, 915
context, 589, 590, 592, 594, 601, 603, 898, 912, 914
coordinator, 424, 914, 922
correlation id, 592, 594, 914
cost, 5, 7, 11, 13, 17, 18, 19, 20, 32, ...
count map, 914, 915
counter, 113, 120, 122, 152, 225, 230, 234, 236, 239, ...
cri-o, 914, 915
critical path, 871, 915
cycle, 1, 15, 16, 23, 24, 26, 27, 28, 29, ...

D

dangerous, 915
dashed, 25, 34, 417, 915
dau, 66, 194, 220, 282, 411, 481, 525, 860, 861, ...
deep-copied, 915
deleted, 10, 52, 55, 88, 91, 92, 96, 97, 99, ...
delta of delta, 915
dependency, 204, 817, 915, 922, 927
depth, 143, 145, 147, 150, 187, 265, 267, 274, 277, ...
depth-first, 485, 494, 495, 519, 698, 806, 809, 915
deque, 6, 14, 17, 19, 306, 309, 315, 321, 322, ...
diminishing returns, 915
direct addressing, 915
domain name, 915
draw two diagrams, 915
duck typing, 915
dummy node, 15, 61, 91, 93, 96, 98, 100, 103, 105, ...

E

earlier, 163, 179, 181, 190, 191, 388, 476, 477, 775, ...
edge cases, 228, 355, 366, 914, 916
edge compute, 916
effect, 6, 10, 17, 119, 284, 291, 295, 296, 467, ...
elastic scaling, 916
election timeout, 916
empty, 4, 15, 21, 22, 24, 39, 40, 42, 47, ...
etl, 292, 334, 338, 400, 408, 423, 426, 427, 428, ...
event object, 916
eventual consistency, 916
exactly, 3, 5, 6, 12, 15, 18, 19, 20, 23, ...
exactly correct, 916
exactly one, 3, 26, 53, 55, 62, 105, 108, 120, 162, ...
exclusive lock, 916
expiry, 197, 202, 206, 208, 210, 215, 216, 217, 542, ...

F

facts, 195, 196, 199, 207, 209, 916, 919, 922
false negative, 916
false positive, 916
fanout, 916

fee, 17, 24, 27, 33, 35, 37, 175, 193, 194, ...
fifo, 205, 206, 208, 210, 545, 547, 549, 555, 560, ...
file, 42, 44, 120, 129, 134, 439, 505, 586, 655, ...
filled diamond, 916
finite state machine, 917
float, 70, 74, 119, 154, 160, 161, 162, 163, 164, ...
four-direction move, 917
fragile, 917
frontend, 917

G

galloping mode, 917
gap, 3, 4, 5, 10, 12, 19, 22, 26, 49, ...
generator expression, 917
greedy simulation, 917
guest operating system, 917, 918

H

hash map, 6, 17, 19, 20, 41, 308, 314, 321, 323, ...
hash ring, 917
hateoas, 917
header interface, 917
heap, 5, 7, 19, 30, 38, 94, 143, 154, 163, ...
heapsort, 917
heartbeat, 917
hidden spof, 917
hiding mechanism, 917
highest node, 917
historical snapshot, 917
hollow triangle, 918
hook, 47, 420, 634, 637, 644, 648, 899, 918
horizontal scaling, 918
hypervisor, 917, 918

I

image, 151, 329, 755, 757, 761, 805, 866, 867, 878, ...
implicit graph, 918
in parallel, 918
in series, 918
in-degree, 918, 922
index, 7, 10, 11, 13, 17, 32, 33, 35, 40, ...
indexed heap, 918
inheritance, 918
inorder, 474, 918, 924, 936
input, 9, 67, 71, 83, 90, 102, 104, 105, 106, ...
integrity, 918
intent plus multiplicity, 918
interface, 2, 25, 29, 68, 70, 73, 114, 156, 197, ...
invariant, 56, 313, 317, 332, 702, 740, 918
is possible, 918, 925
item, 54, 116, 151, 152, 153, 155, 156, 157, 158, ...
iterator, 474, 562, 918, 936, 937

J

json, 451, 539, 587, 588, 590, 592, 594, 596, 597, ...
jump, 3, 176, 221, 224, 230, 231, 233, 255, **717**, ...
jwt, 919

K

k closest points, 919
k most frequent, 919
k-th largest specifically, 919
key, 16, 17, 19, 28, 29, 73, 74, 79, 81, ...
key property, 919

L

lag, 24, 35, 79, 81, 225, 229, 245, 249, 252, ...
largest, 109, 122, 125, 164, 166, 262, 333, 338, 340, ...
latency, 456, 468, 469, **503**, 510, 511, 512, 513, 514, ...
lazy, 371, 376, 381, 388, 389, 390, 413, 428, 432, ...
lazy deletion, 919
lazy loading, 919
lazy refill, **502**, 503, 508, 513, 516, 517, 521, 523, 897, ...
leader, 766, 770, 772, 773, 775, 776, 781, 782, 784, ...
leaf, 569, 573, 574, 575, 582, 605, 616, 676, 701, ...
lease, 25, 30, 31, 32, 35, 79, 117, 371, 375, ...
ledger entries, **327**, 335, 336, 337, 338, 339, 340, 341, 342, ...
left, 5, 19, 21, 63, 66, 68, 71, 113, 118, ...
lfu, **544**, 545, 547, 549, 552, 553, 555, 556, 557, ...
linear scan, 919
linearizability, 919
list, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...
list of lists, 430, 671, 693, 741, 854, 920
load balancer, 863, 920
load factor, 920
load shedding, 920
log-structured merge tree, 920
logical cohesion, 920
logical time, 920
loop, 9, 11, 15, 17, 29, 40, 42, 47, 49, ...
loop body, 58, 875, 920
low cohesion, 920
lru, 3, 6, 17, 18, 19, 20, 41, 45, **544**, ...

M

manifest, 920
many times, 482, 717, 734, 795, 805, 920, 922
many-to-many, 920, 927
matrix, 920
mau, **860**, 868, 920
max-heap, 328, 333, 920
method resolution order, 920

milliseconds, 510, 862, 914, 920
min-heap, 266, 271, 277, 333, 920
minimum spanning forest, 921
mixin, 758, 921
monotone, 921
monotonic deque, 921

N

name mangling, 921
namespace, 684, 690, 921
negative, 11, 229, 333, 401, 483, 489, 758, 916, 921, ...
negative infinity, 921
negatives, 229, 333, 916, 921
nested loop, 921
network partition, 921
never equal, 405, 484, 527, 536, 539, 911, 921
never forgets, **552**, 921
never persisted, 921
no children, 919, 921
node, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...
not a map, 921

O

o(1), 5, 6, 7, 9, 10, 11, 12, 13, **49**, ...
object, 3, 4, 5, 6, 7, 14, 15, 19, 20, ...
observers, 921
occupied, 915, 916, 921
offset, 748, 751, 820, 822, 823, 824, 921
on first touch, 921
one-shot, 922
opens, 48, 79, 90, 107, 115, 151, 262, 379, 380, ...
orchestrator, 817, 912, 922
origin, 98, 99, 185, 270, 357, 359, 360, 529, 794, ...
origin pull, 922
orphan rows, 922
orphans, 912, 922
out-degree, 918, 922
overloading, 922
overriding, 922

P

page, 282, 284, 291, 343, 353, 377, 482, 503, 522, ...
parallel edges, 922
parameters, 314, 922
parent, 593, 678, 682, 683, 685, 686, 687, 827, 843, ...
partial failure, 922
participants, 327, 331, 332, 914, 922, 924
partition key, 912, 922
partitions, 922
path, 35, 109, 111, 113, 119, 122, 123, 124, 125, ...
port, 22, 27, 29, 31, 32, 34, 35, 37, 38, ...
prefix sums, 921, 923

preorder, 923
priority queue, 923
procedure, 923
process, 510, 588, 640, 914, 921, 923, 927, 928
program, 354, 402, 664, 667, 751, 796, 914, 922, 923, ...
protecting state, 923

Q

qps, 839, 859, 860, 861, 862, 863, 864, 866, 867, ...
queue, 23, 29, 37, 41, 73, 90, 91, 107, 108, ...
quorum, 923

R

race condition, 923
ram, 14, 16, 21, 24, 29, 34, 43, 45, 109, ...
rank, 258, 281, 282, 283, 285, 286, 288, 289, 292, ...
rebalancing, 923
rebuilding the tree, 923
recursion, 141, 143, 145, 147, 149, 150, 170, 174, 187, ...
references, 5, 59, 100, 186, 227, 228, 248, 578, 610, ...
refill rate, 913, 923
register, 598, 829, 864, 868, 872, 923
relative epsilon, 923
removal, 105, 307, 309, 310, 316, 318, 321, 322, 323, ...
replica, 863, 865, 866, 868, 911, 913, 923, 924, 928
replica divergence, 924
request-aware routing, 924
resolver, 924
resources, 924, 927
returned, 95, 132, 136, 173, 196, 197, 200, 201, 202, ...
ring buffer, 913, 924
role interface, 924
root, 439, 440, 446, 450, 452, 485, 489, 490, 674, ...
rotation, 924
routine, 65, 542, 586, 587, 924
row, 6, 13, 25, 33, 34, 35, 38, 39, 40, ...
run, 1, 3, 17, 19, 21, 41, 42, 43, 56, ...

S

same remainder, 327, 924
schema, 26, 37, 924
scoping, 924
script, 21, 34, 43, 45, 78, 121, 163, 206, 251, ...
seam, 735, 818, 924
search space, 924
selectivity, 924
sentinel, 308, 309, 311, 312, 314, 315, 316, 319, 320, ...
separator, 924
sequence diagram, 924, 932
serialisable, 924
service credits, 924

session, 864, 919, 924, 925, 926, 927
session guarantees, 925
set, 3, 5, 8, 15, 26, 30, 33, 35, 37, ...
shallow-copied, 925
shape, 31, 37, 38, 43, 44, 45, 50, 85, 94, ...
shard, 35, 44, 204, 379, 381, 382, 384, 389, 390, ...
shared lock, 925
shared store, 925
singleton, 925
singly linked list, 11, 16, 50, 54, 57, 61, 64, 306, 310, ...
slice, 262, 396, 401, 404, 410, 429, 431, 433, 434, ...
smallest, 398, 665, 673, 856, 906, 914, 918, 919, 920, ...
snapshot, 414, 417, 419, 422, 428, 432, 711, 895, 917, ...
snipe, 767, 780, 785, 789, 901, 925
sorted region, 925
sorting everything, 925
splits, 96, 189, 192, 264, 279, 280, 305, 327, 329, ...
spof, 917, 925
squares, 223, 235, 236, 238, 239, 240, 242, 246, 247, ...
stable, 97, 264, 270, 274, 275, 276, 277, 279, 280, ...
stack, 12, 99, 136, 138, 141, 143, 145, 146, 147, ...
stack frame, 143, 265, 273, 396, 398, 403, 436, 437, 452, ...
stack overflow, 925
state, 10, 13, 18, 19, 27, 29, 33, 34, 35, ...
statements, 596, 923, 926, 933
status, 70, 195, 196, 197, 199, 200, 202, 207, 369, ...
still one machine, 919, 926
stonith, 926
strategy, 2, 44, 162, 249, 327, 331, 332, 338, 506, ...
strictly shorter, 483, 926
structural, 923, 926
subarray, 926
subject, 116, 197, 200, 422, 637, 926
subscription list, 926
substring, 926
subtree, 683, 685, 686, 688, 918, 923, 926
surge multiplier especially, 926
swap, 93, 129, 170, 173, 213, 265, 276, 277, 279, ...
system call, 926

T

table, 24, 27, 68, 70, 73, 88, 97, 123, 130, ...
tail call, 440, 441, 451, 926
tail-recursive, 444, 532, 926
tcp, 914, 926, 927
template, 576, 577, 578, 610, 633, 634, 636, 639, 700, ...
test double, 926
the return address, 437, 440, 443, 445, 449, 451, 452, 453, 902, ...
the running median, 917, 926, 927

third table, 920, 927
thread, 30, 293, 440, 446, 447, 550, 553, 556, 557, ...
three distinct orderings, 927
three-way partition, 927
tight coupling, 927
time-to-live, 640, 927
tiny constraint, 927
toast, 927
token, 21, 22, 107, 108, 377, 380, 382, 480, 500, ...
tombstone, 927
top-k problems, 917, 926, 927
total order, 33, 927
transitions itself, 927
transitive dependency, 927
tree, 3, 7, 17, 19, 136, 149, 176, 219, 220, ...
ttl, 26, 30, 39, 41, 108, 119, 120, 121, 125, ...
tuple, 74, 96, 184, 246, 270, 271, 275, 277, 280, ...
two, 3, 4, 5, 6, 7, 8, 9, 10, 393, ...
two implementations, 227, 816, 927

U

udp, 926, 927
under-fetching, 927
union filesystem, 928
upper bound, 666, 671, 672, 756, 759, 760, 762, 763, 788, ...

uri, 131, 145, 225, 467, 712, 804, 814, 833, 926, ...
url, 23, 25, 29, 35, 37, 717, 928

V

value object, 27, 281, 282, 296, 297, 301, 303, 304, 893, ...
version, 23, 96, 136, 143, 145, 228, 229, 275, 283, ...
vertex, 915, 921, 922, 928
vertical scaling, 928
victim, 13, 317, 551, 928
virtual address space, 928
virtual machine, 912, 922, 928
volatile, 928

W

weight, 76, 327, 353, 502, 675, 780, 928
which class, 44, 214, 215, 303, 928
which is hours, 928
within, 28, 37, 76, 81, 98, 110, 118, 121, 198, ...
worst case, 31, 108, 116, 119, 161, 265, 276, 277, 279, ...
write skew, 928
write-around with invalidation, 928
write-through, 928

Y

yes, 21, 24, 26, 37, 47, 71, 80, 90, 656, ...

Two subjects, every day, for one hundred and eighty days.

Most preparation teaches algorithms for three months, then starts again from nothing on system design. Interviews do not work that way. A single loop will ask you to reverse a linked list in one room and design a rate limiter in the next.

So this book teaches both, every day, for one hundred and eighty days. Written for someone starting from zero - not a graduate, not somebody brushing up - and finishing able to answer either kind of question out loud, to a stranger, without notes.

- ◆ One hundred and eighty chapters. Two lessons and a practice sheet in every one.
- ◆ The same nine sections every time, so you learn one reading rhythm and keep it.
- ◆ Every lesson opens with a story about a person, told without a single technical word.
- ◆ Every solution shown in full, built in fragments and then given complete.
- ◆ The arithmetic done out loud, and error messages printed exactly as they appear.
- ◆ Real interviewer phrasings, an opening script, the follow-ups, and a model answer.